

С++ своими руками

От первой строки до четырёх живых консольных проектов — с разбором каждой строки

Серик Мурадов

Содержание

1	Вступление. Зачем эта книга	1
1.1	О чём она	1
1.2	Кому она	2
1.3	Почему C++ в 2026 году	3
1.4	Что мы построим — детальнее	4
1.4.1	Проект 1: текстовая RPG-игра demo-rpg/	4
1.4.2	Проект 2: мини-shell demo-shell/	5
1.4.3	Проект 3: мини-СУБД demo-db/	5
1.4.4	Проект 4: TCP-мессенджер demo-chat/	6
1.5	Что НЕ войдёт (и почему)	7
1.6	Какие у меня соглашения	7
1.7	Как читать	8
1.8	Что дальше	8
2	Глава 1. Что такое компьютер	9
2.1	Зачем это в книге про C++	9
2.2	Что такое компьютер на самом деле	9
2.2.1	Процессор (CPU)	10
2.2.2	Оперативная память (RAM)	11
2.2.3	Кэш процессора	11
2.2.4	Диск (SSD/HDD)	12
2.2.5	Операционная система	12
2.3	Что такое процесс	13
2.4	Что такое поток	14
2.5	Что увидите в C++	15
2.6	На пальцах: жизнь программы Hello	15
2.7	Маленькое упражнение	16
2.8	Что дальше	16
3	Глава 2. Биты, байты, числа, текст	18
3.1	Бит — самая маленькая единица	18
3.2	Байт — группа из 8 бит	19
3.3	Двоичная система счисления	19
3.4	Шестнадцатеричная система	20
3.5	Отрицательные числа: дополнительный код	21
3.6	Дробные числа: IEEE 754	22
3.7	Текст: ASCII	23
3.7.1	Чего не хватало в ASCII	23

3.7.2	Юникод и UTF-8	24
3.7.3	Что это значит для C++	25
3.8	Размеры и единицы	25
3.9	Что увидите в C++	25
3.10	Маленькое упражнение	26
3.11	Что дальше	26
4	Глава 3. Что такое программа и язык программирования	27
4.1	Программа — это последовательность инструкций	27
4.2	Машинный код	28
4.3	Ассемблер	28
4.4	Языки высокого уровня	29
4.4.1	В и С	30
4.4.2	C++	30
4.4.3	Другие популярные языки	31
4.5	Компиляция vs интерпретация	32
4.5.1	Компиляция	32
4.5.2	Интерпретация	32
4.5.3	JIT-компиляция (между двумя мирами)	33
4.5.4	Где здесь C++	33
4.6	Как программа становится исполняемой — общий контур	34
4.7	Линковка чуть подробнее	34
4.8	Стандартная библиотека	36
4.9	Что увидите в C++	36
4.10	Маленькое упражнение	37
4.11	Что дальше	37
5	Глава 4. Алгоритмическое мышление	38
5.1	Что такое алгоритм	38
5.2	Алгоритм в быту: заварить чай	39
5.3	Вход, обработка, выход	39
5.4	Переменные	40
5.5	Операции и выражения	40
5.6	Ветвление (условие)	41
5.7	Цикл	42
5.7.1	Бесконечный цикл	43
5.8	Функция	44
5.8.1	Функция, вызывающая функцию	44
5.9	Декомпозиция как главный навык	45
5.10	Псевдокод и реальный код	46
5.11	Состояние программы	46
5.12	Концепции, которые превратятся в C++	47
5.13	Алгоритм сложнее: найти максимум	47
5.14	Маленькое упражнение	48
5.15	Что дальше	49
6	Глава 5. Терминал, файлы, папки, права	50
6.1	Что такое терминал	50

6.2	Где он живёт	51
6.3	Приглашение к вводу (prompt)	51
6.4	Где я нахожусь — pwd	52
6.5	Что в папке — ls	52
6.6	Перемещение — cd	53
6.7	Пути — абсолютные и относительные	54
6.8	Создать папку — mkdir	55
6.9	Создать или открыть файл	55
6.10	Прочитать файл — cat	55
6.11	Копировать, перемещать, удалять — cp, mv, rm	56
6.12	Маски (wildcards)	57
6.13	Права доступа	57
6.13.1	chmod	58
6.13.2	chown	58
6.14	Переменные окружения	59
6.15	Перенаправление и пайпы	59
6.15.1	Стандартные потоки	59
6.15.2	Запись stdout в файл — >	60
6.15.3	Чтение stdin из файла — <	60
6.15.4	Пайпы — 	60
6.16	Запуск программы и ./	61
6.17	История команд	61
6.18	Автодополнение	61
6.19	Очистить экран	62
6.20	Прервать команду	62
6.21	Полезный набор команд	62
6.22	Маленькое упражнение	63
6.23	Что дальше	64
7	Глава 6. Инструменты и первая программа	65
7.1	Что нам нужно	65
7.2	macOS	66
7.3	Linux	66
7.4	Windows	67
7.5	Редактор	68
7.6	Структура проекта	68
7.7	Первая программа — hello.cpp	69
7.7.1	Строка 1: #include <iostream>	69
7.7.2	Строка 2: пустая	70
7.7.3	Строка 3: int main() {	70
7.7.4	Строка 4: std::cout << "Hello, world!" << std::endl;	70
7.7.5	Строка 5: return 0;	71
7.7.6	Строка 6: }	72
7.8	Компиляция	72
7.9	Запуск	73
7.10	Что произошло — большая картинка	73
7.11	Что если ошибка	74
7.12	Запоминания на сегодня	75

7.13	Маленькое упражнение	75
7.14	Что дальше	75
8	Глава 7. Как программа становится исполняемым файлом	77
8.1	Подопытная программа	77
8.2	Стадия 1: препроцессор	79
8.2.1	Посмотрим своими глазами	79
8.2.2	Макросы	80
8.2.3	Условная компиляция	81
8.2.4	Include guard детальнее	82
8.2.5	Что выучить про препроцессор	83
8.3	Стадия 2: собственно компиляция	83
8.3.1	Манглинг имён	85
8.3.2	Что выучить про компиляцию	85
8.4	Стадия 3: ассемблирование	86
8.4.1	Внутри объектного файла	86
8.4.2	Смотрим на символы	86
8.4.3	Утилита objdump	87
8.5	Стадия 4: линковка	88
8.5.1	А стандартная библиотека?	88
8.5.2	Сообщения об ошибках линковки	89
8.5.3	Статическая и динамическая линковка	89
8.5.4	Что выучить про линковку	91
8.6	Memory layout: что где лежит в памяти процесса	91
8.6.1	.text (код)	91
8.6.2	.rodata (константы)	92
8.6.3	.data (инициализированные глобальные)	92
8.6.4	.bss (неинициализированные глобальные)	92
8.6.5	Куча (heap)	92
8.6.6	Мmap-области	93
8.6.7	Стек (stack)	93
8.6.8	Картинка целиком	93
8.6.9	Утилита size	94
8.6.10	Стек: размер и переполнение	94
8.7	Полная картина: что происходит при сборке	95
8.8	Один важный нюанс: единицы трансляции	95
8.9	Маленькое упражнение	96
8.10	Что дальше	97
9	Глава 8. Переменные и типы	98
9.1	Почему в C++ типы такие важные	98
9.2	Целые числа: семейство int	99
9.2.1	Проверяем sizeof	99
9.2.2	signed и unsigned	100
9.2.3	Когда использовать unsigned	101
9.2.4	Целочисленные типы фиксированного размера	102
9.2.5	Литералы целых чисел	103
9.3	Дробные числа: float и double	103

9.3.1	Литералы дробных чисел	104
9.3.2	Точность — самое больное место	104
9.4	bool	105
9.5	char и символы	106
9.5.1	Escape-последовательности	106
9.5.2	char, signed char, unsigned char	107
9.6	const — переменная, которую нельзя менять	107
9.7	constexpr — константа времени компиляции (C++11)	108
9.8	Способы инициализации	109
9.8.1	Старый способ: с равно	109
9.8.2	С круглыми скобками	109
9.8.3	Единообразная инициализация (C++11): с фигурными скобками	109
9.8.4	Default initialization vs value initialization	110
9.9	auto — компилятор сам выводит тип (C++11)	111
9.9.1	Когда auto хороший выбор	111
9.9.2	Когда auto плохой выбор	111
9.9.3	auto и const/&	112
9.10	Имена переменных: правила и стиль	112
9.11	Полный собранный пример	113
9.12	Преобразования типов: коротко	114
9.13	Главное — что увидите в коде дальше	115
9.14	Маленькое упражнение	116
9.15	Что дальше	117
10	Глава 9. Управляющие конструкции	118
10.1	if — простое ветвление	118
10.2	if/else	119
10.3	Логические операции в условиях	120
10.3.1	Ловушка номер один: = vs ==	121
10.3.2	Short-circuit evaluation	121
10.3.3	Скобки и приоритет	122
10.4	Дополнительный конструктор: if с инициализацией (C++17)	122
10.5	Тернарный оператор	123
10.6	switch — выбор из множества значений	124
10.6.1	Ограничения switch	125
10.6.2	Fall-through — главная ловушка switch	125
10.6.3	default	126
10.7	while — цикл с проверкой в начале	127
10.7.1	Бесконечный цикл нарочно	128
10.8	do-while — цикл с проверкой в конце	129
10.9	for — классический цикл	129
10.9.1	Все три части — опциональны	130
10.9.2	Несколько переменных в for	130
10.10	Range-based for — главная новинка C++11	131
10.10.1	По ссылке vs по значению	131
10.10.2	Если нужен индекс — классический for	132
10.11	break и continue	133
10.12	goto — реабилитация чёрной овцы	134

10.13	Циклы с итераторами (предварительно)	136
10.14	Что выучить про управляющие конструкции	136
10.15	Пара рекомендаций	137
10.16	Пример: «угадай число»	137
10.17	Маленькое упражнение	139
10.18	Что дальше	140
11	Глава 10. Функции	141
11.1	Зачем нужны функции	141
11.2	Анатомия функции	141
11.2.1	Несколько параметров	142
11.2.2	void — функция без возврата	142
11.2.3	Функция без параметров	143
11.3	Объявление и определение	143
11.3.1	Разделение между файлами	145
11.4	Передача параметров: три способа	146
11.4.1	Способ 1: по значению	146
11.4.2	Способ 2: по ссылке	147
11.4.3	Ключевые свойства ссылок	148
11.4.4	По ссылке для эффективности	148
11.4.5	Способ 3: по указателю	149
11.4.6	Указатель vs ссылка	149
11.4.7	Когда что использовать	150
11.4.8	Сводная таблица	150
11.4.9	Правило передачи параметров	150
11.5	Возврат из функции	151
11.5.1	Несколько возвращаемых значений	152
11.6	Перегрузка функций	153
11.7	Параметры по умолчанию	154
11.8	inline	155
11.9	Рекурсия	156
11.9.1	Хвостовая рекурсия	157
11.10	Forward declaration	158
11.11	main — особая функция	158
11.12	Лямбды — короткое упоминание	159
11.13	Указатели на функции	159
11.14	Чистые функции и побочные эффекты	160
11.15	Большой пример: банковская симуляция	160
11.16	Маленькое упражнение	163
11.17	Что дальше	163
12	Глава 11. Память — стек, куча, RAII	164
12.1	Освежим картинку из главы 7	164
12.2	Стек: подробности	165
12.2.1	Стек растёт «вниз»	166
12.2.2	Стек ограничен	166
12.2.3	Преимущества стека	166
12.2.4	Недостатки стека	167

12.3	Куча: подробности	167
12.3.1	new — выделение	167
12.3.2	delete — возврат	168
12.3.3	new[] и delete[] — массивы	168
12.3.4	Размер можно вычислять в рантайме	169
12.4	Утечки памяти	169
12.4.1	Когда утечки нестрашны	170
12.4.2	Пять способов утечь память	170
12.4.3	Висячие указатели	171
12.4.4	Двойное освобождение	171
12.4.5	Использование после освобождения (UAF)	172
12.5	Проблема: слишком много ручной работы	172
12.6	RAII — главный паттерн C++	172
12.6.1	Конструктор и деструктор	172
12.6.2	Почему RAII работает с исключениями	174
12.7	Стандартные RAII-обёртки	174
12.7.1	Пример: вектор вместо ручного массива	175
12.7.2	Пример: файл вместо fopen	175
12.8	smart pointers — предварительно	176
12.9	Стек или куча — что выбрать	176
12.9.1	Конкретные примеры	177
12.10	Инструменты для борьбы с проблемами памяти	177
12.10.1	AddressSanitizer (ASan)	177
12.10.2	Valgrind	178
12.10.3	LeakSanitizer (LSan)	178
12.11	Большой пример: классический ресурс	178
12.11.1	C-стиль (без RAII)	178
12.11.2	C++-стиль (с RAII)	179
12.11.3	Вариант со стандартной библиотекой	180
12.12	Когда нужно писать свой деструктор	181
12.13	Главные правила этой главы	181
12.14	Маленькое упражнение	182
12.15	Что дальше	182
13	Глава 12. Строки и потоки ввода-вывода	184
13.1	Старая школа: C-strings	184
13.2	std::string — основа основ	185
13.2.1	Создание и базовые операции	185
13.2.2	Конкатенация	186
13.2.3	Индексация и обход	186
13.2.4	Сравнение	187
13.2.5	Подстроки	188
13.2.6	Поиск	188
13.2.7	Замена и вставка	188
13.2.8	Преобразование к C-строке	188
13.2.9	Преобразования с числами	189
13.2.10	Что лежит внутри std::string	189
13.2.11	Итог по строкам	189

13.3	<code>std::cout</code> — вывод	190
13.3.1	<code>endl</code> против <code>\n</code>	190
13.3.2	<code>std::cerr</code> — поток ошибок	191
13.3.3	Форматирование вывода	191
13.4	<code>std::cin</code> — ввод	192
13.4.1	Несколько значений подряд	193
13.4.2	Чтение строки целиком: <code>getline</code>	193
13.4.3	Главная ловушка <code>cin</code> : смешивание » и <code>getline</code>	194
13.5	Файловые потоки	194
13.5.1	Чтение из файла	195
13.5.2	Запись в файл	195
13.5.3	Бинарные данные	196
13.5.4	Полностью прочитать файл	197
13.6	<code>std::stringstream</code> — строка как поток	197
13.6.1	Форматированная сборка строк	198
13.7	Сводный пример: разобрать CSV	198
13.8	Пара тонкостей	200
13.8.1	Чтение чисел и точка в качестве разделителя	200
13.8.2	Юникод и <code>std::string</code>	201
13.8.3	<code>sync_with_stdio</code>	201
13.9	Главные правила главы	201
13.10	Маленькое упражнение	202
13.11	Что дальше	202
14	Глава 13. Контейнеры STL	203
14.1	Зачем контейнеры	203
14.2	<code>std::vector</code> — главный контейнер	204
14.2.1	Создание	204
14.2.2	Доступ к элементам	204
14.2.3	Добавление и удаление	205
14.2.4	Размер и ёмкость	205
14.2.5	<code>reserve</code> и оптимизация	206
14.2.6	Инвалидация итераторов и указателей	206
14.2.7	Почему <code>vector</code> обычно быстрее списка	207
14.2.8	Удаление-стирание идиома (<code>erase-remove</code>)	208
14.3	<code>std::array</code> — массив фиксированного размера	208
14.4	<code>std::deque</code> — двусторонняя очередь	209
14.5	<code>std::list</code> — двусвязный список	209
14.6	<code>std::map</code> — упорядоченный словарь	210
14.6.1	Ловушка <code>[]</code> на <code>map</code>	211
14.6.2	Вставка с проверкой существования	211
14.7	<code>std::unordered_map</code> — хеш-таблица	212
14.7.1	Ключи в <code>unordered_map</code>	212
14.7.2	Что внутри <code>unordered_map</code>	212
14.8	<code>std::set</code> и <code>std::unordered_set</code>	213
14.9	Итераторы — кратко	213
14.10	<code>std::pair</code> и <code>std::tuple</code>	214
14.11	Сравнительная таблица	215

14.12	Решающее дерево: какой контейнер брать	215
14.13	Распространённые операции	216
14.13.1	Найти элемент	216
14.13.2	Сортировка	216
14.13.3	Сумма	217
14.13.4	Подсчёт	217
14.13.5	Уникальные значения	217
14.14	Большой пример: подсчёт частоты слов	217
14.15	Главные правила главы	219
14.16	Маленькое упражнение	219
14.17	Что дальше	220
15	Глава 14. Алгоритмы и итераторы	221
15.1	Зачем алгоритмы — снова про переиспользование	221
15.2	Итераторы — обобщённый указатель	222
15.2.1	Что такое <code>begin()</code> и <code>end()</code>	222
15.3	Категории итераторов	223
15.3.1	Input iterator	223
15.3.2	Output iterator	223
15.3.3	Forward iterator	223
15.3.4	Bidirectional iterator	223
15.3.5	Random access iterator	223
15.3.6	Иерархия	224
15.4	Алгоритмы без модификации	224
15.4.1	<code>find</code> — найти первый	224
15.4.2	<code>find_if</code> — найти с предикатом	224
15.4.3	<code>count</code> и <code>count_if</code>	225
15.4.4	<code>all_of</code> , <code>any_of</code> , <code>none_of</code>	225
15.4.5	<code>min_element</code> и <code>max_element</code>	225
15.4.6	<code>equal</code> — сравнить два диапазона	225
15.5	Алгоритмы с модификацией	226
15.5.1	<code>fill</code> — заполнить значением	226
15.5.2	<code>iota</code> — заполнить последовательно	226
15.5.3	<code>transform</code> — применить функцию	226
15.5.4	<code>copy</code> и <code>copy_if</code>	227
15.5.5	<code>remove</code> и <code>remove_if</code>	227
15.5.6	<code>reverse</code> и <code>rotate</code>	228
15.5.7	<code>sort</code> и связанные	228
15.5.8	<code>unique</code>	229
15.6	Численные алгоритмы	229
15.6.1	<code>accumulate</code>	229
15.6.2	<code>inner_product</code>	230
15.6.3	<code>partial_sum</code>	230
15.6.4	<code>adjacent_difference</code>	231
15.7	Перестановки	231
15.8	Бинарный поиск	231
15.9	Лямбды и предикаты — короткое введение	232
15.9.1	Функтор — старый стиль	232

15.10	Алгоритмы vs методы контейнеров	233
15.11	Удалить элемент из map во время прохода	233
15.12	Свой for_each — попытка номер один	234
15.13	Производительность	234
15.13.1	Алгоритмы STL часто оптимизированнее ручных	234
15.13.2	Алгоритмы вызывают функцию для каждого элемента	234
15.13.3	Когда STL медленнее ручного	235
15.14	Большой пример: статистика оценок	235
15.15	Главные правила	238
15.16	Маленькое упражнение	238
15.17	Часть I закрыта	238
15.18	Что дальше	239
16	Глава 15. Дизайн игры и скелет проекта	240
16.1	Что мы строим	240
16.2	Архитектура: какие классы будут	241
16.2.1	Связи между классами	242
16.3	Файловая структура	242
16.4	Создаём папки	243
16.5	Makefile	244
16.5.1	Простой пример	244
16.5.2	-MMD — магия авто-зависимостей	245
16.5.3	Наш Makefile	245
16.6	Скелет кода	250
16.6.1	include/game.h	250
16.6.2	src/game.cpp	251
16.6.3	src/main.cpp	253
16.6.4	tests/test_smoke.cpp	254
16.7	Сборка и проверка	254
16.7.1	Release-сборка	255
16.8	Тонкости и расширения	256
16.8.1	Поддержка отладчика	256
16.8.2	AddressSanitizer	256
16.8.3	Зачем не CMake	256
16.8.4	Зачем не header-only	256
16.9	Главные правила главы	257
16.10	Маленькое упражнение	257
16.11	Что дальше	258
17	Глава 16. Классы — Player, Location, World	259
17.1	struct и class	259
17.2	Простой класс Player	260
17.2.1	Объявление	260
17.2.2	Поля	261
17.2.3	Методы доступа (getters)	261
17.2.4	Модифицирующие методы	262
17.2.5	Конструктор	262
17.2.6	Определение	263

17.2.7	Что такое this	266
17.2.8	Использование Player	266
17.3	Локация: класс посложнее	267
17.3.1	Объявление	267
17.3.2	Определение	268
17.3.3	Использование	269
17.4	Перегрузка операторов — короткий взгляд	269
17.5	Конструктор по умолчанию	270
17.5.1	= delete	271
17.6	Копирование: rule of three	271
17.6.1	Когда rule of zero ломается	272
17.6.2	Rule of five (C++11)	273
17.6.3	Запрет копирования	274
17.7	World: контейнер локаций	274
17.7.1	Объявление	274
17.7.2	find возвращает указатель	275
17.7.3	Заводская функция	275
17.7.4	emplace вместо insert+move	276
17.7.5	Default-конструктор через = default	277
17.8	Game: всё вместе	277
17.8.1	Объявление	277
17.8.2	Конструктор	278
17.8.3	Команды	278
17.9	Сборка и запуск	280
17.10	Что освоено в этой главе	281
17.11	Маленькое упражнение	281
17.12	Что дальше	282
18	Глава 17. Наследование и виртуальные функции	283
18.1	Зачем наследование	283
18.2	Базовый синтаксис наследования	284
18.2.1	public, protected, private — два смысла	285
18.3	Конструктор базы	286
18.4	Виртуальные функции	286
18.4.1	override — обязательная привычка	288
18.4.2	final — запрет дальнейшего переопределения	288
18.5	Чисто виртуальные функции и абстрактные классы	289
18.6	Виртуальный деструктор — обязателен!	290
18.7	vtable — как это работает	291
18.8	Слайсинг — главная ловушка	292
18.9	dynamic_cast — приведение базы к потомку	293
18.10	Множественное наследование — кратко	294
18.11	Item-иерархия в проекте	295
18.12	Тестируем полиморфизм	298
18.13	Несколько тонкостей	299
18.13.1	Виртуальный вызов в конструкторе/деструкторе	299
18.13.2	Вызов метода базы из потомка	300
18.13.3	protected и инвариант класса	300

18.14	Альтернативы наследованию	301
18.15	Главные правила главы	301
18.16	Маленькое упражнение	302
18.17	Что дальше	302
19	Глава 18. Умные указатели — <code>unique_ptr</code>, <code>shared_ptr</code>, <code>weak_ptr</code>	303
19.1	Зачем нужны умные указатели	303
19.2	<code>unique_ptr</code> — основной рабочий	304
19.2.1	Создание	304
19.2.2	Запрет копирования	305
19.2.3	Перемещение разрешено	305
19.2.4	<code>make_unique</code> — фабричная функция	306
19.2.5	Кастомные deleters	306
19.2.6	<code>unique_ptr</code> для массивов	307
19.3	<code>shared_ptr</code> — совладение	307
19.3.1	<code>make_shared</code>	308
19.3.2	Цена <code>shared_ptr</code>	308
19.3.3	Циклы — главная ловушка <code>shared_ptr</code>	308
19.4	<code>weak_ptr</code> — наблюдатель без владения	309
19.5	Когда что	310
19.6	Inventory — наш первый <code>shared_ptr</code> ... ах нет, <code>unique_ptr</code>	310
19.6.1	<code>add</code> — принимаем владение	311
19.6.2	<code>take</code> — отдаём владение	311
19.6.3	<code>find</code> — наблюдение без владения	312
19.6.4	Лямбда с захватом по ссылке	312
19.7	Применение в Location и Player	313
19.8	Команды <code>take</code> / <code>drop</code> / <code>use</code>	314
19.9	Запуск	315
19.10	Сырой указатель — когда уместен	316
19.11	Тонкость: передать <code>unique_ptr</code> нельзя по значению, можно только через <code>move</code>	316
19.12	<code>shared_ptr</code> — куда мы его не применили	316
19.13	<code>weak_ptr</code> — куда мы его не применили	317
19.14	Главные правила главы	317
19.15	Маленькое упражнение	317
19.16	Что дальше	318
20	Глава 19. Move-семантика на бою	319
20.1	Зачем <code>move</code>	319
20.2	<code>lvalue</code> и <code>rvalue</code>	320
20.3	Главное различие на практике	321
20.4	<code>Rvalue</code> -ссылки: <code>T&&</code>	321
20.4.1	Что внутри <code>rvalue</code> -ссылки	321
20.5	Move-конструктор и move-присваивание	322
20.5.1	поexcept на move-конструкторе	323
20.5.2	Move-присваивание	323
20.6	<code>std::move</code> — что это на самом деле	324
20.7	Когда <code>move</code> бесплатен	325
20.8	Rule of five детально	325

20.8.1	Когда компилятор сам генерирует move	326
20.9	RVO и NRVO	326
20.9.1	Когда RVO не работает	327
20.10	Perfect forwarding и std::forward	328
20.11	std::move vs std::forward	328
20.12	Что после move	329
20.13	Демонстрация	329
20.14	Где move применяется в нашем коде	331
20.15	Главные правила главы	332
20.16	Маленькое упражнение	333
20.17	Что дальше	333
21	Глава 20. Шаблоны — EventBus	334
21.1	Зачем шаблоны	334
21.2	typename vs class	335
21.3	Где живут шаблоны: только в заголовках	336
21.4	Шаблоны классов	337
21.5	Дедукция типа	338
21.6	Специализация	339
21.6.1	Частичная специализация	339
21.7	Variadic templates	340
21.7.1	Variadic в нашем make_unique	341
21.8	SFINAE — кратко	341
21.9	Ошибки шаблонов	342
21.10	EventBus	343
21.10.1	События	344
21.10.2	Тест	344
21.11	Концепции и применения	345
21.12	Интеграция в проект (предварительно)	346
21.13	Сторонние шаблонные паттерны	346
21.13.1	Тип-безопасное хранение разнотипных данных	346
21.13.2	Типовые функции	347
21.13.3	Generic algorithms	347
21.14	Шаблоны и компилятор	348
21.15	Главные правила главы	348
21.16	Маленькое упражнение	348
21.17	Что дальше	349
22	Глава 21. Лямбды и std::function	350
22.1	Лямбда — что это	350
22.2	Анатомия захвата	351
22.2.1	[] — без захвата	351
22.2.2	[x] — захват по значению	351
22.2.3	[&x] — захват по ссылке	352
22.2.4	[=] — захват всего по значению	352
22.2.5	[&] — захват всего по ссылке	352
22.2.6	[this] — захват указателя на текущий объект	353
22.2.7	[x, &y] — комбинированный	353

22.2.8	[x = ...] — захват с инициализатором (C++14)	353
22.3	Что лямбда на самом деле	353
22.4	Производительность	354
22.5	std::function — type erasure	355
22.5.1	Цена type erasure	355
22.5.2	Когда использовать что	355
22.6	Лямбды как параметры функций	356
22.7	std::bind — старый стиль (для контекста)	356
22.8	NPC и поведения — намёк на будущее	357
22.9	CommandRegistry — лямбды на работе	357
22.9.1	Регистрация команд	359
22.9.2	Парсинг	360
22.9.3	help автоматический	360
22.9.4	Что выиграли	360
22.10	Запуск	361
22.11	Главные правила главы	361
22.12	Маленькое упражнение	362
22.13	Что дальше	362
23	Глава 22. Исключения и exception safety	363
23.1	Зачем исключения	363
23.2	throw, try, catch	364
23.2.1	Ловите по ссылке, не по значению	365
23.2.2	Несколько catch	365
23.2.3	Rethrow	366
23.3	std::exception и иерархия	366
23.4	Свои типы исключений	367
23.4.1	Базовый тип runtime_error	368
23.5	Применяем: валидация в Player	368
23.6	Главный try/catch в main	369
23.7	Stack unwinding под капотом	370
23.8	ноэкспорт — обещание не бросать	370
23.9	Exception safety — три уровня гарантий	371
23.9.1	Уровень 1: basic guarantee — слабая	371
23.9.2	Уровень 2: strong guarantee — сильная	372
23.9.3	Уровень 3: nothrow guarantee — без бросков	372
23.9.4	Какая гарантия предпочтительна	372
23.10	Copy-and-swap идиома детально	373
23.11	Цена исключений	373
23.12	Почему отключают исключения	374
23.13	Когда что выбирать: исключение vs код ошибки vs assert	374
23.14	Главные правила главы	375
23.15	Маленькое упражнение	375
23.16	Что дальше	376
24	Глава 23. Парсер скриптов локаций	377
24.1	Зачем парсер	377
24.2	Выбор формата	378

24.3	Пример мира	378
24.4	Архитектура парсера	380
24.5	Заголовок	380
24.6	Реализация: вспомогательные функции	381
24.7	Промежуточное состояние	382
24.8	Парсинг строки предмета	382
24.9	Главный цикл парсера	383
24.9.1	name и desc	385
24.9.2	exit	386
24.9.3	item	386
24.9.4	Конец файла	386
24.10	Подключение к Game	386
24.11	Проверка	387
24.12	Уроки про парсинг	388
24.12.1	Состояния парсера	388
24.12.2	Сообщения об ошибках	388
24.12.3	Иммутабельность данных	388
24.12.4	Tokenizer vs парсер	388
24.12.5	Robust vs strict парсер	389
24.13	std::variant — упоминание (C++17)	389
24.14	Главные правила главы	389
24.15	Маленькое упражнение	390
24.16	Что дальше	390
25	Глава 24. Сохранение и загрузка игры	391
25.1	Что сохранять	391
25.2	Текстовый или бинарный формат	391
25.3	Формат сейва	392
25.4	Зачем версия	393
25.5	Зачем контрольная сумма	393
25.6	API менеджера сохранений	394
25.7	Реализация write	395
25.7.1	Сериализация предмета	396
25.7.2	Запись в файл	396
25.7.3	Эскейпинг	397
25.8	Реализация read	397
25.8.1	Двухпроходный парсер	398
25.8.2	Replace player	399
25.9	Команды в Game	399
25.10	Тест	400
25.11	Поломка checksum	401
25.12	Что осталось	402
25.13	Бинарный формат — кратко	402
25.14	Главные правила главы	403
25.15	Маленькое упражнение	403
25.16	Что дальше	403
26	Глава 25. Финал RPG	405

26.1	Зачем юнит-тесты	405
26.2	GoogleTest vs свой простой framework	406
26.3	Свой test-framework	406
26.4	Юнит-тесты модулей	408
26.4.1	Парсер и save/load	409
26.4.2	main и запуск	411
26.5	Сборка на нескольких платформах	411
26.6	Что мы построили — обзор	413
26.7	Что покрыли в C++	413
26.8	Что осталось «за кадром»	414
26.9	Если бы это шло в работу	415
26.10	Итог Части II	416
26.11	Главные правила главы	416
26.12	Маленькое упражнение	417
26.13	Что дальше	417
27	Глава 26. Процессы — fork, exec, wait	418
27.1	Что мы вообще строим	418
27.2	Где это не заработает	419
27.3	Процесс глазами ядра	420
27.4	fork — процесс раздваивается	421
27.4.1	Загадка удвоенной строки	422
27.4.2	Два адресных пространства	423
27.5	exec — процесс превращается в другую программу	425
27.5.1	Что происходит, когда программы нет	426
27.5.2	_exit вместо exit	427
27.5.3	Канонический шаблон	428
27.6	wait — забрать результат	428
27.7	Зомби и сироты	430
27.8	Собираем myshell v1	431
27.9	Что уже сломано	438
27.10	Как v1 превратится в финальный код	439
27.11	Чем смотреть внутрь	439
27.12	Главные правила главы	440
27.13	Упражнения	440
27.14	Что дальше	441
28	Глава 27. Файловые дескрипторы и каналы	442
28.1	Что такое файловый дескриптор	442
28.1.1	Сырые чтение и запись	444
28.2	Дескрипторы переживают fork и exec	444
28.3	pipe — канал в ядре	446
28.4	Канал между процессами и правило закрытия	447
28.4.1	Сколько влезает в канал	448
28.4.2	Что будет, если читателя не стало	450
28.5	dup2 — подменить номер	451
28.6	Цепочка из двух команд	452
28.7	Цепочка из N команд	453

28.8	Реализация в demo-shell	453
28.9	Проверка	460
28.10	Главные правила главы	460
28.11	Упражнения	461
28.12	Что дальше	462
29	Глава 28. Сигналы	463
29.1	Что такое сигнал	463
29.2	Словарь сигналов	464
29.3	Как сигналы посылают	465
29.4	signal и почему им не пользуются	466
29.5	sigaction	466
29.6	Что можно делать в обработчике	467
29.7	EINTR и SA_RESTART	470
29.8	Сигналы не копят	471
29.9	Блокировка сигналов	472
29.10	Что наследуется через fork и exec	473
29.10.1	Побочный эффект, о который легко споткнуться	475
29.11	SIGCHLD	475
29.12	SIGPIPE	476
29.13	SIGTERM и аккуратное завершение	476
29.13.1	Сигнал во время waitpid	477
29.14	Реализация в demo-shell	478
29.15	Проверка	480
29.16	Два приёма, которые пригодятся дальше	480
29.16.1	Кто послал сигнал: SA_SIGINFO	481
29.16.2	Self-pipe: превратить сигнал в обычное событие	482
29.17	Чего мы не сделали	483
29.18	Главные правила главы	483
29.19	Упражнения	484
29.20	Что дальше	485
30	Глава 29. Парсер командной строки	486
30.1	Что должно получиться	486
30.2	Две ступени разбора	487
30.3	Токенизатор	488
30.4	Тесты токенизатора	492
30.5	Парсер	496
30.6	Применение перенаправлений	498
30.7	Порядок перенаправлений решает всё	501
30.8	Перенаправления вместе с конвейером	502
30.9	Проверка	503
30.10	Чего парсер не умеет	504
30.11	Три ступени, если посмотреть сверху	504
30.12	Главные правила главы	505
30.13	Упражнения	505
30.14	Что дальше	506

31	Глава 30. Встроенные команды и история	507
31.1	Три причины быть встроенной командой	507
31.2	Точка входа	508
31.3	Почему встроенные команды не работают в конвейере	509
31.4	Реализации	510
31.4.1	cd	510
31.4.2	pwd	511
31.4.3	export и unset	512
31.4.4	Как окружение попадает в ребёнка	513
31.4.5	echo	514
31.4.6	exit	514
31.4.7	Диспетчер	515
31.5	alias: сокращения для команд	516
31.6	Две разные вещи, которые называют переменными	518
31.7	Как оболочка ищет программу	519
31.7.1	Свой type	520
31.8	История команд	522
31.8.1	Компромисс с сохранением	525
31.9	Почему не работает стрелка вверх	525
31.10	Проверка	526
31.11	Главные правила главы	527
31.12	Упражнения	528
31.13	Что дальше	528
32	Глава 31. Свои cat, wc, grep и head	530
32.1	Модель фильтра	530
32.2	mycat	531
32.2.1	Проверка	533
32.2.2	Так read или ifstream?	534
32.3	mywc	534
32.3.1	UTF-8 в две строки	536
32.3.2	Аргументы и вывод	536
32.3.3	Проверка	538
32.4	mygrep	538
32.4.1	Поиск	539
32.4.2	Компиляция шаблона	540
32.4.3	Код возврата: три состояния	541
32.4.4	Сколько стоит std::regex	542
32.5	myhead и умение вовремя остановиться	542
32.6	Что происходит, когда читатель уходит раньше	545
32.7	Буферизация: почему вывод иногда «залипает»	545
32.8	Всё вместе	546
32.9	Что не сделано	547
32.10	Часть III закончена	547
32.11	Главные правила главы	548
32.12	Упражнения	548
32.13	Что дальше	549

33	Глава 32. Бинарный формат данных	550
33.1	Текст против байтов	550
33.1.1	Насколько именно быстрее	551
33.2	Что компилятор делает со структурой	552
33.3	Почему структуру нельзя писать в файл	554
33.3.1	Дыры не инициализируются	554
33.3.2	Размеры типов разные	555
33.3.3	Порядок байтов	555
33.3.4	Разные компиляторы раскладывают по-разному	555
33.4	Явная сериализация	555
33.4.1	Строки с префиксом длины	558
33.4.2	Знаковые числа	558
33.4.3	Числа с плавающей точкой и правило про memsru	559
33.4.4	Сетевой порядок байтов	560
33.5	Чем смотреть в бинарный файл	560
33.6	Метка формата и версия	562
33.7	Переменная длина: как сэкономить на маленьких числах	563
33.8	Развитие формата	565
33.9	Разберём чужой формат руками	565
33.10	Как выглядит остаток Части IV	566
33.11	Главные правила главы	567
33.12	Упражнения	567
33.13	Что дальше	568
34	Глава 33. Страничный файл и кэш	569
34.1	Почему страницы, а не записи	569
34.1.1	Почему 4096	570
34.2	Адресация: страница номер N	571
34.3	pread и pwrite	572
34.4	Зачем кэш	573
34.4.1	Структура данных для LRU	573
34.5	Грязные страницы	574
34.6	fsync: когда данные действительно на диске	575
34.7	Класс PageManager	576
34.7.1	Открытие	578
34.7.2	Чтение и запись	579
34.7.3	Загрузка в кэш	580
34.7.4	Выделение страницы	581
34.8	Кэш в работе	582
34.9	У операционной системы тоже есть кэш	583
34.10	Сколько страниц держать	583
34.11	Посмотрим на настоящий файл	584
34.12	Что здесь упрощено	585
34.13	Главные правила главы	585
34.14	Упражнения	586
34.15	Что дальше	586
35	Глава 34. B+-дерево	587

35.1	Зачем вообще дерево	587
35.1.1	Почему не двоичное дерево	588
35.1.2	В-дерево и В+-дерево	588
35.2	Узел на странице	589
35.2.1	Чтение и запись узла	590
35.3	Сколько ключей в узле	591
35.4	Поиск	592
35.5	Вставка	593
35.5.1	Разделение листа	594
35.5.2	Разделение внутреннего узла	595
35.5.3	Подъём наверх	596
35.6	Как это выглядит вживую	597
35.7	Обход диапазона	598
35.8	Повторяющиеся ключи	599
35.8.1	Ловушка, которую видно не сразу	600
35.9	Удаление	602
35.10	Чего ещё нет в учебной версии	602
35.11	Главные правила главы	603
35.12	Упражнения	603
35.13	Что дальше	604
36	Глава 35. Журнал упреждающей записи	605
36.1	Почему нельзя просто чаще звать fsync	605
36.2	Идея упреждающей записи	606
36.2.1	Где здесь транзакции	606
36.3	Формат записи	607
36.3.1	Про нашу контрольную сумму	608
36.4	Запись в журнал	608
36.5	Восстановление	609
36.6	Контрольная точка	611
36.7	Проверим на аварии	612
36.8	Сколько это стоит	614
36.9	Разбор по моментам аварии	614
36.10	Как это устроено у взрослых	615
36.11	Чего нет в нашем журнале	616
36.12	Главные правила главы	616
36.13	Упражнения	617
36.14	Что дальше	618
37	Глава 36. Мини-SQL	619
37.1	Зачем вообще язык	619
37.2	Что поддерживаем	620
37.3	Три ступени	620
37.4	Лексер	621
37.5	Парсер	624
37.5.1	Дерево разбора	627
37.5.2	Проверка конца	628
37.6	Исполнитель	628

37.6.1	Создание таблицы	629
37.6.2	Вставка	630
37.6.3	Выборка	630
37.7	Пробуем	632
37.8	Тесты разбора	633
37.9	Главные правила главы	634
37.10	Упражнения	635
37.11	Что дальше	635
38	Глава 37. Вторичные индексы и план запроса	637
38.1	Насколько всё плохо без индекса	637
38.2	Что такое вторичный индекс	638
38.3	Реализация	638
38.3.1	Создание	639
38.3.2	Обновление при вставке	640
38.3.3	Каталог: чтобы индекс пережил перезапуск	641
38.4	Планировщик	643
38.4.1	EXPLAIN	644
38.5	Выполнение	645
38.6	Сколько выигрывают и сколько теряют	646
38.7	Чего мы не сделали	647
38.8	Главные правила главы	648
38.9	Упражнения	648
38.10	Что дальше	649
39	Глава 38. Оболочка базы данных	650
39.1	Цикл	650
39.2	Многострочный ввод	651
39.2.1	Ошибка, которую нашла проверка вручную	652
39.2.2	Чего не умеет наш поиск точки с запятой	652
39.3	Служебные команды	653
39.4	Комментарии	654
39.5	История	655
39.6	Ошибки не должны ронять программу	656
39.7	Завершение	657
39.8	Как это работает целиком	657
39.9	Чего не хватает	658
39.10	Главные правила главы	658
39.11	Упражнения	658
39.12	Что дальше	659
40	Глава 39. Замеры и итоги Части IV	660
40.1	Измеряем	660
40.2	Опыт, который меняет выводы	661
40.3	Где теряется время	662
40.4	Что мы построили	662
40.5	Как это соотносится с настоящими базами	664
40.6	Чему научила эта часть	664

40.7	Главные правила Части IV	665
40.8	Упражнения	665
40.9	Часть IV закрыта	666
40.10	Что дальше	666
41	Глава 40. Сокеты	667
41.1	Что такое сокет	667
41.2	TCP и UDP	668
41.3	Порядок вызовов	668
41.4	Создание сокета	669
41.5	Адрес	669
41.6	bind, listen, accept	670
41.7	Обмен данными	671
41.7.1	TCP — поток, а не сообщения	672
41.8	Клиент	673
41.9	Проверка	674
41.10	TIME_WAIT и SO_REUSEADDR	675
41.11	Чего наш сервер не умеет	676
41.12	Главные правила главы	676
41.13	Упражнения	677
41.14	Что дальше	677
42	Глава 41. Поток и мьютексы	678
42.1	Три способа обслуживать многих	678
42.2	Поток за две строки	679
42.3	Гонка данных вживую	679
42.3.1	Инструмент, который находит гонки	681
42.4	Мьютекс	681
42.5	Сервер чата	682
42.5.1	Обработчик	683
42.6	Проверка	685
42.7	Чем плоха модель «поток на клиента»	685
42.8	Что ещё есть в стандартной библиотеке	686
42.9	Главные правила главы	687
42.10	Упражнения	687
42.11	Что дальше	688
43	Глава 42. Атомарные операции	689
43.1	Почему ++ теряет половину	689
43.2	Мьютекс: правильно и дорого	690
43.3	Атомарная переменная	690
43.3.1	Что умеет std::atomic	691
43.4	Модель памяти и одно правило	692
43.4.1	Насколько редко — измерим	693
43.5	Ложное разделение	693
43.6	Что выбрать	694
43.7	В нашем сервере	695
43.8	Главные правила главы	695

43.9	Упражнения	696
43.10	Что дальше	696
44	Глава 43. Цикл событий	697
44.1	Почему потоков не хватает	697
44.2	poll	698
44.3	Неблокирующий режим	698
44.4	Состояние клиента	699
44.5	Цикл	700
44.5.1	Новые соединения	701
44.5.2	Чтение	702
44.5.3	Запись и ловушка с висящей ссылкой	703
44.6	Проверка	704
44.7	Чем приходится платить	705
44.8	Как это выглядит у взрослых	705
44.9	Главные правила главы	706
44.10	Упражнения	706
44.11	Что дальше	707
45	Глава 44. Свой двоичный протокол	708
45.1	Три способа разметить границы	708
45.2	Формат кадра	709
45.3	Кодирование	710
45.4	Разбор	711
45.5	Проверка на злом входе	712
45.6	Как это встраивается в сервер	713
45.7	Плата за двоичный формат	714
45.8	Чего нет в нашем протоколе	715
45.9	Главные правила главы	715
45.10	Упражнения	715
45.11	Что дальше	716
46	Глава 45. Комнаты	717
46.1	Что добавляется	717
46.2	Две структуры вместо одной	717
46.3	Обработка сообщений	718
46.4	Выход из комнаты: порядок решает	720
46.5	Рассылка	721
46.6	Клиент, понимающий протокол	722
46.7	Проверка	724
46.8	Слабые места	725
46.9	Главные правила главы	725
46.10	Упражнения	726
46.11	Что дальше	726
47	Глава 46. Шифрование и история	727
47.1	Шифр, который придумывают все	727
47.2	Ломаем за пять минут	728

47.3	Что делать вместо	729
47.4	История сообщений	729
47.4.1	Чтение последних N	730
47.4.2	Наивность, о которой надо знать	732
47.5	Как это встроить в сервер	733
47.6	Хранить ли переписку вообще	733
47.7	Главные правила главы	734
47.8	Упражнения	734
47.9	Что дальше	735
48	Глава 47. Сборка, упаковка и итоги Части V	736
48.1	Почему бинарник не переносится	736
48.2	Статическая сборка	737
48.2.1	Что не так со статическим glibc	737
48.2.2	musl	738
48.3	Контейнер	738
48.3.1	Чего в нашем контейнере не хватает	739
48.4	Что построено в Части V	740
48.5	Чему научила эта часть	740
48.6	Главные правила Части V	741
48.7	Упражнения	741
48.8	Что дальше	742
49	Глава 48. C++17: optional, variant, структурные привязки	743
49.1	std::optional: значение или ничего	743
49.2	std::variant: одно из нескольких	745
49.3	Структурные привязки	747
49.4	if constexpr	748
49.5	Что бы изменилось в нашем коде	749
49.6	Что ещё есть в C++17	749
49.7	Главные правила главы	750
49.8	Упражнения	750
49.9	Что дальше	751
50	Глава 49. std::filesystem	752
50.1	Путь — не строка	752
50.2	Операции с файлами и каталогами	753
50.3	Обход каталогов	754
50.4	Ошибки: две версии каждой функции	755
50.5	Чего библиотека не делает	756
50.6	Как это выглядело бы в нашей книге	756
50.7	Главные правила главы	756
50.8	Упражнения	757
50.9	Что дальше	758
51	Глава 50. string_view и параллельные алгоритмы	759
51.1	Проблема копирования строк	759
51.2	Сколько выигрывает	760

51.3	Опасность, которой раньше не было	760
51.4	Ещё одна тонкость: нулевой байт	761
51.5	Где он пригодился бы нам	762
51.6	Параллельные алгоритмы	762
51.6.1	Что нужно понимать до применения	763
51.6.2	А теперь неприятное	763
51.7	Главные правила главы	764
51.8	Упражнения	764
51.9	Что дальше	765
52	Глава 51. Куда дальше — обзор C++20/23 и маршрут роста	766
52.1	Концепты — конец SFINAE-ужасов	766
52.2	Ranges — конвейеры вместо «итератор, итератор»	767
52.3	Корутины — функции, умеющие приостанавливаться	768
52.4	Модули — прощание с #include	769
52.5	Несколько штрихов C++23	769
52.6	Что из этого можно брать уже сейчас	770
52.7	Куда расти после этой книги	771

Глава 1

Вступление. Зачем эта книга

1.1 О чём она

C++ — язык, на котором написана половина того, чем мы каждый день пользуемся: ядра баз данных (PostgreSQL частично, ClickHouse, RocksDB, FoundationDB), браузеры (Chrome, Firefox, Safari), графические движки (Unreal, Unity-native), пакеты типа Photoshop и Blender, торговые системы биржи, прошивки самолётов и автомобилей. На нём же написаны компиляторы JavaScript (V8) и Python (CPython). И при всей этой важности — это, наверное, самый непонятный язык из массово используемых. Я сам, когда первый раз открыл «C++ Primer» Липпмана, на двадцатой странице утонул в `cin >> x;` и захлопнул книгу: половина символов мне ничего не говорила, а автор как будто считал само собой разумеющимся, что я знаю, чем `int*` отличается от `int&`.

Эта книга писалась именно так, как мне самому когда-то нужна была. Я не делаю вид, что читатель — выпускник МГУ с кафедры вычмата. Книга рассчитана и на тех, кто уже знает другой язык (Python, PHP, JS), и на тех, у кого C++ — вообще первый язык программирования. Для вторых первые пять глав (Часть 0a) — это разгон с нуля: что такое компьютер, что такое программа, как устроены биты и байты, что такое алгоритм, как пользоваться терминалом. Без этого даже самый простой C++-код кажется магией. Если у вас уже есть опыт другого языка — пробежите эти главы по диагонали, основное всё равно полезно.

А дальше — за руку, со всеми объяснениями. Если в коде стоит звёздочка, я расскажу, что она означает. Если стоит амперсанд, расскажу, что он означает. Если на одной строке встретятся `&&`, я расскажу, что это вообще не два амперсанда, а отдельная штука. Я не считаю, что у читателя «и так должно быть понятно».

Главное отличие от стандартных C++-учебников — мы не пишем тридцать «hello world» с разными вариациями. Мы делаем **четыре полноценных консольных проекта** по нарастанию сложности. К концу книги у вас в репозитории будут собраны и работают:

1. **Текстовая RPG-игра** в стиле Zork — с локациями, инвентарём, NPC, боями и сохранениями.
2. **Свой мини-shell** — bash-подобная оболочка с pipes, redirections, builtin-командами и историей.
3. **Мини-СУБД** — key-value база данных с B+tree индексом, журналом изменений и собственным мини-SQL.
4. **TCP-мессенджер** — сервер и клиент с многокомнатным чатом, серверная часть на event loop.

Это не игрушки на пятнадцать минут. Это работающие проекты, на которых видно, что C++ — это не «странный язык со звёздочками», а инструмент промышленного уровня. И каждое новое языковое средство я ввожу тогда, когда оно реально нужно для нашего следующего шага в проекте, а не «впрок».

Книга — это не «C++ за выходные». Её читают вечерами по часу, и через пару месяцев получают **понимание**, как устроен язык и что им делать. Каждая глава самодостаточна: вы можете отложить книгу на неделю и вернуться, не потеряв нить. Каждый кусок кода в книге собирается своей командой компиляции, и команда тоже разобрана.

1.2 Кому она

Полным новичкам, для которых C++ — это первый язык программирования. Чаще всего так: подросток заинтересовался играми, услышал что Minecraft был на Java, а его движок Bedrock — на C++, и решил «начну с серьёзного». Или взрослый человек после математической специальности захотел научиться писать программы и не нашёл, с чего начать. Для них Часть 0a — это вход с улицы: без знания терминала, без знания, что такое файл, без знания, как считать в двоичной системе. Я предполагаю только одно: вы умеете пользоваться компьютером на бытовом уровне (открывать программы, искать файлы в файловом менеджере), и у вас есть желание разобраться. Дальше — за руку.

Тем, кто закончил основы программирования на любом другом языке и хочет понять C++. Не «выучить пять команд, чтобы пройти курс в вузе», а понять — как программа на C++ устроена, во что превращается, какие у неё бывают проблемы и как их избегать. Я считаю это самым полезным навыком в индустрии: даже если вы не будете писать на C++ ежедневно, понимание того, что под капотом, делает вас лучше как разработчика на любом другом языке. Когда вы знаете, что строка в Python — это объект с шапкой и указателем на буфер на куче, и что копирование строк не бесплатно, вы пишете лучший Python.

Школьникам и студентам, которым на учебе впахнули C++ и сказали «разбирайтесь». В стандартных учебниках авторы либо начинают с обсуждения «парадигм программирования» (вода), либо сразу прыгают в STL без объяснения, что такое память. Я начинаю с того, что у вас на экране — текстовый редактор, в нём вы пишете символы, и эти символы превращаются в исполняемый файл вот таким способом. Без воды.

Программистам с других языков, которым нужно «прочитать чужой C++-код и не сойти с ума». Особенно если вы пришли из Go, JavaScript или Python — у вас часто кружится голова от move-семантики, шаблонов и `const`. Я тщательно разбираю эти моменты, потому что сам когда-то на них спотыкался.

Кому НЕ подойдёт эта книга:

- если вам нужен `embedded` или микроконтроллеры — я касаюсь `systems-programming`, но не делаю фокус на работе с регистрами и прерываниями;
- если вы хотите быстро «выучить только то, что спросят на собеседовании» — здесь много контекста и историй вокруг, что замедляет «зазубривание»;
- если вам нужна графика (OpenGL, Vulkan, DirectX) — в этой книге только консольные проекты, графика — отдельная огромная тема.

1.3 Почему C++ в 2026 году

«Зачем учить C++, если есть Rust / Go / современные языки?» — вопрос, который вам обязательно зададут. У меня на него четыре ответа.

Первый — **зрелость экосистемы**. C++ существует с 1985 года. За сорок лет на нём написано столько кода, что выкинуть его и переписать на Rust невозможно физически. Это значит: вакансий на поддержку и развитие этого кода десятки тысяч, и они никуда не денутся ещё лет двадцать минимум. Если вы хотите работать в системном программировании, базах, играх, финтехе — вам надо знать C++.

Второй — **производительность**. Когда вы пишете на C++ грамотно, программа выполняется со скоростью железа. Не «как Java с `jit`-ом», не «как Go с GC-паузами», а реально так быстро, насколько способен процессор. Это критично там, где счёт идёт на наносекунды: трейдинг, рендеринг, аудио-обработка, низкоуровневая графика.

Третий — **контроль**. В C++ нет ничего, что бы выполнялось «само». Ни сборщика мусора, ни автоматических аллокаций, ни магических промисов. Каждое выделение памяти — это ваш код. Каждое копирование — ваш выбор. Этот контроль одновременно и сила, и боль: сила, потому что вы понимаете, что происходит; боль, потому что любая ошибка — ваша. И именно из-за этой боли C++ хорошо учит думать.

Четвёртый — **универсальность**. Один и тот же C++-код собирается под x86-64 Linux, ARM Apple Silicon, Windows, Android NDK, iOS, Raspberry Pi, микроконтроллеры (с оговорками), даже под WebAssembly через emscripten. Ни один другой язык такой переносимости не даёт.

Стоит ли C++ называть «устаревшим»? Стандарт C++23 принят в 2024 году, C++26 на подходе. Язык активно развивается: ranges, concepts, modules, coroutines — это всё современные средства, которые на голову выше того, что было даже в C++11. Просто C++ обязан тащить совместимость с кодом сорокалетней давности — отсюда репутация «динозавра».

1.4 Что мы построим — детальнее

Раз уж проекты сквозные, расскажу подробнее, что вы получите.

1.4.1 Проект 1: текстовая RPG-игра demo-rpg/

Игра в духе Zork (Infocom, 1980) и «Колобка» (если вы помните 90-е на советских ПК). Никакой графики — только текст в терминале:

```
> look
Лесная поляна. На западе виднеется тропа, ведущая к деревне.
На востоке — тёмный лес. На земле лежит ржавый меч.
> take меч
Вы подняли ржавый меч.
> go east
Тёмный лес. Ветви скрипят над головой.
Здесь стоит гоблин с дубиной.
> attack гоблин
Вы ударили гоблина мечом. Урон: 8. У гоблина осталось 4 HP.
Гоблин ударил вас дубиной. Урон: 3. У вас осталось 17 HP.
> attack гоблин
Гоблин повержен. Получено 15 опыта.
Из тела выпало: золото (3 монеты), кусок мяса.
> save my_game
Игра сохранена в data/saves/my_game.sav
```

В коде там будет всё, что нужно для понимания ООП в C++: иерархия классов (Entity → Player → Enemy → NPC), отдельная иерархия предметов (Item → Weapon/Armor/Consumable), полиморфные эффекты, шаблонная шина событий, лямбды для AI поведений, умные указатели для управления памятью, парсер сценария игры из текстовых файлов, сохранения

в бинарном и текстовом формате. Игру можно расширять — добавлять новые локации просто редактируя `data/world.txt`, без перекомпиляции.

1.4.2 Проект 2: мини-shell `demo-shell/`

Bash в миниатюре. Поддерживает:

```
$ ls | grep .cpp | wc -l
12
$ echo "hello" > out.txt
$ cat out.txt
hello
$ sleep 5 &
[1] 4823
$ jobs
[1] Running    sleep 5
$ cd ..
```

Внутри: парсер командной строки, который умеет в кавычки и escape-последовательности; пайплайны через `pipe()` и `dup2()`; редиректы `>`, `>>`, `<`, `2>&1`; фоновые задачи через `&`; список `jobs`; история команд в `~/.cpp_shell_history`; навигация стрелками. Плюс три «карманные» утилиты, написанные с нуля: `cat`, `wc`, `grep` — чтобы было что запускать в `pipe`-ах.

Этот проект учит работе с операционной системой через POSIX API: процессы (`fork/exec/wait`), файловые дескрипторы, сигналы, `terminal raw mode`. С Windows тут чуть сложнее — нативные API другие; мы обсудим в самой главе.

1.4.3 Проект 3: мини-СУБД `demo-db/`

Своя key-value база данных с настоящим, не игрушечным движком. По функциям:

```
db> CREATE TABLE users (id INT PRIMARY KEY, name TEXT, age INT);
OK (0.001s)
db> INSERT INTO users VALUES (1, 'Alice', 30);
db> INSERT INTO users VALUES (2, 'Bob', 25);
db> SELECT * FROM users WHERE age > 20;
id | name  | age
---|-----|----
1  | Alice | 30
2  | Bob   | 25
(2 rows, 0.0003s)
db> CREATE INDEX idx_age ON users(age);
```

```
db> EXPLAIN SELECT * FROM users WHERE age = 30;
```

```
Index scan: idx_age
```

```
db> .save
```

```
Database flushed to disk.
```

Внутри: страничный файл (страница 4 KiB, как у настоящих БД), LRU page cache, B+tree индекс с разбиением и слиянием узлов, write-ahead log для атомарности транзакций, парсер мини-SQL (CREATE/INSERT/SELECT/UPDATE/DELETE с WHERE и индексами), REPL с историей команд. Это **реальный** учебный движок БД, не «hello world».

Зачем нужно? Чтобы вы поняли, как устроены настоящие СУБД изнутри. После этого проекта вы по-другому будете писать SQL: вы будете чувствовать, какие запросы быстрые, а какие медленные, и почему.

1.4.4 Проект 4: TCP-мессенджер demo-chat/

Сетевой чат. Сервер слушает порт, клиенты подключаются и общаются в комнатах:

```
$ ./chat-server --port 9001
```

```
[server] listening on :9001
```

```
[server] client connected: 127.0.0.1:54021 (id=1)
```

```
[server] client connected: 127.0.0.1:54033 (id=2)
```

```
[server] [room=general] <user_1> hello!
```

```
[server] [room=general] <user_2> hi there
```

```
# В другом терминале:
```

```
$ ./chat-client --server 127.0.0.1:9001 --nick alice
```

```
* Connected. Type /help for commands.
```

```
/join general
```

```
* Joined room "general"
```

```
hello!
```

```
<user_2> hi there
```

```
/leave
```

```
* Left room "general"
```

```
/quit
```

Внутри: сокеты, многопоточность, atomic-операции, реактор на select/poll/epoll/kqueue, свой бинарный протокол с фреймированием, шифрование XOR pre-key (учебное), история чата на диске. Эта часть — для тех, кто хочет понять, как устроены онлайн-игры, мессенджеры, прокси-серверы и вообще любой network-софт.

1.5 Что НЕ войдёт (и почему)

Книга и так длинная. Поэтому некоторые темы я сознательно вынес «за скобки»:

- **GUI** (Qt, wxWidgets, Win32, GTK) — отдельная огромная тема, требует отдельной книги. Мы строго в консоли.
- **Compiler internals** — мы будем разбирать, как программа компилируется, но без углубления в внутренности LLVM/GCC.
- **Шаблонная метапрограммация на полную глубину** — введение в шаблоны будет; SFINAE, type traits на уровне «что это и где работает», но без <type_traits> от и до. Это материал для второго курса, не первого.
- **Coroutines/concepts/ranges из C++20** — обзорно в последней главе, без глубокого погружения. Книга и так на C++11 как базе.
- **Cross-platform GUI / iOS / Android** — за рамками. В сетевом проекте мы делаем POSIX-ориентированный код с короткими отступлениями про WinSock.
- **Real-time audio/video** — не моя область, не буду делать вид, что эксперт.

1.6 Какие у меня соглашения

Несколько технических моментов, чтобы потом не возвращаться.

Шрифт кода. Весь код будет в моноширинном блоке:

```
int main() {  
    return 0;  
}
```

Если что-то надо подсветить в строке текста — оно тоже в моноширинном виде, как `int` или `std::vector`. Имена файлов — `main.cpp`. Команды терминала отделены долларом:

```
$ g++ -std=c++11 main.cpp -o hello  
$ ./hello
```

Платформы. Я писал и тестировал всё на macOS (Apple Silicon, clang) и Ubuntu Linux 22.04 (gcc). Если будут особенности Windows — отмечу отдельно. Если для какой-то главы Windows нативно не подходит — честно скажу.

Стандарт. Везде, где не указано иное, код собирается командой:

```
$ g++ -std=c++11 -Wall -Wextra -pedantic file.cpp -o file
```

Если строка кода требует C++14 или C++17, я об этом скажу в тексте и в комментарии к коду.

Версии компиляторов. `clang` ≥ 11 , `gcc` ≥ 7 . На более старых может что-то не собраться, но это редкость. На современных Apple Silicon — Apple clang 14+ из коробки.

Структура папок. Все четыре проекта живут в корне репозитория книги: `demo-rpg/`, `demo-shell/`, `demo-db/`, `demo-chat/`. Само текстовое содержимое книги — в `docs/tutorial/00-intro.md`, `01-toolchain.md` и так далее. Сборка PDF из markdown — `build/build-pdf.sh`.

1.7 Как читать

По порядку. Я не делал «книгу-справочник, где можно прыгать по главам». В главе 12 я считаю, что вы уже понимаете, что такое класс и конструктор (глава 11), и не объясняю заново. Если что-то непонятно — лучше вернуться назад, чем читать дальше.

Не пропускайте код. В книге сотни примеров, и я часто говорю «соберите этот файл, запустите, посмотрите вывод». Это не оборот речи — если не запустить, теряется половина смысла. Программирование учится только пальцами.

Не торопитесь. Глава за вечер — идеальный темп. Если за час прочитали три главы — скорее всего, не задержались на коде. Вернитесь.

Делайте свои изменения в коде проектов. Я предлагаю упражнения в конце глав, но самое главное — играть с кодом. Сломайте что-нибудь и посмотрите, что вышло. Это лучший способ понять язык.

1.8 Что дальше

Если вы вообще не знакомы с программированием, дальше идёт **Часть 0a** — пять подготовительных глав. С первой по пятую: устройство компьютера, биты и байты, что такое программа, как думают алгоритмически, как пользоваться терминалом. Без этого фундамента глава про C++-компилятор покажется магией. Если опыт другого языка у вас есть — пробежите по диагонали, но не пропускайте полностью: контекст «зачем C++ устроен именно так» строится именно в этой части.

Глава 6 — про инструменты. Поставим компилятор, разберём первый «hello world» и посмотрим, что происходит между моментом, когда вы нажимаете Enter в терминале, и моментом, когда программа выводит «Hello, world». На это уйдёт целая глава, потому что событий там — десятки, и каждое стоит понять.

Поехали.

Глава 2

Глава 1. Что такое компьютер

Я начну с самого начала — буквально с того, как устроена железка, на которой вы сейчас читаете эту книгу. Если вы уже хорошо понимаете, что такое процессор, оперативная память и операционная система — можете пробежать главу по диагонали. Но даже опытные разработчики иногда путаются в простых вещах вроде «чем отличается ОЗУ от диска», и это потом мешает понимать C++. Так что не лишним будет освежить.

2.1 Зачем это в книге про C++

Большинство современных языков (Python, JavaScript, Java) скрывают железо от программиста. Вы пишете `x = 5`, и думать о том, где именно в физической памяти лежит эта пятёрка, — необязательно. Сам язык за вас об этом заботится: запускает «сборщик мусора», убирает ненужные значения, выделяет память.

C++ — другой. В C++ вы постоянно сталкиваетесь с понятиями вроде «адрес в памяти», «стек», «куча», «размер типа», «cache miss». Без понимания, что это значит на железе, код покажется набором заклинаний. Не «эта программа работает», а «эта программа работает, потому что компилятор разместил эти три переменные подряд в кэш-линии». Если разница пока непонятна — она прояснится по ходу книги. А чтобы было от чего отталкиваться, начнём с устройства железки.

2.2 Что такое компьютер на самом деле

Компьютер — это машина, которая умеет делать одну простую вещь: брать числа, выполнять над ними арифметику или логику, и записывать результат обратно. Всё остальное — текст, картинки, видео, музыка, игры — это надстройка над этой простой механикой. Текст — это последовательность чисел, каждое из которых обозначает букву. Картинка — это сетка чисел,

каждое из которых обозначает цвет точки. Видео — это последовательность таких сеток. И так далее.

Внутри компьютера живут пять ключевых компонентов. Расскажу про каждый.

2.2.1 Процессор (CPU)

Центральный процессор — это собственно «мозг». Кусок кремния размером с ноготь, на котором миллиарды (буквально миллиарды) транзисторов. У современного процессора Apple M3 — около 25 миллиардов транзисторов; у Intel Core i9 — около 12-15 миллиардов; у простенького ARM-процессора для микроконтроллера может быть и несколько миллионов.

Что процессор умеет? Очень мало. По сути — выполнять «инструкции». Каждая инструкция — это маленькое действие: «сложи два числа», «сравни два числа и запомни результат сравнения», «прочитай число из памяти», «запиши число в память», «прыгни в другое место программы». Всё. Никаких «нарисуй картинку» или «открой файл» процессор сам не умеет — это уже надстройки.

Список инструкций, которые понимает процессор, называется его **архитектурой** (или «инструкционным набором»). Самые известные сейчас две: **x86-64** (она же AMD64) — используется в большинстве настольных компьютеров и серверов; и **ARM64** (она же AArch64) — используется в телефонах, планшетах, новых Mac на Apple Silicon. Между ними есть фундаментальные различия (например, x86-64 умеет складывать сразу пять чисел одной инструкцией, ARM64 — за две). Но для нас важно одно: программа, скомпилированная под x86-64, не запустится на ARM64 и наоборот. Нужна перекомпиляция.

Процессор не выполняет инструкции одну за одной с одной скоростью. Современный процессор работает на частоте 3-5 ГГц — это значит «3-5 миллиардов тактов в секунду». На каждом такте процессор может начать новую инструкцию (или несколько — современные процессоры умеют параллелизм внутри одного ядра, об этом потом). Получается, простая инструкция типа сложения выполняется за доли наносекунды. Но обращение в память — куда медленнее, чем сложение. Это ключевой факт, к которому мы будем часто возвращаться.

У процессора есть ещё одна важная штука — **регистры**. Это маленькие «карманы» прямо внутри процессора, в которых лежат числа, с которыми он сейчас работает. На x86-64 регистров общего назначения 16 штук, по 8 байт каждый. На ARM64 — 31 штука. Когда инструкция говорит «сложи два числа», она имеет в виду «сложи числа в этих двух регистрах». Если число лежит не в регистре, а в памяти, его сначала надо в регистр прочитать.

Регистры — это самая быстрая память в системе. Доступ к регистру занимает один такт. К сравнению, доступ к оперативной памяти — сотни тактов. К диску — миллионы. Запомните этот порядок, он часто всплывёт.

2.2.2 Оперативная память (RAM)

RAM (Random Access Memory) — это рабочая память. Большой массив байтов, к каждому из которых можно обратиться по адресу. У современных компьютеров — 8, 16, 32, 64 ГБ. ГБ — гигабайт, миллиард байт (точнее, 2^{30} , но это нюансы).

Слово «random access» — «произвольный доступ» — означает: к любому байту можно обратиться напрямую по адресу, без чтения соседних. Это отличает RAM от ленты — ленту приходится перематывать.

Когда вы запускаете программу — операционная система берёт её код с диска и кладёт в RAM, потому что процессор может читать инструкции и данные только из RAM (не с диска напрямую). Когда программа создаёт переменную — переменная появляется в RAM. Когда программа складывает два числа — она берёт их из RAM, кладёт в регистры процессора, складывает, и результат пишет обратно в RAM.

Память называется **оперативной** или **энергозависимой**, потому что при выключении компьютера её содержимое исчезает. Всё, что вы хотите сохранить — должно быть записано на диск.

Адрес в памяти — это число. На современных 64-битных системах адрес — 64-битное число (8 байт), то есть теоретически можно адресовать $2^{64} \approx 18$ квинтиллионов байт. На практике используется только часть (обычно 48 битов = 256 ТБ). Но факт: каждый байт в памяти имеет свой адрес, и этот адрес — обычное число. Когда мы дойдём до указателей в C++, окажется, что **указатель — это просто число**, которое значит «адрес в памяти». Это сильно поможет потом не путаться.

2.2.3 Кэш процессора

Я сказал выше: обращение в RAM — сотни тактов. Это очень долго по меркам процессора. Если бы каждое чтение действительно занимало 200 тактов — программа летала бы со скоростью улитки.

Поэтому между процессором и RAM встроили **кэш** — маленькая, но очень быстрая память прямо на чипе процессора. Когда процессор обращается к памяти по адресу X, он на самом деле сначала смотрит в кэш — есть ли там копия данных по этому адресу? Если есть — берёт оттуда (1-3 такта). Если нет — обращается в RAM (~200 тактов) и заодно копирует данные в кэш, чтобы следующее обращение было быстрым.

Кэшей обычно несколько уровней: - **L1** — самый быстрый и маленький (32-128 КБ), отдельный для каждого ядра. - **L2** — побольше и помедленнее (256 КБ — 2 МБ), тоже на ядро. - **L3** — общий для всех ядер (4-64 МБ).

Кэш работает «страницами»: когда процессор читает один байт, в кэш загружается целая «кэш-линия» — обычно 64 байта. Это важно! Если ваши данные лежат в памяти подряд — следующее чтение, скорее всего, попадёт в кэш. Если разбросаны по памяти — каждое чтение требует похода в RAM. Поэтому в C++ часто говорят «вектор быстрее списка», «структура данных, дружелюбная к кэшу, важнее теоретической сложности алгоритма».

В этой книге я не буду углубляться в оптимизацию под кэш — это материал отдельной книги. Но знать, что кэш существует, важно, потому что многие решения языка C++ (например, `std::vector` vs `std::list`) объясняются именно через кэш.

2.2.4 Диск (SSD/HDD)

Диск — это место, где данные хранятся при выключенном компьютере. Современные ноутбуки используют SSD (Solid State Drive) — флеш-память, без движущихся частей. Старые компьютеры — HDD (Hard Disk Drive), магнитные диски с механическими головками.

Диск медленный по сравнению с RAM. SSD — в 100-1000 раз медленнее RAM. HDD — в 10000-100000 раз. Поэтому файл с диска сначала читается в RAM, и дальше программа работает с этой копией.

Диск важен для нас в C++ потому, что сохранения игры, файлы базы данных, исходники программ — всё это на диске. В Части III (mini-shell) и Части IV (мини-СУБД) мы будем плотно работать с файлами.

2.2.5 Операционная система

Между вашей программой и железом стоит **операционная система** (ОС). У вас, скорее всего, либо macOS (на Mac), либо Windows, либо Linux (если вы продвинутый пользователь). На телефоне — iOS или Android.

Операционная система выполняет несколько ключевых задач:

1. **Управление процессами.** Процесс — это запущенная программа. На вашем компьютере прямо сейчас работают сотни процессов параллельно: браузер, мессенджер, фоновый монитор обновлений, и так далее. ОС переключает процессор между ними сотни раз в секунду, создавая иллюзию параллельности. Каждому процессу ОС выделяет своё «адресное пространство» — то есть свою куски памяти, отдельные от других. Если бы процессы делили память без изоляции — баг в одном уронил бы все.
2. **Управление памятью.** Когда программа просит «дай мне 100 байт», она обращается на самом деле к ОС. ОС находит свободный кусок памяти и отдаёт. Когда программа закрывается, ОС автоматически возвращает себе всю её память.

3. **Доступ к железу.** Программа не общается с диском напрямую — она просит ОС «прочитай мне такой-то файл». ОС сама общается с диском и возвращает данные. То же с сетью, принтером, экраном.
4. **Файловая система.** Диск — это просто массив байтов. Чтобы хранить там осмысленно «файлы» и «папки», ОС поверх диска организует файловую систему: APFS на Mac, NTFS на Windows, ext4 на Linux. Программа работает с «файлом по пути /Users/me/photo.jpg», а под капотом ОС находит, на каких именно секторах диска лежат данные.
5. **API для программ.** ОС предоставляет программам стандартные функции: открыть файл, прочитать байты, нарисовать окно, услышать клавишу. Это называется **системные вызовы**. На macOS/Linux большинство системных вызовов соответствуют стандарту POSIX. На Windows — другой стандарт (Win32 API). Это причина того, что Windows и macOS-программы не совместимы напрямую.

В C++ системные вызовы делаются через стандартную библиотеку (которая под капотом обращается к ОС). Например, когда вы пишете `std::cout << "hello"`, под капотом происходит системный вызов `write()` (на Unix) или `WriteFile()` (на Windows). Когда вы открываете файл — `open()` или `CreateFile()`. И так далее.

В Части III книги мы будем работать с системными вызовами напрямую, минуя стандартную библиотеку — чтобы написать свой shell. Это даст вам понимание, что под капотом у каждой программы — десятки таких вызовов.

2.3 Что такое процесс

Раз уж я упомянул процессы — расскажу подробнее. Процесс — самое важное понятие, без которого C++ не понять.

Когда вы дважды щёлкаете по программе (или запускаете из терминала) — ОС создаёт новый процесс. Это значит:

1. **Выделяется адресное пространство.** ОС резервирует кусок виртуальной памяти, в котором будет жить процесс. На 64-битной системе размер виртуального пространства — теоретически 256 ТБ, на практике это просто «много». Программа видит непрерывную область адресов от 0 до огромного числа, и думает, что вся память — её. Под капотом ОС динамически отображает виртуальные адреса на физическую RAM по мере надобности.
2. **В адресное пространство загружается код программы.** Бинарный файл с диска копируется в память. Точнее — отображается, через механизм `memory-mapping`; реально данные подгружаются по мере чтения.

3. **Создаётся стек.** Стек — это особая область памяти, где живут локальные переменные функций. О стеке мы будем говорить много, особенно в главе 11.

4. **Запускается `main`.** ОС находит в коде программы функцию `main` и передаёт ей управление. Дальше — работа уже программы.

Процесс изолирован от других процессов. Память процесса А не видна процессу Б. Если процесс А упал — он умирает, а процесс Б продолжает работать. Это огромное достижение операционных систем 70-80-х годов; до этого падение одной программы могло уронить всю систему.

У процесса есть набор «ресурсов» которые ОС ему выделила: - адресное пространство (память); - открытые файлы (через «файловые дескрипторы» — числа-номера); - сетевые соединения (тоже через файловые дескрипторы — в Unix всё файл); - сигналы, которые ему можно посылать (SIGTERM, SIGKILL, SIGINT).

Когда процесс закрывается — ОС автоматически освобождает все его ресурсы. Открытые файлы закрываются, память возвращается в общий пул, сетевые соединения разрываются. Этим часто пользуются: «утечка памяти — не страшно, всё равно через минуту процесс закроется и ОС всё уберёт». Это правда, но привычка такая плохая — на длинных процессах (серверы, демоны) утечки становятся проблемой.

В Части III книги мы будем создавать процессы из C++-кода (системный вызов `fork`), что бывает полезно для shell, серверов и параллельной обработки данных.

2.4 Что такое поток

Один процесс может состоять из нескольких **потоков** (threads). Поток — это «исполняющаяся последовательность инструкций». В простом случае процесс состоит из одного потока (главного), и он по очереди выполняет инструкции в программе. Когда нужен параллелизм — программа создаёт дополнительные потоки.

Чем поток отличается от процесса:

- Потоки одного процесса **делят память**. То есть переменная, созданная в одном потоке, видна другому потоку. Это и быстро (не надо ничего передавать), и опасно (легко получить гонки данных).
- Потоки одного процесса делят также открытые файлы, сетевые соединения, и так далее.
- Каждый поток имеет свой стек (отдельные локальные переменные).
- ОС переключает процессор между потоками точно так же, как между процессами.

Когда ноутбук пишет «8 ядер», это значит — 8 потоков могут реально выполняться одновременно, физически параллельно. На 8-ядерном процессоре десятки тысяч потоков

«крутятся», но реально активных в каждый момент времени — только 8 (плюс ещё «гипертрединг» удваивает это число — но грубо говоря, $8 = 8$ параллельных).

Многопоточность мы будем глубоко изучать в Части V (TCP-чат), где сервер должен обслуживать множество клиентов одновременно.

2.5 Что увидите в C++

Из всего сказанного выше в C++ напрямую отражены вот эти понятия:

- **Указатель** — переменная, в которой лежит адрес в памяти. Чем мощнее, чем боль — будем долго осваивать в главе 11.
- **Размер типа** — каждый тип в C++ имеет фиксированный размер в байтах (`int` обычно 4 байта, `double` 8 байт). Размер зависит от архитектуры процессора. Об этом — в главе 8.
- **Стек и куча** — две основные области памяти процесса. Стек — для локальных переменных, куча — для динамически выделяемой памяти. Глава 11.
- **`new` и `delete`** — обращение к ОС за памятью и возврат памяти ОС. Глава 11.
- **Файлы** — глава 12.
- **Потоки** (`std::thread`) — глава 41.
- **Процессы** (`fork`, `exec`) — глава 26.

Каждое из этих понятий — обёртка над тем, что я описал в этой главе. Когда вы будете встречать `int*` или `delete x`, вспоминайте: за этим стоит совершенно конкретное действие — выделение или возврат куска физической памяти.

2.6 На пальцах: жизнь программы Hello

Чтобы свести вместе всё, что я говорил, представим, что вы скомпилировали программу `hello`, которая выводит «Hello, world», и запустили её. Что произошло:

1. Вы написали `./hello` в терминале и нажали Enter.
2. Терминал — это тоже процесс. Он попросил у ОС создать новый процесс для `hello` (системный вызов `fork`, потом `exec`).
3. ОС нашла на диске файл `hello`, прочитала его заголовок (метаданные о том, как он устроен), выделила в памяти место для кода программы и скопировала туда содержимое файла. На самом деле скопировала не сразу, а отметила в таблице страниц «когда программа запросит этот участок памяти — подгружу данные с диска» (lazy loading через memory-mapping).
4. ОС выделила для процесса начальный стек (обычно 1-8 МБ виртуально, но физически пока только малая часть).

5. ОС нашла в программе функцию `main` и передала ей управление, то есть установила в регистр программного счётчика (RIP на x86-64) адрес первой инструкции `main`.
6. Процессор начал выполнять инструкции одну за одной.
7. Дошёл до инструкции, которая отвечает за `std::cout << "Hello, world"`. На самом деле это много инструкций; конечная цель — вызвать системный вызов `write` с указателем на строку и её длиной.
8. Процесс перешёл в режим ядра, ОС нашла файловый дескриптор 1 (стандартный вывод, обычно — терминал), записала туда байты «Hello, world».
9. Терминал увидел новые байты и отрисовал их на экране.
10. Программа вернулась из `main`, выполнила `return 0`. ОС увидела, что процесс хочет закрыться, освободила его память, файлы, и пометила процесс как завершённый.
11. Терминал увидел, что дочерний процесс умер, и снова показал приглашение к вводу команды.

Между шагами 1 и 11 проходит, наверное, миллисекунды. И за каждой из них стоит то, что я описал в этой главе.

2.7 Маленькое упражнение

Никакого C++ ещё нет, но небольшая разминка. Откройте задачник: попробуйте мысленно (или на бумаге) ответить на эти вопросы.

1. Если на вашем компьютере 16 ГБ RAM, и вы запустили программу, которая просит у ОС 32 ГБ памяти — что произойдёт?
2. Почему программа на C++, скомпилированная под Mac M3, не запустится на Windows-ноутбуке с Intel — даже если функционал у программы одинаковый?
3. Что произойдёт, если в коде есть «утечка памяти» — программа выделяет память и не освобождает её — и эта программа крутится месяцами?
4. Чем процесс отличается от потока? Назовите хотя бы три отличия.

Ответы на эти вопросы я не даю — попробуйте сами, и если затрудняетесь, вернитесь к нужному разделу. Это не контрольная; это способ проверить, что в голове осталось.

2.8 Что дальше

Глава 2 — про биты и байты. Раз компьютер всё хранит в виде чисел, надо понять, как именно. Что такое двоичная система? Почему именно она? Как с её помощью кодируются отрицательные числа, дробные числа, символы алфавита, текст на русском и казахском? Без этого знания нельзя понять, почему `int` в C++ — это «32-битное целое со знаком», и что значит

«переполнение signed int — это undefined behavior». Эту главу старайтесь не пропускать, даже если кажется, что знаете.

Глава 3

Глава 2. Биты, байты, числа, текст

В прошлой главе я несколько раз сказал «всё в компьютере хранится в виде чисел». Сейчас распакуем эту фразу. Откуда «всё»? И что это за «числа», если внутри процессора — миллиарды транзисторов, а не маленьких счётчиков?

Эта глава может казаться скучной — двоичная арифметика, ASCII-таблица, всё это вы могли проходить в школе. Но я прошу не пропускать. Без чёткого понимания, что такое «32-битное целое со знаком» и почему `char` в C++ занимает один байт, многое в дальнейшем будет казаться загадкой. Лучше потратить час сейчас, чем спотыкаться весь курс.

3.1 Бит — самая маленькая единица

Внутри процессора и памяти всё — это электрические сигналы. Сигнал может быть либо «включён» (есть напряжение), либо «выключен» (нет напряжения). Промежуточных состояний нет. Это техническое ограничение: транзисторы быстро и надёжно работают в режиме «открыт/закрыт». Сделать «полуоткрыт» — теоретически можно, но шумы и помехи в реальных схемах сразу превратят такое значение в одно из двух. Поэтому: только два состояния.

Эти два состояния принято обозначать **0** и **1**. И называется одна такая «ячейка» — **бит** (от англ. `binary digit` — двоичная цифра).

Один бит — это очень мало. Им можно различить две вещи: либо «нет», либо «да». Например, «лампочка горит» / «лампочка не горит». «Дверь открыта» / «закрыта». Реальные программы оперируют куда более сложными значениями (числа от 0 до миллиарда, текст, картинки), поэтому биты группируют.

3.2 Байт — группа из 8 бит

Стандартная группа — **8 битов**. Эти 8 битов вместе называются **байтом** (англ. byte). Один байт может принимать $2^8 = 256$ различных значений.

Почему именно 8? Историческая случайность. В 60-х годах разные компьютеры использовали разные размеры: на одних было 6 битов, на других 7, на третьих 9. Но в проекте System/360 (IBM, 1964) был выбран размер 8, потому что: (а) этого хватало для кодирования одной буквы латиницы плюс контрольных символов; (б) 8 удобно делится на 4, 2, 1 (можно использовать как два полу-байта по 4 бита). System/360 стал чрезвычайно популярен, и стандарт «1 байт = 8 бит» прижился. С тех пор все архитектуры — Intel, AMD, ARM, RISC-V — используют 8-битный байт.

В C++ строго гарантировано, что `char` — это один байт. Что байт содержит как минимум 8 битов. Технически стандарт C++ говорит «байт = `CHAR_BIT` битов, и `CHAR_BIT` ≥ 8 »; на всех современных платформах `CHAR_BIT` == 8. Можно считать это законом природы.

3.3 Двоичная система счисления

Раз 8 битов = 256 значений, как именно представить число от 0 до 255 этими восемью битами?

Так же, как мы представляем число в десятичной системе, только основание — 2 вместо 10. В десятичной системе число 247 разбирается так:

$$\begin{aligned} &2 \ 4 \ 7 \\ &2 \times 100 + 4 \times 10 + 7 \times 1 \\ &2 \times 10^2 + 4 \times 10^1 + 7 \times 10^0 \end{aligned}$$

То есть каждая позиция — это степень 10. Справа единицы (10^0), потом десятки (10^1), потом сотни (10^2).

В двоичной системе ровно то же самое, только степень 2:

$$\begin{aligned} &1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \\ &1 \cdot 128 + 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 \\ &= 128 + 64 + 32 + 16 + 4 + 2 + 1 \\ &= 247 \end{aligned}$$

То есть восемь двоичных цифр 11110111 — это то же самое, что десятичное 247. Правый бит — единицы (2^0), потом двойки (2^1), потом четвёрки (2^2), потом восьмёрки (2^3), и так далее. Это называется **позиционная запись**.

Восемь битов могут представить числа от 00000000 (= 0) до 11111111 (= 255). Это 256 значений всего.

Если вам надо больше — берёте больше битов. Два байта (16 бит) = 65536 значений. Четыре байта (32 бита) = ~4.3 миллиарда. Восемь байт (64 бита) = ~18 квинтиллионов.

В C++ есть несколько целочисленных типов, отличающихся именно размером:

Тип	Размер обычно	Диапазон (если без знака)
char	1 байт	0..255
short	2 байта	0..65535
int	4 байта	0..4 294 967 295
long long	8 байт	0..18 446 744 073 709 551 615

Это «обычные» размеры; стандарт C++ их строго не фиксирует, и мы об этом подробно поговорим в главе 8.

3.4 Шестнадцатеричная система

Двоичные числа неудобно читать человеку. 11110111 — это 8 символов, а тут ещё ни одной интересной цифры. Поэтому программисты часто используют **шестнадцатеричную систему** — основание 16.

В шестнадцатеричной системе цифр 16 штук: - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 — как обычно; - A (= 10), B (= 11), C (= 12), D (= 13), E (= 14), F (= 15).

То есть после 9 идёт A, а после F — снова 10 (но это уже «1 шестнадцать + 0 единиц», что в десятичной системе = 16).

Зачем именно 16? Потому что $16 = 2^4$. То есть **одна шестнадцатеричная цифра == четыре бита**. Очень удобно: восьмибитный байт записывается двумя хекс-цифрами. Двухбайтное число — четырьмя. Четырёхбайтное — восемью.

Преобразование байта в хекс делается так: первые 4 бита считаем как одну цифру, последние 4 — как другую.

```

11110111
↓↓↓↓ ↓↓↓↓
1111 0111
  ↓    ↓
 15    7
  ↓    ↓
  F    7

```

То есть 11110111 в двоичной системе = F7 в шестнадцатеричной = 247 в десятичной. Все три записи означают одно и то же число.

В C++ литералы можно писать в любой из этих систем: - десятичный: 247; - шестнадцатеричный: 0xF7 (префикс 0x — особый признак); - двоичный: 0b11110111 (префикс 0b, появился в C++14); - восьмеричный: 0367 (префикс 0 — но не один ноль, а с цифрой; этим почти не пользуются).

0xF7, 0b11110111 и 247 — это три разных способа записать одно и то же число. Компилятор после препроцессинга даже не знает, какая запись была использована. Хранится-то всегда одинаково.

3.5 Отрицательные числа: дополнительный код

Хорошо, до 255 в одном байте мы дошли. А как закодировать минус пять?

Можно было бы сказать «один из битов — знак: 0 = плюс, 1 = минус, остальные 7 битов — модуль числа». Это называется **прямой код**, и он действительно использовался на первых компьютерах. Но у этого подхода две проблемы:

1. Появляется два нуля: $+0 = 00000000$ и $-0 = 10000000$. Это путает и арифметику, и сравнения.
2. Сложение становится сложным: если в $+5$ и -3 бит знака разный, надо сначала разобраться, что вычитать.

Поэтому современные компьютеры используют другую систему — **дополнительный код** (two's complement). Правило простое:

- Положительные числа кодируются как обычно.
- Чтобы получить $-x$ — берётся обычный x , инвертируется (все биты меняются: 0↔1), и добавляется 1.

Пример. Возьмём 8-битный байт.

$+5 = 00000101$ — обычное двоичное представление.

Чтобы получить -5 : 1. Инвертируем все биты: 11111010. 2. Добавляем 1: 11111011.

Итог: -5 в 8-битном дополнительном коде = 11111011 = 0xFB.

Почему так работает арифметика? Возьмём $5 + (-5)$:

```

00000101  (+5)
+ 11111011  (-5)
-----
100000000

```

Получилось 9 битов: 100000000. Но мы складываем в 8-битном байте, поэтому старший бит просто отбрасывается (переполнение). Остаётся 00000000 = 0. То есть $5 + (-5) = 0$. Работает.

Главное преимущество дополнительного кода: сложение положительных и отрицательных чисел не требует никакой особой логики. Процессор просто складывает биты, и результат правильный. Поэтому железо такого процессора — простое и быстрое.

В дополнительном коде в 8-битном байте можно представить числа **от -128 до +127** включительно. Не -127 до +127, как могло бы показаться: дело в том, что -0 в этой системе совпадает с +0 (нет двух нулей), и освободившееся значение 10000000 используется для самого отрицательного числа -128.

Это объясняет, почему в C++ диапазон `signed char` — это -128..+127, а `unsigned char` — это 0..255. Один и тот же байт; разный способ интерпретации битов.

То же для остальных типов: - 16-битный `short`: -32768..+32767 (signed) или 0..65535 (unsigned). - 32-битный `int`: -2147483648..+2147483647 (signed) или 0..4294967295 (unsigned). - 64-битный `long long`: огромные числа в обе стороны.

«Если у меня переменная типа `int` и я в неё кладу значение больше 2147483647 — что произойдёт?» Хороший вопрос. В C++ для unsigned-типов есть строго определённое правило: происходит **арифметика по модулю**, то есть значение «обернётся» через 0. Для signed-типов это **неопределённое поведение** (undefined behavior) — стандарт говорит, что программа может сделать что угодно. На практике обычно происходит то же оборачивание, но компилятор имеет право оптимизировать ваш код на основании предположения «переполнения не будет», и в редких случаях вы получите неожиданный результат. Поэтому правило: **с signed-типами на грани переполнения работать опасно, лучше явно использовать unsigned**.

3.6 Дробные числа: IEEE 754

Целые числа разобрались. А как кодировать 3.14?

Подход называется **числа с плавающей точкой** (floating point). Идея — кодировать число в виде «мантисса $\times 2^{\text{экспонента}}$ ». Например, $3.14 \approx 1.57 \times 2^1$. Мантисса 1.57 хранится приближённо, экспонента 1 — точно.

Стандарт IEEE 754 определяет два размера: - **float** (32 бита): 1 бит на знак, 8 битов на экспоненту, 23 бита на мантиссу. Примерно 7 значимых десятичных цифр точности. - **double** (64 бита): 1 бит на знак, 11 битов на экспоненту, 52 бита на мантиссу. Примерно 15-16 значимых цифр.

Главная неприятность плавающей точки: не каждое десятичное число можно точно представить. Например, 0.1 — это **бесконечная двоичная дробь** (как 1/3 в десятичной). При записи в `float` или `double` она округляется. Поэтому:

```
double x = 0.1 + 0.2;  
// x не равно ровно 0.3, а немного больше: 0.30000000000000004
```

Это не баг, это так устроена IEEE 754. Поэтому сравнивать дробные числа на равенство (`x == 0.3`) почти никогда нельзя — надо проверять «отличается меньше чем на эпсилон». Мы вернёмся к этому в главе 8.

3.7 Текст: ASCII

Хорошо, числа закодировали. А буквы?

Идея простая: каждой букве присвоили число. Латинская A = 65. B = 66. И так далее. Цифра 0 (символ) = 48. Пробел = 32. Перевод строки = 10. И так далее.

Эта таблица соответствий называется **ASCII** (American Standard Code for Information Interchange, принята в 1963). Она использует значения от 0 до 127, то есть помещается в 7 битов (один байт с запасом). В ASCII закодированы: - управляющие символы (0-31): NULL, BACKSPACE, TAB, NEWLINE и так далее; - пунктуация и цифры (32-63): пробел, !, ", ..., 0-9, ..., ?; - латинские заглавные (65-90): A-Z; - больше пунктуации (91-96): [, \,], ^, _, `; - латинские строчные (97-122): a-z; - ещё пунктуация (123-126): {, |, }, ~; - DEL (127).

В C++ строковый литерал "A" — это байт со значением 65. Сравнение символов — это просто сравнение чисел: 'a' < 'b' истинно, потому что 97 < 98.

3.7.1 Чего не хватало в ASCII

ASCII покрывает английский. А русский? Казахский? Китайский?

В 80-90-х годах разные страны делали «расширения ASCII», использующие старший бит (тот самый 128-й). Получались разные **кодировки**:

- **KOI8-R** (Россия, Linux 90-х): русские буквы во второй половине таблицы.
- **CP1251 (Windows-1251)**: русские буквы, но в другом порядке.
- **CP866 (DOS)**: русские буквы плюс псевдографика.
- **ISO 8859-1, 8859-2, 8859-5, ...**: западноевропейские, восточноевропейские, кириллические языки соответственно.

Каждая страна — своя кодировка. Если открыть русский текст в KOI8-R как CP1251 — получите кракозябры. Если в CP866 как KOI8-R — получите другие кракозябры. Это было адом до конца 90-х.

И ещё хуже: даже в одной стране разные системы использовали разные кодировки. Windows — CP1251, Mac — MacCyrillic, Linux — KOI8-R или ISO 8859-5. Открыть документ из одной системы в другой — лотерея.

С китайским/японским/корейским вообще беда: символов десятки тысяч, в один байт никак не помещается. Использовали двухбайтные кодировки (Shift-JIS для японского, GB2312 для китайского, Big5 для традиционного китайского), и они тоже не дружили друг с другом.

3.7.2 Юникод и UTF-8

В начале 90-х придумали **Юникод** — единую таблицу для всех языков мира. Сейчас в Юникоде около 150 тысяч символов: латиница, кириллица, китайские иероглифы, иврит, арабский, индийские письменности, эмодзи (☹ — это тоже символ юникода), даже клингонский язык (из Star Trek) и музыкальные ноты.

Каждый символ юникода имеет номер — **code point**. Латинская А имеет code point U+0041 (= 65 в десятичной, это совместимо с ASCII). Русская А — U+0410. Казахская ә — U+04D8. Эмодзи ☹ — U+1F600.

Но как именно записать code point в файл? У вас уже два-три байта на символ, простая «таблица» не работает. Придумали несколько способов:

- **UTF-32**: каждый символ — ровно 4 байта. Очень просто, но раздуто (для текста на латинице — в 4 раза больше места).
- **UTF-16**: каждый символ — 2 или 4 байта. Используется в Windows и Java внутренне.
- **UTF-8**: каждый символ — от 1 до 4 байт. Самый популярный в вебе и Linux.

UTF-8 устроен хитро: - Символы 0-127 (ASCII) — кодируются одним байтом, как в ASCII. Полная совместимость со старым латинским текстом. - Символы 128-2047 — два байта. - Символы 2048-65535 (включая всю кириллицу, греческий, арабский) — три байта. - Символы 65536+ (китайский, японский, эмодзи) — четыре байта.

Преимущество: английский текст в UTF-8 занимает столько же места, что и в ASCII (он и есть ASCII). Русский — раздувается в 2 раза по сравнению с CP1251 (3 байта против 2 в CP1251). Но зато один формат на всё, никаких кракозябр.

В современной разработке **используйте всегда UTF-8**. Все мои файлы в этой книге — UTF-8. Все исходники C++, которые вы напишете — пусть будут UTF-8. Это де-факто стандарт.

3.7.3 Что это значит для C++

В C++ есть два разных уровня работы с текстом.

Старый уровень — char как байт. `std::string` — это последовательность `char`-ов, то есть последовательность байтов. Длина строки "Hello" — 5 байтов / 5 «символов» (всё совпадает, потому что ASCII). Длина строки "Привет" — **12 байтов**, хотя «символов» там 6. Потому что каждая кириллическая буква в UTF-8 занимает 2 байта.

Это значит: если у вас в коде написано `if (str.size() == 6) ...` для русской строки «Привет» — условие не работает. Длина по байтам — 12.

Современный уровень — `std::u8string` (с C++20). Тип специально для UTF-8 строк. Но в C++11/14/17 — обычно работают на байтовом уровне через `std::string`.

В книге, особенно в проекте RPG, мы будем явно поддерживать русские названия предметов и локаций. Поэтому будет несколько разделов про правильную работу с UTF-8 — например, как корректно посчитать «количество символов» в русском названии.

3.8 Размеры и единицы

Раз мы говорим про память, нужно зафиксировать единицы:

- **1 байт** = 8 бит. Это единица минимальная для адресации. У каждого байта в памяти свой адрес.
- **1 КБ (килобайт)** = **1024 байт** в традиции вычислительной техники (а в системе СИ — 1000, но обычно используется 1024). Это 2^{10} .
- **1 МБ (мегабайт)** = 1024 КБ = $1024 \times 1024 = 1\,048\,576$ байт. Это 2^{20} .
- **1 ГБ (гигабайт)** = 1024 МБ = $2^{30} \approx 1\,073\,741\,824$ байт.
- **1 ТБ (терабайт)** = 1024 ГБ = 2^{40} .

Иногда (особенно на коробках жёстких дисков) производители используют десятичные килобайты: 1 КБ = 1000 байт. Поэтому «диск 1 ТБ» на самом деле часто 931 ГиБ в традиции 1024. Чтобы избежать путаницы, в IT встречаются обозначения KiB (kibibyte), MiB (mebibyte), GiB (gibibyte) — это явно «по 1024». Но в речи обычно «килобайт» = 1024 байт, потому что так удобнее в программировании.

Когда говорят «у меня 16 ГБ оперативки», обычно имеют в виду $16 \text{ ГиБ} = 16 \times 2^{30} = \sim 17.2$ миллиарда байт.

3.9 Что увидите в C++

Когда дойдём до главы 8 «Переменные и типы», эти знания пригодятся:

- `unsigned char` — это байт. Значения 0..255.
- `signed char` — это байт, интерпретированный как число со знаком. Значения -128..+127.
- `char` — это байт, может быть `signed` или `unsigned` (зависит от компилятора и платформы). Для текста используется неявно.
- `int` обычно 4-байтное целое со знаком, диапазон -2 миллиарда..+2 миллиарда.
- `unsigned int` — то же 4 байта, но 0..4 миллиарда.
- `float` — 4 байта, IEEE 754, ~7 десятичных цифр точности.
- `double` — 8 байт, IEEE 754, ~15-16 цифр точности.
- `bool` — формально 1 байт (хотя нужен только 1 бит). Значения `true` (1) и `false` (0).

И ещё: операторы вроде `&` (битовое И), `|` (битовое ИЛИ), `^` (исключающее ИЛИ), `~` (инверсия), `<</>>` (сдвиги) — работают именно на двоичном представлении. То есть `5 & 3` — это «по битам И числа 101 и 011», результат `001 = 1`. К этому вернёмся в главе 8.

3.10 Маленькое упражнение

Без программирования, на бумажке:

1. Преобразуйте число 100 в двоичную систему. Сколько битов понадобится?
2. Преобразуйте `0xCAFE` в десятичную систему. Сколько байт оно занимает?
3. Если число 200 положить в `signed char` (диапазон -128..+127) — что произойдёт?
Подсказка: вспомните дополнительный код.
4. Сколько байт занимает строка "привет" в UTF-8? А строка "hello"?
5. Какое максимальное значение можно представить в 64-битном беззнаковом целом?
Дайте порядок (10 в какой степени).

Если на какой-то вопрос затрудняетесь — вернитесь к соответствующему разделу. Эти штуки реально нужны для C++; их потом сложнее доучивать на ходу.

3.11 Что дальше

Глава 3 — про то, что такое программа и язык программирования. Почему одну программу можно «запустить» сразу (Python: `python file.py`), а другую надо сначала «скомпилировать» (C++: `g++ file.cpp`, потом `./a.out`)? В чём отличие машинного кода от ассемблера? Откуда вообще взялся C++? Это короткая глава, но без неё не понять, почему в C++ так много возни вокруг компилятора.

Глава 4

Глава 3. Что такое программа и язык программирования

В прошлой главе мы выяснили, что внутри компьютера всё — это числа. Процессор умеет с ними работать: складывать, сравнивать, прыгать. А что заставляет процессор делать это в нужной последовательности? **Программа**. Чем программа отличается от просто данных, что значит «язык программирования», почему одни программы компилируются, а другие интерпретируются — об этом и будет глава. Без C++ ни одной строки.

4.1 Программа — это последовательность инструкций

В первой главе я говорил, что процессор умеет выполнять «инструкции» — маленькие действия типа «сложи два числа», «сравни», «прыгни». Программа — это просто **список таких инструкций**. Лежит этот список в памяти, процессор берёт оттуда инструкцию за инструкцией и выполняет.

Хитрый момент: сама программа тоже хранится как набор чисел. Каждой инструкции соответствует своё число (или последовательность чисел). Когда процессор «читает следующую инструкцию» — он на самом деле читает несколько байт из памяти и интерпретирует их как «инструкция такая-то». Точно так же, как программа читает несколько байт и интерпретирует их как число или как букву.

Эта «двойная сущность» — байтов одновременно как данных и как инструкций — называется **архитектурой фон Неймана**, по имени математика Джона фон Неймана, который её предложил в 1945 году. До этого первые компьютеры (ENIAC) хранили программу в коммутационных проводах: чтобы запустить другую программу — приходилось буквально перетягивать провода в нужные гнезда. Фон Нейман предложил: пусть программа лежит в той

же памяти, что и данные. Это позволило компьютерам стать универсальными машинами, на которых можно запустить любую программу без перепайки.

Между прочим, именно из-за этой архитектуры существуют такие штуки, как вирусы и эксплойты: данные, которые программа читает (например, из сети), иногда удаётся подсунуть так, что они начнут выполняться как код. Это огромная тема информационной безопасности; в книге мы её касаемся вскользь, но имейте в виду.

4.2 Машинный код

Числа, в которые превращаются инструкции процессора, называются **машинным кодом** (machine code). Это самый «низкий» уровень, ниже него уже только электрические сигналы.

Покажу пример. Возьмём инструкцию x86-64: «сложить число 5 с содержимым регистра `eax`». В машинном коде она выглядит так:

```
83 c0 05
```

Три байта: `0x83`, `0xc0`, `0x05`. Первый байт `0x83` — код операции (опкод) «прибавь маленькое константное число к регистру». Второй байт `0xc0` — указание, к какому именно регистру (`eax`). Третий байт `0x05` — собственно константа 5.

Если процессор увидит эти три байта подряд — он распознает: «ага, надо прибавить 5 к `eax`», и выполнит.

На x86-64 инструкции бывают разной длины — от 1 до 15 байт. На ARM64 проще: все инструкции ровно 4 байта. Это одна из принципиальных разниц «архитектур с переменной длиной инструкций» (CISC, к ним относится x86) и «архитектур с фиксированной длиной» (RISC, к ним относится ARM).

Так вот, в принципе, программу можно писать прямо в машинном коде: набирать байты в hex-редакторе. И в 50-х так и делали — потому что других вариантов не было. Это было адом: один неверный байт ломал всю программу, никаких имён переменных, ничего. Один человек писал программу неделями. Поэтому очень быстро придумали следующий слой.

4.3 Ассемблер

Ассемблер (assembly language) — это «человеческое» представление машинного кода. Вместо `83 c0 05` пишут:

```
add eax, 5
```

Читается: «прибавь 5 к регистру `eax`». Каждая инструкция машинного кода имеет своё мнемоническое имя (для сложения — `add`, для умножения — `mul`, для прыжка — `jmp`, и так далее). Программист пишет на ассемблере, а специальная программа — **ассемблер** (или транслятор) — превращает текст в машинный код.

Сам по себе ассемблер мало что меняет в сложности: вы по-прежнему пишете каждую инструкцию вручную, программа состоит из десятков тысяч таких строк. Но хотя бы можно читать и редактировать. И сейчас, когда нужно выжать максимум скорости (например, на горячих участках в видеокодеках или криптографических библиотеках), часть кода всё ещё пишется на ассемблере — потому что компилятор C++ при всей своей умности не всегда сгенерирует самый оптимальный код.

В обычной разработке ассемблер вы видеть будете редко. Но иногда полезно — в этой книге, в главе 7, мы посмотрим на ассемблерный листинг нашего `hello.cpp`, чтобы понять, во что превращается C++-код.

Один маленький, но важный нюанс: **ассемблер привязан к архитектуре процессора**. Ассемблер для x86-64 — это один синтаксис, для ARM64 — совсем другой. Программа на ассемблере, написанная под Intel, не работает на Mac M3. Это была одна из причин появления следующего слоя.

4.4 Языки высокого уровня

В середине 50-х годов прозвучала революционная идея: пусть программист пишет не «инструкция за инструкцией», а формулу — а специальная программа сама превращает формулу в нужные инструкции.

Первым успешным языком такого рода стал **Fortran** (1957, IBM). Программа на Fortran выглядела так:

```
X = (A + B) * C
```

Эта одна строка — то, что на ассемблере заняло бы 4-5 строк (загрузить A в регистр, прибавить B, перемножить с C, записать в память). Программа, которая превращает Fortran-текст в машинный код — называется **компилятор**.

Fortran ускорил написание программ в десятки раз. Очень быстро появились другие языки:

- **COBOL** (1959) — для бизнес-приложений, до сих пор работает в банках на серверах IBM.
- **Lisp** (1958) — для научных вычислений и (позже) искусственного интеллекта.
- **ALGOL** (1958) — академический язык, повлиявший на синтаксис почти всего, что появилось позже.

- **BASIC** (1964) — для обучения; на нём в 80-х учились программировать миллионы школьников.

И вот теперь добираемся до нашей родословной.

4.4.1 В и С

В 1969 году Кен Томпсон в Bell Labs писал операционную систему Unix. Он начал на ассемблере, но быстро понял: с ассемблером работать неудобно. Особенно когда хочется переносить Unix на разные машины (а Unix как раз и задумывалась переносимой между разными моделями). Томпсон сделал свой язык **В** — упрощённую версию более старого языка BCPL.

В 1972 году коллега Томпсона Деннис Ритчи доработал В и переименовал в **С**. Главные отличия от В: явные типы данных (int, char, и так далее), структуры, лучшее соответствие железу. На С перенесли Unix — то есть Unix-ядро было написано не на ассемблере, а на С, с очень небольшими вкраплениями ассемблера. Это позволило переносить Unix на новые архитектуры с минимальными усилиями.

С оказался невероятно успешным. К концу 70-х он стал стандартом системного программирования. На С написаны: ядра Linux, macOS, Windows (частично), компиляторы (включая компиляторы самих C/C++), интерпретатор Python (CPython), движок V8 (на C++, но C-стиль), драйверы устройств, протоколы интернета.

Главные черты С: - **Близок к железу**: каждая С-конструкция превращается в небольшое число машинных инструкций. Никакой «магии». - **Без сборщика мусора**: программист сам управляет памятью. - **Минимальная стандартная библиотека**: всё, что хочется делать, надо писать либо самому, либо подключать внешние библиотеки. - **Указатели**: явное указание адресов в памяти.

С по-прежнему живой и активно используется. Стандарты: C89, C99, C11, C17, C23.

4.4.2 C++

В 1979 году Бьярне Страуструп, тоже в Bell Labs, начал работу над расширением С, которое назвал «C with Classes». В 1983 году переименовал в **C++** (плюс-плюс — отсылка к оператору инкремента в С, мол «следующая версия»). Первая официальная версия — 1985 год.

Зачем понадобилось С расширять? Главные идеи:

- **Классы и объекты** (объектно-ориентированное программирование) — для крупных программ удобно описывать сущности с их данными и методами.
- **Шаблоны** (generics) — писать код, который работает с любым типом.

- **Стандартная библиотека:** контейнеры (массивы, списки, хеш-таблицы) поставляются вместе с языком.
- **Исключения** — для обработки ошибок.
- **RAII** — автоматическое освобождение ресурсов через деструкторы.

При этом C++ сохранил всю мощь C: указатели, ручное управление памятью, скорость на уровне ассемблера. Главный девиз: «не платить за то, что не используешь» (You don't pay for what you don't use). Если вам не нужны исключения — компилятор не будет генерировать никакого оверхеда на их поддержку.

Стандарты C++: - **C++98** (1998) — первый официальный стандарт ISO. Очень долго был основным. - **C++03** (2003) — мелкие исправления. - **C++11** (2011) — революция. Move-семантика, лямбды, range-based for, smart pointers, auto, и так далее. Наша книжная база. - **C++14** (2014) — мелкие улучшения, make_unique, generic lambdas. - **C++17** (2017) — optional, variant, filesystem, structured bindings, parallel algorithms. - **C++20** (2020) — concepts, ranges, coroutines, modules — снова большая революция. - **C++23** (2024) — мелкие штрихи к C++20, expected, <print>.

В этой книге мы пишем на C++11 (с пометками про C++14/17 в финальных главах). Это «золотое сечение»: достаточно современно, чтобы код был приятным, и достаточно консервативно, чтобы собиралось у всех.

4.4.3 Другие популярные языки

Чтобы поместить C++ в общую картину:

- **Java** (1995) — компилируется в **байт-код**, который выполняется виртуальной машиной JVM. Кросс-платформенность из коробки. Управление памятью — автоматическое (garbage collector). Используется в банках, Android, серверах.
- **C#** (2000) — почти как Java, только от Microsoft. Используется в Windows-разработке, Unity для игр.
- **Python** (1991) — **интерпретируемый**, динамически типизированный. Просто пишется, медленно выполняется. Для скриптов, анализа данных, AI.
- **JavaScript** (1995) — изначально для браузеров, теперь везде. JIT-компилируется V8/SpiderMonkey/JavaScriptCore.
- **Go** (2009) — Google, типа «упрощённый C++ с автоматическим управлением памятью», для серверов.
- **Rust** (2015) — «C++ без выстрелов в ногу», с проверкой памяти на этапе компиляции. Будущий конкурент C++.
- **Swift** (2014) — Apple, для iOS/macOS, типа «современный, безопасный C++».

C++ в этой компании — мощный, быстрый, опасный (без защиты от ошибок), универсальный. Его учат, потому что (а) на нём написано слишком много существующего кода, чтобы он умер; (б) когда нужна максимальная скорость — C++ её даёт; (в) много концепций C++ влияет на другие языки (если выучите C++, потом легче в Rust, Swift, даже Python станет понятнее).

4.5 Компиляция vs интерпретация

Я уже несколько раз бросил это слово — компилятор, интерпретатор. Сейчас точно объясню разницу.

4.5.1 Компиляция

Компилируемый язык — программа сначала **полностью переводится** в машинный код, и только потом запускается. Цикл выглядит так:

```
file.cpp → [компилятор] → file (исполняемый) → запуск
|
| текст                машинный код
```

Когда вы пишете `g++ hello.cpp -o hello`, компилятор берёт текст, анализирует его, генерирует машинные инструкции, сохраняет в файл `hello`. Дальше вы запускаете `./hello`, и операционная система берёт уже готовый машинный код и отдаёт процессору на выполнение.

Плюсы: - **Быстро на этапе выполнения**: процессор работает с готовым машинным кодом, никаких дополнительных слоёв. - **Контроль ошибок на этапе сборки**: компилятор находит много ошибок (опечатки, неправильные типы) до того, как программа запустится. - **Не нужен компилятор у пользователя**: вы раздаёте исполняемый файл, он запускается без всего.

Минусы: - **Долгий цикл разработки**: правишь файл → перекомпилируешь → запускаешь. На больших проектах перекомпиляция занимает минуты. - **Привязка к платформе**: скомпилированный под Linux x86-64 — не запустится на macOS ARM64. Нужно компилировать заново на каждой целевой платформе.

Компилируемые языки: C, C++, Rust, Go, Swift, Fortran, Pascal.

4.5.2 Интерпретация

Интерпретируемый язык — программа **выполняется построчно** специальной программой-интерпретатором. Никакого «полного перевода в машинный код» не происходит:

```
file.py → [интерпретатор python] → выполнение
```

Когда вы пишете `python script.py`, интерпретатор Python берёт текст, читает первую строку, понимает её, исполняет (используя свои внутренние функции, написанные на C/C++!), читает вторую, исполняет, и так далее. Результат вашей программы — это последовательность действий интерпретатора, а не результат работы машинного кода вашей программы (которого нет).

Плюсы: - **Быстрый цикл разработки:** правишь файл → сразу запускаешь. Никакой пересборки. - **Кросс-платформенность бесплатно:** один файл работает везде, где есть интерпретатор Python (или Ruby, или PHP). - **Простой синтаксис:** не нужно объявлять типы заранее, можно лепить переменные на лету.

Минусы: - **Медленно:** интерпретатор делает много работы на каждое действие (разбор синтаксиса, поиск переменных в словаре, проверки типов). Программы на Python обычно в 10-100 раз медленнее аналогов на C++. - **Меньше ошибок на этапе разработки:** опечатку в имени переменной интерпретатор увидит только тогда, когда выполнение до неё дойдёт. - **Нужен интерпретатор у пользователя:** чтобы запустить ваш Python-скрипт, у пользователя должен стоять Python.

Интерпретируемые языки: классический Python, Ruby, Perl, Shell-скрипты, JavaScript (в браузерах исторически), PHP.

4.5.3 JIT-компиляция (между двумя мирами)

Современные интерпретаторы (V8 для JavaScript, PyPy для Python, JVM для Java) используют JIT (Just-In-Time) компиляцию. Идея такая: интерпретатор замечает, что какая-то функция вызывается часто (горячий код), и налету компилирует её в машинный код, а потом следующие вызовы идут уже по машинному коду. Это даёт скорость, близкую к компилируемым языкам, при сохранении гибкости интерпретации.

JIT — это сложная штука, в книге мы её не рассматриваем. Но знать, что JavaScript в Chrome или Java в банковских системах работает быстрее, чем «честный интерпретируемый Python» — полезно.

4.5.4 Где здесь C++

C++ — **строго компилируемый**. Никакого интерпретатора нет (точнее, есть экспериментальные `cling/Cling`, но это редкость и для специальных задач). Поэтому в C++-разработке цикл всегда такой:

1. Пишете в редакторе `file.cpp`.
2. Сохраняете.
3. Запускаете `g++ file.cpp -o file`.

4. Если есть ошибки — компилятор их выводит. Возвращаетесь к шагу 1.
5. Если ошибок нет — `./file` запускает программу.
6. Если результат не тот — возвращаетесь к шагу 1.

Этот цикл на больших проектах становится длинным. Поэтому появилось много инструментов: системы сборки (которые перекомпилируют только изменившиеся файлы), кэшеры компиляции (ccache, sccache), удалённая сборка (distcc). К ним мы вернёмся ближе к проекту 1.

4.6 Как программа становится исполняемой — общий контур

Раз C++ — компилируемый, у программы на C++ есть несколько стадий жизни от текста до запуска. Я уже намекал на это в первой главе про hello world, но соберу здесь:

1. **Препроцессинг** — текстовая обработка `#include`, `#define`, `#ifdef`. На выходе — «развёрнутый» исходник, обычно намного длиннее (тысячи строк).
2. **Компиляция (в узком смысле)** — превращение C++-текста в **ассемблерный код** для целевой архитектуры.
3. **Ассемблирование** — превращение ассемблерного текста в **объектный файл** (машинный код плюс список «дырок», где должны быть подставлены адреса других функций).
4. **Линковка** — соединение нескольких объектных файлов и стандартной библиотеки в единый исполняемый файл, с подстановкой всех «дырок».

В команде `g++ hello.cpp -o hello` все четыре стадии происходят за один вызов. Но вы можете попросить остановиться после каждой: `-g++ -E hello.cpp -o hello.i` — только препроцессинг. `-g++ -S hello.cpp -o hello.s` — препроцессинг + компиляция (получите ассемблер). `-g++ -c hello.cpp -o hello.o` — препроцессинг + компиляция + ассемблирование (получите объектный файл). `-g++ hello.cpp -o hello` — все четыре стадии (получите исполняемый файл).

В главе 7 мы пройдемся по этим стадиям подробно — что именно происходит, как выглядят результаты, что такое объектные файлы и символы.

4.7 Линковка чуть подробнее

Я упомянул «линковку» — что это значит, разберу сейчас по чуть-чуть. Эта тема — главная боль начинающих C++-разработчиков, потому что «ошибки линковки» отличаются от «ошибок компиляции», и их сообщения часто непонятные.

Пример. Допустим, ваша программа состоит из двух файлов:

`math.cpp` — где вы написали функцию сложения:

```
int add(int a, int b) {  
    return a + b;  
}
```

main.cpp — где вы её используете:

```
int add(int a, int b);  
  
int main() {  
    return add(2, 3);  
}
```

Когда вы компилируете main.cpp, компилятор видит вызов add(2, 3). Тут он знает, что add — это какая-то функция, но не знает, где она находится физически. Поэтому в объектном файле main.o он оставляет «дырку»: «здесь должен быть переход на функцию add». Имя функции add сохраняется как **символ**.

Когда вы компилируете math.cpp, в объектный файл math.o попадает машинный код функции add, и заодно — пометка «здесь определён символ add по такому-то адресу».

Дальше работает **линкер** (ld или эквивалент). Он берёт main.o и math.o, видит в main.o «дырку» с именем add, и в math.o — определение add, и соединяет их. Получается единый исполняемый файл.

Если компилируете отдельно, то g++ за раз делает только компиляцию (через флаг -c):

```
$ g++ -std=c++11 -c math.cpp -o math.o  
$ g++ -std=c++11 -c main.cpp -o main.o  
$ g++ math.o main.o -o program  
$ ./program
```

Чаще, конечно, пишут одной командой, и g++ сам всё разруливает:

```
$ g++ -std=c++11 main.cpp math.cpp -o program
```

Что может пойти не так:

- **undefined reference to add** — линкер не нашёл определения. Возможно, вы забыли скомпилировать math.cpp. Или объявили функцию, но не написали её тело.
- **multiple definition of add** — две версии функции с одним именем. Линкер не знает, какую выбрать.
- **cannot find -lXYZ** — линкер не нашёл библиотеку XYZ. Возможно, забыли установить её, или не указали где искать.

Эти ошибки появляются в самом конце сборки, после того, как все файлы успешно скомпилировались по отдельности. Поэтому новичкам всегда непонятно: «у меня же всё компилировалось, что не так?». Дело в том, что компилятор и линкер — разные программы, и у них разная информация.

К линковке мы вернёмся в главе 7 и потом регулярно — каждый раз, когда наша программа будет состоять больше чем из одного файла.

4.8 Стандартная библиотека

Когда вы пишете `#include <iostream>`, вы подключаете **стандартную библиотеку C++**. Это огромный набор готовых классов и функций, который поставляется вместе с компилятором. Туда входят:

- Контейнеры: `std::vector`, `std::string`, `std::map`, и так далее.
- Алгоритмы: `std::sort`, `std::find`, `std::accumulate`.
- Ввод-вывод: `std::cout`, `std::cin`, `std::ifstream`.
- Потоки: `std::thread`, `std::mutex`.
- Время: `<chrono>`.
- Регулярные выражения: `<regex>`.
- Файловая система (с C++17): `std::filesystem`.

Стандартная библиотека — большая и мощная. Половина книги посвящена её освоению. В коде её ещё называют **STL** (Standard Template Library), хотя STL — это её более узкая часть (контейнеры, итераторы, алгоритмы); полное название — **C++ Standard Library**.

На разных платформах библиотека реализована по-разному, хотя интерфейс один и тот же: - На Linux обычно используется **libstdc++** (от GCC). - На macOS — **libc++** (от LLVM/Clang). - На Windows — **MSVC STL** (от Microsoft).

Все три реализации выполняют одно и то же поведение, описанное стандартом C++. Различия — в деталях реализации и в производительности. Для нашей книги это неважно: код будет одинаково работать со всеми тремя.

4.9 Что увидите в C++

К следующей главе оставим в голове:

- C++ — компилируемый язык. Файл `.cpp` сам не запускается — нужна сборка через компилятор.
- За компиляцию отвечает компилятор (`g++`, `clang++`, `MSVC cl`).

- Программа на C++ часто состоит из нескольких файлов. Каждый компилируется отдельно (получается .o), потом всё соединяется линкером в один исполняемый.
- Машинный код, который генерирует компилятор, привязан к архитектуре процессора. Для x86-64 и ARM64 — разный.
- В C++ есть огромная стандартная библиотека, которая подключается через `#include`.
- Существует много стандартов C++: 98, 11, 14, 17, 20, 23. Каждый добавляет новые возможности. Наша база — 11.

4.10 Маленькое упражнение

На бумажке:

1. Можно ли запустить программу `hello.exe`, скомпилированную под Windows x86-64, на Mac M3 (ARM64)?
2. Можно ли запустить программу `hello.exe`, скомпилированную под Windows x86-64, на Intel Mac с macOS?
3. У вас есть исходник `script.py` на Python. Что нужно, чтобы его запустить?
4. У вас есть исходник `main.cpp` на C++. Что нужно, чтобы его запустить?
5. В чём принципиальная разница между компилятором и интерпретатором?

Ответ к 1 — нет (разные архитектуры). К 2 — нет (формат исполняемого файла Windows .exe не понимает macOS, хотя архитектура та же; есть Wine для эмуляции). К 3 — установленный Python. К 4 — установленный компилятор. К 5 — компилятор превращает текст в машинный код целиком, интерпретатор выполняет текст построчно во время работы.

4.11 Что дальше

Глава 4 — про алгоритмическое мышление. Что такое алгоритм? Как решать задачу «вычислить максимум из трёх чисел» в виде последовательности шагов? Что такое переменная не в смысле «`int x`», а в смысле общей идеи? Условие, цикл, функция — концептуально. Всё на псевдокоде по-русски, без C++. Без этой главы погружение в синтаксис C++ будет «как делать?» вместо «что делать?». Сначала научимся «что», потом «как».

Глава 5

Глава 4. Алгоритмическое мышление

К этой главе вы уже знаете, что компьютер выполняет инструкции, что языки программирования бывают компилируемые и интерпретируемые, что есть машинный код и есть высокоуровневый. Теперь — самое важное для будущего программиста: **как думать алгоритмически**. Это даже важнее, чем знать конкретный язык. Хороший программист отличается от плохого не тем, что лучше помнит синтаксис C++, а тем, что умеет разложить сложную задачу на простые шаги.

В этой главе по-прежнему ни строчки C++. Только русский язык и формальные «шаги». Когда мы доберёмся до главы 8 — там уже все эти концепции попадут в C++-синтаксис, и переход будет мягким.

5.1 Что такое алгоритм

Алгоритм — это **последовательность чётко описанных шагов**, которая решает какую-то задачу. Не «решает любую задачу когда-нибудь как-то», а вот конкретно эту задачу за конечное время.

Слово «алгоритм» — арабского происхождения, искажение имени персидского математика IX века Аль-Хорезми. Он написал учебник по арифметике, где описывал процедуры вычислений шаг за шагом. С тех пор любую такую процедуру и называют алгоритмом.

Алгоритм — это не обязательно про компьютеры. Рецепт борща — алгоритм. Инструкция по сборке Lego — алгоритм. Маршрут «как доехать из дома до работы» — алгоритм. Просто компьютер выполняет алгоритмы по другим правилам: формально, без права на «и так понятно», без интуиции.

5.2 Алгоритм в быту: заварить чай

Самый простой пример. Как заварить чай? Кажется очевидным, но попробуем расписать формально.

Версия 1 (неформальная): 1. Поставить чайник. 2. Подождать пока закипит. 3. Положить в чашку чайный пакетик. 4. Залить кипятком. 5. Подождать минуты три. 6. Выпить.

Ок, для человека сойдёт. Но если бы это писали для робота, возникли бы вопросы. «Поставить чайник» — это куда? Налить в него воду — забыли. Включить — забыли. «Подождать пока закипит» — как робот узнает, что закипел? У него есть датчик?

Версия 2 (для робота): 1. Налить в чайник 200 мл воды. 2. Закрыть чайник крышкой. 3. Включить чайник. 4. **Повторять:** проверить, выключился ли чайник автоматически. Если ещё нет — подождать 5 секунд. Если выключился — перейти к шагу 5. 5. Взять чашку. 6. Положить в чашку чайный пакетик. 7. Залить в чашку 200 мл воды из чайника. 8. **Повторять:** засечь время, прошло ли 3 минуты. Если нет — подождать 10 секунд. Если да — перейти к шагу 9. 9. Вынуть пакетик из чашки. 10. Поставить чашку на стол.

Гораздо подробнее. Но именно так нужно думать про алгоритм. Каждый шаг — простое, однозначное действие. «Повторять» — стандартный приём цикла. Шаг должен быть выполнен без интерпретации; если есть «и так понятно» — это плохой шаг.

5.3 Вход, обработка, выход

Алгоритмы в программировании почти всегда устроены по одной схеме:

ВХОД → ОБРАБОТКА → ВЫХОД

- **Вход** — данные, которые алгоритм получает. Например: список оценок ученика.
- **Обработка** — что алгоритм с ними делает. Например: складывает все оценки и делит на их количество.
- **Выход** — результат. Например: средний балл.

Возьмём конкретную задачу. «**Посчитать средний балл ученика**»:

- **Вход:** список оценок (например, [5, 4, 5, 3, 4, 5]).
- **Обработка:** сложить все оценки, разделить на их количество.
- **Выход:** одно число (средний балл).

Алгоритм: 1. Принять на вход список оценок. 2. Установить переменную *сумма* = 0. 3. Установить переменную *количество* = 0. 4. **Для каждой** оценки в списке: - прибавить оценку к *сумма*; - увеличить *количество* на 1. 5. Вычислить *средний* = *сумма* / *количество*. 6. Вернуть *средний* как результат.

Шесть шагов, любому человеку понятно. Для компьютера тоже подойдёт: каждый шаг чёткий, никаких неоднозначностей.

5.4 Переменные

В алгоритме чая мы говорили «200 мл воды», «3 минуты». В алгоритме среднего балла — «оценки», «сумма», «количество». Эти штуки — **переменные**: именованные «коробки», в которых хранятся значения.

Представить можно буквально: на столе стоит коробка с надписью «сумма», в коробке лежит число (например, 26). Когда мы говорим «прибавить оценку к сумме», мы: 1. Открываем коробку «сумма», достаём оттуда число (26). 2. Берём текущую оценку (например, 4). 3. Складываем: $26 + 4 = 30$. 4. Кладём 30 обратно в коробку «сумма».

После всех шагов в коробке «сумма» лежит общая сумма всех оценок. Коробка одна, но содержимое менялось.

У переменной есть: - **Имя** — как она называется (например, *сумма*, *количество*, *средний*). - **Значение** — что в ней лежит сейчас (число, текст, и так далее). - **Тип** — какого рода значения она хранит (целое число, дробное число, текст).

В C++ переменные пишутся синтаксисом `int сумма = 0;`. `int` — тип (целое число), *сумма* — имя, `= 0` — начальное значение. Когда мы дойдём до главы 8, увидите этот синтаксис; пока думайте про коробки.

Важная мысль: **компьютер не знает, что значит «сумма»**. Для него это просто этикетка. Можно назвать переменную как угодно — `s`, `total`, `sum`, *сумма*, *kazinaki* — компьютер не возражает. Но программа предназначена и для людей, которые её читают потом. Хорошие имена переменных = код, который легко читать. Это будет одно из важнейших правил.

5.5 Операции и выражения

Над переменными можно выполнять **операции**. Самые простые — арифметические:

- Сложение: $5 + 3 = 8$.
- Вычитание: $5 - 3 = 2$.
- Умножение: $5 * 3 = 15$ (звёздочка используется вместо знака умножения, потому что $\times$ нет на клавиатуре).
- Деление: $6 / 3 = 2$. Тут тонкость: если делить целые числа, можно получить дробь, а можно — целое отбрасывая дробную часть. В разных языках по-разному.
- Остаток от деления: $7 \% 3 = 1$ (после 6 идёт остаток 1). Это очень полезная операция.

И операции сравнения: - Равно: $5 == 5 \rightarrow$ истина. $5 == 6 \rightarrow$ ложь. (Знак $==$ с двумя равенствами, потому что одно равенство означает «положить значение», а сравнение — «проверить равенство». Это одна из основных ловушек начинающих в C++.) - Не равно: $5 != 6 \rightarrow$ истина. - Больше: $5 > 3 \rightarrow$ истина. - Меньше: $5 < 3 \rightarrow$ ложь. - Больше-или-равно: $5 >= 5 \rightarrow$ истина. - Меньше-или-равно: $5 <= 4 \rightarrow$ ложь.

Результат сравнения — это **логическое значение** (истина или ложь, в программировании ещё называют true/false или 1/0).

И логические операции над сравнениями: - И (and): (возраст > 18) И (возраст < 65) — истина, если оба условия выполнены. - Или (or): (день = «суббота») ИЛИ (день = «воскресенье») — истина, если хотя бы одно выполнено. - Не (not): НЕ (возраст < 18) — истина, если возраст не меньше 18 (то есть 18 и старше).

Из переменных, операций и значений собираются **выражения**:

- сумма + 5 — выражение, его значение зависит от того, что лежит в *сумма*.
- (возраст $>= 18$) И (гражданство = «РК») — выражение, результат которого истина или ложь.
- (длина * ширина) - (отступ * 2) — выражение, использующее три переменные и две операции.

Главный приём: **результат выражения можно положить в переменную.**

площадь = длина * ширина

Читается: «вычислить длина * ширина, и положить результат в коробку *площадь*». Если *длина* = 5 и *ширина* = 3, то после этой строки в коробке *площадь* лежит 15.

5.6 Ветвление (условие)

В алгоритме чая был такой шаг: «проверить, выключился ли чайник. Если ещё нет — подождать. Если выключился — перейти дальше». Это **ветвление**: алгоритм решает, что делать дальше, в зависимости от условия.

Общий вид:

```
ЕСЛИ <условие> ТО
    <действие 1>
ИНАЧЕ
    <действие 2>
```

Пример. Алгоритм «можно ли войти в кинотеатр на фильм 18+»:

```
ЕСЛИ возраст  $>= 18$  ТО
```

«Можно проходить»

ИНАЧЕ

«Извините, фильм только для взрослых»

Можно делать несколько проверок подряд:

ЕСЛИ возраст < 7 ТО

«На детский билет»

ИНАЧЕ ЕСЛИ возраст < 18 ТО

«На подростковый билет»

ИНАЧЕ ЕСЛИ возраст < 65 ТО

«На взрослый билет»

ИНАЧЕ

«На пенсионный билет»

Главное: проверки идут **сверху вниз**, и срабатывает **первая подходящая**. Если возраст 5, сработает первый блок и остальные не проверяются. Если 30 — сработает третий. Это важно: порядок проверок имеет значение.

Иногда ветвление бывает без альтернативы:

ЕСЛИ температура > 38 ТО

позвонить врачу

Если температура нормальная — ничего не делаем, идём дальше.

5.7 Цикл

Если в чае был «**повторять** проверку каждые 5 секунд» — это цикл. Цикл повторяет какие-то действия до тех пор, пока условие истинно.

Общий вид:

ПОКА <условие>:

<действия>

Пример. Алгоритм «налить в стакан 100 мл воды чайной ложкой» (одна ложка = 5 мл):

налито = 0

ПОКА налито < 100:

зачерпнуть ложкой воду

вылить в стакан

налито = налито + 5

Сначала *налито* = 0. Заходим в цикл, проверяем условие: $0 < 100$ — истина. Выполняем тело: чашка воды, перелив, *налито* теперь 5. Цикл повторяется: $5 < 100$ — истина. И так далее. Когда *налито* станет 100, условие $100 < 100$ будет ложью — цикл прервётся, и алгоритм продолжится дальше.

Бывает и другой вид цикла — **для каждого элемента**:

для КАЖДОЙ оценки В списке оценок:

<действия с оценкой>

Это, по сути, перебор всех элементов какой-то коллекции (списка, массива). В C++ это `for` (`auto` оценка : оценки), мы доберёмся до этого в главе 9.

И ещё один — **повторить N раз**:

ПОВТОРИТЬ 10 РАЗ:

<действия>

Это просто синоним «цикл от 1 до 10».

5.7.1 Бесконечный цикл

Опасность циклов: если условие никогда не станет ложью — цикл будет крутиться вечно. Например:

`x = 5`

ПОКА `x > 0`:

 написать `x`

Здесь `x` всегда 5, никогда не уменьшается. Программа будет бесконечно печатать «5, 5, 5, 5, ...» и никогда не остановится. Это **бесконечный цикл**, и это баг.

Иногда бесконечный цикл — это **сознательное решение**. Например, программа-сервер крутится в цикле, ожидая запросов:

ПОКА истина:

 ждать запрос

 обработать запрос

 отправить ответ

Этот цикл специально вечный — потому что серверу всегда есть, что делать. Прервётся он только при завершении программы.

5.8 Функция

Допустим, нам нужно посчитать средний балл сразу для пяти учеников. Можно тупо повторить алгоритм пять раз, копируя его. Но это плохо: длинно, легко сделать опечатку, если найдём ошибку — придётся править в пяти местах.

Решение — **функция**. Это «алгоритм, упакованный под имя, который можно вызывать сколько угодно раз, передавая разные входные данные».

Описание функции:

ФУНКЦИЯ `средний_балл` (список оценок):

```
сумма = 0
количество = 0
ДЛЯ КАЖДОЙ оценки В списке оценок:
    сумма = сумма + оценка
    количество = количество + 1
ВЕРНУТЬ сумма / количество
```

Здесь: - **средний_балл** — имя функции. - (**список оценок**) — параметр функции, то, что мы ей передаём. - **ВЕРНУТЬ ...** — что функция отдаёт обратно.

Использование:

```
оценки_Ивана = [5, 4, 5, 3, 4, 5]
балл_Ивана = средний_балл(оценки_Ивана)
```

```
оценки_Маши = [5, 5, 5, 5, 4]
балл_Маши = средний_балл(оценки_Маши)
```

Функция написана один раз, используется много раз. Это **переиспользование**. Главная идея в программировании: «не повторяйся».

Функции — основа всего. Большая программа на C++ — это сотни функций, которые друг друга вызывают. Главная функция `main`, которую мы видели в `hello.cpp` — тоже функция, просто особая (с неё начинается выполнение).

5.8.1 Функция, вызывающая функцию

Функции можно собирать в иерархию. Например, мы хотим написать «средний балл нескольких учеников»:

ФУНКЦИЯ `средний_балл` (список оценок):

```
сумма = 0
```

```
количество = 0
ДЛЯ КАЖДОЙ оценки В списке оценок:
    сумма = сумма + оценка
    количество = количество + 1
ВЕРНУТЬ сумма / количество
```

```
ФУНКЦИЯ средний_класса (список_учеников):
    общая_сумма = 0
    количество = 0
    ДЛЯ КАЖДОГО ученика В список_учеников:
        балл_ученика = средний_балл(ученик.оценки)
        общая_сумма = общая_сумма + балл_ученика
        количество = количество + 1
    ВЕРНУТЬ общая_сумма / количество
```

Видим: `средний_класса` использует `средний_балл` внутри себя. Каждая функция занимается своим делом, не зная подробностей других. Это **декомпозиция** — разбиение большой задачи на маленькие.

5.9 Декомпозиция как главный навык

Когда вы видите большую задачу — типа «написать игру», «написать сервер чата», «написать базу данных» — она кажется неподъёмной. Хитрость в том, чтобы разбить её на маленькие задачи, которые сами по себе ясны.

Возьмём «написать RPG-игру» (наш Проект 1): - Игроком нужно как-то управлять → подзадача «парсер команд». - В игре есть локации → подзадача «модель локации». - Локации связаны между собой → подзадача «карта». - В локациях есть предметы → подзадача «предметы». - Игрок может брать предметы → подзадача «инвентарь». - В локациях есть враги → подзадача «враги». - Игрок может сражаться → подзадача «бой».

И так далее. Каждая подзадача — это уже что-то конкретное, что можно решить. Если подзадача всё ещё кажется большой — её можно ещё разбить. Например, «бой»: - Подсчёт урона → подзадача. - Очки здоровья → подзадача. - Очередность ходов → подзадача. - Что случается при смерти врага → подзадача.

В итоге задача «написать RPG» превращается в десятки маленьких задач, и каждая решается за час-два. Эта способность — «дробить» — главное, что отделяет программиста от непрограммиста.

5.10 Псевдокод и реальный код

Всё, что я писал выше — это **псевдокод**. Это формальный способ описать алгоритм, но не на каком-то конкретном языке программирования, а на смеси русского и математической нотации. Псевдокодом удобно описывать алгоритмы в учебниках, на собеседованиях, в дизайн-документах — потому что он не привязан к языку и понятен любому.

Реальный код на C++ для функции «средний балл» выглядит так:

```
double средний_балл(const std::vector<int>& оценки) {  
    int сумма = 0;  
    int количество = 0;  
    for (int оценка : оценки) {  
        сумма = сумма + оценка;  
        количество = количество + 1;  
    }  
    return static_cast<double>(сумма) / количество;  
}
```

Здесь больше деталей: `double` — это тип результата (число с плавающей точкой), `const std::vector<int>&` — параметр (список целых чисел, передаваемый по ссылке без копирования), `static_cast<double>` — преобразование типа. Эти детали мы будем разбирать главами 8-14.

Но логика — та же самая, что в псевдокоде. Сумма, цикл, деление. Когда вы написали псевдокод — вы решили задачу. Перевод в C++ — это в основном про синтаксис и нюансы языка, не про мышление.

Поэтому **сначала думайте на бумажке**, потом пишите код. Многие новички сразу садятся за клавиатуру и пытаются решать задачу прямо в редакторе — потом теряются в синтаксических ошибках и забывают, что вообще решали. Хорошая привычка: на сложную задачу сначала писать псевдокод (10-20 минут), потом переводить в реальный код (быстро).

5.11 Состояние программы

Ещё одно важное понятие: **состояние программы**. Это полный «снимок» всех переменных в какой-то момент времени.

Например, на середине выполнения нашего алгоритма расчёта среднего балла: - *оценки* = [5, 4, 5, 3, 4, 5]; - *сумма* = 12 (после первых трёх оценок); - *количество* = 3; - мы сейчас на четвёртом шаге цикла.

Это и есть состояние. Программа — это последовательность переходов между состояниями. На каждом шаге что-то меняется.

В крупной программе состояний десятки и сотни переменных, и от их сочетания зависит, что будет дальше. Программисту приходится постоянно держать в голове «а в каком состоянии сейчас программа?», чтобы понять, что произойдёт. Это сложно — поэтому хорошие программисты стараются делать так, чтобы состояний было меньше, и они были предсказуемее. К этому ведут такие подходы как «функциональное программирование» (минимум изменяемого состояния), «иммутабельные объекты» (объекты, которые нельзя изменить после создания) и тому подобное.

В С++ ничего этого нет «из коробки» — язык позволяет менять состояние в любой момент. Поэтому хорошие программисты на С++ сами навязывают себе дисциплину. Об этом ещё много раз поговорим.

5.12 Концепции, которые превратятся в С++

Вот что из этой главы мы потом будем видеть на С++:

Псевдокод	С++
коробка с именем <i>x</i>	<code>int x;</code>
положить значение	<code>x = 5;</code>
взять значение и прибавить	<code>y = x + 3;</code>
ЕСЛИ ... ТО ... ИНАЧЕ	<code>if (...) { ... } else { ... }</code>
ПОКА ...:	<code>while (...) { ... }</code>
ДЛЯ КАЖДОГО ... В ...:	<code>for (auto x : container) { ... }</code>
ПОВТОРИТЬ N РАЗ:	<code>for (int i = 0; i < N; ++i) { ... }</code>
ФУНКЦИЯ ИМЯ (параметры):	<code>тип имя(параметры) { ... }</code>
ВЕРНУТЬ	<code>return ...;</code>

В главах 8-10 этот словарь будет наполняться деталями. Но основа уже понятна.

5.13 Алгоритм сложнее: найти максимум

В завершение разберём алгоритм посложнее. **Дан список чисел, нужно найти максимум.**

Псевдокод:

ФУНКЦИЯ максимум (список чисел):

 ЕСЛИ список пуст ТО

ВЕРНУТЬ ошибку «список пуст»

максимум = первый элемент списка

ДЛЯ КАЖДОГО числа В списке (кроме первого):

ЕСЛИ число > максимум ТО

максимум = число

ВЕРНУТЬ максимум

Разбор. Начинаем с проверки: если список пуст — максимума нет. Эту ситуацию надо как-то обработать. Если бы мы её пропустили — на пустом списке программа поломалась бы.

Дальше идея: берём первый элемент как «текущий максимум». Перебираем все остальные элементы. Если встретили число больше — обновляем максимум. В конце возвращаем то, что в *максимум*.

Проверим на примере [3, 7, 2, 9, 4]: 1. *максимум* = 3 (первый элемент). 2. Очередное число 7. $7 > 3$? Да. *максимум* = 7. 3. Очередное число 2. $2 > 7$? Нет. *максимум* = 7. 4. Очередное число 9. $9 > 7$? Да. *максимум* = 9. 5. Очередное число 4. $4 > 9$? Нет. *максимум* = 9. 6. Вернуть 9. ✓

Работает. Это и есть алгоритмическое мышление: сформулировать стратегию решения, перевести её в шаги, проверить на примере.

5.14 Маленькое упражнение

На бумажке (или в Word) опишите псевдокодом алгоритмы:

1. Дан список чисел. Сосчитать, сколько в нём чётных. (Чётное — то, у которого остаток от деления на 2 равен 0.)
2. Дано целое число N . Вычислить $N! = 1 \times 2 \times 3 \times \dots \times N$.
3. Даны два слова. Проверить, не получено ли одно из другого перестановкой букв (анаграмма). Подсказка: один из способов — отсортировать буквы каждого слова и сравнить.
4. Даны два списка имён. Найти имена, которые есть в обоих списках.

Не нужно никакого кода. Просто запишите шаги. На каждое — 5-10 минут. Если запутаетесь — попробуйте на маленьком примере вручную, шаг за шагом, и потом обобщите.

Если задачи дались легко — поздравляю, у вас уже есть алгоритмическое мышление. Если тяжело — это нормально, это навык, он развивается с практикой. Делайте больше задачек.

5.15 Что дальше

Глава 5 — про терминал и файловую систему. Раз будем компилировать C++-программы, нужно уметь обращаться с командной строкой: где я сейчас нахожусь, как переместиться в другую папку, как создать файл, удалить, скопировать, увидеть содержимое. И ещё про права доступа — что значит «executable» и почему `./hello` запускается, а `hello.txt` нет. Это последняя глава Части 0a; после неё начнём ставить компилятор и писать C++.

Глава 6

Глава 5. Терминал, файлы, папки, права

К этой главе у вас в голове должны лежать: что такое компьютер (глава 1), как кодируются числа и текст (глава 2), что такое программа и компиляция (глава 3), и как думать алгоритмически (глава 4). Последняя глава Части 0а — про **терминал**, командную строку. Без него C++-разработка невозможна: вы будете жить в терминале. Если вы привыкли работать только мышкой и боитесь чёрного окна — после этой главы перестанете.

6.1 Что такое терминал

Терминал (он же «консоль», «командная строка», «shell») — это **текстовый интерфейс к компьютеру**. Вместо того чтобы тыкать мышкой в иконки, вы пишете текстовые команды, нажимаете Enter, и компьютер выполняет.

Зачем нужен текстовый интерфейс, если есть удобный графический? Несколько причин.

Скорость. Опытный программист в терминале делает за секунду то, на что в графическом интерфейсе ушли бы 10-20 секунд кликов. Например, скопировать все .cpp файлы из одной папки в другую: команда `cp *.cpp /target/folder/` — секунда. Через Finder/Explorer: открыть две папки, выделить нужные файлы (а ещё мышкой выделить только .cpp без других — отдельная морока), нажать Cmd+C, перейти в другую папку, Cmd+V. Минут пять.

Автоматизация. Команды можно объединять в скрипты. Один раз написал «делай это, потом это, потом это» — и в следующий раз запускаешь скрипт одной командой. Мышкой такое не записать.

Воспроизводимость. Если я в книге напишу «открой Finder, нажми сюда, потом сюда» — у вас может быть другая версия macOS, и кнопки могут стоять иначе. А команда `ls -la` одинакова и сегодня, и десять лет назад, на любой системе.

Удалённая работа. Серверы (где хостится ваш сайт, например) обычно не имеют графической оболочки — только командная строка через SSH. Если хотите работать с серверами — терминал обязателен.

Программирование требует. Все компиляторы C++, все системы сборки, все системы контроля версий (git) — это инструменты командной строки. Графические обёртки есть, но они скрывают то, что внутри, и мешают понять, что происходит.

Не пугайтесь чёрного окна. Сначала медленно, потом быстро.

6.2 Где он живёт

macOS. Программа называется **Terminal** (или **iTerm2**, более продвинутая бесплатная альтернатива; рекомендую поставить через `brew install --cask iterm2`). Открыть: `Cmd+Space`, набрать «terminal», Enter.

Linux. На Ubuntu это **GNOME Terminal**, на других дистрибутивах могут быть **Konsole**, **xterm**, **alacritty**. Обычно открывается сочетанием `Ctrl+Alt+T` или через меню приложений.

Windows. Тут несколько вариантов. **PowerShell** или **Windows Terminal** (новое приложение от Microsoft, рекомендую) — это «нативные» терминалы Windows. Но их синтаксис команд отличается от Unix-овского (macOS/Linux). Для нашей книги проще установить **WSL2** (Windows Subsystem for Linux) — это полноценный Linux внутри Windows, и тогда команды будут такие же, как в этой главе. В главе 6 мы про это поговорим подробнее. Пока, если вы на Windows и читаете эту главу, я рекомендую: либо примите, что некоторые команды у вас будут отличаться, либо поставьте WSL2 (это 10 минут установки).

6.3 Приглашение к вводу (prompt)

Когда вы открываете терминал, видите примерно такое:

```
serik@MacBook-Pro ~ %
```

Или такое:

```
serik@ubuntu:~$
```

Это **приглашение** (prompt). Оно состоит из: - `serik` — имя пользователя (вашей учётной записи в системе). - `MacBook-Pro` или `ubuntu` — имя компьютера. - `~` — текущая папка (тильда =

домашняя папка пользователя, об этом ниже). - % или \$ — символ конца приглашения. После него ждётся ваша команда.

Когда я в книге пишу:

```
$ ls
```

Знак \$ — это и есть приглашение. Команда — `ls`. Сам \$ вы не вводите.

После того, как набрали команду и нажали Enter — терминал передаёт её специальной программе, которая называется **shell** (по-русски «оболочка»). Самые популярные шеллы: - **bash** — Bourne Again SHell, стандартный во многих Linux-дистрибутивах. Старый, но мощный. - **zsh** — Z Shell. С macOS Catalina (2019) это шелл по умолчанию на Mac. Совместим с bash, плюс много улучшений. - **fish** — Friendly Interactive Shell. Удобный, с автодополнением, но синтаксис чуть отличается.

В этой книге я пишу под bash/zsh; команды работают одинаково.

6.4 Где я нахожусь — `pwd`

Каждый раз, когда вы в терминале, вы находитесь **в какой-то папке**. Все команды выполняются в контексте этой папки. Команда `pwd` (print working directory) показывает, где вы сейчас:

```
$ pwd
/Users/serik
```

На macOS домашняя папка — `/Users/<имя>`. На Linux — `/home/<имя>`. На Windows (WSL) — что-то вроде `/home/serik`, но «настоящие» Windows-файлы можно найти под `/mnt/c/Users/<имя>`.

6.5 Что в папке — `ls`

Команда `ls` (list) показывает содержимое текущей папки.

```
$ ls
Desktop Documents Downloads Music Pictures
```

Без аргументов выводит просто имена. С флагом `-l` («длинный формат») — много деталей:

```
$ ls -l
total 0
drwx-----+  5 serik  staff   160 Apr 10 14:22 Desktop
drwx-----+  3 serik  staff    96 Mar 22 09:30 Documents
```

```
drwx-----+  8 serik  staff   256 May 11 11:00 Downloads
...
```

Расшифровка одной строки: - drwx-----+ — права доступа (об этом ниже). - 5 — количество ссылок на файл (для папок — что-то техническое, не важно). - serik — владелец. - staff — группа владельца. - 160 — размер в байтах. - Apr 10 14:22 — дата изменения. - Desktop — имя.

С флагом `-a` (all) показываются **скрытые файлы** — те, чьи имена начинаются с точки:

```
$ ls -a
.  .bash_profile  .gitconfig  Desktop  Downloads
.. .DS_Store      .ssh        Documents Music
```

`.` — текущая папка (всегда есть). `..` — родительская папка. `.bash_profile`, `.gitconfig` — конфиги, которые системные программы прячут от глаз, потому что они не для постоянного редактирования.

Можно комбинировать флаги: `ls -la` или `ls -al` — длинный формат и включая скрытые.

6.6 Перемещение — `cd`

Команда `cd` (change directory) переходит в другую папку:

```
$ pwd
/Users/serik
$ cd Documents
$ pwd
/Users/serik/Documents
$ ls
report.pdf  notes.txt
```

`cd <папка>` — перейти в подпапку. Чтобы вернуться обратно:

```
$ cd ..
$ pwd
/Users/serik
```

`..` — это «на уровень выше». Можно поднимать на несколько уровней: `cd ../..` — на два уровня вверх.

`cd` без аргументов — вернуться в домашнюю папку:

```
$ cd
$ pwd
/Users/serik
```

Эквивалент — `cd ~`. Тильда `~` означает «домашняя папка».

Можно прыгать сразу в любое место по абсолютному пути:

```
$ cd /Users/serik/Desktop/cpp-from-scratch
$ pwd
/Users/serik/Desktop/cpp-from-scratch
```

6.7 Пути — абсолютные и относительные

Это важная концепция, на ней спотыкаются почти все новички.

Абсолютный путь — начинается с корня файловой системы и однозначно указывает на файл/папку. На Unix это что-то вроде `/Users/serik/Desktop/photo.jpg` — начинается со слеша.

Относительный путь — указывает на файл/папку относительно текущей. Например, если вы сейчас в `/Users/serik`, то `Desktop/photo.jpg` — это относительный путь, и он указывает на тот же файл, что абсолютный путь `/Users/serik/Desktop/photo.jpg`.

Специальные относительные пути: `- .` — текущая папка («здесь»). `- ..` — родительская папка («на уровень выше»). `- ~` — домашняя папка.

Примеры. Вы в `/Users/serik/Desktop/cpp-from-scratch/docs/tutorial`:

Путь	На что указывает
<code>00-intro.md</code>	<code>/Users/serik/Desktop/cpp-from-scratch/docs/tutorial/00-intro.md</code>
<code>./00-intro.md</code>	то же самое (явно «здесь»)
<code>../tutorial-original/file.md</code>	соседняя папка
<code>../../README.md</code>	<code>/Users/serik/Desktop/cpp-from-scratch/README.md</code>
<code>~/Downloads/photo.jpg</code>	<code>/Users/serik/Downloads/photo.jpg</code>
<code>/etc/hosts</code>	<code>/etc/hosts</code> (абсолютный)

Относительные пути удобны, когда вы работаете в одном проекте: писать `src/main.cpp` короче, чем `/Users/serik/Desktop/myproject/src/main.cpp`. Абсолютные пути нужны, когда хочется указать на что-то «глобальное» — например, `/usr/bin/g++` (системный компилятор) или `/var/log/system.log` (системный лог).

6.8 Создать папку — `mkdir`

```
$ cd ~  
$ mkdir my-project  
$ ls  
my-project Desktop Documents ...
```

`mkdir <имя>` создаёт папку. Если папка уже есть — выдаёт ошибку.

С флагом `-p` создаются все промежуточные папки:

```
$ mkdir -p projects/2026/cpp/lesson-01  
$ ls projects/2026/cpp/  
lesson-01
```

Если флага `-p` нет, и какой-то промежуточной папки нет — `mkdir` выдаст ошибку.

6.9 Создать или открыть файл

Чтобы создать пустой файл, есть команда `touch`:

```
$ touch hello.cpp  
$ ls  
hello.cpp
```

Если файла не было — создаётся пустой. Если был — обновляется его «время последнего изменения», содержимое не трогается. Это редко нужно в реальной работе, но команда полезная.

Чаще файлы создаются через редактор. Я писал в главе 1 (или, в новой нумерации, в главе 6, которую вы скоро прочтёте), что лучший редактор для нас — VS Code. Чтобы открыть файл в VS Code из терминала:

```
$ code hello.cpp
```

Если файла нет — VS Code откроется с пустым новым файлом этого имени. Сохраните (Cmd+S) — файл создастся.

6.10 Прочитать файл — `cat`

`cat` (concatenate) выводит содержимое файла в терминал:

```
$ cat hello.cpp
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Для маленьких файлов хорошо. Для огромных — терминал захлебнётся (потому что весь файл напечатается за раз).

Если файл большой, лучше использовать `less`:

```
$ less BIG.txt
```

Откроется «листалка» — можно листать стрелками или PgUp/PgDn, поиск через /, выход — q. Это удобнее, чем `cat`, для логов и больших текстовых данных.

И ещё две полезные: `head <файл>` — выводит первые 10 строк (можно задать другое число: `head -n 5`). `tail <файл>` — выводит последние 10 строк.

Особо полезно: `tail -f <файл>` — выводит новые строки по мере их добавления в файл. Используется для слежения за логами в реальном времени.

6.11 Копировать, перемещать, удалять — `cp`, `mv`, `rm`

```
$ cp source.cpp backup.cpp
```

Копирует. Был `source.cpp`, теперь есть `source.cpp` и `backup.cpp` с тем же содержимым.

```
$ mv old-name.cpp new-name.cpp
```

Переименовывает (или перемещает в другую папку). Не оставляет копии.

```
$ rm file.txt
```

Удаляет. Будьте осторожны: в терминале **нет корзины**. Удалённое — удалено навсегда. Особенно опасно `rm -rf <папка>` — удаляет папку со всем содержимым рекурсивно, без подтверждения. Команда `rm -rf /` на Unix-системе с правами `root` в худшем случае стирает всё. Никогда не запускайте таких вещей бездумно.

С флагом `-r` (recursive) можно копировать или удалять папки:

```
$ cp -r my-project my-project-backup
$ rm -r old-project
```

Без `-r` `cp` или `rm` папку не возьмут — нужно явно сказать «да, я хочу с подпапками».

С флагом `-i` (interactive) команды спрашивают подтверждение перед каждым действием:

```
$ rm -i file.txt
remove file.txt? y
```

Полезная страховка для опасных операций.

6.12 Маски (wildcards)

Вместо имени файла можно использовать **маску** — шаблон с подстановочными символами.

Самый частый — `*` (звёздочка). Она означает «любое количество любых символов».

```
$ ls *.cpp           # все .cpp файлы в текущей папке
$ rm *.tmp           # удалить все .tmp файлы
$ cp src/*.cpp build/ # скопировать все .cpp из src в build
```

Менее частый — `?`. Это «ровно один любой символ»:

```
$ ls file?.txt
file1.txt file2.txt fileX.txt
```

Маски разворачиваются **самим шеллом до запуска команды**. То есть когда вы пишете `rm *.tmp`, шелл сначала находит все `.tmp` файлы (например, `a.tmp` и `b.tmp`), и фактически выполняется `rm a.tmp b.tmp`. Сама программа `rm` про звёздочку не знает. Это важно помнить — иногда у вас не сработает что-то по маске именно потому, что шелл подставил пустой список.

6.13 Права доступа

В выводе `ls -l` мы видели:

```
-rw-r--r--  1 serik  staff   1024  May 11 21:00  notes.txt
drwxr-xr-x  3 serik  staff     96  May 11 21:00  my-project
```

Первый «столбец» из 10 символов — это **права доступа**. Разберём.

Первый символ — тип объекта: `-` — обычный файл; `d` — папка (directory); `l` — символическая ссылка; `-b`, `c`, `s`, `p` — специальные файлы (устройства, сокеты).

Следующие 9 символов — три группы по 3: - Первая тройка — права **владельца** файла. - Вторая — права **группы** владельца. - Третья — права **всех остальных**.

В каждой тройке три буквы: - **r** — read (можно читать). - **w** — write (можно изменять). - **x** — execute (можно выполнить; для папки — войти внутрь).

Если права нет — стоит **-**.

Примеры: - **-rw-r--r--**: обычный файл. Владелец читает и пишет, остальные только читают. - **-rwxr-xr-x**: обычный файл, исполняемый. Все могут запустить, владелец может ещё изменить. - **drwxr-xr-x**: папка. Все могут войти и читать, владелец может ещё изменять содержимое. - **-rw-----**: обычный файл, только владелец может читать и писать. Другие — никак. Так выглядят файлы с паролями.

Сразу важный момент. **Чтобы запустить программу — у файла должны быть права x**. Когда вы компилируете C++-программу через `g++ hello.cpp -o hello`, компилятор автоматически проставляет **x** на выходном файле. Поэтому `./hello` срабатывает. А если бы прав **x** не было — была бы ошибка «Permission denied».

6.13.1 chmod

Для изменения прав используется `chmod` (change mode).

```
$ chmod +x my-script.sh      # добавить право выполнения всем
$ chmod -w file.txt          # снять право записи у всех
$ chmod 644 notes.txt        # установить точно: 6 для владельца, 4 для группы, 4 для остальных
```

Цифры в `chmod 644` — это сложение прав: - 4 = read, - 2 = write, - 1 = execute.

Сумма даёт права тройки: - 7 = 4+2+1 = rwx (всё); - 6 = 4+2 = rw- (читать и писать); - 5 = 4+1 = r-x (читать и выполнять); - 4 = r-- (только читать); - 0 = --- (ничего).

`chmod 755 file` — владельцу rwx, остальным r-x. Обычно так выставляют для скриптов и программ.

`chmod 644 file` — владельцу rw-, остальным r--. Обычно для обычных файлов.

6.13.2 chown

Меняет владельца файла:

```
$ sudo chown serik myfile.txt
```

Это редко нужно в обычной разработке, но на серверах часто встречается.

6.14 Переменные окружения

В терминале есть **переменные окружения** — что-то вроде глобальных переменных, которые читают программы. Самая важная — PATH.

PATH — это список папок, в которых шелл ищет программы. Когда вы пишете `g++ file.cpp` — шелл по очереди проверяет: есть ли в `/usr/local/bin/g++`? Если нет, в `/usr/bin/g++`? И так далее. Список папок — это и есть PATH.

Посмотреть его содержимое:

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

Папки разделены двоеточием. Команда `echo` просто выводит то, что ей дают. Знак `$` перед именем переменной означает «возьми её значение».

Другие важные переменные: - HOME — путь к домашней папке (`/Users/serik` или `/home/serik`). - USER — имя текущего пользователя. - SHELL — путь к используемому шеллу. - LANG — текущая локаль (например, `ru_RU.UTF-8` или `en_US.UTF-8`).

Установить свою переменную:

```
$ MY_VAR=hello
$ echo $MY_VAR
hello
```

Но эта переменная видна **только в текущем терминале**. Когда закроете и откроете заново — её не будет. Чтобы сделать постоянной, надо добавить в файл `~/.bashrc` (для `bash`) или `~/.zshrc` (для `zsh`) строку `export MY_VAR=hello`.

В разработке часто пользуются переменными окружения для конфигурации программ. Например, программа может читать `DATABASE_URL` и подключаться к БД по этому адресу. Это удобно для разных окружений (`dev/staging/prod`) и для секретов (не хранить пароли в коде).

6.15 Перенаправление и пайпы

Это одна из самых мощных штук в Unix-философии. Расскажу основы; в проекте Mini-shell (Часть III) мы будем строить такое сами.

6.15.1 Стандартные потоки

У каждой программы есть три «стандартных канала»: - **stdin** (standard input) — стандартный ввод. Откуда программа читает данные. По умолчанию — клавиатура. - **stdout** (standard output)

— стандартный вывод. Куда программа пишет результат. По умолчанию — терминал. - **stderr** (standard error) — стандартный поток ошибок. Куда программа пишет сообщения об ошибках. По умолчанию — тоже терминал.

Эти каналы можно **перенаправлять**.

6.15.2 Запись stdout в файл — >

```
$ ls > files.txt
```

Вместо вывода на экран `ls` пишет результат в файл `files.txt`. Если файл был — перезаписывается. Чтобы добавить в конец, не перезаписывая:

```
$ ls >> files.txt
```

>> дописывает к существующему файлу.

6.15.3 Чтение stdin из файла — <

```
$ sort < unsorted.txt
```

Команда `sort` (которая обычно ждёт данные с клавиатуры или из аргументов) теперь читает из файла `unsorted.txt`.

6.15.4 Пайпы — |

Самое мощное — соединять вывод одной программы со входом другой:

```
$ ls | sort
```

Вывод `ls` идёт во вход `sort`. Получается отсортированный список файлов.

Можно цеплять несколько подряд:

```
$ cat names.txt | grep "Иван" | sort | uniq -c
```

Что произойдёт: 1. `cat names.txt` — вывести содержимое файла. 2. `grep "Иван"` — отфильтровать строки, содержащие «Иван». 3. `sort` — отсортировать оставшиеся. 4. `uniq -c` — подсчитать дубликаты.

Каждая программа делает одну простую вещь, но соединяя их в цепочку — получаем сложную обработку. Это и есть **Unix-философия**: маленькие инструменты, делающие одно дело хорошо, и комбинирование их через пайпы.

6.16 Запуск программы и ./

Вспомним из главы 6 (или из первой версии главы 1): чтобы запустить программу из текущей папки, надо написать `./hello`, а не просто `hello`.

Почему?

Когда вы пишете `hello`, шелл ищет программу с этим именем **в папках из PATH**. Текущую папку — **не** проверяет. Это сделано из соображений безопасности: представьте, вы скачали архив с какой-то программы, распаковали в `/tmp`, перешли туда, и набрали `ls` — вместо системного `ls` шелл мог бы запустить злонамеренный `ls`, лежащий в архиве. Поэтому текущая папка в PATH не входит (на Unix; на Windows входит, и поэтому Windows исторически уязвимее).

`./hello` явно говорит: запустить программу `hello` из текущей папки (`.`). Точка тут — та же самая, что в `cd .` или `ls .` — означает «здесь».

Если хотите запустить программу из другой папки, можно указать путь:

```
$ /Users/serik/Desktop/cpp-from-scratch/demo-rpg/game
```

Или относительный путь:

```
$ ../demo-rpg/game
```

Если вы регулярно используете какую-то программу, её путь добавляют в PATH, и тогда можно вызывать просто по имени.

6.17 История команд

Шелл запоминает все команды, которые вы вводили. Чтобы вернуться к предыдущей — стрелка вверх. Ещё раз — позапрошлая. Стрелка вниз — обратно к новым.

Поиск по истории: `Ctrl+R`, потом начинаете печатать — шелл находит последнюю команду, начинающуюся с этого. Очень удобно.

История хранится в файле `~/.bash_history` или `~/.zsh_history`. Можно её отредактировать руками или экспортировать.

6.18 Автодополнение

Нажмите `Tab` — шелл попытается дополнить незаконченное слово. Например, набираете:

```
$ cd Desk<Tab>
```

И шелл сам допишет до `cd Desktop/`. Если вариантов несколько — `Tab` покажет их все. Это в десятки раз ускоряет работу — не приходится точно набирать длинные имена.

6.19 Очистить экран

Когда терминал засыпан выводом — `Ctrl+L` (или команда `clear`) очищает экран. Полезно, чтобы перевести дух.

6.20 Прервать команду

Запустили долгую команду и поняли, что не то? `Ctrl+C` — прервать (программа получает сигнал `SIGINT`).

`Ctrl+Z` — приостановить (программа замораживается). Чтобы продолжить в фоне — `bg`. Чтобы продолжить на переднем плане — `fg`. Эту функциональность мы будем плотно изучать в проекте `Mini-shell`.

6.21 Полезный набор команд

Сохраню сюда «карманный справочник», к которому буду отсылать в следующих главах:

Команда	Что делает
<code>pwd</code>	где я сейчас
<code>ls</code>	что в этой папке
<code>ls -la</code>	то же, с деталями и скрытыми файлами
<code>cd <папка></code>	перейти в папку
<code>cd ..</code>	подняться на уровень выше
<code>cd ~</code> или <code>cd</code>	в домашнюю папку
<code>mkdir <имя></code>	создать папку
<code>mkdir -p <путь></code>	создать все промежуточные
<code>touch <имя></code>	создать пустой файл
<code>cat <файл></code>	показать файл
<code>less <файл></code>	пролистать большой файл
<code>head <файл></code>	первые 10 строк
<code>tail <файл></code>	последние 10 строк
<code>tail -f <файл></code>	следить за новыми строками
<code>cp <откуда> <куда></code>	копировать файл
<code>cp -r <откуда> <куда></code>	копировать папку рекурсивно

Команда	Что делает
<code>mv <откуда> <куда></code>	переместить или переименовать
<code>rm <файл></code>	удалить файл
<code>rm -r <папка></code>	удалить папку с содержимым
<code>chmod +x <файл></code>	сделать исполняемым
<code>echo <текст></code>	напечатать в stdout
<code>echo \$VAR</code>	значение переменной
<code>export VAR=value</code>	установить переменную
<code>grep <текст> <файл></code>	найти строки, содержащие текст
<code>wc -l <файл></code>	сосчитать строки
<code>find <где> -name <маска></code>	поиск файлов
<code>man <команда></code>	справка по команде

Команда `man` — самая полезная. `man ls`, `man chmod`, `man find` — открывает встроенную справку. Выход — `q`. Туда можно лезть всегда, когда не помните флаги.

6.22 Маленькое упражнение

Откройте терминал и сделайте по шагам:

1. Перейдите в домашнюю папку (`cd ~`).
2. Создайте папку `cpp-practice` (`mkdir cpp-practice`).
3. Зайдите в неё (`cd cpp-practice`).
4. Создайте подпапки `src`, `include`, `build` одной командой (`mkdir src include build`).
5. Создайте пустой файл `src/main.cpp` (`touch src/main.cpp`).
6. Посмотрите структуру (`ls -la` или, если установлен, `tree`).
7. Запишите в `src/main.cpp` одно слово через `echo`: `echo "Hello" > src/main.cpp`.
8. Покажите содержимое: `cat src/main.cpp`.
9. Скопируйте файл: `cp src/main.cpp src/backup.cpp`.
10. Удалите бэкап: `rm src/backup.cpp`.
11. Поднимитесь на уровень выше: `cd ...`.
12. Удалите всю папку `cpp-practice` (только если уверены, что не нужно): `rm -r cpp-practice`.

Если всё прошло без ошибок — освоились. Если запутались — повторите спокойнее. После этой главы должно стать комфортно жить в терминале.

6.23 Что дальше

Часть 0а закончена. Теперь у вас есть фундамент: понимание устройства компьютера, битов и текста, программы и компиляции, алгоритмического мышления, командной строки. На этом фундаменте можно строить C++.

Глава 6 — установка компилятора (clang/gcc) и первая программа «Hello, world». Эта глава уже была написана в первой версии книги; вернётесь к ней и увидите: всё, что там написано про «команды терминала», «папки», «права», теперь укладывается в голову без вопросов. Вы готовы к C++.

Глава 7

Глава 6. Инструменты и первая программа

В этой главе мы поставим C++-компилятор, выберем редактор, напишем самую простую программу, соберём её и запустим. И главное — на каждом шаге я объясню, что именно происходит. Не «вот команда, запустите её», а «вот команда, и теперь покажу, что за этой командой стоит на самом деле».

7.1 Что нам нужно

Для C++-разработки нужны три вещи:

1. **Компилятор** — программа, которая превращает текст вашей программы (.cpp-файл) в исполняемый файл, который умеет запускать процессор.
2. **Текстовый редактор** — где вы пишете этот текст. Любой подойдёт; современные редакторы дополнительно подсказывают синтаксис и подсвечивают ошибки.
3. **Терминал** — программа, в которой вы пишете команды операционной системе. Команды вроде «скомпилируй мне вот этот файл» или «запусти вот этот исполняемый файл». Терминал есть в любой ОС, просто называется по-разному.

Иногда к этому добавляют **отладчик** (программа, которая позволяет остановить вашу программу в любой точке и посмотреть значения переменных) и **систему сборки** (программа, которая знает, какие файлы и в каком порядке компилировать в большом проекте). Их мы поставим, но не в этой главе.

Идём по платформам.

7.2 macOS

На современном Mac (любом, начиная с 2010-х) компилятор называется **clang**. Apple встроила его в систему — вернее, в инструмент Xcode Command Line Tools.

Откройте терминал (Cmd+Space → «Terminal» → Enter) и введите:

```
$ clang++ --version
```

Если в ответ вы видите что-то вроде `Apple clang version 14.0.3 (clang-1403.0.22.14.1)` — компилятор у вас уже есть, переходим к редактору.

Если же терминал предложит установить инструменты разработчика (появится окошко с большой кнопкой «Install») — нажмите её. Через несколько минут установка закончится, и команда `clang++ --version` начнёт работать. На современных Mac с Apple Silicon Xcode Command Line Tools идут как ~700 МБ — это нормально.

Полную Xcode IDE ставить **не нужно** — она огромная (десятки гигабайт), и для нашей книги избыточна. Хватит командной строки.

Альтернатива clang — **gcc** (про него поговорим в Linux-разделе). Поставить его на macOS можно через Homebrew (`brew install gcc`), но смысла особого нет: Apple clang делает всё то же самое. Я буду писать команды как с `g++`, так и с `clang++` — на macOS можете использовать любой, результат будет одинаков. По умолчанию даже команда `g++` на macOS вызывает clang (Apple подсунула символическую ссылку).

7.3 Linux

На Ubuntu/Debian и их производных:

```
$ sudo apt update
$ sudo apt install build-essential
```

Пакет `build-essential` содержит **gcc/g++** (GNU Compiler Collection — компилятор разрабатывается с 1987 года, считается «эталоном» в Linux-мире) плюс инструменты вроде `make` и `libc6-dev` (заголовки стандартной библиотеки). Проверка:

```
$ g++ --version
g++ (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
```

На Fedora/RHEL/CentOS:

```
$ sudo dnf install gcc-c++ make
```

На Arch:

```
$ sudo pacman -S base-devel
```

Если хотите clang — поставьте отдельно: `sudo apt install clang` или эквивалент. Иметь оба не вредно — на больших проектах удобно собирать обоими по очереди, потому что они выдают разные предупреждения, и баги ловятся лучше.

7.4 Windows

С Windows ситуация чуть хитрее, потому что нативный «виндовый» компилятор — Microsoft Visual C++ из Visual Studio — отличается от Linux-овского g++ многими мелочами (синтаксис командной строки, заголовочные файлы, флаги). Я в книге везде пишу команды для g++/clang++, потому что они одинаковые на macOS и Linux.

У вас три варианта.

Вариант А — WSL2 (Windows Subsystem for Linux), рекомендую. Microsoft встроила в Windows 10/11 полноценный Linux. Откройте PowerShell от администратора и наберите:

```
> wsl --install
```

Перезагрузитесь. После перезагрузки в меню «Пуск» появится Ubuntu — запустите, придумайте имя пользователя и пароль, и у вас теперь Linux внутри Windows. Далее следуйте инструкциям из секции «Linux» (`sudo apt install build-essential`). Все команды из книги будут работать «как на Linux».

Вариант В — MSYS2. Это среда, которая ставит на Windows полноценный GNU-toolchain (g++, make, всё как в Linux), но без виртуализации. Скачать: <https://www.msys2.org/>. После установки откройте «MSYS2 MINGW64» и наберите:

```
$ pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-clang
```

Далее команда g++ будет работать. Минус: пути с обратными слешами и другие мелочи Windows иногда вылезают.

Вариант С — Visual Studio Community. Полная IDE от Microsoft, бесплатная для индивидуальной разработки. При установке выберите workload «Desktop development with C++». После этого в меню появится «Developer Command Prompt» — это терминал, где работает команда cl (это виндовый компилятор; синтаксис другой, но первые программы соберёт). Подойдёт, но мои команды надо будет адаптировать.

Я в книге дальше пишу команды для g++/clang++. Если вы на Windows и выбрали вариант С — открывайте дополнительную вкладку с документацией Microsoft по cl.exe и сопоставляйте флаги. Если выбрали А или В — у вас всё будет совпадать с macOS/Linux.

7.5 Редактор

C++ можно писать в Notepad. Я серьёзно — буква в букву это совпадает с тем, как писали в 90-х. Но мы живём в 2026, и есть выбор получше.

VS Code — бесплатный, лёгкий, отлично работает с C++ через расширение «C/C++» от Microsoft и/или «clangd» (более современная альтернатива). Установка: <https://code.visualstudio.com/>, потом в боковой панели Extensions поставьте «C/C++» или «clangd». На macOS/Linux/Windows работает одинаково.

Sublime Text или **Vim/Neovim** — для тех, кто предпочитает классику. Vim требует настройки (plugin clangd), но после неё — летает.

CLion от JetBrains — полноценная IDE с самой умной автодополнялкой и встроенным отладчиком, но платная (студенческая лицензия бесплатно). Если вы любите среду JetBrains (PHPStorm, IntelliJ, PyCharm) — поставьте, разница только в языке.

Я в книге исхожу из того, что у вас VS Code. Это самый распространённый вариант в 2026 году, бесплатно, и достаточно мощно. Все скриншоты — оттуда. Если у вас другой редактор — большой разницы нет, в книге почти не будет «нажмите Ctrl+Shift+P».

7.6 Структура проекта

Создайте где-нибудь на диске папку для нашей книги. Я кладу свои учебные проекты в ~/code/cpp-book/ на macOS (~ — это домашняя папка пользователя). Откройте терминал и создайте папку:

```
$ mkdir -p ~/code/cpp-book/ch01
$ cd ~/code/cpp-book/ch01
```

Команда `mkdir` (make directory) создаёт папку. Флаг `-p` означает «и все промежуточные тоже, если их нет». Команда `cd` (change directory) переходит в эту папку.

Откройте эту папку в VS Code:

```
$ code .
```

Точка `.` означает «текущая папка». Если команды `code` нет — в VS Code откройте Command Palette (Cmd+Shift+P на mac, Ctrl+Shift+P на остальных) и выполните «Shell Command: Install 'code' command in PATH».

7.7 Первая программа — `hello.cpp`

В открытом VS Code создайте файл `hello.cpp` (правой кнопкой по дереву файлов слева → New File). Расширение `.cpp` — стандартное для исходников C++. Бывают ещё `.cxx`, `.cc`, `.C` — это всё C++; конкретно `.cpp` — самое распространённое.

Напишите туда:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Шесть строк (с пустой между ними). Сейчас разберём каждый символ. Серьёзно — каждый.

7.7.1 Строка 1: `#include <iostream>`

Это директива препроцессора. **Препроцессор** — это первая стадия компиляции; он читает исходник, обрабатывает строчки, начинающиеся с `#`, и подменяет их на что-то другое. В нашем случае — берёт содержимое файла `iostream` и вставляет его прямо сюда, где написано `#include`.

Файл `iostream` — это часть стандартной библиотеки C++. Внутри него — описания того, как работать с вводом и выводом (input/output stream). Конкретно нам сейчас нужно `std::cout` — объект, который представляет «стандартный вывод» (то, куда пишет ваша программа: обычно это терминал). Объект `std::cout` объявлен где-то внутри `<iostream>`, поэтому без `#include <iostream>` компилятор не будет знать, что такое `std::cout`, и выдаст ошибку.

Угловые скобки `<...>` говорят: ищи этот файл в системных папках стандартной библиотеки (на macOS это что-то вроде `/Library/Developer/CommandLineTools/usr/include/c++/v1/`, а на Linux — `/usr/include/c++/11/`). Если бы мы написали `#include "iostream"` в кавычках — препроцессор сначала смотрел бы в папке нашего исходника, а только потом в системных. Это правило используется, когда вы подключаете *свои* заголовочные файлы (не стандартные).

`iostream` без расширения — это особенность стандартной библиотеки C++. Когда-то (давно) подключали `<iostream.h>`. В современном C++ — без `.h`. Под капотом файл всё равно есть, просто называется чуть иначе и лежит в специальной папке.

Заметьте: в конце строки **нет точки с запятой**. Препроцессорные директивы — особый случай; они обрабатываются ещё до того, как начинает работать собственно компилятор. Точка с запятой им не нужна, и если её поставить — будет ошибка.

7.7.2 Строка 2: пустая

Просто разделитель. Компилятор её игнорирует. Я люблю отделять блок `#include` от кода пустой строкой — это визуально структурирует файл. Можно не делать.

7.7.3 Строка 3: `int main() {`

Это начало функции `main`. Функция `main` — особенная: это точка входа в программу. Когда вы запускаете исполняемый файл, операционная система ищет в нём функцию с именем `main` и вызывает её. Если такой функции нет — программа не соберётся.

Разберём по частям.

`int` — это **тип возвращаемого значения**. Функция `main` возвращает целое число (`int = integer`). Это число — код возврата программы. По умолчанию `0` означает «всё прошло хорошо», ненулевое значение — «что-то сломалось». Этим кодом возврата пользуются `shell`-скрипты: они проверяют, успешно ли отработала ваша программа, чтобы решить, продолжать или нет.

`main` — имя функции. Это **зарезервированное имя** в C++ — нельзя сделать `main` чем-то другим (например, переменной).

`()` — пустые круглые скобки. Это **список параметров функции**. Сейчас он пуст; на самом деле `main` в C++ может принимать аргументы командной строки (`int main(int argc, char* argv[])`), но мы пока без них.

`{` — открывающая фигурная скобка. Это начало **тела функции**. Всё, что идёт от `{` до соответствующей `}`, выполняется при вызове функции. В C++ блоки кода всегда выделяются фигурными скобками — никаких отступов как в Python.

7.7.4 Строка 4: `std::cout << "Hello, world!" << std::endl;`

Это **выражение-инструкция** — основной строительный кирпичик любой C++ программы. Разбираем по кускам.

Четыре пробела в начале строки. Это отступ. Чисто визуально — чтобы было видно, что строка принадлежит телу функции `main`. Компилятор отступы игнорирует; они для человека. В команде обычно договариваются: 4 пробела или 2 пробела или `tab`. В книге я использую 4 пробела. Большинство C++-стандартов в отрасли — тоже 4 пробела (Google C++ Style Guide — 2 пробела, но в нашей книге держимся 4).

`std::cout` — обращение к объекту `cout` из пространства имён `std`. **Пространство имён** (namespace) — это что-то вроде «папки» для имён, чтобы две библиотеки могли использовать одинаковые названия и не конфликтовать. Вся стандартная библиотека C++ живёт в

пространстве `std`. Чтобы добраться до объекта внутри, пишут `имя_пространства::имя_объекта`. Двоеточия — две штуки подряд (`::`) — называются *оператором разрешения области видимости*.

Объект `cout` — это «стандартный поток вывода». Он наследник класса `std::ostream`. Технически это объект, у которого есть состояние (например, текущая позиция, флаги форматирования), и который умеет принимать данные и отправлять их в терминал.

Оператор `<<` — это **оператор вставки в поток**. Читается так: «отправь в `cout` то, что справа». То есть `std::cout << "Hello, world!"` означает «отправь строку "Hello, world!" в стандартный вывод». Само сочетание `<<` для битового сдвига влево используется (это тот же символ), но в контексте `std::cout` оно перегружено как «вставить в поток». Это перегрузка операторов — мощная вещь C++, доберёмся до неё в главе 11.

"Hello, world!" — это **строковый литерал**. Кавычки двойные. Тип у него — `const char[14]` (массив из 14 символов: 13 видимых + один невидимый «нулевой символ» `\0`, который маркирует конец строки в стиле C). Дальше эта штука неявно сконвертируется в `const char*` (указатель на первый символ массива), и `<<` этим займётся.

Внутри строкового литерала можно использовать **escape-последовательности**: `\n` — перевод строки, `\t` — табуляция, `\"` — кавычка внутри строки, `\\` — обратный слеш. Мы их пока не используем.

Второй `<<` и `std::endl` — снова оператор вставки, но теперь мы вставляем не строку, а специальный объект `std::endl`. Это «end line» — он делает две вещи: вставляет символ перевода строки (`'\n'`) и сбрасывает буфер вывода (то есть форсирует, чтобы текст реально появился в терминале, а не остался лежать в буфере).

В большинстве случаев вместо `std::endl` можно писать `"\n"` — это быстрее, потому что не сбрасывает буфер. Но для нашей первой программы `std::endl` нагляднее.

Точка с запятой ; в конце — обязательна. В C++ почти каждая инструкция оканчивается точкой с запятой. Если её забыть, компилятор выдаст ошибку. Привыкайте сразу.

Итого: эта строка отправляет в стандартный вывод текст «Hello, world!» и перевод строки.

7.7.5 Строка 5: `return 0;`

Возврат из функции `main` со значением 0. Как мы говорили, `main` возвращает целое число — код возврата программы. 0 — «успех». `return` — ключевое слово, которое останавливает выполнение функции и отдаёт значение наружу. Точка с запятой в конце — снова обязательна.

В C++ для функции `main` есть особое правило: если вы забудете `return 0;`, компилятор автоматически добавит его сам. То есть для `main` `return 0;` опционален. Но я всегда пишу явно — так понятнее.

7.7.6 Строка 6: }

Закрывающая фигурная скобка. Конец тела функции `main`. На этом файл и заканчивается.

7.8 Компиляция

Сохранили файл (`Cmd+S` на `mac`, `Ctrl+S` на остальных). Теперь в терминале:

```
$ g++ -std=c++11 -Wall -Wextra hello.cpp -o hello
```

Если у вас вместо `g++` есть только `clang++` — пишите `clang++`, всё то же самое:

```
$ clang++ -std=c++11 -Wall -Wextra hello.cpp -o hello
```

Разбираем по флагам.

g++ или **clang++** — собственно компилятор. Можно ещё `c++` (это символическая ссылка, обычно на системный C++-компилятор).

-std=c++11 — какой стандарт языка использовать. Без этого флага компилятор может взять стандарт по умолчанию (у `gcc 11+` это C++17, у `clang` старше — C++98). Мы фиксируем `-std=c++11`, чтобы код собирался у всех читателей одинаково. Иногда увидите `-std=c++14`, `-std=c++17`, `-std=c++20`, `-std=c++23` — это более новые стандарты.

-Wall — включить большую часть стандартных предупреждений. `w` = warning, `all` — все. Предупреждения это не ошибки — компилятор просто говорит: «ты тут что-то странное написал, может быть, опечатка?» Хорошая практика — всегда собирать с `-Wall`.

-Wextra — включить ещё больше предупреждений, которые не вошли в `-Wall`. Тоже очень полезно.

-pedantic — иногда добавляю. Это «педантичный» режим: компилятор требует строгого соответствия выбранному стандарту, без расширений GNU/clang. Для книги полезно: если код собирается с `-pedantic`, значит, он стандартный.

hello.cpp — имя файла с исходным кодом. Можно несколько: `g++ a.cpp b.cpp c.cpp`.

-o hello — имя выходного файла. `-o` от output. Без этого флага компилятор по умолчанию назовёт результат `a.out` (наследие 70-х, когда так называли исполняемые файлы на Unix). Я предпочитаю явно говорить, как назвать.

Если всё хорошо — никакого вывода не будет. Это normal: «no news is good news» в Unix-философии. Зато в папке появится файл `hello` (или `hello.exe` на Windows). Проверим:

```
$ ls -lh hello
-rwxr-xr-x  1 user  staff   33K hello
```

Файл размером ~33 KB. Видите x в правах доступа (rwxr-xr-x) — это «executable», его можно запустить.

7.9 Запуск

```
$ ./hello
Hello, world!
```

Точка-слеш в начале (./) — это **обязательно**. В Unix-системах, когда вы пишете hello без ./, shell ищет программу с таким именем в специальных папках (/usr/bin, /bin, /usr/local/bin, и так далее — это переменная \$PATH). Текущую папку он *не* проверяет — из соображений безопасности (чтобы вы случайно не запустили вредоносный ls в каком-нибудь скачанном архиве, подменивший системный). Чтобы явно сказать «запусти программу из текущей папки», используют ./ — точка означает текущую папку, слеш — разделитель.

Программа выполнилась, вывела «Hello, world!», и закончилась. Если хотите увидеть код возврата:

```
$ echo $?
0
```

Команда echo \$? показывает код возврата последней запущенной программы. У нас 0, потому что мы написали return 0;. Поменяйте на return 42;, пересоберите, запустите, проверьте — увидите 42.

7.10 Что произошло — большая картинка

Между моментом, когда вы написали g++ hello.cpp -o hello, и моментом, когда появился исполняемый файл, прошло четыре стадии компиляции. Я расскажу о них коротко сейчас, а детально — в следующей главе.

Стадия 1: препроцессор. Компилятор берёт hello.cpp и обрабатывает все #include. То есть он буквально берёт содержимое файла iostream (а это десятки тысяч строк, потому что внутри ещё много #include) и подставляет его в место #include <iostream>. На выходе — один огромный «развёрнутый» исходник. Хотите увидеть? g++ -E hello.cpp (E от «preprocess»). Не пугайтесь, вылетит много текста.

Стадия 2: компилятор. Из «развёрнутого» исходника генерируется *ассемблерный код* — текст на низкоуровневом языке процессора. Хотите увидеть? g++ -S hello.cpp -o hello.s. Потом откройте hello.s — увидите инструкции вроде movq, callq, retq. Это уже почти машинный код, только в текстовом виде.

Стадия 3: ассемблер. Из ассемблерного текста делается *объектный файл* — бинарный файл, содержащий машинный код, но ещё не «соединённый» с другим кодом. `g++ -c hello.cpp -o hello.o`. Файл `hello.o` — это машинный код вашей функции `main`, но с «дырками» там, где она ссылается на чужие функции (например, на `std::cout::operator<<`).

Стадия 4: линкер. Линкер берёт все объектные файлы и подставляет в «дырки» реальные адреса. Например, наша программа ссылается на `std::cout`, который реализован в стандартной библиотеке (на macOS — `libc++`, на Linux — `libstdc++`). Линкер находит библиотеку, привязывает её, и получается готовый исполняемый файл.

Когда вы пишете `g++ hello.cpp -o hello` без флагов `-E`, `-S`, `-c` — компилятор молча проходит все четыре стадии и выдаёт готовый файл. Под капотом это четыре отдельных программы (`cpp`, `cc1plus`, `as`, `ld`), и `g++` их вызывает по очереди. Можете в этом убедиться: `g++ -v hello.cpp -o hello` покажет всё, что происходит, с подробными командами.

Глава 2 будет именно про это: компиляция по шагам, объектные файлы, символы, линковка. Сейчас же запомните: исходник `.cpp` не запускается напрямую. Он сначала превращается в исполняемый файл, и уже исполняемый файл запускает процессор.

7.11 Что если ошибка

Сломаем нашу программу нарочно — уберите точку с запятой в конце пятой строки:

```
    return 0
}
```

Соберите:

```
$ g++ -std=c++11 hello.cpp -o hello
hello.cpp:6:1: error: expected ';' before '}' token
    6 | }
      | ^
```

Компилятор честно говорит: «На строке 6 в начале я ждал точку с запятой перед `}`, но не нашёл». Сообщения об ошибках в современном `gcc/clang` очень подробные — внимательно читайте их, не пугайтесь объёма. Обычно ошибка ровно там, где компилятор её показал.

Несколько типичных ошибок, с которыми вы столкнётесь в первые дни:

error: 'cout' was not declared in this scope — забыли `#include <iostream>` или забыли `std::` перед `cout`.

undefined reference to 'main' — забыли написать функцию `main` (или назвали её по-другому).

expected ';' before ... — где-то выше потеряли точку с запятой.

'std' has not been declared — старый компилятор, который не понимает `std::`. Поставьте новый.

no such file or directory: 'iostream' — у вас не установлена стандартная библиотека C++ (бывает на минималистичных Linux-сборках). Поставьте `libstdc++-dev` или эквивалент.

7.12 Запоминания на сегодня

К следующей главе хорошо бы выучить наизусть:

- Каждый исполняемый файл — это результат компиляции одного или нескольких `.cpp` файлов плюс линковки с библиотеками.
- Функция `main` — точка входа в программу. Возвращает `int`.
- Каждый блок кода обрамляется фигурными скобками `{...}`.
- Каждая инструкция оканчивается точкой с запятой `;`.
- `#include <...>` подключает заголовочный файл.
- Пространство имён `std` — это вся стандартная библиотека. Доступ через `std::имя_объекта`.
- `std::cout << ...` — вывод. `<<` — оператор вставки в поток.
- `std::endl` — перевод строки + сброс буфера.
- Команда сборки: `g++ -std=c++11 -Wall -Wextra файл.cpp -o имя`.
- Запуск программы в текущей папке: `./имя`.

7.13 Маленькое упражнение

Чтобы рука запомнила синтаксис, напишите программу, которая выводит:

Меня зовут Ваше Имя.

Я учу C++.

Это моя первая программа.

Можете использовать `std::endl` или `"\n"` для переводов строки — попробуйте оба варианта. Откройте файл `intro.cpp`, напишите код, соберите, запустите. Если соберётся с первого раза — это очень хороший знак. Если нет — внимательно прочтите ошибку и попробуйте найти, где забыли точку с запятой или скобку. Это первый и самый частый источник ошибок в C++.

7.14 Что дальше

Глава 2 — про то, как программа превращается в исполняемый файл. Мы пройдемся по всем четырём стадиям компиляции, посмотрим, что лежит внутри объектного файла, что такое символы и почему «undefined reference» — это самая частая ошибка C++-новичка. Это не

«теория, которую можно пропустить» — без этого знания невозможно разобраться, когда программа из десятка файлов перестаёт собираться. Поэтому потратим на это целую главу.

Глава 8

Глава 7. Как программа становится исполняемым файлом

В главе 6 мы написали `hello.cpp`, скомпилировали и запустили. И вскользь упомянули, что компиляция идёт в четыре стадии: препроцессинг → компиляция → ассемблирование → линковка. В этой главе разбираем каждую стадию подробно. Что именно происходит, какие файлы появляются на каждом шаге, как выглядят их «потроха», что такое объектный файл, что такое символ, в чём разница между статической и динамической линковкой, и как программа лежит в памяти процесса.

Эта тема — фундамент, на котором всё дальнейшее построено. Без понимания, что компиляция многоступенчатая, и что «ошибка компиляции» отличается от «ошибки линковки», вы будете тонуть в сообщениях `gcc/clang` ближе к проектам. Поэтому глава длинная, но я постараюсь обойтись без лишней теории — будем смотреть на реальные файлы и команды.

8.1 Подопытная программа

Чтобы было что разбирать, заведём чуть более интересный пример, чем простой `hello`. Сделаем программу из двух файлов: главного `main.cpp` и вспомогательного `math.cpp` с функцией сложения.

Создайте папку и три файла.

`math.h`:

```
#ifndef MATH_H
#define MATH_H
```

```
int add(int a, int b);
```

```
#endif
```

math.cpp:

```
#include "math.h"
```

```
int add(int a, int b) {  
    return a + b;  
}
```

main.cpp:

```
#include <iostream>
```

```
#include "math.h"
```

```
int main() {  
    int x = add(2, 3);  
    std::cout << "2 + 3 = " << x << std::endl;  
    return 0;  
}
```

Три файла. Я их буду использовать на протяжении всей главы. Кратко скажу, что тут происходит — детали разберём по ходу.

В `math.h` лежит **объявление** функции `add` — обещание, что где-то есть функция с таким именем, принимающая два `int` и возвращающая `int`. Сам код функции тут не написан.

`#ifndef MATH_H ... #define MATH_H ... #endif` — это **защита от двойного включения** (include guard). Если этот заголовочный файл случайно попадёт в перевод дважды (например, его подключают два других файла, которые в свою очередь подключаются обоим), компилятор увидит, что `MATH_H` уже определён, и пропустит содержимое второй раз. Без этой защиты получим ошибку «redefinition of add». Подробнее про препроцессор — ниже.

В `math.cpp` лежит **определение** функции — собственно её код. Тут она `return a + b`.

В `main.cpp` функция `add` используется. Чтобы компилятор знал, что такая функция существует и что она принимает/возвращает, в начале файла подключаем `math.h` (через `#include "math.h"`). А определение функции живёт в `math.cpp` и попадёт в программу через линковку.

Это классическое разделение в C/C++: **заголовочный файл (.h)** — объявления, чтобы все знали, что есть такая функция; **исходный файл (.cpp)** — определения, реальные тела функций.

Соберём программу одной командой и убедимся, что работает:

```
$ g++ -std=c++11 main.cpp math.cpp -o program
$ ./program
2 + 3 = 5
```

Прекрасно. Теперь — препарируем процесс.

8.2 Стадия 1: препроцессор

Препроцессор — это первая стадия компиляции. Программа, отвечающая за неё, исторически называется `cpp` (C Pre-Processor; не путать с C++!). Что она делает?

Препроцессор работает с текстом. Он не понимает C++. Он не знает, что такое функция, что такое класс, что такое тип `int`. Для него файл `main.cpp` — это просто последовательность символов. Он ищет строчки, начинающиеся с `#`, и применяет к ним свои правила.

Самое частое — `#include`. Когда препроцессор видит `#include <iostream>`, он буквально берёт содержимое файла `iostream` из стандартной библиотеки и **вставляет его прямо в это место**. Текстуально, без анализа.

Также препроцессор обрабатывает: - `#define X Y` — макросы. После этой строки везде в файле `X` будет заменяться на `Y`. - `#ifdef`, `#ifndef`, `#endif` — условная компиляция. Куски кода включаются или исключаются в зависимости от того, определён ли тот или иной макрос. - `#pragma` — указания компилятору (нестандартные расширения). - Комментарии (хоть и не через `#`) — препроцессор их удаляет.

После работы препроцессора на выходе — один большой «развёрнутый» файл, в котором все `#include` подставлены, макросы развёрнуты, комментарии удалены.

8.2.1 Посмотрим своими глазами

GCC и Clang позволяют остановиться после препроцессора, флаг `-E`:

```
$ g++ -std=c++11 -E main.cpp -o main.i
$ wc -l main.i
40826 main.i
```

40 тысяч строк! Из наших 9 строк! Что произошло?

Откройте `main.i` любым редактором. Сверху увидите содержимое `iostream`, потом всех других файлов, которые `iostream` тянет за собой (а тянет он много — около сорока тысяч строк). А внизу, среди этой простыни, найдёте знакомое:

```
int add(int a, int b);

int main() {
    int x = add(2, 3);
    std::cout << "2 + 3 = " << x << std::endl;
    return 0;
}
```

Это и есть наш код. Заметили: `#include <iostream>` исчез — на его месте теперь буквально содержимое `iostream`. `#include "math.h"` тоже исчез — на его месте `int add(int a, int b);`.

8.2.2 Макросы

Препроцессор может ещё определять и подставлять **макросы**. Пример:

main.cpp:

```
#include <iostream>

#define PI 3.14159
#define SQUARE(x) ((x) * (x))

int main() {
    std::cout << "Pi = " << PI << std::endl;
    std::cout << "5 squared = " << SQUARE(5) << std::endl;
    return 0;
}
```

После препроцессора код превращается в:

```
int main() {
    std::cout << "Pi = " << 3.14159 << std::endl;
    std::cout << "5 squared = " << ((5) * (5)) << std::endl;
    return 0;
}
```

Каждое упоминание `PI` буквально заменилось на `3.14159`. Каждое упоминание `SQUARE(x)` развернулось в выражение `((x) * (x))`.

В C макросы были основным способом «не повторяться». В C++ макросы остаются, но их использование сократилось — вместо них есть `constexpr`-константы, `inline`-функции, шаблоны. Они почти всегда лучше макросов, потому что:

- Макрос — это текстовая замена; в нём нет понятия «тип». Если написать `SQUARE("hello")` — препроцессор просто подставит и оставит компилятору разбираться. Компилятор скажет «нельзя умножить строку на строку», но указание ошибки будет на сгенерированной строке, а не на той, где вы написали `SQUARE`.
- Макросы не видны отладчику.
- Макросы не уважают границы пространств имён.

Тем не менее макросы остаются: `include guard` (`#ifndef MATH_H` в нашем `math.h`) — это макрос. Условная компиляция (`#ifdef __APPLE__` — «выполнять этот блок только на macOS») — макросы. Платформенно-зависимый код часто полагается на макросы.

8.2.3 Условная компиляция

Допустим, мы хотим, чтобы наш код по-разному работал на Linux и на macOS:

```
#include <iostream>

int main() {
#ifdef __APPLE__
    std::cout << "Running on macOS\n";
#elif defined(__linux__)
    std::cout << "Running on Linux\n";
#elif defined(_WIN32)
    std::cout << "Running on Windows\n";
#else
    std::cout << "Running on something else\n";
#endif
    return 0;
}
```

Эти `__APPLE__`, `__linux__`, `_WIN32` — макросы, которые компилятор сам определяет в зависимости от того, на какой системе он работает. Препроцессор смотрит, что определено, и оставляет в коде только подходящий блок.

После препроцессора на macOS у вас в файле будет:

```
int main() {
    std::cout << "Running on macOS\n";
    return 0;
}
```

Никаких других веток нет — препроцессор их выкинул. Это **условная компиляция**. На больших проектах ей пользуются часто, чтобы единый код собирался под разные платформы.

8.2.4 Include guard детальнее

Раз про `#ifndef MATH_H` уже несколько раз говорили, разберём окончательно.

Допустим, у нас три файла:

a.h:

```
struct Point { int x; int y; };
```

b.h:

```
#include "a.h"
void draw(Point p);
```

main.cpp:

```
#include "a.h"
#include "b.h"

int main() {
    Point p{1, 2};
    draw(p);
    return 0;
}
```

После препроцессора main.cpp превратится в:

```
struct Point { int x; int y; }; // из a.h, через прямой include
struct Point { int x; int y; }; // из a.h, через b.h
void draw(Point p);           // из b.h
                               // (main.cpp дальше)

int main() {
    Point p{1, 2};
    draw(p);
    return 0;
}
```

Point объявлен дважды! Компилятор ругнётся: «redefinition of struct Point». Это и есть проблема двойного включения.

Решается стандартной защитой:

a.h:

```
#ifndef A_H
#define A_H

struct Point { int x; int y; };

#endif
```

Теперь, когда a.h подключается первый раз, A_H не определён, препроцессор заходит внутрь, определяет A_H и оставляет объявление Point. Когда a.h подключается второй раз (через b.h), A_H уже определён — препроцессор пропускает весь блок до #endif. Point объявлен ровно один раз.

В современном коде защиту иногда заменяют на #pragma once:

```
#pragma once

struct Point { int x; int y; };
```

Это нестандартное расширение, но его поддерживают все основные компиляторы. Удобнее (одна строка вместо трёх), но менее портabelно.

8.2.5 Что выучить про препроцессор

- Препроцессор работает с текстом, до компилятора.
- #include <header> — буквально подставляет содержимое.
- #include "header.h" — то же, но в первую очередь ищет в папке исходника.
- #define создаёт макрос. Старайтесь избегать макросов; для констант используйте const-expr, для функций — inline или шаблоны.
- Каждый .h-файл должен иметь защиту от двойного включения (#ifndef/#define/#endif или #pragma once).
- Платформенно-зависимый код пишут через #ifdef.

8.3 Стадия 2: собственно компиляция

После препроцессора мы получили развёрнутый текст. Теперь **компилятор** (в узком смысле — программа cscpp внутри g++) превращает его в **ассемблерный код** для целевой архитектуры.

Здесь происходит самое интересное:

1. **Лексический анализ.** Текст разбивается на «токены»: int, main, (,), {, и так далее.

2. **Синтаксический анализ.** Из токенов строится **AST** (abstract syntax tree) — дерево, отражающее структуру программы. Узел «функция main» с дочерним узлом «тело», в теле — узел «вызов функции add», и так далее.
3. **Семантический анализ.** Компилятор проверяет, что всё имеет смысл: типы совместимы, имена существуют, функции вызываются с правильным числом аргументов.
4. **Оптимизация.** Компилятор смотрит на код и переписывает его так, чтобы он делал то же самое, но эффективнее. Например, `int x = 2 + 3`; он сразу заменит на `int x = 5`; — это называется **constant folding**.
5. **Генерация кода.** AST превращается в ассемблерные инструкции под нужную архитектуру.

Если на любой из стадий компилятор найдёт проблему — это **ошибка компиляции**. Например, «add is not declared in this scope» — на стадии семантического анализа.

Остановимся после компилятора:

```
$ g++ -std=c++11 -S main.cpp -o main.s
$ wc -l main.s
80 main.s
```

Откройте `main.s` — это ассемблер для вашей архитектуры. На Mac M3 (ARM64) выглядит примерно так:

```
_main:
    sub sp, sp, #48
    stp x29, x30, [sp, #32]
    add x29, sp, #32
    mov w8, #2
    str w8, [sp, #8]
    mov w8, #3
    str w8, [sp, #4]
    ldr w0, [sp, #8]
    ldr w1, [sp, #4]
    bl _Z3addii
    ...
```

Под x86-64 (Intel/AMD) другой синтаксис, но смысл тот же. Каждая строка — одна инструкция.

Что тут особенно интересно: строчка `bl _Z3addii`. `bl` — это «branch with link», то есть вызов функции на ARM64. А `_Z3addii` — это имя функции `add`, преобразованное компилятором.

«Зачем имя так искорёжено?», — справедливый вопрос. Это называется **name mangling**.

8.3.1 Манглинг имён

В C функции называются буквально так, как вы их написали: `add` — это `add`. Это удобно, но непригодно для C++, где функции могут быть **перегружены**:

```
int add(int a, int b);
double add(double a, double b);
int add(int a, int b, int c);
```

Три разные функции, все называются `add`. Как линкер их отличит? Никак, если имена одинаковые.

Поэтому компилятор C++ «кодирует» в имя функции типы её параметров. На GCC/Clang используется схема Itanium ABI: `_Z3addii` = функция `add` с двумя `int` (`i` = `int`). `_Z3adddd` = функция `add` с двумя `double` (`d` = `double`). `_Z3addiii` = функция `add` с тремя `int`.

Линкер видит эти разные имена и спокойно различает.

На MSVC (Microsoft) используется другая схема манглинга, со своими правилами.

Чтобы посмотреть «человеческое» имя из «искорёженного», есть утилита `c++filt`:

```
$ echo "_Z3addii" | c++filt
add(int, int)
```

Очень полезно, когда вы видите в сообщении об ошибке «`undefined reference to _Z6lambda...`».

Иногда нужно отключить манглинг — например, при экспорте функций для C-кода. Для этого пишут `extern "C"`:

```
extern "C" int my_function(int x);
```

В этом случае компилятор имени `my_function` не трогает — линкер видит её как `my_function`. Это используется при создании библиотек, чтобы их могли использовать программы на других языках.

8.3.2 Что выучить про компиляцию

- Компилятор разбирает текст, проверяет смысл, и генерирует ассемблер.
- Ошибки компиляции — несовместимые типы, неизвестные имена, синтаксические ошибки.
- C++-имена кодируются (`mangle`) для поддержки перегрузки.
- Узнать «реальное» имя — через `c++filt`.

8.4 Стадия 3: ассемблирование

После компилятора у нас текстовый ассемблерный файл. Теперь его надо превратить в **машинный код**. Этим занимается **ассемблер** (программа `as`).

Ассемблер — простая программа. Она берёт ассемблерный текст и для каждой инструкции находит её бинарный опкод. На выходе — **объектный файл**.

```
$ g++ -std=c++11 -c main.cpp -o main.o
$ file main.o
main.o: Mach-O 64-bit object x86_64
# или, на ARM64 Mac:
main.o: Mach-O 64-bit object arm64
# на Linux:
main.o: ELF 64-bit LSB relocatable, x86-64, ...
```

Это уже бинарный файл. Прочитать как текст не получится — увидите кракозябры. Но есть утилиты для просмотра.

8.4.1 Внутри объектного файла

Объектный файл — это **не** просто массив машинных инструкций. Это структурированный формат, в котором есть:

1. **Секция кода (.text)** — собственно машинный код функций.
2. **Секция данных (.data)** — глобальные и статические переменные с начальным значением.
3. **Секция BSS (.bss)** — глобальные и статические переменные без начального значения (обнуляются автоматически при загрузке).
4. **Секция read-only (.rodata)** — константы, строковые литералы.
5. **Таблица символов** — список всех функций и переменных, которые объявлены в этом файле.
6. **Таблица релокаций** — список «дырок», где нужно подставить адреса других функций/переменных при линковке.

На разных платформах форматы разные: - **Linux** — ELF (Executable and Linkable Format). - **macOS** — Mach-O. - **Windows** — COFF/PE (Portable Executable).

Все они идейно похожи: секции, символы, релокации.

8.4.2 Смотрим на символы

Утилита `nm` показывает символы внутри объектного файла:


```
$ nm main.o
                 U __ZSt4cout
                 U __ZSt4endl
                 U __Z3addii
0000000000000000 T _main
                 U __ZNSolsEi
                 U __ZNSolsEPKc
```

Буква перед именем (U, T, и так далее) — это тип символа: - T (text) — символ определён в этом файле, в секции кода. У нас `_main` — определён. - U (undefined) — символ нужен, но не определён здесь. Например, `__Z3addii` — это `add(int, int)`, и наш `main.o` ждёт, что она будет где-то определена. - D (data) — определён в секции данных (глобальная переменная с инициализацией). - B (bss) — определён в секции BSS (глобальная переменная без инициализации). - R (rodata) — определён в read-only секции (константа).

Теперь сделаем `math.o`:

```
$ g++ -std=c++11 -c math.cpp -o math.o
$ nm math.o
0000000000000000 T __Z3addii
```

Видим: `__Z3addii` (то есть `add(int, int)`) **определён** в `math.o` (T). А в `main.o` он был **не определён** (U).

Это и есть основа линковки. У одного объектного файла есть «дырка» — символ, нужный, но не определённый. У другого этот же символ есть в готовом виде. Линкер их соединит.

8.4.3 Утилита `objdump`

`objdump` показывает содержимое объектного файла подробнее. Например:

```
$ objdump -d main.o | head -30
main.o: file format mach-o-x86-64

Disassembly of section __TEXT,__text:

_main:
    0:  push    %rbp
    1:  mov     %rsp, %rbp
    4:  sub     $0x20, %rsp
    8:  movl    $0x0, -0x4(%rbp)
    f:  movl    $0x2, %edi
```

```
14:  movl    $0x3, %esi
19:  callq   0x1e <_main+0x1e>
1e:  ...
```

Это уже почти то же самое, что в `.s`-файле, только с числовыми смещениями. `callq 0x1e` — это вызов функции по смещению `0x1e`. Сейчас оно ссылается само на себя (компилятор не знал куда указать). Линкер потом подставит правильный адрес.

8.5 Стадия 4: линковка

Самая интересная стадия. У нас есть два объектных файла: `main.o` и `math.o`. Линкер их соединит, разрешит все «дырки», и получит финальный исполняемый файл.

```
$ g++ main.o math.o -o program
$ ./program
2 + 3 = 5
```

Или, что то же самое, одной командой:

```
$ g++ main.cpp math.cpp -o program
```

Под капотом происходит:

1. Линкер читает все объектные файлы.
2. Собирает таблицу всех символов: какие определены где.
3. Для каждого символа `U` в каком-то файле ищет, где этот символ `T` (определён) — в других объектных файлах или в библиотеках.
4. Соединяет секции: все `.text` склеиваются в одну, все `.data` — в одну, и так далее.
5. Корректирует все ссылки: где была «дырка» с именем — подставляет реальный адрес.
6. Получается готовый исполняемый файл.

8.5.1 А стандартная библиотека?

Замечу: в `main.o` были «undefined» символы вроде `__ZSt4cout` (это `std::cout`). Откуда они придут?

Из **стандартной библиотеки**. Когда вы вызываете `g++`, он автоматически добавляет в линковку стандартную библиотеку C++ (`libstdc++` на Linux, `libc++` на macOS). Туда зашит код `std::cout`, `std::endl`, всех контейнеров, и так далее.

Если бы вы делали линковку вручную через `ld` (а не через `g++` как обёртку), вам пришлось бы явно указывать эти библиотеки. Поэтому-то и используют `g++` как обёртку — он знает, какие библиотеки нужно подцепить.

8.5.2 Сообщения об ошибках линковки

Самые частые ошибки этой стадии:

undefined reference to <symbol>

Линкер нашёл «дырку», но не нашёл, чем её заполнить. Возможные причины: - Забыли скомпилировать нужный .cpp файл. Например, написали `g++ main.cpp -o program` без `math.cpp` — `add` неоткуда взять. - Объявили функцию, но не написали её тело. - Не подключили нужную внешнюю библиотеку (флаг `-l`). - Перепутали манглинг (например, `extern "C"` где надо/не надо).

multiple definition of <symbol>

Два или больше определений одной функции. Линкер не знает, какое выбрать. Часто случается, когда определение функции случайно попадает в заголовочный файл, который подключается из нескольких .cpp-файлов.

Пример. Если бы мы в `math.h` написали не объявление, а определение:

```
// math.h
#ifndef MATH_H
#define MATH_H

int add(int a, int b) {    // ОШИБКА — определение в заголовке!
    return a + b;
}

#endif
```

И этот заголовок подключился бы в `main.cpp` и `math.cpp` — после препроцессинга в обоих файлах оказался бы код `add`. После компиляции — два определения в двух .о. Линкер: «multiple definition».

Решение: в заголовках — только объявления. Определения — в .cpp. (Исключение: шаблоны и `inline`-функции должны быть полностью в заголовках, об этом в главе 20.)

8.5.3 Статическая и динамическая линковка

Линковка бывает двух видов.

Статическая линковка. Все нужные функции из библиотек **копируются прямо в исполняемый файл**. Получается «жирный» бинарь, который работает сам по себе: библиотеки больше не нужны.

Плюсы: - Можно запустить на любом компьютере (нужной версии ОС/архитектуры) без необходимости иметь установленные библиотеки. - Не зависит от версий библиотек в системе.

Минусы: - Большой размер бинарника. - Если в библиотеке нашли уязвимость — нужно пересобрать программу, чтобы обновить.

Динамическая линковка. Линкер только записывает в исполняемый файл «здесь надо подгрузить такую-то библиотеку, и оттуда взять такую-то функцию». Сама библиотека лежит отдельным файлом и подгружается операционной системой при запуске программы.

Файлы динамических библиотек: - *.so (Shared Object) на Linux. - *.dylib на macOS. - *.dll (Dynamic Link Library) на Windows.

Плюсы: - Маленький размер бинарника. - Можно обновить библиотеку в системе — и все программы, использующие её, автоматически получают свежую версию. - Память экономится: если 50 программ используют libc — она в памяти один раз, не 50.

Минусы: - Нужно, чтобы библиотеки требуемых версий были в системе. - Знаменитый «DLL hell» — программа жаждет версию X.Y, а в системе X.Z, и не работает.

По умолчанию компиляторы используют **динамическую линковку для стандартной библиотеки и системных библиотек, и статическую для ваших объектных файлов**. То есть main.o + math.o слипаются в один бинарник, а libstdc++ подгружается во время запуска.

Узнать, какие динамические библиотеки нужны программе:

```
$ ldd ./program # на Linux
linux-vdso.so.1 (0x00007fff...)
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
/lib64/ld-linux-x86-64.so.2 (0x00007f...)

$ otool -L ./program # на macOS
./program:
    /usr/lib/libc++.1.dylib
    /usr/lib/libSystem.B.dylib
```

Чтобы собрать статически (всё внутрь бинарника):

```
$ g++ -static main.cpp math.cpp -o program
```

После этого `ldd program` покажет «not a dynamic executable». Размер бинаря вырастет в десятки раз (с ~30 KB до ~2-5 MB), потому что вся стандартная библиотека теперь внутри. На macOS полностью статическая линковка не поддерживается (Apple использует `dyld` и не разрешает статически линковать `libc`) — там можно только частично.

Статическую сборку часто используют для Docker-контейнеров минимального размера (образ `scratch` + статический бинарь), для подачи бинаря на сервер без установки зависимостей, для `embedded`-устройств.

8.5.4 Что выучить про линковку

- Линковка склеивает несколько `.o` файлов и библиотек в один исполняемый файл.
- «Undefined reference» — линкер не нашёл нужный символ. Проверьте, что нужный `.cpp` тоже компилируется и подключается.
- «Multiple definition» — символ определён в нескольких местах. В заголовках только объявления, не определения.
- Динамическая линковка — библиотеки подгружаются при запуске; статическая — копируются в бинарник.
- `ldd / otool -L` показывают, от каких библиотек зависит бинарник.

8.6 Memory layout: что где лежит в памяти процесса

Программа собрана, бинарник готов. Запускаем — что происходит в памяти?

Операционная система создаёт **процесс** (вспомните главу 1) и **отображает виртуальное адресное пространство процесса** на физическую память. В этом адресном пространстве у программы есть несколько ключевых областей.

Если идти от низких адресов к высоким (типичная картина на Linux x86-64):

8.6.1 `.text` (код)

В самом низу — секция **`.text`**. Это машинный код вашей программы: функции `main`, `add`, всё, что вы написали. Эта секция помечена как **read-only** и **executable**: процессор может её исполнять, но никто не может в неё писать. Если программа попытается записать в код — получит `segfault` (нарушение защиты памяти).

Это защита от того, чтобы программа случайно (или вирусно) не модифицировала собственный код во время работы.

8.6.2 .rodata (константы)

Чуть выше — секция **.rodata** (read-only data). Здесь лежат: - Строковые литералы: "Hello, world!", "2 + 3 = ". - const-константы с фиксированными значениями. - Виртуальные таблицы классов (vtable) — про них в главе 17.

Эта секция тоже read-only. Если вы возьмёте указатель на строковый литерал и попытаетесь через него что-то записать — segfault.

```
char* p = (char*)"hello";  
p[0] = 'H'; // segfault!
```

8.6.3 .data (инициализированные глобальные)

.data — глобальные и static-переменные с **ненулевым** начальным значением.

```
int counter = 5; // окажется в .data  
static int max_size = 100; // тоже в .data
```

Эта секция read-write. Программа может изменять эти переменные во время работы.

8.6.4 .bss (неинициализированные глобальные)

.bss — глобальные и static-переменные **без** начального значения (или с явным = 0). Они автоматически зануляются при загрузке.

```
int errors; // окажется в .bss, значение 0  
int counters[1000]; // .bss, все нули
```

Почему отдельная секция? Потому что .bss занимает в бинарнике ровно ноль байт — там просто записано «у меня нужно столько-то нулей». Заполнение нулями делает уже ОС, при загрузке. Экономия места на диске.

8.6.5 Куча (heap)

Дальше идёт **куча** (heap). Это область, из которой программа **динамически выделяет** память во время работы. Когда вы пишете `new int` или `malloc(100)`, ОС выделяет вам кусок из кучи. Когда вы вернёте через `delete` или `free`, кусок возвращается.

Куча растёт вверх (от низких адресов к высоким) по мере роста потребностей программы. Размер ограничен либо настройками ОС, либо физической памятью.

Куча — это про **долгоживущие** объекты. Например, дерево комнат в нашем RPG-проекте будет жить в куче, потому что оно должно существовать на протяжении всей игры, а не «пока выполняется одна функция».

8.6.6 Мmap-области

Где-то посередине между кучей и стеком (на современных ОС) живут **отображённые в память** регионы. Сюда подгружаются динамические библиотеки (`libstdc++.so`, `libc.so`). Сюда же попадают большие выделения через `mmap` (например, когда вы запрашиваете у ОС 100 MB одним куском).

Это нюанс — для понимания общей картины можно пропустить.

8.6.7 Стек (stack)

В самой верхней части адресного пространства — **стек**. Растёт вниз, в сторону кучи.

Стек — это **локальные переменные функций**, аргументы вызовов, адреса возврата. Когда вы вызываете функцию, на стек кладётся: - адрес, куда вернуться после функции (return address); - значения аргументов; - место под локальные переменные.

Когда функция возвращается — этот «кадр стека» (stack frame) убирается, и стек уменьшается.

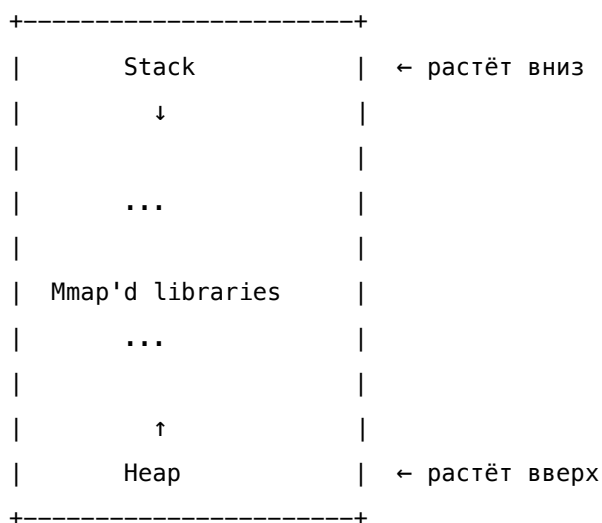
Стек — это про **короткоживущие** объекты. Локальная переменная `int x = 5`; внутри `main` — на стеке. Когда `main` закончится — `x` исчезнет автоматически.

```
void foo() {
    int local = 42;    // на стеке
}                    // local исчезает здесь
```

8.6.8 Картинка целиком

Адреса в типичном Linux-процессе x86-64 (упрощённо):

Высокие адреса



```

|      .bss      |
+-----+
|      .data     |
+-----+
|      .rodata   |
+-----+
|      .text     |
+-----+

```

Низкие адреса

Между стеком и кучей — большое пустое пространство, в которое они оба могут расти. Если стек дойдёт до кучи (или наоборот) — программа упадёт с переполнением.

8.6.9 Утилита `size`

Чтобы увидеть размеры секций бинарника:

```

$ size program
__TEXT __DATA __OBJC  others  dec hex
8192   4096    0  4295106560  4295119248  1000ff850

```

На Linux:

```

$ size program
   text    data     bss      dec      hex filename
  1820     608       24    2452     994 program

```

`text` — размер кода (`.text` + `.rodata`). `data` — инициализированных переменных. `bss` — неинициализированных (но в файле они не лежат, только размер).

8.6.10 Стек: размер и переполнение

Размер стека по умолчанию — на Linux обычно **8 MB**, на macOS — **8 MB** для главного потока и **512 KB** для дополнительных потоков, на Windows — **1 MB**. Можно изменить:

```

$ ulimit -s 16384 # установить 16 MB

```

Стек кончается при глубокой рекурсии:

```

void recur() {
    int big_array[10000]; // 40 KB на каждом вызове
    recur();
}

```


Эта функция вызывает себя бесконечно, каждый раз кладя на стек 40 КБ. Через 200 вызовов стек кончится → **stack overflow** → программа упадёт.

Куча кончается сложнее — обычно вы упрётесь в физическую память раньше, чем в ограничение на размер кучи. На 64-битной системе виртуального пространства хватит на петабайты, а физического — на 16-64 GB обычно.

В Главе 11 «Память — стек, куча, RAII» эти штуки разберём подробно с C++-примерами.

8.7 Полная картина: что происходит при сборке

Соберём весь процесс в одну схему. Команда `g++ main.cpp math.cpp -o program` выполняет:

1. **Препроцессор**: каждый `.cpp` файл проходит через `cpp`. Подставляются `include`'ы, разворачиваются макросы. На выходе — два развёрнутых файла (промежуточно, в памяти).
2. **Компилятор**: каждый развёрнутый файл проходит через `cc1plus`. Получается ассемблер. (Промежуточно.)
3. **Ассемблер**: каждый ассемблерный файл проходит через `as`. Получаются `main.o` и `math.o` (могут быть и не сохранены физически, если делаем одной командой).
4. **Линкер**: все объектные файлы плюс стандартная библиотека → исполняемый файл `program`.

Чтобы увидеть это в подробностях, есть флаг `-v` («verbose»):

```
$ g++ -v main.cpp math.cpp -o program
... много вывода ...
```

Увидите все запускаемые программы с их аргументами. Полезно, когда что-то не собирается и непонятно почему.

8.8 Один важный нюанс: единицы трансляции

В C++ есть понятие **translation unit** (единица трансляции). Это **один .cpp-файл вместе со всеми include'ами**, развёрнутый препроцессором.

Каждый `.cpp`-файл — отдельная `translation unit`. Они компилируются независимо. Это значит: - Изменения в одной TU не требуют пересборки остальных. Если поправили `math.cpp` — `main.cpp` пересобирать не надо, только новый `math.o` слинковать с старым `main.o`. - Декларации, которые видны только внутри одной TU (через `static` или `unnamed namespace`), не видны другим.

Это основа **incremental builds**: на больших проектах системы сборки (Make, CMake, Bazel, ...) пересобирают только те TU, чьи зависимости изменились. На моём ноутбуке Lirotech компилируется с нуля 8 минут; инкрементно — 5 секунд.

Подробнее — в главе 15 (Дизайн проекта RPG) с Makefile.

8.9 Маленькое упражнение

Заведите папку, создайте три файла из начала главы (math.h, math.cpp, main.cpp). Дальше:

1. Скомпилируйте каждый .cpp отдельно (флаг -c):

```
$ g++ -std=c++11 -c main.cpp
$ g++ -std=c++11 -c math.cpp
```

2. Посмотрите символы в каждом:

```
$ nm main.o
$ nm math.o
```

Найдите символ add в обоих. В одном он должен быть U (нужен), в другом T (определён).

3. Слинкуйте:

```
$ g++ main.o math.o -o program
```

4. Запустите.

Теперь нарочно сломайте линковку:

5. Скомпилируйте только main.cpp:

```
$ g++ -std=c++11 main.cpp -o broken
```

Что увидите? Изучите сообщение об ошибке.

6. В main.cpp замените add(2, 3) на multiply(2, 3). Скомпилируйте только main.cpp:

```
$ g++ -std=c++11 -c main.cpp
```

Что произойдёт и почему? Это ошибка компиляции или линковки?

7. Посмотрите ассемблер:

```
$ g++ -std=c++11 -S main.cpp -o main.s
$ less main.s
```

Найдите место вызова add. Обратите внимание на манглинг имени.

8. (Опционально) Посмотрите вывод препроцессора:

```
$ g++ -std=c++11 -E main.cpp | wc -l
```

Сколько строк? Откройте `g++ -E main.cpp -o main.i` и просмотрите. Найдите в конце своё `int main()`.

Эти упражнения покажут «вживую» все понятия главы. Если что-то не получается — перечитайте соответствующий раздел.

8.10 Что дальше

Часть 0 (подготовка) и Часть 0a (программирование с нуля) и Часть 0b (инструменты + компиляция) пройдены. Если вы дочитали досюда — вы знаете больше про сборку программ, чем 80% разработчиков на C++ с 3-летним стажем.

Глава 8 — **Часть I, Язык**. Начнём с самого фундамента C++: переменные и типы. Что такое `int`, какого он размера, чем отличаются `signed` и `unsigned`, как работают плавающие числа, что такое `auto` и почему в C++11 это полезно, какие бывают способы инициализации (`= 5`, `(5)`, `{5}`) и какой когда применять. Глава будет длинная — типы в C++ это правда сложно — но без этой базы дальше никак.

Глава 9

Глава 8. Переменные и типы

Часть 0 пройдена — у вас в голове устройство компьютера, биты, алгоритмическое мышление, терминал, модель компиляции. Теперь начинаем собственно C++ — Часть I, чистый язык, без проекта. Семь глав, в которых разберём базовые конструкции, чтобы к Части II (RPG-игра) подойти подготовленными.

Первая глава Части I — про **типы и переменные**. В Python вы пишете `x = 5` и не задумываетесь, что это число какого-то размера. В C++ всё иначе: при объявлении переменной вы обязаны сказать, какого она типа. И типов целых чисел — пять штук, разных размеров. И ещё типы дробных. И ещё `bool`, `char`. И ещё способы их инициализации — три разных. И ещё `const`, `constexpr`, `auto`. Глава длинная, но через неё надо честно пройти, иначе вся арифметика дальше будет «магией».

9.1 Почему в C++ типы такие важные

В Python `x = 5`; `x = "hello"` — нормально. Переменная меняет тип на лету. В C++ так нельзя. После `int x = 5`; переменная `x` навсегда `int`. Попытка `x = "hello"`; — ошибка компиляции.

Это нужно по двум причинам.

Производительность. Когда компилятор знает, что `x` — это 4-байтное целое, он генерирует машинный код, специально работающий с 4-байтными целыми. Никаких проверок «а вдруг это строка?», никаких словарей типов в рантайме. Поэтому C++ так быстр.

Безопасность. Компилятор находит много ошибок ещё до запуска. Если функция ждёт `int`, а вы пытаетесь передать строку — компилятор ругнётся. В Python такая ошибка вылезла бы только при запуске, и то — если до этого места выполнение бы дошло.

Цена — больше работы при написании кода и больше синтаксиса. Но это того стоит. Особенно когда программа дорастает до сотен тысяч строк.

9.2 Целые числа: семейство `int`

Начнём с самого частого. **Целое число** в C++ можно объявить пятью способами:

```
char    c = 1;    // 1 байт
short   s = 1;    // 2 байта
int     i = 1;    // 4 байта (обычно)
long    l = 1;    // 4 или 8 — зависит!
long long ll = 1; // 8 байт
```

Главный сюрприз: **размер этих типов не зафиксирован стандартом C++**. Стандарт гарантирует только некоторые минимумы и порядок:

- `char` — ровно 1 байт, как минимум 8 бит (де-факто на всех платформах = 8 бит).
- `short` — как минимум 16 бит.
- `int` — как минимум 16 бит.
- `long` — как минимум 32 бит.
- `long long` — как минимум 64 бит.

И ещё гарантия: $\text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$.

На практике на современных 64-битных системах:

Тип	Linux/macOS x86-64	Windows x86-64
<code>char</code>	1	1
<code>short</code>	2	2
<code>int</code>	4	4
<code>long</code>	8	4
<code>long long</code>	8	8

Видите ловушку? **`long` под Linux/macOS — 8 байт, а под Windows — 4!** Поэтому полагаться на размер `long` нельзя. Если вам нужно ровно 8 байт — используйте `long long`. Если ровно 4 — `int`.

9.2.1 Проверяем `sizeof`

`sizeof` — оператор C++, возвращающий размер типа в байтах. Используется так:

```
#include <iostream>

int main() {
    std::cout << "char:      " << sizeof(char) << "\n";
    std::cout << "short:     " << sizeof(short) << "\n";
    std::cout << "int:        " << sizeof(int) << "\n";
    std::cout << "long:       " << sizeof(long) << "\n";
    std::cout << "long long: " << sizeof(long long) << "\n";
    return 0;
}
```

Соберите и запустите. На моём Mac M3 вывод:

```
char:      1
short:     2
int:       4
long:      8
long long: 8
```

`sizeof` — это **оператор времени компиляции**: компилятор вычисляет размер на этапе сборки и подставляет уже число. Поэтому `sizeof(int)` стоит ровно столько же, сколько просто литерал 4 — никакой работы в рантайме.

Можно применять `sizeof` и к переменной: `sizeof(x)` или просто `sizeof x` (без скобок). Тип переменной известен компилятору, размер он подставит.

9.2.2 signed и unsigned

По умолчанию все целочисленные типы — **со знаком** (signed). То есть могут хранить и положительные, и отрицательные значения. Можно явно сказать «без знака»:

```
unsigned int ui = 1;
unsigned long ul = 1;
unsigned char uc = 200;
```

Размер тот же, но интерпретация другая (вспомните главу 2 — дополнительный код).

Тип	Размер	Минимум	Максимум
signed char	1	-128	127
unsigned char	1	0	255
signed short	2	-32 768	32 767
unsigned short	2	0	65 535

Тип	Размер	Минимум	Максимум
signed int	4	-2 147 483 648	2 147 483 647
unsigned int	4	0	4 294 967 295
signed long long	8	-9.2×10^{18}	9.2×10^{18}
unsigned long long	8	0	1.8×10^{19}

Можно сокращать: вместо `signed int` пишут `int`, вместо `unsigned int` — `unsigned`. Это эквивалентно. Но я рекомендую писать `unsigned int` или `unsigned long`, потому что одно слово `unsigned` без типа выглядит загадочно.

9.2.3 Когда использовать `unsigned`

Естественное чувство: «если число не может быть отрицательным — берём `unsigned`». Возраст, количество чего-то, размер массива.

В C++ совет: **берите `signed`, даже если число всегда неотрицательное**. Парадокс? Объясню.

Главная причина — **arithmetic underflow с `unsigned` работает «странно»**. Возьмём пример:

```
unsigned int i = 0;
i = i - 1;
std::cout << i;    // напечатает 4294967295!
```

Что произошло? $0 - 1$ для `unsigned` даёт $2^{32} - 1$ (обернулось через ноль). Это, между прочим, **строго определённое поведение** для `unsigned`, не `undefined behavior`. Стандарт говорит: «арифметика `unsigned` — по модулю 2^N , где N — число битов».

Но из-за этого вы получаете «странные» баги. Самый известный — цикл назад:

```
std::vector<int> v = {1, 2, 3, 4, 5};

// Идём по вектору в обратном порядке, типичный новичковый код:
for (unsigned i = v.size() - 1; i >= 0; --i) {
    std::cout << v[i];
}
```

Цикл должен вывести 5 4 3 2 1 и остановиться. Но условие `i >= 0` для `unsigned i` всегда истинно! `i` уменьшается, на нуле она ещё валидная, на следующей итерации `--i` оборачивает в 4294967295, и цикл продолжается. Программа залезает за границы вектора и крашится (или, хуже, выдаёт мусор).

С `int` такой проблемы нет:

```
for (int i = v.size() - 1; i >= 0; --i) {  
    std::cout << v[i];  
}
```

Когда `i` станет `-1`, условие `>= 0` нарушится, цикл завершится.

Поэтому правило: **используйте `int` или `long (signed)` для индексов и счётчиков, даже если они логически не отрицательные.**

Когда оправдан `unsigned`: - Битовые операции (`AND`, `OR`, `XOR`, сдвиги) — там знак не имеет смысла.

- Когда вам нужны именно полные 64 бита диапазона (`signed long long` даёт только 63). - Когда вы реально хотите `arithmetic mod 2^N` (например, в хэширующих функциях).

9.2.4 Целочисленные типы фиксированного размера

Раз размеры основных типов не зафиксированы, иногда надо именно «8 битов» или именно «32 бита». На этот случай в стандартной библиотеке есть типы из заголовочного файла `<stdint.h>`:

```
#include <stdint.h>  
  
int8_t    a = 1;    // ровно 1 байт со знаком (-128..127)  
uint8_t   b = 1;    // ровно 1 байт без знака (0..255)  
int16_t   c = 1;    // ровно 2 байта со знаком  
uint16_t  d = 1;    // ровно 2 байта без знака  
int32_t   e = 1;    // ровно 4 байта со знаком  
uint32_t  f = 1;    // ровно 4 байта без знака  
int64_t   g = 1;    // ровно 8 байт со знаком  
uint64_t  h = 1;    // ровно 8 байт без знака
```

Эти типы гарантированно одного размера на всех платформах. Удобно для бинарных форматов файлов и сетевых протоколов, где важна точная раскладка байтов.

Внутри это псевдонимы (`typedef` или `using`). На Linux/macOS `int32_t` — это обычно `int`. На Windows — тоже `int`. Но писать `int32_t` яснее: ты сразу видишь «здесь нужны ровно 4 байта».

В этой книге я буду стараться использовать `int` для обычных счётчиков (это коротко, и так все пишут), и `int32_t`/`uint64_t`/etc там, где размер важен (например, форматы файлов в проекте мини-СУБД).

9.2.5 Литералы целых чисел

Когда вы пишете в коде число 42 — это **литерал**. Его тип по умолчанию — самый маленький из (int, long, long long), который вмещает значение. Для большинства чисел — int.

Можно явно указать тип литерала **суффиксом**:

```
42          // int
42L         // long
42LL        // long long
42U         // unsigned int
42UL        // unsigned long
42ULL       // unsigned long long
```

Регистр суффикса роли не играет: 42ll, 42LL, 42lL — всё одно. Но писать большими лучше — маленькая l визуально путается с цифрой 1.

Кроме десятичной формы, литералы можно писать:

```
42          // десятичный
0x2A        // шестнадцатеричный (0x префикс)
0b101010    // двоичный (0b префикс, появился в C++14)
052         // восьмеричный (один ноль в начале)
```

Все четыре записи — одно и то же число 42. Восьмеричной формой почти никто не пользуется — в практике она встречается только в правах доступа Unix (вспомните `chmod 755`).

С C++14 разрешены **разделители разрядов**:

```
1'000'000    // миллион, читается легче чем 1000000
0xFF'FF      // 65535
0b1010'1100   // байт
```

Апостроф между цифрами — просто визуальное разделение, на значение не влияет. Очень удобно для длинных чисел. Но это C++14, и в нашей C++11-базе не работает.

9.3 Дробные числа: float и double

Для дробных чисел в C++ есть три типа:

```
float        f = 3.14f;    // 4 байта
double       d = 3.14;     // 8 байт
long double  ld = 3.14L;   // зависит — 8, 12 или 16
```

В прошлой главе про IEEE 754 я уже упоминал детали. Здесь повторю важное:

- `float` — 4 байта, **примерно 7 значащих десятичных цифр** точности. Диапазон: $\pm 10^{\pm 3}$... $\pm 10^{\pm 3}$.
- `double` — 8 байт, **примерно 15-16 значащих цифр**. Диапазон: $\pm 10^{\pm 3}$... $\pm 10^{\pm 3}$.
- `long double` — на x86-64 Linux обычно 80-битное число с заполнением до 16 байт. На macOS ARM64 — то же, что `double` (Apple не реализовала `extended precision` на ARM). На Windows — то же, что `double`.

Если коротко: **используйте `double`** для всех расчётов. Это разумный дефолт. `float` берите только когда у вас огромный массив чисел и память жмёт (4 vs 8 байт даст экономию вдвое), или когда работаете с библиотекой, которая ждёт именно `float` (GPU, OpenGL, аудио). `long double` — экзотика, не нужная в 99% задач.

9.3.1 Литералы дробных чисел

```
3.14      // double по умолчанию
3.14f     // float
3.14L     // long double
1.0       // double, потому что 1.0 это не целое
1e6       // 1 миллион, double (экспоненциальная форма)
1.5e-3    // 0.0015, double
1.5E-3    // то же — регистр не важен
```

`f`-суффикс для `float` обязателен, если вы хотите именно `float`. Если написать `float f = 3.14;` — компилятор предупредит «`implicit conversion from double to float`», потому что `3.14` без `f` — это `double`, и оно «теряет точность» при присвоении в `float`.

9.3.2 Точность — самое больное место

Двоичная плавающая точка не может точно представить большинство десятичных дробей. Классический пример:

```
#include <iostream>

int main() {
    double x = 0.1 + 0.2;
    std::cout << x << "\n";           // 0.3 — но это печать с округлением!
    std::cout.precision(20);
    std::cout << x << "\n";           // 0.30000000000000004441

    if (x == 0.3) std::cout << "equal\n";
    else          std::cout << "NOT equal\n"; // эта ветка!
```

```
    return 0;
}
```

$0.1 + 0.2$ не равно ровно 0.3 . Поэтому **сравнивать дробные числа на равенство почти никогда нельзя**. Вместо `if (x == 0.3)` пишут:

```
#include <cmath>

if (std::abs(x - 0.3) < 1e-9) {    // отличается меньше чем на  $10^{-9}$ 
    // считаем равными
}
```

Или, для совсем серьёзных случаев, есть `std::numeric_limits<double>::epsilon()` — но это уже тема главы про библиотеку.

И ещё: **никогда не храните деньги в `double`**. Кажется логично — деньги это числа, что не так? А то, что у вас на счёте $100.10\$$. Программа на сервере умножает баланс на процент: $100.10 * 1.05 = 105.105$. Округляете до двух знаков — 105.11 . Но на самом деле в `double` лежало не 100.10 , а 100.09999999999999 . Умножение даёт 105.10499999999999 . Округление — 105.10 . Копейка пропала. Бухгалтерия в шоке.

Для денег используйте либо `int64_t` в копейках/центах (баланс хранится как 10010 копеек), либо специальные библиотеки `decimal-arithmetic`. В нашей книге денег не будет, но имейте в виду на будущее.

9.4 bool

Тип для логических значений:

```
bool a = true;
bool b = false;
bool c = (5 > 3);    // c = true
```

Два значения — `true` и `false`. Размер `sizeof(bool)` обычно 1 байт (хотя нужен только 1 бит — но процессоры не умеют адресовать отдельные биты).

В C++ `bool` неявно конвертируется к `int`: `true` → 1, `false` → 0. И наоборот: любое ненулевое число → `true`, ноль → `false`.

```
int n = 5;
if (n) {                // эквивалентно if (n != 0)
    std::cout << "non-zero\n";
}
```

```
bool b = 42;           // b = true, потому что 42 != 0
int x = true;          // x = 1
```

Эта возможность — приятное наследие C, но иногда мешает. Современный C++ пишет явно:

```
if (n != 0) { ... }    // лучше, чем if (n)
if (ptr != nullptr) { ... } // лучше, чем if (ptr)
```

В компиляторах есть флаги, ругающиеся на неявные конверсии (`-Wconversion`), и я их рекомендую включать на новых проектах.

9.5 char и символы

`char` — целочисленный тип размером 1 байт, традиционно используемый для хранения одного символа ASCII.

```
char c = 'A';
std::cout << c;           // напечатает A
std::cout << (int)c;      // напечатает 65 (код ASCII буквы A)
```

`'A'` — это литерал символа, **одинарные кавычки**, ровно один символ внутри. Его тип — `char`, значение — числовой код символа в ASCII (или Latin-1, или в чём угодно, что компилятор выбрал; для буквы A это всегда 65).

Двойные кавычки — это уже **строка**: `"A"` — это не `char`, а `const char[2]` (массив из двух элементов: `'A'` и невидимого `'\0'` в конце). Принципиально другой тип.

9.5.1 Escape-последовательности

В символьном или строковом литерале нельзя написать буквальный перевод строки или кавычку. Для них используют escape-последовательности:

```
'\n' // перевод строки (newline)
'\t' // табуляция
'\r' // возврат каретки (на Windows используется в паре \r\n)
'\\' // обратный слеш
'\'' // одинарная кавычка
'\"' // двойная кавычка (внутри строкового литерала)
'\0' // нулевой символ (терминатор C-строк)
'\xFF' // байт со значением 0xFF (255)
'\377' // байт со значением 0377 в восьмеричной (= 255)
```

9.5.2 char, signed char, unsigned char

Один из неприятных уголков C++: тип `char` может быть как `signed`, так и `unsigned` — **зависит от компилятора и платформы**. На большинстве — `signed` (диапазон -128..127), но на ARM-Linux исторически `unsigned` (0..255).

В реальности это редко имеет значение для текста (ASCII только 0-127, попадает в обе интерпретации). Но если работаете с байтами как с числами:

```
char c = 0xFF;
int n = c;      // 0xFF → -1 (если char signed) или 255 (если unsigned)
```

Чтобы было определённо, пишите явно:

```
signed char sc = 0xFF;    // -1
unsigned char uc = 0xFF;  // 255
```

В большинстве случаев `char` для текста, `unsigned char` или `uint8_t` для байтов.

9.6 const — переменная, которую нельзя менять

Иногда нужно сказать «вот значение, я его задаю один раз и потом не меняю». Это **константа**. В C++ ставится модификатор `const`:

```
const int max_size = 100;
const double pi = 3.14159;

max_size = 200;    // ОШИБКА компиляции: assignment of read-only variable
```

`const`-переменная инициализируется при создании, и больше её значение менять нельзя. Попытка — ошибка компиляции.

Зачем это нужно?

- **Документация для читателя.** Видя `const int max_size`, читатель сразу понимает: это константа, она никогда не меняется.
- **Защита от случайных правок.** Если коллега полез в код и попытался изменить — компилятор остановит.
- **Возможности оптимизации.** Зная, что значение не меняется, компилятор может его «подставить» прямо в код, не читая из памяти.
- **Корректность функций.** Параметр функции, помеченный `const`, гарантирует: внутри функции его не меняют. Об этом подробно в главе 10.

Запомните железно: в современном C++ всё, что не должно изменяться — должно быть `const`. Это лучшая защита от багов.

9.7 constexpr — константа времени компиляции (C++11)

`const` говорит «не менять». А `constexpr` (новинка C++11) говорит «это значение известно на этапе компиляции».

```
constexpr int max_size = 100;
constexpr double pi = 3.14159;
constexpr int doubled = max_size * 2; // 200, посчитано на компиляции
```

Разница с `const`:

```
int x;
std::cin >> x;
const int a = x; // OK: a — константа, но её значение известно только в рантайме
constexpr int b = x; // ОШИБКА: b должна быть известна на компиляции, а x — нет
constexpr int c = 100; // OK: 100 — литерал, известен на компиляции
```

То есть `constexpr` строже, чем `const`: значение должно быть вычислимо компилятором.

Зачем? Для использования в местах, где нужно именно «значение компилятора». Например, размер массива:

```
const int n = 10;
int array1[n]; // OK в C++ (хотя не в строгом стандарте)

int m;
std::cin >> m;
const int p = m;
int array2[p]; // НЕ OK: размер массива должен быть известен на компиляции
               // (хотя GCC через VLA это разрешает как расширение)
```

`constexpr` решает эту проблему явно: «вот это точно вычислимо на компиляции, не сомневайся».

В C++14+ можно делать `constexpr`-функции, в C++20 — даже выделять память на куче на компиляции. Но в C++11 `constexpr` довольно ограничен — простые арифметические выражения с литералами и другими `constexpr`-переменными.

Правило: для констант предпочитайте `constexpr`. Если нельзя — `const`. Если хотите изменяемость — обычная переменная.

9.8 Способы инициализации

В C++11 есть **три способа** проинициализировать переменную, и они немного отличаются. Это очень частый источник путаницы.

9.8.1 Старый способ: с равно

```
int x = 5;
double pi = 3.14;
char c = 'A';
```

Знакомый по другим языкам. Работает почти всегда.

9.8.2 С круглыми скобками

```
int x(5);
double pi(3.14);
char c('A');
```

Это «конструкторный синтаксис». Для встроенных типов делает то же, что =. Но для классов — вызывает соответствующий конструктор.

Подвох — **most vexing parse**:

```
int x();    // НЕ объявление int x = 0! Это ОБЪЯВЛЕНИЕ ФУНКЦИИ x, возвращающей int
```

Компилятор интерпретирует `int x()` как объявление функции без параметров, возвращающей `int`. Из-за этой ловушки многие предпочитают другие синтаксисы.

9.8.3 Единообразная инициализация (C++11): с фигурными скобками

```
int x{5};
double pi{3.14};
char c{'A'};
```

Этот синтаксис **появился в C++11** и считается рекомендуемым. Преимущества:

1. **Работает везде одинаково**: для встроенных типов, классов, массивов, стандартных контейнеров.

```
int x{5};
std::vector<int> v{1, 2, 3, 4};    // вектор из 4 элементов
```

2. **Защищает от «narrowing» — сужающих преобразований.** Если вы пытаетесь засунуть значение, которое не поместится — будет ошибка компиляции:

```
int x = 3.14;    // OK, x = 3 (молча отбросило 0.14)
int y(3.14);    // OK, y = 3 (тоже молча)
int z{3.14};    // ОШИБКА: narrowing conversion from double to int
```

Эта проверка спасла моих коллег от десятков багов. Серьёзно.

3. **Не страдает от most vexing parse:**

```
int x{};    // x = 0 (value initialization), однозначно
```

В книге я буду использовать {} там, где это естественно, и = для простых случаев. Это слегка непоследовательно, но соответствует общепринятой практике.

9.8.4 Default initialization vs value initialization

Что произойдёт, если объявить переменную **без значения**?

```
int x;    // !!! не инициализирован
```

Для **локальной переменной встроенного типа** значение **не определено**. Внутри переменной — мусор, что было в памяти до этого. Чтение мусора — UB (undefined behavior).

```
int x;
std::cout << x;    // UB — может напечатать что угодно, или упасть
```

Чтобы инициализировать нулём:

```
int x{};    // x = 0, value initialization
int x = 0;    // тоже OK, но явно
```

Для **глобальных и static-переменных** правила другие: они **автоматически** инициализируются нулями (точнее — value initialization). Это потому, что глобалы лежат в .bss-секции, которая зануляется при загрузке (вспомните главу 7).

```
int g;    // глобальная — автоматически 0
static int s;    // static — автоматически 0

int main() {
    int local;    // НЕ инициализирована
    std::cout << g << " " << s << " " << local;    // 0 0 ???
}
```


Правило: всегда инициализируйте локальные переменные. Не пишите `int x;` без значения. Привычка эта спасёт вас от сотен невоспроизводимых багов.

9.9 auto — компилятор сам выводит тип (C++11)

В C++11 появилось ключевое слово `auto`. Оно говорит: «компилятор, сам определи тип из правой части».

```
auto x = 5;           // компилятор видит 5 → int → x это int
auto y = 3.14;        // double
auto s = "hello";     // const char*
auto v = std::vector<int>{1, 2, 3}; // std::vector<int>
```

Зачем? Без `auto` иногда приходится писать длиннющие типы:

```
std::map<std::string, std::vector<int>>::iterator it = m.begin();
```

С `auto`:

```
auto it = m.begin();
```

Куда читабельнее.

9.9.1 Когда auto хороший выбор

- Длинные типы (итераторы, типы из шаблонов).
- Когда тип очевиден из правой части:

```
auto x = std::vector<int>(100); // ясно, что это vector<int>
```

- При работе с лямбдами (тип лямбды нельзя записать без `auto`).

9.9.2 Когда auto плохой выбор

- Когда читатель не понимает, какой тип получится. Например:

```
auto x = some_func(); // что такое x?
```

Если из имени функции не ясно — лучше написать тип явно.

- Когда `auto` скрывает важную деталь. Например:

```
for (auto x : container) { ... }
```

Здесь `x` — копия! Если контейнер хранит большие объекты, копии дорогие.

```
for (const auto& x : container) { ... }
```

А вот это — константная ссылка, без копирования. Гораздо лучше для больших объектов.

9.9.3 auto и const/&

auto берёт тип «как есть», но **отбрасывает const и &**:

```
const int& ref = ...;
auto a = ref;           // a — это int (без const, без &)
const auto& b = ref;    // b — это const int& (как и ref)
```

Поэтому привычное `const auto&` — это «возьми тип, и сделай ссылкой на const». Самый частый паттерн для циклов:

```
for (const auto& item : items) {
    // item — это const-ссылка на элемент контейнера
    // никаких копий, и нельзя случайно изменить
}
```

Запомните этот паттерн — он будет повторяться постоянно.

9.10 Имена переменных: правила и стиль

Правила компилятора: - Имя — буквы, цифры и подчёркивание. - Начинаться с буквы или подчёркивания (но **не с цифры**). - Регистр важен: `count` и `Count` — разные переменные. - Нельзя использовать ключевые слова (`int`, `if`, `while`, и т. д.). - Имена, начинающиеся с подчёркивания + заглавной буквы (`_Foo`) или с двух подчёркиваний (`__bar`), зарезервированы для компилятора и стандартной библиотеки. **Не используйте их**. - В C++ разрешены **юникод-имена**: `cpp int сумма = 0; // работает (если компилятор настроен на UTF-8)` Но я не рекомендую: смешение языков в коде раздражает, и не все системы понимают utf-8 source.

Стиль — это уже вопрос команды. Самые популярные:

- **snake_case** (`max_size`, `current_value`) — преобладает в стандартной библиотеке C++ и в стиле Google.
- **camelCase** (`maxSize`, `currentValue`) — преобладает в Java, popular в C++.
- **PascalCase** (`MaxSize`) — обычно для имён типов (классов, структур), не для переменных.

Я в книге использую **snake_case** для переменных, **PascalCase** для типов:

```
int player_health;
double damage_multiplier;
```

```
class Player;  
struct Inventory;
```

Это близко к стандартной библиотеке (`std::vector`, `std::string`) и читаемо.

Хорошие имена: - Говорят, что переменная значит: `player_health`, `damage_per_round`, `is_alive`.
- Не сокращены до неузнаваемости: `cnt` плохо, `count` лучше, `enemy_count` ещё лучше. - Не повторяют тип в имени: `int_count` плохо (и так видно по типу). - Короткие имена — только для коротких циклов: `i`, `j`, `k` для счётчиков, `it` для итератора, `n` для размера.

Плохие имена: - `temp`, `tmp`, `data`, `value` — не говорят ничего. - `x1`, `x2`, `x3` — порядок появления, не смысл. - `theBestVariableEverInTheWorld` — слишком длинное.

9.11 Полный собранный пример

Соберём всё в работающую программу:

```
#include <iostream>  
#include <cstdlib>  
  
int main() {  
    // Целочисленные типы  
    int count = 42;  
    long long big_number = 1000000000000LL;           // в C++14 для читаемости: 1'000'000'000'000LL  
    unsigned int positive_only = 100;  
    uint8_t one_byte = 255;  
  
    // Дробные  
    double pi = 3.14159265358979323846;  
    float weight_kg = 75.5f;  
  
    // Логический  
    bool game_over = false;  
  
    // Символ и строка (строки разберём в главе 12)  
    char first_letter = 'A';  
    const char* message = "Hello, C++!";  
  
    // Константа времени компиляции  
    constexpr int max_players = 8;
```

```

// auto и инициализация фигурными скобками
auto health = 100;           // int
auto damage = 12.5;         // double
auto enabled = true;        // bool

int explicit_init{0};       // value initialization

// Выводим (тут пока в одну строку, форматирование – в главе 12)
std::cout << "count = " << count << "\n";
std::cout << "big_number = " << big_number << "\n";
std::cout << "pi = " << pi << "\n";
std::cout << "weight_kg = " << weight_kg << "\n";
std::cout << "game_over = " << game_over << "\n";
std::cout << "first_letter = " << first_letter
          << " (code " << (int)first_letter << ") \n";
std::cout << "max_players = " << max_players << "\n";

return 0;
}

```

Сохраните как types_demo.cpp, соберите и запустите:

```

$ g++ -std=c++11 -Wall -Wextra types_demo.cpp -o types_demo
$ ./types_demo

```

Должно вывести что-то осмысленное. Если убрать LL у big_number — на 32-битных платформах будет переполнение и -Wall ругнётся.

9.12 Преобразования типов: коротко

Между разными типами C++ умеет автоматически конвертировать. Правила:

```

int i = 5;
double d = i;           // OK: int → double (расширение)

double pi = 3.14;
int j = pi;             // OK, но компилятор может предупредить – теряем 0.14

unsigned int u = 5;

```

```
int s = u;           // OK
unsigned int u2 = -1; // Ой: -1 как unsigned = 4294967295. Предупреждение от -Wsign-conversion.

bool b = 1;          // OK: 1 → true
int n = false;       // OK: false → 0
```

Преобразования бывают: - **Расширяющие** (widening): меньший тип → больший. Безопасно. - **Сужающие** (narrowing): больший тип → меньший. Может терять данные. - **Со знаком ☒ без знака**: интерпретация битов меняется, но биты те же. Опасно.

Через {} инициализацию (C++11) сужающие конверсии запрещены — это ошибка компиляции. Через = или () — только предупреждение, и то с подходящими флагами.

Явное преобразование пишут через `static_cast`:

```
double pi = 3.14;
int n = static_cast<int>(pi); // явно говорим: знаем что теряем дробную часть
```

Есть ещё `reinterpret_cast` (опасный, переинтерпретация битов) и `const_cast` (удаление `const`). К ним вернёмся в нужных местах книги.

9.13 Главное — что увидите в коде дальше

Этой главы хватит, чтобы понимать почти любую переменную, которая будет встречаться. На примерах:

```
int health = 100;           // 4-байтное целое
double damage = 12.5;       // 8-байтное дробное
bool alive = true;          // логический
char first_letter = 'A';    // 1-байтный символ
const int max_players = 8;   // неизменяемая константа
constexpr double pi = 3.14159; // константа времени компиляции
auto count = 42;             // компилятор сам определит → int
auto it = container.begin(); // компилятор сам определит → итератор
const auto& item = container[0]; // const-ссылка, без копии
int32_t precise_size = 100;  // ровно 4 байта, для протоколов
uint8_t byte_value = 0xFF;   // ровно 1 байт, для байтовых данных
```

К каждой из этих форм будете возвращаться постоянно.

9.14 Маленькое упражнение

Соберите эти программы и проверьте, что вы правильно угадали поведение:

1. Узнайте размеры типов на вашем компьютере:

```
std::cout << sizeof(int) << " " << sizeof(long) << " " << sizeof(long long);
```

На Linux/macOS x86-64 будет 4 8 8. На Windows x86-64 — 4 4 8.

2. Покажите, что unsigned оборачивается:

```
unsigned int x = 0;
std::cout << x - 1 << "\n"; // напечатает 4294967295
```

3. Узнайте, signed или unsigned у вас char:

```
char c = -1;
if (c < 0) std::cout << "char is signed\n";
else      std::cout << "char is unsigned\n";
```

4. Покажите неточность плавающей точки:

```
std::cout.precision(20);
double a = 0.1 + 0.2;
std::cout << a;
```

5. (Самое полезное) Запустите программу с предупреждениями:

```
int main() {
    int x = 3.14;           // -Wfloat-conversion ругнётся
    int y{3.14};           // ошибка narrowing
    unsigned u = -1;       // -Wsign-conversion ругнётся
    int z;
    std::cout << z;         // -Wuninitialized ругнётся
    return 0;
}
```

Команда сборки: `g++ -std=c++11 -Wall -Wextra -Wpedantic -Wconversion test.cpp -o test`. Внимательно читайте каждое предупреждение. Включите эти флаги во все проекты — они помогают.

9.15 Что дальше

Глава 9 — управляющие конструкции. `if/else`, `switch`, циклы `for`, `while`, `do-while`, `range-based for` из C++11, `break/continue`, и почему `goto` — это не зло, а просто инструмент с ограниченной областью применения. Дальше каждая следующая глава будет всё ближе к настоящей программе — после блока про основы языка перейдём к RPG-проекту, где все эти штуки соединятся в реальный код.

Глава 10

Глава 9. Управляющие конструкции

В главе 4 мы обсуждали ветвление и циклы концептуально, на псевдокоде. В главе 8 разобрались с переменными. Теперь склеиваем — пишем настоящие управляющие конструкции на C++. Тут много синтаксиса, но материал в целом простой: после Python/PHP/JS большая часть будет узнаваемой. Главное — пара ловушек, на которых спотыкаются новички, и одна-две особенности C++, которых нет в других языках.

10.1 if — простое ветвление

Базовая форма:

```
if (условие) {  
    действия;  
}
```

Если условие истинно — выполняются действия в {...}. Иначе — пропускаются.

```
int health = 50;  
if (health < 30) {  
    std::cout << "Внимание: мало HP!\n";  
}
```

Скобки (...) вокруг условия — **обязательны**. В отличие от Python или Ruby, в C++ нельзя написать `if health < 30 {...}`.

Фигурные скобки {...} вокруг тела — формально необязательны, если внутри одно действие:

```
if (health < 30)  
    std::cout << "Мало HP!\n";
```


Но я **настоятельно рекомендую всегда писать {...}**. Без скобок легко вляпаться:

```
if (health < 30)
    health = health + 10;
    std::cout << "Heal!\n";    // эта строка выполнится ВСЕГДА!
```

Отступы вводят в заблуждение. Без {...} к if относится только первая строка, второй std::cout выполняется всегда. Внутри if он не входит, потому что нет фигурных скобок. Я лично потерял несколько часов на таких багах в начале карьеры. С скобками такой ошибки не сделать:

```
if (health < 30) {
    health = health + 10;
    std::cout << "Heal!\n";
}
```

Дешёвый страховочный полис. Не экономьте две строки.

10.2 if/else

Альтернативная ветка пишется через else:

```
if (health > 0) {
    std::cout << "Жив\n";
} else {
    std::cout << "Мёртв\n";
}
```

else if для нескольких условий подряд:

```
if (health <= 0) {
    std::cout << "Мёртв\n";
} else if (health < 30) {
    std::cout << "Тяжёлое ранение\n";
} else if (health < 70) {
    std::cout << "Ранен\n";
} else {
    std::cout << "В порядке\n";
}
```

Проверки идут **сверху вниз**. Срабатывает первая истинная. Остальные не проверяются — важно, если в условиях есть вычисления.

Кстати, `else if` в C++ — это не отдельная конструкция, а просто `else` со вложенным `if`. То есть код выше эквивалентен:

```
if (health <= 0) {  
    ...  
} else {  
    if (health < 30) {  
        ...  
    } else {  
        if (...) { ... }  
    }  
}
```

Но первая форма читается лучше, поэтому все так и пишут.

10.3 Логические операции в условиях

Условие — это любое выражение, которое можно интерпретировать как `bool`. Самые частые операции:

```
==    // равно  
!=    // не равно  
<     // меньше  
<=   // меньше или равно  
>     // больше  
>=   // больше или равно  
&&    // логическое И  
||    // логическое ИЛИ  
!     // логическое НЕ
```

Примеры:

```
if (age >= 18 && age < 65) {  
    std::cout << "Работоспособный возраст\n";  
}  
  
if (day == "суббота" || day == "воскресенье") {  
    std::cout << "Выходной\n";  
}  
  
if (!alive) {
```

```
std::cout << "He жив\n"; // эквивалентно alive == false
}
```

10.3.1 Ловушка номер один: = vs ==

Это самый частый баг новичка в C++ (и в C, и в Java, и в JS).

= — это **присваивание**. `x = 5` — положить 5 в переменную `x`.

== — это **сравнение**. `x == 5` — проверить, равен ли `x` пятёрке.

В условии нужно сравнение. Если случайно написать одно равно:

```
if (health = 0) { // ОЙ!
    ...
}
```

— то компилятор не выдаст ошибку. Он подумает: «Ага, программист хочет положить 0 в `health`, а потом проверить, что присвоено». В присваивании результат — само значение, то есть 0. Ноль интерпретируется как `false`. Условие никогда не сработает. И ещё в `health` уляглись 0 — побочный эффект.

Очень коварный баг. Современные компиляторы это ловят:

```
warning: using the result of an assignment as a condition without parentheses
```

— но только с флагами `-Wall`. Включайте флаги.

Есть приём защиты — **сравнение задом наперёд**, когда константа слева:

```
if (0 == health) { // если случайно один знак — ошибка компиляции
    ...
}
```

Если написать `0 = health` — ошибка: нельзя присваивать константе. Но эта запись неестественная (читается «если ноль равен `health`» — задом наперёд). В современном C++ за неё не голосуют. Просто будьте внимательны и используйте флаги.

10.3.2 Short-circuit evaluation

Очень важная особенность: операторы `&&` и `||` **сокращают вычисления**.

Для `A && B`: если `A` ложно — `B` **не вычисляется**. Результат всё равно ложь.

Для `A || B`: если `A` истинно — `B` не вычисляется. Результат всё равно истина.

Зачем это полезно? Защита от ошибок:

```
if (ptr != nullptr && ptr->value > 0) {  
    ...  
}
```

Здесь, если `ptr` нулевой — мы **не** обращаемся к `ptr->value` (что было бы UB и крашем программы). Сначала проверили, что `ptr` не пуст, и только потом разыменовали.

То же самое с делением:

```
if (b != 0 && a / b > 5) {  
    ...  
}
```

Деление на ноль не произойдёт, потому что `b != 0` ложь $\rightarrow$ `a / b` не вычисляется.

Помните этот порядок — он спасает кучу багов.

10.3.3 Скобки и приоритет

Когда операторов несколько, могут возникать вопросы порядка:

```
if (a == 5 || b == 5 && c == 5) {  
    ...  
}
```

Что тут? `(a == 5) || ((b == 5) && (c == 5))`? Или `((a == 5) || (b == 5)) && (c == 5)`?

В C++ `&&` имеет **приоритет выше** `||`. То есть `&&` «связывает крепче». Расшифровка — первая.

Я **очень рекомендую** писать скобки явно:

```
if ((a == 5) || (b == 5 && c == 5)) {  
    ...  
}
```

Скобки лишние? Лучше лишние, чем непонятные. Особенно когда условие сложное.

10.4 Дополнительный конструктор: `if` с инициализацией (C++17)

С C++17 можно объявлять переменную прямо в условии:

```
if (int x = compute(); x > 0) {  
    use(x);  
}  
  
// x здесь уже не виден
```

Это C++17, и в нашей C++11-базе **не работает**. Но удобная штука, упоминаю на будущее.

В C++11 эквивалент:

```
{
    int x = compute();
    if (x > 0) {
        use(x);
    }
}
// x здесь не виден
```

10.5 Тернарный оператор

Краткая форма if/else для простого выбора между двумя значениями:

условие ? значение_если_истина : значение_если_ложь

Пример:

```
int abs_value = (x < 0) ? -x : x;
```

Эквивалент:

```
int abs_value;
if (x < 0) {
    abs_value = -x;
} else {
    abs_value = x;
}
```

Тернарник полезен в выражениях, где хочется выбрать значение:

```
std::cout << "Урон: " << (is_critical ? damage * 2 : damage) << "\n";
```

С тернарником без () вокруг — может быть некрасиво:

```
int x = a > b ? a : b; // максимум, читается ок
```

Но злоупотреблять не стоит. Когда условие сложное или выбираемых значений много — лучше обычный if/else. Тернарник без скобок легко спутать с другими операторами:

```
int y = x > 0 ? 1 : x < 0 ? -1 : 0; // SIGNUM, но читать тяжело
```

Лучше переписать в if/else if/else — будет понятнее.

10.6 switch — выбор из множества значений

Когда у вас много альтернатив на одну и ту же переменную:

```
char direction;
std::cin >> direction;

switch (direction) {
    case 'N':
        std::cout << "Идешь на север\n";
        break;
    case 'S':
        std::cout << "Идешь на юг\n";
        break;
    case 'E':
        std::cout << "Идешь на восток\n";
        break;
    case 'W':
        std::cout << "Идешь на запад\n";
        break;
    default:
        std::cout << "Неизвестное направление\n";
        break;
}
```

Эквивалент через if/else if:

```
if (direction == 'N') {
    ...
} else if (direction == 'S') {
    ...
} else if (...) {
    ...
} else {
    ...
}
```

switch — другая форма того же. Когда вариантов много и все они проверяют одну переменную — switch обычно читается лучше. Плюс, компилятор иногда генерирует **более быстрый код** через таблицу переходов (jump table) вместо последовательных проверок. Не всегда — это оптимизация, и её делает компилятор по своему усмотрению.

10.6.1 Ограничения switch

В switch можно проверять **только целочисленные и перечислимые** типы (int, char, enum, и т. д.). **Нельзя** — double, std::string, объекты классов. Поэтому такое **не сработает**:

```
std::string cmd;
std::cin >> cmd;

switch (cmd) {                                // ОШИБКА компиляции
    case "look": ...
    case "take": ...
}
```

Для строк используют if/else if.

Сами case-значения должны быть **constexpr-выражениями**:

```
const int MAX = 10;
constexpr int MIN = 1;

switch (x) {
    case MAX: ...    // ОК, MAX это const int с известным значением
    case MIN: ...    // ОК, constexpr
    case y:   ...    // ОШИБКА, y не известно на компиляции
}
```

10.6.2 Fall-through — главная ловушка switch

Заметили break после каждого case? Если его убрать — выполнение «просачивается» вниз к следующему case. Это называется fall-through, и оно унаследовано из C.

```
switch (x) {
    case 1:
        std::cout << "one ";
        // нет break – ВНИМАНИЕ
    case 2:
        std::cout << "two ";
        break;
    case 3:
        std::cout << "three ";
        break;
}
```

При `x == 1` напечатается `one two`! Потому что после `case 1` нет `break`, выполнение продолжается в `case 2`.

Чаще всего `fall-through` — это **баг** (программист забыл `break`). Иногда — фича, когда несколько `case` должны делать одно:

```
switch (cmd) {
    case 'h':
    case 'H':
    case '?':
        show_help();
        break;
    case 'q':
    case 'Q':
        quit();
        break;
}
```

Здесь `fall-through` сознательный — обрабатываем верхний и нижний регистр одной веткой. В C++17 для явности добавили атрибут `[[fallthrough]]`:

```
case 1:
    do_one();
    [[fallthrough]]; // явно говорим: «знаю что нет break, это специально»
case 2:
    do_two();
    break;
```

В C++11 такого нет — обходимся комментарием `// fall-through`. Современные компиляторы могут выдавать предупреждение про `fall-through` (`-Wimplicit-fallthrough`), и принимают комментарии в качестве «глушилки».

10.6.3 default

`default` — что делать, если ни один `case` не подошёл. Опционально, но **очень полезно**. Если все случаи перечислены — поставьте `default` с сообщением об ошибке или `assert`:

```
switch (state) {
    case State::Idle:    ...; break;
    case State::Active: ...; break;
    case State::Done:   ...; break;
    default:
```



```
std::cerr << "Unknown state\n";
std::abort();
}
```

Так, если в будущем добавите `State::Error`, но забудете `case` — программа упадёт с понятной ошибкой, а не «странно поведёт себя». Современные компиляторы с `-Wswitch` ругаются, если для `enum` пропущен `case`.

10.7 while — цикл с проверкой в начале

```
while (условие) {
    действия;
}
```

Сначала проверяется условие. Если истинно — выполняются действия, потом снова проверка, и так далее. Когда условие становится ложным — выход.

Если условие изначально ложно — тело не выполнится ни разу.

```
int health = 100;
while (health > 0) {
    health = health - 10;
    std::cout << "HP: " << health << "\n";
}
std::cout << "Игрок умер\n";
```

Циклы — главный источник багов. Самые частые проблемы:

Бесконечный цикл. Условие никогда не становится ложным. Например:

```
int i = 0;
while (i < 10) {
    std::cout << i << "\n";
    // забыли увеличить i — цикл вечный
}
```

Программа печатает `0 0 0 0 ...` бесконечно. Прервите через `Ctrl+C`.

Off-by-one error (ошибка на единицу). Цикл выполняется на один раз больше или меньше, чем надо. Классика — `<` vs `<=`:

```
for (int i = 0; i < 10; ++i) {    // 10 итераций: i = 0, 1, ..., 9
    ...
}
```

```
}  
for (int i = 0; i <= 10; ++i) {    // 11 итераций: i = 0, 1, ..., 10  
    ...  
}
```

Программисты привыкли к первой форме ($< n$), потому что она даёт ровно n итераций и в индексной нотации совпадает с длиной массива.

Изменение неправильной переменной. Особенно во вложенных циклах:

```
int i = 0;  
while (i < 10) {  
    int j = 0;  
    while (j < 5) {  
        std::cout << i << ", " << j;  
        i = i + 1;    // Ой! Меняем i во внутреннем цикле  
        j = j + 1;  
    }  
}
```

Внешний цикл сходит с ума, потому что i меняется внутри. Часто такие баги — результат копипасты.

10.7.1 Бесконечный цикл нарочно

Иногда хочется именно вечный цикл (внутри будет `break` по какому-то условию):

```
while (true) {  
    auto cmd = read_command();  
    if (cmd == "quit") break;  
    execute(cmd);  
}
```

Это идиома REPL (read-eval-print loop). Сервер тоже обычно крутится в `while (true)`.

Эквивалентная запись — `for (;;) :`

```
for (;;) {  
    ...  
}
```

Это тот же бесконечный цикл, just другой синтаксис. Используется реже, но встречается в чужом коде.

10.8 do-while — цикл с проверкой в конце

```
do {  
    действия;  
} while (условие);
```

В отличие от `while`, тело выполняется **минимум один раз** — сначала действие, потом проверка.

```
int answer;  
do {  
    std::cout << "Введите число 1-10: ";  
    std::cin >> answer;  
} while (answer < 1 || answer > 10);
```

Программа просит ввести число, пока не введут что-то в диапазоне. Сначала всегда спрашивает, потом проверяет.

Точка с запятой **после** `while (...)` обязательна — `do-while` это особая конструкция.

`do-while` нужен редко. Большинство случаев лучше переписываются в обычный `while`. Я в своей практике пишу его, может быть, раз в год — когда хочется именно «выполнить и проверить».

10.9 for — классический цикл

Самая частая форма цикла в C/C++:

```
for (инициализация; условие; шаг) {  
    действия;  
}
```

Эквивалент через `while`:

```
инициализация;  
while (условие) {  
    действия;  
    шаг;  
}
```

Классический пример — перебор от 0 до N-1:

```
for (int i = 0; i < 10; ++i) {  
    std::cout << i << "\n";  
}
```

Разбор по частям: - `int i = 0` — выполняется один раз, перед циклом. - `i < 10` — проверяется перед каждой итерацией. Если истина — выполняется тело. - `++i` — выполняется после каждой итерации тела. Увеличивает `i` на 1.

`++i` — это **инкремент**, увеличение на 1. Чуть короче и быстрее, чем `i = i + 1` (для встроенных типов разницы практически нет, для итераторов класса — лучше `++i`).

Есть ещё `i++` — постфиксный инкремент. На встроенных типах в условии цикла **разницы нет**. Но в общем случае: - `++i` — увеличить и вернуть **новое** значение. - `i++` — запомнить старое значение, увеличить, вернуть **старое**.

Префиксный `++i` для итераторов классов чуть быстрее (не делает копию), поэтому в `for`-циклах я пишу `++i`. Это вошло в привычку у всего C++-сообщества.

10.9.1 Все три части — опциональны

Можно опустить любую часть:

```
int i = 0;
for (; i < 10; ++i) {           // инициализации нет
    ...
}

for (int i = 0; ; ++i) {        // условия нет — бесконечный цикл
    if (...) break;
    ...
}

for (int i = 0; i < 10;) {      // шага нет — должен меняться внутри
    ...
    i = i + 2;
}

for (;;) {                     // вечный цикл
    ...
}
```

Чаще всего я пишу классическую полную форму. Опущения — редкая экзотика.

10.9.2 Несколько переменных в `for`

В части инициализации можно объявить несколько переменных:

```
for (int i = 0, j = 10; i < j; ++i, --j) {  
    std::cout << i << " " << j << "\n";  
}
```

Печатает 0 10, 1 9, 2 8, 3 7, 4 6. Останавливается, когда *i* догоняет *j*.

В шаге запятая — это **оператор запятая**, выполняющий несколько выражений по очереди. Используется редко, но иногда удобно для одновременного движения двух переменных.

10.10 Range-based for — главная новинка C++11

Классический `for` хорош, но если нужно просто пройти по всем элементам контейнера — много букв. В C++11 добавили краткую форму:

```
std::vector<int> v = {10, 20, 30, 40, 50};  
  
for (int x : v) {  
    std::cout << x << "\n";  
}
```

Читается «для каждого *x* из *v*». Без индексов, без счётчиков. Эквивалент классического:

```
for (size_t i = 0; i < v.size(); ++i) {  
    int x = v[i];  
    std::cout << x << "\n";  
}
```

Range-based `for` работает для любого, у кого есть `begin()` и `end()` (методы или функции). Это все стандартные контейнеры (`vector`, `array`, `map`, `set`, `string`), а также массивы:

```
int arr[] = {1, 2, 3, 4, 5};  
for (int x : arr) {  
    std::cout << x << "\n";  
}
```

10.10.1 По ссылке vs по значению

В записи `for (int x : v)` *x* — это **копия** элемента. Менять *x* бесполезно — оригинал не изменится:

```
std::vector<int> v = {1, 2, 3};  
for (int x : v) {  
    x = x * 2;    // меняет копию, не оригинал
```

```
}  
// v всё ещё {1, 2, 3}
```

Чтобы менять оригинал — берём по ссылке (&):

```
for (int& x : v) {  
    x = x * 2;  
}  
// v теперь {2, 4, 6}
```

Если контейнер хранит большие объекты (std::string, классы), копировать дорого — берите хотя бы константную ссылку:

```
std::vector<std::string> names = {"Alice", "Bob", "Charlie"};  
  
// Плохо: каждый name копируется (стоит выделения памяти и копирования содержимого)  
for (std::string name : names) {  
    std::cout << name << "\n";  
}  
  
// Хорошо: константная ссылка, без копий  
for (const std::string& name : names) {  
    std::cout << name << "\n";  
}
```

С auto запись становится ещё короче:

```
for (const auto& name : names) {  
    std::cout << name << "\n";  
}
```

Это идиома, которую вы будете писать постоянно. Запоминайте: **for (const auto& x : container)**.

10.10.2 Если нужен индекс — классический for

Range-based for не даёт индекс. Если он нужен (например, чтобы напечатать «элемент номер 3»):

```
for (size_t i = 0; i < v.size(); ++i) {  
    std::cout << i << ": " << v[i] << "\n";  
}
```

(`size_t` — стандартный беззнаковый тип для размеров и индексов; обычно эквивалент `unsigned long`. Можно использовать `int`, но `-Wsign-compare` тогда ругнётся; см. главу 8 — лучше `signed` для индексов всё-таки, но `size_t` — общепринятая практика для итерирования.)

В C++20 появилось `std::views::enumerate`, дающее одновременно индекс и значение. До тех пор — классический `for` с индексом.

10.11 `break` и `continue`

`break` — **выход** из ближайшего цикла или `switch`. Мы видели его в `switch`. В цикле:

```
for (int i = 0; i < 100; ++i) {
    if (found_what_we_want(i)) {
        break;    // выходим из цикла
    }
}
```

`continue` — **переход к следующей итерации** того же цикла. Текущая итерация прерывается:

```
for (int i = 0; i < 10; ++i) {
    if (i % 2 == 0) {
        continue;    // пропускаем чётные
    }
    std::cout << i << "\n";    // печатает только нечётные
}
```

Что важно: `break` и `continue` влияют **только на ближайший цикл**. Если их вызвать из вложенного цикла — выйдут только из внутреннего:

```
for (int i = 0; i < 10; ++i) {
    for (int j = 0; j < 10; ++j) {
        if (found) {
            break;    // выходит ТОЛЬКО из внутреннего for
        }
    }
    // сюда попадаем после break — продолжается внешний цикл
}
```

Это иногда раздражает — хочется выйти сразу из двух циклов. Варианты решения:

1. Флаг:

```
bool done = false;
for (int i = 0; i < 10 && !done; ++i) {
    for (int j = 0; j < 10; ++j) {
        if (found) {
            done = true;
            break;
        }
    }
}
```

Работает, но проверка `&& !done` каждую итерацию — некрасиво.

2. Выделить в функцию + return:

```
auto search = []() -> std::pair<int,int> {
    for (int i = 0; i < 10; ++i) {
        for (int j = 0; j < 10; ++j) {
            if (found) return {i, j};
        }
    }
    return {-1, -1};
};
```

Иногда самый чистый вариант. Лямбды разберём в главе 21.

3. goto. Seriously. Об этом ниже.

10.12 goto — реабилитация чёрной овцы

В каждой книге по программированию пишут «никогда не используйте goto». Это идёт из эссе Дейкстры 1968 года «Goto Considered Harmful». Эссе было верным **в контексте того времени** — программы на ассемблере и Fortran с goto превращались в спагетти, в которых ничего нельзя было понять.

В современном структурированном коде у goto остались два-три **законных** применения, и я о них расскажу.

Синтаксис goto простой:

```
goto метка;
...
```



```
метка:
    действие;
```

goto перепрыгивает на строку с меткой. Всё. Прыгать можно только внутри одной функции, нельзя пересекать инициализации переменных (компилятор не разрешит).

Применение 1: выход из вложенных циклов.

```
for (int i = 0; i < 10; ++i) {
    for (int j = 0; j < 10; ++j) {
        if (found) goto done;
    }
}
done:
    std::cout << "Found at " << i << ", " << j << "\n";
```

Чисто и без лишних флагов. В C++ этот паттерн считается приемлемым.

Применение 2: освобождение ресурсов на ошибке (C-стиль).

```
int* a = malloc(...);
int* b = malloc(...);
int* c = malloc(...);

if (something_failed) {
    goto cleanup;
}

// ... использование ...

cleanup:
    free(c);
    free(b);
    free(a);
```

В C это типичный паттерн «единственная точка очистки». В C++ для этого есть **гораздо лучшие средства** — RAII (мы разберём в главе 11), деструкторы, smart pointers. Поэтому в C++ этот паттерн почти не нужен. Но если читаете старый C-код или общаетесь с C-API — будете видеть.

Что нельзя делать с goto:

- Прыгать вперёд через инициализацию переменной (компилятор не даст).

- Прыгать в другую функцию (только внутри одной).
- Использовать как «удобный if» — `if (...) goto err;` в обычном потоке. Это создаёт путаницу.

В нашей книге `goto` встретится максимум один-два раза — в местах, где он реально лучше альтернатив.

10.13 Циклы с итераторами (предварительно)

В C++ есть концепция **итераторов** — обобщённого способа перебирать элементы. Это основа STL. Подробно — в главе 14, но для полноты картины упомяну:

```
std::vector<int> v = {10, 20, 30};

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";    // *it — это значение по итератору
}
```

`v.begin()` — итератор на первый элемент. `v.end()` — итератор «за последний» (sentinel). `++it` — перейти к следующему. `*it` — получить значение.

В C++11 такой цикл превращается в краткий range-based for, и писать его руками не надо. Но иногда классический for-с-итератором нужен — например, когда хочется иметь итератор для модификации контейнера прямо во время прохода. К этому вернёмся.

10.14 Что выучить про управляющие конструкции

Конструкция	Когда брать
if/else	Простое ветвление, 1-3 ветки.
else if-цепочка	Несколько альтернатив. Если их много и они на одну переменную — рассмотрите switch.
Тернарник ?:	В выражениях, выбор простого значения. Не больше одного на строку.
switch	Много веток на одну целочисленную/enum переменную. Не забывайте break!
while	Цикл с предусловием. Когда количество итераций неизвестно.
do-while	Цикл с пост-условием. Когда нужно гарантировать хотя бы один проход.

Конструкция	Когда брать
for (классический)	Цикл с известным числом итераций (от и до). Когда нужен индекс.
Range-based for	Перебор всех элементов контейнера. Самый частый цикл в C++11+.
break	Выйти из ближайшего цикла или switch.
continue	Перейти к следующей итерации.
goto	Выход из вложенных циклов. В редких случаях.

10.15 Пара рекомендаций

1. **Всегда фигурные скобки** `{...}` **после** `if/else/for/while`, даже если тело — одна строка.
2. **Всегда break в case** (или явный `[[fallthrough]]`/комментарий).
3. **default в switch** — даже если думаете, что не нужен. Будущие изменения добавят варианты.
4. **Range-based for с const auto&** для перебора. Запомните идиому.
5. **Включите флаги** `-Wall -Wextra -Wpedantic` — компилятор найдёт половину ловушек этой главы.

10.16 Пример: «угадай число»

Соберём что-то живое из всего изученного. Простая игра: компьютер загадывает число от 1 до 100, игрок угадывает. Программа подсказывает «больше» или «меньше».

```
#include <iostream>
#include <cstdlib>    // для rand и srand
#include <ctime>      // для time

int main() {
    // Инициализируем генератор случайных чисел текущим временем
    std::srand(static_cast<unsigned>(std::time(nullptr)));

    constexpr int min_value = 1;
    constexpr int max_value = 100;
    constexpr int max_attempts = 10;
```

```
int target = std::rand() % (max_value - min_value + 1) + min_value;

std::cout << "Угадай число от " << min_value << " до " << max_value << "\n";
std::cout << "У тебя " << max_attempts << " попыток.\n";

bool won = false;

for (int attempt = 1; attempt <= max_attempts; ++attempt) {
    std::cout << "\nПопытка " << attempt << "/" << max_attempts << ": ";

    int guess;
    std::cin >> guess;

    if (guess < min_value || guess > max_value) {
        std::cout << "Число вне диапазона. Попробуй ещё.\n";
        continue;
    }

    if (guess == target) {
        std::cout << "Угадал! Это " << target << ".\n";
        won = true;
        break;
    } else if (guess < target) {
        std::cout << "Загаданное больше.\n";
    } else {
        std::cout << "Загаданное меньше.\n";
    }
}

if (!won) {
    std::cout << "\nПопытки кончились. Это было " << target << ".\n";
}

return 0;
}
```

Соберите:

```
$ g++ -std=c++11 -Wall -Wextra guess.cpp -o guess
$ ./guess
```

Что тут использовано: - `constexpr` для констант (глава 8). - `static_cast` для явной конверсии (глава 8). - `for` с условием и шагом. - `if/else if/else` для трёх веток сравнения. - `continue` для пропуска неверного ввода. - `break` для выхода при выигрыше. - Логическое НЕ (`!won`).

Поиграйте, попробуйте сломать (введите букву вместо числа — что будет?), потом исправьте.

10.17 Маленькое упражнение

1. Напишите программу, которая печатает таблицу умножения 10×10 :

```
1  2  3 ...
2  4  6 ...
3  6  9 ...
```

Подсказка: вложенный `for`.

2. Напишите программу, которая читает числа из `stdin` до нуля и считает сумму:

```
$ ./sum
5
3
7
0
Сумма: 15
```

Подсказка: `while (true) { ... if (x == 0) break; }`.

3. Напишите программу, которая находит первое простое число больше `N` (`N` — ввод пользователя).
4. Напишите программу-калькулятор: на вход — два числа и операция (`+` `-` `*` `/`), на выход — результат. Используйте `switch` по операции.
5. (Сложнее) Напишите программу, которая для каждого числа от 1 до 100:

- если делится на 3 — печатает «Fizz»;
- если на 5 — «Buzz»;
- если и на 3, и на 5 — «FizzBuzz»;
- иначе — само число.

Это классическая задача FizzBuzz на собеседованиях.

Если не получается с первого раза — нормально. Если получается с третьего — отлично. Если получилось всё с первого — впечатляет. В любом случае запускайте и проверяйте каждую программу.

10.18 Что дальше

Глава 10 — про функции. Объявление и определение, передача параметров (по значению, по ссылке, по указателю), перегрузка, значения по умолчанию, `inline`, рекурсия и `stack overflow`. После этой главы у вас будет 90% инструментов для написания простых программ — без классов, но уже с собственными модулями. А ещё через две главы (память и RAII) подойдём к тому, чтобы стартовать первый проект.

Глава 11

Глава 10. Функции

С функциями вы уже сталкивались — `main`, `std::cout`, и пара случайных штук. Эта глава — про то, как функции писать самим. Это огромная тема, потому что в C++ передача аргументов в функцию устроена сильно иначе, чем в Python или Java. У вас будет три способа передачи (по значению, по ссылке, по указателю), и каждый со своим смыслом. Я буду долго на этом останавливаться — это центральная вещь языка, и без её понимания дальше будет хаос.

11.1 Зачем нужны функции

В главе 4 мы уже обсудили зачем — переиспользование, декомпозиция. Повторю кратко: функция позволяет один раз написать какой-то кусок логики и потом вызывать его сколько угодно раз. Это бережёт код от копи-пасты, делает структуру программы понятной и упрощает поиск ошибок (если ошибка в функции — правка одна, не пятнадцать).

Когда программа разрастается до тысяч строк — без функций жить нельзя. Хорошее правило: **если кусок кода вы скопировали два раза — пора вынести в функцию.**

11.2 Анатомия функции

Вот простейшая функция:

```
int square(int x) {  
    return x * x;  
}
```

Разбираем по частям.

- **int** в самом начале — **тип возвращаемого значения** (return type). Функция отдаёт обратно `int`. Если функция ничего не возвращает — пишут `void` (про это ниже).

- **square** — **имя функции**. Те же правила, что для переменных (буквы, цифры, подчёркивание; не начинается с цифры; не зарезервированное слово).
- **(int x)** — **список параметров**. У нашей функции один параметр типа `int` по имени `x`. Параметров может быть много, или ноль.
- **{ ... }** — **тело функции**. Код, который выполняется при вызове.
- **return x * x;** — возвращает значение из функции. Управление сразу переходит к месту вызова.

Вызов:

```
int result = square(5);    // result = 25
```

Программа доходит до строки `int result = square(5);`, перепрыгивает в тело `square`, выполняет его (умножает 5 на 5, возвращает 25), потом возвращается обратно, и `result` получает значение 25.

11.2.1 Несколько параметров

```
int max(int a, int b) {  
    if (a > b) return a;  
    else      return b;  
}
```

Можно иметь два, три, четыре параметра. У каждого свой тип и своё имя:

```
double distance(int x1, int y1, int x2, int y2) {  
    double dx = x2 - x1;  
    double dy = y2 - y1;  
    return std::sqrt(dx * dx + dy * dy);  
}
```

(Здесь использован `std::sqrt` из `<cmath>` — квадратный корень.)

11.2.2 void — функция без возврата

Если функция не должна ничего возвращать — указывайте `void`:

```
void print_hello() {  
    std::cout << "Hello!\n";  
}
```

Вызов:


```
print_hello();    // ничего не присваиваем
```

В void-функции можно писать `return;` (без значения) — это выход. Если функция дошла до конца тела без `return` — она тоже завершается, всё нормально:

```
void greet(const std::string& name) {  
    if (name.empty()) {  
        return;    // выход досрочно  
    }  
    std::cout << "Hello, " << name << "!\n";  
}
```

`return` в void-функции с значением — ошибка компиляции.

11.2.3 Функция без параметров

Если параметров нет — пишут пустые скобки:

```
int get_random() {  
    return std::rand();  
}
```

В C++ `()` означает «параметров нет», в отличие от C, где `()` означает «параметры неизвестны»; в C для «нет параметров» пишут `(void)`. В C++ обе формы работают, но `(...)` без `void` — стандарт.

11.3 Объявление и определение

В главе 7 я мельком говорил: **объявление** функции — это обещание её существования, **определение** — это собственно её код.

Объявление:

```
int square(int x);
```

(Точка с запятой в конце, тела нет.)

Определение:

```
int square(int x) {  
    return x * x;  
}
```

(Полное, с телом.)

Зачем разделять? Чтобы можно было использовать функцию **до того, как она написана**. Компилятор работает сверху вниз: дойдя до строки `square(5)`, он должен уже знать, что такое `square`. Если функция определена ниже — компилятор не найдёт.

Поэтому обычно файл выглядит так:

```
#include <iostream>

// объявления вверху
int square(int x);
int max(int a, int b);

// определения ниже
int main() {
    std::cout << square(5) << "\n";
    std::cout << max(3, 7) << "\n";
    return 0;
}

int square(int x) {
    return x * x;
}

int max(int a, int b) {
    return a > b ? a : b;
}
```

Альтернатива — определить функции **до** `main`:

```
int square(int x) {
    return x * x;
}

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    std::cout << square(5) << "\n";
    std::cout << max(3, 7) << "\n";
}
```

```
    return 0;  
}
```

Если функция определена до использования — отдельное объявление не нужно. Каждое определение **само по себе** служит объявлением.

11.3.1 Разделение между файлами

В программе из нескольких файлов схема такая. Допустим, у нас функция `square` нужна и в `main.cpp`, и в `tests.cpp`.

`math.h`:

```
#ifndef MATH_H  
#define MATH_H  
  
int square(int x);    // объявление  
  
#endif
```

`math.cpp`:

```
#include "math.h"  
  
int square(int x) {    // определение  
    return x * x;  
}
```

`main.cpp`:

```
#include "math.h"  
  
int main() {  
    return square(5);  
}
```

`tests.cpp`:

```
#include "math.h"  
  
void run_tests() {  
    if (square(3) != 9) ...  
}
```

Когда `main.cpp` и `tests.cpp` компилируются, они оба видят объявление `square` (через `#include "math.h"`). Компилятор не возражает — он не требует тела на этапе компиляции. Тело берётся при **линковке**, из `math.cpp`. Линкер находит реальную `square` в `math.o` и подставляет.

Это разделение «объявление в `.h`, определение в `.cpp`» — стандартный паттерн в C++. Заголовочный файл — публичный интерфейс модуля. Исходный файл — реализация.

11.4 Передача параметров: три способа

Вот теперь главное место главы. Когда мы пишем:

```
int square(int x) {  
    return x * x;  
}  
  
int main() {  
    int n = 5;  
    int r = square(n);  
}
```

Что происходит с `n` при вызове `square(n)`? Передаётся **сама переменная** `n`? Или её копия? Или ссылка на неё?

В C++ есть **три** способа передачи параметров. Каждый со своим синтаксисом, своим смыслом, своей ценой.

11.4.1 Способ 1: по значению

Самый простой и самый частый. Параметр — обычная переменная.

```
void increment(int x) {  
    x = x + 1;  
    std::cout << "Внутри функции x = " << x << "\n";  
}  
  
int main() {  
    int a = 5;  
    increment(a);  
    std::cout << "После вызова a = " << a << "\n";  
}
```

Вывод:

Внутри функции $x = 6$

После вызова $a = 5$

а не изменилась! Хотя мы написали $x = x + 1$ внутри функции — менялось значение x , а x — это **копия** значения a , отдельная переменная. Внутри функции получили копию, поработали с ней, копия исчезла при возврате.

Это **передача по значению** (pass by value). Главные свойства:

- Параметр — это **новая переменная**, инициализированная значением аргумента.
- Изменения параметра внутри функции **не видны снаружи**.
- Происходит **копирование**. Для `int` — копия одного слова, мгновенно. Для большого `std::vector<std::string>` — копия всего содержимого, может быть мегабайтами.

Передача по значению — основная по умолчанию для «маленьких» типов: `int`, `double`, `bool`, `char`, маленькие структуры.

11.4.2 Способ 2: по ссылке

Если хотим, чтобы функция меняла оригинал — берём параметр **по ссылке**:

```
void increment(int& x) {           // обратите внимание: int& x, а не int x
    x = x + 1;
}

int main() {
    int a = 5;
    increment(a);
    std::cout << "a = " << a << "\n";    // a = 6 !
}
```

Знак `&` после типа параметра — **это ссылка**. `int& x` означает « x — это другое имя для переменной, которая будет передана при вызове». Внутри функции x и снаружи a — это **одна и та же переменная**, доступная под двумя именами. Никаких копий.

Когда вы пишете $x = x + 1$ — меняется именно a .

Ссылка — это **псевдоним**. Аналогии: - В русском языке у Петра Ильича Чайковского можно сказать «Чайковский» или «Петя» — это одно лицо. Ссылка — это «Петя», обычное имя — «Чайковский». - В файловой системе бывают символические ссылки — два пути, ведущие к одному файлу.

11.4.3 Ключевые свойства ссылок

1. **Ссылка должна быть инициализирована при создании.** Нельзя `int& r;` без значения — ошибка.

```
int x = 5;
int& r = x;    // ОК: r это псевдоним x
int& r2;       // ОШИБКА: ссылка должна быть инициализирована
```

2. **Ссылка нельзя «перенаправить» на другую переменную.** После создания она навсегда указывает на ту же.

```
int x = 5;
int y = 10;
int& r = x;    // r это x
r = y;        // НЕ перенаправление! Это присваивание: x теперь равен 10
```

В отличие от указателей (ниже) — ссылка после создания «приклеена» намертво.

3. **К ссылке относятся как к самой переменной.** `r * 2` — это `x * 2`. `++r` — это `++x`. И так далее.

11.4.4 По ссылке для эффективности

Главная причина передавать **большие** объекты по ссылке — **не делать копию**:

```
void print(const std::vector<int>& v) {    // const ссылка — нет копии, нет модификации
    for (int x : v) {
        std::cout << x << "\n";
    }
}
```

`const std::vector<int>&` — это «константная ссылка на вектор». Без копирования (вектор может быть мегабайтным!), и при этом гарантия: внутри функции его не изменят.

Правило: если параметр — большой тип (`std::string`, `std::vector`, класс), и функция его не меняет — передавайте `const T&`. Не `T` (будет копия), не `T&` (без `const` — функция может «случайно» поменять).

Маленькие типы (`int`, `double`, `bool`) — по значению. Копия дешёвая, и компилятор всё равно положит копию в регистр.

11.4.5 Способ 3: по указателю

Указатель — это переменная, в которой лежит **адрес** другой переменной. Это уже не псевдоним, а явная «координата» другой переменной в памяти.

```
void increment(int* p) {           // int* — указатель на int
    *p = *p + 1;                   // *p — значение по адресу p
}

int main() {
    int a = 5;
    increment(&a);                 // &a — адрес a
    std::cout << "a = " << a << "\n"; // a = 6
}
```

Тут много нового. Разбираем.

- **int*** в типе параметра — это «указатель на int». То есть «адрес какой-то ячейки памяти, в которой лежит int».
- **&a** при вызове — оператор «**взять адрес**». Возвращает адрес переменной a. Тип — int*.
- ***p** внутри функции — оператор «**разыменовать**». Возвращает значение по адресу p. То есть a.
- ***p = *p + 1** — взять значение по адресу p, прибавить 1, и положить обратно по тому же адресу. То есть меняем a.

11.4.6 Указатель vs ссылка

Указатели и ссылки делают **похожую** работу — позволяют функции работать с оригиналом, а не с копией. Отличия:

1. **Указатель — это число (адрес)**. Ссылка — псевдоним. Указатель занимает 8 байт (на 64-битной системе), ссылка — формально ноль (компилятор её обычно реализует как тот же указатель, но это деталь).
2. **Указатель может быть nullptr** — «никуда не указывающий». Ссылку нельзя инициализировать «ничем».
3. **Указатель можно перенаправлять** — `p = &b;` меняет, на что указывает. Ссылку — нельзя.
4. Для вызова с указателем нужен **&** у вызывающего и ***** для разыменования — синтаксис заметнее. Со ссылкой — вызов выглядит как обычный.

5. **Указатели — наследие С.** Они везде в С-коде и в системных API. Ссылки — изобретение C++, в С их нет.

11.4.7 Когда что использовать

Если **обязательно** есть что передать (значение точно есть) и функция должна его менять или эффективно прочитать — **ссылка**. Кратко, безопасно.

Если параметр **может отсутствовать** (тогда передаём `nullptr`) или нужно по С-API — **указатель**.

```
// "Дай мне строку, я её обработаю":
void process(const std::string& s);

// "Дай мне строку, ИЛИ ничего":
void process(const std::string* s) {
    if (s == nullptr) {
        std::cout << "Нет строки\n";
    } else {
        // обработка
    }
}
```

В современном C++17+ для «опционального» параметра обычно используют `std::optional`, но до этого далеко. На C++11 — либо указатель, либо специальные значения.

11.4.8 Сводная таблица

Способ	Синтаксис	Можно менять оригинал?	Копирование?
По значению	<code>void f(int x)</code>	Нет	Да
По ссылке	<code>void f(int& x)</code>	Да	Нет
По const-ссылке	<code>void f(const int& x)</code>	Нет	Нет
По указателю	<code>void f(int* p)</code>	Да	Нет
По указателю на const	<code>void f(const int* p)</code>	Нет	Нет

11.4.9 Правило передачи параметров

Я в начале книги обещал, что буду давать «правила без иффов». Вот оно:

1. **Маленькие типы** (`int`, `double`, `bool`, `char`, маленькие структуры до ~16 байт) — **по значению**.

2. **Большие типы** для чтения — `const T&` (по const-ссылке).
3. **Большие типы для модификации** — `T&` (по ссылке). Думайте дважды; часто лучше принять копию и вернуть результат.
4. **Указатели** — только когда нужен `nullptr` или для C-API.

Этим правилом обходитесь в 95% случаев.

11.5 Возврат из функции

С возвратом обычно проще, чем с параметрами. По умолчанию — возврат по значению, копия:

```
std::string make_greeting(const std::string& name) {  
    return "Hello, " + name + "!";  
}  
  
int main() {  
    std::string g = make_greeting("Alice");  
}
```

Здесь возвращается копия строки. На больших объектах это могло бы быть дорого, но **компилятор почти всегда оптимизирует** через RVO/NRVO (return value optimization) — копии не делается, объект сразу строится в нужном месте. Об этом подробно в главе 19 (move semantics).

Можно возвращать ссылку:

```
int& get_element(std::vector<int>& v, int index) {  
    return v[index];  
}  
  
int main() {  
    std::vector<int> nums = {1, 2, 3};  
    get_element(nums, 1) = 99;    // меняет nums[1] на 99  
}
```

Опасный приём! Если вернуть ссылку на **локальную переменную** функции — после возврата эта переменная исчезает, ссылка становится «висячей» (dangling), и обращение к ней — UB:

```
int& bad() {  
    int local = 42;  
    return local;    // ОПАСНО: local исчезает, ссылка указывает в никуда  
}
```

```
int x = bad();    // x получает мусор или программа крашится
```

Современные компиляторы обычно ругаются на такое (`-Wreturn-local-addr`), но не всегда ловят. Поэтому правило: **никогда не возвращайте ссылку или указатель на локальную переменную**.

Возвращать ссылку безопасно, когда: - Вы возвращаете ссылку на параметр. - Вы возвращаете ссылку на `static`-переменную. - Вы возвращаете ссылку на член класса (об этом в главе 16).

11.5.1 Несколько возвращаемых значений

В C++ функция возвращает **только одно** значение. Если хочется два — есть несколько вариантов.

1. Через ссылочные параметры:

```
void divmod(int a, int b, int& quotient, int& remainder) {
    quotient = a / b;
    remainder = a % b;
}

int main() {
    int q, r;
    divmod(17, 5, q, r);    // q = 3, r = 2
}
```

2. Через `std::pair`:

```
#include <utility>

std::pair<int, int> divmod(int a, int b) {
    return std::make_pair(a / b, a % b);
}

int main() {
    auto result = divmod(17, 5);
    std::cout << result.first << " " << result.second;
}
```

`std::pair<int, int>` — стандартная пара. `result.first` — первое значение, `result.second` — второе.

3. Через `std::tuple` (C++11) для трёх и больше:

```
#include <tuple>

std::tuple<int, int, int> get_rgb() {
    return std::make_tuple(255, 128, 64);
}

int main() {
    int r, g, b;
    std::tie(r, g, b) = get_rgb();
}
```

В C++17 появились structured bindings — `auto [q, r] = divmod(17, 5);` — но это не наша база.

4. Через свою структуру — часто чище всех:

```
struct DivResult {
    int quotient;
    int remainder;
};

DivResult divmod(int a, int b) {
    return {a / b, a % b};
}

int main() {
    DivResult r = divmod(17, 5);
    std::cout << r.quotient;
}
```

Я в книге буду использовать в основном именованные структуры — они яснее.

11.6 Перегрузка функций

В C++ можно иметь **несколько функций с одинаковым именем**, если у них разные параметры:

```
int max(int a, int b) {
    return a > b ? a : b;
}
```

```
double max(double a, double b) {
    return a > b ? a : b;
}

int max(int a, int b, int c) {
    return max(max(a, b), c);
}
```

Три функции с именем `max`. Компилятор по типам аргументов выбирает нужную:

```
max(3, 5);           // int max(int, int)
max(3.14, 2.71);     // double max(double, double)
max(1, 2, 3);        // int max(int, int, int)
```

Это **перегрузка** (overloading). Помните манглинг имён из главы 7? Именно для этого компилятор кодирует типы в имена символов — чтобы при линковке три функции `max` были разными `_Z3maxii`, `_Z3maxdd`, `_Z3maxiii`.

Перегрузка различает функции по: - Количеству параметров. - Типам параметров. - `const`-у на параметрах (но обычно не имеет смысла).

Перегрузка **не** различает функции по: - Типу возвращаемого значения. **Это важно!** Нельзя `int f(int); double f(int);` — компилятор не поймёт, какую из них вы хотите.

Перегрузка хороша, когда логически функции делают «одно и то же» с разными типами. Например, `max`, `min`, `print`, `to_string`. Плоха, когда вы пишете перегрузку, чтобы скрыть разную логику под одним именем — путает читателя.

11.7 Параметры по умолчанию

Можно задать значение **по умолчанию** для параметра:

```
void greet(const std::string& name, const std::string& greeting = "Hello") {
    std::cout << greeting << ", " << name << "!\n";
}

int main() {
    greet("Alice");           // Hello, Alice!
    greet("Bob", "Welcome");  // Welcome, Bob!
}
```

Параметры по умолчанию указываются **с конца**. Нельзя `void f(int a = 5, int b)` — компилятор не поймёт, как вызывать `f(7)` (это `b`? или `a`?).

Если функция объявлена и определена отдельно — параметры по умолчанию **только в объявлении**, не в определении:

```
// math.h
int sum(int a, int b, int c = 0);

// math.cpp
int sum(int a, int b, int c) {    // здесь = 0 НЕ повторять
    return a + b + c;
}
```

Параметры по умолчанию хороши, когда у функции «обычный» случай и редкие особые. Например, в большинстве вызовов вы хотите стандартное приветствие — `greet("Bob")` короче, чем `greet("Bob", "Hello")`. Плохо, когда параметров много и непонятно, какие по умолчанию — лучше отдельные функции или структура с параметрами.

11.8 inline

Когда функция очень короткая (одна-две строки), у вызова есть **накладные расходы**: положить аргументы в регистры/на стек, перепрыгнуть, выполнить, вернуться. Для крохотной функции это может быть дольше самого тела.

Решение — **inline-функция**. Это намёк компилятору: «не делай вызов, подставь тело прямо в место вызова».

```
inline int square(int x) {
    return x * x;
}

int main() {
    int r = square(5);
    // компилятор подставит это как:
    // int r = 5 * 5;
}
```

Это называется **инлайнинг** (inlining). После него — никакого вызова функции, просто выражение.

Тонкость: `inline` в современном C++ — **слабая** рекомендация. Компилятор сам решает, инлайнить или нет, и без `inline` тоже может. Но `inline` важен по другой причине — **он разрешает определение в заголовочном файле**:

```
// math.h
#ifndef MATH_H
#define MATH_H

inline int square(int x) {
    return x * x;
}

#endif
```

Без `inline` это привело бы к ошибке `multiple definition` при линковке — потому что заголовок включается в несколько `.cpp`, и компилируется в каждый. С `inline` линкер знает: «это определение, оно может быть в нескольких файлах одинаковое, выбери одно».

В простых случаях — пишите функцию в `.cpp`, не в `.h`. Если выяснилось, что для шаблона или производительности нужно в `.h` — добавьте `inline`.

11.9 Рекурсия

Функция может вызывать **сама себя**. Это **рекурсия**:

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

`factorial(5)` вызывает `factorial(4)`, который вызывает `factorial(3)`, который вызывает `factorial(2)`, который вызывает `factorial(1)`. Последний возвращает 1, дальше — $2 * 1$, $3 * 2$, $4 * 3$, $5 * 24 = 120$.

Главное в рекурсии — **base case** (базовый случай). Условие выхода, при котором функция возвращается без рекурсивного вызова. У нас это `if (n <= 1) return 1;`. Без `base case` рекурсия бесконечная.

Каждый рекурсивный вызов кладёт на стек новый кадр (как мы обсуждали в главе 7). Размер стека ограничен (обычно 8 MB на Linux/macOS, 1 MB на Windows). Когда рекурсия слишком глубокая — **stack overflow**, программа падает.

```
int infinite_recursion(int x) {  
    return infinite_recursion(x);    // никогда не остановится  
}
```

На моей машине это падает где-то на 200 000-й итерации.

Рекурсия — мощный инструмент. Многие задачи естественно рекурсивные: - Обход дерева: посетить корень, потом рекурсивно обход левого поддерева, потом правого. - Алгоритм поиска в файловой системе: открыть папку, для каждого элемента — если файл, обработать; если папка, рекурсивно зайти. - Алгоритмы делящие задачу пополам — quicksort, mergesort, binary search.

Но рекурсии не везде стоит верить. Иногда итеративная версия (через цикл) быстрее и не рискует переполнить стек:

```
int factorial(int n) {  
    int result = 1;  
    for (int i = 2; i <= n; ++i) {  
        result *= i;  
    }  
    return result;  
}
```

Это эквивалентно рекурсивной версии, но без стека. Для факториала такая форма обычно лучше.

11.9.1 Хвостовая рекурсия

Один особый вид рекурсии — **хвостовая**: рекурсивный вызов — последняя операция в функции. Например:

```
int sum_recursive(int n, int acc) {  
    if (n == 0) return acc;  
    return sum_recursive(n - 1, acc + n);  
}  
  
// sum_recursive(100, 0) даёт сумму 1+2+...+100 = 5050
```

В хвостовой рекурсии нет «работы» после рекурсивного вызова. Поэтому теоретически компилятор может заменить такой вызов прыжком — без расхода стека. Это называется **tail call optimization (TCO)**. C++-компиляторы её обычно делают (с включёнными оптимизациями), но не гарантируют. Языки типа Scheme, Haskell — гарантируют. В C++ для гарантированной безстековой рекурсии нужно переписать в цикл.

11.10 Forward declaration

Иногда две функции должны вызывать друг друга:

```
bool is_even(int n);    // forward declaration

bool is_odd(int n) {
    if (n == 0) return false;
    return is_even(n - 1);
}

bool is_even(int n) {
    if (n == 0) return true;
    return is_odd(n - 1);
}
```

`is_odd` использует `is_even`, который определён ниже. Без объявления вверху компилятор бы выдал ошибку «`is_even not declared`». Объявление-«обещание» снимает проблему.

(Алгоритм этот — учебный, в реальной жизни `n % 2 == 0` куда проще. Зато он показывает паттерн взаимной рекурсии.)

11.11 main — особая функция

`main` — точка входа в программу. Особенности:

- **Возвращает `int`.** Это код выхода: 0 — успех, ненулевой — ошибка.
- **Опускать `return 0`;** разрешено — компилятор подставит сам. Я предпочитаю писать явно.
- **Может принимать аргументы командной строки:**

```
int main(int argc, char* argv[]) {
    std::cout << "Количество аргументов: " << argc << "\n";
    for (int i = 0; i < argc; ++i) {
        std::cout << "  argv[" << i << "] = " << argv[i] << "\n";
    }
}
```



```
    }  
    return 0;  
}
```

`argc` (argument count) — сколько аргументов передано (включая имя программы). `argv` (argument vector) — массив строк (на самом деле — массив `char*`-указателей).

```
$ ./program hello world 42
```

Количество аргументов: 4

```
    argv[0] = ./program  
    argv[1] = hello  
    argv[2] = world  
    argv[3] = 42
```

Мы будем это использовать в проектах — например, `./chat-server --port 9001` или `./grep "pattern" file.txt`.

11.12 Лямбды — короткое упоминание

В C++11 появились **лямбды** — функции, объявляемые «прямо на месте»:

```
auto add = [](int a, int b) { return a + b; };  
std::cout << add(3, 4);    // 7
```

Это огромная тема, мы её разберём в главе 21 на проекте RPG. Сейчас просто знайте — такая штука есть.

11.13 Указатели на функции

Можно сохранить функцию в переменную и вызвать через неё:

```
int add(int a, int b) { return a + b; }  
int sub(int a, int b) { return a - b; }  
  
int main() {  
    int (*op)(int, int);    // указатель на функцию, принимающую (int, int) и возвращающую int  
    op = add;  
    std::cout << op(5, 3) << "\n";    // 8  
    op = sub;  
    std::cout << op(5, 3) << "\n";    // 2  
}
```

Синтаксис уродливый. Обычно вместо него используют `std::function` (про неё в главе 21) или `auto`:

```
auto op = add;    // тип выводится
op(5, 3);
```

Указатели на функции редко нужны в современном C++. Но они есть, и в чужом коде встречаются.

11.14 Чистые функции и побочные эффекты

Функция **чистая** (pure), если: - Её результат зависит **только от параметров**. - Она **не меняет** ничего снаружи (никаких глобалов, никакого ввода-вывода, ничего).

```
int square(int x) { return x * x; }    // чистая
```

Чистая, потому что: возвращает $x*x$ (зависит только от x), ничего не меняет.

Не чистая:

```
int counter = 0;

int increment_and_return() {
    counter = counter + 1;    // меняет глобал
    return counter;
}
```

Вызов два раза подряд даёт разные результаты — функция «помнит» состояние.

Чистые функции хороши: - **Тестируются легко** — нет внешних зависимостей. - **Параллелятся** — нет общего состояния, безопасно запускать в разных потоках. - **Кэшируются** — если результат уже посчитан, можно его запомнить. - **Читаются** — поведение полностью описано сигнатурой.

В C++ принудить функцию быть чистой нельзя (нет такого ключевого слова), но писать **по возможности чисто** — хороший стиль. Особенно для «вычислительных» функций. С побочными эффектами лучше работать в маленьких островках; остальной код пусть будет чистым.

11.15 Большой пример: банковская симуляция

Соберём всё в осмысленную программу. Сделаем подобие банка: счета и операции над ними.

```
#include <iostream>
#include <string>

struct Account {
    std::string owner;
    double balance;
};

// Чистая функция: вычисляет процент
double calculate_interest(double balance, double rate) {
    return balance * rate;
}

// Изменяет счёт (передача по ссылке)
void deposit(Account& account, double amount) {
    account.balance = account.balance + amount;
}

// Возвращает true если перевод прошёл, false если денег не хватило
bool transfer(Account& from, Account& to, double amount) {
    if (from.balance < amount) {
        return false;
    }
    from.balance = from.balance - amount;
    to.balance = to.balance + amount;
    return true;
}

// Только читает (const-ссылка)
void print(const Account& account) {
    std::cout << "Счёт " << account.owner << ": " << account.balance << "\n";
}

// Перегрузки – для разных типов суммы
void print(const Account& account, double amount) {
    std::cout << "Счёт " << account.owner << ": " << account.balance
        << " (сумма операции: " << amount << ")\n";
}
```

```
int main() {
    Account alice{"Alice", 1000.0};
    Account bob{"Bob", 500.0};

    print(alice);
    print(bob);

    std::cout << "\nДепозит 200 на счёт Alice\n";
    deposit(alice, 200);
    print(alice);

    std::cout << "\nПеревод 300 от Alice к Bob\n";
    if (transfer(alice, bob, 300)) {
        std::cout << "OK\n";
    } else {
        std::cout << "Недостаточно средств\n";
    }
    print(alice);
    print(bob);

    std::cout << "\nПеревод 999999 от Alice к Bob\n";
    if (transfer(alice, bob, 999999)) {
        std::cout << "OK\n";
    } else {
        std::cout << "Недостаточно средств\n";
    }
    print(alice);

    constexpr double annual_rate = 0.05;
    double interest = calculate_interest(alice.balance, annual_rate);
    std::cout << "\nГодовые Alice по ставке " << annual_rate
                << ": " << interest << "\n";

    return 0;
}
```

Соберите:

```
$ g++ -std=c++11 -Wall -Wextra bank.cpp -o bank
$ ./bank
```

Тут использовано: - `struct` — для определения типа `Account` (узнаем подробнее в главе 16). - Передача по значению (`double amount`). - Передача по ссылке (`Account& from`). - Передача по const-ссылке (`const Account& account`). - Перегрузка (две функции `print`). - Чистая функция (`calculate_interest`). - Возврат `bool` для индикации успеха.

11.16 Маленькое упражнение

1. Напишите функцию `int gcd(int a, int b)` — наибольший общий делитель. Алгоритм Евклида: `gcd(a, 0) = a`, `gcd(a, b) = gcd(b, a % b)`. Рекурсивно.
2. Перепишите `gcd` итеративно (без рекурсии).
3. Напишите функцию `void swap(int& a, int& b)` — обмен значениями. Проверьте, что работает.
4. Напишите функцию `int sum(int n)`, которая возвращает $1 + 2 + \dots + n$. Сделайте две версии: через цикл и через формулу $n * (n + 1) / 2$. Сравните: какая быстрее? Какая понятнее?
5. Напишите функцию `bool is_prime(int n)` — простое ли число. Подсказка: проверять делимость до `sqrt(n)`.
6. Напишите перегрузку `print` для `int`, `double`, `std::string`. Каждая выводит свой тип с подписью «integer: X», «double: Y», «string: Z».
7. Напишите функцию `int read_int()`, которая запрашивает у пользователя число с проверкой ввода (если не число — спросить снова).

11.17 Что дальше

Глава 11 — про **память**: что лежит на стеке, что на куче, `new` и `delete`, утечки памяти, и главный принцип C++ — RAII (Resource Acquisition Is Initialization). После этой главы вы поймёте, **почему** в C++ всё так сложно с указателями и ссылками, и **зачем** придумали `smart pointers` (про них уже на проекте RPG, глава 18). Это самая концептуально тяжёлая глава Части I, но и самая полезная. Готовьтесь.

Глава 12

Глава 11. Память — стек, куча, RAII

Это самая важная глава Части I. После неё вы поймёте, почему C++ устроен именно так — со ссылками, указателями, конструкторами, деструкторами. Поймёте, откуда берутся «утечки памяти», «висячие указатели», «двойное освобождение» — и как современный C++ их предотвращает. Без этого знания дальше можно писать только маленькие игрушки; на больших программах вы будете наткаться на загадочные крэши и недоумевать.

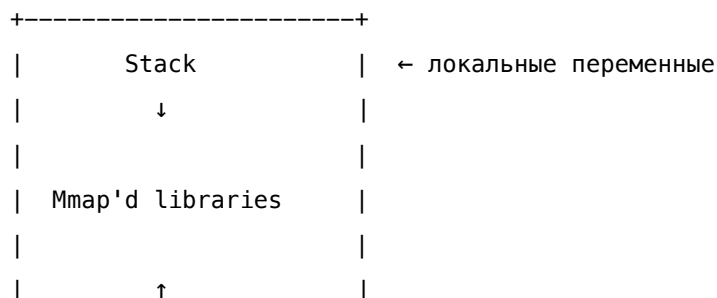
Главное на эту главу: **в C++ программист сам отвечает за память**. В Python, Java, JavaScript есть сборщик мусора — он сам находит ненужные объекты и удаляет. В C++ — нет. Если вы взяли память — вы её и возвращаете. Если забыли — утечка. Если вернули дважды — крэш. Если используете после возврата — UB.

Этой ручной работой можно было бы заняться лично, но за десятилетия C++ выработал паттерн **RAII**, который перекладывает работу на компилятор. RAII — главное изобретение C++, отличающее его от C. Об этом и поговорим.

12.1 Освежим картинку из главы 7

В главе 7 я показывал layout памяти процесса:

Высокие адреса



	Heap		← динамически выделяемая
+-----+			
	.bss		← глобальные неинициализированные
	.data		← глобальные инициализированные
	.rodata		← константы
	.text		← код
+-----+			

Низкие адреса

Из всех областей в этой главе нас интересуют две — **стек** и **куча**. Глобальные данные (.data, .bss) — это редкость в хорошем C++-коде, серьёзный разговор о них не нужен. Код (.text, .rodata) — read-only, программа в них не пишет.

Стек и куча — два главных места жизни данных. Различия в одной фразе: **стек** — **автоматически**, **куча** — **вручную**.

12.2 Стек: подробности

Каждая функция при вызове **получает кусок стека** — называется **stack frame** (кадр стека). В этом кадре лежат: - Параметры функции. - Локальные переменные. - Адрес возврата (куда прыгать после return). - Сохранённые регистры (если нужно).

Когда функция возвращается — её кадр **исчезает**. Локальные переменные, которые в нём лежали, перестают существовать. Если у переменной был деструктор (об этом ниже) — он вызывается прямо в момент исчезновения кадра.

Пример:

```
void foo() {
    int x = 5;
    int y = 10;
    // x и y лежат в кадре foo
    // адреса близкие: &x и &y отличаются на несколько байт
}
// после возврата кадр исчезает, x и y больше нет

int main() {
    foo();    // создаётся кадр foo, потом исчезает
    foo();    // снова создаётся новый кадр на тех же адресах, потом исчезает
}
```

Если бы внутри `foo` была переменная, которую мы хотели использовать после `foo` — её бы не было. Кадр уничтожается.

12.2.1 Стек растёт «вниз»

По соглашению почти всех современных архитектур стек **растёт от высоких адресов к низким**. Когда вы вызываете функцию, указатель стека (`rsp` на x86-64, `sp` на ARM64) **уменьшается** на размер кадра. Когда функция возвращается — увеличивается обратно.

Это особенность исторически такая. С точки зрения программы это неважно — стек просто «есть».

12.2.2 Стек ограничен

Размер стека по умолчанию: - **Linux**: 8 MB (можно изменить через `ulimit -s`). - **macOS**: 8 MB для главного потока, 512 KB для дополнительных. - **Windows**: 1 MB.

Если кадры функций займут больше — **stack overflow**, программа упадёт. Это бывает в двух случаях.

1. Слишком глубокая рекурсия (про это в главе 10).

```
int recurse(int n) {  
    return recurse(n + 1);    // бесконечная рекурсия  
}  
  
// упадёт где-то на 100–200 тысячах вызовов
```

2. Огромные локальные переменные.

```
void bad() {  
    int big_array[10'000'000];    // 40 MB на стеке!  
    // ...  
}
```

40 MB не помещается ни в 8 MB, ни в 1 MB. Программа упадёт ещё до выполнения тела функции. Решение — переместить такой массив в **кучу** (об этом ниже).

12.2.3 Преимущества стека

- **Бесплатное создание и уничтожение.** Никаких системных вызовов, никакого поиска свободного места. Просто перемещение указателя стека на N байт. Это **одна инструкция процессора**.
- **Автоматическое управление.** Программист не думает, когда освобождать — это делается само при выходе из функции.

- **Кэш-дружелюбность.** Локальные переменные функции близки в памяти, скорее всего попадают в одну кэш-линию.
- **Безопасность.** Нет утечек, нет двойного освобождения. Стек сам знает свою жизнь.

12.2.4 Недостатки стека

- **Размер ограничен.** Большие массивы туда нельзя.
- **Размер фиксирован на этапе компиляции.** Нельзя сказать «создай мне массив размером N, где N — переменная». В стандартном C++ — нельзя. (В C99 разрешены VLA — массивы переменной длины, и GCC поддерживает их как расширение для C++. Но в стандартном C++ — нет.)
- **Жизнь привязана к функции.** Если нужно, чтобы объект пережил функцию — на стеке нельзя.

12.3 Куча: подробности

Куча — большая область памяти, из которой программа может выделять кусочки **во время работы**. Размер выделения определяется в рантайме. Жизнь куска **не привязана** к функции — он живёт, пока вы его явно не вернёте.

В C++ есть два уровня работы с кучей: - **Низкий:** оператор `new`, оператор `delete`. Аналоги `malloc/free` из C. - **Высокий:** smart pointers (`std::unique_ptr`, `std::shared_ptr`) и стандартные контейнеры (`std::vector`, `std::string`, ...). Они сами зовут `new/delete` внутри, программисту думать не надо.

В современном коде используют высокий уровень. Низкий — для понимания, как это работает, и для редких случаев. Я разберу низкий уровень детально (это нужно понимать), а потом покажу высокий и объясню, почему он лучше.

12.3.1 new — выделение

Чтобы выделить из кучи память для одного объекта:

```
int* p = new int;           // выделить место для int, p — адрес
*p = 42;                    // положить 42
std::cout << *p << "\n";   // напечатать 42
```

Что произошло:

1. Оператор `new` пошёл к ОС (через библиотеку: `malloc` → `mmap/brk`) и попросил «дай мне 4 байта» (размер `int`).
2. ОС нашла свободный кусок в куче и вернула его адрес.

3. Этот адрес положили в `p` (тип `int*`).
4. Теперь `*p` — это значение по этому адресу. Записали 42.

С инициализацией значения сразу при создании:

```
int* p = new int(42);      // выделить и положить 42
int* q = new int{42};      // то же, через uniform initialization (C++11)
```

Для классов это вызывает соответствующий конструктор — об этом в главе 16.

12.3.2 delete — возврат

Когда объект больше не нужен — обязательно верните память:

```
delete p;
```

Это говорит: «я закончил с памятью по адресу `p`, верни её в общий пул». ОС/менеджер памяти помечает кусок как свободный, и его смогут переиспользовать следующие `new`.

После `delete` указатель **становится опасным**. Адрес тот же, но память уже не «наша». Использовать `*p` после `delete p` — UB:

```
int* p = new int(42);
delete p;
*p = 0;      // UB! Память уже не наша
delete p;    // UB! Двойное освобождение
```

Хорошая привычка — после `delete` присвоить указатель `nullptr`:

```
delete p;
p = nullptr;    // теперь *p вызовет краш сразу, а не UB
delete p;      // delete nullptr — безопасно, ничего не делает
```

12.3.3 new[] и delete[] — массивы

Чтобы выделить массив объектов:

```
int* arr = new int[100];    // массив из 100 int
arr[0] = 1;
arr[99] = 100;

delete[] arr;               // вернуть, ВНИМАНИЕ — delete[], не delete
```

Никогда не путайте `delete` и `delete[]`: - `new int` → `delete` (одиночный). - `new int[N]` → `delete[]` (массив).

Если перепутать — UB. Самое противное: на простых типах (`int`, `float`) это часто «работает», но протекает память. На сложных типах с деструкторами (`std::string[]`) — краш.

12.3.4 Размер можно вычислять в рантайме

В отличие от стека, размер на куче **может быть переменным**:

```
int n;
std::cin >> n;
int* arr = new int[n];    // OK! n может быть любым

for (int i = 0; i < n; ++i) {
    arr[i] = i * i;
}

delete[] arr;
```

Это то, чего не сделать на стеке стандартным C++. Куча — единственный способ.

12.4 Утечки памяти

Утечка — это когда вы выделили память через `new`, но **никогда не освободили** через `delete`. Память остаётся занятой, но программа её больше не видит — потеряна.

```
void leak() {
    int* p = new int(42);
    // забыли delete
}

int main() {
    for (int i = 0; i < 1'000'000; ++i) {
        leak();
    }
    // программа потребляет ~4 МВ лишней памяти
}
```

Один пропуск `delete` в коде, который выполняется миллион раз — миллион утечек. Программа толстеет, начинает занимать гигабайты, ОС начинает свапиться, всё тормозит, в худшем случае — OOM-killer убивает программу.

12.4.1 Когда утечки нестрашны

Парадокс: иногда утечки **не имеют значения**.

- Если программа **выполняется секунду и закрывается** — ОС всё равно освободит её память при выходе. Утечка в `main` в маленькой утилите — никому не вредит.
- Если программа выделила что-то и удерживает до самого конца (а конец близок) — тоже не страшно.

Опасны утечки в **долгоживущих** процессах: серверы, демоны, мобильные приложения. Там каждая утечка остаётся навсегда, и через месяц работы — гигабайты лишнего.

12.4.2 Пять способов утечь память

Я набрал из практики пять типичных:

1. Забыл `delete`.

```
void process() {
    int* p = new int(42);
    use(p);
    // нет delete
}
```

2. Исключение между `new` и `delete`.

```
void process() {
    int* p = new int(42);
    might_throw();    // исключение!
    delete p;         // не выполнится
}
```

3. Ранний `return`.

```
int* find_or_create() {
    int* p = new int(42);
    if (some_condition) {
        return p;    // вернули, вызывающий должен delete
    }
    if (other_condition) {
        return nullptr; // Ой: p потеряно
    }
    return p;
}
```

4. Перезапись указателя.

```
int* p = new int(42);  
p = new int(43);           // адрес 42-го объекта потерян, утечка  
delete p;                  // удаляет только 43-й
```

5. Циклы между `shared_ptr` — про это в главе 18. Сейчас просто упомяну: даже с smart pointers можно утечь, если структура объектов закольцованная.

12.4.3 Висячие указатели

Обратная проблема: `delete`-нули объект, но указатель остался. Использование такого указателя — UB.

```
int* p = new int(42);  
int* q = p;           // q тоже указывает туда  
  
delete p;              // память освобождена  
p = nullptr;  
  
*q = 0;                // q — висячий указатель! UB!
```

Сложнее с возвратом из функции:

```
int* make_int() {  
    int x = 42;  
    return &x;         // !!! &x — адрес локальной переменной  
}  
  
int main() {  
    int* p = make_int(); // p висячий: x уничтожен  
    *p = 0;              // UB  
}
```

Современные компиляторы это ловят (`-Wreturn-local-addr`), но не всегда.

12.4.4 Двойное освобождение

```
int* p = new int(42);  
delete p;  
delete p;           // UB: второе освобождение
```

Что произойдёт — зависит от рантайма. На glibc вы получите красивое сообщение «double free or corruption». На других — UB, может быть SEGFAULT, может быть тихая катастрофа.

12.4.5 Использование после освобождения (UAF)

```
int* p = new int(42);  
delete p;  
std::cout << *p;    // UB: use-after-free
```

Это золотая жила багов безопасности — большинство уязвимостей в браузерах и ядрах ОС в 2010-2020-х годах связаны с UAF.

12.5 Проблема: слишком много ручной работы

Получается, программисту нужно: - Помнить о каждом new — где он, кто делает delete. - Освобождать на всех путях выхода из функции (включая исключения). - Не использовать указатель после delete. - Не освобождать дважды.

На больших программах это нереально. Поэтому в C — постоянная борьба с утечками. В C++ нашли решение.

12.6 RAII — главный паттерн C++

RAII — Resource Acquisition Is Initialization, «получение ресурса есть инициализация». Идея, кажущаяся простой, но меняющая всё:

Привязать жизнь ресурса к жизни объекта на стеке.

Когда объект создаётся (на стеке) — он **захватывает** ресурс. Когда объект уничтожается (выходит из области видимости) — он **освобождает** ресурс.

Стек уничтожает объекты автоматически. Значит, ресурс автоматически освобождается. Никаких ручных delete. Никаких ошибок «забыл освободить».

Это работает для любого ресурса, не только памяти: файлов, мьютексов, сокетов, окон, чего угодно.

12.6.1 Конструктор и деструктор

Чтобы это работало, у объекта должно быть два специальных метода:

- **Конструктор** — выполняется при создании объекта.
- **Деструктор** — выполняется при уничтожении.

Подробно про эти штуки в главе 16 на проекте RPG. Сейчас — лёгкий обзор.

Простой пример. Класс, который сам управляет одним динамически выделенным `int`:

```
class IntHolder {
public:
    // Конструктор: создаём объект, выделяем int
    IntHolder(int value) {
        ptr_ = new int(value);
        std::cout << "Created at " << ptr_ << "\n";
    }

    // Деструктор: освобождаем
    ~IntHolder() {
        std::cout << "Destroying at " << ptr_ << "\n";
        delete ptr_;
    }

    int get() const { return *ptr_; }
    void set(int v) { *ptr_ = v; }

private:
    int* ptr_;
};

int main() {
    IntHolder a(42);
    IntHolder b(100);
    std::cout << a.get() << " " << b.get() << "\n";
} // здесь автоматически вызываются деструкторы a и b
```

Вывод:

```
Created at 0x600003a04020
Created at 0x600003a04030
42 100
Destroying at 0x600003a04030
Destroying at 0x600003a04020
```

Заметили: я не написал ни одного `delete` в `main`. Память освобождается **сама**, потому что деструктор `IntHolder` это делает, и деструктор вызывается автоматически при выходе из `main`.

Деструкторы вызываются в **обратном порядке создания** — это гарантия языка, и она важна, когда объекты зависят друг от друга.

Это и есть RAII в чистом виде. Объект на стеке владеет ресурсом на куче, и его деструктор разруливает уборку.

12.6.2 Почему RAII работает с исключениями

Главная сила RAII — она устойчива к исключениям.

```
void might_throw();

void process() {
    IntHolder x(42);
    might_throw();    // если выкинет исключение — деструктор x всё равно вызовется!
}
```

При возникновении исключения C++ выполняет **stack unwinding** (раскрутка стека): по очереди вызываются деструкторы всех объектов в текущем кадре, потом в кадре функции выше, и так далее до места, где исключение поймают. Утечек нет.

Сравните с прямой работой через new/delete:

```
void process() {
    int* p = new int(42);
    might_throw();    // утечка! delete не выполнится
    delete p;
}
```

Утечка. Можно обернуть в try/catch, но громоздко. RAII делает это автоматически.

12.7 Стандартные RAII-обёртки

В стандартной библиотеке десятки RAII-объектов. Самые частые:

- **std::vector<T>** — массив, который сам управляет памятью. Растёт по мере добавления, сам освобождает в деструкторе.
- **std::string** — строка, аналогично.
- **std::ifstream** / **std::ofstream** — файлы, открываемые в конструкторе и закрываемые в деструкторе.
- **std::lock_guard** — берёт мьютекс в конструкторе, отпускает в деструкторе (про потоки в главе 41).

- `std::unique_ptr<T>` — умный указатель, владеющий объектом и удаляющий его в деструкторе.
- `std::shared_ptr<T>` — умный указатель с подсчётом ссылок (`std::shared_ptr`).

С этими штуками вы **почти никогда не пишете new/delete сами**. Они делают это за вас.

12.7.1 Пример: вектор вместо ручного массива

Так делать **не** надо:

```
int* arr = new int[1000];  
// ...использование...  
delete[] arr;    // надо помнить!
```

Так — надо:

```
std::vector<int> arr(1000);  
// ...использование...  
// arr автоматически освободится
```

Короче, безопаснее, и нет потенциальной утечки.

12.7.2 Пример: файл вместо fopen

Так делать **не** надо (C-стиль):

```
FILE* f = fopen("data.txt", "r");  
if (f == nullptr) return;  
// ...использование...  
fclose(f);    // надо помнить!
```

Так — надо:

```
#include <fstream>  
  
std::ifstream f("data.txt");  
if (!f.is_open()) return;  
// ...использование...  
// f автоматически закроется
```

Меньше кода, нет вероятности забыть `fclose`.

12.8 smart pointers — предварительно

Раз стандартные контейнеры обёртки идут с готовой RAII-логикой, может, и для голого указателя кто-то сделал? Да, и это **smart pointers**.

В C++11 их три:

- `std::unique_ptr<T>` — единоличный владелец. Только один указатель на объект; при уничтожении — удаляет.
- `std::shared_ptr<T>` — совладельцы. Объект живёт, пока есть хоть один `shared_ptr` на него; уничтожается, когда последний пропадает.
- `std::weak_ptr<T>` — наблюдатель, не владеет. Нужен против циклических ссылок `shared_ptr`.

Они подробно разбираются в главе 18 на проекте RPG. Сейчас — лёгкое знакомство.

```
#include <memory>

void use_unique() {
    std::unique_ptr<int> p(new int(42));
    std::cout << *p << "\n";
    // p автоматически удалит int
}

void use_shared() {
    std::shared_ptr<int> p(new int(42));
    auto q = p;           // оба указывают на тот же объект
    std::cout << *p << " " << *q << "\n";
    // объект удалится, когда оба p и q уничтожатся
}
```

В современном C++ почти **весь** код, имеющий дело с памятью на куче — это контейнеры и smart pointers. Ручной `new/delete` — крайне редкая экзотика, обычно только в реализации низкоуровневых структур данных или интерфейсах к C-API.

В C++14 добавили `std::make_unique`, в C++11 его нет — поэтому в нашей C++11-базе будем писать `std::unique_ptr<T>(new T(...))` или сделаем свой шим. Подробнее в главе 18.

12.9 Стек или куча — что выбрать

После всех этих описаний — простое правило:

По умолчанию — стек. Куча — только если есть конкретная причина.

Причины брать кучу:

1. **Размер заранее неизвестен.** Массив n элементов, где n ввёл пользователь.
2. **Объект должен пережить функцию.** Создаём в одной функции, отдаём наружу.
3. **Объект слишком большой для стека.** Например, видеобуфер 10 MB.
4. **Полиморфизм через базовый класс.** Передаём `Item*`, реальный объект — `Weapon` или `Armor`. Об этом в главе 17.

Во всех остальных случаях — стек. Локальная переменная, маленький массив, маленькая структура — на стеке. Это и быстрее, и безопаснее.

12.9.1 Конкретные примеры

```
// Стек:
int x = 42;
double pi = 3.14;
std::vector<int> v = {1, 2, 3};
std::string name = "Alice";
Player player(100, "Hero");

// Куча (через RAII обёртки):
std::vector<int> big_array(1'000'000); // вектор сам хранит данные на куче
std::unique_ptr<Player> hero(new Player(100, "Hero")); // когда нужен полиморфизм
auto enemies = std::vector<std::unique_ptr<Enemy>>{}; // массив указателей на разные типы врагов
```

В каждой строке стек хранит **управляющую часть** (указатель + размер для вектора, указатель для `unique_ptr`). А содержимое — на куче. Деструкторы стеков-управляющих частей освобождают кучу. Это и есть RAII в действии.

12.10 Инструменты для борьбы с проблемами памяти

Несмотря на RAII, иногда что-то протекает или указывает не туда. Есть инструменты, которые помогают находить эти баги.

12.10.1 AddressSanitizer (ASan)

Встроен в clang и gcc. Скомпилируйте программу с `-fsanitize=address`:

```
$ g++ -std=c++11 -fsanitize=address -g program.cpp -o program
$ ./program
```

ASan вставит проверки в каждое обращение к памяти. Если будет: - Использование после освобождения, - Двойное освобождение, - Выход за границы массива, - Утечка,

— программа упадёт с красивым отчётом, какая строка кода виновата.

ASan чуть замедляет программу (раза в 2), но для отладки незаменим. **Всегда** запускайте тесты под ASan.

12.10.2 Valgrind

Раньше (до ASan) самым популярным был **Valgrind** — внешняя программа, эмулирующая выполнение и проверяющая все обращения к памяти. Сейчас на Linux:

```
$ valgrind --leak-check=full ./program
```

Точнее ASan, но в 50 раз медленнее. На macOS работает плохо. Раньше — стандарт; сейчас ASan чаще.

12.10.3 LeakSanitizer (LSan)

Часть ASan. Включается флагом `-fsanitize=leak`. Просто проверяет утечки, не остальные ошибки. Полезно для CI.

12.11 Большой пример: классический ресурс

Соберём пример, который ясно показывает разницу C-стиля и C++-стиля. Класс «лог-файл»: открывает файл, пишет в него, закрывает.

12.11.1 C-стиль (без RAII)

```
class LogFile_BAD {
public:
    LogFile_BAD(const std::string& filename) {
        f_ = std::fopen(filename.c_str(), "w");
    }

    void close() {
        if (f_ != nullptr) {
```

```

        std::fclose(f_);
        f_ = nullptr;
    }
}

void write(const std::string& message) {
    if (f_ != nullptr) {
        std::fprintf(f_, "%s\n", message.c_str());
    }
}

private:
    FILE* f_;
};

void use_bad() {
    LogFile_BAD log("test.log");
    log.write("hello");
    log.close();           // программист ОБЯЗАН вспомнить close
}

```

Проблема: что если программист забудет `close()`? Или между `write` и `close` будет ранний `return`, или исключение? Файл останется открытым, дескриптор утечёт. На длинной программе можно исчерпать лимит ОС на открытые файлы.

12.11.2 C++-стиль (с RAII)

```

class LogFile {
public:
    LogFile(const std::string& filename) {
        f_ = std::fopen(filename.c_str(), "w");
        if (f_ == nullptr) {
            throw std::runtime_error("cannot open " + filename);
        }
    }

    ~LogFile() {
        if (f_ != nullptr) {

```

```

        std::fclose(f_);
    }
}

void write(const std::string& message) {
    std::fprintf(f_, "%s\n", message.c_str());
}

// Запрещаем копирование (об этом в главе 16)
LogFile(const LogFile&) = delete;
LogFile& operator=(const LogFile&) = delete;

private:
    FILE* f_ = nullptr;
};

void use_good() {
    LogFile log("test.log");
    log.write("hello");
    // close не нужен! Деструктор сделает это сам.
}

```

Этот код **физически не может** забыть закрыть файл. Что бы ни случилось — исключение, ранний return, нормальный выход — деструктор сработает и fclose будет вызван. Это и есть сила RAII.

Конструкции = delete запрещают копирование объекта. Иначе можно было бы сделать копию LogFile, и при уничтожении обоих копий случился бы двойной fclose. Об этом подробно в главе 16 (rule of five).

12.11.3 Вариант со стандартной библиотекой

Ещё короче — std::ofstream уже RAII:

```

#include <fstream>

void use_modern() {
    std::ofstream log("test.log");
    log << "hello\n";
}

```

```
// ничего больше не надо  
}
```

Стандартная библиотека уже использует RAII для большинства ресурсов. Поэтому **в реальной работе вы редко пишете деструкторы сами** — обычно завернуть в готовый `vector/string/fstream/unique_ptr` достаточно.

12.12 Когда нужно писать свой деструктор

Когда стандартные обёртки не подходят. Например:

- Работа с C-API, где нужно вызвать специфическую очистку (`SDL_FreeSurface`, `curl_easy_cleanup`, и так далее).
- Свой контейнер с особой логикой.
- Объект, владеющий несколькими ресурсами сразу.

Правило **rule of five** — если у класса есть нетривиальный деструктор, ему **скорее всего** нужны:

- Конструктор копирования, - Оператор присваивания при копировании, - Конструктор перемещения, - Оператор присваивания при перемещении.

— или явно запретить все эти операции через `= delete`.

Об этом в главе 16. Сейчас просто запомните, что свой деструктор тянет за собой много работы — поэтому **избегайте писать свой деструктор**, если можно обойтись стандартной обёрткой.

12.13 Главные правила этой главы

1. **По умолчанию — переменная на стеке.** Только если нужна гибкость или объект должен пережить функцию — куча.
2. **На куче — через RAII-обёртки.** `std::vector`, `std::string`, `std::unique_ptr`, `std::shared_ptr`. **Не** через сырой `new/delete`.
3. **Каждый ресурс — в одном объекте.** Если класс владеет несколькими ресурсами — оберните каждый в свою RAII-обёртку, не пишите общий деструктор.
4. **new без delete или со «свободным» указателем — звоночек.** Используйте `std::unique_ptr` или поместите в контейнер.
5. **Запускайте тесты под ASan.** Многие баги памяти он находит мгновенно.
6. **Помните про двойной delete и UAF.** Они выглядят страшнее, чем «легко исправимые опечатки» — это часто эксплуатируемые уязвимости.

12.14 Маленькое упражнение

1. Напишите класс `IntArray`, который владеет динамическим массивом `int`:

- конструктор `IntArray(size_t n)` выделяет `new int[n]`.
- деструктор `~IntArray()` делает `delete[]`.
- метод `at(size_t i)` возвращает `int&` на элемент.

Запустите программу, создающую `IntArray`, заполняющую его, и убедитесь, что деструктор зовётся.

2. Скомпилируйте программу из пункта 1 с `-fsanitize=address`. Запустите. Должно работать без жалоб.
3. Намеренно сломайте: уберите `delete[]` из деструктора. Скомпилируйте под `ASan`, запустите. `ASan` должен сказать про утечку.
4. Намеренно сломайте: вернитесь к рабочей версии, но в `main` после уничтожения `IntArray` попытайтесь обратиться к освобождённой памяти через сохранённый указатель. `ASan` должен сказать про `use-after-free`.
5. Напишите программу:

```
void deep(int n) {  
    int x = n;  
    deep(n + 1);  
    (void)x;    // чтобы компилятор не выкинул  
}
```

Запустите `deep(0)`. Программа должна упасть через секунды. Если есть `ulimit -s 1024` — упадёт быстрее.

6. (Опционально, важно). Перепишите упражнение 1 через `std::unique_ptr<int[]>` (использует `delete[]` под капотом):

```
std::unique_ptr<int[]> arr(new int[100]);
```

Никакого пользовательского деструктора. Память по-прежнему освобождается. Это и есть современный C++.

12.15 Что дальше

Глава 12 — про строки (`std::string`) и потоки ввода-вывода (`std::cin/std::cout/std::ifstream`). После 11-й главы, на которой мы дотащили концептуальную базу, эти штуки уже не загадка

— `std::string` это RAII-обёртка над `char*` в куче, `std::ofstream` — RAII над `FILE*`. Дальше изученное только закрепляется.

Часть I на этой главе перевалила за середину: остались главы 12 (строки и I/O), 13 (контейнеры), 14 (алгоритмы). После них стартует Часть II — RPG-проект, и весь язык начнёт работать вместе.

Глава 13

Глава 12. Строки и потоки ВВОДА-ВЫВОДА

Программы редко обходятся без текста. Прочитать имя пользователя, вывести список товаров, открыть файл с настройками, разобрать команду «attack goblin» — всё это строки. В этой главе разберём, как с ними работать в C++.

C++ — язык с историей. Поэтому строки тут двух видов: **C-style** (наследие C, `char*` с нулевым байтом в конце) и **C++-style** (`std::string`, RAII-обёртка с современным интерфейсом). Я расскажу про обе, но писать в реальном коде будем `std::string`. Глава 11 уже дала нужный фундамент: `std::string` — это просто RAII-объект, который владеет куском памяти на куче.

13.1 Старая школа: C-strings

В C, до появления `std::string`, строкой назывался **массив `char` с нулевым байтом в конце**:

```
char str[] = "Hello";  
// в памяти: 'H' 'e' 'l' 'l' 'o' '\0'  
// размер: 6 байт (5 видимых + терминатор)
```

Терминатор `'\0'` нужен, чтобы все функции знали, где строка кончается. `strlen("Hello")` пробегает по символам до первого нуля и возвращает 5. Нет терминатора — выйдет за границы массива в чужую память, UB.

В C++ литералы `"Hello"` имеют тип `const char[6]` (массив из 6 элементов), который легко конвертируется в `const char*` (указатель на первый символ):

```
const char* msg = "Hello";
std::cout << msg << "\n";    // OK
std::cout << msg[1] << "\n"; // 'e'
```

C-строки до сих пор используются: - В системных API (`open(const char*, ...)`, `printf(const char*, ...)`). - В литералах в коде — "Hello" это C-строка, и компилятор её хранит в `.rodata`. - В производительности — иногда нужно избежать аллокации `std::string`.

Но писать **новый** код на C-строках почти никогда не стоит. Слишком много ловушек:

- **Нельзя просто склеить.** "hello" + " world" в C — UB (это сложение двух указателей).
- **Размер не известен из типа.** `char[]` теряет размер при передаче в функцию (там это уже `char*`).
- **Терминатор легко забыть.** Все функции типа `strcat`, `strcpy` пишут в чужую память, если буфер маловат.

Поэтому весь современный C++-код использует `std::string`. C-строку оставляем для интерфейса с C-API.

13.2 `std::string` — основа основ

`std::string` — это объект, который **владеет** куском памяти с текстом. Управляет им через RAII: выделяет в конструкторе, освобождает в деструкторе. Никаких ручных `delete`. Можно копировать, передавать в функции, возвращать — всё работает корректно.

Подключается через заголовок `<string>`:

```
#include <string>
```

(Обычно подтянется автоматически через `<iostream>`, но лучше явно.)

13.2.1 Создание и базовые операции

```
std::string a;                // пустая строка
std::string b = "Hello";      // из C-строки
std::string c("World");       // через конструктор
std::string d{"!"};           // uniform initialization
std::string e(5, 'x');         // "xxxxx" — 5 одинаковых символов
std::string f = b;             // копия b

// Размер
std::cout << b.size() << "\n"; // 5
```

```
std::cout << b.length() << "\n"; // 5 (синоним size)
std::cout << b.empty() << "\n"; // 0 (false)
std::cout << a.empty() << "\n"; // 1 (true)
```

`size()` возвращает количество **байтов**. Для ASCII совпадает с количеством символов. Для UTF-8 — нет (русская буква занимает 2 байта; мы говорили в главе 2).

13.2.2 Конкатенация

Главная сила `std::string` — её можно склеивать оператором `+`:

```
std::string first = "Hello";
std::string second = "World";
std::string greeting = first + ", " + second + "!"; // "Hello, World!"
```

И дописывать в существующую через `+=`:

```
std::string s = "Hello";
s += ", "; // "Hello, "
s += "World"; // "Hello, World"
s += '!'; // "Hello, World!" (даже одиночный char можно добавить)
```

Под капотом конкатенация — это **аллокация новой памяти, копирование старой строки, копирование добавляемой части, освобождение старой**. Не бесплатная операция, но обычно достаточно быстрая.

Если строите длинную строку через много `+=`, `std::string` иногда выделяет больше, чем нужно, чтобы следующие `+=` не вызывали аллокацию. Это похоже на `std::vector`. Для критичной производительности можно явно зарезервировать через `s.reserve(1000)` — выделить место под 1000 символов сразу.

13.2.3 Индексация и обход

```
std::string s = "Hello";
char c = s[0]; // 'H'
s[0] = 'J'; // s теперь "Jello"

// Обход через range-based for (C++11)
for (char ch : s) {
    std::cout << ch << "\n";
}
```

```
// Обход с индексами
for (size_t i = 0; i < s.size(); ++i) {
    std::cout << i << ": " << s[i] << "\n";
}
```

`s[i]` **не проверяет границы**. `s[100]` на 5-символьной строке — UB. Если нужна проверка с исключением — `s.at(i)`:

```
try {
    char c = s.at(100);    // выкинет std::out_of_range
} catch (const std::out_of_range& e) {
    std::cout << "out of range\n";
}
```

В цикле проверки замедлили бы код, поэтому почти всегда используют `[]`, а правильность границ обеспечивают сами.

13.2.4 Сравнение

```
std::string a = "apple";
std::string b = "banana";
std::string c = "apple";

if (a == c) ...    // true
if (a != b) ...    // true
if (a < b) ...     // true (лексикографически по байтам)
```

`==`, `!=`, `<`, `>`, `<=`, `>=` работают как ожидаешь — сравнение байт за байтом. Регистр учитывается, заглавные < строчных (по таблице ASCII).

В C-стиле сравнивать через `==` нельзя:

```
const char* a = "apple";
const char* b = "apple";
if (a == b) ...    // ОШИБКА в смысле: сравниваются указатели, не содержимое!
```

Это сравнение указателей. Чтобы сравнить содержимое, нужен `strcmp(a, b) == 0`. Ещё один повод использовать `std::string`.

13.2.5 Подстроки

```
std::string s = "Hello, World!";

std::string sub = s.substr(7, 5);    // "World" — с позиции 7, длиной 5
std::string sub2 = s.substr(7);     // "World!" — с позиции 7 до конца
```

13.2.6 Поиск

```
std::string s = "Hello, World, World!";

size_t pos = s.find("World");        // 7 — позиция первого вхождения
size_t pos2 = s.find("World", 8);    // 14 — поиск с позиции 8

if (s.find("Foo") == std::string::npos) {
    std::cout << "не найдено\n";
}
```

`std::string::npos` — специальное значение (обычно максимум `size_t`), возвращаемое при «не найдено». Запомните проверку `== npos`.

13.2.7 Замена и вставка

```
std::string s = "Hello, World!";
s.replace(7, 5, "C++");            // Hello, C++!
s.insert(0, "Greetings: ");        // Greetings: Hello, C++!
s.erase(0, 11);                   // Hello, C++!
```

Не особенно частая операция, но иногда нужна.

13.2.8 Преобразование к C-строке

Когда нужно передать в C-API, который ждёт `const char*`:

```
std::string s = "test.txt";
FILE* f = std::fopen(s.c_str(), "r");
```

`s.c_str()` возвращает `const char*` на содержимое строки, с гарантированным `'\0'` на конце. Указатель **валиден, пока строка не изменена** — после `s += "x"` он может стать невалидным. Не сохраняйте `c_str()` надолго.

13.2.9 Преобразования с числами

```
#include <string>

// Число → строка
std::string s1 = std::to_string(42);           // "42"
std::string s2 = std::to_string(3.14);         // "3.140000" (увы, форматирование кучее)

// Строка → число
int n = std::stoi("42");                       // 42
double d = std::stod("3.14");                  // 3.14
long ll = std::stoll("1234567890");            // 1234567890

// Если строка не парсится — выкинет исключение
try {
    int bad = std::stoi("not a number");
} catch (const std::invalid_argument& e) {
    std::cout << "не число\n";
}
```

`std::to_string` для дробных даёт «6 цифр после точки» — не всегда хорошо. Для приличного форматирования используют `std::stringstream` (об этом ниже).

13.2.10 Что лежит внутри `std::string`

Под капотом `std::string` обычно содержит три поля: - Указатель на буфер с символами (на куче). - Текущая длина. - Текущая ёмкость (`capacity`) — сколько символов помещается без новой аллокации.

То есть это **указатель + два размера** ≈ 24 байта на 64-битной системе. Сама строка лежит отдельно, в куче.

Современные реализации хорошо оптимизированы: для **коротких** строк (~15 символов на GCC libstdc++, ~22 на clang libc++) данные хранятся **внутри** самой `std::string`, без аллокации. Это называется **SSO** — Small String Optimization. Поэтому `std::string s = "hi"` обычно вообще не выделяет память на куче. А `std::string s = "очень длинная строка..."` — выделит.

13.2.11 Итог по строкам

- Используйте `std::string` везде.
- C-строки — только для интерфейса с C-API.

- `s.size()` — количество **байтов**, не символов (для UTF-8 будут отличаться).
- `s[i]` без проверки, `s.at(i)` с исключением.
- `==`, `<` сравнивают содержимое, а не указатели.
- Поиск через `find`, проверка `==` `npos`.
- `c_str()` — для передачи в C-API; не сохранять надолго.
- `to_string`, `stoi`, `stod` — конверсии с числами.

13.3 `std::cout` — вывод

С `std::cout` мы знакомы с главы 6. Здесь — детали.

`std::cout` — это объект класса `std::ostream`. Оператор `<<` — это **сдвиг влево**, но для потоков перегружен как «вставить в поток». Возвращает сам поток, поэтому можно цеплять:

```
std::cout << "Имя: " << name << ", возраст: " << age << "\n";
```

Это эквивалентно:

```
(((((std::cout << "Имя: ") << name) << ", возраст: ") << age) << "\n";
```

Каждый `<<` принимает поток и значение, кладёт значение в поток, возвращает поток.

`<<` поддерживает все встроенные типы плюс `std::string`, `const char*`. Для своих типов перегрузку можно написать самому — об этом в главе 16.

13.3.1 `endl` против `\n`

Что лучше:

```
std::cout << "Hello" << std::endl;  
std::cout << "Hello\n";
```

Оба выводят «Hello» с переводом строки. Разница: - `std::endl` дополнительно **сбрасывает буфер** — заставляет немедленно записать содержимое в терминал/файл. - `"\n"` — просто символ перевода строки, ничего не сбрасывает.

Когда программа выводит много, флушинг каждой строки — заметная трата. Для производительности предпочитайте `"\n"`. `std::endl` оставляйте для случаев, когда нужно немедленно показать пользователю (например, прогресс).

Современный код часто пишет `"\n"` везде, и я тоже буду так.

13.3.2 std::cerr — поток ошибок

std::cerr — отдельный поток для ошибок. По умолчанию связан с тем же терминалом, что и std::cout, но **не буферизирован**. Каждая запись сразу выводится.

```
std::cerr << "Error: cannot open file\n";
```

Зачем разделять? В Unix-философии: - std::cout — основной вывод, его можно перенаправить (./program > result.txt). - std::cerr — диагностика, она остаётся в терминале (даже после перенаправления).

```
$ ./program > out.txt
Error: cannot open file      # это std::cerr — на экране
$ cat out.txt
обычные выводы программы   # это std::cout — в файле
```

Это полезно: пользователь видит ошибки сразу, даже если стандартный вывод куда-то перенаправлен.

std::clog — ещё один поток ошибок, **буферизированный**. Используется редко.

13.3.3 Форматирование вывода

По умолчанию числа выводятся «как есть»:

```
std::cout << 3.14159265 << "\n";    // 3.14159
std::cout << 1234567 << "\n";        // 1234567
std::cout << true << "\n";           // 1
```

Чтобы управлять форматом — манипуляторы из <iomanip>:

```
#include <iomanip>

// Число знаков после запятой:
std::cout << std::fixed << std::setprecision(2) << 3.14159 << "\n";    // 3.14

// Ширина поля (выравнивание):
std::cout << std::setw(10) << 42 << "\n";          // "          42" — справа
std::cout << std::left << std::setw(10) << 42 << "\n"; // "42          " — слева

// Заполнитель:
std::cout << std::setfill('0') << std::setw(4) << 42 << "\n";    // "0042"

// Системы счисления:
```

```
std::cout << std::hex << 255 << "\n";    // ff
std::cout << std::oct << 8 << "\n";      // 10
std::cout << std::dec << 255 << "\n";    // 255 (десятичная — стандартная)

// Логические как true/false:
std::cout << std::boolalpha << true << "\n";    // true
std::cout << std::noboolalpha << true << "\n";  // 1
```

Большинство манипуляторов **остаются в силе** на следующие выводы — это «состояние потока». Если установили `std::hex` — все числа дальше будут в hex до явного `std::dec`. Это иногда подвох. Принято возвращать поток в дефолт после нестандартного формата.

Чёткое управление шириной полей — для красивых таблиц:

```
std::cout << std::left
    << std::setw(15) << "Name"
    << std::setw(5) << "Age"
    << "\n";

for (const auto& person : people) {
    std::cout << std::left
        << std::setw(15) << person.name
        << std::setw(5) << person.age
        << "\n";
}
```

Получится:

Name	Age
Alice	30
Bob	25
Charlie	42

В современном C++20 появилась `std::format` (как Python f-strings), но в C++11 её нет.

13.4 std::cin — ввод

`std::cin` — стандартный ввод. По умолчанию читает с клавиатуры.

```
int age;
std::cout << "Сколько вам лет? ";
```

```
std::cin >> age;
std::cout << "Вы ввели " << age << "\n";
```

Оператор >> («извлечь из потока»): - Пропускает ведущие пробелы и переводы строк. - Читает до первого пробела/перевода строки. - Преобразует прочитанное в нужный тип.

Если ввели нечто, что не парсится как `int` — поток переходит в **failure state**, переменная остаётся неинициализированной (или со старым значением). После `cin >> age` хорошо проверить:

```
int age;
if (std::cin >> age) {
    std::cout << "OK, возраст " << age << "\n";
} else {
    std::cout << "Не число\n";
    std::cin.clear(); // сбросить failure
    std::cin.ignore(1000, '\n'); // пропустить плохие символы до конца строки
}
```

Эта проверка работает потому, что `std::cin >> age` возвращает сам поток, который в булевом контексте даёт `true` если всё ОК, `false` если ошибка.

13.4.1 Несколько значений подряд

```
int a, b, c;
std::cin >> a >> b >> c; // три числа, разделённых пробелами или переводами строк
```

Если ввести «3 5 7» и Enter — `a=3`, `b=5`, `c=7`. Так же сработает «3\n5\n7» — пробельные символы любые.

13.4.2 Чтение строки целиком: `getline`

`std::cin >> s` для строки прочитает до первого пробела. Если хочется прочитать **всю строку** (включая пробелы):

```
std::string name;
std::getline(std::cin, name);
```

`getline` читает до перевода строки (не включая) и сохраняет в `name`. Идеально для имён, фраз, путей.

13.4.3 Главная ловушка `cin`: смешивание » и `getline`

Самый частый баг новичка в I/O. Код такой:

```
int age;
std::string name;

std::cin >> age;           // вводят 25 + Enter
std::getline(std::cin, name); // вводят "John"

std::cout << age << " " << name;
```

Ожидаешь: 25 John. На самом деле: 25 (имя пустое!).

Почему? Когда вы вводите 25 Enter, в буфере остаётся 25\n. `cin >> age` читает 25 и **оставляет** \n в буфере. Далее `getline` сразу видит \n — и считает строку пустой.

Решение — после `>>` явно пропустить остаток строки:

```
std::cin >> age;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, name);
```

`std::cin.ignore(N, ch)` пропускает до N символов или до встречи `ch`. С большим N (`max`) — пропустит до конца строки.

Громоздко, но необходимо. Альтернатива — **всегда** читать через `getline` и парсить руками:

```
std::string line;
std::getline(std::cin, line);
int age = std::stoi(line);

std::string name;
std::getline(std::cin, name);
```

Этот стиль безопаснее. В книге я буду чередовать в зависимости от ситуации.

13.5 Файловые потоки

Файлы открываются через `std::ifstream` (для чтения) и `std::ofstream` (для записи). Обе — RAII-обёртки, как мы видели в главе 11.

13.5.1 Чтение из файла

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream file("data.txt");
    if (!file.is_open()) {
        std::cerr << "Не удалось открыть файл\n";
        return 1;
    }

    std::string line;
    while (std::getline(file, line)) {
        std::cout << line << "\n";
    }

    return 0;
    // файл закроется автоматически в деструкторе
}
```

Проверка `if (!file.is_open())` — открыть могли не удать (нет файла, нет прав). `while (std::getline(file, line))` — стандартный паттерн чтения построчно: цикл продолжается, пока `getline` успешно читает; на конце файла `getline` ставит failure state, цикл завершается.

Также можно читать через `>>` — числа, отдельные слова:

```
std::ifstream file("numbers.txt");
int n;
while (file >> n) {
    std::cout << n * 2 << "\n";
}
```

13.5.2 Запись в файл

```
#include <fstream>

int main() {
    std::ofstream file("output.txt");
```

```
if (!file.is_open()) {  
    return 1;  
}  
  
file << "Hello, file!\n";  
file << "Number: " << 42 << "\n";  
  
return 0;  
}
```

По умолчанию `std::ofstream` открывает файл **на перезапись**: если он был — содержимое затирается. Чтобы дописать в конец:

```
std::ofstream file("log.txt", std::ios::app);  
file << "новая запись\n";
```

`std::ios::app` — флаг «append». Файл открывается так, что каждая запись добавляется в конец.

Другие флаги: - `std::ios::in` — для чтения (`ifstream` по умолчанию это и делает). - `std::ios::out` — для записи (`ofstream` по умолчанию). - `std::ios::app` — append. - `std::ios::ate` — открыть и сразу перейти в конец (для записи или чтения). - `std::ios::binary` — бинарный режим. По умолчанию текстовый — на Windows `\n` превращается в `\r\n` при записи. Бинарный — без преобразований. - `std::ios::trunc` — очистить файл при открытии (по умолчанию для `ofstream`).

Флаги можно комбинировать через `|`:

```
std::ofstream file("data.bin", std::ios::out | std::ios::binary);
```

13.5.3 Бинарные данные

Когда нужно записать «сырые» байты:

```
int values[] = {1, 2, 3, 4, 5};  
std::ofstream out("data.bin", std::ios::binary);  
out.write(reinterpret_cast<const char*>(values), sizeof(values));
```

`write(const char*, size_t)` пишет заданное количество байтов. `reinterpret_cast<const char*>` — это говорит компилятору «тракуй этот указатель как `const char*`». Об этом ужасе в проекте мини-СУБД (глава 32).

Аналогично `read`:

```
int loaded[5];
std::ifstream in("data.bin", std::ios::binary);
in.read(reinterpret_cast<char*>(loaded), sizeof(loaded));
```

13.5.4 Полностью прочитать файл

Иногда хочется прочитать весь файл в строку:

```
#include <fstream>
#include <sstream>
#include <string>

std::string read_file(const std::string& filename) {
    std::ifstream in(filename);
    if (!in.is_open()) return "";

    std::stringstream buffer;
    buffer << in.rdbuf();
    return buffer.str();
}
```

`in.rdbuf()` — это «внутренний буфер» потока. Перенаправляя его в `stringstream`, мы копируем всё содержимое. Далее `buffer.str()` возвращает результат.

13.6 `std::stringstream` — строка как поток

Иногда нужны возможности потока для работы со строками. Например, разобрать строку в числа:

```
#include <sstream>

std::string line = "1 2 3 4 5";
std::stringstream ss(line);

int n;
while (ss >> n) {
    std::cout << n * 2 << "\n";
}
```

`std::stringstream` — поток, работающий с памятью, а не с файлом. Поддерживает `<<` и `>>`.
Полезно для: - Разбора строк (как пример выше). - Сборки строк с форматированием.

13.6.1 Форматированная сборка строк

`std::to_string` для дробных мало настраивается. С `stringstream` точнее:

```
#include <sstream>
#include <iomanip>

std::stringstream ss;
ss << std::fixed << std::setprecision(2) << 3.14159;
std::string s = ss.str();    // "3.14"
```

Это идиома, которую вы будете использовать постоянно для красивого форматирования.

Класс отдельно для записи — `std::ostringstream`. Для чтения — `std::istringstream`.
`std::stringstream` — оба сразу.

13.7 Сводный пример: разобрать CSV

Соберём осмысленный пример. Файл `people.csv`:

```
Alice,30,Engineer
Bob,25,Designer
Charlie,42,Manager
```

Прочитаем его и выведем красивой таблицей:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>

struct Person {
    std::string name;
    int age;
    std::string job;
};
```



```
std::vector<Person> read_csv(const std::string& filename) {
    std::vector<Person> people;
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не могу открыть " << filename << "\n";
        return people;
    }

    std::string line;
    while (std::getline(file, line)) {
        std::stringstream ss(line);
        Person p;

        std::getline(ss, p.name, ',');

        std::string age_str;
        std::getline(ss, age_str, ',');
        p.age = std::stoi(age_str);

        std::getline(ss, p.job);

        people.push_back(p);
    }

    return people;
}

void print_table(const std::vector<Person>& people) {
    std::cout << std::left
        << std::setw(15) << "Name"
        << std::setw(5) << "Age"
        << "Job\n";
    std::cout << std::string(30, '-') << "\n";

    for (const auto& p : people) {
        std::cout << std::left
            << std::setw(15) << p.name
            << std::setw(5) << p.age
```

```

        << p.job << "\n";
    }
}

int main() {
    auto people = read_csv("people.csv");
    print_table(people);
    return 0;
}

```

Создайте файл `people.csv` с указанным содержимым, соберите программу:

```

$ g++ -std=c++11 -Wall -Wextra csv.cpp -o csv
$ ./csv

```

Name	Age	Job
Alice	30	Engineer
Bob	25	Designer
Charlie	42	Manager

Что использовано: - `std::ifstream` для чтения файла. - `std::getline` с **разделителем** (третий параметр) — читает до указанного символа. - `std::stringstream` для разбора одной строки на поля. - `std::stoi` для конвертации строки в число. - Форматирование таблицы через `std::setw` и `std::left`. - Заполнение разделителя через `std::string(30, '-') (30 минусов)`.

13.8 Пара тонкостей

13.8.1 Чтение чисел и точка в качестве разделителя

Если у вас файл с дробными числами в локали с **запятой** в качестве разделителя (русская локаль):

```

3,14
2,71

```

Стандартный `>>` на дробное число ждёт **точку**. Поэтому он прочитает 3 и остановится. Чтобы поток понимал запятую — нужно настроить локаль:

```

std::cin.imbue(std::locale("ru_RU.UTF-8"));

```

В книжных примерах я везде использую точку. На реальных проектах с локалями отдельно разбираться.

13.8.2 Юникод и `std::string`

Что будет если прочитать русский текст?

```
std::string s;  
std::cin >> s;    // вводим "Привет"  
std::cout << s;   // печатается "Привет"
```

Работает! `std::string` хранит байты, и поток ввода/вывода тоже работает с байтами. Никаких преобразований. Если терминал на UTF-8 (на современных macOS/Linux так), всё проходит через без потерь.

Но: `s.size()` будет 12 байтов (русские буквы по 2 в UTF-8), не 6 символов. И `s[0]` будет первым байтом многобайтного символа, не первой буквой. Для правильной работы с символами нужно конвертировать в `std::u32string` (UTF-32) или использовать библиотеку типа ICU. В нашей книге обходимся байтами — для большинства задач достаточно.

13.8.3 `sync_with_stdio`

В C из тех же файлов можно писать через `printf/scanf`. По умолчанию C++ потоки **синхронизируются** с C-потоками, чтобы можно было смешивать `printf` и `std::cout`. Это замедляет ввод-вывод.

Если в вашей программе только C++-потоки и нужно ускорить (актуально для соревновательного программирования):

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```

В обычных проектах не нужно.

13.9 Главные правила главы

1. **Всегда `std::string`, не C-строки.** Исключение — только при работе с C-API.
2. **"\n" вместо `std::endl`** для скорости.
3. **`std::cerr` для ошибок, `std::cout` для основного вывода.** Не смешивайте.
4. **После `cin >> x` пропускайте остаток строки**, если потом `getline`.
5. **Проверяйте, что файл открылся** через `is_open()`.
6. **Файловые потоки сами закрываются** в деструкторе — не нужен явный `close()`, можно и не нужно.
7. **`std::stringstream` — мощная штука** для парсинга и форматирования.

13.10 Маленькое упражнение

1. Напишите программу, которая просит ввести строку и выводит её задом наперёд. Подсказка: `std::reverse` или ручной цикл.
2. Напишите программу, которая читает файл `numbers.txt` (по одному числу на строке) и выводит их сумму.
3. Напишите программу-калькулятор, читающую выражение вида `5 + 3` (число, оператор, число). Используйте `std::stringstream`.
4. Напишите программу, читающую строки до пустой строки (или `Ctrl+D`), и выводящую их обратно отсортированными.
5. Прочитайте файл (`std::ifstream`) и подсчитайте: сколько строк, сколько слов, сколько символов. По сути, это утилита `wc`.
6. (Сложнее) Напишите программу, которая принимает аргументом командной строки имя файла, открывает его как CSV, и выводит первые 10 строк красивой таблицей.

13.11 Что дальше

Главы 13-14 — про **STL контейнеры** (`vector`, `array`, `list`, `map`, `set`, `unordered_map`, `unordered_set`) и **алгоритмы** (`sort`, `find`, `transform`, `accumulate`). Это последние главы Части I — после них стартуем RPG-проект.

`std::vector` уже мелькал в предыдущих главах. Дальше посмотрим внимательно, как он работает изнутри (вместе с `capacity` vs `size` и инвалидацией итераторов), какие ещё контейнеры есть, и когда какой брать. Тема большая, но фундаментально не новая — много RAII, который мы уже понимаем.

Глава 14

Глава 13. Контейнеры STL

Стандартная библиотека C++ даёт целый зоопарк готовых контейнеров. Это огромное преимущество языка — почти любую задачу хранения данных закрывает что-то стандартное, и писать своё не надо. В этой главе разберём основные: `vector`, `array`, `list`, `map`, `unordered_map`, `set`, `unordered_set`. Расскажу, как они устроены изнутри, что у каких быстро/медленно, и в каком случае какой выбирать.

Эта глава очень практическая. Контейнеры — то, чем вы будете пользоваться **постоянно** в любом коде на C++. Без свободного владения хотя бы `vector` далеко не уехать.

14.1 Зачем контейнеры

В Python вы пишете `[1, 2, 3]` — это список. `{1, 2, 3}` — множество. `{"a": 1}` — словарь. Под капотом всё это разные структуры данных, но интерфейс однообразный и удобный.

В C++ исторически был только сырой массив (`int arr[10]`), который мы рассмотрели в главе 11. У него есть проблемы: размер на этапе компиляции, не растёт, нет удобных операций. Поэтому в стандартную библиотеку добавили контейнеры — каждый со своей структурой данных и интерфейсом.

Все контейнеры являются **шаблонами** (templates) — вы указываете, какой тип они будут хранить:

```
std::vector<int> numbers;           // вектор целых
std::vector<std::string> names;     // вектор строк
std::vector<std::vector<int>> grid; // вектор векторов (двумерная таблица)
```

Про шаблоны подробно — в главе 20. Сейчас достаточно знать: `Container<T>` — это контейнер, хранящий элементы типа `T`.

Все контейнеры — **РАП-объекты**. Они владеют своими данными, копируются и удаляются корректно, никаких ручных delete не нужно.

14.2 std::vector — главный контейнер

Если бы пришлось выбрать **один** контейнер на всю программу — выбрали бы vector. На моих проектах он покрывает 80% всех случаев хранения данных.

Это **динамический массив**: элементы лежат в памяти подряд, размер может расти. Подключается через <vector>:

```
#include <vector>
```

14.2.1 Создание

```
std::vector<int> a;           // пустой
std::vector<int> b(10);       // 10 элементов, все 0
std::vector<int> c(10, 42);   // 10 элементов, все 42
std::vector<int> d{1, 2, 3, 4, 5}; // initializer list — 5 указанных элементов
std::vector<int> e = {1, 2, 3}; // тоже initializer list
std::vector<int> f(c);         // копия c

std::cout << d.size(); // 5
```

std::vector<int> v(10) и std::vector<int> v{10} — **разные!** Первое — «10 нулей», второе — «один элемент 10» (инициализация-списком). Это та самая ловушка () vs {} из главы 8. Будьте внимательны.

14.2.2 Доступ к элементам

```
std::vector<int> v = {10, 20, 30, 40};

int x = v[0];           // 10 (без проверки границ)
int y = v.at(1);         // 20 (с исключением при выходе за границу)

int first = v.front();   // 10
int last  = v.back();    // 40
```

v[i] быстрее, чем v.at(i) (нет проверки), но опаснее. В новом коде с непроверенными индексами я обычно использую [] плюс ручной assert(i < v.size()) в отладке.

14.2.3 Добавление и удаление

```
std::vector<int> v;

v.push_back(1);    // [1]
v.push_back(2);    // [1, 2]
v.push_back(3);    // [1, 2, 3]

v.pop_back();      // [1, 2]

v.insert(v.begin() + 1, 99);    // [1, 99, 2] — вставить перед позицией 1
v.erase(v.begin());            // [99, 2] — удалить первый

v.clear();              // []
```

`push_back` — **главная** операция, добавление в конец. **Амортизированно $O(1)$** — обычно мгновенно, иногда долго (при росте `capacity`, об этом ниже).

`insert` и `erase` в середине — **$O(n)$** : нужно сдвинуть все элементы, которые идут после. Не используйте `insert` для частых вставок не в конец.

14.2.4 Размер и ёмкость

Здесь начинается самое интересное. У `vector` есть **два** размера:

- **`size()`** — сколько элементов реально хранится.
- **`capacity()`** — сколько элементов может поместиться без новой аллокации памяти.

```
std::vector<int> v;
std::cout << v.size() << " " << v.capacity() << "\n";    // 0 0

v.push_back(1);
std::cout << v.size() << " " << v.capacity() << "\n";    // 1 1

v.push_back(2);
std::cout << v.size() << " " << v.capacity() << "\n";    // 2 2

v.push_back(3);
std::cout << v.size() << " " << v.capacity() << "\n";    // 3 4

v.push_back(4);
```

```
std::cout << v.size() << " " << v.capacity() << "\n"; // 4 4

v.push_back(5);

std::cout << v.size() << " " << v.capacity() << "\n"; // 5 8
```

Размер растёт по 1. Ёмкость — **скачками**: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$. На GCC (libstdc++) и clang (libc++) это умножение на 2, на MSVC — на 1.5. Каждое «уплотнение» — это:

1. Выделить новую память (вдвое больше).
2. Скопировать (или переместить — об этом в главе 19) старые элементы.
3. Уничтожить старые.
4. Освободить старую память.

Это медленно. Но **амортизированная** сложность всё равно $O(1)$: если рост экспоненциальный, то операций «скопировать N элементов» происходит редко (логарифм от размера), и общее время на N вставок — $O(N)$.

14.2.5 reserve и оптимизация

Если **заранее известно**, сколько элементов будет — лучше зарезервировать:

```
std::vector<int> v;
v.reserve(1000);           // capacity = 1000 заранее
for (int i = 0; i < 1000; ++i) {
    v.push_back(i);        // ни одной реаллокации
}
```

reserve(N) гарантирует: первые N push_back пройдут без перевыделения памяти. Хорошая привычка, когда размер можно прикинуть.

shrink_to_fit() — обратное: убрать запас, сделать capacity() == size(). Используется редко, обычно когда хотите вернуть ОС лишнюю память.

14.2.6 Инвалидация итераторов и указателей

Серьёзная ловушка vector. После любой операции, которая может вызвать **перевыделение** памяти (push_back, insert, reserve, ...), все указатели и итераторы на элементы **становятся невалидными**.

```
std::vector<int> v = {1, 2, 3};

int* p = &v[0];           // p указывает на первый элемент
v.push_back(4);
```



```
v.push_back(5);
v.push_back(6);           // одно из этих push_back вызвало реаллокацию

*p = 99;                   // UB! p указывает на старую освобождённую память
```

То же с итераторами. То же с range-based for:

```
std::vector<int> v = {1, 2, 3};
for (int& x : v) {
    if (x == 2) {
        v.push_back(99);    // UB! Меняем контейнер во время итерации
    }
}
```

Правило: пока вы итерируете по вектору, **не меняйте его** — не добавляйте, не удаляйте, не вызывайте `reserve`. Если нужно — итерируйте по индексам и заранее запоминайте размер:

```
size_t n = v.size();
for (size_t i = 0; i < n; ++i) {
    if (v[i] == 2) {
        v.push_back(99);    // OK, n зафиксировано
    }
}
```

Или собирайте «что добавить» в отдельный список, и добавляйте после цикла.

14.2.7 Почему vector обычно быстрее списка

Элементы `vector` лежат в памяти **подряд**. Это значит: - При доступе к `v[i]` адрес вычисляется как `&v[0] + i * sizeof(T)` — одна арифметическая операция. - При проходе подряд процессор подгружает соседние элементы в кэш сразу. Это очень быстро. - Никаких «прыжков» по адресам, никаких разыменований указателей.

Сравните с **связным списком**: каждый элемент — отдельный узел на куче, с указателем на следующий. Чтобы перейти к следующему, нужно прочитать указатель — это обращение в RAM (кэш-промах) → ~200 тактов.

В среднем, проход по `vector<int>` из миллиона элементов — миллисекунды. По `list<int>` — десятки миллисекунд. Разница в 10-50 раз. Это про **кэш-дружелюбность**, о которой я говорил в главе 1.

Поэтому современный C++ совет: **используйте vector по умолчанию**. Список бери только если есть очень особые причины.

14.2.8 Удаление-стирание идиома (erase-remove)

Удалить **все** элементы, удовлетворяющие условию:

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};
// убрать все чётные

v.erase(
    std::remove_if(v.begin(), v.end(),
        [](int x) { return x % 2 == 0; }),
    v.end());
// v теперь {1, 3, 5}
```

Это идиома **erase-remove**. `std::remove_if` сдвигает «оставляемые» элементы в начало и возвращает итератор «за концом нужного». `erase` удаляет хвост. Лямбды и алгоритмы — главы 14 и 21, но идиому стоит знать.

В C++20 есть прямой `std::erase_if(v, predicate)`, но в C++11 — только составной приём.

14.3 std::array — массив фиксированного размера

`std::array<T, N>` — обёртка над сырым массивом, но с интерфейсом вектора. Размер **фиксирован на компиляции**, элементы лежат на стеке.

```
#include <array>

std::array<int, 5> a = {1, 2, 3, 4, 5};

std::cout << a.size(); // 5 (всегда константа)
std::cout << a[0];     // 1
a.front();
a.back();

for (int x : a) {
    std::cout << x;
}
```

Когда брать `array`: - Размер заранее известен и не меняется (например, RGB-цвет = `array<unsigned char, 3>`). - Хотим то же удобство, что у `vector`, но без аллокации на куче.

Когда `vector`: - Размер меняется. - Размер известен только в рантайме. - Размер очень большой (не уместится на стеке).

Размер `std::array` — это часть его типа. `std::array<int, 5>` и `std::array<int, 10>` — **разные типы**. Поэтому функция, принимающая `array`, должна знать размер на компиляции. Передача массивов разных размеров — тема шаблонов (глава 20).

14.4 `std::deque` — двусторонняя очередь

`deque` (double-ended queue, читается «дэк») — как `vector`, но эффективно добавляет и удаляет с **обоих концов**:

```
#include <deque>

std::deque<int> d;
d.push_back(1);
d.push_back(2);
d.push_front(0);    // эффективно, в отличие от vector::insert(begin(), ...)
// d = [0, 1, 2]
```

Внутри — список «блоков» (массивов небольшого размера). Поэтому индексация $O(1)$, но элементы **не подряд в памяти** — кэш-эффективность хуже, чем у `vector`.

Когда брать: реальная очередь FIFO, иногда буферы. На практике — редко. `vector` хватает в 90% случаев.

14.5 `std::list` — двусвязный список

`std::list` — двусвязный список (у каждого узла указатели на следующий и предыдущий).

```
#include <list>

std::list<int> l = {1, 2, 3, 4, 5};
l.push_back(6);
l.push_front(0);

// нет индексации! l[2] не компилируется
auto it = l.begin();
std::advance(it, 2);    // перейти на 3-ю позицию
std::cout << *it;      // 3
```

Преимущества: - Вставка/удаление в **середине** — $O(1)$, если у вас уже есть итератор на нужное место. - Итераторы **не инвалидируются** при добавлении/удалении (кроме удалённого).

Недостатки: - Доступ по индексу — $O(N)$, потому что нужно пройти от начала. - Каждый узел — отдельная аллокация на куче (24 байта overhead на узел плюс выравнивание). Огромный overhead для маленьких элементов. - Плохая кэш-эффективность.

Когда брать list: - Очень частые вставки/удаления в середину. - Стабильные итераторы важны.

В моей практике list нужен раз в год. Для большинства случаев — vector.

14.6 std::map — упорядоченный словарь

std::map<K, V> — словарь, отображающий ключи K в значения V. Реализован как **красно-чёрное дерево** (балансированное двоичное дерево поиска).

```
#include <map>

std::map<std::string, int> ages;
ages["Alice"] = 30;
ages["Bob"] = 25;
ages["Charlie"] = 42;

std::cout << ages["Alice"];    // 30
std::cout << ages.size();      // 3

// Проверка наличия
if (ages.count("Alice") > 0) ...      // C++11 способ
if (ages.find("Alice") != ages.end()) ...    // более общий способ

// Удаление
ages.erase("Bob");

// Обход — отсортированно по ключу
for (const auto& pair : ages) {
    std::cout << pair.first << " = " << pair.second << "\n";
}
// "Alice = 30"
// "Charlie = 42"
```

Главные свойства: - **Все операции $O(\log N)$** — find, insert, erase. Это не «быстро», но прилично.
 - **Элементы отсортированы по ключу.** При обходе вы получите их по возрастанию.
 - **Стабильные итераторы:** операции с другими элементами не инвалидируют итератор на элемент.

14.6.1 Ловушка [] на map

map[key] имеет **побочный эффект**:

```
std::map<std::string, int> ages;
ages["Alice"] = 30;
std::cout << ages["Bob"];    // 0, и Bob появился в карте с значением 0!
std::cout << ages.size();    // 2 !
```

Если ключа нет, [] **создаёт** его со значением по умолчанию (для int — 0, для std::string — пустая). Это иногда удобно (счётчики), иногда — опасно.

Безопасный доступ:

```
auto it = ages.find("Bob");
if (it != ages.end()) {
    std::cout << it->second;
} else {
    std::cout << "не найден";
}
```

it->first — ключ, it->second — значение.

Или через at, который кидает исключение:

```
try {
    int x = ages.at("Bob");
} catch (const std::out_of_range&) {
    std::cout << "не найден";
}
```

14.6.2 Вставка с проверкой существования

```
auto result = ages.insert({"Alice", 50});
if (result.second) {
    std::cout << "Добавлен\n";
} else {
```

```
std::cout << "Alice уже была, значение " << result.first->second << "\n";  
}
```

insert возвращает пару: итератор на элемент и bool «была ли вставка».

14.7 std::unordered_map — хеш-таблица

std::unordered_map<K, V> — словарь на основе **хеш-таблицы**. Интерфейс тот же, что у map, но другие свойства:

```
#include <unordered_map>  
  
std::unordered_map<std::string, int> ages;  
ages["Alice"] = 30;  
ages["Bob"] = 25;  
  
std::cout << ages["Alice"]; // 30
```

Главные свойства: - **Поиск/вставка/удаление** — **O(1)** в среднем. **O(N)** в худшем случае (когда хеши плохо распределены, и все элементы попали в одно «ведро»). - **Элементы НЕ отсортированы**. Обход — в произвольном порядке. - **Перехеширование** при росте — инвалидация итераторов (как vector при push_back).

Когда брать unordered_map: - Не нужно отсортированное хранение. - Нужны быстрые операции.

Когда map: - Нужен отсортированный обход. - Нужно «найти первый элемент с ключом $\geq X$ » — есть lower_bound. - Хочется стабильное поведение в худшем случае (map гарантированно $O(\log N)$, unordered_map может стать $O(N)$).

В большинстве случаев берите unordered_map — он быстрее.

14.7.1 Ключи в unordered_map

Чтобы тип K мог быть ключом unordered_map, для него должна быть определена **функция хеширования**. Для стандартных типов (int, std::string, double) она уже есть. Для своих классов придётся определять — std::hash<MyClass>::operator(). Об этом коротко в главе 20.

14.7.2 Что внутри unordered_map

Это массив «**вёдер**» (buckets). Хеш-функция превращает ключ в число; число делится по модулю на количество вёдер, получается индекс. В этом ведре лежит список (обычно односвязный) всех элементов с тем же индексом.

Если все элементы попадают в одно ведро — поиск становится $O(N)$, потому что нужно пройти по списку. Хорошая хеш-функция распределяет равномерно, чтобы в каждом ведре было ~ 1 элемент.

14.8 `std::set` и `std::unordered_set`

Те же, что `map/unordered_map`, но **без значения** — только ключи. То есть **множество**.

```
#include <set>
#include <unordered_set>

std::set<std::string> taken_names;
taken_names.insert("Alice");
taken_names.insert("Bob");

if (taken_names.count("Charlie") == 0) {
    std::cout << "Charlie свободно\n";
}

std::unordered_set<int> visited;
visited.insert(5);
visited.insert(10);
```

Свойства: - `set` — отсортированное, $O(\log N)$, на дереве. - `unordered_set` — без сортировки, $O(1)$ в среднем, на хеш-таблице.

Когда брать: проверка «есть ли», подсчёт уникальных, удаление дубликатов.

Простой алгоритм «убрать дубликаты из вектора»:

```
std::vector<int> v = {1, 2, 2, 3, 3, 3, 4};
std::set<int> unique_set(v.begin(), v.end());
std::vector<int> unique(unique_set.begin(), unique_set.end());
// unique = {1, 2, 3, 4}, отсортированный
```

Через `unordered_set` — было бы быстрее, но порядок утратился бы.

14.9 Итераторы — кратко

Все контейнеры поддерживают **итераторы** — обобщённый способ обхода. Это будем подробно в главе 14, но базово:

```
std::vector<int> v = {10, 20, 30};

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";
}
```

- `v.begin()` — итератор на первый элемент.
- `v.end()` — итератор «за последний», `sentinel`.
- `++it` — следующий элемент.
- `*it` — значение по итератору.

`range-based for` под капотом — это `for` с итераторами. Тот же код:

```
for (int x : v) {
    std::cout << x << "\n";
}
```

— компилируется в эквивалент классического `for (auto it = v.begin(); it != v.end(); ++it)`. Поэтому `range-for` работает для всего, у чего есть `begin()` и `end()`.

В главе 14 разберём, как использовать итераторы с алгоритмами.

14.10 `std::pair` и `std::tuple`

Малые «контейнеры» для группировки разнородных значений.

`std::pair<A, B>` — пара:

```
std::pair<std::string, int> entry{"Alice", 30};
std::cout << entry.first << ": " << entry.second;
```

Используется внутри `map` (элементы `map` — это `pair<key, value>`).

`std::tuple<A, B, C, ...>` — кортеж из нескольких значений (любого количества):

```
#include <tuple>

std::tuple<std::string, int, double> t{"Alice", 30, 5.7};
std::cout << std::get<0>(t) << " " << std::get<1>(t) << " " << std::get<2>(t);
```

В C++17 появились `structured bindings` (`auto [name, age, height] = t;`), но в C++11 — через `std::get<N>(t)` или `std::tie(name, age, height) = t`.

`Tuple` используется нечасто; обычно лучше определить именованную `struct`.

14.11 Сравнительная таблица

Контейнер	Доступ по индексу	Вставка в конец	Вставка в начало	Вставка в середину	Удаление в середине	Память
vector	$O(1)$	$O(1)$ аморт.	$O(N)$	$O(N)$	$O(N)$	компактно, на куче
array	$O(1)$	—	—	—	$O(N)$	компактно, обычно на стеке
deque	$O(1)$	$O(1)$	$O(1)$	$O(N)$	$O(N)$	блочко, на куче
list	—	$O(1)$	$O(1)$	$O(1)$	$O(N)$	разрозненно, с итератором на куче
map	—	—	—	$O(\log N)$	$O(\log N)$	дерево на куче
unordered_map	—	—	—	$O(1)$	$O(1)$	хеш-среднеесреднее на куче
set	—	—	—	$O(\log N)$	$O(\log N)$	дерево на куче
unordered_set	—	—	—	$O(1)$	$O(1)$	хеш-среднеесреднее на куче

14.12 Решающее дерево: какой контейнер брать

Нужна последовательность с порядком?

└─ Да: упорядочена и фиксирована? → array

| упорядочена, размер меняется? → vector

- | часто вставляем и в начало, и в конец? → deque
- | очень частые вставки/удаления в середину? → list
- └ Нет: нужны пары ключ-значение?
 - | Да: нужен порядок ключей? → map
 - | только быстрый доступ? → unordered_map
 - └ Нет (только уникальные элементы):
 - нужен порядок? → set
 - только быстрый доступ? → unordered_set

По умолчанию — vector или unordered_map. Они покрывают 90% случаев и в среднем самые быстрые благодаря кэш-эффективности и хешированию соответственно.

14.13 Распространённые операции

14.13.1 Найти элемент

```
std::vector<int> v = {1, 2, 3, 4, 5};

auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) {
    std::cout << "Found at index " << (it - v.begin()) << "\n";
}
```

std::find из <algorithm> — работает с любым контейнером.

В map / set — встроенный find быстрее, потому что использует структуру (дерево или хеш):

```
auto it = ages.find("Alice");
```

14.13.2 Сортировка

```
std::vector<int> v = {5, 2, 4, 1, 3};
std::sort(v.begin(), v.end());
// v = {1, 2, 3, 4, 5}

// По убыванию:
std::sort(v.begin(), v.end(), std::greater<int>());
// v = {5, 4, 3, 2, 1}
```

std::sort — алгоритм из главы 14. Работает для контейнеров с произвольным доступом (vector, deque, array). Не работает для list — у списка свой sort метод.

14.13.3 Сумма

```
std::vector<int> v = {1, 2, 3, 4, 5};
int total = std::accumulate(v.begin(), v.end(), 0);
// 15
```

Третий аргумент — начальное значение. Можно использовать для произвольной операции (не только сложение), но это уже глава 14.

14.13.4 Подсчёт

```
std::vector<int> v = {1, 2, 2, 3, 2, 4};
int count = std::count(v.begin(), v.end(), 2);
// 3
```

14.13.5 Уникальные значения

```
std::vector<int> v = {1, 2, 2, 3, 3, 3, 4};
std::sort(v.begin(), v.end());
v.erase(std::unique(v.begin(), v.end()), v.end());
// v = {1, 2, 3, 4}
```

`std::unique` работает только на **отсортированных** — поэтому сначала `sort`.

14.14 Большой пример: подсчёт частоты слов

Соберём программу, читающую текст и выводящую самые частые слова:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <unordered_map>
#include <string>
#include <algorithm>

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: " << argv[0] << " <file>\n";
        return 1;
    }
}
```

```
}

std::ifstream file(argv[1]);
if (!file.is_open()) {
    std::cerr << "Cannot open " << argv[1] << "\n";
    return 1;
}

// Считаем частоту
std::unordered_map<std::string, int> freq;
std::string word;
while (file >> word) {
    // Приведём к нижнему регистру (только ASCII – упрощение)
    for (char& c : word) {
        c = std::tolower(static_cast<unsigned char>(c));
    }
    // Уберём знаки препинания (упрощённо)
    word.erase(
        std::remove_if(word.begin(), word.end(),
            [](char c) { return !std::isalpha(static_cast<unsigned char>(c)); }),
        word.end()
    );
    if (!word.empty()) {
        freq[word] = freq[word] + 1;
    }
}

// Переложим в вектор и отсортируем по убыванию частоты
std::vector<std::pair<std::string, int>> pairs(freq.begin(), freq.end());
std::sort(pairs.begin(), pairs.end(),
    [](const std::pair<std::string, int>& a,
        const std::pair<std::string, int>& b) {
        return a.second > b.second;
    });

// Вывод первых 10
int shown = 0;
for (const auto& p : pairs) {
```

```
std::cout << p.first << ": " << p.second << "\n";
if (++shown >= 10) break;
}

return 0;
}
```

Использовано: - `std::unordered_map<std::string, int>` для подсчёта (быстро, без сортировки).
- `std::vector<std::pair<...>>` для переноса в сортируемый формат. - `std::sort` с лямбдой-компаратором (глава 14 + 21). - `std::remove_if` для очистки от пунктуации. - `range-based for` с `const auto&`. - `argv` для имени файла.

Сборка:

```
$ g++ -std=c++11 -Wall -Wextra freq.cpp -o freq
$ ./freq book.txt
```

14.15 Главные правила главы

1. По умолчанию — `std::vector` для последовательностей, `std::unordered_map` для словарей.
2. `reserve` перед циклом `push_back`, если знаете размер.
3. Не модифицируйте контейнер во время `range-based for` — UB.
4. `map::[]` создаёт элемент, если его не было. Используйте `find` или `at` для безопасного чтения.
5. `list` нужен очень редко. Не давайте в это «теории алгоритмов».
6. Помните про инвалидацию итераторов при изменении контейнера.
7. `const T&` в `range-for` для больших типов, без копий.

14.16 Маленькое упражнение

1. Прочитайте N чисел из `stdin` в `std::vector<int>`. Найдите минимум, максимум, среднее.
2. Прочитайте слова и выведите их по алфавиту, без дубликатов.
3. Имея `std::vector<int>`, удалите все отрицательные.
4. Имея `std::map<std::string, int>` оценок учеников, найдите ученика с максимальной оценкой.
5. Прочитайте имена из файла. Выведите частоту каждого имени, отсортированную по убыванию.

6. (Сложнее) У вас граф, представленный как `std::unordered_map<int, std::vector<int>>` (соседи каждой вершины). Реализуйте BFS — поиск кратчайшего пути от вершины 0 до вершины N.

14.17 Что дальше

Глава 14 — алгоритмы STL (`sort`, `find`, `transform`, `accumulate`, ...) и итераторы подробно. Многие из них мы уже видели мельком; теперь они получают своё место. После 14-й главы Часть I закрыта, и мы готовы к **Части II — RPG-проекту**, где все эти инструменты заработают вместе на реальной программе.

Глава 15

Глава 14. Алгоритмы и итераторы

В прошлой главе мы освоили контейнеры — где хранить данные. Эта глава — про то, что с ними **делать**: искать, сортировать, преобразовывать, сворачивать. Всё это уже написано в стандартной библиотеке через `<algorithm>` и `<numeric>`. Самим алгоритмы писать почти никогда не нужно — `std::sort` от Google написан умнее, чем любая реализация на коленке за вечер. Ваша работа — научиться этими готовыми штуками пользоваться.

После этой главы Часть I закрывается, и мы стартуем RPG-проект.

15.1 Зачем алгоритмы — снова про переиспользование

Допустим, у вас есть `std::vector<int>` и нужно найти, есть ли там значение 42. Можно написать вручную:

```
bool found = false;
for (size_t i = 0; i < v.size(); ++i) {
    if (v[i] == 42) {
        found = true;
        break;
    }
}
```

Или короче:

```
auto it = std::find(v.begin(), v.end(), 42);
bool found = (it != v.end());
```

Второе короче, читается яснее, и не зависит от того, `vector` у вас или `list` или `array`. `std::find` работает с **любым** контейнером, у которого есть итераторы. Это и есть главная идея STL — **алгоритмы не привязаны к контейнерам**.

15.2 Итераторы — обобщённый указатель

В прошлой главе я обещал детали по итераторам. Итак.

Итератор — это объект, который ведёт себя как указатель на элемент контейнера: - `*it` — получить значение. - `++it` — перейти к следующему. - `it1 == it2 / it1 != it2` — сравнить.

В разных контейнерах итераторы устроены по-разному: - В `vector` — это просто `T*` (или почти). - В `list` — указатель на узел. - В `map` — указатель на узел дерева.

Но интерфейс одинаков. Алгоритм типа `std::find` пишется в общем виде через шаблоны:

```
template <typename Iterator, typename Value>
Iterator find(Iterator begin, Iterator end, const Value& value) {
    for (; begin != end; ++begin) {
        if (*begin == value) return begin;
    }
    return end;
}
```

(Шаблоны мы подробно разберём в главе 20, но идею вы уже видите.)

Алгоритм не знает, что за контейнер. Знает только, что итератор умеет `++`, `*` и `==`. Поэтому один и тот же `std::find` работает с `vector`, `list`, `map`, и так далее.

15.2.1 Что такое `begin()` и `end()`

`v.begin()` — итератор на **первый** элемент.

`v.end()` — итератор **за последний** (sentinel, «маркер конца»). Это **не последний элемент**, а позиция «после последнего». Разыменовывать `end()` — UB.

Размер контейнера: `end() - begin()` (для случайно-доступных итераторов).

Пустой контейнер: `begin() == end()`.

«Полуоткрытый интервал» `[begin, end)` — стандарт для всех алгоритмов STL. Принимайте как данность.

15.3 Категории итераторов

Не все итераторы одинаково мощные. STL делит их на **категории**, и каждый алгоритм требует определённой категории.

15.3.1 Input iterator

Самый слабый. Можно только **читать**, и только **один раз**: - *it — прочитать. - ++it — перейти к следующему. - Сравнивать.

Пример: итератор `std::istream_iterator` (читает из потока).

Со слабым итератором работают алгоритмы типа `find`, `count` — они один раз проходят и не возвращаются.

15.3.2 Output iterator

Можно только **писать**: - *it = value — записать. - ++it — перейти.

Пример: `std::ostream_iterator` (пишет в поток).

С слабым output итератором работают `copy`, `transform` (пишущие куда-то).

15.3.3 Forward iterator

Можно читать **много раз**, перебирать в одну сторону.

Пример: итераторы `std::forward_list`.

15.3.4 Bidirectional iterator

Прямой плюс возможность идти **назад** (`--it`).

Пример: итераторы `std::list`, `std::map`, `std::set`.

15.3.5 Random access iterator

Самый мощный. Можно прыгать на N позиций (`it + N`, `it - N`), вычислять разность, индексировать.

Пример: итераторы `std::vector`, `std::deque`, `std::array`, сырые указатели.

Со случайно-доступными работает `std::sort`, потому что ему нужно прыгать туда-сюда.

15.3.6 Иерархия

Random access → Bidirectional → Forward → Input/Output
 (vector) (list) (flist) (потоки)

Каждая следующая категория — расширение предыдущей. Random access умеет всё, что Forward, плюс прыжки.

Если алгоритм требует Random access, а у вас Forward — не сработает. Поэтому `std::sort` нельзя применить к `std::list`. У `list` есть свой метод `l.sort()` для этого случая.

В большинстве кода вы не задумываетесь о категориях. Знаете только: «vector и array — Random access, list и map — Bidirectional, остальное реже».

15.4 Алгоритмы без модификации

Не меняют контейнер, только читают.

15.4.1 find — найти первый

```
std::vector<int> v = {1, 2, 3, 4, 5};

auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) {
    int index = it - v.begin(); // для random access
    std::cout << "Found at index " << index << "\n";
}
```

15.4.2 find_if — найти с предикатом

«Предикат» — это функция или лямбда, возвращающая `bool`. Найти первый элемент, удовлетворяющий условию:

```
auto it = std::find_if(v.begin(), v.end(),
                      [](int x) { return x > 3; });
if (it != v.end()) {
    std::cout << "Первый > 3: " << *it << "\n"; // 4
}
```

`[](int x) { return x > 3; }` — это **лямбда**, безымянная функция. Глава 21 — детально. Сейчас можно читать как «функция, принимающая `int` и возвращающая, больше ли он трёх».

15.4.3 count и count_if

```
std::vector<int> v = {1, 2, 2, 3, 2, 4};

int n = std::count(v.begin(), v.end(), 2);    // 3 – сколько двоек
int m = std::count_if(v.begin(), v.end(),
                      [](int x) { return x > 2; });    // 2 – сколько > 2
```

15.4.4 all_of, any_of, none_of

Проверки «все», «хоть один», «ни один»:

```
bool all_positive = std::all_of(v.begin(), v.end(),
                                [](int x) { return x > 0; });
bool has_negative = std::any_of(v.begin(), v.end(),
                                [](int x) { return x < 0; });
bool no_zeros     = std::none_of(v.begin(), v.end(),
                                 [](int x) { return x == 0; });
```

15.4.5 min_element и max_element

Найти минимальный и максимальный:

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

auto min_it = std::min_element(v.begin(), v.end());
auto max_it = std::max_element(v.begin(), v.end());

std::cout << "Min: " << *min_it << "\n";    // 1
std::cout << "Max: " << *max_it << "\n";    // 9
```

std::min/std::max — это просто между двумя значениями:

```
int x = std::min(5, 7);    // 5
int y = std::max({5, 3, 9, 2});    // 9 – из списка (C++11)
```

15.4.6 equal — сравнить два диапазона

```
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {1, 2, 3};
```

```
bool same = std::equal(a.begin(), a.end(), b.begin()); // true
```

15.5 Алгоритмы с модификацией

Меняют содержимое контейнера.

15.5.1 fill — заполнить значением

```
std::vector<int> v(10);  
std::fill(v.begin(), v.end(), 42);  
// v = {42, 42, 42, 42, 42, 42, 42, 42, 42, 42}
```

15.5.2 iota — заполнить последовательно

```
#include <numeric>  
  
std::vector<int> v(5);  
std::iota(v.begin(), v.end(), 10);  
// v = {10, 11, 12, 13, 14}
```

«iota» — старое слово, означает «единица» (греческая буква йота). Алгоритм заполняет начиная с указанного значения с шагом 1.

15.5.3 transform — применить функцию

Применить функцию к каждому элементу:

```
std::vector<int> v = {1, 2, 3, 4};  
std::vector<int> doubled(v.size());  
  
std::transform(v.begin(), v.end(),  
               doubled.begin(),  
               [](int x) { return x * 2; });  
// doubled = {2, 4, 6, 8}
```

Параметры: начало входа, конец входа, начало выхода, функция.

Можно писать в **тот же** контейнер:

```
std::transform(v.begin(), v.end(),
               v.begin(),
               [](int x) { return x * 2; });
// v = {2, 4, 6, 8}
```

15.5.4 copy и copy_if

Копировать элементы:

```
std::vector<int> src = {1, 2, 3, 4, 5};
std::vector<int> dst(5);

std::copy(src.begin(), src.end(), dst.begin());
// dst = {1, 2, 3, 4, 5}
```

С предикатом — только подходящие:

```
std::vector<int> src = {1, 2, 3, 4, 5, 6};
std::vector<int> evens;

std::copy_if(src.begin(), src.end(),
             std::back_inserter(evens),
             [](int x) { return x % 2 == 0; });
// evens = {2, 4, 6}
```

`std::back_inserter(evens)` — особый итератор, который при записи вызывает `evens.push_back()`. Если бы мы написали `evens.begin()`, было бы UB — мы бы писали в пустой вектор.

15.5.5 remove и remove_if

Хитрая семантика: эти алгоритмы **не удаляют** элементы (контейнер они не знают). Они **сдвигают** «оставляемые» в начало и возвращают итератор «нового конца». Хвост остаётся в неопределённом состоянии.

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto new_end = std::remove(v.begin(), v.end(), 3);
// v = {1, 2, 4, 5, ?}   (5 элементов всё ещё, последний — мусор)
// new_end указывает на позицию «после 5»

v.erase(new_end, v.end());
// v = {1, 2, 4, 5}
```

Эта пара `remove` + `erase` — идиома **erase-remove**, мы её видели в главе 13. Запомните как одно действие.

С предикатом — `remove_if`:

```
v.erase(
    std::remove_if(v.begin(), v.end(),
        [](int x) { return x < 0; }),
    v.end()
);
```

15.5.6 reverse и rotate

```
std::vector<int> v = {1, 2, 3, 4, 5};

std::reverse(v.begin(), v.end());
// v = {5, 4, 3, 2, 1}

std::rotate(v.begin(), v.begin() + 2, v.end());
// «провернуть» так, чтобы элемент с позиции 2 стал началом
```

15.5.7 sort и связанные

Главный алгоритм сортировки. Использует introsort (комбинацию quicksort, heapsort и insertion sort), $O(N \log N)$ в среднем и в худшем случае.

```
std::vector<int> v = {5, 2, 4, 1, 3};

std::sort(v.begin(), v.end());
// v = {1, 2, 3, 4, 5}

// По убыванию через компаратор:
std::sort(v.begin(), v.end(),
    [](int a, int b) { return a > b; });
// v = {5, 4, 3, 2, 1}

// Сортировка строк по длине:
std::vector<std::string> words = {"apple", "hi", "world", "a"};
std::sort(words.begin(), words.end(),
    [](const std::string& a, const std::string& b) {
```

```

        return a.size() < b.size();
    });
// words = {"a", "hi", "apple", "world"}

```

Компаратор — это **двухаргументная функция**, возвращающая `true`, если первый «меньше» второго.

`std::stable_sort` — то же, но сохраняет относительный порядок равных элементов. Чуть медленнее, но иногда важно.

`std::partial_sort` — отсортировать только первые `N`.

`std::nth_element` — поставить `N`-й элемент на нужное место (как если бы массив был отсортирован). Быстрее, чем полная сортировка, если нужен только медиана или `N`-й.

15.5.8 `unique`

Убрать **подряд идущие** дубликаты:

```

std::vector<int> v = {1, 1, 2, 2, 3, 1, 1};

auto new_end = std::unique(v.begin(), v.end());
v.erase(new_end, v.end());
// v = {1, 2, 3, 1} ← одиночка 1 в конце осталась!

```

Чтобы убрать **все** дубликаты, сначала отсортируйте:

```

std::vector<int> v = {3, 1, 2, 1, 3, 2};
std::sort(v.begin(), v.end());
// v = {1, 1, 2, 2, 3, 3}
v.erase(std::unique(v.begin(), v.end()), v.end());
// v = {1, 2, 3}

```

15.6 Численные алгоритмы

Из заголовка `<numeric>`:

15.6.1 `accumulate`

Свернуть в одно значение:

```

#include <numeric>

```

```
std::vector<int> v = {1, 2, 3, 4, 5};
int sum = std::accumulate(v.begin(), v.end(), 0);
// 15
```

Третий аргумент — начальное значение. Тип результата равен типу начального значения. Поэтому осторожно:

```
std::vector<double> v = {1.5, 2.5, 3.5};
auto bad = std::accumulate(v.begin(), v.end(), 0);    // int! результат 6, не 7.5
auto good = std::accumulate(v.begin(), v.end(), 0.0); // double! 7.5
```

Можно задать **операцию**:

```
// Произведение
int product = std::accumulate(v.begin(), v.end(), 1,
                             std::multiplies<int>());
// 120 (= 1*2*3*4*5)

// Через лямбду
int sum_of_squares = std::accumulate(v.begin(), v.end(), 0,
                                     [](int acc, int x) {
                                         return acc + x * x;
                                     });
// 55 (= 1 + 4 + 9 + 16 + 25)
```

В общем `accumulate(first, last, init, op)` делает: начать с `init`, для каждого элемента `e` сделать `result = op(result, e)`. Это **fold**, тот самый из функциональных языков.

15.6.2 inner_product

Скалярное произведение двух последовательностей:

```
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {4, 5, 6};
int dot = std::inner_product(a.begin(), a.end(), b.begin(), 0);
// 32 (= 1*4 + 2*5 + 3*6)
```

15.6.3 partial_sum

Префиксная сумма:


```
std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> sums(v.size());
std::partial_sum(v.begin(), v.end(), sums.begin());
// sums = {1, 3, 6, 10, 15}
```

15.6.4 adjacent_difference

Обратная — разности соседних:

```
std::vector<int> sums = {1, 3, 6, 10, 15};
std::vector<int> diffs(sums.size());
std::adjacent_difference(sums.begin(), sums.end(), diffs.begin());
// diffs = {1, 2, 3, 4, 5}
```

15.7 Перестановки

```
std::vector<int> v = {1, 2, 3};

do {
    for (int x : v) std::cout << x;
    std::cout << "\n";
} while (std::next_permutation(v.begin(), v.end()));
```

Выведет все 6 перестановок. Удобно для перебора всех вариантов.

15.8 Бинарный поиск

На **отсортированных** диапазонах:

```
std::vector<int> v = {1, 3, 5, 7, 9, 11};

bool present = std::binary_search(v.begin(), v.end(), 7); // true

auto it = std::lower_bound(v.begin(), v.end(), 6); // первый >= 6 → итератор на 7
auto it2 = std::upper_bound(v.begin(), v.end(), 7); // первый > 7 → итератор на 9
```

$O(\log N)$. На несортированных данных результат непредсказуем.

15.9 Лямбды и предикаты — короткое введение

Я уже много раз писал лямбды. Скажу о них минимум, чтобы освоиться. Подробно — в главе 21.

Лямбда — это безымянная функция, объявленная «на месте»:

```
auto is_even = [](int x) { return x % 2 == 0; };
std::cout << is_even(4);    // 1
std::cout << is_even(5);    // 0
```

Синтаксис: [capture](параметры) { тело }. Возвращаемый тип компилятор выводит сам.

[] — пустой **захват** (capture). Лямбда может «захватить» переменные из окружения:

```
int threshold = 10;
auto big_enough = [threshold](int x) { return x > threshold; };

std::vector<int> v = {3, 12, 7, 15, 1};
int count = std::count_if(v.begin(), v.end(), big_enough);
// 2 (12 и 15)
```

[threshold] — копировать threshold в лямбду по значению. [&threshold] — ссылка на оригинал (если изменится — лямбда увидит). [=] — копировать все используемые переменные. [&] — все по ссылке.

Это короткие безымянные функции, идеально подходящие для алгоритмов. Подробнее — глава 21.

15.9.1 Функтор — старый стиль

До C++11 вместо лямбд использовали **функторы** — классы с перегруженным operator():

```
struct IsBigger {
    int threshold;
    IsBigger(int t) : threshold(t) {}
    bool operator()(int x) const { return x > threshold; }
};

IsBigger pred(10);
int count = std::count_if(v.begin(), v.end(), pred);
```

В современном коде функторы редкость, лямбды короче. Но в чужом коде встречаются.

15.10 Алгоритмы vs методы контейнеров

Иногда один и тот же алгоритм есть и как свободная функция, и как метод контейнера. **Какой брать?**

Правило: **если у контейнера есть свой метод — используйте его**. Он знает структуру и работает быстрее.

Примеры:

Алгоритм	Свободная функция (медленнее)	Метод контейнера (быстрее)
Сортировка list	не работает	<code>l.sort()</code>
Поиск в map	<code>std::find</code> — $O(N)$	<code>m.find(key)</code> — $O(\log N)$
Поиск в un- ordered_map	<code>std::find</code> — $O(N)$	<code>m.find(key)</code> — $O(1)$
Поиск в set	<code>std::find</code> — $O(N)$	<code>s.find(value)</code> — $O(\log N)$

`std::find` для vector/array — нормально, $O(N)$ всё равно. Для map/set — глупо, у них свой `find` использует дерево.

15.11 Удалить элемент из map во время прохода

Хитрая операция:

```
std::map<std::string, int> m = {...};

// Так нельзя — итератор инвалидируется при erase
for (auto it = m.begin(); it != m.end(); ++it) {
    if (it->second < 0) {
        m.erase(it);    // UB: it после этого мёртв
    }
}

// Правильно — `erase` возвращает следующий итератор
for (auto it = m.begin(); it != m.end(); ) {
    if (it->second < 0) {
        it = m.erase(it);    // двигаемся через возврат
    } else {
        ++it;
    }
}
```

```

    }
}

```

Для `vector` — лучше через `erase-remove` идиому.

15.12 Свой `for_each` — попытка номер один

Зная итераторы и лямбды, можем написать свой простой `for_each`:

```

template <typename Iterator, typename Function>
void my_for_each(Iterator first, Iterator last, Function f) {
    for (; first != last; ++first) {
        f(*first);
    }
}

int main() {
    std::vector<int> v = {1, 2, 3, 4};
    my_for_each(v.begin(), v.end(),
        [](int x) { std::cout << x << " "; });
    // 1 2 3 4
}

```

Шаблон обобщает тип итератора и функции. Алгоритмы STL устроены примерно так же, только сложнее (есть проверки, оптимизации, специализации).

15.13 Производительность

Несколько практических замечаний.

15.13.1 Алгоритмы STL часто оптимизированнее ручных

Реализация `std::sort` в современных компиляторах — десятки тысяч строк, с микро-оптимизациями под архитектуру. Ваш ручной `quicksort` скорее всего будет в 2-5 раз медленнее.

Поэтому: **используйте STL для очевидно стандартных задач**. Не переписывайте `sort` или `find_if`, если нет особых причин.

15.13.2 Алгоритмы вызывают функцию для каждого элемента

В цикле:

```
for (int x : v) {  
    process(x);  
}
```

process вызывается на каждый элемент — нет оверхеда. Через `std::for_each`:

```
std::for_each(v.begin(), v.end(), [](int x) { process(x); });
```

Лямбда тоже вызывается на каждый элемент, плюс лямбда создаётся как объект. Компилятор обычно инлайнит, и оверхед нулевой. Но в общем — не страшно.

15.13.3 Когда STL медленнее ручного

Очень редко. Бывает в случаях: - Хочется ранний выход из алгоритма, которого нет в STL. - Особая структура данных, где STL не знает. - Нужно несколько преобразований за один проход (STL делает их за несколько).

В 99% случаев STL — не узкое место.

15.14 Большой пример: статистика оценок

Собирающий пример. У нас оценки учеников; нужно посчитать среднее, медиану, отклонение, найти лучших, отсортировать.

```
#include <iostream>  
#include <vector>  
#include <algorithm>  
#include <numeric>  
#include <cmath>  
#include <string>  
  
struct Student {  
    std::string name;  
    int score;  
};  
  
double average(const std::vector<int>& scores) {  
    if (scores.empty()) return 0.0;  
    double sum = std::accumulate(scores.begin(), scores.end(), 0.0);  
    return sum / scores.size();  
}
```

```
double median(std::vector<int> scores) {    // копия — модифицируем
    if (scores.empty()) return 0.0;
    std::sort(scores.begin(), scores.end());
    size_t n = scores.size();
    if (n % 2 == 1) {
        return scores[n / 2];
    } else {
        return (scores[n / 2 - 1] + scores[n / 2]) / 2.0;
    }
}

double stddev(const std::vector<int>& scores) {
    if (scores.size() < 2) return 0.0;
    double mean = average(scores);
    double sum_sq = std::accumulate(scores.begin(), scores.end(), 0.0,
                                     [mean](double acc, int x) {
                                         double diff = x - mean;
                                         return acc + diff * diff;
                                     });
    return std::sqrt(sum_sq / (scores.size() - 1));
}

int main() {
    std::vector<Student> students = {
        {"Alice", 85},
        {"Bob", 72},
        {"Charlie", 90},
        {"Dave", 65},
        {"Eve", 78},
        {"Frank", 95},
        {"Grace", 58},
    };

    // Выдернем оценки в отдельный вектор
    std::vector<int> scores(students.size());
    std::transform(students.begin(), students.end(),
                  scores.begin(),
```

```

        [](const Student& s) { return s.score; });

std::cout << "Учеников: " << students.size() << "\n";
std::cout << "Среднее: " << average(scores) << "\n";
std::cout << "Медиана: " << median(scores) << "\n";
std::cout << "Ст. отклонение: " << stddev(scores) << "\n";

auto max_it = std::max_element(scores.begin(), scores.end());
auto min_it = std::min_element(scores.begin(), scores.end());
std::cout << "Лучшая оценка: " << *max_it << "\n";
std::cout << "Худшая: " << *min_it << "\n";

// Сколько прошло порог 70
int passed = std::count_if(scores.begin(), scores.end(),
                           [](int s) { return s >= 70; });
std::cout << "Прошли (>=70): " << passed << "\n";

// Отсортированный по убыванию рейтинг
std::sort(students.begin(), students.end(),
          [](const Student& a, const Student& b) {
              return a.score > b.score;
          });

std::cout << "\nРейтинг:\n";
int rank = 1;
for (const auto& s : students) {
    std::cout << rank++ << ". " << s.name << " (" << s.score << ")\n";
}

return 0;
}

```

Соберите:

```

$ g++ -std=c++11 -Wall -Wextra stats.cpp -o stats
$ ./stats

```

Использовано: `accumulate` (с лямбдой для квадратов отклонений), `transform` (выдернуть поле из структур), `min_element/max_element`, `count_if`, `sort` с лямбдой-компаратором.

Это **идиоматичный** современный C++. Никаких ручных циклов для поиска max/min, для подсчёта, для сортировки. Все стандартное.

15.15 Главные правила

1. Для известной задачи — берите алгоритм STL. Не пишите свой sort или find.
2. Для map/set/unordered_* — find метод контейнера, не std::find.
3. erase-remove для удаления из вектора по предикату.
4. accumulate с типом начального значения — 0 для int, 0.0 для double.
5. Лямбды с захватом по ссылке ([&]) — только когда понимаете, что переменная переживёт лямбду.
6. На отсортированных — бинарный поиск (binary_search, lower_bound).
7. Помните инвалидацию итераторов при erase — используйте возвращаемое значение.

15.16 Маленькое упражнение

1. Дан std::vector<int>. Найдите сумму всех **положительных** элементов. Используйте accumulate с лямбдой.
2. Дан std::vector<std::string>. Отсортируйте по длине, при равных длинах — лексикографически.
3. Дан std::vector<int>. Используя transform, постройте новый вектор, где каждый элемент — квадрат исходного.
4. Дан std::vector<int>. Используя partial_sum, постройте префиксные суммы. Используя их, ответьте на запрос «сумма от индекса L до R включительно» за O(1).
5. Дан std::vector<std::pair<std::string, int>> — оценки. Найдите имя ученика с максимальной оценкой через max_element с компаратором.
6. (Сложнее) Дан std::vector<int>. Сделайте функцию, которая удаляет все дубликаты, но **сохраняет первоначальный порядок** (то есть sort + unique не подходит).

15.17 Часть I закрыта

На этом Часть I заканчивается. Перечислю кратко, что у вас теперь есть:

- **Глава 8:** типы и переменные. int, double, bool, char, const, constexpr, auto.
- **Глава 9:** управляющие конструкции. if/else, switch, while, for, range-based for.
- **Глава 10:** функции. Объявление/определение, передача по значению/ссылке/указателю, перегрузка, рекурсия.

- **Глава 11:** память. Стек/куча, new/delete, RAII — главный паттерн C++.
- **Глава 12:** строки и I/O. `std::string`, `cin/cout`/файловые потоки, `stringstream`.
- **Глава 13:** контейнеры. `vector`, `array`, `list`, `map`, `unordered_map`, `set`.
- **Глава 14:** алгоритмы и итераторы. `find`, `sort`, `transform`, `accumulate`, лямбды.

Этого хватит, чтобы писать осмысленные программы. Но это всё ещё «низший уровень» C++ — мы пока не делали свои классы, не пользовались наследованием, не работали с умными указателями. Это всё в Части II.

15.18 Что дальше

Часть II — RPG-проект `demo-rpg/`. Мы начинаем писать большую программу: текстовая игра с локациями, инвентарём, NPC, боями, сохранениями. Все знания Части I заработают вместе, плюс мы введём: - Классы и объекты (глава 16). - Наследование и виртуальные функции (глава 17). - Smart pointers `unique_ptr/shared_ptr/weak_ptr` (глава 18). - Move-семантику и `rvalue`-ссылки (глава 19). - Шаблоны функций и классов (глава 20). - Лямбды и `std::function` (глава 21) — после кратких намёков. - Исключения и `exception safety` (глава 22). - Парсер скриптов локаций (глава 23). - Сохранение/загрузку (глава 24).

Сначала — глава 15: дизайн игры и скелет проекта. Мы разработаем архитектуру, создадим файловую структуру `demo-rpg/`, напомним `Makefile` с автогенерацией зависимостей, и подготовим почву для следующих глав. Это будет существенно отличаться от глав «учебных примеров»: реальная разработка с реальными решениями.

Глава 16

Глава 15. Дизайн игры и скелет проекта

С этой главы начинается **Часть II — RPG-проект**. До сих пор у нас были «учебные программы»: коротенькие файлы, демонстрирующие одну концепцию. Теперь мы строим полноценное приложение — `demo-rpg/`. Игра текстовая, в стиле *Zork*. Десяток локаций, инвентарь, NPC, бои, сохранения. К 25-й главе всё это будет работать и собираться одной командой.

В этой главе — **только подготовка**. Дизайн игры, архитектура классов, файловая структура, сборка, минимальный скелет, который компилируется. Никакого кода игровой логики ещё нет; это начнётся с главы 16. Но фундамент мы зложим прочный.

16.1 Что мы строим

Игрок управляет персонажем, путешествующим по миру. Все действия — текстовыми командами:

```
=== RPG-демо ===
```

```
Введите 'help' для списка команд.
```

```
> look
```

```
Лесная поляна. На западе видна тропа к деревне.
```

```
На востоке — тёмный лес.
```

```
> go east
```

```
Тёмный лес. Здесь стоит гоблин.
```

```
> attack гоблин
```

```
Вы ударили гоблина мечом. Урон 8. У гоблина 4 HP.
```

Гоблин ударил вас дубиной. Урон 3. У вас 17 HP.

```
> attack гоблин
```

Гоблин повержен. Получено 15 опыта.

```
> inventory
```

Инвентарь:

- ржавый меч
- кожаная броня

```
> save game1
```

Игра сохранена в data/saves/game1.sav

```
> quit
```

Что должно быть в финальной версии:

- **Мир** из ~15 локаций. Загружается из текстового файла, не зашит в код.
- **Команды:** look, go <direction>, take <item>, drop <item>, inventory/i, examine <thing>, attack <enemy>, talk <npc>, save <slot>, load <slot>, help, quit.
- **Инвентарь:** предметы можно брать, носить, использовать. Разные типы — оружие, броня, расходники.
- **NPC:** с ними можно говорить, иногда давать или брать предметы.
- **Враги:** бой пошаговый. У игрока и врага HP; кто первым опустился до 0 — проиграл.
- **Квесты:** например, «принеси меч кузнецу — получишь золото».
- **Сохранения:** в файл, по слотам. Можно загрузить в любой момент.

Это не пародия на RPG, а серьёзный игровой движок. Расширять его (новые локации, предметы, квесты) можно правя только текстовые файлы — без перекомпиляции.

16.2 Архитектура: какие классы будут

С прицелом на дальнейшие главы. Здесь сразу зафиксируем «карту» проекта.

Класс	Файлы	Что хранит	Глава
Game	game.{h,cpp}	Главный цикл, владеет миром и игроком.	15 (скелет)
World	world.{h,cpp}	Карта локаций.	16
Location	location.{h,cpp}	Описание комнаты, выходы, предметы, NPC.	16
Player	player.{h,cpp}	Состояние игрока: HP, инвентарь, опыт.	16
Entity (база)	entity.{h,cpp}	Общая база для всего живого.	17
Enemy : Entity	enemy.{h,cpp}	Враги: HP, урон, AI.	17

Класс	Файлы	Что хранит	Глава
NPC : Entity	npc.{h,cpp}	Дружественный персонаж с диалогами.	17
Item (база)	item.{h,cpp}	Базовый предмет.	17
Weapon : Item	weapon.{h,cpp}	Оружие: модификатор урона.	17
Armor : Item	armor.{h,cpp}	Броня: модификатор защиты.	17
Consumable : Item	consumable.{h,cpp}	Расходники: эффект при использовании.	17
Inventory	inventory.{h,cpp}	Управление предметами игрока.	18
EventBus<T>	event_bus.h	Шаблонная шина игровых событий.	20
CommandRegistry	commands.{h,cpp}	Парсер и диспетчер команд.	21
WorldParser	world_parser.{h,cpp}	Чтение мира из data/world.txt.	23
SaveManager	save_manager.{h,cpp}	Сохранения.	24

Может показаться, что классов много. Но каждый отвечает за **одно** дело, и связи между ними понятные. Лучше десять простых классов, чем один «БогОбъект» на тысячу строк.

16.2.1 Связи между классами

Грубо: Game владеет World и Player. World владеет коллекцией Location-ов. Location хранит список Item*-ов и Enemy*/NPC*-ов. Player имеет свой Inventory. Команды (CommandRegistry) при выполнении меняют состояние мира и игрока.

Что нового и страшного для свежее-прошедших Часть I:

- **Item, Weapon, Armor, Consumable** — иерархия классов с **полиморфизмом**. Локация хранит `unique_ptr<Item>`, а внутри может быть конкретный тип. Это глава 17 + 18.
- **EventBus** — шаблон. Глава 20.
- **CommandRegistry** — словарь имени команды на лямбду-обработчик. Глава 21.

Сейчас всего этого нет — у нас только Game с зашитыми двумя командами. По мере глав будем достраивать.

16.3 Файловая структура

Как организовать файлы — большая тема в C++-проектах. Я пользуюсь стандартной для типичных Open Source-проектов структурой:

```
demo-rpg/
├─ Makefile          — сборка
├─ include/          — заголовочные файлы (публичный интерфейс)
│   └─ game.h
│   └─ world.h       (появится в гл. 16)
│   └─ location.h
│   └─ ...
├─ src/              — реализации (.cpp)
│   └─ main.cpp       (вход в программу)
│   └─ game.cpp
│   └─ world.cpp
│   └─ ...
├─ tests/            — тесты (каждый .cpp — отдельная программа)
│   └─ test_smoke.cpp
├─ data/             — данные игры (мир, сейвы)
│   └─ world.txt      (появится в гл. 23)
│   └─ saves/
└─ build/            — собранное (НЕ коммитим в git!)
    └─ rpg            (исполняемый файл игры)
    └─ *.o, *.d       (промежуточные)
    └─ tests/*
```

Несколько правил:

1. **include/ для интерфейсов, src/ для реализаций.** Чёткое разделение. В include/ ничего исполняемого — только декларации (которые попадают через #include в код пользователя).
2. **build/ — артефакты сборки.** В git не коммитится (добавляется в .gitignore). Можно удалить и пересобрать.
3. **data/ — статические данные,** не код. Карта мира, диалоги. Можно править без перекомпиляции.
4. **tests/ — отдельный мир.** Тесты не входят в rpg-бинарь; они собираются отдельными исполняемыми файлами.

Если присмотреться, такая структура — почти стандарт. PostgreSQL, LLVM, SQLite, redis — все так устроены, с вариациями.

16.4 Создаём папки

В корне проекта:

```
$ cd cpp-from-scratch/demo-rpg
$ mkdir -p include src tests data/saves build
```

Файл .gitignore (если работаете под git):

```
build/
*.o
*.d
data/saves/*
!data/saves/.gitkeep
```

Это говорит git’у: не коммитить артефакты сборки, не коммитить пользовательские сейвы (только пустую папку с .gitkeep).

16.5 Makefile

Дальше — сборка. На больших проектах редко пишут команды компиляции руками; обычно используют **систему сборки**. Самая старая и универсальная — make со своими Makefile.

Современные альтернативы — CMake, Bazel, Meson, Ninja. Они мощнее, но make достаточно для нашего проекта, и хорошо иллюстрирует, что система сборки делает под капотом.

Идея make: вы описываете **правила** вида «чтобы сделать X, нужно Y и Z, и команда <cmd>». Make сам строит граф зависимостей, видит, что изменилось, и пересобирает только нужное.

16.5.1 Простой пример

```
build/rpg: src/main.cpp src/game.cpp
    g++ -std=c++11 -Iinclude src/main.cpp src/game.cpp -o build/rpg
```

Прочитайте как: «build/rpg зависит от src/main.cpp и src/game.cpp. Если они новее, чем build/rpg, выполнить команду».

Команда **должна** начинаться с Tab (не пробелов!). Это историческая особенность make, плохо стареющая, но изменить уже нельзя.

С такой простой версией каждое изменение пересобирает **всё**, включая файлы, которые не трогали. На больших проектах — больно.

Лучше: компилировать каждый .cpp в свой .o, потом линковать .o-файлы.

```
build/rpg: build/main.o build/game.o
    g++ build/main.o build/game.o -o build/rpg
```

```
build/main.o: src/main.cpp
    g++ -std=c++11 -Iinclude -c src/main.cpp -o build/main.o

build/game.o: src/game.cpp
    g++ -std=c++11 -Iinclude -c src/game.cpp -o build/game.o
```

Теперь, изменив `src/game.cpp`, мы пересобираем только `game.o` и линкуем. Уже лучше.

Но есть проблема: **зависимости от заголовков**. Если изменился `include/game.h`, нужно пересобрать `main.o` и `game.o` (потому что они включают `game.h`). Make об этом не знает! Он видит зависимости только от того, что мы написали явно. Если мы написали `build/game.o: src/game.cpp` — изменения в `game.h` make пропустит. Будет ошибка несоответствия (старая версия компилировалась с одним интерфейсом, новая — с другим).

16.5.2 -MMD — магия авто-зависимостей

Компиляторы умеют сами генерировать списки зависимостей. Флаг `-MMD` говорит: «при компиляции `.cpp` создавай рядом `.d`-файл со списком `.h`, которые входят».

Пример. Файл `src/game.cpp` подключает `include/game.h` и `<iostream>`. После `g++ -MMD -c src/game.cpp -o build/game.o` рядом появится `build/game.d`:

```
build/game.o: src/game.cpp include/game.h
```

Это **готовое правило make**! Если мы его подключим через `include` в наш Makefile — make узнает все зависимости автоматически.

Это и есть то, ради чего флаг `-MMD` стоит в любом современном проекте. Без него вы либо пишете зависимости вручную (и забываете), либо `make clean` перед каждой сборкой (медленно).

16.5.3 Наш Makefile

Собранный (с пояснениями ниже):

```
CXX      := g++
CXXSTD   := -std=c++11
WARNINGS := -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion
DEBUG    := -g -O0
RELEASE  := -O2 -DNDEBUG

MODE     ?= debug
ifeq ($(MODE),release)
```

```

    OPT := $(RELEASE)
else
    OPT := $(DEBUG)
endif

CXXFLAGS := $(CXXSTD) $(WARNINGS) $(OPT) -Iinclude -MMD -MP
LDFLAGS :=

SRCS      := $(filter-out src/main.cpp, $(wildcard src/*.cpp))
OBSJS     := $(patsubst src/%.cpp, build/%.o, $(SRCS))
DEPS      := $(OBSJS:.o=.d)

all: build/rpg

build/rpg: build/main.o $(OBSJS)
    @mkdir -p build
    $(CXX) $(CXXFLAGS) $^ -o $@ $(LDFLAGS)

build/main.o: src/main.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@

build/%.o: src/%.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
    rm -rf build

-include $(DEPS)

.PHONY: all clean

```

Разберём по частям.

Переменные

```

CXX      := g++
CXXSTD   := -std=c++11

```


`:=` — это **немедленное присваивание** (в отличие от `=`, ленивое). Хорошая практика — всегда `:=` для переменных-констант.

```
WARNINGS := -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion
```

Список флагов предупреждений. Расшифровка: - `-Wall` — большая часть стандартных предупреждений. - `-Wextra` — дополнительные. - `-Wpedantic` — строгое соответствие стандарту C++11. - `-Wshadow` — предупреждать о затенении переменных (когда локальная переменная скрывает внешнюю с тем же именем — типичный баг). - `-Wconversion` — неявные сужающие конверсии (`int` → `char`, `double` → `int` и т. д.). - `-Wsign-conversion` — конверсии `signed` ↔ `unsigned`.

Эти флаги ловят кучу багов. С ними новичкам приходится переписывать код несколько раз, пока компилятор не успокоится — но код потом качественнее.

```
DEBUG      := -g -O0
RELEASE    := -O2 -DNDEBUG
```

Два режима. **Debug**: `-g` (включить символы отладки, чтобы можно было через `gdb/lldb`), `-O0` (никаких оптимизаций, чтобы код и переменные соответствовали исходнику). **Release**: `-O2` (умеренная оптимизация), `-DNDEBUG` (определить макрос `NDEBUG`, который отключает `assert` и подобные проверки).

```
MODE      ?= debug
ifeq ($(MODE),release)
    OPT := $(RELEASE)
else
    OPT := $(DEBUG)
endif
```

Переменная `MODE` — режим сборки. Если не задана пользователем, по умолчанию `debug`. Через `make MODE=release` собираем в `release`. `?` — присваивание «если не задано».

```
CXXFLAGS := $(CXXSTD) $(WARNINGS) $(OPT) -Iinclude -MMD -MP
```

Финальный набор флагов: - стандарт C++11; - предупреждения; - режим (`debug/release`); - `-Iinclude` — добавить `include/` в пути поиска заголовков; - `-MMD` — генерировать `.d`-файлы зависимостей; - `-MP` — добавлять «пустые цели» для заголовков (защита от ошибок, когда удалят файл).

Списки файлов

```
SRCS      := $(filter-out src/main.cpp, $(wildcard src/*.cpp))
OBJS      := $(patsubst src/%.cpp, build/%.o, $(SRCS))
DEPS      := $(OBJS:.o=.d)
```

- `$(wildcard src/*.cpp)` — все `.cpp` в `src/`.
- `$(filter-out X, list)` — убрать `X` из списка. Мы убираем `main.cpp`, потому что он будет отдельно (в правиле для `rpg`).
- `$(patsubst src/%.cpp, build/%.o, ...)` — преобразование путей: `src/foo.cpp` → `build/foo.o`.
- `$(OBJSEX:.o=.d)` — то же, заменяя `.o` на `.d`.

После этих манипуляций: `- SRCS = src/game.cpp src/world.cpp ...` (всё кроме `main`). `- OBJSEX = build/game.o build/world.o ...` `- DEPS = build/game.d build/world.d ...`

Удобство: при добавлении нового `.cpp` ничего не надо править в `Makefile`.

Правила сборки

```
build/rpg: build/main.o $(OBJSEX)
    @mkdir -p build
    $(CXX) $(CXXFLAGS) $^ -o $@ $(LDFLAGS)
```

`build/rpg` зависит от `build/main.o` и всех остальных `OBJSEX`. Если что-то новое — выполняется команда.

В команде: `@mkdir -p build` — `@` означает «не выводить команду в терминал». `-p` для `mkdir` мы знаем из главы 5. `-$^` — все зависимости (`build/main.o build/game.o ...`). `-$@` — имя цели (`build/rpg`).

Это **автоматические переменные** `make`. Удобство — не повторять имена.

```
build/%.o: src/%.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@
```

Шаблонное правило (pattern rule): для любого `build/X.o`, который соответствует `src/X.cpp`, выполнить команду. `-$<` — первая зависимость (то есть конкретный `.cpp`).

Так одно правило обрабатывает все исходники: `game.cpp`, `world.cpp`, `player.cpp` и так далее.

`-include` — подключение зависимостей

```
-include $(DEPS)
```

Это **подключение файлов** в `Makefile`. Дефис в начале означает «не ошибка, если файла нет» (в первый запуск `.d`-файлов ещё нет — это нормально).

После первой сборки в `build/` появятся `game.o`, `main.o` со списками зависимостей. На следующих запусках `make` их прочитает и знает, что и от чего зависит.

.PHONY

```
.PHONY: all clean
```

Объявляет цели «фиктивными» — они не создают файла с именем `all` или `clean`. Без этого `make clean` мог бы перестать работать, если бы в каталоге случайно создался файл `clean`.

Запуск

```
$ make
g++ -std=c++11 -Wall -Wextra ... -c src/main.cpp -o build/main.o
g++ -std=c++11 -Wall -Wextra ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
```

Все исходники компилируются и линкуются. Запустить:

```
$ ./build/rpg
```

При следующем `make` без изменений:

```
$ make
make: Nothing to be done for 'all'.
```

Make видит, что ничего не поменялось, и не делает работу заново.

Изменим `src/game.cpp` (без изменения `game.h`):

```
$ touch src/game.cpp
$ make
g++ ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
```

Только `game.o` пересоберётся. Линковка — да, потому что `game.o` изменился. `main.o` не трогается.

Изменим `include/game.h`:

```
$ touch include/game.h
$ make
g++ ... -c src/main.cpp -o build/main.o
g++ ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
```

Оба .cpp пересобрались, потому что оба зависят от game.h (через #include). Make узнал об этом из .d-файлов.

Это и есть **инкрементальная сборка**. Без -MMO пришлось бы перечислять зависимости руками — на десятке файлов уже невозможно.

16.6 Скелет кода

Готов Makefile. Теперь — минимальный код. Цель: компилируется и запускается, печатает приглашение, понимает help/look/quit. Игровой логики нет.

16.6.1 include/game.h

```
#ifndef RPG_GAME_H
#define RPG_GAME_H

#include <string>

namespace rpg {

class Game {
public:
    Game();
    int run();

private:
    bool step();
    void execute(const std::string& cmd);

    bool running_;
};

} // namespace rpg

#endif
```

Что нового:

Пространство имён `namespace rpg { ... }`. Все наши классы будут в нём. Это защита от коллизии имён: если кто-то подключит наш `Game` рядом с другим `Game` из чужой библиотеки, конфликта не будет — наш `rpg::Game`, его `other::Game`.

Class declaration:

```
class Game {
public:
    Game();           // конструктор
    int run();        // публичный метод

private:
    bool step();      // приватные методы
    void execute(const std::string& cmd);

    bool running_;    // приватное поле
};
```

`public:` — что видно снаружи. `private:` — только внутри класса.

Конвенция: **поля класса с подчёркиванием в конце** (`running_`). Это отличает их от локальных переменных и параметров.

`const std::string&` для параметра `cmd` — то, что мы изучили в главе 10 (по `const`-ссылке, без копии).

Глубокий разбор синтаксиса класса — глава 16. Сейчас — увидели, что есть, и идём дальше.

16.6.2 src/game.cpp

```
#include "game.h"

#include <iostream>
#include <string>

namespace rpg {

Game::Game() : running_(true) {
}

int Game::run() {
    std::cout << "=== RPG-демо ===\n";
```

```
std::cout << "Введите 'help' для списка команд.\n\n";

while (running_) {
    if (!step()) {
        break;
    }
}

std::cout << "До свидания.\n";
return 0;
}

bool Game::step() {
    std::cout << "> ";

    std::string line;
    if (!std::getline(std::cin, line)) {
        // EOF (Ctrl+D) – корректный выход.
        std::cout << "\n";
        return false;
    }

    if (line.empty()) {
        return true;
    }

    execute(line);
    return running_;
}

void Game::execute(const std::string& cmd) {
    if (cmd == "help") {
        std::cout << "Доступные команды:\n";
        std::cout << "  help    – этот список\n";
        std::cout << "  look   – осмотреться (пока заглушка)\n";
        std::cout << "  quit   – выход\n";
    } else if (cmd == "look") {
        std::cout << "Вы стоите на пустой поляне. Здесь пока ничего нет.\n";
    }
}
```

```

    } else if (cmd == "quit" || cmd == "q" || cmd == "exit") {
        running_ = false;
    } else {
        std::cout << "Неизвестная команда: '" << cmd << "'. Введите 'help'.\n";
    }
}

} // namespace rpg

```

Имена методов класса записываются как `Game::run` (член-функция). Двоеточие — оператор разрешения области видимости, тот же, что в `std::cout`.

`Game::Game() : running_(true) {}` — **конструктор с инициализационным списком**. После двоеточия — инициализация полей. Эквивалент:

```

Game::Game() {
    running_ = true;
}

```

Но через инициализационный список **правильнее** — поле сразу создаётся с нужным значением, а не сначала по умолчанию, потом перезаписывается. Для `bool` разницы практически нет, для классов — большая. Подробно в главе 16.

16.6.3 src/main.cpp

```

#include "game.h"

int main() {
    rpg::Game game;
    return game.run();
}

```

Создаём `Game` на стеке, вызываем `run()`, возвращаем код.

Заметили — `rpg::Game`, не просто `Game`. Без префикса `rpg::` компилятор не нашёл бы класс. Альтернатива:

```

using namespace rpg;

int main() {
    Game game;
}

```

```
    return game.run();  
}
```

`using namespace` втаскивает имена пространства в текущую область. Удобно, но опасно: на больших проектах можно случайно «втащить» имя, конфликтующее с чем-то ещё. Поэтому я в `main.cpp` обхожусь без `using namespace`.

16.6.4 tests/test_smoke.cpp

Совсем маленький smoke-тест:

```
#include "game.h"  
#include <iostream>  
#include <sstream>  
  
int main() {  
    std::istringstream empty_input("");  
    std::cin.rdbuf(empty_input.rdbuf());  
  
    rpg::Game game;  
    int rc = game.run();  
  
    if (rc != 0) {  
        std::cerr << "FAIL\n";  
        return 1;  
    }  
  
    std::cout << "OK\n";  
    return 0;  
}
```

«Smoke test» — простейшая проверка, что код вообще запускается. Здесь мы подменяем `std::cin` на пустой поток через `rdbuf`, чтобы `getline` сразу вернул `false` и игра завершилась.

Полноценные тесты с `GoogleTest` появятся в главе 25.

16.7 Сборка и проверка

```
$ cd demo-rpg  
$ make
```



```
g++ -std=c++11 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion \
    -g -O0 -Iinclude -MMD -MP -c src/main.cpp -o build/main.o
g++ ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
Built: build/rpg (debug)
```

```
$ ./build/rpg
```

```
=== RPG-демо ===
```

```
Введите 'help' для списка команд.
```

```
> help
```

```
Доступные команды:
```

```
help    — этот список
look    — осмотреться (пока заглушка)
quit    — выход
```

```
> look
```

```
Вы стоите на пустой поляне. Здесь пока ничего нет.
```

```
> quit
```

```
До свидания.
```

Работает. Запустим тест:

```
$ make tests
```

```
$ ./build/tests/test_smoke
```

```
=== RPG-демо ===
```

```
...
```

```
OK
```

Скелет готов. Все следующие главы — наполнение его настоящей игрой.

16.7.1 Release-сборка

Сейчас собралось в debug-режиме. Соберём в release:

```
$ make clean
```

```
$ make MODE=release
```

```
g++ -std=c++11 ... -O2 -DNDEBUG ...
```

```
Built: build/rpg (release)
```

```
$ ls -lh build/rpg
```

```
-rwxr-xr-x 1 user staff 48K ... build/rpg
```

Размер 48 KB (в debug было ~100 KB). С -O2 компилятор оптимизирует код — программа быстрее, бинарь меньше (debug-символы выкинуты).

16.8 Тонкости и расширения

Несколько вещей, которые могут пригодиться позже.

16.8.1 Поддержка отладчика

В debug-режиме файлы скомпилированы с -g, можно открыть в отладчике:

```
$ gdb ./build/rpg
(gdb) break Game::run
(gdb) run
(gdb) next
(gdb) print running_
```

Или lldb на macOS (то же).

16.8.2 AddressSanitizer

Чтобы собрать с ASan для поиска багов памяти, добавьте в Makefile (или вызовите):

```
$ make CXXFLAGS="$(make CXXFLAGS) -fsanitize=address -fno-omit-frame-pointer"
```

В реальных проектах делают отдельный режим MODE=asan для удобства. Я добавлю это в проекте на середине Части II.

16.8.3 Зачем не CMake

CMake — современный стандарт. Я не использую его в книге специально: - Makefile проще для обучения (~50 строк, всё видно). - CMake требует знания «третьего языка» (CMakeLists.txt). - На нашем масштабе проекта (~30 файлов) разница незаметна.

В реальных коммерческих проектах — конечно, CMake. Когда дойдёте до глав 26+ (mini-shell и далее), стоит уже посмотреть на CMake — синтаксис похожий, идея та же.

16.8.4 Зачем не header-only

Иногда видишь библиотеки, где все классы написаны прямо в .h-файлах, без .cpp (Catch2, например). Это называется header-only.

Удобно для пользователя — одна `#include`-строка и поехали. Но плохо для **времени компиляции**: каждый файл, который включает заголовок, компилирует все его реализации. На больших проектах — десятки секунд лишних.

В нашем проекте — классический подход. Объявления в `.h`, реализации в `.cpp`. Компилятор работает быстрее, организация чище.

16.9 Главные правила главы

1. **Файловая структура** `include/src/tests/data/build/` — стандарт, придерживайтесь.
2. **-MMD -MP для авто-зависимостей**. Никаких ручных списков `.h` в `Makefile`.
3. **Жёсткие флаги предупреждений** с самого начала. Лучше две недели потратить на исправление, чем потом неделю искать редкий баг.
4. **Заголовок класса с `include guard` или `#pragma once`**. Без них — `multiple definition`.
5. **Класс в `namespace`** для проекта. Защита от коллизий имён.
6. **Поля приватные**, доступ через методы. Подробно в главе 16.
7. **Один класс — один файл**. `Game` → `game.h/game.cpp`. Облегчает навигацию.

16.10 Маленькое упражнение

1. Соберите проект (`make`), запустите (`./build/rpg`). Поиграйтесь с тремя командами.
2. Добавьте новую команду: `name <текст>` — установить имя игрока, и `whoami` — показать текущее имя. Имя храните в новом поле `Game`. Подсказка: для парсинга используйте `std::stringstream` (глава 12).
3. Соберите в `release` (`make MODE=release`). Сравните размер бинарника.
4. Сделайте `make clean`, потом `make`. Засеките время.
5. Добавьте к флагам предупреждений `-Werror` (превращает все предупреждения в ошибки). Соберите. Если будут срабатывания — исправляйте код, а не глушите флаги.
6. Откройте `build/game.d` и `build/main.d`. Посмотрите, что там за зависимости. Убедитесь, что после изменения `game.h` пересобираются оба `.o`.
7. Создайте `src/world.cpp` с пустой функцией:

```
namespace rpg {  
void hello_world() {}  
}
```

Запустите `make`. Должен сам подхватить новый файл без правки `Makefile`.

16.11 Что дальше

Глава 16 — наполняем скелет настоящими классами. Начнём с `Player` и `Location`. Введём наследование, конструкторы, деструкторы, инициализаторы, инкапсуляцию. После этой главы у нас уже будет работающий игрок, который может «осмотреть локацию» с конкретными деталями. Потом, глава за главой, добавим инвентарь, врагов, бои, NPC, сохранения.

Часть II — самая длинная и интересная: настоящий код, настоящие решения, и весь язык в работе.

Глава 17

Глава 16. Классы — Player, Location, World

В прошлой главе мы собрали скелет проекта. Класс `Game` там уже был, но мы прошли по нему по верхам. Эта глава — глубокий разбор **классов** в C++ на примере трёх настоящих сущностей игры: `Player`, `Location`, `World`. К концу главы у игрока будут параметры (НР, имя), в мире — три локации, и можно ходить из одной в другую.

Главы 16-19 — самые «тяжёлые» в Части II. Это про объектно-ориентированный C++: классы, наследование, виртуальные функции, `smart pointers`, `move`-семантика. Если эти главы пройдут плотно — дальше будет легко.

17.1 `struct` и `class`

В C++ есть два ключевых слова для определения пользовательских типов: `struct` и `class`. Технически они почти не отличаются. Единственная разница:

- В `struct` поля и методы по умолчанию **публичные**.
- В `class` поля и методы по умолчанию **приватные**.

```
struct Point {  
    int x;  
    int y;  
};  
  
class Person {  
    std::string name;    // приватное по умолчанию  
    int age;             // тоже приватное
```

```
public:
    Person(std::string n, int a) : name(std::move(n)), age(a) {}
};
```

Конвенция: - **struct** — для простых наборов данных без поведения. Point, Color, Vector3, DivvResult. Поля торчат наружу, никакой инкапсуляции. - **class** — для всего с инкапсуляцией, инвариантами, нетривиальной логикой.

Эту конвенцию никто не запрещает нарушать, но я в книге придерживаюсь. И в стандартной библиотеке так же: `std::pair` — `struct`, `std::vector` — `class`.

17.2 Простой класс Player

Начнём с Player. У игрока должны быть: - Имя. - Текущее здоровье (hp). - Максимальное здоровье (max_hp). - Методы: получить урон, вылечиться, проверить, жив ли.

17.2.1 Объявление

include/player.h:

```
#ifndef RPG_PLAYER_H
#define RPG_PLAYER_H

#include <string>

namespace rpg {

class Player {
public:
    Player(std::string name, int max_hp);

    const std::string& name() const;
    int hp() const;
    int max_hp() const;
    bool alive() const;

    void take_damage(int amount);
    void heal(int amount);

private:
```

```
    std::string name_;  
    int max_hp_;  
    int hp_;  
};  
  
} // namespace rpg  
  
#endif
```

Разберём по элементам.

17.2.2 Поля

```
private:  
    std::string name_;  
    int max_hp_;  
    int hp_;
```

Это **поля** (fields) — данные класса. У каждого объекта Player будут свои name_, max_hp_, hp_.

Соглашение об именовании: поля заканчиваются на подчёркивание. Это популярная конвенция (Google C++ Style Guide и многие другие). Зачем? Чтобы внутри метода легко различить:

```
void set_name(const std::string& name) {  
    name_ = name;    // присвоить полю значение параметра  
}
```

Без подчёркивания пришлось бы либо использовать `this->name = name;`, либо переименовать параметр. Подчёркивание убирает путаницу.

Поля **приватные**: снаружи к name_ напрямую обратиться нельзя. Это **инкапсуляция** — главная идея ООП. Класс отвечает за свои данные, никто извне их случайно не испортит.

17.2.3 Методы доступа (getters)

```
public:  
    const std::string& name() const;  
    int hp() const;  
    int max_hp() const;  
    bool alive() const;
```

Это **getters**. Они возвращают значение полей, никак их не меняя. Особенности:

`const std::string& name() const` — два `const` в одной строке, и они **разные**.

- Первый `const` — у возвращаемого типа: «возвращаю ссылку на неизменяемую строку». Так читатель не сможет через возвращённую ссылку поменять `name_`.
- Второй `const` (в конце) — это **const-метод**: «обещаю, что метод не меняет состояние объекта». Внутри `name()` нельзя написать `name_ = ...` — компилятор не позволит.

`const`-методы можно вызывать на **константных объектах**:

```
const Player p("Hero", 100);
std::cout << p.name();      // ОК: name() это const-метод
p.take_damage(5);           // ОШИБКА: take_damage не const
```

Это важно. Если у вас в коде где-то `const Player& p` (как параметр функции), на нём можно вызывать только `const`-методы.

Правило: все методы, не меняющие объект, помечайте `const`. Это документация для читателя и контракт для компилятора. Привычка `const-correctness` — отличает уверенного C++-программиста.

17.2.4 Модифицирующие методы

```
public:
    void take_damage(int amount);
    void heal(int amount);
```

Эти меняют состояние — не `const`.

17.2.5 Конструктор

```
public:
    Player(std::string name, int max_hp);
```

Конструктор — особый метод, имя которого совпадает с именем класса. Возвращаемого типа нет (даже `void` не пишут).

Вызывается **автоматически** при создании объекта:

```
Player hero("Alice", 100);
// неявно вызывается Player::Player("Alice", 100)
```

Параметры конструктора `name` (по значению `std::string`) и `max_hp` (по значению `int`) — это входные данные для инициализации.

17.2.6 Определение

src/player.cpp:

```
#include "player.h"

#include <utility>

namespace rpg {

Player::Player(std::string name, int max_hp)
    : name_(std::move(name)),
      max_hp_(max_hp),
      hp_(max_hp) {}

const std::string& Player::name() const {
    return name_;
}

int Player::hp() const {
    return hp_;
}

int Player::max_hp() const {
    return max_hp_;
}

bool Player::alive() const {
    return hp_ > 0;
}

void Player::take_damage(int amount) {
    if (amount < 0) amount = 0;
    hp_ -= amount;
    if (hp_ < 0) hp_ = 0;
}

void Player::heal(int amount) {
```

```

    if (amount < 0) amount = 0;
    hp_ += amount;
    if (hp_ > max_hp_) hp_ = max_hp_;
}

} // namespace rpg

```

Каждый метод определяется как `ClassName::method`. Это правильное использование квалифицированного имени.

Инициализационный список — самое важное

Конструктор содержит **инициализационный список** после двоеточия:

```

Player::Player(std::string name, int max_hp)
    : name_(std::move(name)),
      max_hp_(max_hp),
      hp_(max_hp) {
}

```

Каждое поле инициализируется в скобках. Это **правильный** способ. Альтернатива через присваивание в теле:

```

Player::Player(std::string name, int max_hp) {
    name_ = name;
    max_hp_ = max_hp;
    hp_ = max_hp;
}

```

— работает, но **хуже**. Что происходит при таком стиле:

1. Сначала каждое поле создаётся **по умолчанию**. `name_` становится пустой строкой, `max_hp_` и `hp_` — неинициализированными.
2. Потом в теле конструктора им **присваиваются** нужные значения. То есть `std::string::operator=` вызывается, что лишняя работа.

С инициализационным списком — поля сразу создаются с правильными значениями, без промежуточного «пустого» состояния. Это **быстрее и правильнее**.

Для `int` разница невелика. Для `std::string` или собственных классов с дорогим конструктором — заметная.

И есть случаи, когда инициализационный список **обязателен**: - Поля типа `const` (нельзя присвоить — только инициализировать). - Поля-ссылки (то же самое). - Поля без `default-конструктора` (классы, у которых нет конструктора без параметров).

Поэтому: **всегда инициализируйте поля в инициализационном списке, не в теле.**

Порядок в списке должен **совпадать с порядком объявления полей**. Если в классе:

```
private:
    std::string name_;
    int max_hp_;
    int hp_;
```

Инициализация **именно** в этом порядке. Иначе компилятор молча перенаправит (поля всё равно инициализируются в порядке объявления), и `-Wreorder` это поймает.

std::move в конструкторе

Почему `name_(std::move(name))`?

В прошлой главе видел такое выражение, не разбирали детально. Объясню кратко (полностью — в главе 19).

`std::move(name)` — это «преобразование к `rvalue`». Грубо: «эту переменную я больше использовать не буду, можно её ограбить».

Без `std::move`:

```
Player::Player(std::string name, int max_hp)
    : name_(name),          // КОПИЯ строки name
    ...
```

`name_(name)` — это **копия** строки. Аллокация на куче, копирование байтов. Если переданная строка большая — дорого.

С `std::move`:

```
    : name_(std::move(name)),
```

`name_(std::move(name))` — **перемещение**: `name_` забирает себе внутренний указатель и буфер `name`. Никакой копии. Источник (`name`) становится «пустым», но это нормально — мы её больше не используем.

В новом C++-коде вы будете часто видеть `std::move` в конструкторах для строк и других «дорогих» типов. Это идиома.

Тело конструктора

После инициализационного списка идёт обычное тело `{ }`. Здесь можно делать дополнительную работу — выводы, проверки, что угодно. Часто оно пустое.

В нашем `Player::Player` — пусто. Все поля инициализированы списком.

17.2.7 Что такое `this`

Внутри методов класса есть невидимый параметр `this` — указатель на текущий объект. Когда вы пишете внутри метода `hp_`, на самом деле подразумевается `this->hp_`.

```
int Player::hp() const {
    return hp_;          // эквивалент: return this->hp_;
}
```

Часто `this->` опускают для краткости (поэтому подчёркивание в имени поля помогает не путать с параметрами). Но иногда нужен явный `this`:

```
void Player::set_hp(int hp) {
    this->hp_ = hp;      // явно, чтобы не путать с параметром
}
```

Или когда нужно вернуть указатель/ссылку на сам объект:

```
Player& Player::heal(int amount) {
    hp_ += amount;
    if (hp_ > max_hp_) hp_ = max_hp_;
    return *this;        // вернуть ссылку на сам объект
}
```

Это позволяет цепочки: `player.heal(5).heal(10).heal(15)` (правда, в нашем коде мы возвращаем `void`).

17.2.8 Использование Player

В `Game::Game()` мы теперь создаём игрока:

```
Game::Game()
: running_(true),
  world_(make_demo_world()),
  player_("Герой", 20),
  current_location_id_("glade") {
}
```

player_("Герой", 20) — вызов конструктора Player. Это и есть инициализационный список самого Game.

В Game::run():

```
std::cout << "Привет, " << player_.name() << "!\n";
```

player_.name() — вызов const-метода. Возвращает const std::string&. Дальше << для потока умеет работать со строкой.

17.3 Локация: класс посложнее

Location хранит: - ID (короткая строка-идентификатор). - Имя (для отображения). - Описание.
- Выходы — словарь «направление → ID соседней локации».

17.3.1 Объявление

include/location.h:

```
#ifndef RPG_LOCATION_H
#define RPG_LOCATION_H

#include <string>
#include <unordered_map>

namespace rpg {

class Location {
public:
    Location(std::string id, std::string name, std::string description);

    const std::string& id() const;
    const std::string& name() const;
    const std::string& description() const;

    void add_exit(const std::string& direction, const std::string& target_id);
    std::string exit(const std::string& direction) const;

    const std::unordered_map<std::string, std::string>& exits() const;

private:
```

```

    std::string id_;
    std::string name_;
    std::string description_;
    std::unordered_map<std::string, std::string> exits_;
};

} // namespace rpg

#endif

```

Очень похоже на Player, только полей побольше.

17.3.2 Определение

```

Location::Location(std::string id, std::string name, std::string description)
    : id_(std::move(id)),
      name_(std::move(name)),
      description_(std::move(description)) {
}

```

Конструктор инициализирует строки через `std::move`. `exits_` не упомянут — он автоматически создаётся пустым `unordered_map`-ом (default constructor).

```

void Location::add_exit(const std::string& direction, const std::string& target_id) {
    exits_[direction] = target_id;
}

std::string Location::exit(const std::string& direction) const {
    auto it = exits_.find(direction);
    if (it == exits_.end()) return {};
    return it->second;
}

```

`add_exit` — обычный setter. `exit` — поиск с возвратом пустой строки при отсутствии. Я предпочитаю возвращать пустую строку, а не выкидывать исключение — для нашей логики (не каждый ход надо проверять направления) это удобнее.

`return {};` — пустая инициализация для `std::string`. Эквивалент `return std::string();`.

Замечание: метод **называется** `exit`, но это не выход из программы. У нас в `namespace rpg`, имя не конфликтует с системным `exit()`. Так можно.

17.3.3 Использование

```
Location glade("glade", "Лесная поляна",  
              "Залитая солнцем поляна. На западе тропа к деревне.");  
glade.add_exit("west", "village");  
glade.add_exit("east", "forest");
```

Создаём, заполняем выходами. Простой и понятный код.

17.4 Перегрузка операторов — короткий взгляд

В стандартной библиотеке вы видели:

```
std::string a = "hello";  
std::string b = a + " world";           // оператор +  
std::cout << a;                         // оператор <<  
if (a == "hello") ...                  // оператор ==
```

Откуда эти операторы для `std::string`? Они **перегружены** в самом классе. C++ позволяет определить операторы для своих типов.

Подробный разбор — глава 20 (там вместе с шаблонами). Сейчас для контекста:

```
class Vector2 {  
public:  
    Vector2(double x, double y) : x_(x), y_(y) {}  
  
    Vector2 operator+(const Vector2& other) const {  
        return Vector2(x_ + other.x_, y_ + other.y_);  
    }  
  
private:  
    double x_, y_;  
};  
  
int main() {  
    Vector2 a(1, 2);  
    Vector2 b(3, 4);  
    Vector2 c = a + b;           // вызов a.operator+(b), даёт {4, 6}  
}
```

В нашей игре операторов мы пока не делаем — обходимся методами `add_exit`, `exit`. Но потом, в иерархии предметов и в сохранениях, будем перегружать `<<` для потоков (как `std::cout << player`).

17.5 Конструктор по умолчанию

Если вы не пишете **никакого** конструктора, компилятор создаёт **конструктор по умолчанию** — без параметров. Он инициализирует поля их default-значениями.

```
class Empty {
    int x;
    std::string y;
};

Empty e;      // ОК: x неинициализирован (мусор), y пустая строка
```

Но как только вы написали **любой** свой конструктор — компилятор перестаёт автоматически создавать default. Поэтому:

```
class Player {
public:
    Player(std::string name, int max_hp);    // мы написали свой
};

Player p;    // ОШИБКА: нет конструктора без параметров
```

Если вам нужен ещё и default:

```
class Player {
public:
    Player();    // отдельно объявляем
    Player(std::string name, int max_hp);
};

Player::Player() : Player("Unnamed", 100) {}
// делегирование — вызов другого конструктора (C++11)
```

В нашем `Player` default-конструктор не нужен. В `World` — нужен:

```
class World {
public:
    World();    // явно объявляем
```



```
// ...
};

World::World() = default;
```

= default — особый синтаксис: «компилятор, создай мне дефолтную реализацию». Полезно, когда хочется явно показать, что default-конструктор есть, но писать его руками нечего.

17.5.1 = delete

Антипод = default. Запрещает конструктор/метод:

```
class Singleton {
public:
    Singleton(const Singleton&) = delete;           // запрет копирования
    Singleton& operator=(const Singleton&) = delete; // запрет присваивания
};
```

Если кто-то попытается скопировать Singleton — ошибка компиляции. Полезно для классов, которые не должны копироваться (RAII-обёртки для уникальных ресурсов).

17.6 Копирование: rule of three

Что произойдёт, если мы скопируем объект?

```
Player a("Hero", 100);
Player b = a;           // копия
```

Компилятор автоматически генерирует **конструктор копирования**:

```
Player(const Player& other)
    : name_(other.name_),
      max_hp_(other.max_hp_),
      hp_(other.hp_) {}
```

Поле за полем. Для встроенных типов и std::string это работает корректно (std::string сам знает, как себя копировать).

Также генерируется **оператор присваивания при копировании**:

```
Player& operator=(const Player& other) {
    name_ = other.name_;
```

```

    max_hp_ = other.max_hp_;
    hp_ = other.hp_;
    return *this;
}

```

И **деструктор** (для Player он тривиальный — встроенные типы и `std::string` сами уничтожаются).

Это «**rule of zero**» (правило нуля): если ваш класс хранит только стандартные типы и контейнеры — **ничего не писать**, компилятор всё сгенерирует правильно.

17.6.1 Когда rule of zero ломается

Если поле — **сырой указатель** на динамическую память:

```

class IntHolder {
public:
    IntHolder(int v) : ptr_(new int(v)) {}
    ~IntHolder() { delete ptr_; }    // вот, освобождаем

private:
    int* ptr_;
};

IntHolder a(42);
IntHolder b = a;           // !!! компилятор скопирует ptr_ как есть
// теперь у a и b ОДИН указатель на тот же int
// при уничтожении обоих — двойной delete, UB!

```

Это типичный баг. Компилятор копирует **по умолчанию** поверхностно (shallow copy) — то есть копирует значения полей. Для указателя значение — адрес. Глубокого копирования (создать новый объект и скопировать содержимое) не делает.

Решение — «**rule of three**»: если у класса есть **нетривиальный деструктор** (нужен для управления ресурсом), скорее всего нужны и: - Конструктор копирования (правильно копировать ресурс). - Оператор присваивания (то же).

```

class IntHolder {
public:
    IntHolder(int v) : ptr_(new int(v)) {}

    ~IntHolder() { delete ptr_; }

```

```
IntHolder(const IntHolder& other) : ptr_(new int(*other.ptr_)) {}

IntHolder& operator=(const IntHolder& other) {
    if (this != &other) {                // защита от self-assignment
        int* new_ptr = new int(*other.ptr_);
        delete ptr_;
        ptr_ = new_ptr;
    }
    return *this;
}

private:
    int* ptr_;
};
```

Целая программа из-за одного указателя. Поэтому в современном C++ **сырые указатели почти не используются** — берут `std::unique_ptr` (глава 18), у которого правильное копирование/перемещение из коробки.

17.6.2 Rule of five (C++11)

В C++11 добавились **move-семантика**: перемещение объекта без копирования. Это потребовало двух новых специальных функций: - Конструктор перемещения. - Оператор присваивания при перемещении.

Если у класса нетривиальный деструктор, скорее всего нужны **пять** функций — это **rule of five**:

1. Деструктор.
2. Конструктор копирования.
3. Оператор присваивания при копировании.
4. Конструктор перемещения (новинка).
5. Оператор присваивания при перемещении (новинка).

Подробно про move — в главе 19. Сейчас знайте, что в современном C++:

- **Rule of zero** — лучший выбор: всё стандартно, ничего писать не надо.
- **Rule of five** — если нужно управлять ресурсами руками.
- **Rule of three** — устаревшее, для C++03.

17.6.3 Запрет копирования

Иногда копирование нежелательно (например, наш `LogFile_BAD` из главы 11). Запрещаем явно:

```
class LogFile {
public:
    LogFile(const std::string& filename);
    ~LogFile();

    LogFile(const LogFile&) = delete;           // запрет копирования
    LogFile& operator=(const LogFile&) = delete;
};
```

Это удобно для RAII-классов: копировать файл (значит, иметь два владельца одного дескриптора) — глупость.

17.7 World: контейнер локаций

World хранит коллекцию Location-ов и предоставляет доступ по ID.

17.7.1 Объявление

include/world.h:

```
#ifndef RPG_WORLD_H
#define RPG_WORLD_H

#include "location.h"

#include <string>
#include <unordered_map>

namespace rpg {

class World {
public:
    World();

    void add(Location location);
    const Location* find(const std::string& id) const;
    std::size_t size() const;
};
```

```
private:
    std::unordered_map<std::string, Location> locations_;
};

World make_demo_world();

} // namespace rpg

#endif
```

Ключевое поле — `locations_` типа `unordered_map<string, Location>`. Карта строка-в-локацию. `World` владеет всеми локациями — они хранятся **по значению** внутри карты. При уничтожении `World` все локации уничтожатся (RAII!).

17.7.2 `find` возвращает указатель

```
const Location* World::find(const std::string& id) const;
```

Возвращает `const Location*`. Почему указатель, а не ссылка?

Потому что **может не найтись**. Ссылка не может быть «никакой» — она всегда на что-то указывает. Указатель может быть `nullptr`. В нашем случае:

```
const Location* loc = world_.find("unknown_id");
if (loc == nullptr) {
    // не найдено
}
```

Это вариант «**optional**». В C++17 есть `std::optional<T>`, но в C++11 — обходимся указателями.

17.7.3 Заводская функция

```
World make_demo_world();
```

Это **свободная функция** в `namespace rpg`, не метод класса. Создаёт демо-мир из трёх локаций. Возвращает `World` по значению.

В реализации:

```
World make_demo_world() {
    World w;
```

```

Location glade("glade", "Лесная поляна", "...");
glade.add_exit("west", "village");
glade.add_exit("east", "forest");

Location village("village", "Деревня", "...");
village.add_exit("east", "glade");

Location forest("forest", "Тёмный лес", "...");
forest.add_exit("west", "glade");

w.add(std::move(glade));
w.add(std::move(village));
w.add(std::move(forest));

return w;
}

```

Создаём World, три Location, перемещаем их в World, возвращаем сам World. `std::move` тут потому, что после `add` локальные `glade`, `village`, `forest` не нужны — пусть World забирает их «внутренности» без копирования.

Возврат World по значению — тоже без копирования благодаря **RVO** (return value optimization), о которой говорили в главе 10. Компилятор просто строит World сразу в нужном месте.

17.7.4 `emplace` вместо `insert+move`

В `World::add`:

```

void World::add(Location location) {
    std::string id = location.id();
    locations_.emplace(std::move(id), std::move(location));
}

```

`emplace` — это «вставка с конструированием на месте». Эффективнее `insert`:

- `insert(std::make_pair(id, location))` сначала создаёт пару, потом копирует/перемещает её в карту.
- `emplace(id, location)` пересылает аргументы прямо в конструктор пары внутри карты. Никаких промежуточных копий.

Тонкость: `emplace` ожидает аргументы конструктора `std::pair<key, value>`, и таких пар бывает два вида (с разными порядками копирования/перемещения). На наших размерах локаций разница незаметна, но привычка `emplace` вместо `insert` полезна.

17.7.5 Default-конструктор через `= default`

```
World::World() = default;
```

Я мог бы написать `World::World() {}` — пустое тело. Но `= default` явно говорит «дефолтная реализация». Тонкие отличия (для агрегатной инициализации и других вещей) для нас сейчас неважны; стиль `c = default` — более явный.

17.8 Game: всё вместе

Обновлённый `Game` теперь владеет `Player` и `World`. Карта команд расширилась.

17.8.1 Объявление

```
class Game {
public:
    Game();
    int run();

private:
    bool step();
    void execute(const std::string& cmd);

    void cmd_help() const;
    void cmd_look() const;
    void cmd_go(const std::string& direction);
    void cmd_status() const;

    bool running_;
    World world_;
    Player player_;
    std::string current_location_id_;
};
```

Каждая команда — отдельный метод. `cmd_help`, `cmd_look`, `cmd_status` — не меняют состояние (`const`). `cmd_go` меняет (`current_location_id_`).

17.8.2 Конструктор

```
Game::Game()  
    : running_(true),  
      world_(make_demo_world()),  
      player_("Герой", 20),  
      current_location_id_("glade") {  
}
```

Все поля инициализируются в списке. `world_(make_demo_world())` — вызов заводской функции, она возвращает `World` по значению, `world_` инициализируется этим значением (через `move`-конструктор `World`, который компилятор сгенерировал нам автоматически — *rule of zero!*).

17.8.3 Команды

```
void Game::execute(const std::string& cmd) {  
    std::istringstream ss(cmd);  
    std::string verb;  
    ss >> verb;  
  
    if (verb == "help") {  
        cmd_help();  
    } else if (verb == "look" || verb == "l") {  
        cmd_look();  
    } else if (verb == "go") {  
        std::string dir;  
        ss >> dir;  
        if (dir.empty()) {  
            std::cout << "Куда идти?\n";  
        } else {  
            cmd_go(dir);  
        }  
    } else if (verb == "status" || verb == "stat") {  
        cmd_status();  
    } else if (verb == "quit" || verb == "q" || verb == "exit") {  
        running_ = false;  
    }  
}
```



```

    } else {
        std::cout << "Неизвестная команда\n";
    }
}

```

Распознавание команд — пока через if/else. В главе 21 заменим на словарь имя → лямбда. Сейчас простой код.

```

void Game::cmd_look() const {
    const Location* loc = world_.find(current_location_id_);
    if (!loc) {
        std::cout << "Где-то в пустоте.\n";
        return;
    }
    std::cout << "== " << loc->name() << " ==\n";
    std::cout << loc->description() << "\n";

    if (!loc->exits().empty()) {
        std::cout << "Выходы:";
        for (const auto& exit_pair : loc->exits()) {
            std::cout << " " << exit_pair.first;
        }
        std::cout << "\n";
    }
}

```

world_.find(current_location_id_) — получаем указатель. Проверяем на nullptr. Если есть — печатаем имя, описание, выходы.

for (const auto& exit_pair : loc->exits()) — стандартная идиома обхода карты по const-ссылке.

```

void Game::cmd_go(const std::string& direction) {
    const Location* loc = world_.find(current_location_id_);
    if (!loc) return;

    std::string target = loc->exit(direction);
    if (target.empty()) {
        std::cout << "Туда не пройти.\n";
        return;
    }
}

```

```
    current_location_id_ = target;
    cmd_look();
}
```

Получаем текущую локацию, ищем выход в нужном направлении, если есть — меняем `current_location_id_`. После — `cmd_look()` показывает новую локацию.

17.9 Сборка и запуск

```
$ make clean && make
g++ ... -c src/main.cpp -o build/main.o
g++ ... -c src/game.cpp -o build/game.o
g++ ... -c src/location.cpp -o build/location.o
g++ ... -c src/player.cpp -o build/player.o
g++ ... -c src/world.cpp -o build/world.o
g++ ... build/main.o build/game.o build/location.o build/player.o build/world.o -o build/rpg
Built: build/rpg (debug)
```

Заметили? `make` сам подхватил новые `.cpp` файлы — wildcard `src/*.cpp` в `Makefile` собрал их все. Никаких ручных правок.

```
$ ./build/rpg
=== RPG-демо ===
Привет, Герой! Введите 'help' для справки.

== Лесная поляна ==
Залитая солнцем поляна. На западе видна тропа к деревне, на востоке — тёмный лес.
Выходы: east west

> status
Герой: HP 20/20

> go east
== Тёмный лес ==
Ветви скрипят над головой. Здесь сумрачно и тихо. На запад уходит тропа.
Выходы: west

> go west
```

```
== Лесная поляна ==
```

```
Залитая солнцем поляна. На западе видна тропа к деревне, на востоке — тёмный лес.
```

```
Выходы: east west
```

```
> go north
```

```
Туда не пройти.
```

```
> quit
```

```
До свидания.
```

Игра. Не богатая, но игра.

17.10 Что освоено в этой главе

1. **class vs struct** — разница в умолчании доступа.
2. **Поля, методы, инкапсуляция** — private/public.
3. **Конвенция _ в конце поля** — отличает от параметров.
4. **Конструктор и инициализационный список** — правильный способ инициализации полей.
5. **std::move в конструкторе** — для дорогих параметров.
6. **this** — указатель на текущий объект.
7. **const-методы** — обещание «не мутирую состояние».
8. **= default и = delete** — явные специальные функции.
9. **Rule of zero / rule of three / rule of five** — иерархия правил.
10. **Возврат указателя из find** — паттерн для «может не найтись».
11. **emplace вместо insert** — без промежуточных копий.

17.11 Маленькое упражнение

1. Добавьте в Player поле `int xp_` (опыт). Метод `void gain_xp(int amount)` и `int xp() const`. В `cmd_status` показывайте опыт.
2. Добавьте в игру новую локацию — пещеру (`cave`), доступную с лесной поляны на юг. Из пещеры можно вернуться на север.
3. Добавьте команду `where` — выводит ID текущей локации (для дебага).
4. Сделайте `Player::heal` так, чтобы он отказывался лечить мёртвого игрока (если `hp_ == 0`, ничего не делать, выдать сообщение).

5. Перепишите `Location::exit` так, чтобы он бросал `std::out_of_range`, если направление не найдено. Сравните с текущей версией — какая удобнее для нашего `cmd_go`?
6. (Сложнее) Добавьте `Location::add_item(std::string item_name)` и `std::vector<std::string> items()`. В `cmd_look` показывайте список предметов: «На полу лежит: меч, золото».
7. (Сложнее) Прочитайте код `World::add`. Объясните, почему `std::string id = location.id();` копирует строку, а `std::move(id)` дальше уже её перемещает.

17.12 Что дальше

Глава 17 — **наследование и виртуальные функции**. Создадим иерархию `Entity` → `Player`/`Enemy`/`NPC` и параллельную `Item` → `Weapon`/`Armor`/`Consumable`. Это полиморфизм — главная идея ООП. Узнаем, что такое `vtable`, почему виртуальный деструктор обязателен, что такое `override` и `final`.

После 17-й главы у нас будут враги, оружие, броня. Бои добавятся в главе 18 (заодно введём `unique_ptr` для управления предметами).

Темп Части II: новый класс или две — каждую главу, постепенно. К 25-й главе соберём всё в живую игру.

Глава 18

Глава 17. Наследование и виртуальные функции

В этой главе — главная тема ООП в C++: **наследование** и **полиморфизм**. На примере нашего проекта построим иерархию предметов: базовый `Item`, и три потомка — `Weapon`, `Armor`, `Consumable`. Локация и инвентарь смогут хранить указатели на базу `Item*` — а внутри будут лежать конкретные оружия, брони, расходники. Один и тот же `item->describe()` будет работать по-разному в зависимости от реального типа объекта. Это и есть полиморфизм.

К концу главы вы поймёте: что такое `virtual`, зачем `override`, почему виртуальный деструктор обязателен, что такое `vtable`, в чём проблема «slicing». Тема концептуально тяжёлая, но если разберётесь — большая часть страшности C++ останется позади.

18.1 Зачем наследование

Предположим, нам в игре нужны три типа предметов: оружие (даёт урон), броня (даёт защиту), расходник (даёт эффект при использовании). У всех общее: - Имя. - Вес. - Возможность отобразить в инвентаре.

У каждого свои особенности: - `Weapon`: урон. - `Armor`: защита. - `Consumable`: что делает при использовании.

Можно было бы написать три отдельных класса без связи. Локация бы тогда хранила три коллекции:

```
class Location {  
    std::vector<Weapon> weapons_;  
    std::vector<Armor> armors_;
```

```
std::vector<Consumable> consumables_;  
};
```

Дальше команда `take <name>` искала бы предмет в трёх местах. Команда `look` показывала бы три цикла. Если завтра появится «волшебный артефакт» — четвёртая коллекция, четвёртый цикл. И повторение кода: имена, вес, описание — у всех одинаковые.

Решение — **общая база**. Создаём абстрактный класс `Item`, от него наследуем три конкретных. Локация хранит один список **указателей на базу**:

```
class Location {  
    std::vector<Item*> items_;  
};
```

Когда нужно «отобразить» предмет, мы вызываем `item->describe()` — компилятор сам выберет нужную реализацию в зависимости от реального типа. Это и есть полиморфизм.

18.2 Базовый синтаксис наследования

```
class Item {  
public:  
    Item(std::string name, int weight);  
  
    const std::string& name() const;  
    int weight() const;  
  
protected:  
    std::string name_;  
    int weight_;  
};  
  
class Weapon : public Item {  
public:  
    Weapon(std::string name, int weight, int damage);  
  
    int damage() const;  
  
private:  
    int damage_;  
};
```

`class Weapon : public Item` — **Weapon наследует от Item**. Это значит:

1. У `Weapon` есть **все поля и методы** `Item` плюс свои.
2. `Weapon` — это «тоже `Item`» в каком-то смысле: можно положить `Weapon*` туда, где ждут `Item*`.

Объявление `Weapon`:

```
Weapon sword("ржавый меч", 5, 8);

std::cout << sword.name();      // OK, унаследовано от Item
std::cout << sword.weight();    // OK, унаследовано
std::cout << sword.damage();    // OK, собственное
```

18.2.1 `public`, `protected`, `private` — два смысла

В заголовке `class Weapon : public Item` слово `public` определяет **тип наследования**:

- `public Item` — публичное наследование. `Weapon` — это `Item` снаружи. Самый частый вид.
- `protected Item` — наследование «для своих». Внешний код не знает о связи.
- `private Item` — наследование как «реализация через». `Weapon` использует `Item` внутри, но снаружи не виден.

В нашей книге будем использовать только **`public`** — самый частый и осмысленный.

Внутри класса слова `public/protected/private` определяют **доступ к членам**. С `protected` мы пока не встречались. Это «доступно классу и его потомкам, но не снаружи»:

```
class Item {
protected:
    std::string name_;
    // доступен снаружи: только через name()
    // доступен потомкам: напрямую как this->name_ или просто name_
};

class Weapon : public Item {
public:
    std::string describe() const {
        return name_ + " (оружие)"; // OK: protected доступно потомкам
    }
};
```

Если бы `name_` был `private` — `Weapon` не смог бы к нему обращаться напрямую. Пришлось бы через `name()` (геттер).

Когда что использовать: - `private` — для внутренних деталей класса, потомкам не нужно лезть. - `protected` — для общих полей/методов базы, к которым потомки хотят прямой доступ. - `public` — то, что доступно снаружи.

В нашем `Item` поля `name_` и `weight_` — `protected`, чтобы потомки могли их печатать в своём `describe`.

18.3 Конструктор базы

При создании `Weapon` сначала вызывается конструктор `Item`, потом — собственный конструктор `Weapon`. Вы обязаны указать, как звать конструктор базы:

```
Weapon::Weapon(std::string name, int weight, int damage)
    : Item(std::move(name), weight),    // вызов конструктора базы
      damage_(damage) {
}
```

В инициализационном списке **первой** идёт инициализация базы — синтаксис как для поля, но с именем класса. Перед собственными полями. Без этого:

```
Weapon::Weapon(std::string name, int weight, int damage)
    : damage_(damage) {                // забыли про Item
}
```

— компилятор попытается вызвать `Item()` (default-конструктор). Раз у `Item` его нет (мы написали свой с параметрами) — будет ошибка компиляции.

18.4 Виртуальные функции

Теперь центральная вещь. Допустим, у `Item` есть метод `describe()`, который возвращает описание:

```
class Item {
public:
    std::string describe() const {
        return name_ + " (предмет)";
    }
};
```

И у `Weapon` мы хотим, чтобы `describe()` показывал и урон:


```
class Weapon : public Item {
public:
    std::string describe() const {
        return name_ + " (оружие, урон " + std::to_string(damage_) + ")";
    }
};
```

Что произойдёт?

```
Weapon sword("меч", 5, 10);
std::cout << sword.describe();    // "меч (оружие, урон 10)" □

Item& item = sword;
std::cout << item.describe();      // "меч (предмет)" □
```

Через ссылку базы вызывается метод **базы**, а не потомка. Это **static dispatch** — компилятор смотрит на тип переменной (Item&), не на реальный объект (Weapon).

Это не то, что мы хотим. Хотим: «какой бы тип реально ни был, вызови подходящий метод». Для этого — **virtual**:

```
class Item {
public:
    virtual std::string describe() const {
        return name_ + " (предмет)";
    }
};

class Weapon : public Item {
public:
    std::string describe() const override {
        return name_ + " (оружие)";
    }
};
```

Ключевое слово **virtual** в базе говорит: «этот метод можно переопределить, и при вызове через ссылку/указатель базы выбирай реализацию по реальному типу».

Теперь:

```
Item& item = sword;
std::cout << item.describe();    // "меч (оружие)" □ dynamic dispatch
```

18.4.1 `override` — обязательная привычка

Слово `override` в потомке — **не магия, а проверка**. Оно говорит компилятору: «эта функция переопределяет виртуальную функцию базы». Если что-то не так — ошибка компиляции.

Без `override` опечатки молча создают новую функцию:

```
class Item {
public:
    virtual std::string describe() const { ... }
};

class Weapon : public Item {
public:
    std::string describe() const { ... }    // ОПЕЧАТКА: describe вместо describe
};
```

Без `override` компилятор подумает: «нормально, метод `describe` в `Weapon`, рядом с унаследованным `describe`». Тестирование укажет, что переопределение не сработало. Пол часа поисков, недоумение.

С `override`:

```
std::string describe() const override { ... }
// error: 'describe' marked 'override' but does not override
```

Компилятор сразу указывает на проблему.

Правило: ВСЕГДА ставьте `override` на переопределяющие методы. Это бесплатный страховочный полис.

18.4.2 `final` — запрет дальнейшего переопределения

Слово `final` запрещает потомкам переопределять метод:

```
class Item {
public:
    virtual std::string describe() const { ... }
};

class Weapon : public Item {
public:
    std::string describe() const override final { ... }
};
```

```
class MagicSword : public Weapon {
public:
    std::string describe() const override { ... }    // ОШИБКА: переопределение final
};
```

`final` можно применить и к классу — запретит наследование вообще:

```
class Weapon final : public Item {
    // от Weapon нельзя наследоваться
};
```

В нашем `Item.h` я как раз помечу `Weapon`, `Armor`, `Consumable` как `final` — на текущем этапе нам не нужно их расширять, и `final` помогает компилятору оптимизировать (статически разрешить виртуальные вызовы).

18.5 Чисто виртуальные функции и абстрактные классы

В `Item` есть метод `kind()` — возвращает тип («оружие», «броня», ...). У базового `Item` непонятно, что вернуть — это зависит от потомка. Делаем метод **чисто виртуальным**:

```
class Item {
public:
    virtual std::string kind() const = 0;    // = 0 значит «нет реализации»
};
```

`= 0` в конце — это синтаксис «pure virtual». Реализации нет; потомки **обязаны** её предоставить.

Класс, у которого есть хоть одна чисто виртуальная функция — **абстрактный**. Объекты абстрактного класса нельзя создать:

```
Item item("...", 5);    // ОШИБКА: cannot allocate object of abstract type
Item* p = new Item(...);    // тоже ошибка
```

Можно только указатели и ссылки:

```
Item* p = new Weapon(...);    // ОК
Item& r = some_weapon;        // ОК
```

Это удобно: запрещает «безсмысленные» объекты базы.

Конкретные потомки должны реализовать все чисто виртуальные методы:

```
class Weapon : public Item {
public:
    std::string kind() const override { return "оружие"; }    // обязательно
};
```

Если забыть — `Weapon` останется абстрактным, его тоже нельзя будет создать. Компилятор быстро выявит.

18.6 Виртуальный деструктор — обязателен!

Главный подвох наследования. Покажу на ошибке.

```
class Item {
public:
    Item(std::string n) : name_(n) {}
    ~Item() { std::cout << "Item dtor\n"; }    // HE virtual!
protected:
    std::string name_;
};

class Weapon : public Item {
public:
    Weapon(std::string n, int* big_array)
        : Item(n), buffer_(big_array) {}
    ~Weapon() {
        std::cout << "Weapon dtor\n";
        delete[] buffer_;
    }
private:
    int* buffer_;
};

int main() {
    Item* p = new Weapon("...", new int[1000]);
    delete p;    // ВНИМАНИЕ
}
```

Что произойдёт при `delete p`? `p` имеет тип `Item*`. Компилятор смотрит на тип указателя, видит — деструктор `Item`. Вызывает `~Item()`. `~Weapon()` **НЕ вызывается!**

Результат: `Weapon::buffer_` не освобождён, утечка памяти 4 КВ. И это ещё «милый» случай — если `Weapon` владел файлом или мьютексом, последствия серьёзнее.

Это **undefined behavior** в строгом смысле: стандарт C++ говорит, что `delete` через указатель базы без виртуального деструктора — UB. На практике обычно «работает» с утечкой; но в общем случае может быть что угодно.

Решение: деструктор базы должен быть **virtual**:

```
class Item {
public:
    virtual ~Item() = default;    // вот так
};
```

Тогда при `delete p` компилятор смотрит в **vtable** реального объекта, находит правильный деструктор (`~Weapon`), вызывает его. Внутри `~Weapon` потом автоматически вызывается `~Item` базы. Цепочка корректная, утечки нет.

`= default` — потому что для `Item` деструктор тривиальный (только освободить `std::string name_`, что и так само). `virtual ~Item() = default;` — идиома для «у меня виртуальный деструктор, но особой логики нет».

Правило: если класс задумывается как база для наследования, его деструктор обязан быть **virtual**. Без исключений.

Если класс **не** задумывается как база — деструктор пусть будет неvirtуальным или вовсе не объявлен. Виртуальность стоит лишние байты (vtable pointer внутри объекта).

18.7 vtable — как это работает

Под капотом виртуальные вызовы устроены так.

Каждый класс с хотя бы одной виртуальной функцией имеет **vtable** (virtual table) — массив указателей на методы. Где-то в `.rodata`-секции бинарника лежит:

```
vtable_Item:
    &Item::describe    // или &Weapon::describe, ...
    &Item::kind        // или &Weapon::kind, ...
    &Item::~~Item      // или &Weapon::~~Weapon, ...
    ...
```

Каждый объект класса с виртуальными функциями имеет **скрытый указатель** на свою vtable (обычно как первое поле). Размер объекта увеличивается на 8 байт.

`Item* p` → объект `Weapon` в памяти:

```

[vtable ptr: → vtable_Weapon]  <- скрытое поле
[name_: "меч"]                  <- из Item
[weight_: 5]                    <- из Item
[damage_: 10]                   <- из Weapon

```

Когда вы пишете `p->describe()`:

1. Компилятор знает, что `describe` — виртуальный.
2. Генерирует код: «возьми `vtable` из объекта по адресу `p`, найди в ней `describe`, вызови».
3. Поскольку у `Weapon` своя `vtable`, в ней `describe` указывает на `Weapon::describe`.

Это **дополнительный косвенный вызов** — чуть медленнее, чем прямой. Обычно — единицы наносекунд, не критично. Но если виртуальный метод вызывается миллионы раз за секунду — может стать узким местом. Тогда смотрят: правда ли нужен полиморфизм здесь?

18.8 Слайсинг — главная ловушка

```

Weapon sword("меч", 5, 10);

Item item = sword;          // !!!
std::cout << item.describe();

```

Это копирование `sword` в **новый объект** `item` типа `Item`. При копировании поля `Weapon` (`damage_`) **отсекаются** — у `Item` их нет. Это **slicing** (срезание).

Объект `item` — настоящий `Item`, не `Weapon`. Даже если методы виртуальные, `item.describe()` вызовет `Item::describe()`.

Чтобы избежать `slicing`:

- **Никогда не присваивайте объект потомка в переменную базы.** Только указатели или ссылки.
- Если нужно «как-то хранить разнотипные объекты» — `std::vector<Item*>` или `std::vector<std::unique_ptr<Item>>` (глава 18).

В нашем `Item` я сделал ещё страховку:

```

class Item {
public:
    Item(const Item&) = delete;
    Item& operator=(const Item&) = delete;
    Item(Item&&) = delete;

```

```
Item& operator=(Item&&) = delete;
};
```

Запрет копирования и перемещения. Если кто-то попытает `Item item = sword;` — ошибка компиляции. Безопасно: объект `Item` живёт ровно один раз, доступ к нему только через указатели/ссылки.

В реальной жизни иногда копирование нужно. Тогда определяют **виртуальный `clone()`-метод**:

```
class Item {
public:
    virtual std::unique_ptr<Item> clone() const = 0;
};

class Weapon : public Item {
public:
    std::unique_ptr<Item> clone() const override {
        return std::unique_ptr<Item>(new Weapon(*this));
    }
};
```

`item->clone()` создаёт копию реального типа. Это идиома **prototype**. У нас в книге она появится позже — для копирования предметов в инвентаре.

18.9 `dynamic_cast` — приведение базы к потомку

Иногда у вас на руках `Item*`, и нужно «если это оружие — узнать урон». В C++ для этого есть `dynamic_cast`:

```
Item* p = ...;
if (Weapon* w = dynamic_cast<Weapon*>(p)) {
    int dmg = w->damage();
    ...
}
```

`dynamic_cast<Weapon*>(p)`: - Если объект **реально** `Weapon` или его потомок — возвращает `Weapon*`.
- Иначе — `nullptr`.

То же для ссылок (но при «не тот тип» бросает `std::bad_cast`, ловите через `try/catch`):

```
try {  
    Weapon& w = dynamic_cast<Weapon>(some_item);  
    ...  
} catch (const std::bad_cast&) {  
    std::cout << "не оружие\n";  
}
```

`dynamic_cast` использует **RTTI** (runtime type information) — компилятор записывает в `vtable` имя класса, и `dynamic_cast` это проверяет. Требуется, чтобы у базы был **хотя бы один виртуальный метод** (иначе RTTI не работает; обычно работает, потому что виртуальный деструктор уже есть).

`dynamic_cast` — относительно медленная операция (несколько указательных переходов). Не злоупотребляйте — если в каждой второй функции нужен `dynamic_cast`, скорее всего, дизайн классов плохой. Подумайте: можно ли вместо `if (Weapon*)` сделать ещё один виртуальный метод?

В нашей игре `dynamic_cast` пригодится в команде «надеть» — отличить `Armor` от `Weapon` чтобы понять, в какой слот вставлять.

18.10 Множественное наследование — кратко

В C++ можно наследоваться **сразу от нескольких классов**:

```
class Swimmer { virtual void swim() = 0; };  
class Flyer { virtual void fly() = 0; };  
  
class Duck : public Swimmer, public Flyer {  
    void swim() override { ... }  
    void fly() override { ... }  
};
```

Это **множественное наследование** (multiple inheritance). Считается «острым» — много граблей (diamond problem, неоднозначность имён). Java и C# его не разрешают вовсе; вместо этого — наследование от одного класса + реализация многих интерфейсов.

В C++ совет: **избегайте множественного наследования**, кроме одного случая:

Множественное наследование от интерфейсов — классов с одними чисто виртуальными функциями, без полей. Это безопасно и похоже на интерфейсы Java/C#.


```

class Printable {
public:
    virtual ~Printable() = default;
    virtual std::string to_string() const = 0;
};

class Comparable {
public:
    virtual ~Comparable() = default;
    virtual int compare(const Comparable& other) const = 0;
};

class Item : public Printable, public Comparable {
    ...
};

```

В нашем коде множественного наследования не будет.

18.11 Item-иерархия в проекте

Соберём всё. include/item.h:

```

#ifndef RPG_ITEM_H
#define RPG_ITEM_H

#include <string>

namespace rpg {

class Item {
public:
    Item(std::string name, int weight);
    virtual ~Item() = default;

    Item(const Item&) = delete;
    Item& operator=(const Item&) = delete;
    Item(Item&&) = delete;
    Item& operator=(Item&&) = delete;
};

```

```
const std::string& name() const;
int weight() const;

virtual std::string kind() const = 0;
virtual std::string describe() const;

protected:
    std::string name_;
    int weight_;
};

class Weapon final : public Item {
public:
    Weapon(std::string name, int weight, int damage);

    int damage() const;
    std::string kind() const override;
    std::string describe() const override;

private:
    int damage_;
};

class Armor final : public Item {
public:
    Armor(std::string name, int weight, int defense);

    int defense() const;
    std::string kind() const override;
    std::string describe() const override;

private:
    int defense_;
};

class Consumable final : public Item {
public:
    Consumable(std::string name, int weight, int heal_amount);
```

```

    int heal_amount() const;
    std::string kind() const override;
    std::string describe() const override;

private:
    int heal_amount_;
};

} // namespace rpg

#endif

```

Разберём ключевые моменты:

База Item: - Виртуальный деструктор `virtual ~Item() = default;` — обязательно. - Запрет копирования/перемещения через `= delete` — защита от slicing. - Поля `protected`, чтобы потомки могли их печатать. - `kind()` чисто виртуальный — `Item` абстрактный, его нельзя создать. - `describe()` обычный виртуальный — с дефолтной реализацией.

Потомки: - `final` после имени класса — нельзя дальше наследоваться. - `override` на всех переопределяемых методах.

`src/item.cpp` (фрагменты):

```

// Реализация базы
Item::Item(std::string name, int weight)
    : name_(std::move(name)), weight_(weight) {}

const std::string& Item::name() const { return name_; }
int Item::weight() const { return weight_; }

std::string Item::describe() const {
    std::ostringstream ss;
    ss << name_ << " (" << kind() << ", вес " << weight_ << ")";
    return ss.str();
}

```

В `Item::describe()` мы вызываем `kind()` — виртуальный! Хотя сами мы в базе, во время выполнения через `vtable` будет вызван `kind()` реального объекта. Это нормально — внутри методов базы виртуальные вызовы тоже работают полиморфно.

```
// Реализация Weapon
Weapon::Weapon(std::string name, int weight, int damage)
    : Item(std::move(name), weight), damage_(damage) {
}

int Weapon::damage() const { return damage_; }

std::string Weapon::kind() const { return "оружие"; }

std::string Weapon::describe() const {
    std::ostringstream ss;
    ss << name_ << " (оружие, урон " << damage_ << ", вес " << weight_ << ")";
    return ss.str();
}
```

Weapon явно переопределяет describe() для добавления урона. Если бы не переопределили — работала бы Item::describe(), использующая kind() = "оружие". Тоже бы вышло осмысленно, но без урона.

18.12 Тестируем полиморфизм

tests/test_items.cpp:

```
#include "item.h"
#include <iostream>
#include <vector>

int main() {
    using namespace rpg;

    std::vector<Item*> items;
    items.push_back(new Weapon("ржавый меч", 5, 8));
    items.push_back(new Armor("кожаный нагрудник", 8, 4));
    items.push_back(new Consumable("зелье лечения", 1, 15));

    // Полиморфный обход
    for (Item* it : items) {
        std::cout << "- " << it->describe() << "\n";
    }
}
```

```

// dynamic_cast
for (Item* it : items) {
    if (Weapon* w = dynamic_cast<Weapon*>(it)) {
        std::cout << w->name() << " наносит " << w->damage() << "\n";
    }
}

for (Item* it : items) {
    delete it;
}

return 0;
}

```

Запуск:

- ржавый меч (оружие, урон 8, вес 5)
- кожаный нагрудник (броня, защита 4, вес 8)
- зелье лечения (расходник, лечит 15, вес 1)

ржавый меч наносит 8 урона

Видим: один и тот же `it->describe()` даёт разный вывод для разных реальных типов. Один и тот же `dynamic_cast<Weapon*>` срабатывает только на `Weapon`. Это полиморфизм в работе.

В цикле `delete it` мы освобождаем память. Без виртуального деструктора в `Item` это вызывало бы UB и утечку `damage_/defense_/heal_amount_`. С виртуальным — корректно вызывается `~Weapon/~Armor/~Consumable`, потом `~Item`, всё чисто.

В главе 18 заменим `Item*` и `new/delete` на `std::unique_ptr<Item>` — будет ещё короче и безопаснее.

18.13 Несколько тонкостей

18.13.1 Виртуальный вызов в конструкторе/деструкторе

В конструкторе базы виртуальный механизм **не работает**:

```

class Item {
public:
    Item() {
        std::cout << "creating " << kind(); // !!! вызовет Item::kind, не Weapon::kind
    }
}

```

```
    }  
    virtual std::string kind() const = 0;  
};
```

На момент конструктора `Item` объект ещё не «стал» `Weapon`. `Vtable` указывает на `vtable Item`. Для чисто виртуального метода (`= 0`) это вообще приведёт к **UB** или к ошибке линковки.

В деструкторе то же — `vtable` уже «откатилась» к базе.

Правило: не вызывайте виртуальные методы из конструктора/деструктора базы.

Если нужно похожее поведение — отдельный метод `init()`, который вызывается после конструктора.

18.13.2 Вызов метода базы из потомка

Иногда из переопределённого метода хочется вызвать базовую реализацию:

```
class Weapon : public Item {  
public:  
    std::string describe() const override {  
        return Item::describe() + ", урон " + std::to_string(damage_);  
    }  
};
```

`Item::describe()` — явный квалифицированный вызов. Обходит виртуальный механизм, вызывает именно версию `Item`.

Удобно, когда хочется «дополнить» базовое поведение, не дублируя его. У нас в книге пока не пользуемся, но идиома известная.

18.13.3 `protected` и инвариант класса

Делать поля `protected` — иногда соблазнительно, но риск.

Если поле `private`, доступ только через методы. Можете в методе валидировать (например, `take_damage` не пускает `hp` в отрицательные). Потомки тоже идут через метод.

Если поле `protected`, потомки могут менять напрямую. Инварианты класса могут нарушиться.

Рекомендация: лучше держать поля `private` и давать `protected` getters/setters, если потомкам нужно. Но это в идеале; в нашем `Item` поля `protected` для удобства (потомки печатают их в `describe`).

18.14 Альтернативы наследованию

Наследование не единственный способ переиспользования. Современный C++ часто советует **композицию вместо наследования**:

```
// Вместо наследования
class Magic : public Weapon { ... };

// Композиция
class Weapon {
    MagicEffect effect_; // оружие имеет эффект
};
```

«Оружие имеет эффект» — композиция (has-a). «Магическое оружие — это оружие» — наследование (is-a). Если is-a сомнительно, лучше композиция.

Наследование добавляет **связанность**: потомок зависит от базы. Менять базу больно — все потомки могут сломаться. Композиция связана слабее: меняете внутреннее устройство, наружу не видно.

В нашей игре Item → Weapon/Armor/Consumable — настоящий is-a. «Меч — это предмет». Полностью оправдано.

Но не делайте наследование «потому что в учебнике написано». Если можно композицией — обычно лучше композицией.

18.15 Главные правила главы

1. **Виртуальный деструктор** для любого класса, который задумывается как база. **Обязательно.**
2. **override** на каждом переопределении. Бесплатная защита от опечаток.
3. **final** на классах/методах, которые не будут расширять — даёт компилятору оптимизировать.
4. **Никогда не присваивайте объект потомка в переменную базы по значению** — slicing.
5. **Запрещайте копирование** для классов в иерархии — = delete. Доступ только через указатели/ссылки.
6. **dynamic_cast** для безопасного приведения — но злоупотребление — признак плохого дизайна.
7. **Виртуальные методы не работают в конструкторе/деструкторе базы** — не зовите их там.

8. **Композиция часто лучше наследования** — наследуйте только когда is-a реально и иерархия плоская.

18.16 Маленькое упражнение

1. Добавьте новый тип предмета — `KeyItem` (квестовый предмет, нельзя выбросить). Чисто наследник `Item` с переопределённым `kind()` = "квестовый" и описанием.
2. Создайте `Item* it = new Weapon(...)` без виртуального деструктора в `Item` (нарочно уберите `virtual ~Item()`). Запустите под `ASan` (`make CXXFLAGS=... с -fsanitize=address`). `ASan` должен указать утечку.
3. Попробуйте `Item item("test", 5);` — должна быть ошибка компиляции (абстрактный класс). Прочтите сообщение.
4. Уберите `override` у `Weapon::describe()` и переименуйте его в `describe()`. Что произойдёт при сборке? Что произойдёт при попытке вызвать `weapon.describe()`?
5. Добавьте в `Item` виртуальный метод `int value() const` — «цена предмета». В `Weapon` — `damage_ * 10`, в `Armor` — `defense_ * 15`, в `Consumable` — `heal_amount_ * 5`. Используйте в тесте.
6. (Сложнее) Добавьте `virtual std::unique_ptr<Item> clone() const` для копирования предметов. Реализуйте в потомках. Какие проблемы это создаёт?
7. (Сложнее) Перепишите `Item` так, чтобы `Weapon/Armor/Consumable` не наследовались, а использовали композицию: `Item` хранит `enum Type` и `union` для специфических данных. Сравните, какой подход проще для добавления нового типа.

18.17 Что дальше

Глава 18 — **умные указатели**: `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`. Применим их к нашим `Item`: уберём ручной `new/delete`, заменим на `unique_ptr<Item>`. Добавим **Inventory** — игрок сможет таскать предметы. Локации тоже смогут содержать предметы — командой `take` будем их подбирать.

Это завершит «инфраструктуру предметов». После — бои, NPC, парсер мира из файла, сейвы. К 25-й главе соберётся всё.

Глава 19

Глава 18. Умные указатели — unique_ptr, shared_ptr, weak_ptr

В предыдущей главе мы построили иерархию `Item`. Тестировали полиморфизм через `std::vector<Item*>`, выделяя предметы через `new` и освобождая через `delete`. Это работало, но было хрупко: забудьте `delete` — утечка, забудьте про виртуальный деструктор — UB, выкиньте исключение между `new` и `delete` — снова утечка.

Эта глава — про **умные указатели** (smart pointers). Они владеют объектом на куче и сами заботятся о его освобождении в нужный момент. Никаких ручных `delete`. Никаких утечек. Это и есть «современный C++» в одном понятии.

Параллельно построим класс `Inventory`, и в игре наконец-то можно будет брать и бросать предметы.

19.1 Зачем нужны умные указатели

Вспомним проблему. У нас был такой код:

```
std::vector<Item*> items;
items.push_back(new Weapon("меч", 5, 10));
// ...

for (Item* it : items) {
    delete it;
}
items.clear();
```

Что может пойти не так:

1. **Забыли `delete`.** Память не освобождена.
2. **Между `new` и `delete` исключение.** Деструкторы не вызовутся для оставшихся.
3. **Возвращаем `Item*` из функции.** Кто его освобождает? Без документации непонятно.
4. **Два указателя на один объект.** Оба пытаются `delete` — UB.
5. **Использование после `delete`.** Висячий указатель — UB.

В главе 11 я ввёл RAII как решение этих проблем. Умные указатели — это RAII для куча-указателей. Они конкретные классы, которые **владеют** объектом на куче и в деструкторе вызывают `delete`.

Их три:

- `std::unique_ptr<T>` — единоличный владелец. Только один указатель на объект.
- `std::shared_ptr<T>` — совладельцы. Объект живёт, пока есть хотя бы один указатель.
- `std::weak_ptr<T>` — наблюдатель. Сам не владеет, но может проверить, жив ли объект.

Все три — в заголовке `<memory>`.

19.2 `unique_ptr` — основной рабочий

`std::unique_ptr<T>` — самый частый. Если вы выделяете объект на куче и есть **один** ответственный за его жизнь — это `unique_ptr`.

19.2.1 Создание

```
#include <memory>

std::unique_ptr<int> p(new int(42));
std::cout << *p << "\n";    // 42
```

`p` владеет `int`-ом со значением 42. Когда `p` выходит из области видимости (или уничтожается как-то иначе), его деструктор делает `delete` на хранимый указатель.

Полная безопасность: даже если будет исключение между `new int(42)` и концом функции — `p` всё равно уничтожится, `int` освободится.

Использование почти как обычный указатель:

```
*p = 100;           // разыменование
p->method();         // вызов метода (если T — класс)
T* raw = p.get();    // получить сырой указатель (без передачи владения)
bool ok = bool(p);   // проверка на nullptr — p != nullptr
```

19.2.2 Запрет копирования

Главное свойство: unique_ptr **нельзя копировать**.

```
std::unique_ptr<int> p(new int(42));  
std::unique_ptr<int> q = p;    // ОШИБКА компиляции
```

Почему? Если бы можно было — у нас стало бы два владельца, оба бы попытались delete. Двойное освобождение.

Поэтому копи-конструктор unique_ptr помечен = delete. Создатели стандартной библиотеки об этом подумали.

19.2.3 Перемещение разрешено

Что можно — **переместить** владение:

```
std::unique_ptr<int> p(new int(42));  
std::unique_ptr<int> q = std::move(p);    // q теперь владеет, p пустой  
  
if (!p) std::cout << "p пустой\n";    // выведется  
std::cout << *q << "\n";              // 42
```

std::move(p) говорит: «забери у p всё, что он держит». После этого p становится nullptr, владельцем становится q.

Это идиома **передачи владения**. В функциях:

```
std::unique_ptr<Item> make_sword() {  
    return std::unique_ptr<Item>(new Weapon("меч", 5, 10));  
}  
  
void use_item(std::unique_ptr<Item> item) {  
    // функция взяла владение  
    std::cout << item->describe();  
    // в конце item уничтожится, delete вызовется  
}  
  
int main() {  
    std::unique_ptr<Item> sword = make_sword();  
    // ...  
    use_item(std::move(sword));
```

```
// sword теперь пустой
}
```

Возврат unique_ptr из функции — нормально (move выполняется автоматически). Передача в функцию — через std::move.

19.2.4 make_unique — фабричная функция

В C++14 добавили std::make_unique:

```
auto p = std::make_unique<Weapon>("меч", 5, 10);
// эквивалент: std::unique_ptr<Weapon>(new Weapon("меч", 5, 10))
```

Преимущество — нет голого new. И ещё чуть-чуть исключение-безопасности (между new и конструктором unique_ptr теоретически могло что-то выкинуться, с make_unique — нет).

В C++11 make_unique нет. У нас C++11-база. Поэтому пишем шим:

```
// include/util.h
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Поместил в namespace rpg, чтобы не конфликтовать с системным std::make_unique (хотя в C++11 его и нет — но в C++14+ есть, и наш код тогда должен работать).

Args&&... — variadic template, передаёт аргументы как они есть (perfect forwarding). Шаблоны и &&... — это глава 20. Сейчас просто принимайте как идиому: «rpg::make_unique<T>(args...) создаёт unique_ptr<T> с передачей args в конструктор T».

Использование:

```
auto sword = rpg::make_unique<Weapon>("меч", 5, 10);
```

Это и есть современный способ выделения на куче в нашем коде.

19.2.5 Кастомные deleters

unique_ptr обычно использует delete для освобождения. Иногда нужно другое — например, для C-API нужно вызвать fclose:

```
auto deleter = [](FILE* f) { if (f) std::fclose(f); };
std::unique_ptr<FILE, decltype(deleter)> file(std::fopen("test.txt", "r"), deleter);
// file сам закроет файл
```

Это редко нужно — обычно лучше написать свой RAII-класс. Но в системном коде встречается.

19.2.6 unique_ptr для массивов

Для массивов есть особая форма unique_ptr<T[]>:

```
std::unique_ptr<int[]> arr(new int[100]);
arr[0] = 1;
arr[99] = 100;
// в деструкторе автоматически delete[]
```

Использует delete[] (не delete). Поэтому отдельный синтаксис.

В современном C++ почти всегда вместо unique_ptr<T[]> берут std::vector<T>. Вектор тот же массив на куче, плюс знает свой размер. Используйте unique_ptr<T[]> редко, для совместимости со старым кодом.

19.3 shared_ptr — совладение

Иногда у объекта несколько владельцев. Пример: окно UI, на которое ссылаются и менеджер окон, и виджет внутри окна. Какой из них освобождает? Если кто-то один — другой может потом обратиться к освобождённой памяти.

Решение — **подсчёт ссылок**. Объект живёт, пока есть хоть один указатель на него. Уничтожается, когда последний пропадает.

```
std::shared_ptr<int> a(new int(42));
std::cout << a.use_count();    // 1

{
    std::shared_ptr<int> b = a;    // КОПИРУЕТСЯ! не move
    std::cout << a.use_count();    // 2
    std::cout << b.use_count();    // 2
}
// здесь b уничтожается, счётчик становится 1

std::cout << a.use_count();    // 1
// в конце a уничтожится, счётчик = 0, объект delete'ится
```

В отличие от unique_ptr, **shared_ptr копируется** — это безопасно благодаря подсчёту.

19.3.1 make_shared

```
auto p = std::make_shared<Weapon>("меч", 5, 10);
```

Это C++11 — у нас работает.

make_shared ещё лучше make_unique: он делает **одну** аллокацию для объекта и для счётчика (внутри shared_ptr есть «control block»). Если выделять через shared_ptr<T>(new T(...)) — будут две аллокации (объект и control block отдельно). Маленькая, но реальная экономия.

19.3.2 Цена shared_ptr

shared_ptr не бесплатный:

1. **Размер:** 16 байт (два указателя — на объект и на control block), вместо 8 у unique_ptr.
2. **Счётчик:** каждое копирование увеличивает счётчик (атомарная операция!), каждое уничтожение уменьшает. Это медленнее простого копирования.
3. **Косвенный доступ:** чтобы добраться до объекта, нужно идти через control block.

Поэтому правило: **используйте unique_ptr по умолчанию. shared_ptr только когда совладение реально нужно.**

В моих проектах — shared_ptr нужен раза в десять реже, чем unique_ptr.

19.3.3 Циклы — главная ловушка shared_ptr

Допустим, у нас два класса, ссылающиеся друг на друга:

```
struct Node {
    std::shared_ptr<Node> next;
    ~Node() { std::cout << "destroyed\n"; }
};

int main() {
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();
    a->next = b;
    b->next = a;    // цикл!
}
```

Что произойдёт в конце main?

1. b уничтожается, счётчик на b-объекте становится 1 (он держится из a->next).
2. a уничтожается, счётчик на a-объекте становится 1 (держится из b->next).

3. Никого не уничтожаем — счётчики не дошли до нуля.

Утечка. Деструкторы не вызовутся, "destroyed" не выведется.

Это серьёзная проблема. shared_ptr не умеет автоматически разрешать циклы (для этого нужен сборщик мусора). Решение — weak_ptr.

19.4 weak_ptr — наблюдатель без владения

std::weak_ptr<T> — указатель на объект, который не **держит** его. То есть не влияет на счётчик. Но может проверить, жив ли объект ещё.

```
std::shared_ptr<int> s = std::make_shared<int>(42);
std::weak_ptr<int> w = s;

std::cout << s.use_count();    // 1 (weak не считается)

if (auto locked = w.lock()) {   // попытка временно получить shared_ptr
    std::cout << *locked << "\n";    // 42
} else {
    std::cout << "объект уничтожен\n";
}

s.reset();    // уничтожаем

if (auto locked = w.lock()) {
    std::cout << *locked << "\n";
} else {
    std::cout << "объект уничтожен\n";    // эта ветка
}
```

w.lock() пытается превратить weak_ptr в shared_ptr. Если объект ещё жив — возвращает рабочий shared_ptr. Если уничтожен — возвращает пустой shared_ptr.

Это позволяет разорвать цикл:

```
struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> back;    // обратная ссылка слабая
};

auto a = std::make_shared<Node>();
```

```

auto b = std::make_shared<Node>();
a->next = b;
b->back = a;    // weak — НЕ увеличивает счётчик a

// В конце оба уничтожаются нормально:
// b уничтожается → счётчик a остаётся 1 (только переменная a)
// a уничтожается → счётчик a = 0 → delete → a->next пустеет → b уже мёртв

```

В нашей игре weak_ptr нам пригодится для «NPC помнит, кто ему дал квест» — без weak это создавало бы цикл.

19.5 Когда что

Сводная таблица:

Указатель	Когда брать	Стоимость
T* (raw)	Только для «не владею, просто смотрю» — например, параметр функции.	бесплатно
unique_ptr<T>	Единственный владелец динамического объекта.	8 байт
shared_ptr<T>	Несколько владельцев.	16 байт + control block + атомарный счётчик
weak_ptr<T>	Наблюдатель в графе объектов с shared_ptr.	16 байт

По умолчанию — unique_ptr. Используйте shared_ptr только когда реально несколько владельцев. Используйте weak_ptr против циклов. Используйте сырой указатель когда не владеете.

19.6 Inventory — наш первый shared_ptr... ах нет, unique_ptr

В нашей игре каждым предметом владеет ровно **один** держатель: либо локация (предмет на земле), либо инвентарь игрока. При подъёме — передача владения. При выбрасывании — обратная.

Это **классический случай** для unique_ptr<Item>.

include/inventory.h:


```

class Inventory {
public:
    Inventory() = default;

    void add(std::unique_ptr<Item> item);
    std::unique_ptr<Item> take(const std::string& name);
    const Item* find(const std::string& name) const;
    bool empty() const;
    const std::vector<std::unique_ptr<Item>>& items() const;
    int total_weight() const;

private:
    std::vector<std::unique_ptr<Item>> items_;
};

```

Поле items_ — vector из unique_ptr<Item>. Каждый предмет хранится через указатель на базу (Item* под капотом), что даёт **полиморфизм**: в одном векторе лежат Weapon, Armor, Consumable. При уничтожении вектора каждый unique_ptr сделает delete на свой указатель, виртуальный деструктор позаботится о правильной очистке.

19.6.1 add — принимаем владение

```

void Inventory::add(std::unique_ptr<Item> item) {
    if (item) {
        items_.push_back(std::move(item));
    }
}

```

Параметр std::unique_ptr<Item> item — **по значению**. Чтобы передать — нужно std::move(...) снаружи:

```

inv.add(rpg::make_unique<Weapon>("меч", 5, 10)); // временный, move автоматический

```

Внутри add мы перемещаем item в вектор (push_back(std::move(item))). После этого локальный item пустеет.

19.6.2 take — отдаём владение

```

std::unique_ptr<Item> Inventory::take(const std::string& name) {
    auto it = std::find_if(items_.begin(), items_.end(),

```

```

        [&name](const std::unique_ptr<Item>& p) {
            return p && p->name() == name;
        });
    if (it == items_.end()) {
        return nullptr;
    }
    std::unique_ptr<Item> result = std::move(*it);
    items_.erase(it);
    return result;
}

```

Находим предмет по имени через `find_if` с лямбдой. Лямбда: «возьми указатель из вектора, проверь, что он не пустой и что имя предмета совпадает».

Если нашли — `std::move(*it)` забирает предмет из вектора (теперь там пустой `unique_ptr`), а `erase` удаляет эту пустышку. Возвращаем найденный.

Вызывающий получает `unique_ptr<Item>` — владение перешло к нему.

19.6.3 find — наблюдение без владения

```

const Item* Inventory::find(const std::string& name) const {
    auto it = std::find_if(items_.begin(), items_.end(),
        [&name](const std::unique_ptr<Item>& p) {
            return p && p->name() == name;
        });
    if (it == items_.end()) return nullptr;
    return it->get();
}

```

Возвращает **сырой** `const Item*`. Это «я не владею, я просто хочу посмотреть». Точно так же, как ссылка, но может быть `nullptr`.

Главное: **не вызывайте delete на этом указателе**. Владение остаётся у инвентаря.

`it->get()` — метод `unique_ptr`, возвращает сырой указатель без передачи владения.

19.6.4 Лямбда с захватом по ссылке

В лямбдах `find_if`:

```
[&name](const std::unique_ptr<Item>& p) { ... }
```

[&name] — захват по ссылке. Лямбда видит снаружи переменную name через ссылку. Это безопасно: лямбда живёт ровно столько же, сколько find_if (несколько миллисекунд), name не уничтожится за это время.

Если бы захват был по значению ([name]), копировалась бы строка. Для коротких имён ничего страшного, но привычка & для read-only — лучше.

19.7 Применение в Location и Player

Локация теперь хранит Inventory:

```
class Location {
public:
    // ...
    Inventory& items();
    const Inventory& items() const;
private:
    // ...
    Inventory items_;
};
```

Игрок тоже:

```
class Player {
public:
    // ...
    Inventory& inventory();
    const Inventory& inventory() const;
private:
    // ...
    Inventory inventory_;
};
```

И заполняем локации в make_demo_world:

```
glade.items().add(rpg::make_unique<Weapon>("ржавый меч", 5, 8));
village.items().add(rpg::make_unique<Consumable>("зелье лечения", 1, 15));
forest.items().add(rpg::make_unique<Armor>("кожаный нагрудник", 8, 4));
```

Каждая локация теперь имеет один предмет.

19.8 Команды take / drop / use

В Game добавились методы:

```
void Game::cmd_take(const std::string& item_name) {
    Location* loc = world_.find(current_location_id_);
    if (!loc) return;

    std::unique_ptr<Item> picked = loc->items().take(item_name);
    if (!picked) {
        std::cout << "Здесь нет '" << item_name << "'.\n";
        return;
    }
    std::cout << "Вы подняли: " << picked->name() << "\n";
    player_.inventory().add(std::move(picked));
}
```

Логика: вынимаем предмет из инвентаря локации (получаем unique_ptr), кладём в инвентарь игрока. **Между двумя строками владение временно у локальной переменной picked** — без проблем, потому что в конце мы её отдаём через std::move.

Заметили: World::find теперь возвращает Location* (без const) — потому что мы модифицируем содержимое локации. Добавили перегрузку:

```
Location* World::find(const std::string& id);           // для модификации
const Location* World::find(const std::string& id) const; // для чтения
```

cmd_drop — наоборот:

```
void Game::cmd_drop(const std::string& item_name) {
    std::unique_ptr<Item> dropped = player_.inventory().take(item_name);
    if (!dropped) {
        std::cout << "У вас нет '" << item_name << "'.\n";
        return;
    }
    Location* loc = world_.find(current_location_id_);
    // ...
    loc->items().add(std::move(dropped));
}
```

cmd_use — пример полиморфизма через dynamic_cast:

```

void Game::cmd_use(const std::string& item_name) {
    const Item* item = player_.inventory().find(item_name);
    if (!item) {
        std::cout << "У вас нет '" << item_name << "'.\n";
        return;
    }

    if (const Consumable* c = dynamic_cast<const Consumable*>(item)) {
        int heal = c->heal_amount();
        std::cout << "Вы выпили " << c->name() << " (" << heal << " HP).\n";
        player_.heal(heal);
        player_.inventory().take(item_name);    // удаляет
        cmd_status();
    } else {
        std::cout << "Это нельзя «использовать».\n";
    }
}

```

find даёт сырой const Item* — наблюдение. dynamic_cast<const Consumable*> проверяет, расходник ли. Если да — лечим и удаляем предмет. Если нет (оружие, броня) — отказ.

19.9 Запуск

```

$ make && echo -e "look\ntake ржавый меч\ninventory\nquit" | ./build/rpg
== Лесная поляна ==
...
На земле:
- ржавый меч (оружие, урон 8, вес 5)
> Вы подняли: ржавый меч
> Инвентарь (общий вес 5):
- ржавый меч (оружие, урон 8, вес 5)
> До свидания.

```

Игра умеет: - Брать предметы с земли (take). - Носить их в инвентаре (inventory). - Бросать обратно (drop). - Использовать расходники (use).

И всё это **без единого ручного delete**. Каждый предмет с момента создания через make_unique и до конца программы безопасно владеется одним unique_ptr. Передачи владения через std::move явны.

19.10 Сырой указатель — когда уместен

Может показаться: unique_ptr так хорош, что сырые указатели вообще не нужны. Это не так.

Сырой T* уместен, когда вы не владеете объектом. Например, как параметр функции:

```
void print_item(const Item* it) {    // не владею, просто смотрю
    if (it) std::cout << it->describe();
}
```

Или возврат «может не найтись» (как find):

```
const Item* find(const std::string& name) const;
```

Сырой указатель == **non-owning view**. Это его смысл в современном C++.

Если же функция **выделяет** объект и возвращает — берите unique_ptr. Если **принимает владение** — принимайте unique_ptr по значению. Без unique_ptr ваш интерфейс будет неясен: «кто освобождает? непонятно».

19.11 Тонкость: передать unique_ptr нельзя по значению, можно только через move

Часто новички пишут:

```
auto p = rpg::make_unique<Weapon>("меч", 5, 10);
inventory.add(p);    // ОШИБКА: unique_ptr нельзя копировать
```

Компилятор укажет на это. Решение:

```
inventory.add(std::move(p));    // ОК: передача владения
// после этого p пустой
```

Если же предмет создаётся «временным» — move происходит автоматически:

```
inventory.add(rpg::make_unique<Weapon>("меч", 5, 10));    // ОК без move
```

Временный объект — это rvalue, для него unique_ptr автоматически перемещается. Подробности — в главе 19.

19.12 shared_ptr — куда мы его не применили

В нашей игре shared_ptr не появился, и это правильно. Каждый предмет имеет одного владельца — unique_ptr достаточно.

Кейс, где может пригодиться `shared_ptr`: - **Шаблоны предметов**. Допустим, у нас 100 одинаковых «зелий лечения» в магазине, и игрок носит «ссылку» на шаблон. `shared_ptr<ItemTemplate>` — все 100 ссылок указывают на один шаблон. При удалении магазина шаблон жив, пока игрок носит зелье. - **NPC, на которого указывают разные квесты**. Если один и тот же NPC участвует в 5 квестах — `shared_ptr<NPC>` от каждого квеста.

В нашем коде до 25-й главы `shared_ptr` появится в одном-двух местах, не больше. `unique_ptr` — рабочая лошадка.

19.13 `weak_ptr` — куда мы его не применили

В нашей игре циклов нет (предмет не знает свой инвентарь, инвентарь знает предметы). Поэтому `weak_ptr` пока не нужен.

В большом проекте `weak_ptr` появляется в момент, когда вы заметили утечку через цикл `shared_ptr`. Тогда одно из направлений ссылок делаете `weak_ptr` — обычно «обратное» или «менее важное».

19.14 Главные правила главы

1. **Никогда не пишите `new T(...)` без `unique_ptr` или эквивалента.** Голый `new` в C++17+ — почти всегда баг.
2. **`unique_ptr<T>` по умолчанию.** Самый дешёвый и понятный.
3. **`make_unique<T>(args)` через шим в C++11, через стандарт в C++14+.**
4. **`shared_ptr<T>` только когда совладение реально нужно.** Цена — двойной размер, атомарный счётчик.
5. **`weak_ptr<T>` против циклов в `shared_ptr`-графе.**
6. **Сырой `T*` означает “не владею”.** Только для параметров и наблюдения.
7. **Передача `unique_ptr` через `std::move`, временные — автоматически.**
8. **`get()` даёт сырой указатель без передачи владения.** Используйте только когда нужно отдать в C-API.
9. **`reset()` на `unique_ptr` освобождает текущий и опционально берёт новый.**

19.15 Маленькое упражнение

1. Добавьте на одну локацию второй предмет (например, в лесу не только броня, но и палка-weapon). Проверьте, что `look` показывает оба, `take` берёт по имени.
2. Добавьте проверку «вес инвентаря не больше 30» — если превышен, `take` отказывает.

3. Команда `examine <item>` — показать описание предмета **в инвентаре**, не вынимая. Использовать `find` (тот, что возвращает `const Item*`).
4. Перепишите `Game::cmd_use` так, чтобы для оружия выдавало «Это оружие используется в бою через `wield`», для брони — аналогично. Используйте `dynamic_cast`.
5. (Сложнее) Добавьте поле `Item* equipped_weapon_` в `Player` — текущее оружие. Команда `wield <item>` берёт оружие из инвентаря и «надевает». Заметьте: `Item*` — сырой, потому что **владелец — инвентарь**. `equipped_weapon_` — лишь наблюдает.
6. (Сложнее) Сравните размеры под разные указатели:

```
std::cout << sizeof(Item*) << "\n";           // 8
std::cout << sizeof(std::unique_ptr<Item>) << "\n"; // 8
std::cout << sizeof(std::shared_ptr<Item>) << "\n"; // 16
std::cout << sizeof(std::weak_ptr<Item>) << "\n";  // 16
```

Объясните, почему `unique_ptr` остался 8.

7. Создайте циклический `shared_ptr` (две `Node`, каждая держит `shared_ptr` на другую). Запустите под ASan. Что покажет (или не покажет)?

19.16 Что дальше

Глава 19 — **move-семантика** глубоко. Что такое `rvalue`, `lvalue`, `rvalue`-ссылки. Что делает `std::move` на самом деле. Зачем нужны `move`-конструктор и `move`-присваивание. Когда компилятор сам инлайнит вызов и не делает копий (RVO/NRVO). После этой главы у вас не останется загадок вокруг производительности.

Глава 20 — шаблоны. Сделаем `EventBus<T>` для игровых событий. После — лямбды и `std::function` (21), исключения и `exception safety` (22). На главе 23 начнём парсер скриптов локаций — мир будет загружаться из текстового файла.

Глава 20

Глава 19. Move-семантика на бою

Move-семантика — одна из главных новинок C++11 и одна из самых страшно выглядящих. На самом деле идея простая: вместо копирования объекта **перемещать** его. Если объект тяжёлый — это огромная экономия. Если временный — это вообще без накладных расходов.

Мы уже несколько глав используем `std::move` и `&&` в сигнатурах, не объясняя глубоко. Эта глава — подробный разбор: что такое rvalue, что такое rvalue-ссылки, что **на самом деле** делает `std::move`, как пишут move-конструктор, что такое RVO и почему компилятор иногда вообще ничего не копирует.

После этой главы у вас в голове сложится цельная картина того, как объекты живут, копируются, перемещаются и умирают в C++.

20.1 Зачем move

Допустим, у нас функция, возвращающая большую коллекцию:

```
std::vector<int> generate(int n) {
    std::vector<int> result(n);
    for (int i = 0; i < n; ++i) result[i] = i * i;
    return result;
}

int main() {
    auto v = generate(1000000);
}
```

Что происходит?

1. Внутри `generate` создаётся `result` — миллион `int`-ов на куче.
2. `return result;` — возврат.
3. В `main` создаётся `v` — копия?

В C++98 — **да**, это была копия. Миллион `int`-ов выделить заново и скопировать — несколько мегабайт памяти. Дорого.

В C++11 благодаря move-семантике (и RVO ещё раньше) — **нет**. Никакой копии. `v` получает «внутренности» `result` напрямую. Скорость — наносекунды.

Этот сдвиг изменил, как пишут на C++. Большие объекты можно свободно возвращать из функций, передавать в контейнеры. Раньше использовали указатели и аллокации руками, а в современном C++ — просто значения. И они быстрые.

20.2 lvalue и rvalue

Начнём с фундамента. В C++ есть две категории выражений (упрощённо):

lvalue — то, что имеет адрес в памяти, на что можно «указать». Имеет имя и/или живёт дольше одного выражения.

```
int x = 5;    // x — lvalue, у неё есть адрес &x
int& r = x;   // r — тоже lvalue (другое имя для x)
x = 10;       // ОК: можно присвоить
```

rvalue — временное значение. Живёт ровно одно выражение. Адреса нет (формально).

```
5           // rvalue (литерал)
x + 1       // rvalue (результат вычисления)
foo()       // rvalue (если foo возвращает по значению)
std::move(x) // rvalue (об этом ниже)
```

Можно попробовать:

```
&5;         // ОШИБКА: cannot take the address of an rvalue
&(x + 1);   // ОШИБКА: то же
&x;         // ОК: x это lvalue
```

«L» от «left side of assignment»: `x = 5;` — слева lvalue, справа rvalue. «R» — соответственно «right».

Это упрощённая модель. В современном C++ категорий выражений больше (lvalue, xvalue, prvalue, glvalue), но для нашего обсуждения хватит lvalue/rvalue.

20.3 Главное различие на практике

С rvalue можно делать вещи, которых нельзя с lvalue: - «Ограбить» — забрать у него внутренности. Раз он временный — никто не заметит.

С lvalue вообще-то тоже можно (объявлено как `std::move`), но это **программистское решение**: «я знаю, что эту переменную больше использовать не буду, можно её опустошить».

20.4 Rvalue-ссылки: T&&

Чтобы функция могла различать lvalue и rvalue, в C++11 добавили **rvalue-ссылки**:

```
void f(int& x) { std::cout << "lvalue\n"; }
void f(int&& x) { std::cout << "rvalue\n"; }

int main() {
    int a = 5;
    f(a);      // "lvalue"
    f(5);      // "rvalue"
    f(a + 1);  // "rvalue"
    f(std::move(a)); // "rvalue"
}
```

`int&` — обычная ссылка на lvalue. `int&&` — ссылка на rvalue.

Двойной амперсанд `&&` это **отдельный знак**, не два амперсанда. Логического И тут нет.

Обе ссылки — это «псевдоним переменной». Различие — в том, **какую категорию выражения** они принимают.

20.4.1 Что внутри rvalue-ссылки

Парадокс: переменная-rvalue-ссылка сама является **lvalue**:

```
void f(int&& x) {
    // x — это rvalue-ссылка, но сама x — lvalue
    int& r = x;    // OK: lvalue

    // чтобы передать дальше как rvalue:
    int&& r2 = std::move(x);
    // или
```

```
other_function(std::move(x));
}
```

Это путает. Запомните: rvalue-ссылка — это **способ принять rvalue**, но внутри функции переменная имеет имя и адрес, то есть lvalue.

20.5 Move-конструктор и move-присваивание

Раз можно перегружать функции по rvalue, можно перегрузить и **конструктор**. Это и есть move-конструктор:

```
class Buffer {
public:
    // Обычный (copy) конструктор: принимает lvalue
    Buffer(const Buffer& other);

    // Move-конструктор: принимает rvalue
    Buffer(Buffer&& other) noexcept;
};
```

Зачем? Сору-конструктор делает **глубокую копию** — выделяет новую память, копирует байты. Move-конструктор просто **забирает** указатель из источника:

```
class Buffer {
public:
    // сору: новая память + копирование
    Buffer(const Buffer& other)
        : data_(new int[other.size_]),
          size_(other.size_) {
        for (std::size_t i = 0; i < size_; ++i) {
            data_[i] = other.data_[i];
        }
    }

    // move: забираем чужие внутренности
    Buffer(Buffer&& other) noexcept
        : data_(other.data_),
          size_(other.size_) {
        other.data_ = nullptr;    // обнуляем источник
        other.size_ = 0;
    }
};
```

```

    }

    ~Buffer() {
        delete[] data_; // если data_ == nullptr, delete[] просто ничего не делает
    }

private:
    int* data_;
    std::size_t size_;
};

```

Сравним стоимость: - Сору: `new int[N] + memcpy(N*sizeof(int))`. На миллионе элементов — миллисекунды. - Move: четыре присваивания указателей и чисел. **Наносекунды.**

В сто-тысячу раз быстрее. Особенно для строк и контейнеров.

20.5.1 поехсепт на move-конструкторе

Заметили поехсепт в сигнатуре move-конструктора? Это обещание «не бросает исключений». Важно!

Если move-конструктор может бросить — `std::vector` при перевыделении (когда `push_back` исчерпал `capacity`) **откажется** его использовать и упадёт на сору. Стандарт говорит: контейнерам нужна **сильная exception guarantee**. Они полагаются на поехсепт move.

Правило: помечайте move-конструктор/присваивание поехсепт, если не делаете ничего, что может выкинуть. Обычно так и есть — простые присваивания указателей и чисел.

20.5.2 Move-присваивание

Тот же принцип для оператора `=`:

```

class Buffer {
public:
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            delete[] data_;
            data_ = new int[other.size_];
            size_ = other.size_;
            for (std::size_t i = 0; i < size_; ++i) {
                data_[i] = other.data_[i];
            }
        }
    }
};

```

```

    }
    return *this;
}

Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {
        delete[] data_;           // освобождаем своё
        data_ = other.data_;      // забираем чужое
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}
};

```

Move-присваивание сначала освобождает свои ресурсы, потом забирает чужие. И обнуляет источник.

`if (this != &other)` — проверка от **self-assignment** (`a = std::move(a);`). Без неё `delete[] data_` уничтожил бы то, что мы потом пытаемся забрать у того же объекта.

20.6 std::move — что это на самом деле

Я говорил «`std::move` — это перемещение». На самом деле — **не совсем**.

`std::move(x)` **ничего не перемещает**. Это просто **приведение типа** — оно превращает `x` в `rvalue`.

Грубо:

```

template <typename T>
typename std::remove_reference<T>::type&& move(T&& x) noexcept {
    return static_cast<typename std::remove_reference<T>::type&&>(x);
}

```

Это шаблон, который принимает что угодно и кастует к `rvalue`-ссылке. Никакого «перемещения» в этой функции нет. Имя `move` — обманчивое, лучше бы было `rvalue_cast`.

Перемещение делает потом — `move`-конструктор или `move`-присваивание, к которым приводит эта `rvalue`.

```
Buffer a(100);
Buffer b = std::move(a);
// std::move(a) превратило a в rvalue
// потом вызывается Buffer(Buffer&&) — move-конструктор
// внутри он опустошает a
```

Сам по себе `std::move` не делает ничего. Без move-конструктора (если вы его не определили) `std::move` бесполезен — компилятор откатится на `copy`.

20.7 Когда move бесплатен

Move-конструктор имеет смысл, когда у класса есть **дорогой ресурс**: динамическая память, файл, сокет, мьютекс.

Для классов с одними тривиальными полями (`int`, `double`, `bool`) `move` = `copy`. Никакой разницы.

Стандартные классы, у которых полезный `move`:

- `std::string` — для длинных строк.
- `std::vector` — внутренний массив переходит к новому владельцу.
- `std::unique_ptr` — указатель переходит (мы уже это делали).
- `std::shared_ptr` — указатель + контрол блок (но без атомарного увеличения счётчика).
- `std::map`, `std::unordered_map`, `std::set`, `std::list`, и так далее.

Для собственных классов с динамической памятью — пишите `move`.

20.8 Rule of five детально

В главе 11 и 16 я упоминал `rule of five`. Теперь полная картина.

Если ваш класс **управляет ресурсом** (нетривиальный деструктор), вам нужны **пять** специальных функций:

1. **Деструктор** — освобождает ресурс.
2. **Сору-конструктор** — глубокая копия (или запрет).
3. **Сору-присваивание** — глубокое копирование с предварительным освобождением.
4. **Move-конструктор** — забор ресурса с обнулением источника.
5. **Move-присваивание** — освобождение своего + забор + обнуление.

Это **rule of five**. C++11 разделил его на «обязательную часть» (1-3) и «опциональный плюс» (4-5).

Если все пять не нужны, явно их пишите как `= delete` или `= default`:

```
class Unique {
public:
    Unique() = default;
    ~Unique() = default;

    Unique(const Unique&) = delete;
    Unique& operator=(const Unique&) = delete;

    Unique(Unique&&) noexcept = default;
    Unique& operator=(Unique&&) noexcept = default;
};
```

«Я не копирую, перемещаю по умолчанию».

20.8.1 Когда компилятор сам генерирует move

По умолчанию (без `= default`) move-конструктор и move-присваивание **генерируются автоматически**, но только если:

- Вы не объявили **никакой** из специальных функций (деструктора, сору-конструктора, сору-присваивания).
- Все поля — движимые.

Если объявили деструктор — move **не** генерируется. Если объявили сору — тоже.

Это «правило zero»: для класса с только стандартными полями (`std::string`, `std::vector`, ...) — ничего не пишете, всё генерируется правильно.

20.9 RVO и NRVO

В коде:

```
Buffer make_buffer() {
    Buffer b(100);
    return b;
}

int main() {
    Buffer x = make_buffer();
}
```

Сколько раз вызывается конструктор?

В C++11 — **один** раз, благодаря RVO/NRVO.

RVO (return value optimization): компилятор замечает, что возвращаемый объект — это конкретный возвращаемый объект. Вместо «создать в `make_buffer`, потом скопировать в `main`» — он строит **x прямо в стеке `main`**. Никакого `make_buffer::b` отдельно нет. Всё в одном месте.

NRVO (named RVO) — то же, но для именованных переменных (`Buffer b` в нашем коде). Чуть сложнее реализовать; компилятор делает это эвристически.

С C++17 **RVO стало обязательным** — компилятор гарантированно не делает копию для `rvalue`. Для NRVO — пока на усмотрение компилятора (обычно делает).

Это значит: вы пишете `return some_local;` для большого объекта **без страха**. Компилятор сам оптимизирует.

20.9.1 Когда RVO не работает

NRVO ломается в нескольких случаях:

```
Buffer make_buffer(bool flag) {  
    Buffer a(100);  
    Buffer b(200);  
    return flag ? a : b;    // !! NRVO не работает  
}
```

Компилятор не знает заранее, какой возвращать. Не может построить «прямо в месте назначения». Тут будет `move` (если есть) или `copy`.

Ещё:

```
Buffer make_buffer() {  
    Buffer b(100);  
    return std::move(b);    // !!! НЕ ДЕЛАЙТЕ ТАК  
}
```

Многие новички думают «помогу компилятору с `move`». На самом деле — **испортите**. С `std::move` компилятор не может применить NRVO; вместо нуля копий получится `move`. `Move` дешевле, чем `copy`, но дороже, чем ничего.

Правило: не пишите `std::move` в `return` на локальную переменную. Компилятор сам разберётся.

20.10 Perfect forwarding и std::forward

Мы уже использовали std::forward в шиме make_unique:

```
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Что тут происходит?

Forwarding reference (universal reference) — T&& в шаблоне. Особенный зверь: он принимает и lvalue, и rvalue. Компилятор «свёртывает» ссылки:

- Вызвали make_unique<Weapon>(arg) где arg — lvalue (std::string& name):
 - T = std::string&, T&& = std::string& && = std::string& (свёртывание).
 - Параметр args принимает по lvalue-ссылке.
- Вызвали make_unique<Weapon>(std::move(arg)):
 - T = std::string, T&& = std::string&&.
 - Параметр принимает по rvalue-ссылке.

Внутри функции args — это переменная (всегда lvalue), и нужно **передать дальше с сохранением категории**. Для этого — std::forward<T>(args). Это похоже на std::move, но условный: - Если T — std::string& — вернёт lvalue-ссылку. - Если T — std::string — вернёт rvalue-ссылку.

Это и есть **perfect forwarding** — переслать аргументы дальше «как есть», без потери rvalue-ности или lvalue-ности.

Используется в шаблонах, особенно в фабричных функциях. В обычном коде редко нужен.

20.11 std::move vs std::forward

- **std::move** — безусловное приведение к rvalue. Используйте, когда **знаете**, что переменная — lvalue, и хотите её переместить.
- **std::forward<T>** — условное. Используется в шаблонах с forwarding reference. Сохраняет «категорию» аргумента.

Если в обычном коде хотите переместить — std::move. std::forward оставьте для шаблонов.

20.12 Что после move

Состояние объекта **после** перемещения: - Валидное, но **неопределённое**. Можно вызывать любые методы, которые не требуют конкретного содержимого. - Деструктор сработает корректно. - Присвоить новое значение — можно. - Использовать содержимое — нельзя (точнее, оно есть, но какое — неопределено).

Для стандартной библиотеки конкретно: - `std::string` после `move` — пустая или содержит что попало; вызвать `.size()` можно, но какое значение — неизвестно. - `std::vector` после `move` — обычно пустой. - `std::unique_ptr` после `move` — точно `nullptr`.

Иногда говорят «moved-from object is in valid but unspecified state». Запомните: после `std::move(x)` использовать `x` нельзя, только присвоить новое значение или уничтожить.

```
std::string s = "hello";
std::string t = std::move(s);

// s сейчас в moved-from состоянии
std::cout << s;           // OK, но что внутри — неизвестно
s.size();                 // OK, но размер непредсказуем
s = "new";                // OK, можно присвоить новое
```

20.13 Демонстрация

Соберём пример. Класс `Buffer` владеет массивом, печатает все вызовы:

```
class Buffer {
public:
    explicit Buffer(std::size_t n, const std::string& tag)
        : data_(new int[n]), size_(n), tag_(tag) {
        std::cout << "[" << tag_ << "]" ctor\n";
    }

    ~Buffer() {
        std::cout << "[" << tag_ << "]" dtor\n";
        delete[] data_;
    }

    Buffer(const Buffer& other)
        : data_(new int[other.size_]),
          size_(other.size_),
```

```

        tag_(other.tag_ + "_copy") {
            std::cout << "[" << tag_ << "]" copy ctor\n";
            for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i];
        }

Buffer(Buffer&& other) noexcept
    : data_(other.data_),
      size_(other.size_),
      tag_(std::move(other.tag_) + "_moved") {
    std::cout << "[" << tag_ << "]" move ctor\n";
    other.data_ = nullptr;
    other.size_ = 0;
}

private:
    int* data_;
    std::size_t size_;
    std::string tag_;
};

int main() {
    std::cout << "--- 1. Копирование ---\n";
    Buffer a(5, "a");
    Buffer b = a;    // copy

    std::cout << "\n--- 2. Перемещение ---\n";
    Buffer c = std::move(a);    // move

    std::cout << "\n--- 3. vector с резервом ---\n";
    std::vector<Buffer> v;
    v.reserve(2);
    v.push_back(Buffer(10, "x"));    // временный → move
    v.push_back(Buffer(20, "y"));
}

```

Вывод:

```

--- 1. Копирование ---
[a] ctor (size=5)

```

```
[a_copy] copy ctor from a
```

```
--- 2. Перемещение ---
```

```
[a_moved] move ctor
```

```
--- 3. Push в vector с резервом ---
```

```
[x] ctor (size=10)
```

```
[x_moved] move ctor
```

```
[] dtor
```

```
[y] ctor (size=20)
```

```
[y_moved] move ctor
```

```
[] dtor
```

```
[y_moved] dtor
```

```
[x_moved] dtor
```

```
[a_moved] dtor
```

```
[a_copy] dtor
```

```
[] dtor
```

Видно: - Копирование `Buffer b = a` вызывает `copy ctor` — `[a_copy] copy ctor from a`. - `Buffer c = std::move(a)` вызывает `move ctor` — `[a_moved] move ctor`. - `v.push_back(Buffer(10, "x"))` создаёт временный `x`, потом `vector` использует `move ctor` (потому что он `noexcept!`) — `[x_moved] move ctor`. После — деструктор пустого источника — `[] dtor` (тег пуст, потому что `tag_` был перемещён).

Этот тест входит в `demo-rpg/tests/test_move.cpp` — соберите и убедитесь сами.

20.14 Где move применяется в нашем коде

В `demo-rpg/` мы уже несколько раз использовали `std::move`:

1. В конструкторах для передачи параметров без копирования:

```
Player::Player(std::string name, int max_hp)
    : name_(std::move(name)), // забираем у параметра
    ...
```

`name` — параметр по значению. Внутри функции — `lvalue`. `std::move(name)` превращает в `rvalue`, что позволяет `name_` забрать байты строки.

Альтернатива — принимать параметр по `const &`:

```
Player::Player(const std::string& name, int max_hp)
    : name_(name),    // копия
```

Здесь нет move, потому что параметр — const-ссылка. Какая лучше?

- **std::string name + std::move**: одна копия при передаче в функцию (при Player("Alice", ...) — для строкового литерала; для Player(s, ...) с lvalue — копия s), один move.
- **const std::string& name + сору**: одна копия в любом случае (внутри).

Для коротких строк (SSO) разницы нет. Для длинных — оба варианта примерно одинаковы. Но **std::string name + move** позволяет вызывающему передать **std::move(s)**, и тогда копий не будет вообще. Так чуть гибче.

2. В World::add:

```
void World::add(Location location) {
    std::string id = location.id();
    locations_.emplace(std::move(id), std::move(location));
}
```

location принят по значению — он наш. **std::move(location)** отдаёт его в **emplace** без копии.

3. В Inventory::add:

```
void Inventory::add(std::unique_ptr<Item> item) {
    if (item) {
        items_.push_back(std::move(item));
    }
}
```

unique_ptr **нельзя** копировать. Только перемещать. Без **std::move** была бы ошибка компиляции.

20.15 Главные правила главы

1. **Rule of zero** — лучший выбор. Стандартные поля → ничего не пишете, всё генерируется правильно.
2. **Rule of five** — если управляете ресурсами руками. Реализуйте все пять или явно = delete/= default.
3. **поexcept на move** — обязательно, иначе контейнеры откатятся на сору.
4. **std::move** — это **приведение типа**, не само перемещение. Перемещает move-конструктор.
5. **Не пишите std::move в return** — мешает RVO.
6. **После std::move(x)** объект x в valid-but-unspecified состоянии. Не используйте.

7. `std::forward` — для шаблонов, обычный код обходится `std::move`.
8. Принимать параметр по значению + `std::move` — гибче, чем `const T&`. Особенно для конструкторов.

20.16 Маленькое упражнение

1. Скомпилируйте `test_move.cpp`. Сопоставьте вывод с тем, что я показал.
2. Закомментируйте `noexcept` в `move`-конструкторе `Buffer`. Запустите снова. В `push_back` изменится: теперь `vector` сделает `copy` вместо `move` (потому что не доверяет «бросающему» `move`). Увидите ли вы `copy ctor` в логе?
3. Уберите `reserve(2)` в `vector`. Что произойдёт при втором `push_back`? Пересмотр (`capacity` → 2, опять `move` старых элементов).
4. Замените `Buffer c = std::move(a)` на просто `Buffer c = a`. Будет `copy`. Сравните логи.
5. Добавьте операторы `= default` для `move`-конструктора (`Buffer(Buffer&&) noexcept = default;`). Что сгенерирует компилятор? Будет ли это правильным? (Подсказка: данные-член `data_` — сырой указатель, копирование которого не делает `move`. Дефолтный `move` просто скопирует адрес — но не обнулит источник. Это значит, что и `a`, и `c` имеют один указатель, и оба будут пытаться `delete[]`. Сломается.)
6. (Сложнее) Сделайте функцию `Buffer make_big_buffer(int n)`. Внутри один путь возврата — RVO сработает (вы увидите только `ctor` и `dtor`, без `move/copy`). Добавьте `if (n < 0) return Buffer(0, "small"); else return Buffer(n, "big");` — RVO может не сработать. Проверьте.
7. (Сложнее) Перепишите `Buffer c std::unique_ptr<int[]>` вместо сырого `int*`. Большая часть `rule of five` станет тривиальной — `unique_ptr` сам перемещается. Можно ли использовать `= default` для всех специальных функций?

20.17 Что дальше

Глава 20 — шаблоны (`templates`). Самая мощная и страшная часть C++. Сделаем `EventBus<T>` для шины игровых событий — типобезопасный механизм рассылки сообщений между подсистемами. Заодно узнаем `typename` vs `class`, частичную специализацию, `variadic templates`, и почему ошибки шаблонов так нечитаемы.

Часть II приближается к середине. После шаблонов (20), лямбд (21), исключений (22) дойдём до главы 23 — парсера скриптов локаций. Мир из текстового файла, не зашитый в C++. Это даст наглядное «программа делает то, что хочет пользователь».

Глава 21

Глава 20. Шаблоны — EventBus

В этой главе — **шаблоны** (templates), одна из самых мощных и одновременно самых страшных частей C++. Шаблоны позволяют писать код, работающий с **любым типом**, без потери производительности. STL без них не было бы — `std::vector<int>` и `std::vector<std::string>` это два разных типа, но кода в стандартной библиотеке для них один.

Мы построим `EventBus<T>` — шаблонную шину событий. Игровые подсистемы смогут публиковать события (`ItemPickedEvent`, `PlayerMovedEvent`), а подписчики получать колбэки. Это паттерн **observer**, реализованный обобщённо.

Главное обещание главы: после неё вы будете понимать большую часть кода стандартной библиотеки, плюс уметь писать свои простые шаблоны. Это шаг от «пользователя C++» к «продвинутому пользователю».

21.1 Зачем шаблоны

Вернёмся к функции `max`:

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

Работает для `int`. А для `double`? Нужна другая функция. Для `std::string`? Третья. Для какого-нибудь `Player`-а с оператором `<`? Четвёртая.

В Python с этим проще: `def max(a, b): return a if a > b else b`. Один код для всего.

В C++ можно перегрузкой:


```
int max(int a, int b) { return a > b ? a : b; }
double max(double a, double b) { return a > b ? a : b; }
std::string max(const std::string& a, const std::string& b) { return a > b ? a : b; }
```

Три функции, идентичный код. Если потребуется четвёртый тип — четвёртая функция. Это копи-паст.

Шаблон — это «функция-фабрика». Вы пишете один раз, компилятор сам создаёт версии для нужных типов:

```
template <typename T>
T max(T a, T b) {
    return a > b ? a : b;
}
```

template <typename T> объявляет шаблон. T — параметр-тип. Может быть int, double, чем угодно.

Использование:

```
int a = max(3, 7); // T выводится как int
double b = max(3.14, 2.71); // T выводится как double
std::string c = max(std::string("apple"), std::string("banana")); // T = std::string

// Можно явно указать:
int d = max<int>(3, 7);
```

Компилятор для каждого использования создаёт реальную функцию: max<int>, max<double>, и так далее. Это называется **инстанциация** шаблона.

После компиляции в бинарнике лежат именно сгенерированные конкретные функции. Сам шаблон — это «рецепт», не существующий в финальном коде.

21.2 typename vs class

В template <typename T> слово typename означает «следующий идентификатор — это тип-параметр». Можно также писать class:

```
template <typename T>
T max(T a, T b);

template <class T>
T max(T a, T b);
```

— оба варианта **полностью эквивалентны**. Различий нет.

Исторически: ключевое слово `class` для шаблонов появилось первым, и до C++ не было отдельного слова для «параметра-типа». Потом ввели `typename` как более понятное. Сейчас обычно пишут **`typename`** — оно лучше отражает смысл. Но в чужом коде увидите оба.

В шаблоне `T` может быть не только пользовательским типом, но и встроенным (`int`, `double`), даже указателем (`int*`). Главное — у него должны быть операции, которые шаблон использует. У нас `max` использует `operator>` — значит, у `T` должна быть эта операция. Иначе компилятор выдаст ошибку при инстанцииции.

21.3 Где живут шаблоны: только в заголовках

В главе 7 я говорил: объявления в `.h`, определения в `.cpp`. Для шаблонов это **не работает**.

Шаблоны должны быть **полностью видны** в момент инстанцииции. То есть тело шаблона должно быть в заголовочном файле. Если положить `template T max(T a, T b) { return ... }` в `.cpp` — другой файл, использующий `max<int>`, не увидит реализацию и выдаст ошибку линковки.

Поэтому шаблоны живут **полностью в .h-файле**:

```
// math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

template <typename T>
T max(T a, T b) {
    return a > b ? a : b;
}

#endif
```

Никакого `math_utils.cpp`. Реализация в заголовке.

Это имеет последствия:

- **Раздувание кода.** Каждый `.cpp`, использующий шаблон, перекомпилирует его тело. Это медленно.
- **Утечка реализации.** Все детали шаблона видны пользователю.
- **Реальная инстанцииция в нескольких translation units.** Линкер потом убирает дубликаты (через `inline`, шаблоны автоматически).

Альтернатива — **explicit instantiation**: явно говорим, какие версии шаблона нужны, генерируем их в одном .cpp. Сложновато; в этой книге не пользуемся.

21.4 Шаблоны классов

То же с классами:

```
template <typename T>
class Vec {
public:
    Vec() : data_(nullptr), size_(0), cap_(0) {}

    void push_back(const T& value) {
        if (size_ == cap_) grow();
        data_[size_++] = value;
    }

    T& operator[](std::size_t i) { return data_[i]; }

    std::size_t size() const { return size_; }

private:
    void grow() { /* ... */ }

    T* data_;
    std::size_t size_;
    std::size_t cap_;
};
```

Это сокращённый std::vector. Использование:

```
Vec<int> v;
v.push_back(1);
v.push_back(2);
std::cout << v[0] << "\n"; // 1
```

Vec<int> — это **тип**, сгенерированный из шаблона с T = int. У него есть свои методы, конструкторы, всё как у обычного класса.

Когда вы пишете методы шаблонного класса **вне** класса, синтаксис громоздкий:

```

template <typename T>
class Vec {
public:
    void push_back(const T& value);
};

// Реализация:
template <typename T>
void Vec<T>::push_back(const T& value) {
    // ...
}

```

Каждое определение метода требует префикса `template <typename T>` и `Vec<T>::`. Если параметров шаблона много — становится трудно читать. Многие пишут все методы прямо внутри класса (как у нашего `EventBus`), чтобы не повторять.

21.5 Дедукция типа

Когда вы вызываете `max(3, 7)`, компилятор сам выводит `T = int`. Это **template argument deduction**.

Дедукция работает по аргументам функции. Если параметр объявлен `T`, и вы передаёте `int` — `T = int`. Если параметр `const T&`, передаёте `int` — `T = int` всё равно (`const &` отбрасывается при дедукции).

Иногда дедукция не работает — например, для возвращаемого типа:

```

template <typename T>
T from_string(const std::string& s);

int i = from_string<int>("42");      // явно: T = int
int j = from_string("42");           // ОШИБКА: не из чего вывести T

```

Тогда указываем явно: `from_string<int>("42")`.

Для классов в C++11 дедукция не работала (`std::vector v{1, 2, 3}` — нужен был `std::vector<int> v{1, 2, 3}`). С C++17 добавили **CTAD** (class template argument deduction), но это не наша база.

21.6 Специализация

Иногда хочется для **одного конкретного** типа сделать особую реализацию. Это **специализация шаблона**.

Пример. У нас шаблон `print<T>`:

```
template <typename T>
void print(const T& value) {
    std::cout << value;
}
```

Для `bool` хотим выводить `"true"/"false"`, а не `1/0`:

```
template <> // явная специализация
void print<bool>(const bool& value) {
    std::cout << (value ? "true" : "false");
}
```

`template <>` (пустые скобки) — синтаксис **полной специализации**. Дальше `print<bool>` указывает, какой тип специализируем. Когда компилятор увидит `print(true)` — выберет специализацию, не общий шаблон.

Использование:

```
print(42); // "42"
print(true); // "true"
print("hello"); // "hello"
```

В стандартной библиотеке полно специализаций. Например, `std::vector<bool>` — особая реализация (упаковывает в биты), отличающаяся от `std::vector<T>` для других типов.

21.6.1 Частичная специализация

Только для классов (для функций нет — пишут перегрузки). Допустим, у нас шаблон `Wrapper<T, N>`, и для случая `N == 0` нужна особая реализация:

```
template <typename T, int N>
class Wrapper {
public:
    void show() { std::cout << "general " << N << "\n"; }
};

// Частичная специализация для N = 0
```

```
template <typename T>
class Wrapper<T, 0> {
public:
    void show() { std::cout << "specialized: zero\n"; }
};

Wrapper<int, 5> a; a.show(); // "general 5"
Wrapper<int, 0> b; b.show(); // "specialized: zero"
```

В нашей книге частичная специализация почти не появится — это глубокая тема. Просто знайте, что она есть.

21.7 Variadic templates

Шаблон может принимать **переменное число** параметров:

```
template <typename... Args>
void print_all(Args... args) {
    // ...
}
```

Args... — это «пакет параметров» (parameter pack). Принимает любое количество типов. Можно вызвать:

```
print_all(1, 2.5, "hello", true); // 4 параметра
print_all();                       // 0 параметров
```

Внутри функции пакет нужно «развернуть». Самый частый способ — рекурсия:

```
void print_all() {
    std::cout << "\n";
}

template <typename T, typename... Rest>
void print_all(T first, Rest... rest) {
    std::cout << first << " ";
    print_all(rest...); // рекурсивный вызов
}

int main() {
```

```
print_all(1, 2.5, "hello", true);
}
```

Каждый вызов «отрывает» первый элемент и идёт дальше. В конце — пустая база (terminator).

Вывод: 1 2.5 hello 1 (true печатается как 1).

В C++17 появилось **fold expression** — более короткий синтаксис, но он не наша база.

21.7.1 Variadic в нашем make_unique

Помните rpg::make_unique?

```
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Args... args — пакет аргументов любых типов. std::forward<Args>(args)... — развёртывание пакета. Каждый args пересылается через std::forward, который сохраняет lvalue/rvalue-ность.

Это **perfect forwarding** в действии. Можем создавать любые объекты с любыми параметрами конструктора:

```
auto p1 = make_unique<int>(42);
auto p2 = make_unique<std::string>(10, 'x'); // строка из 10 'x'
auto p3 = make_unique<Player>("Hero", 100);
```

И всё это **без копирования** параметров — они пересылаются как есть.

21.8 SFINAE — кратко

«Substitution Failure Is Not An Error». Один из главных механизмов шаблонной магии.

Идея: если при подстановке типа в шаблон возникает ошибка — компилятор не выдаёт error, а **молча игнорирует** этот шаблон. Это позволяет писать шаблоны, работающие только с определёнными типами.

Пример:

```
template <typename T>
typename std::enable_if<std::is_integral<T>::value, T>::type
square(T x) {
    return x * x;
```

```

}

square(5);           // OK
square(3.14);        // ОШИБКА: square отброшен, потому что double не integral

```

`std::enable_if` — шаблон-машина: если первый параметр истина, есть тип `::type`; иначе ошибка. SFINAE ловит «нет `::type`», молча отбрасывает шаблон.

Синтаксис страшный. В C++20 для этого появились **концепты** (concepts), которые гораздо чище. В C++11 — только SFINAE.

В нашей книге SFINAE не используем. Это упоминание для тех, кто будет читать чужой код.

21.9 Ошибки шаблонов

Если в шаблонном коде ошибка — компилятор выдаёт **очень длинные** сообщения. Иногда десятки строк на одну ошибку.

```

template <typename T>
T sum(const T& a, const T& b) {
    return a + b;
}

struct Foo {};

int main() {
    Foo a, b;
    sum(a, b);
}

```

Компилятор:

```

error: no match for 'operator+' (operand types are 'const Foo' and 'const Foo')
note: candidate: 'T sum(const T&, const T&) [with T = Foo]'
note: in instantiation of function template specialization 'sum<Foo>' requested here
...

```

Видны и шаблон, и место использования. На сложных шаблонах (`std::vector<std::map<int, std::vector<...>>>`) ошибка может занять страницу.

Совет: читайте сообщение **снизу вверх**. Снизу — место ошибки. Над ним — что попытался сделать компилятор. И там, где конкретно «не подошло» — оригинальная причина.

В C++20 концепты делают сообщения чище. Но C++11 — терпите.

21.10 EventBus

Соберём всё в нашем EventBus. Это шаблонный класс, который для каждого типа события держит свой список колбэков:

```
template <typename Event>
class EventBus {
public:
    using Handler = std::function<void(const Event&);>;

    void subscribe(Handler handler) {
        handlers_.push_back(std::move(handler));
    }

    void publish(const Event& event) const {
        for (const auto& h : handlers_) {
            h(event);
        }
    }

    std::size_t subscriber_count() const {
        return handlers_.size();
    }

private:
    std::vector<Handler> handlers_;
};
```

Разберём.

template <typename Event> — шаблон по типу события.

using Handler = std::function<void(const Event&);>; — псевдоним типа. Handler — это «функция, принимающая `const Event&` и не возвращающая ничего». `std::function` — тип-«стиратель» (type-erased wrapper), который умеет хранить любую callable: лямбду, функтор, указатель на функцию. Подробно — в главе 21.

using — современная альтернатива `typedef`:

```
typedef std::function<void(const Event&)> Handler;    // старо
using Handler = std::function<void(const Event&)>;    // ново, читается лучше
```

В шаблонных классах `using` особенно ценен, потому что работает с шаблонными параметрами без проблем.

subscribe добавляет обработчик в вектор. Перемещаем, чтобы избежать копии.

publish проходит по всем и вызывает. Метод `const` — публикация не меняет список подписчиков.

Простой, но работающий код. Около 25 строк.

21.10.1 События

Сами события — просто структуры:

```
// events.h
struct ItemPickedEvent {
    std::string player_name;
    std::string item_name;
};

struct PlayerMovedEvent {
    std::string player_name;
    std::string from_id;
    std::string to_id;
};
```

Никакой иерархии, ничего сложного. Каждый тип события — своя структура. `EventBus` параметризуется конкретным типом, и события только этого типа в неё попадают.

21.10.2 Тест

```
EventBus<ItemPickedEvent> pickup_bus;
EventBus<PlayerMovedEvent> move_bus;

// Подписчик-логгер
pickup_bus.subscribe([](const ItemPickedEvent& e) {
    std::cout << "[log] " << e.player_name << " поднял " << e.item_name << "\n";
});
```

```
// Второй подписчик — ачивка за первый предмет
bool first = true;
pickup_bus.subscribe([&first](const ItemPickedEvent& e) {
    if (first) {
        std::cout << "[ачивка] первый предмет!\n";
        first = false;
    }
});

move_bus.subscribe([](const PlayerMovedEvent& e) {
    std::cout << "[log] " << e.player_name << ": "
               << e.from_id << " -> " << e.to_id << "\n";
});

pickup_bus.publish({"Герой", "ржавый меч"});
pickup_bus.publish({"Герой", "зелье"});
move_bus.publish({"Герой", "glade", "village"});
```

Каждая лямбда — это объект `std::function<void(const Event&)>`. Лямбды и `std::function` — глава 21.

Вывод:

```
[log] Герой поднял ржавый меч
[ачивка] первый предмет!
[log] Герой поднял зелье
[log] Герой: glade -> village
```

Каждое `publish` вызвало всех подписчиков по очереди.

21.11 Концепции и применения

Что нам это даёт?

1. **Развязывание подсистем.** Бой не знает про логирование, но при ударе шлёт `DamageEvent`. Логгер подписан на `DamageEvent` и сам решает, что писать.
2. **Лёгкость расширения.** Новая фича (например, «достижения за 100 убийств») — отдельный подписчик. Старый код не меняется.
3. **Тестирование.** Подписать тест-подписчика — проверить, что событие выстрелило.

Это паттерн **observer** или **pub-sub**. На больших играх — основа архитектуры. В нашей книге используется частично.

21.12 Интеграция в проект (предварительно)

Я не делаю интеграцию EventBus в Game в этой главе — добавим в финальной (25). Но архитектура такая:

```
class Game {
private:
    EventBus<ItemPickedEvent> pickup_bus_;
    EventBus<PlayerMovedEvent> move_bus_;
    // ...

    void cmd_take(const std::string& name) {
        // ... поднимаем предмет ...
        pickup_bus_.publish({player_.name(), item_name});
    }

    void cmd_go(const std::string& dir) {
        // ... меняем локацию ...
        move_bus_.publish({player_.name(), old_id, new_id});
    }
};
```

В начале программы — подписываемся:

```
Game game;
game.subscribe_pickup([](const auto& e) { ... });
```

Удобно? Очень. Но в нашей маленькой игре он не настолько критичен — оставим этот рефакторинг на потом.

21.13 Сторонние шаблонные паттерны

Несколько других мест, где шаблоны блистают.

21.13.1 Тип-безопасное хранение разнотипных данных

```

template <typename T>
class Holder {
public:
    Holder(T value) : value_(std::move(value)) {}
    T& get() { return value_; }
    const T& get() const { return value_; }
private:
    T value_;
};

Holder<int> a(42);
Holder<std::string> b("hi");

a.get() += 1;           // a содержит 43
b.get() += " world";   // b содержит "hi world"

```

21.13.2 Типовые функции

```

template <typename Container, typename Predicate>
auto count_if(const Container& c, Predicate p) -> std::size_t {
    std::size_t n = 0;
    for (const auto& x : c) {
        if (p(x)) ++n;
    }
    return n;
}

std::vector<int> v = {1, 2, 3, 4, 5};
auto evens = count_if(v, [](int x) { return x % 2 == 0; });

```

(Стандарт уже даёт `std::count_if`, но мы могли бы написать свой.)

21.13.3 Generic algorithms

Вся STL — шаблоны. `std::sort`, `std::find`, `std::accumulate` — все принимают любой контейнер и любые типы. Это даёт огромную переиспользуемость.

21.14 Шаблоны и компилятор

Под капотом шаблоны работают так:

1. **На этапе парсинга** компилятор проверяет синтаксис шаблона, но **не компилирует его в код**. Просто записывает «вот рецепт».
2. **При использовании** (`max<int>(3, 7)`) компилятор делает **инстанциацию** — берёт рецепт, подставляет `T = int`, и компилирует получившуюся конкретную функцию.
3. **Каждая инстанциация — новая функция**. `max<int>` и `max<double>` — два разных символа в объектном файле.
4. **Линкер убирает дубликаты**. Если два `.cpp`-файла оба используют `max<int>` — оба генерируют его, но линкер оставляет одну копию.

Поэтому в больших проектах с шаблонами: - **Долгая компиляция** — каждый файл компилирует все используемые шаблоны. - **Большой код** — много дубликатов до линковки. - **Сложные сообщения об ошибках** — раскрыт каждый уровень шаблона.

Современные компиляторы это знают и оптимизируют. Но факт остаётся: шаблоны не бесплатны на `compile-time`. На `run-time` же они часто **быстрее** обычных функций — компилятор знает все типы и может агрессивно инлайнить.

21.15 Главные правила главы

1. **Шаблоны для обобщённого кода**. Если есть `N` копий функции, отличающейся только типом — шаблон.
2. **Тело шаблона в .h-файле**. Реализации в `.cpp` не работают для шаблонов.
3. **typename или class** — выбирайте `typename`, оно лучше отражает смысл.
4. **Дедукция типа** работает для функций. Можно явно: `func<int>(arg)`.
5. **using лучше typedef** для псевдонимов типов.
6. **... для variadic** — переменное число параметров. Используется через рекурсию.
7. **Ошибки шаблонов** читайте снизу вверх.
8. **Шаблоны не бесплатны на compile-time** — длительная сборка на больших проектах.
9. **std::function для type-erased callable** — подробнее в следующей главе.

21.16 Маленькое упражнение

1. Напишите шаблон `min<T>(T a, T b)` по аналогии с `max`. Используйте для `int`, `double`, `std::string`.

2. Напишите шаблон `swap<T>(T& a, T& b)`. Сравните со `std::swap`.
3. Напишите шаблон `print_pair<A, B>(const std::pair<A, B>& p)`, выводящий `"(first, second)"`.
4. Сделайте EventBus методом `clear()` — убрать всех подписчиков.
5. Добавьте к EventBus метод `subscribe_once(handler)` — обработчик удалится после первого вызова. Подсказка: можно через флаг внутри лямбды, или специальной обёртке.
6. (Сложнее) Сделайте шаблон `Stack<T>` — стек на массиве с операциями `push`, `pop`, `top`, `empty`, `size`. Используйте `std::vector<T>` внутри.
7. (Сложнее) Создайте шаблон `Function<R(Args...)>` — упрощённый аналог `std::function`. Поддерживает только хранение лямбд без захвата. Это сложно — подсказка: внутри используйте `R (*func_)(Args...)` (указатель на функцию).

21.17 Что дальше

Глава 21 — **лямбды и `std::function`**. Мы уже много раз использовали лямбды, теперь разберём детально: захваты (`[]`, `[=]`, `[&]`, `[this]`, `[var]`, `[&var]`), `mutable`, тип лямбды, `std::function` как тип-стиратель, performance trade-offs. После — исключения (22), парсер мира (23), сохранения (24), финал RPG (25).

Глава 22

Глава 21. Лямбды и `std::function`

В предыдущих главах мы уже много раз использовали лямбды, особо не вглядываясь. В `std::sort` с компаратором, в `std::count_if` с предикатом, в `Inventory::take` с предикатом поиска, в `EventBus<T>::subscribe` для обработчиков. Эта глава — детальный разбор: что такое лямбда под капотом, что значит каждый вид захвата, зачем `mutable`, что такое `std::function`, и где между ними разница.

Дополнительно построим `CommandRegistry` — реестр игровых команд, в котором имя команды отображается в обработчик-лямбду. Заменяем длинную цепочку `if/else if` в `Game::execute` на короткий `commands_.dispatch(verb, rest)`.

22.1 Лямбда — что это

Лямбда в C++ — это **анонимная функция**, объявленная прямо в коде, где она нужна.

```
auto add = [](int a, int b) { return a + b; };
std::cout << add(3, 4);    // 7
```

`[](int a, int b) { return a + b; }` — это и есть лямбда. Расшифровка по частям:

- `[]` — **захват** (capture). Какие переменные снаружи лямбда «видит».
- `(int a, int b)` — параметры, как у обычной функции.
- `{ return a + b; }` — тело.
- (опционально) Между `()` и `{}` можно указать `-> int` — возвращаемый тип. Если опущен — компилятор выводит сам.

Тип лямбды компилятор генерирует автоматически. Программисту имя этого типа неизвестно. Поэтому используем `auto` или `std::function` для хранения.


```
auto plus = [](int a, int b) { return a + b; };           // auto – самое короткое
std::function<int(int, int)> plus2 = ...;                 // через std::function
```

22.2 Анатомия захвата

Самая сложная часть. Захват — что лямбда «запоминает» из окружающего кода.

```
int x = 10;
auto f = [x]() { return x + 1; };    // x захвачен по значению
std::cout << f();    // 11

x = 100;
std::cout << f();    // всё ещё 11! Лямбда хранит свою копию x
```

Возможные варианты:

22.2.1 [] — без захвата

Лямбда не видит ничего из окружения. Только параметры и глобалы.

```
auto inc = [](int x) { return x + 1; };
```

Самые «свободные» лямбды — могут конвертироваться в указатель на функцию, удобны как параметры в C-API.

22.2.2 [x] — захват по значению

Делает **копию** переменной x в момент создания лямбды. Изменения x снаружи не влияют.

```
int counter = 0;
auto f = [counter]() { return counter; };
counter = 100;
std::cout << f();    // 0 – захватили старое значение
```

Внутри лямбды захваченные по значению переменные **const** по умолчанию — изменять нельзя:

```
int n = 5;
auto f = [n]() { n = 10; };    // ОШИБКА
```

Чтобы разрешить менять — mutable:

```
auto f = [n]() mutable { n = 10; std::cout << n; };  
f();
```

Но изменение влияет только на копию внутри лямбды. Снаружи `n` не меняется.

22.2.3 `[&x]` — захват по ссылке

Лямбда «помнит» **ссылку** на `x`. Все изменения снаружи видны внутри, и наоборот.

```
int counter = 0;  
auto f = [&counter]() { counter++; };  
f();  
f();  
f();  
std::cout << counter;    // 3
```

Это часто используется для накопления состояния.

Опасность: ссылка живёт, пока лямбда живёт. Если лямбда переживёт `counter` — будет висячая ссылка → UB.

```
std::function<void()> bad() {  
    int x = 5;  
    return [&x]() { std::cout << x; };    // !!! x уничтожится, ссылка висячая  
}
```

Запоминайте: **захват по ссылке только когда лямбда живёт меньше, чем источник**. Часто это локальные функции и алгоритмы STL — там безопасно.

22.2.4 `[=]` — захват всего по значению

Все переменные, которые лямбда использует, копируются.

```
int x = 10, y = 20;  
auto f = [=]() { return x + y; };    // x и y скопированы
```

Удобно, но опасно — можно случайно «прихватить» что-то лишнее. В современных стилях `[=]` часто не рекомендуют. Лучше явно указать, что захватываете.

22.2.5 `[&]` — захват всего по ссылке

Все используемые переменные — по ссылке.

```
int x, y, z;  
auto f = [&]() { x = 1; y = 2; z = 3; };
```

Удобно для писателей в окружение. Та же опасность с висячими ссылками.

22.2.6 [this] — захват указателя на текущий объект

Внутри методов класса можно захватить this:

```
class Game {  
public:  
    void register_commands() {  
        commands_.add("look", [this](const std::string&) {  
            cmd_look();    // вызов метода этого Game  
        });  
    }  
};
```

[this] копирует **указатель** this. Через него лямбда может вызывать методы и обращаться к полям.

Опасность: если Game уничтожится, а лямбда останется — this будет висячим. У нас Game живёт всё время программы, проблем нет.

В C++17 появился [*this] — копирует **сам объект**. В C++11 нет.

22.2.7 [x, &y] — комбинированный

Можно смешивать:

```
int x, y;  
auto f = [x, &y]() { ... };    // x по значению, y по ссылке
```

22.2.8 [x = ...] — захват с инициализатором (C++14)

```
auto f = [n = 10]() { return n; };
```

В C++11 нет. У нас не появится.

22.3 Что лямбда на самом деле

Лямбда — это **синтаксический сахар** для создания **функтора** (объекта-функции). Компилятор за вас генерирует класс примерно такого вида:

```
int x = 10;
auto f = [x](int y) { return x + y; };

// эквивалент:
class __lambda_42 {
    int x;
public:
    __lambda_42(int x_) : x(x_) {}
    int operator()(int y) const { return x + y; }
};

auto f = __lambda_42(x);
```

То есть лямбда — это объект некоторого анонимного класса с `operator()`. Захват — это поля класса. Параметры — параметры `operator()`. Тело — тело `operator()`.

mutable лямбда — это объект без `const` на `operator()`:

```
auto f = [n=10]() mutable { ++n; return n; };

// эквивалент:
class __lambda_42 {
    int n = 10;
public:
    int operator()() { return ++n; } // нет const
};
```

Это помогает понять любую лямбду: «это безымянный объект, вызываемый через `()`».

22.4 Производительность

Лямбды **дешёвые**. Они: - Не выделяют память на куче (объект на стеке). - Не используют виртуальные вызовы (вызов `operator()` инлайнится компилятором). - Без захватов превращаются в указатель на функцию.

В STL-алгоритмах (`std::sort`, `std::find_if`) использование лямбды vs обычной функции в скорости **не отличается**. Иногда даже быстрее — потому что компилятор видит код лямбды на месте и инлайнит.

22.5 std::function — type erasure

Лямбды все разного типа. Даже две одинаково выглядящие лямбды — два разных типа. Это создаёт проблему: как **хранить** лямбду в поле класса, если её тип не назвать?

auto работает только для **локальных** переменных. В классе как поле — нельзя.

Решение — **std::function<Signature>**. Это **type-erased** контейнер для любой callable.

```
#include <functional>

std::function<int(int, int)> op = [](int a, int b) { return a + b; };
std::cout << op(3, 4);    // 7

op = [](int a, int b) { return a * b; };
std::cout << op(3, 4);    // 12

op = std::plus<int>();    // функтор тоже ОК
std::cout << op(3, 4);    // 7

int my_func(int a, int b) { return a - b; }
op = my_func;            // указатель на функцию тоже ОК
```

std::function<int(int, int)> принимает **любой** callable с сигнатурой «принимает (int, int), возвращает int». Лямбды, функторы, указатели на функции — всё подходит.

22.5.1 Цена type erasure

std::function не бесплатный:

1. **Размер:** обычно 32-64 байта (внутри SBO — small buffer optimization, как у std::string).
2. **Аллокация для больших захватов:** если лямбда с захватом большего чем SBO размера, std::function выделит память на куче.
3. **Косвенный вызов:** внутри std::function хранится указатель на «виртуальную таблицу». Каждый вызов — это разыменование + вызов через указатель.

В критичных к производительности местах (горячий цикл, миллион вызовов) std::function чувствуется. В обычном коде (как наш EventBus или CommandRegistry) — не проблема.

22.5.2 Когда использовать что

- **Шаблонный параметр** — самый дешёвый, без type erasure. Подходит, когда тип лямбды известен на компиляции:

```
template <typename F>
void run(F f) { f(); }
```

- **std::function** — когда нужно **хранить** callable без знания его типа: поле класса, элемент в vector, возврат из функции с разными возможными лямбдами.
- **Указатель на функцию** — для C-API. Работает только без захватов.

В нашем EventBus<T> мы использовали std::function<void(const Event&)> — потому что нужно хранить разные обработчики в vector. Это типичный случай.

22.6 Лямбды как параметры функций

Два способа принять лямбду:

```
// 1. Шаблонный параметр — компилятор инлайнит, быстро
template <typename F>
void apply(F f) { f(); }

// 2. std::function — type-erased, проще читается
void apply(std::function<void()> f) { f(); }
```

STL использует первый подход — алгоритмы шаблонные.

Если функция большая и часто использует callback — std::function упрощает интерфейс. Если функция маленькая и часто вызывается — шаблон даёт лучшую производительность.

22.7 std::bind — старый стиль (для контекста)

До лямбд для «частичного применения» функции использовали std::bind:

```
#include <functional>

int sub(int a, int b) { return a - b; }

auto from_10 = std::bind(sub, 10, std::placeholders::_1);
std::cout << from_10(3); // 10 - 3 = 7
```

Сложный синтаксис, ограничения, странные placeholders. В современном C++ это **полностью** заменяется лямбдами:

```
auto from_10 = [](int x) { return sub(10, x); };
```

Лямбды короче, читаемее, эффективнее. std::bind в новом коде не пишите — но в старом увидите.

22.8 NPC и поведения — намёк на будущее

Главу можно назвать «поведения NPC через лямбды», и вот пример, к чему мы потом придём:

```
class NPC {
public:
    using Tick = std::function<void(NPC&, Game&)>;

    void on_tick(Tick t) { tick_ = std::move(t); }
    void update(Game& g) { if (tick_) tick_(*this, g); }

private:
    Tick tick_;
};

// Использование: NPC, который патрулирует туда-сюда
auto patroller = std::make_shared<NPC>();
patroller->on_tick([direction = 1](NPC& self, Game& g) mutable {
    // ходим вперёд-назад, переключая direction
    // ...
});
```

Каждый NPC имеет свой tick-обработчик. Один — патрулирует, другой — отвечает на приветствие, третий — продаёт товары. Все через лямбды с захватом своего состояния.

В нашей текущей RPG NPC ещё нет (добавятся за пределами книги, как упражнение). Но шаблон поведения вы уже понимаете.

22.9 CommandRegistry — лямбды на работе

Главная интеграция этой главы. У нас в Game::execute была длинная цепочка:

```
if (verb == "help") {
    cmd_help();
} else if (verb == "look" || verb == "l") {
```

```

    cmd_look();
} else if (verb == "go") {
    std::string dir; ss >> dir;
    if (dir.empty()) std::cout << "Куда идти?\n";
    else cmd_go(dir);
}
// ...ещё восемь веток...
else {
    std::cout << "Неизвестная команда\n";
}

```

Уродливо, длинно, и для каждой новой команды нужно добавлять `else if`. Перепишем через **словарь имя → лямбда**.

include/commands.h:

```

class CommandRegistry {
public:
    using Handler = std::function<void(const std::string&)>;

    void add(const std::string& name, Handler handler);
    void add_alias(const std::string& alias, const std::string& target);
    bool dispatch(const std::string& verb, const std::string& rest) const;
    std::vector<std::string> names() const;

private:
    std::unordered_map<std::string, Handler> handlers_;
    std::unordered_map<std::string, std::string> aliases_;
};

```

Реализация — простая:

```

bool CommandRegistry::dispatch(const std::string& verb, const std::string& rest) const {
    std::string resolved = verb;
    auto alias_it = aliases_.find(verb);
    if (alias_it != aliases_.end()) {
        resolved = alias_it->second;
    }

    auto it = handlers_.find(resolved);
    if (it == handlers_.end()) return false;
}

```



```

        it->second(rest);
        return true;
    }

```

Сначала разрешаем алиас (l → look), потом ищем обработчик, потом вызываем с аргументом.

22.9.1 Регистрация команд

В Game::register_commands():

```

void Game::register_commands() {
    commands_.add("help",      [this](const std::string&) { cmd_help(); });
    commands_.add("look",      [this](const std::string&) { cmd_look(); });
    commands_.add("status",    [this](const std::string&) { cmd_status(); });
    commands_.add("inventory", [this](const std::string&) { cmd_inventory(); });
    commands_.add("go",        [this](const std::string& rest) {
        if (rest.empty()) std::cout << "Куда идти?\n";
        else cmd_go(rest);
    });
    commands_.add("take",      [this](const std::string& rest) {
        if (rest.empty()) std::cout << "Что взять?\n";
        else cmd_take(rest);
    });
    commands_.add("drop",      [this](const std::string& rest) { /* ... */ });
    commands_.add("use",       [this](const std::string& rest) { /* ... */ });
    commands_.add("quit",      [this](const std::string&) { running_ = false; });

    commands_.add_alias("l",   "look");
    commands_.add_alias("i",   "inventory");
    commands_.add_alias("inv", "inventory");
    commands_.add_alias("stat","status");
    commands_.add_alias("get", "take");
    commands_.add_alias("q",   "quit");
    commands_.add_alias("exit","quit");
}

```

Каждая команда — лямбда с захватом this. [this] копирует указатель, через него лямбда вызывает метод cmd_xxx. CommandRegistry ничего о Game не знает — он просто хранит обработчики.

В сигнатуре `Handler = std::function<void(const std::string&)>` — обработчик принимает «остаток строки» (всё после `verb`). Это позволяет вытащить аргумент:

```
// Ввод: "go east"
// verb = "go", rest = "east"
```

22.9.2 Парсинг

В `Game::step()`:

```
std::istringstream ss(line);
std::string verb;
ss >> verb;

std::string rest;
std::getline(ss, rest);
rest = trim_left(rest);

if (!commands_.dispatch(verb, rest)) {
    std::cout << "Неизвестная команда: '" << verb << "'.\n";
}
```

Извлекаем первое слово как `verb`, остальное как `rest` (через `getline` после `>>` забирает остаток строки), убираем ведущие пробелы. Вся логика — в `CommandRegistry`.

22.9.3 help автоматический

```
void Game::cmd_help() const {
    std::cout << "Команды:\n";
    for (const auto& name : commands_.names()) {
        std::cout << "  " << name << "\n";
    }
}
```

`commands_.names()` возвращает список всех зарегистрированных. `Help` автоматически показывает что есть. Добавьте новую команду — она появится в `help` без правок.

22.9.4 Что выиграли

- Цепочка `if/else` исчезла.
- Новая команда — одна строка `commands_.add(...)`.

- Алиасы централизованы.
- Help автогенерируется.
- Парсер изолирован от логики команд.

Это **стандартный паттерн** организации команд в REPL/CLI программах: словарь обработчиков плюс единая точка диспетчеризации. В нашей mini-shell (часть III) сделаем то же самое.

22.10 Запуск

```
$ make && echo -e "help\nl\ninv\nq" | ./build/rpg
```

Команды:

```
drop
go
help
inventory
look
quit
status
take
use
```

== Лесная поляна ==

...

Инвентарь пуст.

До свидания.

Работает: алиасы `l`, `inv`, `q` разрешаются в `look`, `inventory`, `quit`.

22.11 Главные правила главы

1. **Лямбда** — это объект анонимного класса с `operator()`. Не магия.
2. **Захват `[var]` копирует, `[&var]` ссылка.** Запоминайте опасность висячих ссылок.
3. **`mutable`** для изменения захваченных по значению.
4. **`[this]`** для лямбд в методах класса.
5. **`auto` для хранения лямбды**, `std::function<Signature>` для типостёртого хранения в полях/контейнерах.
6. **`std::function` не бесплатен** — стоимость аллокации для больших захватов и косвенного вызова.
7. **Шаблонный параметр** для callback в шаблонных функциях — быстрее, чем `std::function`.
8. **`std::bind` устарел** — заменяйте лямбдами.

22.12 Маленькое упражнение

1. Напишите лямбду `is_even`, принимающую `int` и возвращающую `bool`. Используйте в `std::count_if` на векторе.
2. Захватите счётчик по ссылке и сделайте лямбду, увеличивающую его на каждый вызов. Вызовите 5 раз. Проверьте, что счётчик стал 5.
3. Лямбда без захватов конвертируется в указатель на функцию:

```
void (*p)(int) = [](int x) { std::cout << x; };  
p(42);
```

Проверьте. С захватом не сработает — попробуйте.

4. Добавьте команду `where` в `CommandRegistry`, выводящую текущий ID локации.
5. Добавьте алиас `whereami` → `where`.
6. (Сложнее) Реализуйте функцию `times(int n, F func)`, вызывающую `func()` ровно `n` раз. Шаблонный параметр `F`.
7. (Сложнее) Сделайте простой `Function<int(int)>` — упрощённая копия `std::function` без аллокации (только для маленьких захватов). Подсказка: внутри буфер `char[32]` и указатель на «vtable» (`void (*invoke)(void*, int)`, и так далее).

22.13 Что дальше

Глава 22 — **исключения и exception safety**. Узнаем, как `try/catch/throw` работает на самом деле, что такое `stack unwinding` (мельком в главе 11), что такое слабая/сильная/`no-throw` гарантия безопасности при исключениях, идиома `copy-and-swap`, почему многие C++-проекты исключения отключают.

Затем — главы 23 (парсер мира из текстового файла), 24 (сохранения), 25 (финал RPG). После этого Часть II закрыта.

Глава 23

Глава 22. Исключения и exception safety

Исключения — механизм сигнализации об ошибках в C++. Они мощные и удобные, но имеют свою цену и кучу подводных камней. Эта глава — детальный разбор: как работают throw/try/catch, что такое stack unwinding, какие бывают гарантии безопасности при исключениях, и почему некоторые большие проекты (Google, ядро Chrome, LLVM) исключения **отключают**.

В коде добавим иерархию ошибок (GameError → ValidationError, WorldError), научимся ловить исключения в main, и проверим валидацию через них.

23.1 Зачем исключения

В функции что-то пошло не так. Файл не открылся. Память не выделилась. Параметр невалидный. Как сообщить об ошибке вызывающему?

Подход 1: возвращаемое значение.

```
int divide(int a, int b, int& result) {
    if (b == 0) return -1;    // код ошибки
    result = a / b;
    return 0;
}

int r;
if (divide(10, 0, r) != 0) {
```

```
std::cerr << "ошибка\n";  
}
```

Проблем здесь три. Возвращаемое значение занято под код ошибки, и настоящий результат приходится отдавать через выходной параметр. Каждый вызов надо проверять, а забытая проверка становится ошибкой, которую компилятор не заметит. И, самое неприятное, ошибка из глубины стека вызовов должна всплыть наверх через все промежуточные функции — каждая проверяет, каждая возвращает, и половина кода превращается в передачу кодов возврата.

Подход 2: глобальная переменная `errno` (C-стиль).

```
double d = std::strtod("not a number", nullptr);  
if (errno == ERANGE) { ... }
```

Те же проблемы плюс **глобальное состояние**, плохо с потоками.

Подход 3: исключения.

```
int divide(int a, int b) {  
    if (b == 0) throw std::invalid_argument("division by zero");  
    return a / b;  
}  
  
try {  
    int r = divide(10, 0);  
} catch (const std::exception& e) {  
    std::cerr << "ошибка: " << e.what() << "\n";  
}
```

Все три проблемы исчезают. Возвращаемое значение свободно для результата. Проигнорировать ошибку случайно невозможно: необработанное исключение завершает программу, а не проходит незамеченным. И распространяется оно само, пробивая все уровни стека, — промежуточным функциям об этом знать не нужно.

Недостатки тоже есть, и к концу главы мы разберём их все — включая причины, по которым крупные проекты исключения отключают.

23.2 throw, try, catch

Базовый синтаксис:

```
try {
    // код, который может бросить
    do_something();
} catch (const SpecificException& e) {
    // ловим конкретный тип
} catch (const std::exception& e) {
    // ловим базу всех стандартных
} catch (...) {
    // ловим всё, что осталось (плохой стиль, но иногда нужно)
}
```

Бросить исключение — throw expression:

```
throw std::runtime_error("что-то пошло не так");
throw std::invalid_argument("плохой параметр");
throw 42;    // да, можно бросить int — но не надо
```

throw ищет ближайший try в стеке вызовов, у которого есть подходящий catch. Если найден — управление прыгает туда, и **все локальные переменные** между throw и catch уничтожаются по обычным правилам (вызываются деструкторы). Это и есть **stack unwinding**.

Если ни одного подходящего catch нет — программа вызывает std::terminate, что обычно приводит к падению с сообщением вроде terminate called after throwing an instance of 'std::runtime_error'.

23.2.1 Ловите по ссылке, не по значению

```
catch (std::exception e) { ... }    // ПЛОХО: копия
catch (const std::exception& e) { ... } // ХОРОШО: ссылка
```

Если ловим по значению — исключение копируется. Хуже того: если оно полиморфное (бросили std::runtime_error, ловите как std::exception), при копировании происходит **slicing** (помните из главы 17). Теряются дополнительные данные.

Ссылка избегает копии и slicing. **Всегда ловите по const T&.**

23.2.2 Несколько catch

Можно ловить разные типы:

```
try {
    risky();
```

```

} catch (const std::invalid_argument& e) {
    // плохие параметры
} catch (const std::runtime_error& e) {
    // что-то сломалось в рантайме
} catch (const std::exception& e) {
    // все остальные стандартные
} catch (...) {
    // вообще всё (последняя линия защиты)
}

```

Проверяются **сверху вниз**, срабатывает первый подходящий. Полиморфизм: `const std::exception&` поймает любой потомок, поэтому ставьте конкретные типы **выше** общих, иначе общий перехватит всё.

`catch (...)` — без типа — ловит **любое** исключение. Удобно для логирования и продолжения работы, но информации никакой (даже `e.what()` нельзя). Использовать редко.

23.2.3 Rethrow

Внутри `catch` можно «передать дальше»:

```

try {
    risky();
} catch (const std::exception& e) {
    log("ошибка: ", e.what());
    throw;    // та же ошибка летит дальше
}

```

`throw;` без аргументов — повторно бросает текущее исключение. Можно и новое:

```

catch (const std::runtime_error& e) {
    throw std::runtime_error("обёрнуто: " + std::string(e.what()));
}

```

Это иногда нужно для добавления контекста.

23.3 std::exception и иерархия

В стандартной библиотеке есть **базовый класс** `std::exception` с методом `what()`:

```

class std::exception {
public:

```



```
virtual const char* what() const noexcept;
virtual ~exception();
};
```

От него наследуются:

Иерархия делится надвое по смыслу ошибки. `std::logic_error` и его потомки — `invalid_argument`, `out_of_range`, `length_error`, `domain_error` — означают ошибку программиста: то, чего можно было избежать, проверив аргументы. `std::runtime_error` и его потомки — `range_error`, `overflow_error`, `underflow_error`, `system_error` — означают внешнюю причину, которую программа предвидеть не могла.

Отдельно стоят `std::bad_alloc`, который бросает `new` при нехватке памяти, и `std::bad_cast` — неудачное приведение `dynamic_cast` к ссылке.

Деление на «ошибку программиста» и «внешнюю ошибку» полезно не само по себе, а тем, что подсказывает реакцию: на первую надо чинить код, на вторую — обрабатывать и продолжать.

Эта иерархия даёт **тип-безопасное** ловление. Если ловите `std::out_of_range` — точно знаете, что это «индекс за границей». Если ловите `std::exception` — поймает что угодно стандартное.

Свои ошибки лучше **наследовать** от подходящих стандартных. Тогда любой код, ловящий `std::exception`, поймает и ваши.

23.4 Свои типы исключений

В нашем проекте сделаем три:

```
// include/errors.h
class GameError : public std::runtime_error {
public:
    explicit GameError(const std::string& msg)
        : std::runtime_error(msg) {}
};

class ValidationError : public GameError {
public:
    explicit ValidationError(const std::string& msg)
        : GameError("Validation error: " + msg) {}
};

class WorldError : public GameError {
```

```
public:
    explicit WorldError(const std::string& msg)
        : GameError("WorldError: " + msg) {}
};
```

Корень иерархии — `GameError`; благодаря ему все ошибки игры ловятся одним `catch (const GameError&)`, если разбираться в подробностях не нужно. Ниже два потомка: `ValidationError` для недопустимых данных (пустое имя, отрицательное здоровье) и `WorldError` для несогласованного состояния мира (локация не найдена, выход ведёт в никуда).

`explicit` у конструктора — запрещает неявную конверсию из `std::string`. Без него можно было бы случайно сделать `throw "some text"` (точнее, не совсем, но всё же).

23.4.1 Базовый тип `runtime_error`

Я наследую от `std::runtime_error`, потому что:

1. Это стандартный класс. Любой код, ловящий `std::exception` или `std::runtime_error`, поймает мою ошибку.
2. У него уже есть конструктор от `std::string` и реализация `what()`. Не надо переписывать.

Альтернатива — наследовать прямо от `std::exception` и самим реализовывать `what()`. Сложнее, без явных плюсов.

23.5 Применяем: валидация в `Player`

В `src/player.cpp`:

```
Player::Player(std::string name, int max_hp)
    : name_(std::move(name)),
      max_hp_(max_hp),
      hp_(max_hp) {
    if (name_.empty()) {
        throw ValidationError("имя игрока не может быть пустым");
    }
    if (max_hp <= 0) {
        throw ValidationError("max_hp должен быть положительным");
    }
}
```

Важно: проверяем после инициализации. К моменту `throw` поля уже инициализированы. Это нормально — при выходе из конструктора через исключение поля корректно уничтожатся (их деструкторы вызовутся).

Что произойдёт, если `Player p("", 100)`:

1. Конструктор вызывается.
2. `name_` инициализируется пустой строкой через `move`.
3. `max_hp_` и `hp_` = 100.
4. Проверка: `name_.empty()` истина.
5. `throw ValidationError(...)` — исключение полетело.
6. Stack unwinding: `name_` уничтожается (вызывается `~std::string()`), `max_hp_/hp_` — тривиально.
7. Объект `p` **никогда не создан**. Его как будто не было.
8. Исключение летит к ближайшему `catch`.

Это **критично важное свойство**: если конструктор бросил, объекта нет. Нельзя получить «полусоздан» объект. RAII работает безопасно.

23.6 Главный try/catch в main

Чтобы не падать с ужасным сообщением `terminate after throwing...`, оборачиваем `main`:

```
int main() {
    try {
        rpg::Game game;
        return game.run();
    } catch (const rpg::GameError& e) {
        std::cerr << "Игра не запустилась: " << e.what() << "\n";
        return 1;
    } catch (const std::exception& e) {
        std::cerr << "Неожиданная ошибка: " << e.what() << "\n";
        return 2;
    } catch (...) {
        std::cerr << "Неизвестная ошибка\n";
        return 3;
    }
}
```

Три уровня:

1. **Наши ошибки** — выводим как «ожидаемые», возвращаем 1.

2. **Стандартные ошибки** — как «неожиданные», возвращаем 2.
3. **Что-то совсем странное** — возвращаем 3.

Это **best practice**: ваш `main` должен быть единой точкой обработки всех исключений. Внутри программы — бросайте свободно; в `main` ловите и форматируйте для пользователя.

23.7 Stack unwinding под капотом

Когда `throw` срабатывает, среда выполнения C++ делает примерно следующее:

1. **Создаёт объект исключения** в специальной памяти (не на стеке вызвавшего, потому что стек будет очищаться).
2. **Идёт вверх по стеку**, проверяя для каждого кадра: есть ли в нём активный `try` с подходящим `catch`?
3. Для каждого пройденного кадра **вызывает деструкторы** локальных переменных. Это и есть `unwinding`.
4. Когда найден подходящий `catch` — управление переходит туда.

Этим механизмом пользуется **RAII** (глава 11). Все RAII-объекты в `stack frame` получают свой деструктор бесплатно, даже когда исключение разрушает выполнение.

```
void foo() {  
    std::ofstream f("output.txt");    // RAII: файл откроется  
    risky_operation();                // бросает исключение  
    f << "это не запишется";          // эта строка не выполнится  
    // ~f вызывается АВТОМАТИЧЕСКИ во время unwinding — файл корректно закроется  
}
```

В C-стиле пришлось бы оборачивать в `try/catch` и закрывать вручную. С C++-RAII — само.

23.8 noexcept — обещание не бросать

Функция, помеченная `noexcept`, обещает не бросать исключений:

```
void safe_thing() noexcept {  
    // не бросает  
}
```

Если **всё-таки бросит** — программа вызывает `std::terminate` (как при необработанном). То есть `noexcept` это **строгое** обещание; нарушение фатально.

Нужно это по трём причинам, и третья — самая практическая.

Первая — документация: читатель кода видит обещание, что функция не бросит, и может на него опираться. Вторая — оптимизация: компилятор вправе не создавать для такой функции таблицы раскрутки стека.

Третья касается контейнеров. `std::vector` при увеличении вместимости перемещает элементы только если их конструктор перемещения помечен `noexcept`; иначе он их копирует — потому что при исключении посреди перемещения вектор остался бы в разрушенном состоянии, а при копировании старые элементы целы. Забыть `noexcept` у конструктора перемещения — значит потерять всю выгоду от перемещения, о которой шла речь в главе 19, и не получить об этом ни одного предупреждения.

Деструкторы в C++11 помечены `noexcept` неявно, и это правильно: бросать из деструктора почти всегда ошибка.

Помечать стоит деструкторы (хотя это и происходит само), конструкторы и операторы перемещения, а также простые методы доступа, которые ничего не выделяют и ничего не вычисляют.

Не ставьте `noexcept` на функции, которые могут позвать аллокацию памяти (`new`, `std::vector::push_back`) — они могут бросить `std::bad_alloc`.

23.9 Exception safety — три уровня гарантий

Когда функция бросает, что с **состоянием программы**? Этот вопрос изучили глубоко, выделили три уровня.

23.9.1 Уровень 1: basic guarantee — слабая

После исключения программа **в валидном состоянии**, нет утечек ресурсов, инвариантов не нарушено. Но конкретное содержимое может быть «частично применённым» — то есть какие-то изменения сделаны, какие-то нет.

Пример. Функция, добавляющая элементы в два контейнера:

```
void add_to_both(std::vector<int>& a, std::vector<int>& b, int v) {  
    a.push_back(v);  
    b.push_back(v);    // если эта строка бросит — a уже изменён, b нет  
}
```

Basic guarantee: оба вектора целы, утечек нет, но программа **не может предположить**, что обе вставки сделаны.

23.9.2 Уровень 2: strong guarantee — сильная

Либо операция полностью успешна, либо состояние **точно как было**. «Транзакционность» — всё или ничего.

```
// Сильная гарантия:
void add_to_both(std::vector<int>& a, std::vector<int>& b, int v) {
    std::vector<int> a_new = a;    // копия
    a_new.push_back(v);

    std::vector<int> b_new = b;
    b_new.push_back(v);

    // Если до сюда дошли — операции прошли. Теперь атомарно меняем.
    a.swap(a_new);    // swap у vector — noexcept
    b.swap(b_new);
}
```

Если любая из вставок бросает — оригиналы *a* и *b* нетронуты. Это потому, что мы делали копию, и только в конце атомарно меняли.

Это **copy-and-swap** идиома — частый паттерн для сильной гарантии.

23.9.3 Уровень 3: nothrow guarantee — без бросков

Функция **гарантированно не бросает**. Например, деструкторы, простые геттеры, перемещения. Идеал.

```
void operator=(const T& other) noexcept { ... }
```

23.9.4 Какая гарантия предпочтительна

Ориентир по уровням такой. Деструкторы обязаны не бросать вовсе: исключение во время раскрутки стека, вызванной другим исключением, немедленно завершает программу. Операции перемещения тоже должны быть без исключений — иначе контейнеры откажутся ими пользоваться. Простые методы доступа обычно не бросают сами собой. А сложные методы должны давать хотя бы базовую гарантию, и лучше строгую — через приём с копией и обменом, о котором ниже.

В нашей RPG большая часть кода имеет *basic guarantee*. Это часто достаточно, особенно с RAII.

23.10 Copy-and-swap идиома детально

Классический пример: оператор присваивания с сильной гарантией.

```
class Buffer {
public:
    Buffer(const Buffer& other);    // copy ctor
    ~Buffer();

    // Простая copy-and-swap
    Buffer& operator=(Buffer other) { // принимаем по значению — компилятор сам копирует
        swap(*this, other);          // меняемся местами с временной
        return *this;
    }

    friend void swap(Buffer& a, Buffer& b) noexcept {
        using std::swap;
        swap(a.data_, b.data_);
        swap(a.size_, b.size_);
    }

private:
    int* data_;
    std::size_t size_;
};
```

Что происходит: 1. Параметр `other` создаётся как **копия** аргумента. Если копирование бросит — `*this` нетронут. 2. `swap` (без `std::`, для нахождения через ADL — argument-dependent lookup) меняет внутренности. Он `noexcept`, не бросает. 3. В конце `other` уничтожается, забирая с собой **бывшие данные** `*this`.

Если бы кому-то пришло на ум присваивать самому себе (`a = a`), сначала бы скопировался `a` в `other`, потом `swap`, потом `other` уничтожился. Всё корректно.

Это **исключительно элегантный** паттерн. Использует RAII (`other` уничтожается сама), move-семантику (передача параметра), и одну операцию `swap` с гарантией `noexcept`. Сильная гарантия за дёшево.

23.11 Цена исключений

«Цена» — спорный вопрос. Зависит от ABI и оптимизаций.

На «happy path» (нет исключений): Пока исключение не брошено, современные соглашения о вызовах (Itanium на Unix, x64 на Windows) не стоят почти ничего: таблицы раскрутки лежат в памяти, но не исполняются. Совсем нулевой цена всё же не бывает — компилятор осторожнее оптимизирует функции, из которых может вылететь исключение, а таблицы занимают место в бинарнике.

Когда исключение полетело: А вот сам бросок дорог: поиск подходящего обработчика, раскрутка стека, вызовы деструкторов — это десятки микросекунд. Отсюда правило: исключения нужны для ошибок, а не для обычного управления ходом программы. Выходить из цикла броском исключения технически можно, но это способ сделать программу и медленной, и непонятной.

В обычном коде overhead исключений не виден. В код горячих циклов и реал-тайм-систем — заметен.

23.12 Почему отключают исключения

Несколько громких проектов отключают `-fno-exceptions`:

Причины у всех разные. В Google исключения запрещены стилем в основном исторически: старый код к ним не готов, а переписывать миллионы строк никто не станет; сюда же добавляются сложности стыковки с кодом на C. Во встраиваемых системах не устраивает расход памяти на таблицы раскрутки, а иногда и сама непредсказуемость времени выполнения. В игровых движках важна стабильность кадровой частоты, и внезапная задержка в десятки микросекунд нежелательна. А в LLVM исключения отключены и заменены собственным типом `llvm::Error`, который заставляет проверять результат.

При `-fno-exceptions`: С этим флагом `throw` превращается в ошибку компиляции или в немедленное завершение, а места стандартной библиотеки, которые могли бы бросить, заменяются на аварийный выход. Вместо исключений в таких проектах пользуются кодами ошибок, собственными типами результата или `std::expected` из C++23, о котором пойдёт речь в главе 51.

В нашей книге исключения используем — это идиоматический C++ для приложений. Но знайте, что в индустрии бывает по-разному.

23.13 Когда что выбирать: исключение vs код ошибки vs assert

Грубая модель:

`assert` — для того, чего в правильной программе случиться не может: деления на ноль там, где ноль исключён проверкой выше, пустого указателя там, где он гарантированно не пуст. Такая проверка ловит ошибку программиста, а в сборке для выпуска отключается вовсе.

Возврат признака — `bool`, `std::optional`, код — для исходов, которые неудачей не являются. «Не нашли элемент» — обычный ответ, а не ошибка, и исключение здесь избыточно.

Исключения — для настоящих ошибок, которые должен обработать кто-то выше по стеку: файл не открылся, память не выделилась, данные не проходят проверку.

В нашей RPG: В нашем коде это выглядит так. `Inventory::take` возвращает пустой указатель, если предмета нет, — обычная ситуация, игрок просто ошибся с названием. `World::find` тоже возвращает пустой указатель: локации с таким именем нет, и это ответ, а не сбой. А вот конструктор `Player` с пустым именем бросает — потому что пустое имя означает ошибку в коде, который его создал.

23.14 Главные правила главы

1. **Ловите по `const T&`** — не по значению, не по указателю.
2. **Бросайте конкретные типы**, наследованные от `std::runtime_error` или `std::logic_error`.
3. **Главный `try/catch` в `main`** — единая точка форматирования ошибок.
4. **Деструкторы и `move`** — по `except`. Без исключений.
5. **`assert` для невозможных ситуаций**, исключение для реальных ошибок.
6. **`copy-and-swap`** для сильной гарантии оператора присваивания.
7. **Не используйте исключения для управления потоком** — они дорогие при бросании.
8. **`-fno-exceptions`** — реальная опция в некоторых проектах. Подумайте, как ваш код переживёт это.

23.15 Маленькое упражнение

1. Запустите `test_exceptions.cpp` (`make tests && ./build/tests/test_exceptions`). Изучите вывод.
2. Добавьте в `Inventory::take` бросок `WorldError`, если предмет не найден (вместо возврата `nullptr`). Что произойдёт с `cmd_take` в игре? Какая логика проще: возврат `nullptr` или `try/catch`?
3. Перепишите `Location::exit` так, чтобы возвращал `std::string` (сейчас и так), но при отсутствии направления **бросал** `WorldError`. Команда `cmd_go` должна это ловить и говорить «не пройти». Сравните с текущим кодом.
4. Сделайте функцию, которая нарочно бросает в деструкторе. Запустите. Что произойдёт?

5. Сделайте функцию `divide` с двумя `int`-параметрами. Бросайте `std::invalid_argument` при делении на 0. Поймайте в `main`. Выведите `e.what()`.
6. Прочитайте `errors.h`. Объясните, почему конструкторы `ValidationError/WorldError` принимают `const std::string&`, а не `std::string` по значению.
7. (Сложнее) Сделайте функцию, бросающую `std::runtime_error`. Поймайте, **залогируйте**, и **перебросьте** (`throw;`). Поймайте на следующем уровне. Это типичный паттерн добавления контекста к ошибке.

23.16 Что дальше

Глава 23 — **парсер скриптов локаций**. Мир игры будет загружаться из текстового файла (`data/world.txt`), а не зашиваться в C++. Это огромный шаг: можно править контент без перекомпиляции, добавлять локации руками, делиться мирами. Используем `std::ifstream` (глава 12), `std::stringstream` (12), `std::unordered_map` (13), и наши `Item`-классы (17, 18). Это будет реальная мини-задача парсинга — пример того, что вы сможете применять везде.

Затем — глава 24 (сохранения), глава 25 (финал RPG), и Часть II закрыта.

Глава 24

Глава 23. Парсер скриптов локаций

До сих пор наш мир из трёх локаций был **зашит в C++**: функция `make_demo_world()` явно создавала их вручную. Чтобы добавить пещеру, нужно было править `world.cpp`, пересобирать программу, запускать. Это работает для трёх локаций, но не масштабируется.

В этой главе мы построим **парсер**: мир будет описываться в текстовом файле `data/world.txt`, и программа будет его читать. Добавил локацию — сохранил файл — запустил, никаких пересборок. Это огромный шаг: контент отделён от кода.

Заодно — это первая в книге задача парсинга. Очень полезный навык. Любая программа, читающая конфиги, JSON, XML, CSV, файлы данных — это тот же парсинг. Освойте принцип на простом примере — лучше поймёте сложные форматы.

24.1 Зачем парсер

Несколько причин, по которым контент не зашивают в код:

1. **Скорость итерации.** Дизайнер игры (или вы сами) меняет локацию — сохранил файл — посмотрел в игре. Без перекомпиляции.
2. **Расширяемость пользователями.** Кто-то скачивает игру, открывает `world.txt`, добавляет свои локации. Получает свою версию.
3. **Локализация.** Тексты в файлах легко переводить, не лезя в C++-код.
4. **Разделение компетенций.** Программисту не приходится «писать тексты», дизайнеру — лезть в код.
5. **Скрипты vs данные.** Можно держать «движок» в C++, а «контент» в текстовых файлах.

Большие игры (Skyrim, World of Warcraft) хранят сотни тысяч объектов в файлах. Жёсткое разделение «движок против контента» — стандарт индустрии.

24.2 Выбор формата

Какой формат файла? Варианты:

JSON. Стандарт. Хорошо парсится, есть библиотеки. Но многословный и требует кавычек:

```
{
  "id": "glade",
  "name": "Лесная поляна",
  "description": "...",
  "exits": {"west": "village", "east": "forest"},
  "items": [
    {"type": "weapon", "name": "меч", "weight": 5, "damage": 8}
  ]
}
```

YAML. Менее многословно, но синтаксис со значимыми отступами тонкий.

TOML. Хорошо для конфигов, но плохо для иерархии.

Свой формат. Можно сделать максимально простой под свои нужды.

Для учебной цели свой формат лучше всего: меньше зависимостей, понятнее, что и как парсится. На реальном проекте — JSON или TOML через готовую библиотеку.

Наш формат — **линейно-ключевой**:

```
# Комментарий
```

```
location <id>
name <текст до конца строки>
desc <текст до конца строки>
exit <direction> <target_id>
item <type> <name> <weight> <extra>
```

Каждая строка — одна команда. Парсер читает построчно. Локация начинается с `location <id>`, дальше идут её свойства, пока не встретится новая `location` или конец файла.

24.3 Пример мира

data/world.txt:

```
# Описание мира RPG-демо.
# Имена предметов с подчёркиванием — превратится в пробел.
```

```
location glade
name Лесная поляна
desc Залитая солнцем поляна. На западе тропа, на востоке лес.
exit west village
exit east forest
exit south mountain
item weapon ржавый_меч 5 8
```

```
location village
name Деревня
desc Маленькая деревня из десятка домов.
exit east glade
item consumable зелье_лечения 1 15
item consumable хлеб 1 5
```

```
location forest
name Тёмный лес
desc Ветви скрипят над головой.
exit west glade
exit north cave
item armor кожаный_нагрудник 8 4
```

```
location cave
name Пещера
desc Сырая пещера. Капает вода.
exit south forest
item weapon боевой_топор 7 12
item armor стальной_щит 6 6
```

```
location mountain
name Горная тропка
desc Узкая тропка вдоль обрыва.
exit north glade
item consumable вода 1 3
```

Пять локаций, граф связан, есть и оружие, и броня, и расходники. Без C++.

24.4 Архитектура парсера

Парсер состоит из:

1. **Read line** — читаем строку файла.
2. **Trim** — убираем пробелы по краям.
3. **Skip** — пропускаем комментарии и пустые строки.
4. **Tokenize** — извлекаем keyword и аргументы.
5. **Dispatch** — выполняем нужное действие на основе keyword.

Состояние парсера — текущая «черновая» локация. Каждый name, desc, exit, item модифицирует её. Каждая новая location — сохраняет старую и начинает новую.

Это **state machine**: два состояния — «нет активной локации» и «активная локация». Переход — по ключу location.

24.5 Заголовок

include/world_parser.h:

```
#ifndef RPG_WORLD_PARSER_H
#define RPG_WORLD_PARSER_H

#include "world.h"

#include <iosfwd>
#include <string>

namespace rpg {

    // Парсер мира из текстового формата. Бросает WorldError при ошибках.
    World parse_world(std::istream& in);
    World load_world(const std::string& path);

} // namespace rpg

#endif
```

Две функции: - parse_world(istream&) — для тестов и для случаев, когда хотим читать не из файла. - load_world(path) — удобная обёртка, открывает файл и зовёт parse_world.

<iosfwd> — заголовок с **forward declarations** для потоков. Объявляет `std::istream` и подобные, но без полных определений. Достаточно для параметров. Полная <iostream> тяжёлая (десятки тысяч строк через <algorithm> и прочие транзитивные include). <iosfwd> лёгкий.

В .cpp мы подключаем уже полные <fstream> и <sstream> — там нужно работать с потоками реально.

24.6 Реализация: вспомогательные функции

src/world_parser.cpp:

```
namespace { // анонимное пространство — функции видны только в этом файле

std::string trim(const std::string& s) {
    auto b = s.find_first_not_of(" \t\r");
    if (b == std::string::npos) return {};
    auto e = s.find_last_not_of(" \t\r");
    return s.substr(b, e - b + 1);
}

std::string unescape_name(const std::string& s) {
    std::string r = s;
    std::replace(r.begin(), r.end(), '_', ' ');
    return r;
}

} // namespace
```

trim — стандартная штука: убрать пробелы/табуляции/\r (на Windows-файлах в конце строки) с обоих краёв.

unescape_name — наш способ обработки пробелов в именах. В файле пишем ржавый_меч, в игре отображаем ржавый меч.

Анонимное пространство (namespace { ... }) — для функций, видимых только в этом .cpp. Альтернатива static. Я в книге использую namespace { ... } — это идиоматично в современном C++.

24.7 Промежуточное состояние

Один блок в файле описывает локацию **постепенно** — сначала name, потом desc, потом несколько exit и item. Мы не можем создать Location сразу, нужно собирать.

Решение — **промежуточная структура**:

```
struct LocationDraft {
    std::string id;
    std::string name;
    std::string desc;
    std::vector<std::pair<std::string, std::string>> exits;
    std::vector<std::unique_ptr<Item>> items;
};

Location finalize(LocationDraft& draft) {
    Location loc(std::move(draft.id), std::move(draft.name), std::move(draft.desc));
    for (auto& e : draft.exits) {
        loc.add_exit(e.first, e.second);
    }
    for (auto& item : draft.items) {
        loc.items().add(std::move(item));
    }
    return loc;
}
```

LocationDraft — простой POD (plain old data, набор полей). Мы её наполняем по мере чтения, а в конце finalize превращает в настоящий Location. После этого можно создать новый draft для следующей локации.

Это типичный паттерн **builder**: собираем «чертёж», в конце «строим».

24.8 Парсинг строки предмета

```
std::unique_ptr<Item> parse_item_line(std::istream& ss, int line_no) {
    std::string type, name;
    int weight = 0, extra = 0;
    if (!(ss >> type >> name >> weight >> extra)) {
        throw WorldError("строка " + std::to_string(line_no)
            + ": неполная строка item");
    }
}
```



```

}
std::string pretty = unescape_name(name);

if (type == "weapon")    return make_unique<Weapon>(pretty, weight, extra);
if (type == "armor")    return make_unique<Armor>(pretty, weight, extra);
if (type == "consumable") return make_unique<Consumable>(pretty, weight, extra);

throw WorldError("строка " + std::to_string(line_no)
                 + ": неизвестный тип предмета '" + type + "'");
}

```

ss >> type >> name >> weight >> extra — извлекает четыре поля через `std::istream` (мы их подробно рассмотрели в главе 12). Если поток в failure state — что-то не извлеклось, кидаем `WorldError`.

Дальше по типу диспатчируем: создаём `Weapon`, `Armor` или `Consumable`. Если тип неизвестен — ошибка.

Каждое исключение содержит **номер строки** — это критично для диагностики. Без него «строка item плохая» — где её искать?

24.9 Главный цикл парсера

```

World parse_world(std::istream& in) {
    World world;
    LocationDraft draft;
    bool have_draft = false;

    auto commit_draft = [&]() {
        if (have_draft) {
            world.add(finalize(draft));
            have_draft = false;
            draft = LocationDraft{};
        }
    };

    std::string line;
    int line_no = 0;
    while (std::getline(in, line)) {

```

```
++line_no;
line = trim(line);
if (line.empty() || line[0] == '#') continue;

std::istringstream ss(line);
std::string keyword;
ss >> keyword;

if (keyword == "location") {
    commit_draft();
    ss >> draft.id;
    if (draft.id.empty()) {
        throw WorldError("строка " + std::to_string(line_no)
            + ": location без id");
    }
    have_draft = true;
    continue;
}

if (!have_draft) {
    throw WorldError("строка " + std::to_string(line_no)
        + ": '' до 'location'");
}

if (keyword == "name") {
    std::string rest;
    std::getline(ss, rest);
    draft.name = trim(rest);
} else if (keyword == "desc") {
    std::string rest;
    std::getline(ss, rest);
    draft.desc = trim(rest);
} else if (keyword == "exit") {
    std::string dir, target;
    ss >> dir >> target;
    if (dir.empty() || target.empty()) {
        throw WorldError("строка " + std::to_string(line_no)
            + ": exit требует <direction> <target_id>");
    }
}
```

```

    }
    draft.exits.emplace_back(std::move(dir), std::move(target));
} else if (keyword == "item") {
    draft.items.push_back(parse_item_line(ss, line_no));
} else {
    throw WorldError("строка " + std::to_string(line_no)
                    + ": неизвестный ключ '" + keyword + "'");
}
}

commit_draft();
return world;
}

```

Разберём.

Лямбда `commit_draft` — фиксирует «черновую» локацию в мире. Зачем лямбда? Используется дважды: при встрече новой location и в конце файла. Не хочется дублировать код. Захват [&] — нужно менять draft, have_draft, world снаружи.

Цикл `while (std::getline(in, line))` — читаем строки до конца файла. `getline` возвращает поток в булевом контексте: true пока всё ОК, false при EOF/ошибке.

`if (line.empty() || line[0] == '#') continue;` — пропуск пустых строк и комментариев.

`std::istringstream ss(line)` — превращаем строку в поток для дальнейшего парсинга >>. Мы уже видели этот трюк в главе 12 и 13.

Дальше — диспатч по ключевому слову. Каждая ветка ожидает свои аргументы.

24.9.1 name и desc

```

if (keyword == "name") {
    std::string rest;
    std::getline(ss, rest);
    draft.name = trim(rest);
}

```

`getline(ss, rest)` берёт **весь остаток строки**, включая пробелы. Это нужно, потому что name Лесная поляна — поле из двух слов. Если бы мы делали `ss >> ss >> ...`, потеряли бы «поляна».

`trim(rest)` убирает ведущий пробел (тот, что между name и началом значения).

24.9.2 exit

```

} else if (keyword == "exit") {
    std::string dir, target;
    ss >> dir >> target;
    if (dir.empty() || target.empty()) {
        throw WorldError(...);
    }
    draft.exits.emplace_back(std::move(dir), std::move(target));
}

```

Два слова: направление и id цели. `emplace_back` конструирует `std::pair` прямо в векторе — без промежуточных копий (это глава 13).

24.9.3 item

```

} else if (keyword == "item") {
    draft.items.push_back(parse_item_line(ss, line_no));
}

```

Делегируем в `parse_item_line`. Внутри — четыре поля.

24.9.4 Конец файла

```

commit_draft();
return world;

```

Не забыть зафиксировать последний `draft`. До этого `commit_draft` вызывался только при встрече **новой** `location`; последний блок завершается концом файла.

24.10 Подключение к Game

Меняем `Game::Game()`:

```

Game::Game()
: running_(true),
  world_(load_world("data/world.txt")), // <--- было make_demo_world()
  player_("Герой", 20),
  current_location_id_("glade") {
    register_commands();
}

```

Одна строка изменилась. Вся остальная логика игры не знает, откуда мир. RAII работает, как раньше.

Если файл недоступен — `load_world` бросит `WorldError`, и наш `main-catch` покажет «Игра не запустилась: `WorldError`: не открылся файл мира». Графично.

24.11 Проверка

```
$ make && echo -e "look\ngo south\nlook\ngo north\ngo east\ngo north\nlook\nq" | ./build/rpg
== Лесная поляна ==
Залитая солнцем поляна. На западе видна тропа к деревне, на востоке — тёмный лес. На юг ведёт горная тропа.
Выходы: south east west
На земле:
  - ржавый меч (оружие, урон 8, вес 5)
  ...
== Горная тропка ==
Узкая тропка вьётся вдоль обрыва. Виден весь лес.
  ...
== Тёмный лес ==
Ветви скрипят над головой. Здесь сумрачно и тихо.
Выходы: north west
На земле:
  - кожаный нагрудник (броня, защита 4, вес 8)
== Пещера ==
Сырая пещера. Где-то в глубине капает вода.
Выходы: south
На земле:
  - боевой топор (оружие, урон 12, вес 7)
  - стальной щит (броня, защита 6, вес 6)
```

Пять локаций (вместо трёх в `make_demo_world`). Все из текстового файла. Пещера, тропка, лес и предметы появились **без правок C++-кода**.

Откройте `data/world.txt`, добавьте ещё одну локацию — пересоберите без `make clean` (только `Makefile` видит изменения). Если изменили только `world.txt` — даже `make` не нужно! Программа просто прочтёт новый файл при следующем запуске.

24.12 Уроки про парсинг

Несколько наблюдений из этой главы.

24.12.1 Состояния парсера

У нас всего два: «нет активного блока» и «активный блок». Переход — `keyword location`. Это **machine с очень малым количеством состояний**.

Для большего формата (например, вложенные блоки) понадобились бы: - Стек состояний (`current score`). - Или рекурсивный парсинг (каждое состояние — отдельная функция).

JSON-парсер, например, использует рекурсию.

24.12.2 Сообщения об ошибках

Каждая ошибка содержит номер строки. Это критично. Без номера пользователь не найдёт проблему.

Хороший парсер на ошибку: - Указывает, где (файл, строка, иногда колонка). - Говорит, что ожидалось. - Говорит, что получено.

Мы это делаем. Можно ещё добавить контекст (несколько строк вокруг), но для учебного достаточно.

24.12.3 Иммутабельность данных

Заметили: `Location` после конструктора **не меняется**. `name`, `desc`, `exits`, `items` устанавливаются один раз. Это упрощает логику и делает класс безопасным к ошибкам.

Если бы у `Location` были сеттеры (`set_name`, `set_desc`), можно было бы строить её итеративно. Но тогда повсюду риск: «а вдруг кто-то поменяет уже созданную локацию во время игры?». Мы избегаем этого через паттерн **builder** (`LocationDraft` → `finalize`).

Это **полу-immutable** дизайн. У `World` сеттер есть (`add`), но он используется только при загрузке.

24.12.4 Tokenizer vs парсер

В нашем парсере токенизация и парсинг **смешаны**: `std::istream` сама делит строку на слова, и мы прямо в логике диспатчим.

В **больших парсерах** (компилятор C++, JSON-движок) это разделяют: 1. **Lexer/tokenizer** превращает текст в поток токенов (`KEYWORD`, `IDENTIFIER`, `NUMBER`, ...). 2. **Parser** работает с токенами, строит AST.

Для нашей задачи разделение `overkill`. Но знайте, что в больших задачах оно есть.

24.12.5 Robust vs strict парсер

Наш парсер **строгий**: любая ошибка — `throw`. Запустился — мир загружен. Не запустился — игра не работает вообще.

Альтернатива — **прощающий** парсер: пропускать неизвестные ключи, не падать на одной плохой локации. Подходит для пользовательских моддинг-форматов: пользователь добавил кривую локацию — остальное всё равно загрузится.

Что лучше — зависит от задачи. Для разработки лучше строгий (баги видны сразу). Для пользователей — мягкий.

24.13 `std::variant` — упоминание (C++17)

В нашем коде у Item-иерархии **три типа**: `Weapon`/`Armor`/`Consumable`. Мы держим их через `std::unique_ptr<Item>` и проверяем через `dynamic_cast`. Это классический ОО-подход.

В C++17 появился `std::variant<Weapon, Armor, Consumable>` — он хранит **один из** перечисленных типов. Доступ через `std::visit` (компилятор сам выбирает нужный код в зависимости от того, что сейчас в `variant`'е).

Преимущества `std::variant`: - Нет виртуальных функций — быстрее. - Тип фиксирован на компиляции — компилятор может оптимизировать. - Нет наследования — проще структура.

Недостатки: - Закрытое множество типов — не добавить четвёртый тип без переделки. - Сложнее API (visitor pattern).

У нас C++11. `std::variant` нет. Альтернатива — **tagged union**: `struct { Type tag; union { Weapon w; Armor a; Consumable c; }; }`. Старо, многословно, но работает. В нашем коде проще остаться на наследовании.

24.14 Главные правила главы

1. **Контент в файлах, движок в коде.** Разделение.
2. **Builder паттерн** для постепенного сбора объекта.
3. **Каждая ошибка с номером строки.** Без этого диагностика плачет.
4. **Анонимные пространства** для функций видимых только в `.cpp`.
5. `std::istream` для парсинга одной строки.
6. `getline(ss, rest)` для получения остатка строки (включая пробелы).
7. **Builder-объект + finalize** для постепенной инициализации иммутабельных классов.

8. **Большие парсеры разделяют tokenizer и parser**, маленькие — нет.

24.15 Маленькое упражнение

1. Запустите игру (`make && ./build/rpg`). Попробуйте все 5 локаций.
2. Добавьте в `data/world.txt` локацию `tower` — высокая башня. Из деревни выход `north tower`, из башни — `south village`. Положите в неё `weapon` магический_посох 3 20. Запустите. Никаких изменений в C++ не делайте.
3. Намеренно сломайте `data/world.txt`: уберите `name` у одной из локаций. Запустите игру. Что произойдёт? Сообщение об ошибке понятное?
4. Добавьте в формат строку `start <id>` — указание стартовой локации. Сейчас зашито `glade` в `Game::Game()`. Сделайте, чтобы стартовая локация читалась из файла.
5. Добавьте поддержку `# inline` комментариев — комментарий **на конце строки**, например:

```
exit west village # дорога вниз
```

Подсказка: найти `#` в строке и обрезать до него.
6. (Сложнее) Сделайте парсер прощающим: при ошибке вместо `throw` логировать предупреждение и продолжать. Подсказка — заменить `throw` на `std::cerr << ...; continue;` или собирать ошибки в `std::vector<std::string>` для итогового отчёта.
7. (Сложнее) Сделайте формат поддерживающим **NPC** — пока без логики, просто запись имени и тек. Расширьте `Location`, чтобы хранил список NPC-имён.

24.16 Что дальше

Глава 24 — **сохранение и загрузка игры**. Текстовый формат vs бинарный, версии формата, обратная совместимость, контрольная сумма. Похоже на парсер мира, но в обе стороны: можем сохранить и можем загрузить.

Затем — глава 25, **финал RPG**: модульные тесты с `GoogleTest`, сборка под три платформы, итоги, что осталось «как упражнение». На этом Часть II закроется, и мы перейдём к проекту `mini-shell`.

Глава 25

Глава 24. Сохранение и загрузка игры

Текущее состояние игры — это **снимок** мира и игрока в момент времени. Чтобы можно было выключить программу и продолжить позже — нужно этот снимок куда-то сохранить и потом восстановить. В этой главе сделаем систему сохранений: формат файла, версионирование, контрольную сумму. Добавим команды `save <slot>` и `load <slot>`.

Тема очень практическая. Любая нетривиальная программа рано или поздно сталкивается с **сериализацией**: сохранением состояния. Игры, базы данных, мессенджеры, графические редакторы — все хранят что-то на диске.

25.1 Что сохранять

Состояние нашей игры состоит из:

- **Игрок**: имя, текущее `hp`, `max hp`, инвентарь (список предметов).
- **Текущая локация**: `id`.
- **Мир**: остаточные предметы по локациям (часть взяли — этих предметов больше нет).

Что **не** сохранять: - Имена локаций, описания, выходы — это статика из `world.txt`. - Список команд, обработчики — это код.

То есть сохраняется **разница** между «свежим миром из `world.txt`» и «текущим состоянием». Это уменьшает размер файла и облегчает миграции (если поменяем мир — старые сейвы продолжат работать в том, что не задела).

25.2 Текстовый или бинарный формат

Два подхода.

Бинарный: - Записываем байты значений напрямую (через `out.write(&value, sizeof(value))`). - Компактно, быстро. - Нечитаемо без декодера. - Зависит от `endianness`, размеров типов.

Текстовый: - Каждое значение — строка. - Больше места, медленнее. - Читается глазами. - Легко версионировать и отлаживать.

Для нашей RPG — **текстовый**. Размер сейва — килобайты, скорость неважна. Зато можно открыть в редакторе и посмотреть, что не так. В мини-СУБД (Часть IV) будет бинарный — там скорость и размер критичны.

25.3 Формат сейва

```
SAVE 1
name Герой
hp 20 20
loc mountain
inv weapon ржавый_меч 5 8
wloc cave weapon боевой_топор 7 12
wloc cave armor стальной_щит 6 6
wloc mountain consumable вода 1 3
wloc forest armor кожаный_нагрудник 8 4
wloc village consumable зелье_лечения 1 15
wloc village consumable хлеб 1 5
CHK 5e54e64c
```

Структура:

- **SAVE <version>** — первая строка, версия формата.
- **name <player_name>** — имя игрока (_ вместо пробелов).
- **hp <current> <max>** — здоровье.
- **loc <location_id>** — где игрок.
- **inv <type> <name> <weight> <extra>** — предмет в инвентаре. Несколько строк.
- **wloc <loc_id> <type> <name> <weight> <extra>** — предмет, оставшийся в локации.
- **CHK <8_hex_digits>** — контрольная сумма для проверки целостности.

Похоже на формат мира из главы 23, но с другими ключами. Один и тот же стиль.

25.4 Зачем версия

Программа развивается. Завтра вы добавите экипировку — у Player появится поле `equipped_weapon`. Старые сейвы не знают о нём.

Без версии: чтение крашится на отсутствии новых полей. Игрок теряет прогресс.

С версией: программа смотрит, какая версия в файле, и применяет соответствующую логику парсинга. Например: - v1 — без экипировки. - v2 — с экипировкой; для v1-сейвов считать «не экипирован».

В нашем коде:

```
namespace save {  
    constexpr int CURRENT_VERSION = 1;  
    // ...  
}
```

При загрузке проверяем:

```
if (ver != CURRENT_VERSION) {  
    throw GameError("несовместимая версия сейва: " + std::to_string(ver));  
}
```

Это **строгая** реакция. Альтернатива — **миграции**: при $v < \text{CURRENT_VERSION}$ вызывать функцию `migrate_v1_to_v2`, потом загружать как v2. Так делают серьёзные приложения. У нас пока строгое отклонение.

25.5 Зачем контрольная сумма

Файлы повреждаются. Прервалась запись (отключили питание), повредился диск, кто-то открыл в редакторе и неудачно правил. Загружать **повреждённый** сейв опасно — можно получить непредсказуемое состояние, краш, или хуже — испорченную игровую логику.

Контрольная сумма (checksum) — функция, дающая на основе содержимого короткое значение. Если содержимое изменилось — значение тоже. На загрузке сравниваем сохранённую сумму с пересчитанной по содержимому. Не сходится — отказ.

Простые алгоритмы: - **Сумма байтов** — очень простая, плохо ловит «компенсирующие» изменения. - **CRC32** — стандартное, точное обнаружение. - **Хеш-функции** (MD5, SHA-1) — криптографические, но избыточны для нашей задачи.

У нас — простой 32-битный «накатывающий хеш»:

```
std::uint32_t checksum(const std::string& text) {
    std::uint32_t sum = 0;
    for (char ch : text) {
        sum = sum * 31u + static_cast<unsigned char>(ch);
    }
    return sum;
}
```

Это **не криптография**. Злонамеренный пользователь подделает её за минуту. Защита только от **случайных** повреждений. Для нашей цели — достаточно.

31 — стандартный множитель в хешах строк (используется в Java String.hashCode). unsigned char — чтобы избежать UB при отрицательных значениях char (см. главу 8).

25.6 API менеджера сохранений

include/save_manager.h:

```
namespace rpg {
namespace save {

constexpr int CURRENT_VERSION = 1;

void write(const std::string& path,
           const Player& player,
           const World& world,
           const std::string& current_location_id);

void read(const std::string& path,
           Player& player,
           World& world,
           std::string& current_location_id);

} // namespace save
} // namespace rpg
```

Две функции: write и read. Оба бросают GameError при проблемах.

Заметьте: save — это **внутреннее пространство имён** внутри rpg. Так не «загрязняем» основной namespace. Использование: rpg::save::write(...).

25.7 Реализация write

```

void write(const std::string& path,
          const Player& player,
          const World& world,
          const std::string& current_location_id) {
    std::ostringstream body;
    body << "SAVE " << CURRENT_VERSION << "\n";
    body << "name " << escape(player.name()) << "\n";
    body << "hp " << player.hp() << " " << player.max_hp() << "\n";
    body << "loc " << current_location_id << "\n";

    for (const auto& item : player.inventory().items()) {
        body << "inv " << item_line(*item) << "\n";
    }
    for (const auto& kv : world.locations()) {
        const Location& loc = kv.second;
        for (const auto& item : loc.items().items()) {
            body << "wloc " << loc.id() << " " << item_line(*item) << "\n";
        }
    }

    std::string content = body.str();
    std::uint32_t sum = checksum(content);

    std::ofstream out(path);
    if (!out.is_open()) {
        throw GameError("не открыт для записи: " + path);
    }
    out << content << "CHK " << to_hex(sum) << "\n";
}

```

Алгоритм:

1. **Собираем тело в stringstream**, чтобы посчитать checksum.
2. Каждое поле — одна строка.
3. Инвентарь игрока через цикл.
4. Все локации мира через `world.locations()` (World теперь возвращает свой `unordered_map`).
5. **Считаем checksum от тела.**

6. Записываем в файл: тело + строка СНК <hex>.

Зачем сначала в stringstream, а потом сразу в файл? Чтобы: 1. Посчитать checksum от того же текста, что попадёт в файл. 2. Если запись частично прошла и оборвалась — можно повторить.

25.7.1 Сериализация предмета

```
std::string item_line(const Item& it) {
    std::ostringstream ss;
    if (auto w = dynamic_cast<const Weapon*>(&it)) {
        ss << "weapon " << escape(w->name()) << " " << w->weight() << " " << w->damage();
    } else if (auto a = dynamic_cast<const Armor*>(&it)) {
        ss << "armor " << escape(a->name()) << " " << a->weight() << " " << a->defense();
    } else if (auto c = dynamic_cast<const Consumable*>(&it)) {
        ss << "consumable " << escape(c->name()) << " " << c->weight() << " " << c->heal_amount();
    } else {
        throw GameError("неизвестный тип предмета при сохранении");
    }
    return ss.str();
}
```

Через `dynamic_cast` определяем реальный тип и пишем нужное «extra» поле (урон для оружия, защиту для брони, лечение для расходника).

Этот код **дублирует** логику парсера из главы 23 — там по типу мы создавали потомков, здесь по типу пишем. Это типичная неприятность сериализации: чтение и запись должны быть **синхронны**.

Альтернативно можно было бы добавить виртуальный метод `Item::serialize(std::ostream&)`, и `Weapon`/`Armor`/`Consumable` его переопределили бы. Это чище — логика типа лежит в самом классе. Но требует трогать `Item.h`. Для учебного — `dynamic_cast` проще.

25.7.2 Запись в файл

```
std::ofstream out(path);
if (!out.is_open()) {
    throw GameError("не открыт для записи: " + path);
}
```

Если файл не открылся — бросаем. Чаще всего причина — нет папки `data/saves/`.

25.7.3 Эскейпинг

`escape(name)` заменяет пробелы на `_`. На загрузке `unescape` обратно. Если бы оставить пробелы, поток `>>` разделил бы имя на части и парсинг сломался.

Альтернатива — заключать имена в кавычки и парсить их специально. Делать сложнее, у нас обходимся подчёркиваниями.

25.8 Реализация `read`

```
void read(const std::string& path,
          Player& player,
          World& world,
          std::string& current_location_id) {
    std::ifstream in(path);
    if (!in.is_open()) throw GameError("сейв не открыт: " + path);

    std::ostringstream buf;
    buf << in.rdbuf();
    std::string text = buf.str();
    // ...
}
```

Сначала читаем **весь файл в строку**. Зачем? Чтобы проверить checksum: нужен текст без строки CHK.

```
auto chk_pos = text.rfind("\nCHK ");
if (chk_pos == std::string::npos) {
    throw GameError("в сейве нет CHK — файл повреждён");
}

std::string body = text.substr(0, chk_pos + 1);
std::string chk_line = text.substr(chk_pos + 1);

std::istringstream chk_ss(chk_line);
std::string chk_word, chk_hex;
chk_ss >> chk_word >> chk_hex;

std::uint32_t expected = static_cast<std::uint32_t>(std::stoul(chk_hex, nullptr, 16));
std::uint32_t actual = checksum(body);
if (expected != actual) {
```

```
throw GameError("чек-сумма не сходится — сейв повреждён");
}
```

`rfind("\nCHK ")` — ищем строку CHK с конца. `text.substr(0, chk_pos + 1)` — всё до CHK (включая `\n`). `text.substr(chk_pos + 1)` — сама строка CHK.

Парсим hex через `std::stoul(s, nullptr, 16)`. 16 — основание системы.

Сравниваем суммы. Не сходятся — бросаем.

25.8.1 Двухпроходный парсер

Главная тонкость загрузки: **не применять состояние сразу**. Если посередине файла ошибка, у нас окажется наполовину загруженная игра — нелогичное состояние.

Решение — **двухпроходно**: сначала всё парсим во **временные структуры**, проверяем валидность, **потом** атомарно применяем.

```
std::string new_name = player.name();
int new_hp = 0, new_max = 1;
std::string new_loc;
std::vector<std::unique_ptr<Item>> player_items;
std::unordered_map<std::string, std::vector<std::unique_ptr<Item>>> location_items;

while (std::getline(lines, line)) {
    // парсим в new_name / new_hp / player_items / location_items
}

// Валидация
for (const auto& kv : location_items) {
    if (!world.find(kv.first)) {
        throw GameError("сейв ссылается на неизвестную локацию: " + kv.first);
    }
}

// Применение — теперь точно ничего не упадёт
player = Player(new_name, new_max);
player.set_hp(new_hp);
for (auto& it : player_items) {
    player.inventory().add(std::move(it));
}
```



```

for (const auto& kv : world.locations()) {
    Location* mloc = world.find(kv.first);
    if (mloc) mloc->items().clear();
}
for (auto& kv : location_items) {
    Location* mloc = world.find(kv.first);
    for (auto& it : kv.second) {
        mloc->items().add(std::move(it));
    }
}
current_location_id = new_loc;

```

Это **strong exception guarantee** (глава 22): если что-то бросит до «применения» — состояние программы не изменилось. Если применение пошло — гарантированно завершится (нет операций, которые могут бросить на этом этапе; перемещения `unique_ptr` это поexcept).

25.8.2 Replace player

```

player = Player(new_name, new_max);
player.set_hp(new_hp);

```

Move-assignment `player_ = ....`. Поскольку `Player` move-assignable (rule of zero — все поля движимы), это работает. Старый `Player` уничтожается, новый занимает его место.

После пересоздания `player` — пустой инвентарь. Далее добавляем сохранённые предметы.

25.9 Команды в Game

```

void Game::cmd_save(const std::string& slot) {
    std::string path = "data/saves/" + slot + ".sav";
    try {
        save::write(path, player_, world_, current_location_id_);
        std::cout << "Сохранено в " << path << "\n";
    } catch (const GameError& e) {
        std::cout << "Ошибка сохранения: " << e.what() << "\n";
    }
}

void Game::cmd_load(const std::string& slot) {

```

```

std::string path = "data/saves/" + slot + ".sav";
try {
    World fresh = load_world("data/world.txt");
    save::read(path, player_, fresh, current_location_id_);
    world_ = std::move(fresh);
    std::cout << "Загружено из " << path << "\n";
    cmd_look();
} catch (const GameError& e) {
    std::cout << "Ошибка загрузки: " << e.what() << "\n";
}
}

```

cmd_save собирает путь, вызывает write. Исключения ловит и показывает по-человечески.

cmd_load сначала загружает **свежий мир** из world.txt (статика — имена локаций, описания, выходы), потом накладывает сейв на этот мир (динамика — игрок, оставшиеся предметы). Если всё ок — заменяет world_ на fresh.

Регистрация:

```

commands_.add("save", [this](const std::string& rest) {
    if (rest.empty()) std::cout << "Имя слота?\n";
    else cmd_save(rest);
});
commands_.add("load", [this](const std::string& rest) {
    if (rest.empty()) std::cout << "Имя слота?\n";
    else cmd_load(rest);
});

```

25.10 Тест

```

$ mkdir -p data/saves
$ make
$ echo -e "take ржавый меч\ngo south\nsave slot1\nq" | ./build/rpg
...
> Вы подняли: ржавый меч
== Горная тропка ==
...
> Сохранено в data/saves/slot1.sav

```

```
$ cat data/saves/slot1.sav
SAVE 1
name Герой
hp 20 20
loc mountain
inv weapon ржавый_меч 5 8
wloc cave weapon боевой_топор 7 12
wloc cave armor стальной_щит 6 6
wloc mountain consumable вода 1 3
wloc forest armor кожаный_нагрудник 8 4
wloc village consumable зелье_лечения 1 15
wloc village consumable хлеб 1 5
CHK 5e54e64c

$ echo -e "load slot1\nstatus\ninv\nq" | ./build/rpg
== Лесная поляна ==
...
> Загружено из data/saves/slot1.sav
== Горная тропка ==
> Герой: HP 20/20
> Инвентарь (общий вес 5):
  - ржавый меч (оружие, урон 8, вес 5)
```

Сохранение/загрузка работают.

Заметьте: после загрузки **меч в инвентаре** (мы его взяли до сейва), на **поляне его нет** (мы его подняли), мы **на горной тропке** (мы туда переместились). Состояние восстановлено в точности.

25.11 Полломка checksum

```
$ sed -i '' 's/CHK 5e54e64c/CHK 00000000/' data/saves/slot1.sav
$ echo "load slot1" | ./build/rpg
> Ошибка загрузки: GameError: чек-сумма не сходится — сейв повреждён
```

Изменили чек-сумму руками — load отказался. Это и есть страховка.

Удалите CHK строку совсем:

```
$ sed -i '' '/^CHK /d' data/saves/slot1.sav
$ echo "load slot1" | ./build/rpg
> Ошибка загрузки: GameError: в сейве нет CHK — файл повреждён
```

Опять отказ. Файл без чек-суммы — повреждён.

25.12 Что осталось

В реальной игре сделали бы ещё:

- **Метаданные сейва:** дата создания, имя игры, скриншот.
- **Несколько слотов** с автоматическими именами (slot1, slot2, ..., autosave).
- **Quicksave / quickload** на горячие клавиши.
- **Журнал изменений** (autosave каждые N минут).
- **Облачная синхронизация.**

Это всё надстройка над тем, что мы сделали. Базовая работа — write / read с версией и checksum — закрыта.

25.13 Бинарный формат — кратко

Для контраста: тот же сейв в бинарном виде мог выглядеть так:

```
out.write(reinterpret_cast<const char*>(&version), sizeof(version)); // 4 байта
out.write(reinterpret_cast<const char*>(&name_len), sizeof(name_len));
out.write(name.data(), name_len);
out.write(reinterpret_cast<const char*>(&hp), sizeof(hp));
out.write(reinterpret_cast<const char*>(&max_hp), sizeof(max_hp));
// ...
```

Преимущества: - Меньше места: int это 4 байта, не 1-10 символов как в тексте. - Быстрее запись и чтение. - Не нужен парсинг — читаем по байтам в нужные позиции.

Недостатки: - Нечитаемо без декодера. - Зависит от endianness (на разных архитектурах байты в int могут лежать по-разному). - Сложнее версионировать (добавил поле — старый файл неправильно интерпретируется). - Сложнее отлаживать.

В Части IV (мини-СУБД) мы будем плотно работать с бинарными форматами — там скорость и размер критичны. Для игры — текст лучше.

25.14 Главные правила главы

1. **Сохранение и парсер дополняют друг друга** — формат должен уметь сериализовать что парсер понимает.
2. **Версия в первой строке** — для обратной совместимости.
3. **Checksum для целостности** — простой хеш достаточен.
4. **Двухпроходный загрузчик** — сначала всё в локальные переменные, потом применяем (strong guarantee).
5. **Текст для отладки, бинарь для производительности.**
6. **Сохраняйте только разницу со статикой** — не дублируйте world.txt в сейве.
7. **Эскейпинг для значений с пробелами** — `_` → или кавычки.
8. **Метаданные** (дата, имя) полезны на больших проектах.

25.15 Маленькое упражнение

1. Запустите игру, поиграйте, сохраните в slot1. Откройте data/saves/slot1.sav в редакторе. Изучите формат.
2. Сделайте slot2, slot3. Все работают независимо.
3. Сломайте чек-сумму руками — проверьте, что load отказывает.
4. Добавьте поддержку **saves** — команда выводящая список существующих сейвов в data/saves/. Подсказка: `std::filesystem` в C++17 / для C++11 — `opendir` из `<dirent.h>` или просто пробег по предсказуемым именам.
5. Добавьте в сейв **дату создания** (число секунд от эпохи). Используйте `std::time(nullptr)`. На загрузке игнорируйте, но при списке сейвов выводите.
6. Добавьте версию 2: в неё дополнительное поле «опыт игрока» (поле в Player ещё не добавлено — добавьте). Сделайте код, который при `v=1` ставит опыт в 0, при `v=2` читает.
7. (Сложнее) Замените простую сумму на CRC32. Готовая реализация — `<zlib.h>` или таблица CRC32 руками. Сравните, как ловит специально подобранные повреждения.
8. (Сложнее) Перепишите сейв в бинарном формате. Сравните размер с текстовым.

25.16 Что дальше

Глава 25 — **финал RPG**. Закрепляем всё, что построили. Модульные тесты с GoogleTest (или свой простой test framework). Сборка на macOS/Linux/Windows. Итоги. Что осталось как упражнения для читателя. Это закрывает Часть II.

После — Часть III (mini-shell), Часть IV (мини-СУБД), Часть V (ТСР-чат), Часть VI (C++17 бонусы). Тут разворачивается остальная половина книги.

Глава 26

Глава 25. Финал RPG

Часть II подходит к концу. У нас в репозитории `demo-rpg/` — больше 1600 строк C++, полностью рабочий мини-движок текстовой RPG. Игрок ходит по миру, поднимает предметы, использует расходники, сохраняет и загружает игру. Мир — в текстовом файле, можно править без перекомпиляции.

В этой финальной главе мы: 1. Добавим **юнит-тесты** для основных классов — практика, которой пользуются на любых проектах. 2. Соберём итог: что построили в C++, какие приёмы освоили. 3. Поговорим про сборку на других платформах. 4. Обсудим, что осталось «за кадром» и можно добавить в качестве упражнений.

После этой главы Часть II закрыта. Дальше — три новых проекта (`mini-shell`, мини-СУБД, ТСП-чат) и обзорная Часть VI про C++17 как бонус.

26.1 Зачем юнит-тесты

Программа из 1600 строк уже достаточно большая, чтобы случайные изменения ломали неожиданные вещи. Вы правите `Inventory::take`, а тут ломается `cmd_drop`. Юнит-тесты — **первая линия защиты** от регрессий.

Хороший тест: - Покрывает один сценарий (одна функция, один аспект). - Запускается за миллисекунды. - Не зависит от других тестов. - Чётко говорит, что прошло и что упало.

Юнит-тесты пишутся **на ту же кодовую базу**, ловят баги до того, как они доберутся до пользователя, и заодно служат документацией: «вот так этот класс используется».

26.2 GoogleTest vs свой простой framework

Стандартный фреймворк тестирования C++ — **GoogleTest**. Он мощный (макросы `EXPECT_EQ`, фикстуры, параметризованные тесты), но требует подключения внешней зависимости.

Для нашего учебного проекта возьмём **свой минимальный test-runner**. Это: - Никаких внешних зависимостей. - Видна вся логика подсчёта. - Достаточно для нашего масштаба.

В реальном проекте на нескольких десятках тысяч строк — GoogleTest или Catch2. Идея та же, просто фреймворк удобнее.

26.3 Свой test-framework

tests/test_framework.h:

```
#ifndef RPG_TEST_FRAMEWORK_H
#define RPG_TEST_FRAMEWORK_H

#include <iostream>
#include <sstream>
#include <string>

namespace test {

struct Stats {
    int total = 0;
    int passed = 0;

    int report(const std::string& suite) const {
        std::cout << "[" << suite << "]" " << passed << "/" << total
                    << " прошло\n";
        return (passed == total) ? 0 : 1;
    }
};

} // namespace test

#define CHECK(stats, cond) \
    do { \
        (stats).total++; \
    }
```



```

    if (cond) {
        (stats).passed++;
    } else {
        std::cerr << "FAIL " << __FILE__ << ":" << __LINE__
            << " - " #cond << "\n";
    }
} while (0)

#define CHECK_EQ(stats, a, b)
do {
    (stats).total++;
    auto&& _a = (a);
    auto&& _b = (b);
    if (_a == _b) {
        (stats).passed++;
    } else {
        std::ostringstream _ss;
        _ss << "FAIL " << __FILE__ << ":" << __LINE__
            << " - " #a " == " #b " (got " << _a << " vs " << _b << ")";
        std::cerr << _ss.str() << "\n";
    }
} while (0)

#endif

```

Разбор макросов.

do { ... } while (0) — стандартный приём, чтобы макрос вёл себя как одна инструкция. Без этого `if (cond) CHECK(...)`; сломался бы — `do/while(0)` фиксирует «один блок, требующий ; после».

#cond — stringification: имя выражения превращается в строку. `CHECK(s, x > 5)` напечатает `"x > 5"`.

__FILE__ и __LINE__ — макросы препроцессора, подставляют имя файла и номер строки.

auto&& _a = (a); — захват выражения в локальную ссылку, чтобы вычислить **один раз** (выражение могло иметь побочные эффекты — например, `++i`).

Это не идеальный фреймворк (нет фикстур, параметризации), но достаточный.

26.4 Юнит-тесты модулей

tests/test_units.cpp. Каждая функция — отдельная группа тестов:

```
void test_player(test::Stats& s) {
    Player p("Alice", 50);
    CHECK_EQ(s, p.name(), std::string("Alice"));
    CHECK_EQ(s, p.hp(), 50);
    CHECK(s, p.alive());

    p.take_damage(20);
    CHECK_EQ(s, p.hp(), 30);
    p.heal(100);           // не выше max
    CHECK_EQ(s, p.hp(), 50);
    p.take_damage(1000);   // не ниже 0
    CHECK_EQ(s, p.hp(), 0);
    CHECK(s, !p.alive());
}
```

Проверяем: конструктор работает, ограничения работают (heal не превысит max, damage не загонит в минус), alive() правильно меняется.

```
void test_player_validation(test::Stats& s) {
    bool empty_caught = false;
    try { Player p("", 10); }
    catch (const ValidationError&) { empty_caught = true; }
    CHECK(s, empty_caught);

    bool neg_caught = false;
    try { Player p("ok", -1); }
    catch (const ValidationError&) { neg_caught = true; }
    CHECK(s, neg_caught);
}
```

Проверяем валидацию: пустое имя и отрицательный hp бросают исключение. Это **обратная сторона**: показываем, что баги (поломки) тоже работают как ожидаешь.

```
void test_inventory(test::Stats& s) {
    Inventory inv;
    CHECK(s, inv.empty());
}
```

```

    inv.add(make_unique<Weapon>("меч", 5, 8));
    inv.add(make_unique<Armor>("шлем", 3, 4));
    CHECK_EQ(s, inv.items().size(), static_cast<std::size_t>(2));
    CHECK_EQ(s, inv.total_weight(), 8);

    auto taken = inv.take("меч");
    CHECK(s, static_cast<bool>(taken));
    CHECK_EQ(s, taken->name(), std::string("меч"));
    CHECK_EQ(s, inv.items().size(), static_cast<std::size_t>(1));

    auto miss = inv.take("щит");
    CHECK(s, !miss);

    inv.clear();
    CHECK(s, inv.empty());
}

```

Полный жизненный цикл инвентаря: пустой → добавили → проверили → взяли → проверили → попытались взять несуществующее → проверили → очистили.

`static_cast<std::size_t>(2)` — для подавления `-Wsign-compare`: `size()` возвращает `size_t` (unsigned), а 2 — это `int` (signed). С строгими флагами компилятор требует, чтобы оба были одного знака.

26.4.1 Парсер и save/load

```

void test_world_parser(test::Stats& s) {
    std::string source =
        "location start\n"
        "name Старт\n"
        "desc Тут начало.\n"
        "exit east finish\n"
        "item weapon кинжал 2 4\n"
        "\n"
        "location finish\n"
        "name Финиш\n"
        "desc Конец.\n";
    std::istringstream in(source);
    World w = parse_world(in);
}

```

```

CHECK_EQ(s, w.size(), static_cast<std::size_t>(2));

const Location* start = w.find("start");
CHECK(s, start != nullptr);
CHECK_EQ(s, start->name(), std::string("Старт"));
CHECK_EQ(s, start->exit("east"), std::string("finish"));
CHECK_EQ(s, start->items().items().size(), static_cast<std::size_t>(1));
}

```

Парсер тестируем **на строке**, не на файле. Так быстрее и тест не зависит от внешних файлов. `parse_world` принимает `std::istream&` — через `std::istreamstream` подаём строку.

```

void test_save_load_roundtrip(test::Stats& s) {
    World w;
    Location loc("home", "Дом", "Тут уютно.");
    w.add(std::move(loc));

    Player p("Hero", 100);
    p.take_damage(30);
    p.inventory().add(make_unique<Consumable>("яблоко", 1, 5));

    std::string path = "/tmp/rpg_test_save.sav";
    save::write(path, p, w, "home");

    // Восстанавливаем в другой Player/World
    Player p2("Other", 50);
    World w2;
    Location loc2("home", "Дом", "Тут уютно.");
    w2.add(std::move(loc2));
    std::string cur;
    save::read(path, p2, w2, cur);

    CHECK_EQ(s, p2.name(), std::string("Hero"));
    CHECK_EQ(s, p2.hp(), 70);
    CHECK_EQ(s, p2.max_hp(), 100);
    CHECK_EQ(s, cur, std::string("home"));
    CHECK_EQ(s, p2.inventory().items().size(), static_cast<std::size_t>(1));
}

```

Roundtrip тест: записали → прочли → проверили совпадение. Если что-то сломали в сериализации — тест упадёт.

26.4.2 main и запуск

```
int main() {
    test::Stats s;
    test_player(s);
    test_player_validation(s);
    test_inventory(s);
    test_location(s);
    test_world(s);
    test_world_parser(s);
    test_world_parser_errors(s);
    test_save_load_roundtrip(s);
    return s.report("unit-tests");
}
```

Все тесты передают одну и ту же Stats. В конце — отчёт.

Сборка и запуск:

```
$ make tests
$ ./build/tests/test_units
[unit-tests] 37/37 прошло
$ echo $?
0
```

37 проверок, все прошли. Код возврата 0.

Если запустить через CI (Continuous Integration) — например, GitHub Actions — этот тест автоматически запускается на каждый push. Любая регрессия будет видна.

26.5 Сборка на нескольких платформах

Makefile мы заточили под GCC/Clang с Unix-инструментами. Это работает: - **macOS** — clang/g++ из Xcode Command Line Tools (глава 6). - **Linux** — gcc из build-essential. - **Windows + WSL2** — Linux-окружение в Windows.

Что **не работает** на нативном Windows (без WSL): - `mkdir -p` — на Windows это другое. - `rm -rf` — на `cmd.exe` нет `rm`. - Возможно проблемы с UTF-8 в исходниках (MSVC требует особых флагов).

Решений несколько:

1. Использовать CMake вместо Makefile. CMake генерирует Visual Studio проекты на Windows, Makefile на Unix, Ninja-файлы — везде.

```
cmake_minimum_required(VERSION 3.10)
project(rpg CXX)
set(CMAKE_CXX_STANDARD 11)

add_executable(rpg
    src/main.cpp
    src/game.cpp
    src/commands.cpp
    src/item.cpp
    src/inventory.cpp
    src/location.cpp
    src/player.cpp
    src/world.cpp
    src/world_parser.cpp
    src/save_manager.cpp
)

target_include_directories(rpg PRIVATE include)
target_compile_options(rpg PRIVATE -Wall -Wextra)
```

Команда:

```
$ cmake -B build
$ cmake --build build
```

Получаете build/rpg независимо от платформы.

2. WSL2 на Windows. Простейший вариант: пользователь на Windows ставит WSL2, дальше всё как на Linux.

3. Условные правила в Makefile. Можно усложнить наш Makefile, чтобы он автоопределял ОС:

```
ifeq ($(OS),Windows_NT)
    MKDIR := mkdir
    RM     := rmdir /S /Q
else
    MKDIR := mkdir -p
```

```
RM      := rm -rf
endif
```

— и так далее. Возможно, но рассыпает простоту.

Для учебной книги — **простой Makefile + рекомендация WSL на Windows**. На реальном проекте перешли бы на CMake.

26.6 Что мы построили — обзор

Структура demo-rpg/:

```
include/
├─ commands.h      — реестр команд
├─ errors.h        — иерархия исключений GameError/ValidationError/WorldError
├─ event_bus.h     — шаблонная шина событий
├─ events.h        — типы игровых событий
├─ game.h          — главный класс
├─ inventory.h     — инвентарь
├─ item.h          — иерархия предметов (abstract Item + Weapon/Armor/Consumable)
├─ location.h      — локация с выходами и предметами
├─ player.h        — игрок
├─ save_manager.h  — сохранение/загрузка
├─ util.h          — make_unique-шим
├─ world.h         — мир (коллекция локаций)
└─ world_parser.h  — парсер мира из text-файла
src/               — реализации (.cpp на каждый заголовок)
tests/             — юнит-тесты
data/
├─ world.txt       — описание мира
└─ saves/          — слоты сохранений
build/             — артефакты сборки (в .gitignore)
Makefile
```

13 заголовков, 11 реализаций, 6 тестовых файлов. Около 1700 строк C++.

26.7 Что покрыли в C++

Полезно перечислить, что именно из языка пошло в дело — не в виде списка тем из оглавления, а по месту применения.

Из основ работает всё, что было в Части I. Типы и переменные — в каждой строке. Ветвления и циклы, в том числе цикл по диапазону, — в обходах инвентаря и выходов из локации. Разные способы передачи параметров: по значению там, где объект дешёвый, по константной ссылке для чтения, по значению с последующим перемещением для передачи владения. Память на стеке и в куче с освобождением по правилу RAII. Строки и потоки ввода-вывода — в парсере мира и в сохранении. Контейнеры `vector` и `unordered_map` — в инвентаре и в мире. Алгоритмы `find_if`, `sort` и `replace`.

Из продвинутого в проекте задействовано почти всё, о чём шла речь в Части II. Классы с инкапсуляцией — `Player`, `Location`, `World`, `Inventory`, `Item`. Списки инициализации в конструкторах и правильная расстановка `const` у методов и параметров. Наследование с виртуальными методами, `override`, `final`, виртуальным деструктором и чистыми виртуальными функциями — иерархия предметов. Запрет копирования через `= delete` там, где объект владеет уникальным ресурсом. Умные указатели `std::unique_ptr<Item>` в инвентаре и перемещение вместо копирования при передаче предметов. Шаблоны — `EventBus<T>` и наш собственный `make_unique<T>` с переменным числом аргументов и совершенной передачей. Лямбды с захватами и `std::function` в реестре команд. Исключения со своей иерархией и строгой гарантией безопасности в загрузке сохранения. Разбор текстовых файлов, сериализация в файл и обратно, контрольная сумма для проверки целостности. И пространства имён, которыми всё это разложено по местам.

Список выглядит внушительно, но суть не в количестве. Суть в том, что каждая из этих вещей появилась не сама по себе, а потому что понадобилась: виртуальный деструктор — когда предметы стали храниться по указателю на базу, перемещение — когда предмет понадобилось передать из локации игроку, исключения — когда обнаружилось, что ошибку загрузки некуда возвращать. Этого достаточно, чтобы читать почти любой C++-проект из открытых источников; незнакомые места останутся, но словарь у вас уже есть.

26.8 Что осталось «за кадром»

Игру можно расширять, и дизайн для этого готов. Перечислю направления — не как список задач, а чтобы показать, куда именно ложится каждое расширение в уже существующей архитектуре.

Бой требует нового класса `Enemy` с очками здоровья и уроном, команды `attack` в реестре команд и обработки смерти — игрока или врага. Класс предметов трогать не придётся: оружие уже знает свой урон, а `dynamic_cast` из главы 17 позволит найти оружие в инвентаре.

Задания — это состояние, которого у нас пока нет: у игрока появляется список взятых заданий, а у локаций — персонажи, которые их выдают. Проверка выполнения удобно ложится на шину событий: подписались на событие «предмет подобран» и сверяем с условием задания.

Несколько персонажей означают, что сохранение перестаёт быть одним файлом. Формат менять не придётся — достаточно завести каталог на игрока, и как раз пригодится версия формата, которую мы заложили в главе 24.

Шина событий сейчас используется в одном месте. С ней можно сделать журнал всего урона, систему достижений, звуковые эффекты — всё это подписчики, о которых издатель ничего не знает. Это и есть польза от разделения, ради которого она затевалась.

Оформление в терминале — библиотека `ncurses` на Unix или `PCurses` на Windows: псевдо-окна, цвета, карта мира в виде рисунка из символов. Настоящая графика — `SDL2` или `SFML` — тоже возможна, но это уже другая книга.

Сетевая игра — несколько игроков в одном мире через TCP. После Части V навыков для этого хватит; правда, придётся решать вопросы, которых нет в одиночной игре: одновременные действия, согласование состояния, что делать при обрыве связи.

Ни одно из этих расширений не требует ломать существующее — новые классы добавляются рядом, новые команды регистрируются в реестре, новые подписчики появляются у шин. Это и есть проверка архитектуры на прочность.

26.9 Если бы это шло в работу

Учебный проект и рабочий отличаются не кодом, а обвязкой. Вот чего в нашем не хватает, если бы его пришлось выпускать всерьёз.

Сборка через `CMake` вместо `Makefile` — чтобы собиралось на трёх системах без правок. Тесты на готовом фреймворке вроде `GoogleTest` или `Catch2` — со внятными сообщениями и запуском по маске. Непрерывная интеграция, которая на каждый коммит собирает проект на Linux, macOS и Windows и прогоняет тесты; без неё тесты рано или поздно перестают запускаться.

Единый стиль кода, поддерживаемый `clang-format`, чтобы споры о расстановке скобок закончились раз и навсегда. Статический анализ `clang-tidy`, который находит подозрительные места до запуска. Санитайзеры адресов и неопределённого поведения в отладочной сборке.

Для парсеров — фаззинг: автоматическая подача случайного мусора в `world_parser` и загрузчик сохранений, чтобы найти падения, до которых не додумался бы ни один тест. Наш разбор мира к такому обращению, скорее всего, не готов, и это стоило бы проверить.

Плюс измерение покрытия тестами, документация в исходниках для `doxygen` и нормальное журналирование вместо печати в поток вывода.

Каждый пункт — от получаса до дня работы, а вместе они превращают «работает у меня» в «работает у всех». Для учебного проекта мы сделали разумный минимум: тесты, разделение на модули, обработка ошибок, версия формата.

26.10 Итог Части II

С главы 15 (дизайн) до главы 25 (финал) мы построили рабочую программу. Не «hello world», не «упражнение из учебника», а **проект, в который не стыдно показать собеседнику**. С тестами, документацией в коде, разделением модулей, обработкой ошибок, версионированием формата сохранений.

Если вы шли по главам последовательно, к этому моменту у вас должно быть ощущение «я могу написать C++-программу». Не «зазубрил синтаксис», а **могу разложить задачу, выбрать структуру данных, написать класс с правильной семантикой, обработать ошибки**. Это и есть **навык**.

Дальше — три новых проекта, каждый в своей области. В Части III мы напишем командную оболочку и познакомимся с системным интерфейсом Unix: процессами, файловыми дескрипторами, каналами, сигналами. В Части IV построим базу данных со страничным файлом, B+-деревом, журналом и собственным SQL — это работа о корректности и производительности на диске. В Части V сделаем сетевой чат: сокеты, потоки, цикл событий, свой двоичный протокол. Часть VI, короткая, покажет, что добавили к языку стандарты после C++11.

Каждая часть — отдельный проект в своей папке: `demo-shell/`, `demo-db/`, `demo-chat/`. Структура везде та же, что в RPG: заголовки в `include/`, реализации в `src/`, проверки в `tests/`, сборка одним `make`.

26.11 Главные правила главы

1. Тесты нужны не ради процента покрытия, а ради возможности менять код и сразу узнавать, что сломалось.
2. Один класс — один тестовый файл; свалка из всех проверок в одном месте перестаёт читаться на третьей сотне строк.
3. Проверка «записал, прочитал, сравнил» — самый ценный вид теста для любой сериализации.
4. Тесты, которые не запускаются автоматически, живут ровно до тех пор, пока о них помнят.
5. Makefile хорош для учёбы, CMake — для проекта, который собирают на разных системах.
6. Санитайзеры адресов и неопределённого поведения должны быть включены в отладочной сборке, а не откладываться на потом.
7. Хорошая архитектура проверяется добавлением новой возможности: если для этого не пришлось трогать старое — она хорошая.

8. Цель — не запомнить синтаксис, а научиться разбирать задачу; синтаксис доучивается по справочнику.

26.12 Маленькое упражнение

1. Запустите `make tests && ./build/tests/test_units`. Все 37 прошли?
2. Поломайте одну строку в `Player::heal` (например, уберите ограничение по `max_hp`). Запустите тесты — увидите `fail`.
3. Добавьте тест для команды `cmd_take`: создайте `Game`, имитируйте ввод (это сложно для CLI; можно через `Game::cmd_take(...)` напрямую через `friend` или `public` метод для теста).
4. Установите CMake (`brew install cmake` или `apt install cmake`). Напишите `CMakeLists.txt` по примеру выше. Соберите через `cmake -B build && cmake --build build`.
5. Реализуйте класс `Enemy` (имя, `hp`, `damage`). Добавьте поддержку в `world.txt`: `enemy гоблин 10 3` — кладёт врага в локацию. В `cmd_look` показывайте врагов.
6. Реализуйте команду `attack <enemy>` — пошаговый бой. Используйте `dynamic_cast` для определения «оружия» в инвентаре игрока.
7. (Сложнее) Добавьте `EventBus<DamageEvent>` в `Game`. Подпишите логгер в `register_commands`. В команде `attack` публикуйте события — увидите, как декаплинг работает.
8. (Сложнее) Прогоните игру под **AddressSanitizer**:

```
make CXXFLAGS="-std=c++11 -Wall -fsanitize=address -g"
```

Если ASan находит проблему — это серьёзный сигнал. Чистая программа не должна ничего давать.

26.13 Что дальше

Часть III — mini-shell demo-shell/ (главы 26–31): свой `bash`-подобный `shell` с пайпами, редиректами, `builtins`, историей. Это другая половина C++: системное программирование через POSIX API. Узнаем `fork`, `exec`, `wait`, `pipe`, `dup2`, сигналы, `terminal raw mode`.

Затем — **Часть IV (главы 32–39)**: мини-СУБД с `B+tree`, страничным файлом, WAL, мини-SQL.

Часть V (главы 40–47) — TCP-чат с реактор-циклом, `atomic`, многопоточностью.

Часть VI (главы 48–51) — C++17 как бонус.

Всё это — впереди. До встречи в Части III.

Глава 27

Глава 26. Процессы — `fork`, `exec`, `wait`

Начинается Часть III, и в ней мы шесть глав подряд будем писать собственную командную оболочку — свой маленький `bash`. Называться она будет `myshell`, лежать в папке `demo-shell/`, и к концу части в ней заработают кавычки, пайпы, перенаправления, встроенные команды, история и даже собственные `cat`, `wc` и `grep`.

Это другая половина C++. В Части II мы строили RPG: классы, наследование, умные указатели, шаблоны — всё внутри программы, всё в своей песочнице. Теперь мы выходим наружу и начинаем разговаривать с операционной системой напрямую. Новых языковых конструкций почти не будет: те же функции, те же `std::vector` и `std::string`. Зато появится новый словарь — системные вызовы. `fork`, `exec`, `wait`, `pipe`, `dup2`, `signal`. Это интерфейс, через который любая программа в Unix просит ядро что-то сделать.

Эта глава — про запуск программ. Разберёмся, что такое процесс с точки зрения ядра, как процесс раздваивается через `fork`, как он превращается в другую программу через `exec` и как родитель забирает результат через `wait`. А в конце соберём первую рабочую версию оболочки: вы вводите `ls /tmp`, она запускает настоящий `ls`, дожидается его и печатает приглашение обратно.

27.1 Что мы вообще строим

Оболочка — это программа, которая читает ваши команды и запускает другие программы. Когда вы открываете терминал, вас встречает `zsh` или `bash`; вы пишете `ls`, они находят `/bin/ls`, запускают его, ждут и снова печатают приглашение. Вот и всё, если убрать украшения. Украшений, правда, накопилось много: автодополнение по Tab, подсветка, история со стрелками, скрипты с циклами и функциями, работа в фоне, управление заданиями. Мы возьмём скелет — и он окажется удивительно небольшим.

Так будет выглядеть работа с нашей оболочкой в конце Части III:

```
$ ./build/myshell
=== myshell ===
Builtins: cd, pwd, export, unset, history, echo, exit.
Кавычки, пайпы, редиректы (>, >>, <, 2>, 2>&1).

myshell$ echo "привет, мир"
привет, мир
myshell$ cat Makefile | ./build/mygrep CXX | ./build/mywc -l
7
myshell$ ls nosuchfile 2> errors.txt
myshell$ cd /tmp
myshell$ pwd
/tmp
myshell$ exit
```

А в этой главе мы сделаем первый шаг — версию, которая умеет ровно одно: запустить внешнюю программу и дождаться её. Всё остальное вырастет из неё.

Зачем вообще писать оболочку в книге про C++? Первая причина — так вы узнаете POSIX. Системные вызовы `fork`, `exec`, `pipe`, `dup2` лежат в фундаменте всего, что работает на Linux и macOS: веб-сервер, запускающий обработчик, система сборки, вызывающая компилятор, Docker, запускающий контейнер, — все они делают то же, что будем делать мы. Вторая причина практичнее: «запустить внешнюю программу из своей» — задача, которая рано или поздно возникает в любом проекте. И третья: C++ здесь показывает лицо, о котором часто забывают. Не шаблоны и не Boost, а сырой системный вызов, возвращающий `-1` и выставляющий `errno`.

27.2 Где это не заработает

POSIX — стандарт мира Unix. Linux, macOS, FreeBSD его поддерживают. Нативная Windows — нет: там своё API, `CreateProcess`, `WaitForSingleObject`, `CreatePipe`. Идеи похожи, но ни одна строка из этой части на `cmd.exe` не соберётся.

Если вы на Windows, поставьте WSL2 — как это делается, было в главе 6. Внутри WSL2 живёт настоящий Linux, и всё из Части III там работает без единой правки. Отдельный компилятор под Windows ставить не нужно.

27.3 Процесс глазами ядра

В главе 1 процесс был описан просто: «запущенная программа», у которой есть своё адресное пространство. Пора добавить деталей, потому что дальше мы будем этими деталями пользоваться.

Когда ядро создаёт процесс, оно заводит на него учётную запись. В ней лежит номер процесса, PID — уникальное число, по которому процесс адресуют все остальные («убей 4712», «дождись 4712»). Рядом хранится PPID — номер родителя, того, кто этот процесс породил.

У процесса есть собственное адресное пространство. Это важнейшая вещь: код, глобальные данные, куча и стек живут в виртуальной памяти, которую ядро выделило именно этому процессу. Указатель `0x7ff7be13ca9c` в одном процессе и такой же указатель в другом ведут в разные физические байты. Именно поэтому упавшая программа не роняет соседнюю.

Ещё у процесса есть таблица открытых файловых дескрипторов. Дескриптор — это просто маленькое целое число, индекс в этой таблице. Три первых заведены заранее: 0 — стандартный ввод, 1 — стандартный вывод, 2 — поток ошибок. Всё, что вы открываете через `open`, получает следующие свободные номера. В главе 27 мы будем эту таблицу переставлять руками, и вся магия пайпов окажется именно в ней.

Наконец, у процесса есть текущая рабочая папка (относительно неё разрешаются все относительные пути), переменные окружения, идентификатор владельца UID и маска сигналов — набор указаний, что делать при каждом сигнале.

Процессы связаны в дерево. У каждого есть родитель; у корня дерева, процесса с PID 1 (`init`, а на современных Linux — `systemd`), родителя нет. Посмотреть на дерево можно так — вот настоящий вывод на моей машине:

```
$ ps -o pid,ppid,command -ax | head -4
PID  PPID COMMAND
    1     0 /sbin/launchd
  357    1 /System/Library/ExtensionKit/Extensions/...
```

На macOS корень дерева называется `launchd`, на Linux — `systemd`, но роль одинаковая. Когда вы запускаете `ls` в терминале, ваша оболочка становится его родителем, и цепочка выглядит так: `launchd` → терминал → `zsh` → `ls`.

Родство — не формальность. Когда процесс завершается, его код возврата ядро сохраняет и отдаёт **только родителю**. Никто другой этот код получить не может. На этом построен весь механизм `wait`, к которому мы придём через две страницы.

27.4 fork — процесс раздваивается

В большинстве языков «запустить программу» — это одна функция: `subprocess.run` в Python, `Process.Start` в C#. В Unix всё устроено иначе и на первый взгляд странно: сначала процесс копирует сам себя, а потом копия превращается в нужную программу. Два шага вместо одного. Шаг первый — `fork`.

`fork()` объявлена в `<unistd.h>` и не принимает аргументов. Она создаёт копию текущего процесса. После её вызова процессов становится два, и оба продолжают выполнение с одного и того же места — со строки, следующей за `fork`.

Различить их можно по возвращаемому значению. В родителе `fork` возвращает PID новорождённого ребёнка (положительное число), в ребёнке — ноль, а если создать процесс не удалось — `-1` в единственном оставшемся процессе. Отсюда канонический вид проверки: сначала ошибка, потом ребёнок, потом родитель.

Соберём первую программу. Создайте файл `fork1.cpp`:

```
#include <iostream>
#include <unistd.h>

int main() {
    std::cout << "до fork, я PID " << getpid() << "\n";

    pid_t pid = fork();

    if (pid < 0) {
        std::cerr << "fork не удался\n";
        return 1;
    }
    if (pid == 0) {
        std::cout << "ребёнок: мой PID " << getpid()
                  << ", родитель " << getppid() << "\n";
    } else {
        std::cout << "родитель: мой PID " << getpid()
                  << ", ребёнок " << pid << "\n";
    }
    return 0;
}
```

Здесь три новых имени. `pid_t` — тип для номера процесса; на всех живых системах это знаковое целое, но пишут именно `pid_t`, потому что стандарт не обещает конкретный размер. `getpid()` возвращает собственный PID, `getppid()` — PID родителя. Обе объявлены в `<unistd.h>` — этот заголовок вообще главный вход в POSIX-часть системы.

Собираем и запускаем:

```
$ g++ -std=c++11 -Wall -Wextra -pedantic fork1.cpp -o fork1
$ ./fork1
до fork, я PID 49674
родитель: мой PID 49674, ребёнок 49675
ребёнок: мой PID 49675, родитель 49674
```

Одна программа, один `main`, но строк вывода три. Строка «до fork» выполнилась один раз — тогда процесс был один. Дальше процессов стало два, и каждый напечатал свою строку. У родителя PID 49674, у ребёнка 49675, и ребёнок правильно видит родителя в `getppid()`.

Запустите ещё раз — номера будут другими, а порядок двух последних строк может поменяться местами. Это нормально и важно понять сразу: после `fork` два процесса выполняются независимо, и кто из них первым доберётся до `std::cout`, решает планировщик ядра. Никакого порядка тут не гарантировано. Если в вашей программе порядок важен — его надо обеспечивать явно, через `wait` или другие средства синхронизации.

27.4.1 Загадка удвоенной строки

Теперь тот же бинарник, но вывод перенаправлен в файл:

```
$ ./fork1 > out.txt
$ cat out.txt
до fork, я PID 49682
родитель: мой PID 49682, ребёнок 49683
до fork, я PID 49682
ребёнок: мой PID 49683, родитель 1
```

Строка «до fork» напечаталась дважды. Причём одним и тем же PID — 49682. Программу мы не меняли, только перенаправили вывод. Что произошло?

Дело в буферизации. `std::cout` не отдаёт байты ядру по одному символу — это было бы чудовищно медленно, каждый символ стоил бы системного вызова. Вместо этого символы копят в буфере внутри процесса и уходят наружу пачкой. Когда вывод идёт в терминал, поток буферизуется построчно: увидели `\n` — сбросили. Когда вывод идёт в файл или в канал, буфер полный: он копится до нескольких килобайт и сбрасывается только при заполнении или при завершении программы.

Дальше всё складывается само. При выводе в файл строка «до fork» осталась лежать в буфере. fork скопировал память процесса целиком — вместе с буфером и его содержимым. Теперь непрочитанная строка есть у обоих. И родитель, и ребёнок, завершаясь, честно сбросили свои буферы — отсюда два экземпляра одной строки.

Это классическая ловушка, на которой спотыкаются все, кто начинает работать с fork. Лечится она одной строкой — сбросить буферы до раздвоения:

```
std::cout << "до fork, я PID " << getpid() << "\n";
std::cout.flush();           // <--- буфер пуст, копировать нечего

pid_t pid = fork();
```

Запомните правило: **перед fork буферы должны быть пусты**. В нашей оболочке мы будем звать `std::cout.flush()` перед выводом приглашения, и это не только про приглашение — это ещё и защита от удвоения.

Обратите внимание на вторую странность в том же выводе: ребёнок написал «родитель 1». Не 49682, а 1. Это значит, что к моменту, когда ребёнок дошёл до `getppid()`, родитель уже успел завершиться — и осиротевшего ребёнка усыновил процесс номер 1. Про сирот поговорим отдельно, но обратите внимание, как легко это увидеть: два процесса живут своей жизнью, и родитель совершенно не обязан пережить ребёнка.

27.4.2 Два адресных пространства

Ребёнок получает копию памяти родителя. Именно копию, а не общий доступ: изменения в одном процессе другой не видит. Проверим это опытом, который заодно покажет, как устроена виртуальная память. Файл `cow.cpp`:

```
#include <iostream>
#include <string>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    std::string message = "исходная строка";
    int counter = 0;

    pid_t pid = fork();
    if (pid == 0) {
        message = "ребёнок переписал";
        counter = 100;
```

```

        std::cout << "ребёнок: counter=" << counter
                    << " по адресу " << static_cast<const void*>(&counter)
                    << ", message=" << message << "\n";
        std::cout.flush();
        _exit(0);
    }

    waitpid(pid, nullptr, 0);
    std::cout << "родитель: counter=" << counter
                << " по адресу " << static_cast<const void*>(&counter)
                << ", message=" << message << "\n";

    return 0;
}

```

`static_cast<const void*>(&counter)` нужен, чтобы `std::cout` напечатал адрес, а не значение; для `int*` перегрузки печати адреса нет, приведение к `void*` включает нужную. Запускаем:

```

$ ./cow
ребёнок: counter=100 по адресу 0x7ff7be13ca9c, message=ребёнок переписал
родитель: counter=0 по адресу 0x7ff7be13ca9c, message=исходная строка

```

Адрес совпадает до последней цифры. Значение — разное. Это и есть виртуальная память в чистом виде: `0x7ff7be13ca9c` — не адрес планки памяти, а адрес внутри личного адресного пространства процесса, и ядро отображает эти два одинаковых виртуальных адреса на две разные физические страницы.

Дословное копирование всей памяти было бы разорительным: породить ребёнка от процесса с гигабайтом данных значило бы скопировать гигабайт. Поэтому ядра давно делают копирование при записи, *copy-on-write*. Сразу после `fork` физические страницы у процессов общие и помечены как «только для чтения». Пока оба процесса читают — копий нет. Как только кто-то пишет в страницу, срабатывает аппаратное прерывание, ядро копирует именно эту страницу и разрешает запись в копию. Поэтому `fork` дешёв даже для толстых процессов, а платить приходится только за реально изменённые страницы.

Кроме памяти ребёнок наследует таблицу открытых дескрипторов (включая 0, 1 и 2 — благодаря этому запущенная из оболочки программа пишет в тот же терминал), текущую папку, переменные окружения, маску сигналов и права владельца. Свои у него только PID, PPID и обнулённые счётчики использованного процессорного времени; ожидающие доставки сигналы родителя ребёнку не передаются, файловые блокировки `flock` тоже не наследуются.

27.5 exec — процесс превращается в другую программу

fork даёт нам второй процесс, но это копия нас же. Чтобы он стал `ls`, нужен второй шаг — `exec`.

`exec` — не одна функция, а семейство из шести. Все они делают одно и то же, различаясь только формой вызова:

Функция	Аргументы	Поиск в \$PATH	Окружение
<code>execv</code>	массив	нет	наследуется
<code>execvp</code>	массив	да	наследуется
<code>execvpe</code>	массив	да	своё (Linux)
<code>execl</code>	список	нет	наследуется
<code>execlp</code>	список	да	наследуется
<code>execle</code>	список	да	своё

Буква `v` означает `vector` — аргументы передаются массивом `char* const argv[]`, и это удобно, когда их число известно только во время работы. Буква `l` (`list`) — аргументы перечисляются прямо в вызове через запятую, что удобно для фиксированных команд. Буква `p` включает поиск программы в `$PATH`; без неё имя трактуется как путь к файлу. Буква `e` позволяет передать своё окружение вместо унаследованного.

Нам нужен `execvp`: имя команды пользователь вводит как есть (`ls`, а не `/bin/ls`), а количество аргументов заранее неизвестно.

Посмотрим на превращение вживую. Файл `execdemo.cpp` — без всякого `fork`, просто один процесс:

```
#include <cerrno>
#include <cstring>
#include <iostream>
#include <unistd.h>

int main() {
    std::cout << "до exec: я myprog, PID " << getpid() << "\n";
    std::cout.flush();

    char prog[] = "/bin/echo";
    char arg1[] = "я теперь echo";
    char* const argv[] = { prog, arg1, nullptr };
}
```

```

    execv(prog, argv);

    std::cerr << "execv не сработал: " << std::strerror(errno) << "\n";
    return 127;
}

```

```

$ ./execdemo
до exec: я мурprog, PID 49737
я теперь echo
$ echo $?
0

```

Тот же процесс, тот же PID — а программа другая. `execv` не создала процесс: она выбросила из него всё старое содержимое и загрузила на его место `/bin/echo`. Код, данные, куча, стек — всё заменено. Дескрипторы, PID, рабочая папка и окружение остались. Строчка `std::cerr << ...` в этом запуске не выполнялась никогда: выполнять её было уже некому, старая программа перестала существовать.

Отсюда важнейшее свойство: **exec не возвращается**. Если управление вернулось на следующую строку — значит, запустить программу не удалось. Проверять код возврата `exec` бессмысленно, надо просто писать обработку ошибки сразу после вызова.

Почему написано так подробно — `char prog[]` вместо строкового литерала? Потому что `execv` принимает `char* const argv[]`, то есть массив указателей на изменяемые символы. Строковый литерал в C++ имеет тип `const char[]`, и передать его напрямую нельзя — компилятор откажется. Это наследие C семидесятых, когда `const` ещё не придумали. В реальном коде обычно пишут `const_cast<char*>(s.c_str())`; функция ничего не меняет в переданных строках, так что это безопасно, и через несколько страниц мы сделаем именно так.

27.5.1 Что происходит, когда программы нет

Проверим ошибку. Файл `execfail.cpp`:

```

#include <cerrno>
#include <cstring>
#include <iostream>
#include <unistd.h>

int main() {
    char prog[] = "definitely-not-a-real-command";

```

```
char* const argv[] = { prog, nullptr };
execvp(prog, argv);
std::cerr << "execvp: errno=" << errno << " (" << std::strerror(errno) << ")\n";
return 127;
}
```

```
$ ./execfail
execvp: errno=2 (No such file or directory)
$ echo $?
127
```

`errno` — глобальная (точнее, потоко-локальная) переменная из `<cerrno>`, куда системные вызовы кладут код последней ошибки. `std::strerror` из `<cstring>` переводит код в человеческую строку. Коды, которые вы будете видеть чаще всего при неудачном `exec`: `ENOENT` (2) — файла нет ни в одном каталоге из `$PATH`; `EACCES` (13) — файл есть, но нет права на исполнение; `ENOEXEC` (8) — файл есть, но это не исполняемый формат.

Возврат 127 — не случайное число. Это соглашение всех Unix-оболочек: 127 значит «команда не найдена», 126 — «найдена, но не запускается». Наш `myshell` будет соблюдать эту традицию, и любой скрипт, который его вызовет, поймёт код правильно.

27.5.2 `_exit` вместо `exit`

Ещё один пункт, где легко ошибиться. После неудачного `exec` ребёнка нужно завершить — но каким способом?

`exit(N)` — стандартная функция C. Она аккуратно закрывает программу: вызывает деструкторы глобальных объектов, выполняет обработчики, зарегистрированные через `atexit`, сбрасывает буферы всех потоков и только потом просит ядро завершить процесс. `_exit(N)` из `<unistd.h>` не делает ничего из этого — она немедленно уходит в ядро.

В ребёнке после неудачного `exec` правильный выбор — `_exit`. Причина в том, что ребёнок — точная копия родителя, и все глобальные объекты родителя существуют в нём в виде копий. Позвать деструкторы этих копий значит выполнить чужую логику завершения дважды: закрыть файл, который в родителе ещё нужен, дописать хвост в лог, отправить прощальный пакет по сети. Ничего хорошего из этого не выйдет.

Расплата за `_exit` — несброшенные буферы. Именно поэтому в примере `cow.cpp` перед `_exit(0)` стоит `std::cout.flush()`: без него строка ребёнка просто пропадёт, если вывод перенаправлен в файл. Я это проверил, убрав `flush`, — строка исчезает бесследно. В коде оболочки мы будем писать сообщения об ошибках в `std::cerr`, который не буферизуется вовсе, но привычку звать `flush` перед `_exit` лучше завести сразу.

27.5.3 Канонический шаблон

Соберём два шага вместе. Вот скелет, который в том или ином виде есть в каждой Unix-программе, запускающей другую:

```
pid_t pid = fork();
if (pid < 0) {
    // не удалось создать процесс — обрабатываем ошибку
} else if (pid == 0) {
    // мы ребёнок
    execvp(argv[0], argv);
    // сюда попадаем ТОЛЬКО при ошибке
    _exit(127);
} else {
    // мы родитель: pid — номер ребёнка
    waitpid(pid, &status, 0);
}
```

Fork — сделали заготовку. Exec — превратили заготовку в нужную программу. Wait — дождались результата. Windows делает всё то же самое одним вызовом CreateProcess, и на первый взгляд это выглядит разумнее. Но у разделения на два шага есть большое преимущество: между fork и exec ребёнок — ещё наш собственный код, и мы можем настроить всё что угодно перед запуском чужой программы. Перенаправить его вывод в файл или в канал, сменить рабочую папку, поменять переменные окружения, сбросить обработчики сигналов. Именно в этот промежуток мы вставим dup2 в главе 27 — и получим пайпы.

27.6 wait — забрать результат

Осталось третье действие. Когда ребёнок завершился, ядро не выбрасывает его учётную запись сразу: там лежит код возврата, который ещё никто не забрал. Забирает его родитель вызовом wait или waitpid из <sys/wait.h>.

wait(&status) ждёт любого ребёнка. waitpid(pid, &status, flags) гибче: в первом аргументе можно указать конкретный PID, -1 для «любого», 0 для «любого из моей группы процессов». Третий аргумент — флаги; 0 означает «блокировать до завершения», а WNOHANG — «не блокировать, просто посмотреть»; во втором случае функция вернёт 0, если никто ещё не готов. Возвращает waitpid номер завершившегося ребёнка или -1 при ошибке.

Самое интересное — второй аргумент. status — это не код возврата, а упакованное в одно int описание того, как процесс закончил жизнь. Распаковывают его макросами.

Напишем программу, которая запускает разные команды и показывает сырое число рядом с расшифровкой. Файл `waitstatus.cpp`:

```
#include <iostream>
#include <string>
#include <vector>
#include <sys/wait.h>
#include <unistd.h>

int spawn(const std::vector<std::string>& args) {
    std::vector<char*> c_argv;
    for (std::size_t i = 0; i < args.size(); ++i) {
        c_argv.push_back(const_cast<char*>(args[i].c_str()));
    }
    c_argv.push_back(nullptr);

    pid_t pid = fork();
    if (pid == 0) {
        execvp(c_argv[0], c_argv.data());
        _exit(127);
    }

    int status = 0;
    waitpid(pid, &status, 0);

    std::cout << args[0] << ": raw status = " << status << " -> ";
    if (WIFEXITED(status)) {
        std::cout << "завершился сам, код " << WEXITSTATUS(status) << "\n";
    } else if (WIFSIGNALED(status)) {
        std::cout << "убит сигналом " << WTERMSIG(status) << "\n";
    }
    return status;
}

int main() {
    spawn({"true"});
    spawn({"false"});
    spawn({"sh", "-c", "exit 42"});
    spawn({"sh", "-c", "kill -SEGV $$"});
}
```

```
spawn({"sh", "-c", "kill -9 $$"});  
return 0;  
}
```

Вывод — настоящий, с моей машины:

```
true: raw status = 0 -> завершился сам, код 0  
false: raw status = 256 -> завершился сам, код 1  
sh: raw status = 10752 -> завершился сам, код 42  
sh: raw status = 11 -> убит сигналом 11  
sh: raw status = 9 -> убит сигналом 9
```

Числа объясняют устройство упаковки лучше любой документации. Код возврата 1 дал status 256, то есть 1×256 . Код 42 дал $10752 = 42 \times 256$. Код возврата хранится в старшем байте, младший байт при этом нулевой. А если процесс убит сигналом, номер сигнала пишется в младший байт: SIGSEGV (11) дал ровно 11, SIGKILL (9) — ровно 9.

Отсюда понятно, что делают макросы. `WIFEXITED(status)` проверяет, что младший байт нулевой, — значит, процесс завершился сам. `WEXITSTATUS(status)` достаёт старший байт, то есть делит на 256. `WIFSIGNALED(status)` проверяет обратное, а `WTERMSIG(status)` возвращает младший байт. Есть ещё `WIFSTOPPED` и `WSTOPSIG` — для приостановленных процессов, они нужны при отладке и при управлении заданиями (Ctrl+Z), но нам в этой части не понадобятся.

Пользоваться макросами обязательно. Писать `status / 256` вместо `WEXITSTATUS(status)` нельзя — раскладка байтов не гарантирована стандартом, а на некоторых системах она действительно другая. Макросы прячут эту разницу.

27.7 Зомби и сироты

Раз код возврата хранится до тех пор, пока его не заберут, возникает вопрос: что если родитель его так и не заберёт?

Тогда ребёнок становится зомби. Он уже не выполняется, памяти у него нет, но запись в таблице процессов и его PID остаются занятыми — ядро держит их ради кода возврата, который кто-то, возможно, ещё спросит. Посмотрим. Файл `zombie.cpp`:

```
#include <iostream>  
#include <unistd.h>  
  
int main() {  
    pid_t pid = fork();  
    if (pid == 0) {
```



```

    _exit(0);                // ребёнок умирает сразу
}
std::cout << "ребёнок " << pid << " уже мёртв, но я его не жду\n";
std::cout.flush();
sleep(30);                  // родитель занят чем-то другим
return 0;
}

```

Запускаем в фоне и смотрим на процессы:

```

$ ./zombie &
ребёнок 49787 уже мёртв, но я его не жду
$ ps -o pid,ppid,stat,command -ax | grep 49785
49785      1 SN    ./zombie
49787 49785 ZN    <defunct>

```

Вот он: PID 49787, состояние Z, команда <defunct> — «покойник». Он висит и будет висеть, пока жив родитель. Как только родитель завершится (через тридцать секунд), зомби усыновит процесс 1, тот немедленно заберёт код возврата, и запись исчезнет.

Один зомби безвреден. Проблема начинается, когда программа порождает детей в цикле и не забирает их: PID — ресурс конечный, на Linux по умолчанию их около 32 тысяч, и долгоживущий сервер с такой ошибкой рано или поздно упрётся в потолок и не сможет создать новый процесс. Отсюда правило: **каждому fork — свой waitpid**. В нашей оболочке это выполняется буквально: мы ждём ребёнка сразу после запуска.

Сирота — обратная ситуация: ребёнок жив, а родитель умер раньше. Мы её уже видели в самом первом примере, когда ребёнок напечатал «родитель 1». Сироту усыновляет процесс с PID 1, и он же потом заберёт её код возврата. Никакой утечки тут нет, и сирота — состояние совершенно нормальное; на нём, например, построен классический способ запустить демона.

27.8 Собираем myshell v1

Теории достаточно. Создаём проект.

```

$ mkdir -p demo-shell/src demo-shell/include demo-shell/build
$ cd demo-shell

```

Makefile берём тот же, что служил нам в demo-rpg, — с автоматическим отслеживанием зависимостей через `-MMD` и с `wildcard`, чтобы новые файлы подхватывались без правок:

```
CXX      := g++
CXXSTD   := -std=c++11
WARNINGS := -Wall -Wextra -Wpedantic -Wshadow
DEBUG    := -g -O0
RELEASE  := -O2 -DNDEBUG

MODE     ?= debug
ifeq ($(MODE),release)
    OPT := $(RELEASE)
else
    OPT := $(DEBUG)
endif

CXXFLAGS := $(CXXSTD) $(WARNINGS) $(OPT) -Iinclude -MMD -MP

SRCS      := $(filter-out src/main.cpp, $(wildcard src/*.cpp))
OBJS      := $(patsubst src/%.cpp, build/%.o, $(SRCS))
DEPS      := $(OBJS:.o=.d)

all: build/myshell

build/myshell: build/main.o $(OBJS)
    @mkdir -p build
    $(CXX) $(CXXFLAGS) $^ -o $@
    @echo "Built: $@ ($(MODE))"

build/%.o: src/%.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@

build/main.o: src/main.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@

run: build/myshell
    ./build/myshell

clean:
```

```
rm -rf build

-include $(DEPS)

.PHONY: all run clean
```

main.cpp вынесен из SRCS через filter-out и собирается отдельным правилом — так позже будет проще подключить тесты, которым нужны все объектные файлы, кроме main.o с его функцией main. В главе 31 мы добавим в этот файл ещё правила для утилит из utils/, а пока хватит и такого.

Теперь заголовок. include/exec_runner.h:

```
#ifndef SHELL_EXEC_RUNNER_H
#define SHELL_EXEC_RUNNER_H

#include <string>
#include <vector>

namespace shell {

// Разбить строку на слова по пробелам. Кавычки и escape пока не понимаем —
// это глава 29.
std::vector<std::string> tokenize(const std::string& line);

// Запустить внешнюю программу и дождаться её. Возвращает код завершения
// (или 128 + номер сигнала, если ребёнка убили).
int run_external(const std::vector<std::string>& argv);

} // namespace shell

#endif
```

Всё содержимое проекта живёт в пространстве имён shell — та же договорённость, что была с rpg:: в Части II. Include guard в стиле SHELL_EXEC_RUNNER_H — из главы 7.

Реализация, src/exec_runner.cpp:

```
#include "exec_runner.h"

#include <cerrno>
```

```
#include <cstring>
#include <iostream>
#include <sstream>

#include <sys/wait.h>
#include <unistd.h>

namespace shell {

std::vector<std::string> tokenize(const std::string& line) {
    std::vector<std::string> tokens;
    std::istringstream ss(line);
    std::string word;
    while (ss >> word) {
        tokens.push_back(word);
    }
    return tokens;
}

int run_external(const std::vector<std::string>& argv) {
    if (argv.empty()) return 0;

    std::vector<char*> c_argv;
    c_argv.reserve(argv.size() + 1);
    for (std::size_t i = 0; i < argv.size(); ++i) {
        c_argv.push_back(const_cast<char*>(argv[i].c_str()));
    }
    c_argv.push_back(nullptr);

    pid_t pid = fork();
    if (pid < 0) {
        std::cerr << "fork: " << std::strerror(errno) << "\n";
        return -1;
    }

    if (pid == 0) {
        execvp(c_argv[0], c_argv.data());
        std::cerr << "myshell: " << argv[0] << ": "
```

```

        << std::strerror(errno) << "\n";
    std::cerr.flush();
    _exit(127);
}

int status = 0;
if (waitpid(pid, &status, 0) < 0) {
    std::cerr << "waitpid: " << std::strerror(errno) << "\n";
    return -1;
}

if (WIFEXITED(status)) return WEXITSTATUS(status);
if (WIFSIGNALED(status)) {
    std::cerr << "[прерван сигналом " << WTERMSIG(status) << "]\n";
    return 128 + WTERMSIG(status);
}
return -1;
}

} // namespace shell

```

Пройдёмся по коду.

tokenize пока предельно наивна. std::istringstream из главы 12 читает из строки как из потока, а оператор >> для std::string сам пропускает пробелы и табуляции и отдаёт слово целиком. Четыре строки — и разбиение по пробелам готово. Кавычки, конечно, не работают, и очень скоро мы в этом убедимся.

Дальше готовим аргументы для execvp. Наши строки лежат в std::vector<std::string>, а системному вызову нужен массив указателей на char, оканчивающийся нулевым указателем. Собираем его вручную: reserve на один элемент больше — под завершающий nullptr, чтобы вектор не перевыделялся; const_cast<char*>(argv[i].c_str()) снимает const, потому что подпись execvp досталась нам из семидесятых. Строки при этом никто не меняет, так что приведение безопасно. А вот c_argv.push_back(nullptr) в конце — не формальность: без завершающего нуля execvp пойдёт читать память за границей массива, и это неопределённое поведение в чистом виде.

Обратите внимание на порядок: весь этот массив мы строим **до** fork. Так короче путь между fork и exec, а чем короче этот путь, тем меньше шансов сделать в ребёнке что-нибудь опасное. Дело в том, что в ребёнке безопасно вызывать далеко не всё: malloc, а с ним и любое

выделение памяти в `std::string` или `std::vector`, может оказаться заблокированным, если в момент `fork` другой поток держал внутренний замок аллокатора. В однопоточной программе это не страшно, но привычку готовить всё заранее стоит завести сразу.

В ветке ребёнка после `execvp` идёт сообщение об ошибке в формате, принятом в Unix: имя программы, двоеточие, имя команды, двоеточие, причина. Именно так пишут `bash` и `ls`, и такой формат легко разбирать глазами в длинном логге. `std::cerr` не буферизуется, но `flush` перед `_exit` я оставил намеренно — как напоминание о правиле.

В ветке родителя — `waitpid` с флагом `0`, то есть блокирующий. Оболочка засыпает, пока ребёнок работает, и просыпается, когда он закончил. Далее разбираем статус: нормальное завершение отдаём как есть, а убитого сигналом превращаем в `128 + номер`. Это соглашение всех оболочек: `SIGINT` (2) даёт 130, `SIGKILL` (9) — 137, `SIGSEGV` (11) — 139. Увидев в терминале код 139, опытный человек сразу говорит «сегфолт», не заглядывая в документацию.

Наконец, `src/main.cpp` с циклом чтения команд:

```
#include "exec_runner.h"

#include <iostream>
#include <string>

int main() {
    std::cout << "=== myshell v1 ===\n";

    while (true) {
        std::cout << "myshell$ ";
        std::cout.flush();

        std::string line;
        if (!std::getline(std::cin, line)) {
            std::cout << "\n";
            break;
        }

        std::vector<std::string> tokens = shell::tokenize(line);
        if (tokens.empty()) continue;
        if (tokens[0] == "exit" || tokens[0] == "quit") break;

        int rc = shell::run_external(tokens);
        if (rc != 0) {
```

```

        std::cerr << "[exit " << rc << "]\n";
    }
}
return 0;
}

```

Это REPL — read-eval-print loop, тот же цикл, что крутился в `Game::step` из главы 15, только вместо игровых команд здесь программы.

`std::cout.flush()` после приглашения — обязателен. Приглашение `myshell$` не заканчивается переводом строки, а значит, при построчной буферизации останется в буфере, и вы увидите пустой экран вместо приглашения. Программа при этом будет исправно ждать ввода, что выглядит как зависание. Это, пожалуй, самая частая ошибка в самодельных оболочках.

Проверка `if (!std::getline(...))` ловит конец ввода. В терминале это `Ctrl+D`, при запуске из скрипта — конец файла. Без этой проверки при `Ctrl+D` цикл завертится вхолостую на бесконечном чтении пустых строк.

Собираем и пробуем:

```

$ make
g++ -std=c++11 -Wall -Wextra -Wpedantic -Wshadow -g -O0 -Iinclude -MMD -MP -c src/main.cpp -o build/main.o
g++ -std=c++11 -Wall -Wextra -Wpedantic -Wshadow -g -O0 -Iinclude -MMD -MP -c src/exec_runner.cpp -o build/exec_runner.o
g++ ... build/main.o build/exec_runner.o -o build/myshell
Built: build/myshell (debug)

$ ./build/myshell
=== myshell v1 ===
myshell$ echo привет из myshell
привет из myshell
myshell$ pwd
/Users/serik/Desktop/From-scratch/cpp-from-scratch/demo-shell
myshell$ ls Makefile
Makefile
myshell$ date
Thu Jul 30 13:35:54 +05 2026
myshell$ nosuchcmd
myshell: nosuchcmd: No such file or directory
[exit 127]
myshell$ exit

```

Сто с небольшим строк кода — и у нас настоящая оболочка. Она запускает любую программу, видимую в `$PATH`, показывает её вывод, сообщает про ненулевые коды возврата. Базовый цикл здесь ровно тот же, что в `bash`: прочитать строку, разбить, `fork`, `exec`, `wait`.

27.9 Что уже сломано

Похвалили — теперь ломаем. Наивный `tokenize` не понимает кавычек, и это видно сразу:

```
myshell$ sh -c "exit 42"
42": -c: line 0: unexpected EOF while looking for matching `"'
42": -c: line 1: syntax error: unexpected end of file
[exit 2]
```

Строка развалилась на слова `sh`, `-c`, `"exit, 42"` — кавычки остались внутри слов, а пробел между ними разрезал аргумент пополам. `sh` получил на вход мусор и честно пожаловался. Починим это в главе 29, когда напишем настоящий токенизатор с состояниями.

Вторая заметная дыра — `cd`. Попробуйте:

```
myshell$ cd /tmp
myshell$ pwd
/Users/serik/Desktop/From-scratch/cpp-from-scratch/demo-shell
```

Папка не сменилась. На `macOS` ошибки при этом нет вовсе — там существует файл `/usr/bin/cd`, который честно запускается, честно ничего полезного не делает и возвращает ноль. На `Linux` такого файла обычно нет, и вы увидите `myshell: cd: No such file or directory`. Причина в обоих случаях одна: `cd` выполняется в **ребёнке**, меняет рабочую папку ребёнку, после чего ребёнок умирает и уносит изменение с собой. Рабочая папка — свойство процесса, и поменять её у родителя, запустив отдельную программу, физически невозможно. Поэтому `cd` во всех оболочках мира — встроенная команда, выполняемая без `fork`. Этим займёмся в главе 30.

Третья дыра — `Ctrl+C`. Запустите `sleep 30` и нажмите `Ctrl+C`: умрёт не только `sleep`, но и сама оболочка, потому что сигнал приходит всей группе процессов сразу. Настоящая оболочка сигналы у себя отключает и раздаёт детям — это глава 28.

И, конечно, нет ни пайпов (`ls | wc -l`), ни перенаправлений (`> out.txt`), ни истории команд. Всё это мы добавим, и в такой последовательности: каналы и перенаправления в главе 27, сигналы в 28, полноценный парсер в 29, встроенные команды и история в 30, свои утилиты в 31.

27.10 Как v1 превратится в финальный код

Чтобы вы не потерялись, когда начнёте сверяться с готовым `demo-shell/` из репозитория, вот карта превращений.

Функция `run_external` — временная. В главе 27 её место займёт `run_pipeline` из `src/pipeline.cpp`, которая умеет запускать цепочку команд, а не одну; одиночная команда для неё — частный случай цепочки длиной один. Поэтому в финальном коде `run_external` вы не найдёте, и это не ошибка книги, а нормальный ход эволюции проекта.

Функция `tokenize` останется на своём месте — в `exec_runner.cpp`, — но полностью изменится внутри: вместо четырёх строк с `istringstream` там будет конечный автомат на сотню строк, понимающий кавычки, `escape`-последовательности и специальные токены. Заголовок `exec_runner.h` в итоге объявляет только `tokenize`.

Файл `main.cpp` обрастёт: в него добавятся установка обработчиков сигналов, загрузка и сохранение истории, попытка выполнить встроенную команду перед запуском внешней. Каркас цикла — приглашение, `getline`, проверка на `exit` — доживёт до конца книги без изменений.

Такой способ работы — обычная практика. Никто не пишет сразу финальную версию; пишут простую, упираются в её ограничения и переделывают именно то место, которое мешает. Мы будем идти этим путём всю Часть III, и каждая глава начинается с того, что предыдущая версия перестала справляться.

27.11 Чем смотреть внутрь

Пара инструментов, которые пригодятся, если что-то пойдёт не так.

`ps -o pid,ppid,stat,command -ax` показывает все процессы вместе с родителями и состоянием. Колонка `STAT`: `R` — выполняется, `S` — спит, `Z` — зомби, `T` — остановлен. Именно так мы поймали зомби несколько страниц назад.

Трассировка системных вызовов позволяет увидеть глазами, что программа просит у ядра. На Linux это `strace ./build/myshell`, на macOS — `sudo dtruss ./build/myshell` (нужны права администратора, а для системных бинарников ещё и отключённая защита SIP). Запустите оболочку под трассировкой, введите `ls` — и увидите ровно то, о чём говорилось в главе: `fork`, затем `execve`, затем `wait4`. Очень отрезвляющее зрелище: никакой магии, три вызова.

`echo $?` в вашей обычной оболочке печатает код возврата последней команды — удобно, чтобы проверить, что наш `myshell` возвращает то, что обещает.

27.12 Главные правила главы

1. fork создаёт копию процесса, exec заменяет содержимое процесса другой программой. Запуск внешней команды — это всегда пара, а не одно действие.
2. fork возвращает 0 ребёнку, PID ребёнка родителю и -1 при ошибке. Проверять надо все три случая.
3. Перед fork сбрасывайте буферы вывода, иначе строки продублируются.
4. exec не возвращается. Код после него выполняется только при ошибке.
5. В ребёнке на пути ошибки — _exit, не exit: деструкторы копий родительских объектов трогать нельзя.
6. Массив аргументов для exec обязан заканчиваться nullptr.
7. На каждый fork должен приходиться waitpid, иначе накапливаются зомби.
8. Статус из waitpid разбирают только макросами WIFEXITED/WEXITSTATUS/WIFSIGNALED/WTERMSIG, а не арифметикой.

27.13 Упражнения

1. Соберите demo-shell и поиграйте с ним: ls, pwd, whoami, date, echo hello, head -3 Makefile. Убедитесь, что вывод программ попадает прямо в терминал без всяких усилий с нашей стороны, и объясните, почему так происходит.
2. Запустите ./build/myshell > log.txt, введите пару команд и посмотрите файл. Что попало в log.txt, а что осталось на экране? Объясните, куда делось приглашение и почему сообщения об ошибках видны на экране.
3. Уберите std::cout.flush() после приглашения, пересоберите, запустите. Опишите, как выглядит поломка. Теперь верните flush и вместо него попробуйте std::endl — что изменилось и почему?
4. Запустите sh -c "kill -9 \$\$" в нашей оболочке. Какой код вы увидите в [exit ...]? Сойдётся ли он с формулой 128 + номер сигнала?
5. Напишите программу из двадцати строк, которая делает fork дважды подряд — так, чтобы получилось три процесса: родитель и два ребёнка. Каждый печатает свой PID и PPID, родитель дважды вызывает waitpid(-1, &status, 0). Запустите несколько раз и объясните, почему порядок строк меняется.
6. Воспроизведите зомби: fork, ребёнок сразу _exit(0), родитель sleep(60). Найдите зомби в ps. Теперь добавьте waitpid — и убедитесь, что зомби исчез. Сколько времени зомби живёт после смерти родителя?

7. Добавьте в `run_external` измерение времени: запомните `std::chrono::steady_clock::now()` до `fork`, посчитайте разницу после `waitpid` и печатайте `[0.123s]` после каждой команды. Сравните время `true` и `sleep 1`.
8. (Сложнее) Прочитайте `man 2 fork`, `man 3 execvp` и `man 2 waitpid`. Найдите в описании `waitpid` флаг `WUNTRACED` и объясните, зачем он нужен. Затем добавьте в `run_external` обработку `WIFSTOPPED` и проверьте её, нажав `Ctrl+Z` во время работы `sleep 30`.

27.14 Что дальше

Глава 27 — каналы и перенаправления. Разберёмся, что такое `pipe()`, как `dup2` подменяет файловые дескрипторы и почему `ls | grep txt | wc -l` — это три процесса, соединённые двумя каналами, а не одна хитрая функция. Заодно поймём, откуда взялась вся философия Unix с маленькими программами, которые делают одно дело и умеют соединяться в цепочки.

Глава 28

Глава 27. Файловые дескрипторы и каналы

В прошлой главе оболочка научилась запускать одну программу. В этой она научится соединять программы в цепочку: `ls | grep .cpp | wc -l`. Три процесса, два канала, один результат.

Это, пожалуй, самая красивая идея Unix. Вместо одной огромной программы, умеющей всё, — набор маленьких, каждая из которых делает ровно одно дело и умеет читать со стандартного входа и писать в стандартный выход. Соединяются они не хитрым API, а вертикальной чертой. `wc` ничего не знает про `ls`, `grep` ничего не знает ни про того, ни про другого, но вместе они считают строки в отфильтрованном списке файлов.

Реализуется всё это на удивление небольшим количеством кода, но требует дисциплины: один незакрытый дескриптор — и цепочка зависает намертво. Мы такое зависание сейчас специально устроим, чтобы вы увидели его своими глазами и потом узнавали в собственном коде.

28.1 Что такое файловый дескриптор

В Unix почти всё выглядит как файл. Дисковый файл, терминал, сетевое соединение, канал, устройство `/dev/null` — со всем этим работают через один и тот же набор действий: открыть, прочитать, записать, закрыть. Разница спрятана внутри ядра.

Открытая сущность в программе представлена файловым дескриптором — маленьким неотрицательным целым числом. Это индекс в личной таблице процесса, а уже запись в таблице ведёт к настоящему объекту внутри ядра.

Три первых индекса заняты с самого запуска:

Дескриптор	Имя	Макрос	Назначение
0	stdin	STDIN_FILENO	стандартный ввод
1	stdout	STDOUT_FILENO	стандартный вывод
2	stderr	STDERR_FILENO	поток ошибок

Когда программу запускают из терминала, все три связаны с терминалом. Когда вы пишете `./program > out.txt`, оболочка перед запуском подменяет дескриптор 1 на файл — и программа даже не подозревает об этом. Она как писала в дескриптор 1, так и пишет.

Посмотрим, как раздаются номера. Файл `fds.cpp`:

```
#include <iostream>
#include <fcntl.h>
#include <unistd.h>

int main() {
    std::cout << "STDIN_FILENO = " << STDIN_FILENO << "\n"
              << "STDOUT_FILENO = " << STDOUT_FILENO << "\n"
              << "STDERR_FILENO = " << STDERR_FILENO << "\n";

    int a = open("/etc/hosts", O_RDONLY);
    int b = open("/etc/hosts", O_RDONLY);
    std::cout << "первый open вернул " << a << ", второй " << b << "\n";

    close(a);
    int c = open("/etc/hosts", O_RDONLY);
    std::cout << "после close(" << a << ") новый open вернул " << c << "\n";
    return 0;
}
```

```
$ g++ -std=c++11 -Wall -Wextra -pedantic fds.cpp -o fds && ./fds
```

```
STDIN_FILENO = 0
```

```
STDOUT_FILENO = 1
```

```
STDERR_FILENO = 2
```

```
первый open вернул 3, второй 4
```

```
после close(3) новый open вернул 3
```

Правило простое и жёстко зафиксированное в стандарте: `open` всегда возвращает **наименьший свободный номер**. Освободили тройку — следующий `open` снова получит тройку. Это не

любопытный факт, а рабочий инструмент: старый способ перенаправить вывод состоял в том, чтобы закрыть дескриптор 1 и сразу открыть файл — он неизбежно занимал освободившееся первое место. Мы будем делать то же самое явно, через `dup2`, потому что так понятнее и безопаснее.

28.1.1 Сырые чтение и запись

В главе 12 мы работали с файлами через `<fstream>` — и в обычном коде так и надо делать. Но каналы живут на уровень ниже, поэтому здесь придётся звать системные вызовы напрямую:

```
int fd = open("data.txt", O_RDONLY);      // <fcntl.h>
char buf[100];
ssize_t n = read(fd, buf, sizeof(buf));   // <unistd.h>
close(fd);
```

`read(fd, buf, n)` пытается прочитать до `n` байт. Возвращает, сколько реально прочитал, `0` — если данные кончились (конец файла), `-1` — при ошибке. `write(fd, buf, n)` симметрична.

Слово «до» здесь принципиально. `read` имеет полное право вернуть меньше, чем запрошено, даже если данные ещё не кончились, — так бывает с терминалом, сетью и каналами постоянно. Точно так же `write` может записать не всё. Поэтому любой честный код читает и пишет в цикле, пока не наберёт нужное. В Части V мы напишем такой цикл для сокетов и будем разбирать его подробно; здесь нам хватит того, что `read` возвращает `0` ровно тогда, когда источник закончился.

Тип `ssize_t` — знаковый размер: он должен вмещать и обычные размеры, и `-1`.

28.2 Дескрипторы переживают `fork` и `exec`

Теперь ключевой факт, на котором держится вся глава. При `fork` таблица дескрипторов копируется в ребёнка — но копируются **номера**, а сами объекты внутри ядра остаются общими. Ребёнок и родитель смотрят на один и тот же открытый файл.

Насколько «один и тот же», показывает опыт. Файл `sharedfd.cpp`:

```
#include <iostream>
#include <fcntl.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    int fd = open("shared.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
pid_t pid = fork();
if (pid == 0) {
    write(fd, "ребёнок\n", 15);
    std::cout << "ребёнок: позиция после записи = "
               << lseek(fd, 0, SEEK_CUR) << "\n";
    std::cout.flush();
    _exit(0);
}
waitpid(pid, nullptr, 0);

write(fd, "родитель\n", 17);
std::cout << "родитель: позиция после записи = "
           << lseek(fd, 0, SEEK_CUR) << "\n";
close(fd);
return 0;
}
```

`lseek(fd, 0, SEEK_CUR)` ничего не сдвигает, а просто возвращает текущую позицию в файле. Смотрим:

```
$ ./sharedfd
ребёнок: позиция после записи = 15
родитель: позиция после записи = 32
$ cat shared.txt
ребёнок
родитель
```

Ребёнок записал 15 байт и сдвинул позицию на 15. Родитель начал писать не с нуля, а с пятнадцати — и получил 32. Позиция чтения-записи не принадлежит процессу; она хранится в объекте ядра, общем для обоих. Отсюда, кстати, понятно, почему `./prog >> log` из нескольких процессов не перемешивает строки как попало, а почему `>` из двух процессов затирает данные.

При `ехес` дескрипторы тоже сохраняются — если на них не выставлен флаг `FD_CLOEXEC` («закрывать при `ехес`»). Именно поэтому запущенный из оболочки `ls` пишет в тот же терминал: он унаследовал дескриптор 1, и никто ему ничего специально не передавал.

Из этих двух фактов складывается вся техника, которой посвящена глава. Оболочка готовит дескрипторы **до** `fork`, ребёнок их наследует, между `fork` и `ехес` подменяет ими свои 0 и 1 — и запускаемая программа работает с каналом, будучи уверенной, что общается с терминалом.

28.3 pipe — канал в ядре

Системный вызов `pipe` создаёт однонаправленную трубу: буфер внутри ядра с двумя концами. Принимает он массив из двух `int` и заполняет его: `fds[0]` — конец для чтения, `fds[1]` — конец для записи. Что записали в первый, прочитается из второго; порядок байтов сохраняется.

Запомнить, какой конец где, помогает аналогия со стандартными номерами: 0 — чтение, как `stdin`; 1 — запись, как `stdout`.

Файл `pipe1.cpp` — канал внутри одного процесса:

```
#include <iostream>
#include <string>
#include <unistd.h>

int main() {
    int fds[2];
    if (pipe(fds) < 0) { perror("pipe"); return 1; }
    std::cout << "pipe дал fd " << fds[0] << " (чтение) и "
               << fds[1] << " (запись)\n";

    const char* msg = "привет через ядро";
    ssize_t written = write(fds[1], msg, std::char_traits<char>::length(msg));

    char buf[64];
    ssize_t n = read(fds[0], buf, sizeof(buf));
    std::cout << "записано " << written << " байт, прочитано " << n
               << ": " << std::string(buf, static_cast<std::size_t>(n)) << "\n";

    close(fds[0]);
    close(fds[1]);
    return 0;
}
```

```
$ ./pipe1
pipe дал fd 3 (чтение) и 4 (запись)
записано 32 байт, прочитано 32: привет через ядро
```

Номера 3 и 4 — те самые «наименьшие свободные». А 32 байта вместо семнадцати символов — потому что строка русская: в UTF-8 каждая кириллическая буква занимает два байта. Ядро про буквы ничего не знает, оно считает байты.

Внутри одного процесса канал бесполезен: проще положить строку в переменную. Смысл появляется, когда концы оказываются в разных процессах, — а для этого достаточно сделать pipe до fork.

28.4 Канал между процессами и правило закрытия

Соберём канал между родителем и ребёнком — и заодно устроим то самое зависание. Файл hang.cpp, у него есть режим с закрытием пишущего конца и режим без:

```
#include <iostream>
#include <string>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char** argv) {
    bool close_write = (argc > 1 && std::string(argv[1]) == "close");

    int fds[2];
    pipe(fds);

    pid_t pid = fork();
    if (pid == 0) {
        close(fds[1]); // ребёнок-читатель: пишущий конец не нужен
        char buf[64];
        ssize_t n;
        while ((n = read(fds[0], buf, sizeof(buf))) > 0) {
            std::cout << "ребёнок прочитал " << n << " байт\n";
        }
        std::cout << "ребёнок: read вернул " << n << " — это EOF\n";
        std::cout.flush();
        _exit(0);
    }

    write(fds[1], "данные", 12);
    if (close_write) close(fds[1]); // <-- вот эта строка решает всё
    close(fds[0]);

    waitpid(pid, nullptr, 0);
    std::cout << "родитель: ребёнок завершился\n";
}
```

```
return 0;
}
```

Сначала правильный вариант:

```
$ ./hang close
ребёнок прочитал 12 байт
ребёнок: read вернул 0 — это EOF
родитель: ребёнок завершился
```

Теперь тот же бинарник без аргумента, то есть без `close(fds[1])`:

```
$ ./hang &
$ sleep 3; ps -o pid,ppid,stat,command -ax | grep hang
50443 50442 SN    ./hang
50445 50443 SN    ./hang
```

Оба процесса живы и будут жить вечно. Ребёнок сидит в `read` и ждёт данных, родитель сидит в `waitpid` и ждёт ребёнка. Классическая взаимная блокировка, и оба виноваты одинаково.

Разберём, почему так. Ядро считает ссылки на каждый конец канала. Ребёнок закрыл свою копию пишущего конца, но у родителя копия осталась открытой — значит, писатели ещё есть, значит, данные ещё могут прийти, значит, `read` обязан ждать. Конец файла на канале наступает только тогда, когда закрыты **все** копии пишущего конца во всех процессах.

Отсюда правило, которое стоит выучить наизусть: **каждый процесс закрывает все концы, которые ему не нужны, и делает это сразу**. Не «в конце функции», не «перед выходом», а немедленно после `fork`. Забытый конец не вызывает ошибки, не печатает предупреждения — он просто однажды подвешивает вашу программу, и искать причину вы будете долго.

Кстати, о том, как искать. Если что-то зависло, посмотрите, какие дескрипторы открыты у процесса: `ls -p PID` на macOS и Linux, `ls -l /proc/PID/fd` на Linux. Лишний FIFO или pipe в списке — и виновник найден.

28.4.1 Сколько влезает в канал

Канал — это буфер конечного размера. Что будет, если писать в него, а не читать? Проверим. Файл `pipesize.cpp` переводит пишущий конец в неблокирующий режим, чтобы вместо зависания получить ошибку:

```
#include <cerrno>
#include <cstring>
#include <iostream>
```

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fds[2];
    pipe(fds);

    int flags = fcntl(fds[1], F_GETFL, 0);
    fcntl(fds[1], F_SETFL, flags | O_NONBLOCK);

    char chunk[1024];
    std::memset(chunk, 'x', sizeof(chunk));

    long total = 0;
    while (true) {
        ssize_t n = write(fds[1], chunk, sizeof(chunk));
        if (n < 0) {
            std::cout << "write остановился на " << total << " байт, errno = "
                      << std::strerror(errno) << "\n";
            break;
        }
        total += n;
    }
    close(fds[0]);
    close(fds[1]);
    return 0;
}
```

```
$ ./pipesize
```

```
write остановился на 65536 байт, errno = Resource temporarily unavailable
```

64 килобайта — обычная ёмкость канала и на macOS, и на Linux. Без `O_NONBLOCK` вместо ошибки `EAGAIN` писатель просто заснул бы в `write`, пока читатель не разберёт накопившееся.

Это ограничение — не недостаток, а именно то, что делает конвейер конвейером. Когда вы пишете `cat huge.log | grep error`, файл не читается целиком в память: `cat` наполняет буфер, засыпает, `grep` разбирает, `cat` просыпается и пишет дальше. Два процесса работают одновременно, а памяти тратится 64 килобайта независимо от того, гигабайтный лог или

терабайтный. По той же причине `head -1` на огромном файле возвращается мгновенно — до конца никто не дочитывает.

28.4.2 Что будет, если читателя не стало

Обратная ситуация: писатель пишет в канал, у которого закрыт читающий конец. Так бывает постоянно — `yes | head -1` устроен ровно так, `head` берёт строку и уходит.

Файл `sigpipe.cpp`:

```
#include <cerrno>
#include <cstring>
#include <iostream>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    int fds[2];
    pipe(fds);

    pid_t pid = fork();
    if (pid == 0) {
        close(fds[1]);
        close(fds[0]);          // читатель сразу уходит и закрывает свой конец
        _exit(0);
    }

    close(fds[0]);
    sleep(1);                   // даём ребёнку время закрыться

    ssize_t n = write(fds[1], "кому это надо?", 26);
    std::cerr << "write вернул " << n << ", errno = " << std::strerror(errno) << "\n";
    waitpid(pid, nullptr, 0);
    return 0;
}
```

```
$ ./sigpipe
```

```
$ echo $?
```

```
141
```

Сообщение не напечаталось вовсе, а код возврата — 141. Узнаёте? По формуле из прошлой главы $141 = 128 + 13$, а сигнал 13 — это SIGPIPE. Ядро увидело запись в канал без читателей и убило писателя, не дав ему дойти до `std::cerr`.

Поведение это разумное: в конвейере из пяти команд бессмысленно, чтобы первая продолжала работать, когда последняя уже закончила. Но для сервера, который пишет клиенту в момент обрыва связи, смерть от SIGPIPE — катастрофа. Поэтому сигнал часто отключают. Добавим одну строку в начало `main`:

```
#include <csignal>
...
std::signal(SIGPIPE, SIG_IGN);
```

```
$ ./sigpipe2
write вернул -1, errno = Broken pipe
$ echo $?
0
```

Теперь вместо смерти — честная ошибка: `write` вернул `-1`, `errno` равен `PIPE`. Программа продолжила работу и завершилась нормально. Наша оболочка так и сделает, но уже в следующей главе, целиком посвящённой сигналам. В Части V, когда будем писать сервер чата, эта же проблема встретится снова — и там она куда острее.

28.5 dup2 — подменить номер

Осталась последняя деталь механизма. Мы умеем создавать канал, но как заставить чужую программу читать из него? Программа-то будет читать из дескриптора 0, ничего не зная про наш канал с номером 3.

Ответ — `dup2(oldfd, newfd)`. Этот вызов делает `newfd` вторым именем того же объекта ядра, на который смотрит `oldfd`. Если `newfd` был чем-то занят, он предварительно закрывается — молча и автоматически.

```
int file = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644); // допустим, дал 3
dup2(file, STDOUT_FILENO); // теперь дескриптор 1 ведёт в out.txt
close(file); // номер 3 больше не нужен, объект остаётся жив

std::cout << "эта строка окажется в файле\n";
```

Три строки, и весь вывод программы уходит в файл. Причём `std::cout` ничего не заметил: он как писал в дескриптор 1, так и пишет, просто теперь за этим номером стоит файл.

Порядок здесь важен. Сначала `dup2`, потом `close` — и ни в коем случае не наоборот: закрыв `file` до `dup2`, вы получите `EBADF` и молчаливую поломку. А закрывать `file` нужно потому, что после `dup2` объект ядра доступен через оба номера, и лишний номер — это ровно тот забытый дескриптор, из-за которого потом не приходит EOF.

`STDOUT_FILENO` — это просто макрос для 1, объявленный в `<unistd.h>`. Можно писать и цифру, но с макросом код читается лучше.

28.6 Цепочка из двух команд

Соберём `ls | grep .cpp` вручную, из уже знакомых деталей. Порядок действий такой: создать канал, породить первого ребёнка и подменить ему дескриптор 1 на пишущий конец, породить второго и подменить ему дескриптор 0 на читающий конец, закрыть все лишние копии везде, дожждаться обоих.

```
int fds[2];
pipe(fds);

pid_t p1 = fork();
if (p1 == 0) {                                // левая команда: ls
    dup2(fds[1], STDOUT_FILENO);              // вывод уходит в канал
    close(fds[0]);
    close(fds[1]);
    execvp("ls", argv_ls);
    _exit(127);
}

pid_t p2 = fork();
if (p2 == 0) {                                // правая команда: grep
    dup2(fds[0], STDIN_FILENO);               // ввод приходит из канала
    close(fds[0]);
    close(fds[1]);
    execvp("grep", argv_grep);
    _exit(127);
}

close(fds[0]);                                // родителю канал не нужен вовсе
close(fds[1]);
```

```
waitpid(p1, nullptr, 0);
waitpid(p2, nullptr, 0);
```

Обратите внимание на три места, где ошибиться проще всего.

Первое: `pipe` вызывается до обоих `fork`. Если создать канал после, ребёнок его не унаследует — дескрипторы копируются в момент раздвоения, и позже созданные объекты в чужую таблицу не попадают.

Второе: каждый ребёнок закрывает **оба** конца, включая тот, который только что продублировал. Нужный конец уже живёт под номером 0 или 1, копия под номером 3 или 4 не нужна и вредна.

Третье, и самое коварное: родитель закрывает оба конца **до** `waitpid`. Если этого не сделать, `grep` никогда не увидит конца данных — ведь в таблице родителя всё ещё открыт пишущий конец, — и вся конструкция повиснет ровно так, как мы уже видели на `hang.cpp`.

28.7 Цепочка из N команд

Обобщение прямолинейное. Для n команд нужно $n-1$ каналов: между первой и второй, между второй и третьей и так далее. Ребёнок с номером i берёт вход из канала $i-1$ (если он не первый) и отдаёт выход в канал i (если он не последний).

Для `ls | grep .cpp | wc -l` картина такая:

```

           канал 0           канал 1
ls  — write —> read — grep — write —> read — wc
(i=0)                (i=1)                (i=2)
```

```
i=0: stdout -> pipes[0][1]
```

```
i=1: stdin  <- pipes[0][0],  stdout -> pipes[1][1]
```

```
i=2: stdin  <- pipes[1][0]
```

Каждый ребёнок после двух возможных `dup2` закрывает вообще все концы всех каналов — и своих, и чужих. Чужие ему тем более не нужны, а держать их открытыми опасно: если, скажем, `wc` оставит открытым пишущий конец первого канала, то `grep` не дожждётся EOF.

28.8 Реализация в demo-shell

Теперь всё это надо уложить в наш проект. Начнём с типов. Новый файл `include/pipeline.h`:

```
#ifndef SHELL_PIPELINE_H
#define SHELL_PIPELINE_H
```

```

#include <string>
#include <vector>

namespace shell {

struct Command {
    std::vector<std::string> argv;
};

struct Pipeline {
    std::vector<Command> commands;

    bool empty() const { return commands.empty(); }
    std::size_t size() const { return commands.size(); }
};

// Разбирает строку в цепочку команд, разделённых '|'.
Pipeline parse_pipeline(const std::string& line);

// Запускает цепочку и возвращает код последней команды.
int run_pipeline(const Pipeline& p);

} // namespace shell

#endif

```

Command пока состоит из единственного поля, и такая структура выглядит избыточной — казалось бы, хватило бы `std::vector<std::vector<std::string>>`. Но в главе 29 у команды появятся ещё и перенаправления (`> out.txt, 2>&1`), и тогда структура окажется на своём месте. Заводить её сразу дешевле, чем потом переписывать сигнатуры.

Разбор строки, `src/pipeline.cpp`:

```

#include "pipeline.h"
#include "exec_runner.h"

#include <array>
#include <cerrno>
#include <cstring>

```



```
#include <iostream>
#include <stdexcept>
#include <vector>

#include <sys/wait.h>
#include <unistd.h>

namespace shell {

Pipeline parse_pipeline(const std::string& line) {
    Pipeline p;
    std::vector<std::string> tokens = tokenize(line);
    if (tokens.empty()) return p;

    Command current;
    for (std::size_t i = 0; i < tokens.size(); ++i) {
        if (tokens[i] == "|") {
            if (current.argv.empty()) {
                throw std::runtime_error("пустая команда перед '|');");
            }
            p.commands.push_back(std::move(current));
            current = Command();
        } else {
            current.argv.push_back(tokens[i]);
        }
    }

    if (current.argv.empty()) {
        throw std::runtime_error("пустая команда после '|');");
    }
    p.commands.push_back(std::move(current));
    return p;
}
```

Проходим по токенам и накапливаем слова в `current`, пока не встретится `|`; тогда накопленное уходит в список, а накопитель начинается заново. Две проверки на пустоту ловят `| ls` и `ls |` — команды без имени запускать нечего, и лучше сказать об этом сразу, чем получить непонятную ошибку из `execvp`. Ошибки мы бросаем исключением, а ловит его `main`; в главе

22 мы договорились, что исключения — для ситуаций, которые обнаружены глубоко, а обрабатываются высоко, и разбор командной строки подходит под это описание идеально.

Пока `tokenize` из прошлой главы режет строку по пробелам, `|` должен быть окружён пробелами: `ls|grep` пойдёт одним словом и не разберётся. В главе 29 токенизатор поумнеет и это ограничение исчезнет.

Дальше две вспомогательные функции. Обе нужны только внутри этого файла, поэтому прячем их в анонимное пространство имён — тот же приём, что в парсере мира из главы 23:

```
namespace {

void close_all_pipes(std::vector<std::array<int, 2>>& pipes) {
    for (std::size_t i = 0; i < pipes.size(); ++i) {
        if (pipes[i][0] >= 0) { close(pipes[i][0]); pipes[i][0] = -1; }
        if (pipes[i][1] >= 0) { close(pipes[i][1]); pipes[i][1] = -1; }
    }
}

std::vector<char*> to_c_argv(const std::vector<std::string>& args) {
    std::vector<char*> r;
    r.reserve(args.size() + 1);
    for (std::size_t i = 0; i < args.size(); ++i) {
        r.push_back(const_cast<char*>(args[i].c_str()));
    }
    r.push_back(nullptr);
    return r;
}

} // namespace
```

В `close_all_pipes` каждый закрытый дескриптор помечается как `-1`. Это защита от повторного закрытия: функция вызывается и в детях, и в родителе, а закрыть один и тот же номер дважды — ошибка не безобидная. Между двумя `close` ядро может выдать этот номер новому `open` в другом потоке, и второй `close` прикончит чужой файл. Такие ошибки ищут неделями, поэтому дешёвая пометка `-1` стоит своих двух строк.

Теперь главное — запуск:

```
int run_pipeline(const Pipeline& p) {
    if (p.empty()) return 0;
    const int n = static_cast<int>(p.commands.size());
```

```
std::vector<std::array<int, 2>> pipes(n > 0 ? n - 1 : 0);
for (int i = 0; i < n - 1; ++i) {
    if (pipe(pipes[i].data()) < 0) {
        std::cerr << "pipe failed: " << std::strerror(errno) << "\n";
        close_all_pipes(pipes);
        return -1;
    }
}

std::vector<pid_t> children(n);
for (int i = 0; i < n; ++i) {
    pid_t pid = fork();
    if (pid < 0) {
        std::cerr << "fork failed: " << std::strerror(errno) << "\n";
        close_all_pipes(pipes);
        return -1;
    }
    if (pid == 0) {
        if (i > 0) {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        if (i < n - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }
        close_all_pipes(pipes);

        std::vector<char*> argv = to_c_argv(p.commands[i].argv);
        execvp(argv[0], argv.data());
        std::cerr << "exec '" << argv[0] << "' failed: "
                    << std::strerror(errno) << "\n";
        _exit(127);
    }
    children[i] = pid;
}

close_all_pipes(pipes);
```

```
int last_status = 0;
for (int i = 0; i < n; ++i) {
    int status = 0;
    if (waitpid(children[i], &status, 0) < 0) {
        std::cerr << "waitpid: " << std::strerror(errno) << "\n";
        continue;
    }
    if (i == n - 1) {
        if (WIFEXITED(status)) last_status = WEXITSTATUS(status);
        else if (WIFSIGNALED(status)) last_status = 128 + WTERMSIG(status);
        else last_status = -1;
    }
}
return last_status;
}

} // namespace shell
```

Функция читается сверху вниз как список действий, но в ней есть несколько мест, о которых стоит сказать отдельно.

Каналы создаются все сразу, до первого `fork`. Иначе третий ребёнок не унаследовал бы второй канал.

`std::array<int, 2>` вместо `int[2]` — потому что массивы в стиле C нельзя хранить в `std::vector`, у них нет ни копирования, ни присваивания. `pipes[i].data()` отдаёт указатель на первый элемент, а `pipe` только этого и ждёт.

Ветка ребёнка — самая важная. Первый в цепочке не трогает свой ввод: он читает откуда читал, обычно с терминала. Последний не трогает вывод: пишет туда же, куда писала оболочка. Все промежуточные подменяют оба конца. Затем — обязательное `close_all_pipes()`, закрывающее в ребёнке вообще все дескрипторы каналов; нужные уже продублированы под номерами 0 и 1, а всё остальное — балласт, из-за которого потом не придёт EOF.

Родитель закрывает каналы сразу после цикла запуска, не дожидаясь никого. Тут легко ошибиться и написать `waitpid` внутри цикла — казалось бы, разница невелика. Разница огромна: пока родитель ждёт первого ребёнка, второй ещё не запущен, и первому некуда девать вывод; заполнив 64 килобайта, он заснёт навсегда. Именно поэтому сначала запускаем всех, потом закрываем свои копии концов, и только затем ждём.

Код возврата цепочки берётся у последней команды — так делает `bash`. `ls nosuchfile | wc -l` вернёт ноль, потому что `wc` отработал прекрасно, хотя `ls` и пожаловался. Поведение спорное, и в `bash` его можно изменить опцией `set -o pipefail`, после которой цепочка возвращает код первой упавшей команды. Наша оболочка следует поведению по умолчанию; сделать `pipefail` — хорошее упражнение в конце главы.

Заметьте: цепочка длиной один здесь тоже работает. Каналов создаётся ноль, `dup2` не вызывается ни разу, остаётся `fork + exec + wait` — ровно то, что делала `run_external` в прошлой главе. Поэтому `run_external` больше не нужна, и из `exec_runner.h` она уходит; в файле остаётся один `tokenize`.

Подключаем в `src/main.cpp` — меняется всего пара строк:

```
#include "pipeline.h"

...

shell::Pipeline p;
try {
    p = shell::parse_pipeline(line);
} catch (const std::exception& e) {
    std::cerr << "syntax: " << e.what() << "\n";
    continue;
}
if (p.empty()) continue;

const std::vector<std::string>& first = p.commands[0].argv;
if (!first.empty() && (first[0] == "exit" || first[0] == "quit")) {
    break;
}

int rc = shell::run_pipeline(p);
if (rc != 0) {
    std::cerr << "[exit " << rc << "]\n";
}
```

Проверка на `exit` теперь смотрит в первую команду цепочки, а не в первый токен строки. Разбор обёрнут в `try`, потому что `parse_pipeline` умеет бросать: синтаксическая ошибка не должна ронять оболочку, она должна печатать сообщение и возвращать приглашение.

28.9 Проверка

```
$ make
$ ./build/myshell
=== myshell ===
myshell$ ls -l src | wc -l
      5
myshell$ ls -l | sort | head -3
Makefile
build
include
myshell$ echo hello world | tr a-z A-Z
HELLO WORLD
myshell$ ls src | grep pipe
pipeline.cpp
myshell$ exit
```

Работает и на две команды, и на три. Обратите внимание на пробелы перед 5 — это `wc` выравнивает число по колонке, и наша оболочка тут ни при чём: она просто отдаёт байты как есть.

Стоит попробовать и то, что показывает потоковую природу канала:

```
myshell$ yes | head -3
y
y
y
```

`yes` печатает `y` бесконечно, `head` берёт три строки и закрывается. `yes` получает `SIGPIPE` и умирает — именно тот механизм, который мы разбирали. Без него команда работала бы вечно.

28.10 Главные правила главы

1. Дескриптор — это индекс в таблице процесса; 0, 1 и 2 заняты стандартными потоками, `open` всегда выдаёт наименьший свободный номер.
2. При `fork` дескрипторы копируются, но объект ядра остаётся общим — включая позицию в файле.
3. `pipe` вызывают до `fork`, иначе ребёнку нечего наследовать.

4. Конец данных на канале наступает, только когда закрыты все копии пишущего конца во всех процессах.
5. После `dup2` оригинальный дескриптор закрывают; в ребёнке закрывают вообще все концы каналов.
6. Родитель закрывает свои копии концов до `waitpid`, иначе цепочка виснет.
7. Запись в канал без читателей даёт `SIGPIPE` и код 141; с `SIG_IGN` — `write` возвращает `-1` и `ERPIPE`.
8. Код возврата цепочки берут у последней команды, если не включён `pipefail`.

28.11 Упражнения

1. Соберите оболочку и проверьте цепочки: `ls | wc`, `ls | grep .cpp | sort`, `cat Makefile | head -5 | tr a-z A-Z`. Затем `ls nosuchfile | wc -l` — объясните, почему нет `[exit 1]`.
2. Закомментируйте `close_all_pipes(pipes)` в ветке ребёнка, пересоберите и запустите `ls | wc -l`. Зависло? Верните строку на место, а вместо неё уберите `close_all_pipes` в родителе — зависнет ли теперь и почему?
3. Повторите опыт `sharedfd.cpp`, но уберите `waitpid` из родителя, чтобы записи шли одновременно. Запустите несколько раз. Что происходит с содержимым файла и почему позиция всё равно не «перезаписывается»?
4. Найдите ёмкость канала на своей машине через `pipesize.cpp`. Затем на Linux попробуйте изменить её вызовом `fcntl(fd, F_SETPIPE_SZ, 1048576)` и измерьте снова.
5. Напишите программу, которая соединяет каналом два **своих** куска кода без `exec`: родитель отправляет миллион строк, ребёнок читает и считает их. Замерьте время через `<chrono>` и посчитайте, сколько строк в секунду проходит через канал.
6. Добавьте в `run_pipeline` поведение `pipefail`: если любая команда цепочки завершилась ненулевым кодом, возвращать код самой левой упавшей. Проверьте на `ls nosuchfile | wc -l`.
7. (Сложнее) Сделайте так, чтобы оболочка печатала диагностику при `MYSHELL_DEBUG=1` в окружении: для каждого ребёнка — его индекс, PID и какие `dup2` были выполнены. Пригодится в следующих главах.
8. (Сложнее) Прочитайте `man 2 pipe2` и `man 2 fcntl` про `FD_CLOEXEC`. Объясните, почему флаг `O_CLOEXEC` не избавляет нас от `close_all_pipes` в ребёнке, хотя и выглядит похоже.

28.12 Что дальше

Глава 28 — сигналы. Разберёмся, что происходит при Ctrl+C, почему сейчас он убивает нашу оболочку вместе с запущенной программой, чем `sigaction` лучше `signal` и какие функции вообще можно звать внутри обработчика сигнала. Заодно наконец отключим SIGPIPE, о котором в этой главе шла речь.

Глава 29

Глава 28. Сигналы

У нашей оболочки есть неприятная особенность, которую вы наверняка уже заметили. Запустите `sleep 5` и нажмите `Ctrl+C` — умрёт не только `sleep`. Умрёт и сама оболочка, и вы окажетесь обратно в `zsh`.

Проверим это аккуратно, без терминала. Запустим версию из прошлой главы, дадим ей команду `sleep 5` и пошлём `SIGINT` всей группе процессов — ровно то, что делает терминал при `Ctrl+C`:

```
$ kill -INT -$(pgrep -f sleep)
$ ps -o pid,pgid,stat,command -ax | awk -v g="$(pgrep -f sleep)" '$2==g'
(пусто — процессов не осталось)
```

Ни оболочки, ни `sleep`. Всё правильно с точки зрения ядра и совершенно неправильно с точки зрения пользователя: прерывать он хотел запущенную программу, а не свой рабочий инструмент.

Эта глава — про сигналы: что это такое, как их посылать и принимать, почему обработчик сигнала — самая опасная функция в вашей программе, и как сделать так, чтобы `Ctrl+C` убивал только ребёнка. Заодно наконец разберёмся с `SIGPIPE`, о котором шла речь в главе 27.

29.1 Что такое сигнал

Сигнал — это короткое асинхронное уведомление, которое ядро доставляет процессу. Всё содержание уведомления — его номер. Никаких данных, никакого текста, никакой очереди сообщений: просто «тебе пришёл номер 2, реши, что с этим делать».

У каждого сигнала есть номер, имя вида SIGINT и действие по умолчанию — то, что произойдёт, если процесс никак сигнал не обрабатывает. Чаще всего действие по умолчанию — немедленная смерть.

Слово «асинхронно» здесь надо понимать буквально. Сигнал приходит не тогда, когда программа готова, а когда его послали: посреди вычисления, между двумя машинными инструкциями, в середине работы `std::cout`, внутри `malloc`. Ядро прерывает выполнение, вызывает обработчик, и после его возврата программа продолжает с того же места, будто ничего не было. Из этой особенности растут все сложности, о которых пойдёт речь дальше.

29.2 Словарь сигналов

Часто встречающиеся сигналы стоит знать наизусть. Обратите внимание на две колонки с номерами — они различаются:

Имя	Linux	macOS/BSD	По умолчанию	Когда приходит
SIGHUP	1	1	завершить	оборвалась связь с терминалом, закрыли SSH
SIGINT	2	2	завершить	Ctrl+C
SIGQUIT	3	3	завершить + core	Ctrl+\
SIGKILL	9	9	завершить, не перехватывается	kill -9
SIGSEGV	11	11	завершить + core	обращение по недопустимому адресу
SIGPIPE	13	13	завершить	запись в канал без читателей
SIGALRM	14	14	завершить	сработал alarm()
SIGTERM	15	15	завершить	kill без аргументов — вежливая просьба уйти
SIGSTOP	19	17	приостановить, не перехватывается	kill -STOP
SIGTSTP	20	18	приостановить	Ctrl+Z
SIGCONT	18	19	продолжить	fg, bg
SIGCHLD	17	20	игнорируется	ребёнок умер или остановлен

Первые полтора десятка номеров совпадают везде — они закреплены исторически. А дальше начинается разнობой: SIGCHLD на Linux имеет номер 17, а на macOS — 20, и на том же месте 17 у macOS стоит SIGSTOP. Мораль простая: **в коде пишите имена, а не числа**. `kill(pid, 9)` формально работает, но `kill(pid, SIGKILL)` работает всегда и читается лучше. Посмотреть таблицу своей системы можно командой `kill -l`.

Два сигнала — особые. SIGKILL и SIGSTOP невозможно ни перехватить, ни проигнорировать, ни заблокировать. Это страховка операционной системы: как бы ни был написан процесс, ядро гарантированно сможет его убить или остановить. Поэтому `kill -9` и работает там, где обычный `kill` бессилён, — но по той же причине процесс не получает шанса закрыть файлы и сохранить состояние.

Остальные сигналы процесс может обрабатывать тремя способами: оставить действие по умолчанию (SIG_DFL), полностью проигнорировать (SIG_IGN) или установить свою функцию-обработчик.

29.3 Как сигналы посылают

Из командной строки — утилитой `kill`, которая, вопреки названию, посылает любой сигнал, а не только смертельный:

```
$ kill -INT 1234      # SIGINT процессу 1234
$ kill -9 1234       # SIGKILL
$ kill 1234          # SIGTERM, если сигнал не указан
$ kill -INT -1234    # SIGINT всей группе процессов 1234
```

Последняя форма — с минусом перед номером — посылает сигнал не процессу, а целой группе. Именно так работает терминал.

Из программы — системным вызовом `kill` из `<csignal>`:

```
int rc = kill(pid, SIGTERM); // 0 при успехе, -1 при ошибке
```

Ошибка бывает, если процесса с таким номером нет (ESRCH) или если нет прав его трогать (EPERM) — послать сигнал чужому процессу может только его владелец или администратор.

Терминал посылает сигналы сам, когда вы нажимаете управляющие сочетания: Ctrl+C превращается в SIGINT, Ctrl+ — в SIGQUIT, Ctrl+Z — в SIGTSTP. Причём посылает он их не одному процессу, а всей группе переднего плана — той, что сейчас владеет терминалом. Когда вы в `bash` запускаете `sleep 10`, `bash` помещает `sleep` в отдельную группу и делает её группой переднего плана; сигнал от Ctrl+C уходит туда, а сам `bash` остаётся в стороне.

Наша оболочка так не умеет — она не создаёт группы и не передаёт владение терминалом. Поэтому и она, и её дети сидят в одной группе, и Ctrl+C накрывает всех. Полноценное управление группами (job control) — тема на отдельную главу, и в конце этой я расскажу, чего именно оно стоит. Обойдёмся более простым решением, которым, к слову, пользуются многие настоящие программы.

29.4 signal и почему им не пользуются

Старый способ установить обработчик — функция `signal`:

```
#include <csignal>

void handler(int sig) {
    // что-то делаем
}

int main() {
    std::signal(SIGINT, handler);
    ...
}
```

Выглядит просто, и в учебниках прошлого века только она и встречается. Проблема в том, что её поведение исторически различалось между ветками Unix. В System V после срабатывания обработчик автоматически сбрасывался в `SIG_DFL`, и программа должна была устанавливать его заново — а между сбросом и переустановкой оставалась щель, в которую мог влететь второй сигнал и убить процесс. В BSD обработчик оставался на месте. Современные системы следуют варианту BSD, но обещать это стандарт не берётся.

Вторая беда — отсутствие управления. Нельзя сказать, какие сигналы блокировать на время работы обработчика; нельзя выбрать, перезапускать ли прерванные системные вызовы. Всё это решили в `sigaction`, и именно её надо использовать в новом коде.

29.5 sigaction

Вызов принимает номер сигнала и структуру с описанием желаемого поведения:

```
#include <csignal>

void on_sigint(int sig) {
    ...
}
```

```
}

struct sigaction sa;
sa.sa_handler = on_sigint;    // что делать: функция, SIG_IGN или SIG_DFL
sigemptyset(&sa.sa_mask);    // какие сигналы блокировать внутри обработчика
sa.sa_flags = 0;             // флаги поведения

sigaction(SIGINT, &sa, nullptr);
```

Поле `sa_handler` — самое главное: туда кладут либо указатель на функцию с сигнатурой `void(int)`, либо `SIG_IGN`, чтобы сигнал игнорировался, либо `SIG_DFL`, чтобы вернуть действие по умолчанию.

Поле `sa_mask` — набор сигналов, которые ядро временно заблокирует на время выполнения обработчика. Заполнять его напрямую нельзя, для этого есть функции: `sigemptyset` очищает набор, `sigfillset` заполняет всеми сигналами, `sigaddset` добавляет один. Сам обрабатываемый сигнал блокируется всегда, независимо от маски, — так обработчик не может прерваться сам собой.

Поле `sa_flags` собирает модификаторы поведения; ниже разберём самый нужный из них, `SA_RESTART`.

Третий аргумент `sigaction` — место, куда сохранить прежнее поведение, если оно вам понадобится, чтобы потом восстановить. Нам не нужно, поэтому `nullptr`.

Забывать `sigemptyset` нельзя. Локальная структура не инициализируется, и в `sa_mask` окажется мусор со стека — ядро заблокирует случайный набор сигналов, и поведение программы станет невоспроизводимым.

29.6 Что можно делать в обработчике

Теперь самое важное, и здесь я буду настойчив, потому что ошибка тут дорогая и не воспроизводится в тестах.

Обработчик прерывает программу в произвольной точке. Представьте, что сигнал пришёл ровно в тот момент, когда `std::cout` дописывал байты в свой буфер и обновлял внутренний указатель. Обработчик вызывает `std::cout` снова — и второй вызов работает с наполовину обновлённой структурой. Или сигнал пришёл внутри `malloc`, когда список свободных блоков временно несогласован, а обработчик выделяет память. Результат — испорченные данные, зависание на внутреннем замке или падение, причём в одном запуске из тысячи.

POSIX перечисляет функции, которые безопасно звать из обработчика; их называют `asyn-signal-safe`. Список короткий и весь состоит из системных вызовов: `write`, `read`, `open`, `close`, `_exit`, `kill`, `getpid`, `sigaction`, `signal`, `waitpid` и ещё несколько десятков. Полный перечень — в `man 7 signal-safety` на Linux и `man 2 sigaction` на macOS.

Чего в списке нет: `printf`, `malloc`, `free`, `new`, `delete`, любые потоки C++, любые контейнеры, которые выделяют память. То есть почти всё, чем мы пользуемся каждый день.

Отсюда единственное разумное правило: **обработчик выставляет флаг и немедленно возвращается**. Вся работа делает основной код, который сам выбирает удобный момент.

```
volatile sig_atomic_t got_sigint = 0;

void on_sigint(int) {
    got_sigint = 1;           // только флаг, больше ничего
}
```

Тип `sig_atomic_t` — целое, чтение и запись которого происходят одной неделимой операцией; стандарт гарантирует, что вы не поймаете «половину» значения. Ключевое слово `volatile` запрещает компилятору кэшировать переменную в регистре: без него оптимизатор имеет полное право решить, что в цикле `while (!got_sigint) {}` переменная не меняется, и превратить цикл в бесконечный.

Соберём демонстрацию целиком. Файл `flag.cpp`:

```
#include <csignal>
#include <iostream>
#include <unistd.h>

volatile sig_atomic_t got_sigint = 0;

void on_sigint(int) {
    got_sigint = 1;
}

int main() {
    struct sigaction sa;
    sa.sa_handler = on_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, nullptr);
```

```

std::cout << "мой PID " << getpid() << "\n";
std::cout.flush();

for (int tick = 1; tick <= 5; ++tick) {
    unsigned left = sleep(1);           // sleep вернёт остаток, если его прервали
    if (got_sigint) {
        got_sigint = 0;
        std::cout << "тик " << tick << ": поймал SIGINT (sleep прерван, "
                    << "недоспал " << left << " с)\n";
    } else {
        std::cout << "тик " << tick << ": обычная работа\n";
    }
    std::cout.flush();
}
std::cout << "штатное завершение\n";
return 0;
}

```

Запускаем и посылаем два сигнала со стороны:

```

$ ./flag &
мой PID 50709
$ sleep 1.4; kill -INT 50709
$ sleep 1;   kill -INT 50709
тик 1: поймал SIGINT (sleep прерван, недоспал 1 с)
тик 2: обычная работа
тик 3: поймал SIGINT (sleep прерван, недоспал 1 с)
тик 4: обычная работа
тик 5: обычная работа
штатное завершение

```

Программа пережила оба сигнала, потому что действие по умолчанию заменено обработчиком. Печатает при этом не обработчик, а основной цикл — печатать из обработчика было бы той самой ошибкой.

Заодно видно ещё кое-что: `sleep(1)` вернул единицу, то есть был прерван, не доспав. Об этом — следующий раздел.

29.7 EINTR и SA_RESTART

Когда сигнал приходит во время блокирующего системного вызова — программа ждёт ввода в read, спит в sleep, сидит в waitpid, — ядро должно решить, что делать с этим вызовом. Вариантов два: прервать его, вернув -1 с errno равным EINTR, или после обработчика продолжить ожидание как ни в чём не бывало.

Выбирает поведение флаг SA_RESTART. Проверим на опыте. Файл `eintr.cpp` ставит будильник на секунду и пытается прочитать строку:

```
#include <cerrno>
#include <csignal>
#include <cstring>
#include <iostream>
#include <string>
#include <unistd.h>

void on_alarm(int) { }           // ничего не делаем, важен сам факт доставки

int main(int argc, char** argv) {
    bool restart = (argc > 1 && std::string(argv[1]) == "restart");

    struct sigaction sa;
    sa.sa_handler = on_alarm;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = restart ? SA_RESTART : 0;
    sigaction(SIGALRM, &sa, nullptr);

    alarm(1);                    // через секунду придёт SIGALRM

    char buf[64];
    ssize_t n = read(STDIN_FILENO, buf, sizeof(buf));
    if (n < 0) {
        std::cout << "read вернул -1, errno = " << std::strerror(errno)
                    << " (EINTR = " << EINTR << ", получили " << errno << ")\n";
    } else {
        std::cout << "read вернул " << n << " байт\n";
    }
    return 0;
}
```


Без флага, если ввода не давать вовсе:

```
$ sleep 3 | ./eintr
read вернул -1, errno = Interrupted system call (EINTR = 4, получили 4)
```

С флагом, если через две секунды подать строку:

```
$ (sleep 2; echo "текст") | ./eintr restart
read вернул 11 байт
```

В первом случае будильник разорвал чтение, и `read` вернулся ни с чем. Во втором сигнал пришёл точно так же, обработчик отработал, а `read` продолжил ждать и дождался — одиннадцать байт, потому что «текст» в UTF-8 занимает десять байт плюс перевод строки.

Практический вывод: если вы ставите обработчик и не хотите проверять `EINTR` в каждом вызове, ставьте `SA_RESTART`. Если хотите, чтобы сигнал прерывал ожидание (а иногда именно это и нужно — например, чтобы выйти из `read` по таймауту), не ставьте, но тогда обрабатывайте `EINTR` явно:

```
ssize_t n;
do {
    n = read(fd, buf, sizeof(buf));
} while (n < 0 && errno == EINTR);
```

Такой цикл — обычное дело в системном коде, и в Части V мы будем писать его вокруг сетевых вызовов регулярно.

29.8 Сигналы не копятя

Есть у сигналов свойство, которое регулярно ломает чужие ожидания: они не образуют очередь. Если процессу послали двадцать одинаковых сигналов, пока он был занят, он получит их не двадцать раз, а один.

Механика такая. У каждого процесса ядро держит набор ожидающих сигналов — именно набор, а не список: один бит на каждый номер. Пришёл `SIGUSR1` — бит поднялся. Пришёл ещё раз, пока первый не доставлен, — бит уже поднят, и поднимать нечего. Когда процесс в следующий раз возвращается из ядра в пользовательский код, ядро смотрит на биты и вызывает обработчики; после доставки бит гасится.

Убедимся. Файл `counting.cpp` считает срабатывания обработчика:

```
#include <csignal>
#include <iostream>
#include <unistd.h>
```

```
volatile sig_atomic_t count = 0;

void on_sigusr(int) { ++count; }

int main() {
    struct sigaction sa;
    sa.sa_handler = on_sigusr;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGUSR1, &sa, nullptr);

    std::cout << getpid() << "\n";
    std::cout.flush();

    sleep(3); // пока мы спим, в нас стреляют
    std::cout << "обработчик сработал " << count << " раз\n";
    return 0;
}
```

Пошлём один сигнал — обработчик срабатывает один раз, `sleep` при этом прерывается. Пошлём двадцать подряд, пока процесс спит, — увидите число заметно меньше двадцати, обычно единицу или двойку. Ничего не потерялось «из-за нагрузки»: так сигналы устроены.

Отсюда практическое следствие. Сигнал сообщает «событие произошло», а не «событие произошло N раз». Обработчик `SIGCHLD`, написанный как один `waitpid`, рано или поздно пропустит ребёнка — потому и нужен цикл с `WNOHANG`, о котором речь ниже. Реальный счётчик событий на сигналах не построишь.

29.9 Блокировка сигналов

Иногда сигнал нужно не проигнорировать, а отложить: например, программа обновляет структуру данных, к которой обработчик тоже обращается, и прерывание посередине оставит её в несогласованном виде. Для этого есть маска сигналов процесса и вызов `sigprocmask`.

```
sigset_t mask, old;
sigemptyset(&mask);
sigaddset(&mask, SIGUSR1);

sigprocmask(SIG_BLOCK, &mask, &old); // с этого момента SIGUSR1 не доставляется
```

```
// ... критическая секция ...
```

```
sigprocmask(SIG_SETMASK, &old, nullptr); // вернули прежнюю маску
```

Заблокированный сигнал не теряется — он остаётся ожидающим и доставляется сразу после разблокировки. Проверим оба свойства сразу. Файл `blocked.cpp` блокирует `SIGUSR1`, спит две секунды и печатает счётчик до и после разблокировки:

```
sigprocmask(SIG_BLOCK, &mask, &old);
```

```
std::cout << getpid() << "\n";
```

```
std::cout.flush();
```

```
sleep(2); // сигналы приходят, но не доставляются
```

```
std::cout << "внутри критической секции count = " << count << "\n";
```

```
sigprocmask(SIG_SETMASK, &old, nullptr); // разблокировали
```

```
std::cout << "сразу после разблокировки count = " << count << "\n";
```

Пока программа спала, я послал ей пять сигналов:

```
$ for i in $(seq 5); do kill -USR1 50971; done
```

```
внутри критической секции count = 0
```

```
сразу после разблокировки count = 1
```

Оба свойства как на ладони. Внутри критической секции обработчик не сработал ни разу — блокировка держит. А после разблокировки счётчик стал равен единице, а не пятёрке: пять сигналов слились в один ожидающий бит.

Восстанавливать маску лучше через `SIG_SETMASK` с сохранённым значением, а не через `SIG_UNBLOCK`. Разница видна во вложенных критических секциях: `SIG_UNBLOCK` снимет блокировку, даже если внешний код её тоже устанавливал, а `SIG_SETMASK` вернёт ровно то состояние, которое было до вашего вмешательства.

Есть и ещё одно применение маски: `sa_mask` в структуре `sigaction` — это временная блокировка на время работы обработчика. Если обработчик `SIGINT` трогает те же данные, что обработчик `SIGTERM`, добавьте `SIGTERM` в `sa_mask` — и они не перебьют друг друга.

29.10 Что наследуется через `fork` и `exec`

Для нашей задачи это самый важный раздел. Судьба настроек сигналов при `fork` и `exec` различается, и различается хитро:

Настройка	После fork	После exec
SIG_DFL	сохраняется	сохраняется
SIG_IGN	сохраняется	сохраняется
свой обработчик	сохраняется	сбрасывается в SIG_DFL

Логика понятна: после exec в процессе живёт другая программа, и адрес старой функции-обработчика в ней указывает в никуда — ядро обязано сбросить его. А вот SIG_IGN — не адрес, а просто пометка «этот сигнал не доставлять», и она переживает замену программы.

Из этого следует неочевидное: если оболочка поставит себе SIG_IGN на SIGINT и запустит ребёнка, **ребёнок унаследует игнорирование**. Ctrl+C перестанет работать вообще для всех запускаемых программ. Проверим. Файл inherit.cpp — родитель игнорирует SIGINT, запускает sleep 5 и посылает ему SIGINT; в режиме restore ребёнок перед exec возвращает поведение по умолчанию:

```
pid_t pid = fork();
if (pid == 0) {
    if (restore) {
        struct sigaction dfl;
        dfl.sa_handler = SIG_DFL;
        sigemptyset(&dfl.sa_mask);
        dfl.sa_flags = 0;
        sigaction(SIGINT, &dfl, nullptr);
    }
    char prog[] = "sleep";
    char arg[] = "5";
    char* const av[] = { prog, arg, nullptr };
    execvp(prog, av);
    _exit(127);
}

sleep(1);
kill(pid, SIGINT);

int status = 0;
waitpid(pid, &status, 0);
```

```
$ ./inherit
ребёнок завершился сам, код 0
$ ./inherit restore
ребёнок убит сигналом 2
```

В первом запуске `sleep` унаследовал игнорирование и спокойно доспал свои пять секунд, не заметив сигнала. Во втором — умер, как и полагается. Разница ровно в четырёх строках, восстанавливающих `SIG_DFL` перед `exec`.

Это и есть решение нашей задачи. Оболочка игнорирует `SIGINT` у себя, а каждый ребёнок перед `exec` возвращает поведение по умолчанию. `Ctrl+C` летит всей группе, но убивает только ребёнка.

29.10.1 Побочный эффект, о который легко споткнуться

Когда я готовил примеры к этой главе, первый запуск дал странный результат: я послал `SIGINT` группе, и не умер никто — ни `sleep`, ни оболочка, у которой вообще не было никакой обработки сигналов.

Разгадка в том, что оболочку я запустил в фоне, через `&`. А `bash` и `zsh`, запуская фоновую задачу, ставят ей `SIG_IGN` на `SIGINT` — чтобы `Ctrl+C`, предназначенный переднему плану, не задевал фоновые процессы. Дальше сработало правило из таблицы: `SIG_IGN` пережил и `fork`, и `exec`, и достался всей цепочке, включая `sleep`.

Мораль двойная. Первая: правило про наследование `SIG_IGN` — не теория, оно проявляется в самых обычных ситуациях. Вторая: если ваша программа обязана реагировать на сигнал, не полагайтесь на унаследованное состояние — ставьте `SIG_DFL` или свой обработчик явно.

29.11 SIGCHLD

Когда ребёнок умирает, родителю приходит `SIGCHLD`. По умолчанию он игнорируется, и это единственный из распространённых сигналов, чьё умолчание безобидно.

Забрать код возврата всё равно надо, иначе останется зомби (глава 26). Есть три способа. Первый — синхронный `waitpid` сразу после запуска: так делаем мы, и никакого `SIGCHLD` нам не нужно. Второй — обработчик, который подбирает всех завершившихся:

```
void on_sigchld(int) {
    while (waitpid(-1, nullptr, WNOHANG) > 0) { }
}
```

Цикл здесь не для красоты: сигналы не копятсь в очереди, и пока обработчик работает, могли умереть ещё двое детей — но сигнал придёт один. Поэтому забирать надо всех, до тех пор пока `waitpid` с `WNOHANG` не скажет, что готовых больше нет. Такой обработчик понадобится, когда появятся фоновые задачи (`cmd &`) — тогда дети умирают в непредсказуемый момент.

Третий способ — сказать `signal(SIGCHLD, SIG_IGN)` явно. По POSIX это не просто игнорирование: ядро само подбирает завершившихся детей и не создаёт зомби. Удобно для программ, которым коды возврата не важны, но нам они как раз важны, так что этот путь не наш.

29.12 SIGPIPE

С этим сигналом мы уже познакомились в прошлой главе: запись в канал, у которого не осталось читателей, убивает писателя, и код возврата получается 141.

Для нашей оболочки это опасно. Представьте цепочку `./generator | head -1`. Если оболочка сама что-то пишет в этот же канал — а она может, если мы добавим встроенные команды с выводом в конвейер, — SIGPIPE прикончит её вместо программы. Поэтому оболочки почти всегда ставят себе SIG_IGN на SIGPIPE, а ошибку EPIPE от `write` обрабатывают в коде.

29.13 SIGTERM и аккуратное завершение

Отдельно стоит сказать про SIGTERM, потому что с ним вы столкнётесь в любой серверной работе. Это сигнал «пожалуйста, заверши работу»: его посылает `kill` без аргументов, его посылает `systemctl stop`, его же посылает Docker при остановке контейнера. И почти всегда за ним следует SIGKILL, если процесс не ушёл сам за отведённое время — у Docker по умолчанию это десять секунд.

Разница между двумя сигналами принципиальная. SIGTERM можно перехватить, а значит — успеть дописать буферы на диск, закрыть соединения, снять блокировки, сообщить балансировщику «я больше не принимаю запросы». SIGKILL перехватить нельзя: процесс исчезает мгновенно, недописанные данные пропадают, временные файлы остаются.

Канонический скелет аккуратного завершения выглядит так:

```
volatile sig_atomic_t should_stop = 0;

void on_term(int) { should_stop = 1; }

int main() {
    struct sigaction sa;
    sa.sa_handler = on_term;
```

```
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;           // без SA_RESTART: хотим прервать ожидание
sigaction(SIGTERM, &sa, nullptr);
sigaction(SIGINT, &sa, nullptr);

while (!should_stop) {
    // ... обычная работа: принять запрос, обработать, ответить ...
}

// сюда попадаем после сигнала — здесь можно всё
save_state();
close_connections();
return 0;
}
```

Отсутствие SA_RESTART здесь намеренное: мы хотим, чтобы блокирующий вызов внутри цикла прервался с EINTR и цикл дошёл до проверки флага. С SA_RESTART программа продолжила бы ждать нового запроса и не заметила бы просьбы завершиться до самого SIGKILL.

Именно поэтому `kill -9` — плохая привычка. Он работает всегда и потому кажется удобным, но лишает программу возможности прибраться за собой. Правильная последовательность: `kill (SIGTERM)`, подождать, и только если процесс не отреагировал — `kill -9`.

29.13.1 Сигнал во время `waitpid`

Есть тонкость, которая касается прямо нашей оболочки. Пока она ждёт ребёнка в `waitpid`, ей может прийти сигнал. Если бы у нас стоял обработчик без SA_RESTART, `waitpid` вернул бы `-1` с `errno == EINTR`, и код решил бы, что ребёнок пропал.

Нас спасает то, что оболочка не ставит обработчиков вовсе: SIG_IGN не прерывает системные вызовы, потому что доставки не происходит. Но как только вы добавите настоящий обработчик — например, для SIGTERM из примера выше или для SIGCHLD ради фоновых задач, — придётся защитить ожидание циклом:

```
int status = 0;
pid_t r;
do {
    r = waitpid(children[i], &status, 0);
} while (r < 0 && errno == EINTR);
```

Это тот же шаблон, что и для `read`. В системном программировании он встречается так часто, что многие заводят себе обёртки вроде `retry_on_eintr(...)` и больше об этом не думают.

29.14 Реализация в `demo-shell`

Всё сводится к двум коротким функциям. Новый файл `include/signals.h`:

```
#ifndef SHELL_SIGNALS_H
#define SHELL_SIGNALS_H

namespace shell {

    // Установить обработчики сигналов для shell.
    //  SIGINT (Ctrl+C) – игнорируем; будет идти только в ребёнка.
    //  SIGQUIT (Ctrl+\) – то же.
    //  SIGPIPE – игнорируем (write вернёт EPIPE вместо смерти).
    void install_shell_signal_handlers();

    // Восстановить дефолтные обработчики в ребёнке после fork (до exec).
    void restore_default_signal_handlers();

} // namespace shell

#endif
```

`src/signals.cpp`:

```
#include "signals.h"

#include <csignal>

namespace shell {

    void install_shell_signal_handlers() {
        // sigaction вместо signal – переносимое и контролируемое.
        struct sigaction sa;
        sa.sa_handler = SIG_IGN;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = 0;
```



```

    sigaction(SIGINT, &sa, nullptr);
    sigaction(SIGQUIT, &sa, nullptr);
    sigaction(SIGPIPE, &sa, nullptr);
    // SIGCHLD оставляем по умолчанию — мы синхронно waitpid'аем.
    // Иначе дети-зомби были бы проблемой.
}

void restore_default_signal_handlers() {
    struct sigaction sa;
    sa.sa_handler = SIG_DFL;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, nullptr);
    sigaction(SIGQUIT, &sa, nullptr);
    sigaction(SIGPIPE, &sa, nullptr);
}

} // namespace shell

```

Ни одного обработчика-функции — только SIG_IGN и SIG_DFL. Это осознанный выбор: раз обработчик почти ничего не может делать безопасно, самый надёжный обработчик — тот, которого нет. Всё, что нам нужно, достигается переключением между «игнорировать» и «как обычно».

SIGQUIT игнорируем по той же причине, что и SIGINT: Ctrl+ шлёт его всей группе, а по умолчанию он не просто убивает, но ещё и сбрасывает дампы памяти на диск. Терять оболочку и получать файл core на пустом месте не хочется.

Подключаем в src/main.cpp — одна строка в самом начале:

```

int main() {
    shell::install_shell_signal_handlers();
    ...
}

```

И в src/pipeline.cpp, первой же строкой в ветке ребёнка, до всех dup2:

```

if (pid == 0) {
    restore_default_signal_handlers();
}

```

```
if (i > 0) {
    dup2(pipes[i - 1][0], STDIN_FILENO);
}
...
}
```

Почему именно здесь, а не после `dup2`? Порядок в данном случае не принципиален — важно лишь, чтобы восстановление случилось до `exec`. Но ставить его первой строкой удобнее: сразу видно, что ребёнок начинает жизнь с чистого листа, а не наследует настройки родителя.

Не забудьте `#include "signals.h"` в обоих файлах.

29.15 Проверка

Собираем и повторяем тот же опыт, что открывал главу: команда `sleep 5`, затем `SIGINT` всей группе процессов.

```
$ make
$ ./build/myshell
=== myshell ===
myshell$ sleep 5
[exit 130]
myshell$ echo я жив после Ctrl+C
я жив после Ctrl+C
myshell$ exit
```

`sleep` умер, оболочка выжила и продолжила принимать команды. Код 130 — это, как мы помним, $128 + 2$, где 2 — номер `SIGINT`; оболочка получила его из `WTERMSIG` и сообщила по общей конвенции.

Стоит проверить и то, что сигналы теперь доходят только куда надо. Запустите `cat` без аргументов — он будет ждать ввода — и нажмите `Ctrl+C`. Умрёт `cat`, приглашение вернётся. Запустите `yes | head -3` — увидите три строки и приглашение: `yes` получил `SIGPIPE`, потому что в ребёнке восстановлены умолчания, а оболочка от того же сигнала защищена.

29.16 Два приёма, которые пригодятся дальше

Наша оболочка обошлась `SIG_IGN` и `SIG_DFL`, но в Части V, когда мы будем писать сервер, простого игнорирования уже не хватит. Два приёма стоит знать заранее.

29.16.1 Кто послал сигнал: SA_SIGINFO

Обычный обработчик получает только номер сигнала. Если добавить флаг SA_SIGINFO и использовать поле sa_sigaction вместо sa_handler, обработчик получит структуру с подробностями — в том числе номер процесса-отправителя:

```
#include <csignal>
#include <cstring>
#include <iostream>
#include <unistd.h>

volatile sig_atomic_t sender_pid = 0;

void on_term(int, siginfo_t* info, void*) {
    sender_pid = info->si_pid;    // кто нас об этом попросил
}

int main() {
    struct sigaction sa;
    std::memset(&sa, 0, sizeof(sa));
    sa.sa_sigaction = on_term;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_SIGINFO;
    sigaction(SIGTERM, &sa, nullptr);

    std::cout << getpid() << "\n";
    std::cout.flush();

    while (sender_pid == 0) pause();
    std::cout << "SIGTERM пришёл от процесса " << sender_pid << "\n";
    return 0;
}
```

```
$ мой PID: 50993, шлю SIGTERM процессу 50996
SIGTERM пришёл от процесса 50993
```

Кроме si_pid в структуре лежат si_uid (владелец отправителя), si_code (откуда сигнал — от kill, от ядра, от таймера) и, для SIGSEGV, si_addr — адрес, обращение к которому вызвало падение. Последнее бесценно при отладке: обработчик может записать адрес в файл, и вы узнаете, куда именно программа полезла.

`std::memset(&sa, 0, sizeof(sa))` здесь не лишний. У `struct sigaction` поля `sa_handler` и `sa_sigaction` на большинстве систем лежат в одном объединении, и остатки мусора в неинициализированной структуре приводят к тому, что ядро зовёт не ту функцию.

29.16.2 Self-pipe: превратить сигнал в обычное событие

Главная беда обработчика — в нём почти ничего нельзя делать. Есть красивый обходной приём, придуманный ещё в девяностых: обработчик пишет один байт в канал, а основной цикл читает этот канал как обычный источник данных. Запись в канал через `write` *async-signal-safe*, а всё остальное происходит в спокойной обстановке главного цикла.

```
int sig_pipe[2];

void on_sigint(int sig) {
    char c = static_cast<char>(sig);
    ssize_t ignored = write(sig_pipe[1], &c, 1);    // write – async-signal-safe
    (void)ignored;
}

int main() {
    pipe(sig_pipe);

    struct sigaction sa;
    sa.sa_handler = on_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGINT, &sa, nullptr);

    char c;
    while (read(sig_pipe[0], &c, 1) == 1) {
        std::cout << "из главного цикла: пришёл сигнал "
                    << static_cast<int>(c) << "\n";
        std::cout.flush();
        if (c == SIGINT) break;
    }
    return 0;
}
```

```
$ kill -INT 51010
```

```
из главного цикла: пришёл сигнал 2
```

Ценность приёма не в самом переносе печати, а в том, что сигнал становится обычным дескриптором. Значит, его можно положить в тот же `poll`, который следит за сетевыми соединениями, — и цикл событий узнает о сигнале ровно тем же способом, что и о пришедших данных. Именно так устроены настоящие серверы, и в главе 43, когда будем писать реактор на `poll`, мы к этому приёму вернёмся.

Приведение `(void)ignored` нужно, чтобы компилятор с `-Wunused-result` не ругался на проигнорированный результат `write`. Проверять его здесь бессмысленно: обработчик всё равно ничего не сможет сделать с ошибкой.

29.17 Чего мы не сделали

Наше решение — простое и рабочее, но настоящий `bash` устроен сложнее, и стоит понимать, чего мы лишились.

Перед запуском команды `bash` создаёт для неё отдельную группу процессов вызовом `setpgid`, а затем передаёт этой группе владение терминалом через `tcsetpgrp`. После этого `Ctrl+C` доходит только до группы переднего плана, а сама оболочка сигнала вообще не видит — не потому, что игнорирует, а потому, что он ей не адресован.

Это даёт то, чего у нас нет: управление задачами. `Ctrl+Z` останавливает программу и возвращает приглашение, `jobs` показывает список приостановленных, `fg` возвращает задачу на передний план, `bg` продолжает её в фоне. Всё это построено на группах процессов, `SIGTSTP`, `SIGCONT` и передаче владения терминалом.

Цена — заметно более тонкий код с настоящими гонками: между `fork` и `setpgid` в родителе и ребёнке существует окно, из-за которого группе приходится устанавливать в обоих процессах сразу; `tcsetpgrp` из фоновой группы сам порождает `SIGTTOU`, который надо предварительно заблокировать. Для учебной оболочки это лишний уровень сложности, и я сознательно оставил `job control` за бортом. Если захочется — это отличное большое упражнение; начать стоит с `man 2 setpgid` и главы про `job control` в POSIX.

29.18 Главные правила главы

1. `sigaction` вместо `signal`: предсказуемое поведение, маска и флаги.
2. `sigemptyset(&sa.sa_mask)` обязателен — структура не инициализируется сама.
3. В обработчике можно только выставить флаг типа `volatile sig_atomic_t`; `printf`, `malloc` и потоки C++ не `async-signal-safe`.
4. Флаг `SA_RESTART` перезапускает прерванные системные вызовы; без него проверяйте `errno == EINTR`.

5. SIG_IGN переживает `exec`, а свой обработчик — нет: он сбрасывается в SIG_DFL.
6. Ребёнок обязан восстановить умолчания до `exec`, иначе унаследует игнорирование от родителя.
7. SIGKILL и SIGSTOP не перехватываются — на этом держится возможность убить любой процесс.
8. Сигналы не копятя: двадцать одинаковых сигналов подряд дают одно срабатывание обработчика.
9. Заблокированный через `sigprocmask` сигнал не теряется, а ждёт разблокировки — но тоже в одном экземпляре.
10. В коде пишите имена сигналов, а не номера: они различаются между Linux и macOS.

29.19 Упражнения

1. Соберите обновлённую оболочку. Запустите `sleep 5`, нажмите Ctrl+C. Убедитесь, что оболочка выжила и напечатала `[exit 130]`. Затем запустите `sleep 5` и нажмите Ctrl+ — что изменилось и почему?
2. Закомментируйте `restore_default_signal_handlers()` в `pipeline.cpp`, пересоберите. Запустите `sleep 5` и нажмите Ctrl+C. Объясните результат, опираясь на таблицу наследования.
3. Запустите `cat` без аргументов и нажмите Ctrl+C, потом ещё раз `cat` и нажмите Ctrl+D. Чем отличаются два способа выйти и какой код возврата в каждом случае?
4. Повторите опыт `flag.cpp`, но замените в обработчике присваивание флага на `std::cout << "поймал\n"`. Соберите, нажимайте Ctrl+C часто и подряд. Удалось ли увидеть перемешанный или потерянный вывод?
5. Повторите опыт `blocked.cpp`, но добавьте в критическую секцию второй сигнал: заблокируйте и SIGUSR1, и SIGUSR2, пошлите оба и посмотрите, в каком порядке они доставятся после разблокировки. Зависит ли порядок от того, в каком порядке вы их посылали?
6. Напишите программу с `alarm(1)` и обработчиком SIGALRM, который переустанавливает будильник и увеличивает счётчик тиков. После десяти тиков — выход. Сравните поведение с SA_RESTART и без него, если в главном цикле стоит `read` со стандартного ввода.
7. Сделайте так, чтобы наша оболочка при получении SIGTERM (`kill PID` снаружи) успевала напечатать «до свидания» и корректно выйти. Подсказка: обработчик выставляет флаг, а цикл REPL проверяет его после `getline`.

8. (Сложнее) Прочитайте `man 2 setpgid` и `man 2 tcsetpgrp`. Реализуйте создание отдельной группы процессов для каждой запускаемой команды и передачу ей терминала. Проверьте, что `Ctrl+C` работает, даже если убрать `SIG_IGN` из оболочки.

29.20 Что дальше

Глава 29 — парсер командной строки. Сейчас `tokenize` режет строку по пробелам, и `echo "hello world"` разваливается на части. Напишем настоящий токенизатор — конечный автомат с состояниями «внутри двойных кавычек», «внутри одинарных», «после обратного слэша» — и научим оболочку понимать перенаправления `>`, `>>`, `<`, `2>` и `2>&1`. Заодно у структуры `Command`, которую мы завели в главе 27, наконец появится второе поле.

Глава 30

Глава 29. Парсер командной строки

Наш токенизатор до сих пор состоял из четырёх строк: положили строку в `istringstream` и вычерпали слова оператором `>>`. Для `ls -l | wc -l` этого хватало, но стоит появиться кавычкам — и всё рассыпается:

```
myshell$ sh -c "exit 42"
42": -c: line 0: unexpected EOF while looking for matching `"'
[exit 2]
```

Строка развалилась на четыре слова: `sh`, `-c`, `"exit, 42"`. Кавычки остались внутри слов, а пробел между ними разрезал аргумент пополам.

В этой главе напишем настоящий токенизатор — с двойными и одинарными кавычками, с экранированием обратным слэшем, — и научим оболочку понимать перенаправления `>`, `>>`, `<`, `2>`, `2>>` и `2>&1`. Заодно у структуры `Command`, заведённой в главе 27, наконец появится второе поле, а в проекте — первые тесты.

Задача эта родственна разбору мира в главе 23, но требовательнее: там формат придумывали мы сами и могли сделать его удобным, здесь синтаксис задан сорока годами традиции, и отступать от него нельзя — пользователь ждёт, что `echo "a b"` сработает точно так же, как в `bash`.

30.1 Что должно получиться

Возьмём строку:

```
grep "hello world" file.txt > result.txt 2>&1
```

Разобрать её нужно так: команда `grep` с аргументами `grep`, `hello world` и `file.txt`, плюс два перенаправления — вывод в `result.txt` и поток ошибок туда же, куда вывод. Кавычки при

этом должны исчезнуть: программа `grep` получит аргумент `hello world` без них, ведь кавычки — синтаксис оболочки, а не часть данных.

Структуры для этого уже почти готовы. Дополним `include/pipeline.h`:

```
struct Redirect {
    enum Type {
        Input,          // < file
        Output,         // > file
        Append,         // >> file
        ErrOutput,       // 2> file
        ErrAppend,       // 2>> file
        ErrToOut,        // 2>&1 (без файла)
    };
    Type type;
    std::string target; // имя файла; пусто для ErrToOut
};

struct Command {
    std::vector<std::string> argv;
    std::vector<Redirect> redirects;
};

struct Pipeline {
    std::vector<Command> commands;

    bool empty() const { return commands.empty(); }
    std::size_t size() const { return commands.size(); }
};
```

`Redirect` — тип и цель. `ErrToOut` единственный обходится без цели: `2>&1` не открывает файл, а связывает один дескриптор с другим.

Порядок перенаправлений хранится в `vector` не случайно. Как увидим ближе к концу главы, `> out 2>&1` и `2>&1 > out` дают разный результат, и единственный способ воспроизвести поведение `bash` — применять их строго в том порядке, в каком они написаны.

30.2 Две ступени разбора

Разбор традиционно делят на две независимые ступени.

Токенизатор (его же называют лексером) работает с символами и ничего не знает про команды. Его дело — понять, где кончается одно слово и начинается другое, снять кавычки, обработать экранирование и выделить служебные символы в отдельные токены. На выходе — плоский список строк.

Парсер работает уже с токенами и ничего не знает про кавычки. Его дело — увидеть в списке структуру: разбить по | на команды, отделить перенаправления от аргументов, проверить, что после > действительно стоит имя файла.

Разделение стоит держать в голове, потому что оно окупается сразу. Каждая ступень получается небольшой и проверяемой отдельно; ошибка «незакрытая кавычка» рождается в токенизаторе, ошибка «редирект без файла» — в парсере, и по тексту сообщения сразу понятно, куда смотреть. Ровно так же устроены настоящие компиляторы, только там между лексером и парсером стоит ещё грамматика посерьёзнее.

30.3 Токенизатор

Требований к нему шесть.

Пробелы и табуляции разделяют слова — но только вне кавычек. Двойные кавычки `"..."` собирают всё внутри в одно слово, при этом обратный слэш внутри них экранирует четыре символа: `"`, `\`, `$` и обратную кавычку. Одинарные кавычки `'...'` берут содержимое буквально, без единого исключения — внутри них не работает даже экранирование. Обратный слэш вне кавычек экранирует любой следующий символ. Служебные `|`, `<`, `>`, `>>` становятся отдельными токенами, даже если написаны вплотную к слову. И, наконец, `2>`, `2>>`, `2>&1` тоже отдельные токены, но распознаются только в начале слова — иначе `file2.txt` развалилось бы на куски.

Всё это естественно ложится на конечный автомат с тремя состояниями: «обычный текст», «внутри двойных кавычек», «внутри одинарных». Переписываем `src/exec_runner.cpp` целиком:

```
#include "exec_runner.h"

#include <stdexcept>
#include <string>
#include <vector>

namespace shell {

namespace {
```

```
// Распознать на позиции i спец-токен. Возвращает длину поглощённых символов
// и сам токен. 0 – не спец.
std::size_t match_special(const std::string& s, std::size_t i, std::string& out) {
    if (i >= s.size()) return 0;
    char c = s[i];

    if (c == '|') { out = "|"; return 1; }
    if (c == '<') { out = "<"; return 1; }
    if (c == '>') {
        if (i + 1 < s.size() && s[i + 1] == '>') { out = ">>"; return 2; }
        out = ">"; return 1;
    }
    return 0;
}

} // namespace

std::vector<std::string> tokenize(const std::string& line) {
    std::vector<std::string> tokens;
    std::string current;
    bool in_dq = false; // double quote
    bool in_sq = false; // single quote

    auto flush = [&]() {
        if (!current.empty()) {
            tokens.push_back(std::move(current));
            current.clear();
        }
    };

    for (std::size_t i = 0; i < line.size(); ++i) {
        char c = line[i];

        if (in_sq) {
            if (c == '\\') in_sq = false;
            else current += c;
            continue;
        }
```

```
if (in_dq) {
    if (c == '"') {
        in_dq = false;
    } else if (c == '\\' && i + 1 < line.size() &&
               (line[i + 1] == '"' || line[i + 1] == '\\' ||
                line[i + 1] == '$' || line[i + 1] == '`')) {
        current += line[++i];
    } else {
        current += c;
    }
    continue;
}

// Вне кавычек.
if (c == '\') { in_sq = true; continue; }
if (c == '"') { in_dq = true; continue; }

if (c == '\\' && i + 1 < line.size()) {
    current += line[++i];
    continue;
}

if (c == ' ' || c == '\t') {
    flush();
    continue;
}

// 2>, 2>>, 2>&1 – только в начале слова.
if (current.empty() && c == '2' && i + 1 < line.size() && line[i + 1] == '>') {
    if (i + 3 < line.size() &&
        line[i + 2] == '&' && line[i + 3] == '1') {
        tokens.push_back("2>&1");
        i += 3;
        continue;
    }
    if (i + 2 < line.size() && line[i + 2] == '>') {
        tokens.push_back("2>>");
    }
}
```

```

        i += 2;
        continue;
    }
    tokens.push_back("2>");
    i += 1;
    continue;
}

std::string spec;
std::size_t consumed = match_special(line, i, spec);
if (consumed > 0) {
    flush();
    tokens.push_back(spec);
    i += consumed - 1;
    continue;
}

current += c;
}

if (in_sq || in_dq) {
    throw std::runtime_error("незакрытая кавычка");
}

flush();
return tokens;
}

} // namespace shell

```

Разберём по частям.

Лямбда `flush` сбрасывает накопленное слово в список токенов и очищает накопитель. Условие `if (!current.empty())` избавляет нас от пустых токенов при двойных пробелах: строка `ls -l` даст два слова, а не четыре. У этой мелочи есть побочный эффект, о котором скажу ниже, — `echo ""` теряет пустой аргумент.

Ветка `in_sq` — самая простая во всём коде, и это отражает суть одинарных кавычек. Внутри них нет ничего, кроме символов: закрывающая кавычка заканчивает режим, всё остальное копируется как есть. Поэтому вставить одинарную кавычку внутрь одинарных кавычек

невозможно в принципе — ни в нашей оболочке, ни в `bash`. Классический обход выглядит пугающе: `'don\'\'t'` — закрыли кавычку, поставили экранированный апостроф, открыли снова.

Ветка `in_dq` сложнее ровно на одно правило: обратный слэш экранирует, но лишь четыре символа. Это не наша прихоть, а поведение `bash`: внутри двойных кавычек `\n` остаётся двумя символами, слэшем и буквой, а вот `\"` превращается в кавычку. Смысл в том, что внутри двойных кавычек специальными остаются только `$`, обратная кавычка и сама кавычка — экранировать имеет смысл лишь их.

Дальше идёт обычный режим. Кавычки включают соответствующее состояние и в слово не попадают. Обратный слэш забирает следующий символ буквально — так работает `echo a\ b`, дающий одно слово `a b`. Пробел и табуляция завершают слово.

Отдельного внимания заслуживает условие `current.empty()` перед разбором `2>`. Без него строка `cat file2.txt` распалась бы на `cat`, `file`, `2>`, `.txt` — токенизатор увидел бы двойку с угловой скобкой в середине имени файла и радостно принял бы её за перенаправление. Проверка «накопитель пуст» означает «мы в начале слова», и этого достаточно. В `bash`, кстати, правило то же самое, только там оно сформулировано через понятие «номер дескриптора перед оператором».

Функция `match_special` возвращает не токен, а число поглощённых символов — так вызывающий код узнаёт, на сколько сдвинуть позицию. Для `>>` это два символа, для `>` один. В цикле пишем `i += consumed - 1`, потому что `++i` в заголовке `for` добавит недостающую единицу. Такие смещения — обычный источник ошибок «на единицу», и тесты, которые мы сейчас напишем, ловят их первыми.

Незакрытая кавычка в конце строки — синтаксическая ошибка, и мы бросаем исключение. `bash` в этой ситуации ведёт себя дружелюбнее: печатает вторичное приглашение `>` и ждёт продолжения, собирая многострочную команду. Нам такое поведение стоило бы переделки цикла `REPL`, и я его сознательно не делаю — зато в конце главы есть упражнение.

30.4 Тесты токенизатора

Токенизатор — идеальный кандидат для юнит-тестов: чистая функция, строка на входе, вектор строк на выходе, никаких системных вызовов. Наш `Makefile` из главы 26 уже умеет собирать тесты (цель `tests`), но каталог `tests/` до сих пор пустовал. Создадим `tests/test_tokenize.cpp`:

```
#include "exec_runner.h"
#include "pipeline.h"
```

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

int failures = 0;
int checks = 0;

std::string join(const std::vector<std::string>& v) {
    std::string r;
    for (std::size_t i = 0; i < v.size(); ++i) {
        if (i) r += " | ";
        r += "[" + v[i] + "];"
    }
    return r;
}

void expect_tokens(const std::string& line,
                  const std::vector<std::string>& expected) {
    ++checks;
    std::vector<std::string> got = shell::tokenize(line);
    if (got != expected) {
        ++failures;
        std::cout << "FAIL tokenize(\"" << line << "\")\n"
                  << "   ждали:  " << join(expected) << "\n"
                  << "   выдало: " << join(got) << "\n";
    }
}

void expect_throw(const std::string& line) {
    ++checks;
    try {
        shell::tokenize(line);
        ++failures;
        std::cout << "FAIL tokenize(\"" << line << "\") – ждали исключение\n";
    } catch (const std::runtime_error&) {
    }
```

```

        // как и задумано
    }
}

} // namespace

```

Никакого фреймворка — та же самодельная схема, что в тестах RPG из главы 25: счётчик проверок, счётчик провалов, понятное сообщение при расхождении. Сравнение векторов строк работает само: `operator==` для `std::vector` сравнивает поэлементно, а для `std::string` — посимвольно.

Сами проверки:

```

int main() {
    // Простые слова.
    expect_tokens("echo hello", {"echo", "hello"});
    expect_tokens("  ls  -l  ", {"ls", "-l"});
    expect_tokens("", {});

    // Кавычки.
    expect_tokens("echo \"hello world\"", {"echo", "hello world"});
    expect_tokens("echo 'hello world'", {"echo", "hello world"});
    expect_tokens("echo 'one \"two\"'", {"echo", "one \"two\""});
    expect_tokens("echo \"a'b\"", {"echo", "a'b"});
    expect_tokens("echo \"\"", {"echo"}); // пустое слово теряется

    // Escape.
    expect_tokens("echo a\\ b", {"echo", "a b"});
    expect_tokens("echo \"a\\\"b\\\"", {"echo", "a\"b"});
    expect_tokens("echo 'a\\b'", {"echo", "a\\b"}); // внутри ' escape не работает

    // Спец-токены.
    expect_tokens("ls | wc", {"ls", "|", "wc"});
    expect_tokens("ls|wc", {"ls", "|", "wc"});
    expect_tokens("cmd > file.txt", {"cmd", ">", "file.txt"});
    expect_tokens("cmd >> file.txt", {"cmd", ">>", "file.txt"});
    expect_tokens("cmd < in.txt", {"cmd", "<", "in.txt"});
    expect_tokens("cmd 2> err.txt", {"cmd", "2>", "err.txt"});
    expect_tokens("cmd 2>> err.txt", {"cmd", "2>>", "err.txt"});
    expect_tokens("cmd 2>&1", {"cmd", "2>&1"});
}

```



```
// Цифра 2 в середине слова — не редирект.
expect_tokens("cat file2.txt", {"cat", "file2.txt"});

// Незакрытые кавычки.
expect_throw("echo \"no end");
expect_throw("echo 'no end");
...
```

Обратите внимание на двойное экранирование в строковых литералах C++. Чтобы проверить пользовательский ввод `echo "hello world"`, в исходнике надо написать `"echo \"hello world\""`; чтобы проверить `echo a\ b` — написать `"echo a\\ b"`. Первый слой экранирования съедает компилятор C++, второй — наш токенизатор. Путаться тут нормально, и лучший способ не запутаться — держать в голове, что именно увидит пользователь.

Три теста заслуживают отдельного слова.

`expect_tokens("ls|wc", {"ls", "|", "wc"})` подтверждает, что теперь вертикальная черта не нуждается в пробелах. В главе 27 наивный токенизатор этого не умел, и я обещал, что ограничение исчезнет, — вот оно и исчезло, само собой, потому что `match_special` разрезает слово в любом месте.

`expect_tokens("cat file2.txt", {"cat", "file2.txt"})` — тот самый случай, ради которого написано условие `current.empty()`. Уберите его — и тест немедленно покраснеет.

`expect_tokens("echo \"\"", {"echo"})` фиксирует известное расхождение с `bash`. Настоящая оболочка передала бы `echo` пустой аргумент, и вывелась бы пустая строка; у нас пустое слово теряется в `flush`. Тест написан не для того, чтобы одобрить это поведение, а чтобы оно было зафиксировано: если однажды вы почините пустые аргументы, тест напомним о себе и его нужно будет обновить. Тесты, описывающие текущие ограничения, полезны не меньше тестов, описывающих правильную работу.

Собираем и запускаем:

```
$ make tests
g++ ... -c tests/test_tokenize.cpp -o build/tests/test_tokenize.o
g++ ... build/tests/test_tokenize.o build/builtins.o build/exec_runner.o \
      build/pipeline.o build/signals.o -o build/tests/test_tokenize

$ ./build/tests/test_tokenize
все проверки прошли: 31/31
```

Теперь любую правку токенизатора можно проверить за секунду, а не вылавливать её потом в живой оболочке.

30.5 Парсер

Токены есть, пора собирать из них структуру. Работа делится на две функции: внешняя режет список по |, внутренняя разбирает один кусок.

```
Pipeline parse_pipeline(const std::string& line) {
    Pipeline p;
    auto tokens = tokenize(line);
    if (tokens.empty()) return p;

    // Разбиваем по `|` на сегменты, каждый – Command.
    auto seg_start = tokens.begin();
    for (auto it = tokens.begin(); it != tokens.end(); ++it) {
        if (*it == "|") {
            if (seg_start == it) {
                throw std::runtime_error("пустая команда перед '|'");
            }
            p.commands.push_back(parse_command(seg_start, it));
            seg_start = it + 1;
        }
    }
    if (seg_start != tokens.end()) {
        p.commands.push_back(parse_command(seg_start, tokens.end()));
    } else {
        throw std::runtime_error("пустая команда после '|'");
    }

    // Проверка: каждая команда имеет хотя бы один аргумент.
    for (const auto& cmd : p.commands) {
        if (cmd.argv.empty()) {
            throw std::runtime_error("команда без названия");
        }
    }
    return p;
}
```

Сегменты задаются парой итераторов — от `seg_start` до найденной черты. Такой способ передачи диапазона привычен по алгоритмам STL из главы 14 и избавляет от копирования кусков вектора.

Три проверки покрывают все способы ошибиться с чертой. `seg_start == it` означает, что между началом сегмента и чертой ничего нет, — так ловится `| ls и ls || wc` (две черты подряд). Условие `seg_start != tokens.end()` после цикла проверяет, что за последней чертой что-то есть, — так ловится `ls |`. И финальный проход по командам ловит случай, когда токены есть, а имени команды нет: `> out.txt` состоит из двух токенов, но запускать нечего.

Последняя проверка нужна ещё и потому, что `parse_command` спокойно возвращает команду без `argv`, если ей достались одни перенаправления. Разделение обязанностей здесь такое: `parse_command` разбирает то, что видит, а осмысленность целого проверяет `parse_pipeline`.

Теперь разбор одной команды:

```
bool is_redirect_token(const std::string& t) {
    return t == "<" || t == ">" || t == ">>" ||
           t == "2>" || t == "2>>" || t == "2>&1";
}

Redirect::Type redirect_type_for(const std::string& t) {
    if (t == "<") return Redirect::Input;
    if (t == ">") return Redirect::Output;
    if (t == ">>") return Redirect::Append;
    if (t == "2>") return Redirect::ErrOutput;
    if (t == "2>>") return Redirect::ErrAppend;
    if (t == "2>&1") return Redirect::ErrToOut;
    throw std::runtime_error("не редирект: " + t);
}

Command parse_command(std::vector<std::string>::iterator begin,
                      std::vector<std::string>::iterator end) {
    Command cmd;
    for (auto it = begin; it != end; ++it) {
        if (is_redirect_token(*it)) {
            Redirect r;
            r.type = redirect_type_for(*it);
            if (r.type == Redirect::ErrToOut) {
                // 2>&1 — без аргумента-файла
            } else {
```

```

        if (it + 1 == end || is_redirect_token(*(it + 1)) || *(it + 1) == "|") {
            throw std::runtime_error("редирект '" + *it + "' без файла");
        }
        ++it;
        r.target = *it;
    }
    cmd.redirects.push_back(std::move(r));
} else {
    cmd.argv.push_back(*it);
}
}
return cmd;
}

```

Логика простая: встретили служебный токен — значит, началось перенаправление, и следующий токен (кроме случая `2>&1`) должен быть именем файла. Всё остальное — аргументы.

Проверка перед `++it` разбирает три способа написать перенаправление без файла: строка кончилась (`ls >`), дальше идёт другое перенаправление (`ls > > x`), дальше идёт черта (`ls > | wc`). Без неё `++it` шагнул бы за конец диапазона — и мы получили бы чтение чужой памяти вместо внятного сообщения об ошибке.

Важная деталь: перенаправления вынимаются из потока аргументов, а не остаются в нём. Команда `ls -l > out.txt` даст `argv` из двух элементов, `ls` и `-l`; про файл знает только `Redirect`. Программе `ls` о перенаправлении знать незачем, для неё вывод как шёл в дескриптор 1, так и идёт.

30.6 Применение перенаправлений

Разбор закончен, пора выполнять. Перенаправления применяются в ребёнке, после `fork` и до `exec`, — в том самом промежутке, ради которого в Unix и разделили создание процесса на два шага.

```

// Применить редиректы в текущем процессе (вызывается в ребёнке).
// При ошибке завершается с _exit.
void apply_redirects(const std::vector<Redirect>& redirects) {
    for (const auto& r : redirects) {
        switch (r.type) {
            case Redirect::Input: {
                int fd = open(r.target.c_str(), O_RDONLY);

```

```
    if (fd < 0) {
        std::cerr << r.target << ": " << std::strerror(errno) << "\n";
        _exit(1);
    }
    dup2(fd, STDIN_FILENO);
    close(fd);
    break;
}

case Redirect::Output: {
    int fd = open(r.target.c_str(),
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        std::cerr << r.target << ": " << std::strerror(errno) << "\n";
        _exit(1);
    }
    dup2(fd, STDOUT_FILENO);
    close(fd);
    break;
}

case Redirect::Append: {
    int fd = open(r.target.c_str(),
                  O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd < 0) {
        std::cerr << r.target << ": " << std::strerror(errno) << "\n";
        _exit(1);
    }
    dup2(fd, STDOUT_FILENO);
    close(fd);
    break;
}

case Redirect::ErrOutput: {
    int fd = open(r.target.c_str(),
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        std::cerr << r.target << ": " << std::strerror(errno) << "\n";
        _exit(1);
    }
    dup2(fd, STDERR_FILENO);
```

```

        close(fd);
        break;
    }
    case Redirect::ErrAppend: {
        int fd = open(r.target.c_str(),
                     O_WRONLY | O_CREAT | O_APPEND, 0644);
        if (fd < 0) {
            std::cerr << r.target << ": " << std::strerror(errno) << "\n";
            _exit(1);
        }
        dup2(fd, STDERR_FILENO);
        close(fd);
        break;
    }
    case Redirect::ErrToOut: {
        // stderr → туда же, куда stdout сейчас идёт.
        dup2(STDOUT_FILENO, STDERR_FILENO);
        break;
    }
}
}
}
}
}

```

Код повторяется, и это осознанный выбор: пять почти одинаковых веток читаются легче, чем одна с тремя флагами-параметрами. Если такое повторение вас смущает, вынесите открытие в маленькую функцию `open_or_die(target, flags)` — это упражнение в конце главы.

Каждая ветка делает одно и то же в три шага: открыть файл с нужными флагами, `dup2` на нужный стандартный дескриптор, закрыть оригинал. Порядок последних двух действий важен, и мы разбирали его в главе 27: сначала дублируем, потом закрываем.

Флаги `open` стоит запомнить, они встречаются постоянно. `O_RDONLY` — только чтение, для `<`. Комбинация `O_WRONLY | O_CREAT | O_TRUNC` — только запись, создать, если файла нет, обрезать до нуля, если есть; это `>`. Комбинация `O_WRONLY | O_CREAT | O_APPEND` — то же самое, но каждая запись идёт в конец файла; это `>>`. Флаг `O_APPEND` не просто ставит позицию в конец при открытии — он делает это атомарно перед каждой записью, поэтому два процесса, дописывающих в один лог, не затирают строки друг друга.

Третий аргумент `0644` — права доступа для нового файла, в восьмеричной записи: владельцу чтение и запись, остальным только чтение. Ноль в начале и означает восьмеричную систему.

Права из этого числа реальный файл получит не целиком: ядро вычитет из них маску `umask` процесса, о которой шла речь в главе 5. При обычной маске `022` из `0644` получится `0644`, а вот `0666` превратится в `0644`.

Ошибка открытия обрабатывается жёстко — сообщение и `_exit(1)`. Мягче нельзя: мы уже в ребёнке, отступить некуда, а запускать программу с перенаправленным выводом было бы хуже, чем не запускать вовсе. Именно так ведёт себя `bash`: `cat < nosuchfile` не выполняет `cat` вообще.

Ветка `ErrToOut` вообще не открывает файлов — `dup2(STDOUT_FILENO, STDERR_FILENO)` направляет поток ошибок туда же, куда в этот момент направлен вывод. Слова «в этот момент» здесь — самое главное.

30.7 Порядок перенаправлений решает всё

Возьмём программу, которая пишет в оба потока:

```
#include <iostream>
int main() {
    std::cout << "это stdout\n";
    std::cerr << "это stderr\n";
    return 0;
}
```

И запустим её в нашей оболочке двумя способами:

```
myshell$ /tmp/errout > /tmp/o1.txt 2>&1
myshell$ /tmp/errout 2>&1 > /tmp/o2.txt
это stderr
```

```
$ cat /tmp/o1.txt
это stdout
это stderr
$ cat /tmp/o2.txt
это stdout
```

В первом случае в файле оба потока. Во втором — только вывод, а ошибка осталась на экране, хотя написано, казалось бы, то же самое.

Разгадка в порядке применения. В первом случае сначала `>` подменяет дескриптор 1 файлом, а затем `2>&1` копирует дескриптор 1 — уже указывающий на файл — в дескриптор 2. Оба потока в файле.

Во втором случае сначала `2>&1` копирует дескриптор 1 — а он в этот момент ещё указывает на терминал — в дескриптор 2. Ошибки прибиты к терминалу. И только потом `>` перенаправляет дескриптор 1 в файл; дескриптора 2 это уже не касается, он давно скопирован.

Это самый известный подвох `bash`, и запоминается он одной фразой: `2>&1` копирует то, куда дескриптор указывает **сейчас**, а не создаёт вечную связь. Отсюда правило: `> file 2>&1` отправляет в файл всё, а `2>&1 > file` — только вывод.

Наша оболочка воспроизводит это поведение бесплатно, потому что хранит перенаправления в векторе по порядку и применяет их подряд. Стоило бы нам, скажем, отсортировать их или сложить в `map` по типу — и совместимость с `bash` сломалась бы.

30.8 Перенаправления вместе с конвейером

Что будет, если у команды есть и черта, и перенаправление?

```
myshell$ ls Makefile > /tmp/o3.txt | wc -l
0
```

```
$ cat /tmp/o3.txt
Makefile
```

Вывод `ls` ушёл в файл, а `wc` не получил ничего и насчитал ноль строк. Так и должно быть: побеждает то, что применено последним, а порядок в нашем коде задан явно:

```
if (pid == 0) {
    restore_default_signal_handlers();

    if (i > 0) {
        dup2(pipes[i - 1][0], STDIN_FILENO);
    }
    if (i < n - 1) {
        dup2(pipes[i][1], STDOUT_FILENO);
    }
    close_all_pipes(pipes);

    // Редиректы применяются ПОСЛЕ pipe-dup2, поэтому они «победнее».
    // То есть `cmd > file | other` — stdout cmd идёт в file, не в pipe.
    apply_redirects(p.commands[i].redirects);

    auto argv = to_c_argv(p.commands[i].argv);
    execvp(argv[0], argv.data());
}
```



```
std::cerr << "exec '" << argv[0] << "' failed: "  
    << std::strerror(errno) << "\n";  
_exit(127);  
}
```

Сначала конвейер, потом перенаправления — значит, перенаправление затирает конвейер. `bash` ведёт себя точно так же, и `wc` в нём тоже напечатает ноль. Заодно обратите внимание, что `wc` не завис: пишущий конец канала закрыт всеми, кто его держал, поэтому конец данных наступил сразу.

30.9 Проверка

```
$ make  
$ ./build/myshell  
=== myshell ===  
myshell$ echo "hello world"  
hello world  
myshell$ echo 'single quotes'  
single quotes  
myshell$ echo a\ b  
a b  
myshell$ sh -c "exit 42"  
[exit 42]  
myshell$ echo hello > /tmp/out.txt  
myshell$ cat /tmp/out.txt  
hello  
myshell$ echo ещё > /tmp/out.txt  
myshell$ echo дописали >> /tmp/out.txt  
myshell$ cat /tmp/out.txt  
ещё  
дописали  
myshell$ ls nosuchfile 2> /tmp/err.txt  
[exit 1]  
myshell$ cat /tmp/err.txt  
ls: nosuchfile: No such file or directory  
myshell$ wc -l < /tmp/out.txt  
2  
myshell$ exit
```

Команда `sh -c "exit 42"`, с которой начиналась глава, теперь отрабатывает правильно: код 42 доходит до нас в целости, потому что аргумент `exit 42` дошёл до `sh` одним куском.

30.10 Чего парсер не умеет

Настоящая оболочка умеет намного больше, и честнее перечислить пропущенное, чем делать вид, что его не существует. Ни одну из этих возможностей мы реализовывать не будем — но каждая упирается в понятную работу, а не в магию.

Последовательности `;`, `&&` и `||` требуют ещё одного уровня разбора: сейчас вершина нашего дерева — конвейер, а нужна цепочка конвейеров со связками между ними. Работы примерно на одну функцию `parse_sequence` и небольшой цикл выполнения, который смотрит на код возврата предыдущего конвейера.

Подстановка переменных `$HOME` живёт в токенизаторе: увидели `$`, собрали имя, вызвали `getenv`, подставили значение. Тонкость в том, что подстановка работает вне кавычек и внутри двойных, но не внутри одинарных, — как раз то различие, ради которого в нашем автомате уже есть три состояния.

Раскрытие шаблонов `*.crr` делается после токенизации и до запуска: каждое слово с `*`, `?` или `[...]` разворачивается в список подходящих имён файлов. Руками это писать не нужно, в POSIX есть функция `glob` из `<glob.h>`.

Пустой аргумент `echo ""` теряется — про это мы уже говорили, и лечится оно флагом «слово было в кавычках, сбрось его даже пустым».

Многострочный ввод (незакрытая кавычка, вложенный документ `<< EOF`) требует переделки цикла `REPL`: оболочка должна понимать, что команда не закончилась, и запрашивать продолжение.

Фоновый запуск & вообще не про парсер — распознать амперсанд легко, а вот дальше нужны список задач, обработчик `SIGCHLD` и команды `jobs`, `fg`, `bg`. Это управление задачами, о котором шла речь в конце главы 28.

30.11 Три ступени, если посмотреть сверху

Если отойти на шаг, наша оболочка выглядит так:

строка от пользователя

↓ `tokenize` — состояния: текст, `"..."`, `'...'`, escape

список токенов

↓ `parse_pipeline` — режет по `'|'`, проверяет структуру

```
↓ parse_command  — отделяет редиректы от аргументов
Pipeline
↓ run_pipeline   — fork, dup2, exec, waitpid
результат
```

Это в миниатюре та же схема, по которой устроен любой компилятор или интерпретатор: лексер превращает текст в токены, парсер — токены в дерево, исполнитель обходит дерево и что-то делает. В gcc между второй и третьей ступенью помещаются ещё оптимизации, а в Python дерево сначала превращается в байт-код, но каркас узнаётся сразу.

Именно поэтому имеет смысл писать парсеры руками хотя бы раз. Дальше вы будете брать готовые библиотеки — но уже понимая, что внутри.

30.12 Главные правила главы

1. Лексер и парсер разделяют: первый знает про символы и кавычки, второй — про структуру команды.
2. Токенизатор с кавычками — это конечный автомат; состояний ровно столько, сколько видов кавычек, плюс обычный режим.
3. Значение символа зависит от позиции: 2> в начале слова — перенаправление, в середине — часть имени.
4. Проверяйте границы перед тем, как заглянуть в следующий токен, иначе выйдете за конец диапазона.
5. Перенаправления применяются в ребёнке между fork и exec, строго в порядке написания.
6. 2>&1 копирует текущее назначение дескриптора, поэтому > f 2>&1 и 2>&1 > f дают разный результат.
7. Ошибка открытия файла в ребёнке означает _exit, а не запуск программы с перенаправленным выводом.
8. Чистая функция вроде токенизатора обязана быть покрыта тестами — это дёшево и окупается сразу.

30.13 Упражнения

1. Соберите оболочку и тесты (make && make tests && ./build/tests/test_tokenize). Проверьте вручную: echo "Привет, мир", echo it\'s, echo 'внутри "двойные"', cat < Makefile | head -3.
2. Уберите условие current.empty() из разбора 2>, пересоберите тесты. Какой тест упал и что он теперь выдаёт? Верните условие обратно.

3. Напишите программу `errout`, печатающую строку в `stdout` и строку в `stderr`. Запустите её в оболочке четырьмя способами: `> f`, `2> f`, `> f 2>&1`, `2>&1 > f`. Для каждого объясните, что оказалось в файле и что на экране.
4. Добавьте перенаправление `&>` (весь вывод в файл, сокращение для `> f 2>&1`). Понадобится новый токен в `match_special`, новый тип в `Redirect` и новая ветка в `apply_redirects`.
5. Вынесите повторяющееся открытие файла из `apply_redirects` в функцию `int open_or_die(const std::string& path, int flags)`. Проверьте, что все тесты по-прежнему проходят и оболочка ведёт себя как раньше.
6. Реализуйте подстановку `$VAR` и `${VAR}` в токенизаторе. Внутри одинарных кавычек подстановки быть не должно. Добавьте тесты: `echo $HOME`, `echo "$HOME/x"`, `echo '$HOME'`.
7. (Сложнее) Реализуйте `;`, `&&` и `||`. Заведите структуру `Sequence` со списком конвейеров и связей, разберите её в `parse_sequence`, а в `main` выполняйте с учётом кодов возврата.
8. (Сложнее) Сделайте многострочный ввод: если `tokenize` бросил «незакрытая кавычка», оболочка печатает вторичное приглашение `>`, читает следующую строку, склеивает через `\n` и пробует разобрать снова.

30.14 Что дальше

Глава 30 — встроенные команды и история. Мы уже знаем, почему `cd` не может быть отдельной программой: она меняет состояние самой оболочки, а запущенный ребёнок унесёт изменение с собой в могилу. Разберёмся, какие команды обязаны быть встроенными, как устроены `export` и `unset`, почему `echo` существует и как встроенная, и как отдельная программа, и научим оболочку помнить историю команд между запусками.

Глава 31

Глава 30. Встроенные команды и история

Оболочка у нас получилась работающая: запускает программы, соединяет их в цепочки, перенаправляет потоки, разбирает кавычки, не умирает от Ctrl+C. Но одна простейшая команда до сих пор не работает:

```
myshell$ cd /tmp
myshell$ pwd
/Users/serik/Desktop/From-scratch/cpp-from-scratch/demo-shell
```

Папка не сменилась. Мы уже знаем почему: `cd` запустился в ребёнке, сменил рабочую папку ребёнку и умер вместе со своим достижением. Рабочая папка — свойство процесса, и никакая запущенная программа не может поменять её родителю.

Это не недоработка, а фундаментальное ограничение, и обходят его во всех оболочках мира одинаково: такие команды выполняются **внутри** оболочки, без `fork` и `exec`. Их называют встроенными. В этой главе добавим девять встроенных команд — `cd`, `pwd`, `export`, `unset`, `echo`, `history`, `type`, `alias` и `unalias` — и научим оболочку помнить историю между запусками.

31.1 Три причины быть встроенной командой

Команды становятся встроенными по трём разным причинам, и их полезно различать.

Первая причина — необходимость. Команда меняет состояние самой оболочки, а значит, обязана выполняться в её процессе. Таких немного: `cd` меняет рабочую папку, `export` и `unset` — переменные окружения, `exit` завершает оболочку, `alias` заводит сокращения для будущих команд, `source` выполняет скрипт в текущем процессе, чтобы его переменные остались. Внешней программой ни одну из них сделать нельзя в принципе.

Вторая причина — скорость. `echo`, `pwd`, `test`, `l`, `printf` вполне могли бы быть отдельными программами — и они существуют, в `/bin/echo` и `/bin/pwd` можно ткнуть пальцем. Но каждый их запуск стоит `fork` плюс `exec` плюс загрузка исполняемого файла плюс инициализация библиотеки времени выполнения. В скрипте с тысячей проверок это заметно.

Насколько заметно, легко измерить. Я прогнал через нашу оболочку триста команд `echo hi` — сперва встроенной версией, потом явно указав `/bin/echo`:

```
300 раз echo (builtin): 11 мс
300 раз /bin/echo (exec): 920 мс
отношение: 83.7x
```

Восемьдесят три раза. Три миллисекунды на запуск процесса против сотых долей на вызов функции — вот и вся арифметика. Именно поэтому `l` в `bash` встроен, хотя файл `/bin/l` тоже лежит на диске ради совместимости со старыми скриптами.

Третья причина — доступ к внутренним данным оболочки. Команда `history` печатает список, который хранится в памяти нашего процесса; внешней программе его просто неоткуда взять. Сюда же относятся `jobs`, `fg` и `bg`, знающие про список фоновых задач.

31.2 Точка входа

Весь механизм сводится к одной функции. Новый файл `include/builtins.h`:

```
#ifndef SHELL_BUILTINS_H
#define SHELL_BUILTINS_H

#include <string>
#include <vector>

namespace shell {

// Попытаться выполнить встроенную команду. Возвращает true, если команда
// встроенная (вне зависимости от успеха/неуспеха выполнения; код в `out_code`).
// Возвращает false, если команда не встроенная — её надо запускать через exec.
bool try_builtin(const std::vector<std::string>& argv, int& out_code);

// Раскрыть алиас в первом слове строки. Возвращает строку без изменений,
// если алиас не найден. Раскрывается один уровень — рекурсию не поддерживаем.
std::string expand_alias(const std::string& line);
```

```
// История.
void history_add(const std::string& line);
void history_load(const std::string& path);
void history_save(const std::string& path);
const std::vector<std::string>& history();

} // namespace shell

#endif
```

Обратите внимание на разделение двух вещей, которые легко перепутать. Возвращаемое значение `bool` отвечает на вопрос «это встроенная команда?», а не «получилось ли». Результат выполнения уходит в `out_code`. Если бы мы совместили их в одном `int`, вызывающему коду пришлось бы гадать, означает ли ненулевое значение ошибку команды или её отсутствие.

Подключение в `main.cpp`:

```
// Builtin — только если одна команда и без редиректов (упрощение).
if (p.commands.size() == 1 && p.commands[0].redirects.empty()) {
    int code = 0;
    if (shell::try_builtin(first, code)) {
        if (code != 0) std::cerr << "[exit " << code << "]\n";
        continue;
    }
}

int rc = shell::run_pipeline(p);
```

Сначала пробуем встроенную, и только если её нет — запускаем внешнюю. Порядок именно такой во всех оболочках: встроенная версия побеждает одноимённую программу на диске. Поэтому `echo` в `bash` — это встроенный `echo`, а не `/bin/echo`, и чтобы позвать именно программу, приходится писать путь целиком.

31.3 Почему встроенные команды не работают в конвейере

Условие `p.commands.size() == 1 && redirects.empty()` — упрощение, и стоит понять, что мы упростили.

Пусть пользователь написал `cd /tmp | cat`. Формально это конвейер из двух команд, значит, наше условие не выполняется, и `cd` уходит в `run_pipeline` — то есть запускается как внешняя программа в ребёнке. Проверим, что из этого выходит:

```
myshell$ cd /tmp | cat
myshell$ pwd
/Users/serik/Desktop/From-scratch/cpp-from-scratch/demo-shell
```

Папка не сменилась — ровно как в начале главы. И это правильное поведение: `bash` ведёт себя точно так же. Каждая команда конвейера выполняется в своём процессе, а изменение рабочей папки в чужом процессе бессмысленно по определению. Писать `cd` в конвейере просто не имеет смысла, и никакая оболочка не станет ради этого выдумывать особую механику.

С перенаправлениями сложнее. Команда `pwd > out.txt` вполне осмысленна, и `bash` её выполняет: он форкается, применяет перенаправление в ребёнке и там же выполняет встроенную команду. Мы этого не делаем — `pwd > out.txt` уйдёт во внешний `/bin/pwd`, и результат будет тот же самый, но лишним процессом. А вот `cd /tmp > log.txt` у нас сменит папку не там, где надо. Случай экзотический, и я оставил упрощение в силе, но в упражнениях есть задание его убрать.

31.4 Реализации

Все команды живут в `src/builtins.cpp`, каждая — отдельная функция в анонимном пространстве имён, а наружу торчит только `try_builtin`.

31.4.1 `cd`

```
int builtin_cd(const std::vector<std::string>& argv) {
    std::string target;
    if (argv.size() < 2) {
        const char* home = std::getenv("HOME");
        if (!home) {
            std::cerr << "cd: HOME не задана\n";
            return 1;
        }
        target = home;
    } else {
        target = argv[1];
    }
    if (chdir(target.c_str()) < 0) {
        std::cerr << "cd: " << target << ": " << std::strerror(errno) << "\n";
        return 1;
    }
}
```



```
    return 0;
}
```

Вся команда — это один системный вызов `chdir` из `<unistd.h>`, меняющий рабочую папку текущего процесса. Он возвращает 0 при успехе и -1 при ошибке, а причину кладёт в `errno`: `ENOENT`, если папки нет, `ENOTDIR`, если путь ведёт к файлу, `EACCES`, если нет прав войти.

Без аргументов `cd` уходит домой. Домашняя папка берётся из переменной окружения `HOME` функцией `std::getenv` из `<cstdlib>`; она возвращает указатель на строку внутри окружения процесса или `nullptr`, если переменной нет. Проверка на `nullptr` не паранойя: окружение можно очистить полностью, и тогда `HOME` действительно не будет.

Настоящий `cd` умеет больше: `cd -` возвращает в предыдущую папку, `cd ~user` ведёт в чужой дом, `cd` без аргументов в некоторых оболочках учитывает переменную `CDPATH`. Наш вариант — минимальный, и первые две возможности несложно добавить самому.

31.4.2 `pwd`

```
int builtin_pwd() {
    char buf[4096];
    if (!getcwd(buf, sizeof(buf))) {
        std::cerr << "pwd: " << std::strerror(errno) << "\n";
        return 1;
    }
    std::cout << buf << "\n";
    return 0;
}
```

`getcwd(buf, size)` записывает в буфер абсолютный путь текущей папки и возвращает указатель на него — или `nullptr`, если не влезло (`ERANGE`) либо папка была удалена у нас из-под ног (`ENOENT`, и такое бывает).

Размер 4096 взят из `PATH_MAX` — предельной длины пути на Linux; на macOS предел меньше, 1024. Пользоваться константой `PATH_MAX` напрямую соблазнительно, но она не так надёжна, как кажется: на некоторых системах её вообще нет, а на Linux она не учитывает, что путь длиннее предела всё-таки может существовать, если собирать его по частям. Надёжный вариант — передать `getcwd` нулевой указатель, тогда на Linux и macOS она сама выделит память нужного размера, а вы потом её освободите. Для учебной оболочки четырёх килобайт на стеке достаточно.

Полезно понимать, что `getcwd` не читает никакой переменной, а спрашивает ядро. Поэтому она всегда говорит правду — в отличие от переменной `PWD`, которую оболочки поддерживают сами и которая может отстать от реальности.

31.4.3 `export` и `unset`

```
int builtin_export(const std::vector<std::string>& argv) {
    if (argv.size() < 2) {
        // Без аргументов — печать всех переменных. Упростим: пропустим.
        std::cerr << "export: укажите KEY=VALUE\n";
        return 1;
    }
    for (std::size_t i = 1; i < argv.size(); ++i) {
        const std::string& kv = argv[i];
        auto eq = kv.find('=');
        if (eq == std::string::npos) {
            std::cerr << "export: ожидался формат KEY=VALUE: " << kv << "\n";
            return 1;
        }
        std::string key = kv.substr(0, eq);
        std::string val = kv.substr(eq + 1);
        if (setenv(key.c_str(), val.c_str(), 1) < 0) {
            std::cerr << "export: " << std::strerror(errno) << "\n";
            return 1;
        }
    }
    return 0;
}

int builtin_unset(const std::vector<std::string>& argv) {
    for (std::size_t i = 1; i < argv.size(); ++i) {
        if (unsetenv(argv[i].c_str()) < 0) {
            std::cerr << "unset: " << std::strerror(errno) << "\n";
            return 1;
        }
    }
    return 0;
}
```

Разбор KEY=VALUE делается через `find('=')` и два `substr`. Значение может содержать знаки равенства — `substr(eq + 1)` берёт всё до конца строки, так что `export X=a=b` даст переменной значение `a=b`, как и положено.

Третий аргумент `setenv` — флаг перезаписи. Единица означает «перезаписать, если переменная уже есть»; с нулём существующее значение осталось бы нетронутым.

Стоит убедиться, что это действительно работает — то есть что запущенные программы видят наши переменные:

```
myshell$ export F00=bar
myshell$ sh -c "echo видно ли: $F00"
видно ли: bar
myshell$ env | grep F00
F00=bar
myshell$ unset F00
myshell$ env | grep F00
[exit 1]
```

Обратите внимание на вторую строку: подстановку `$F00` выполнил не наш токенизатор (он этого не умеет), а запущенный `sh`, которому строка досталась дословно. И значение он взял из своего окружения — унаследованного от нас. Последний `grep` не нашёл ничего и вернул код 1, что оболочка честно показала.

31.4.4 Как окружение попадает в ребёнка

Раз уж речь зашла об окружении, стоит пояснить механику, потому что она устроена неочевидно.

Окружение процесса — это массив строк вида KEY=VALUE, оканчивающийся нулевым указателем. Он доступен через глобальную переменную `environ`, объявленную как `extern char** environ`, и через третий, малоизвестный параметр `main`:

```
int main(int argc, char** argv, char** envp) {
    for (char** p = envp; *p; ++p) {
        std::cout << *p << "\n";
    }
}
```

`setenv` и `unsetenv` правят именно этот массив, а функция `execvp`, которой мы запускаем программы, молча передаёт текущий `environ` новому образу. Отсюда и получается наследование: мы изменили массив у себя — `fork` скопировал его ребёнку — `exec` передал

далее. Если бы понадобилось запустить программу с другим окружением, мы бы взяли `exesve`, у которой окружение — явный третий аргумент.

И маленькое предупреждение про `getenv`: она возвращает указатель на строку внутри этого массива, а не копию. Указатель становится недействительным после следующего `setenv`, потому что массив может быть перевыделен. Правило простое: понадобилось значение надолго — скопируйте его в `std::string` сразу.

31.4.5 echo

```
int builtin_echo(const std::vector<std::string>& argv) {
    bool newline = true;
    std::size_t start = 1;
    if (argv.size() > 1 && argv[1] == "-n") {
        newline = false;
        start = 2;
    }
    for (std::size_t i = start; i < argv.size(); ++i) {
        if (i > start) std::cout << " ";
        std::cout << argv[i];
    }
    if (newline) std::cout << "\n";
    return 0;
}
```

Печатаем аргументы через пробел; условие `i > start` ставит разделитель перед каждым аргументом, кроме первого, — обычный приём, чтобы не получить лишний пробел в конце. Флаг `-n` отменяет перевод строки.

Разделение аргументов пробелом важно понимать правильно. Оболочка уже разобрала строку на слова, и `echo` печатает их заново через один пробел. Поэтому `echo a b` выведет `a b` с одним пробелом — лишние съел токенизатор, — а `echo "a b"` сохранит все пробелы, потому что это одно слово.

Флаг `-e`, включающий в `bash` разбор `\n` и `\t` внутри строки, мы не делаем: экранирование у нас уже отработал токенизатор из главы 29.

31.4.6 exit

Единственная команда, которую мы обрабатываем не через `try_builtin`, а прямо в цикле:

```
const auto& first = p.commands[0].argv;
if (!first.empty() && (first[0] == "exit" || first[0] == "quit")) {
    break;
}
```

Причина в том, что `exit` должен закончить цикл, а функция `try_builtin` этого сделать не может — она умеет только вернуть код. Пришлось бы либо заводить особое значение «пора выходить», либо звать `std::exit` изнутри. Второй путь хуже: `std::exit` прерывает работу немедленно, и `history_save` в конце `main` уже не выполнится — история потеряется. Явный `break` оставляет управление там, где ему положено быть.

`quit` как синоним `exit` — наша вольность, в настоящих оболочках такой команды нет.

31.4.7 Диспетчер

```
bool try_builtin(const std::vector<std::string>& argv, int& out_code) {
    if (argv.empty()) {
        out_code = 0;
        return true;
    }
    const std::string& cmd = argv[0];

    if (cmd == "cd")        { out_code = builtin_cd(argv);    return true; }
    if (cmd == "pwd")       { out_code = builtin_pwd();        return true; }
    if (cmd == "export")    { out_code = builtin_export(argv); return true; }
    if (cmd == "unset")     { out_code = builtin_unset(argv);  return true; }
    if (cmd == "history")   { out_code = builtin_history();    return true; }
    if (cmd == "echo")      { out_code = builtin_echo(argv);   return true; }

    return false;
}
```

Цепочка сравнений выглядит примитивно, и рука тянется заменить её на `std::unordered_map` из имени в функцию — тем более что в главе 21 мы именно так сделали реестр команд RPG. Здесь я сознательно оставил простой вариант: команд девять, сравнение коротких строк стоит наносекунды, а читается такой код без единой мысли. Заводить таблицу стоит, когда команд становится два-три десятка или когда их нужно регистрировать динамически — например, для `alias`.

31.5 alias: сокращения для команд

Ещё одна команда, которая обязана быть встроенной, — `alias`. Она заводит короткое имя для длинной команды, и заведённое должно жить в оболочке, а не в умершем ребёнке.

Хранилище — обычная упорядоченная таблица:

```
#include <map>

namespace {
    std::map<std::string, std::string> g_aliases;
}
```

`std::map` вместо `std::unordered_map` выбран не случайно: он держит ключи в отсортированном порядке, и печать всех алиасов сразу выходит по алфавиту, без отдельной сортировки. Алиасов десятки, скорость поиска тут никого не волнует.

Сама команда работает в трёх режимах — показать все, показать один, задать:

```
// alias          — показать все алиасы
// alias ll="ls -l" — задать алиас
int builtin_alias(const std::vector<std::string>& argv) {
    if (argv.size() < 2) {
        for (std::map<std::string, std::string>::const_iterator it = g_aliases.begin();
             it != g_aliases.end(); ++it) {
            std::cout << "alias " << it->first << "=" << it->second << "'\n";
        }
        return 0;
    }
    int rc = 0;
    for (std::size_t i = 1; i < argv.size(); ++i) {
        std::size_t eq = argv[i].find('=');
        if (eq == std::string::npos) {
            std::map<std::string, std::string>::const_iterator it = g_aliases.find(argv[i]);
            if (it == g_aliases.end()) {
                std::cerr << "alias: " << argv[i] << ": не найден\n";
                rc = 1;
            } else {
                std::cout << "alias " << it->first << "=" << it->second << "'\n";
            }
            continue;
        }
    }
```

```

    }
    g_aliases[argv[i].substr(0, eq)] = argv[i].substr(eq + 1);
}
return rc;
}

int builtin_unalias(const std::vector<std::string>& argv) {
    for (std::size_t i = 1; i < argv.size(); ++i) {
        g_aliases.erase(argv[i]);
    }
    return 0;
}

```

Разбор KEY=VALUE здесь тот же, что в `export`. Обратите внимание, что кавычки в `alias ll='ls -l'` до нас не доходят — их снял токенизатор, и в `argv[1]` лежит готовая строка `ll=ls -l`.

Теперь главное — подстановка. Алиас надо раскрыть **до** разбора строки, потому что он может содержать что угодно, включая перенаправления и черты конвейера:

```

std::string expand_alias(const std::string& line) {
    // Берём первое слово строки — только оно может быть алиасом.
    std::size_t start = line.find_first_not_of(" \t");
    if (start == std::string::npos) return line;
    std::size_t end = line.find_first_of(" \t", start);
    std::string first = line.substr(start, end == std::string::npos
                                    ? std::string::npos
                                    : end - start);

    std::map<std::string, std::string>::const_iterator it = g_aliases.find(first);
    if (it == g_aliases.end()) return line;

    std::string rest = (end == std::string::npos) ? "" : line.substr(end);
    return it->second + rest;
}

```

И вызов в цикле `main`, перед разбором:

```

// Алиас раскрываем до разбора: `ll -a` → `ls -l -a`.
std::string expanded = shell::expand_alias(line);

shell::Pipeline p;

```

```
try {  
    p = shell::parse_pipeline(expanded);  
    ...  
}
```

Раскрывается только первое слово и только один раз. Настоящие оболочки идут дальше: раскрывают алиас рекурсивно (если раскрытое снова начинается с алиаса), а если значение алиаса кончается пробелом — проверяют и следующее слово. Рекурсию мы не делаем сознательно: без защиты от заикливания `alias ll="ls -l"` подвесил бы оболочку намертво, а с защитой код перестаёт быть примером на десять строк. В `bash` от этой ловушки спасает правило «имя алиаса не раскрывается внутри собственного значения».

Заметьте ещё, что в историю попадает исходная строка, а не раскрытая. Так удобнее: пролистав историю, человек увидит то, что набирал.

```
myshell$ alias ll="ls -l"  
myshell$ alias  
alias ll='ls -l'  
myshell$ ll src  
builtins.cpp  
exec_runner.cpp  
main.cpp  
pipeline.cpp  
signals.cpp  
myshell$ unalias ll  
myshell$ ll src  
exec 'll' failed: No such file or directory  
[exit 127]
```

31.6 Две разные вещи, которые называют переменными

С `export` связана путаница, которая живёт даже у опытных людей. В настоящей оболочке есть **два** вида переменных, и они устроены совершенно по-разному.

Переменные оболочки живут только в её памяти. Создаются присваиванием без всяких команд:

```
$ X=5  
$ echo $X  
5
```


Переменные окружения живут в массиве `environ` и передаются всем запускаемым программам. Обычная переменная становится переменной окружения после `export`.

Разницу видно в один опыт:

```
$ bash -c 'X=5; echo "в самом bash: $X"; sh -c "echo в ребёнке: [\ $X]"'
в самом bash: 5
в ребёнке: []
```

Переменная есть в самой оболочке и отсутствует у ребёнка. Добавьте `export X` — и в квадратных скобках появится пятёрка.

Наша оболочка умеет только вторую половину: `export F00=bar` работает, а простое присваивание — нет.

```
myshell$ X=5
exec 'X=5' failed: No such file or directory
[exit 127]
```

Строка `X=5` не распознаётся как присваивание и уходит в `execvp` как имя программы. Чтобы это заработало, в цикл нужно добавить проверку: если единственное слово команды содержит знак равенства до первого пробела и перед ним стоит допустимое имя, то это присваивание, и его надо положить в собственную таблицу переменных оболочки. А `export X` без значения должен переносить переменную из этой таблицы в окружение.

Заводить своё хранилище переменных мы не станем — это потянуло бы за собой подстановку `$X`, без которой переменные оболочки бесполезны. Но понимать разницу между двумя видами стоит: именно из-за неё скрипт, работающий при запуске вручную, ломается в `cron`, где окружение почти пустое.

31.7 Как оболочка ищет программу

Ещё одна вещь, которую полезно понимать про встроенные команды, — порядок поиска. Когда вы вводите слово, `bash` проверяет варианты в строгой очерёдности: сначала функции оболочки, затем встроенные команды, и только потом ищет исполняемый файл в каталогах из `PATH`.

Убедиться можно командой `type`, которая как раз и рассказывает, чем окажется имя:

```
$ type echo
echo is a shell builtin
$ type ls
ls is /bin/ls
```

```
$ type cd
cd is a shell builtin
$ type -a echo
echo is a shell builtin
echo is /bin/echo
```

Последний вызов с ключом `-a` показывает сразу все варианты — и видно, что `/bin/echo` существует, но проигрывает встроенной версии.

Поиск по `PATH` за нас делает `execvp` — та самая буква `p` в её имени. Она берёт переменную `PATH`, разбивает по двоеточиям и пробует каждый каталог по очереди, пока не найдёт исполняемый файл. Если в имени команды есть косая черта (`./build/myshell`), поиск не выполняется вовсе — путь берётся как есть.

Отсюда, кстати, известное правило безопасности: текущий каталог `.` не должен входить в `PATH`. Иначе достаточно положить в общедоступную папку файл с именем `ls`, дожидаться, пока кто-то зайдёт туда и наберёт привычную команду, — и чужая программа запустится от его имени. По этой же причине запускать программу из текущего каталога приходится с явным `./`.

31.7.1 Свой type

Раз поиск устроен так просто, сделаем такую же команду у себя — заодно это будет седьмая встроенная и хороший пример работы с `PATH` руками:

```
// type — рассказать, чем окажется имя: встроенной командой или файлом из PATH.
```

```
int builtin_type(const std::vector<std::string>& argv) {
    if (argv.size() < 2) {
        std::cerr << "type: укажите имя команды\n";
        return 1;
    }

    int rc = 0;
    for (std::size_t i = 1; i < argv.size(); ++i) {
        const std::string& name = argv[i];

        if (is_builtin_name(name)) {
            std::cout << name << " — встроенная команда\n";
            continue;
        }
    }
}
```

```
// Имя с косой чертой ищем как путь, без — перебираем PATH (так же,
```

```
// как это делает exesvr).
if (name.find('/') != std::string::npos) {
    if (access(name.c_str(), X_OK) == 0) {
        std::cout << name << " - " << name << "\n";
    } else {
        std::cerr << "type: " << name << ": не найдено\n";
        rc = 1;
    }
    continue;
}

const char* path = std::getenv("PATH");
std::string dirs = path ? path : "";
bool found = false;
std::size_t start = 0;
while (start <= dirs.size()) {
    std::size_t colon = dirs.find(':', start);
    std::string dir = dirs.substr(start, colon == std::string::npos
                                   ? std::string::npos
                                   : colon - start);

    if (dir.empty()) dir = ".";
    std::string candidate = dir + "/" + name;
    if (access(candidate.c_str(), X_OK) == 0) {
        std::cout << name << " - " << candidate << "\n";
        found = true;
        break;
    }
    if (colon == std::string::npos) break;
    start = colon + 1;
}
if (!found) {
    std::cerr << "type: " << name << ": не найдено\n";
    rc = 1;
}
return rc;
}
```

Здесь три вещи, о которых стоит сказать.

`access(path, X_OK)` спрашивает у ядра, можно ли запускать этот файл, — не открывая его. Кроме `X_OK` бывают `R_OK`, `W_OK` (право на чтение и запись) и `F_OK` (просто существует ли). Пользоваться `access` для проверки «а можно ли, прежде чем делать» вообще-то не рекомендуют: между проверкой и действием мир может измениться, и это классическая уязвимость гонки. Но для справочной команды вроде `type`, которая ничего не делает, а только рассказывает, — самое подходящее применение.

Разбор `PATH` вручную выглядит громоздко из-за возни с `find(':')` и `substr`, и это цена работы со строками в `C++11` — в `C++17` появился бы `string_view`, а с ним и деление без копирования (глава 50). Пустой элемент в `PATH` (два двоеточия подряд или двоеточие в начале) по традиции означает текущий каталог — отсюда `if (dir.empty()) dir = ".";`.

Порядок проверок повторяет настоящий: сначала встроенные, потом файлы. Поэтому наш `type echo` честно скажет, что победит встроенная версия.

```
myshell$ type echo
echo — встроенная команда
myshell$ type ls
ls — /bin/ls
myshell$ type cd
cd — встроенная команда
myshell$ type ./build/mycat
./build/mycat — ./build/mycat
myshell$ type nosuchprogram
type: nosuchprogram: не найдено
[exit 1]
```

31.8 История команд

История — это просто список строк в памяти, который переживает выход из оболочки за счёт текстового файла.

```
namespace {
    std::vector<std::string> g_history;
}

void history_add(const std::string& line) {
    if (line.empty()) return;
    if (!g_history.empty() && g_history.back() == line) return; // не дублируем
```

```
    g_history.push_back(line);  
}
```

Проверка на совпадение с последней записью убирает самый частый мусор: пять раз подряд нажали `make` — в истории одна строка. `bash` управляет этим переменной `HISTCONTROL` со значением `ignoredups`; мы включили такое поведение всегда.

Загрузка и сохранение — обычные потоки из главы 12:

```
void history_load(const std::string& path) {  
    std::ifstream in(path);  
    if (!in.is_open()) return;  
    std::string line;  
    while (std::getline(in, line)) {  
        if (!line.empty()) g_history.push_back(line);  
    }  
}  
  
void history_save(const std::string& path) {  
    std::ofstream out(path);  
    if (!out.is_open()) return;  
    for (const auto& line : g_history) {  
        out << line << "\n";  
    }  
}
```

Формат — одна команда на строку. Никаких меток времени, никакого экранирования; многострочные команды мы и не умеем принимать. Если файла нет — молча выходим: при первом запуске истории просто ещё не существует, и ругаться не на что. Точно так же молча переживаем невозможность записи: не сохранённая история — не повод портить пользователю сессию сообщением об ошибке.

Печать с номерами:

```
int builtin_history() {  
    std::size_t i = 1;  
    for (const auto& line : g_history) {  
        std::cout << " " << i << " " << line << "\n";  
        ++i;  
    }  
}
```

```
    return 0;
}
```

Путь к файлу собирается в `main.cpp`:

```
namespace {

std::string history_path() {
    const char* home = std::getenv("HOME");
    if (!home) return "./.myshell_history";
    return std::string(home) + "/.myshell_history";
}

} // namespace

int main() {
    shell::install_shell_signal_handlers();
    shell::history_load(history_path());

    while (true) {
        ...
        shell::history_add(line);
        ...
    }

    shell::history_save(history_path());
    return 0;
}
```

Точка в начале имени файла делает его скрытым — соглашение Unix, по которому настроечные файлы не мозолят глаза в `ls`. Запасной вариант `./.myshell_history` на случай отсутствия `HOME` нужен для запуска в необычном окружении, например внутри контейнера с очищенным окружением.

`history_add` вызывается сразу после разбора строки, но до выполнения команды. Так в историю попадают и команды с ошибками — и это правильно: чаще всего вспомнить хотят как раз ту команду, которая не сработала, чтобы её исправить.

31.8.1 Компромисс с сохранением

Мы записываем историю один раз, при выходе. Это самый простой вариант, и у него есть очевидный недостаток: если оболочку убьют через `kill -9` или она упадёт, история сессии пропадёт целиком.

Настоящие оболочки поступают иначе — дописывают каждую команду в файл сразу, открывая его в режиме `O_APPEND`. Тогда переживает даже внезапная смерть, а из-за атомарности `O_APPEND` (о ней шла речь в главе 29) две одновременно запущенные оболочки не портят файл друг другу.

Цена — системный вызов на каждую команду. Для оболочки это несущественно: человек набирает команды в тысячи раз медленнее. В упражнениях есть задание переделать сохранение на этот лад.

31.9 Почему не работает стрелка вверх

Раз есть история, хочется листать её стрелками. Не выйдет — и стоит объяснить, почему, потому что причина не в нашей лени.

Терминал по умолчанию работает в каноническом режиме. Драйвер терминала сам собирает строку, сам обрабатывает забой, сам показывает набираемое, и только по нажатию `Enter` отдаёт программе готовую строку. Наш `std::getline` получает её уже целиком.

Стрелка вверх в этом режиме не имеет никакого особого значения — терминал просто передаёт три байта, которые она порождает:

```
$ printf '\033[A\033[B' | cat -v
^[A^[B
```

`^[` — это символ эскапе с кодом 27, дальше идут `[` и буква. Стрелка вверх — `ESC [ A`, вниз — `ESC [ B`, вправо и влево — `C` и `D`. Пока мы читаем строки целиком, эти байты просто попадут в середину команды и испортят её.

Чтобы листать историю, нужен неканонический (сырой) режим. Схема такая: через `tcsetattr` получить текущие настройки терминала, снять флаги `ICANON` (построчная сборка) и `ECNO` (автоматический показ введённого), применить `tcsetattr`. После этого `read` начинает возвращать каждый нажатый байт немедленно — и всю работу приходится делать самому: показывать символы, обрабатывать забой, узнавать управляющие последовательности, двигать курсор теми же последовательностями (`ESC [ K` стирает остаток строки, `ESC [ nD` двигает курсор влево на `n` позиций). Не забыть восстановить настройки при выходе, в том числе аварийном, — иначе пользователь останется в терминале без эха, где ничего не видно при наборе.

Работы тут на пару сотен строк, и она не про C++, а про особенности терминалов, которых накопилось немало. Поэтому в реальных проектах берут готовую библиотеку: GNU readline или её более свободный по лицензии заменитель libedit. Подключается она в три строки и приносит сразу всё — историю со стрелками, поиск по Ctrl+R, автодополнение, редактирование строки. Есть упражнение и на это.

Кое-что, впрочем, работает и без сырого режима, потому что делается драйвером терминала: Ctrl+U стирает строку, Ctrl+W — последнее слово, Ctrl+D в пустой строке даёт конец файла и завершает оболочку.

31.10 Проверка

```
$ ./build/myshell
=== myshell ===
Builtins: cd, pwd, export, unset, history, echo, type, alias, exit.
Кавычки, пайпы, редиректы (>, >>, <, 2>, 2>&1).

myshell$ pwd
/Users/serik/Desktop/From-scratch/cpp-from-scratch/demo-shell
myshell$ cd /tmp
myshell$ pwd
/private/tmp
myshell$ cd
myshell$ pwd
/Users/serik
myshell$ export F00=bar
myshell$ env | grep F00
F00=bar
myshell$ echo -n без перевода
без переводаmyshell$ history
 1  pwd
 2  cd /tmp
 3  pwd
 4  cd
 5  pwd
 6  export F00=bar
 7  env | grep F00
 8  echo -n без перевода
```



```
9 history
myshell$ exit
$ cat ~/.myshell_history
pwd
cd /tmp
pwd
cd
pwd
export FOO=bar
env | grep FOO
echo -n без перевода
history
exit
```

В файл попала и сама команда `exit` — мы добавляем строку в историю до того, как проверяем её на выход. `bash` поступает так же.

Одна деталь в этом выводе стоит объяснения: после `cd /tmp` команда `pwd` показывает `/private/tmp`, а не `/tmp`. Так устроена `macOS` — `/tmp` там символическая ссылка на `/private/tmp`, а `getcwd` спрашивает ядро о настоящем положении дел и разрешает все ссылки. На `Linux` вывод был бы `/tmp`. Кстати, `bash` в такой ситуации напечатает `/tmp`: он ведёт собственную переменную `PWD` и показывает путь, каким его набрал пользователь. Наш вариант честнее, `bash` — привычнее.

31.11 Главные правила главы

1. Команды, меняющие состояние оболочки (`cd`, `export`, `unset`, `exit`), обязаны быть встроенными — внешняя программа физически не может этого сделать.
2. Встроенная версия побеждает одноимённую программу на диске; чтобы позвать программу, указывают полный путь.
3. Скорость встроенной команды выше на два порядка: `fork` с `exec` стоят миллисекунды, вызов функции — наносекунды.
4. Встроенные команды в конвейере смысла не имеют: каждое звено выполняется в своём процессе.
5. `setenv/unsetenv` меняют массив `environ`, который `exec` передаёт запускаемой программе, — так работает наследование окружения.
6. `getenv` возвращает указатель внутрь окружения; нужное значение копируйте сразу.
7. Историю надёжнее дописывать после каждой команды в режиме `O_APPEND`, а не сохранять при выходе.

8. Стрелки и редактирование строки требуют сырого режима терминала; в настоящем проекте для этого берут `readline` или `libedit`.

31.12 Упражнения

1. Соберите оболочку и проверьте все встроенные команды. Убедитесь, что `cd` без аргументов уходит в домашнюю папку, а `cd /nosuchdir` печатает внятную ошибку и код 1.
2. Выясните опытом, что происходит с `cd` в конвейере: выполните `cd /tmp | cat`, затем `pwd`. Объясните результат и скажите, как ведёт себя в этом случае ваш обычный `bash` или `zsh`.
3. Повторите замер из начала главы на своей машине: прогоните триста `echo hi` и триста `/bin/echo hi`, сравните время. Затем добавьте в оболочку встроенный `true` и сравните его с `/usr/bin/true`.
4. Добавьте `cd --` — возврат в предыдущую папку. Понадобится статическая переменная с прошлым путём, которую заполняет `getcwd` перед каждым `chdir`.
5. Реализуйте `export` без аргументов: печать всех переменных окружения по одной на строку. Пройдите по `environ` или по третьему параметру `main`.
6. Переделайте сохранение истории на дописывание после каждой команды. Откройте файл в режиме `std::ios::app` (или через `open` с `O_APPEND`) и добавляйте строку сразу в `history_add`. Проверьте, что история переживает `kill -9`.
7. Уберите упрощение с перенаправлениями: пусть `pwd > out.txt` тоже работает. Подсказка — форкнуть, в ребёнке применить `apply_redirects` и вызвать `try_builtin`, а в родителе дожидаться. Проверьте, что `cd /tmp > log.txt` при этом по-прежнему не меняет папку оболочки, и объясните почему.
8. (Сложнее) Подключите `libedit` (`brew install libedit` или `apt install libedit-dev`), замените `std::getline` на `readline("myshell$ ")` и передайте историю в `add_history`. Получите стрелки, поиск по `Ctrl+R` и редактирование строки. Не забудьте освободить строку, которую вернул `readline`.

31.13 Что дальше

Глава 31 — собственные `cat`, `wc`, `grep` и `head`. Мы написали оболочку, которая соединяет чужие программы; теперь напишем программы, которые она будет соединять. Разберёмся, почему `cat` без аргументов читает стандартный ввод, как считать символы в UTF-8, не

разбирая кодировку целиком, и как устроен поиск по регулярному выражению средствами стандартной библиотеки. На этом Часть III закончится, и мы перейдём к мини-СУБД.

Глава 32

Глава 31. Свои cat, wc, grep и head

Мы написали оболочку, которая умеет соединять чужие программы. Осталось написать программы, которые она будет соединять.

В этой главе сделаем четыре классические утилиты Unix — cat, wc, grep и head. Не игрушечные подоби́я, а рабочие: они читают стандартный ввод, принимают файлы аргументами, соблюдают соглашения о кодах возврата и без проблем встают в цепочку рядом с системными. Назовём их `mycat`, `mywc`, `mygrep` и `myhead`, чтобы не путались с настоящими.

Смысл упражнения не в том, чтобы заменить `/bin/cat`. Смысл в том, что программа-фильтр — самая частая форма командной утилиты вообще. Прочитать поток, что-то с ним сделать, отдать дальше. Написав их, вы получите шаблон, по которому дальше пишется любая своя утилита: конвертер логов, обработчик выгрузки, генератор отчёта.

Заодно разберёмся с вещами, которые в прикладном коде обычно проходят мимо: почему `read` может вернуть меньше запрошенного, как считать символы в UTF-8, не разбирая кодировку целиком, и почему `std::regex` в шесть раз медленнее системного `grep`.

32.1 Модель фильтра

У всех четырёх утилит одинаковый каркас, и он же — общее соглашение Unix.

Если аргументов нет, программа читает стандартный ввод. Это важнейшее правило: именно оно позволяет ставить утилиту в середину конвейера, где никакого файла нет, а есть только канал. Если аргументы есть, они считаются именами файлов, и каждый обрабатывается по очереди. Отдельный аргумент — означает «здесь читай стандартный ввод», что позволяет смешивать файлы и поток: `mycat header.txt - footer.txt`.

Результат всегда идёт в стандартный вывод, ошибки — в поток ошибок. Разделение не формальность: благодаря ему `mygrep foo *.log > found.txt` кладёт в файл только найденные строки, а жалобы на нечитаемые файлы остаются на экране.

Код возврата — ноль при успехе, ненулевой при неудаче, и у каждой утилиты свои оттенки. У `grep` их три, и мы к этому вернёмся.

32.2 mycat

Начнём с самой простой. Задача — прочитать источник и без изменений вывалить его в стандартный вывод.

```
// mycat — простая копия Unix-утилиты cat.
// Читает файлы (или stdin), пишет в stdout.
// Поддержка: один или несколько файлов, "-" как stdin.

#include <cerrno>
#include <cstring>
#include <iostream>
#include <string>
#include <vector>

#include <fcntl.h>
#include <unistd.h>

namespace {

// Скопировать всё из fd в stdout. Возвращает 0 при успехе, 1 при ошибке.
int cat_fd(int fd, const std::string& name) {
    char buf[4096];
    while (true) {
        ssize_t n = read(fd, buf, sizeof(buf));
        if (n == 0) return 0;           // EOF
        if (n < 0) {
            if (errno == EINTR) continue; // прервано сигналом — попробуем снова
            std::cerr << "mycat: " << name << ": " << std::strerror(errno) << "\n";
            return 1;
        }
        // Записываем всё, что прочитали (с учётом short writes).
```

```
    ssize_t written = 0;
    while (written < n) {
        ssize_t m = write(STDOUT_FILENO, buf + written, n - written);
        if (m < 0) {
            if (errno == EINTR) continue;
            std::cerr << "mycat: write: " << std::strerror(errno) << "\n";
            return 1;
        }
        written += m;
    }
}

} // namespace

int main(int argc, char* argv[]) {
    if (argc < 2) {
        return cat_fd(STDIN_FILENO, "<stdin>");
    }

    int rc = 0;
    for (int i = 1; i < argc; ++i) {
        std::string name = argv[i];
        if (name == "-") {
            if (cat_fd(STDIN_FILENO, "<stdin>") != 0) rc = 1;
            continue;
        }
        int fd = open(name.c_str(), O_RDONLY);
        if (fd < 0) {
            std::cerr << "mycat: " << name << ": " << std::strerror(errno) << "\n";
            rc = 1;
            continue;
        }
        if (cat_fd(fd, name) != 0) rc = 1;
        close(fd);
    }
    return rc;
}
```

Сорок строк, и почти все они — про обработку ошибок. Так и должно быть в системном коде: полезное действие занимает две строки, остальное — честная реакция на то, что может пойти не так.

Разберём внутренний цикл, потому что он важнее, чем кажется.

`read` возвращает три разных вида ответа, и все три надо различать. Ноль означает конец данных — файл кончился или собеседник закрыл канал. Отрицательное значение означает ошибку, и тут есть особый случай: `EINTR` говорит, что вызов прервал сигнал (глава 28), и это не настоящая ошибка — надо просто повторить. Положительное значение — количество прочитанных байт, и оно **может быть меньше** размера буфера, даже если данные ещё не кончились. Для файла на диске такое почти не случается, а вот при чтении из канала или сокета — постоянно: сколько успело прийти, столько и вернули.

Ровно та же история с `write`, и здесь она опаснее, потому что о ней чаще забывают. Вызов имеет право записать меньше, чем попросили, — например, если буфер канала заполнился наполовину. Если не проверять возвращённое значение, программа молча потеряет хвост данных. Поэтому запись обёрнута в цикл со счётчиком `written`, который двигает указатель и уменьшает остаток, пока не отдаст всё.

Возьмите этот цикл за образец. В Части V, когда мы будем писать сетевой код, он повторится почти дословно — там короткие чтения и записи станут не редкостью, а нормой.

Размер буфера в четыре килобайта выбран не наугад: это типичный размер страницы памяти и блока файловой системы. Меньше — больше системных вызовов на тот же объём; больше — заметного выигрыша уже не даёт, а память тратится.

В `main` стоит обратить внимание на две мелочи. Ошибка на одном файле не прекращает обработку остальных, но запоминается в `rc` — так ведёт себя настоящий `cat`. И имя файла хранится в `std::string`, а не в `const char*`, чтобы не думать о времени жизни, — `argv` живёт до конца программы, но привычка полезная.

32.2.1 Проверка

```
$ ./build/mycat Makefile | cmp - Makefile
mycat идентичен исходному файлу
$ echo "текст со стандартного ввода" | ./build/mycat
текст со стандартного ввода
$ ./build/mycat nosuchfile
mycat: nosuchfile: No such file or directory
$ echo $?
1
```

cmp – сравнивает наш вывод с исходным файлом побайтово и молчит, если различий нет. Для утилиты, обязанной копировать точно, это лучшая проверка.

32.2.2 Так read или ifstream?

В главе 12 мы читали файлы через <fstream>, а тут вдруг сырые системные вызовы. Есть ли смысл?

Я измерил на файле в 16 мегабайт, при прогревом кэше, все варианты собраны с -O2:

```
mycat (read/write, буфер 4 КБ):    0.01 с
ifstream + getline построчно:     0.05 с
ifstream.read блоками по 4 КБ:     0.01 с
системный cat:                     0.00 с
```

Вывод получается не тот, который обычно ожидают. Дело не в том, что системные вызовы быстрее потоков: ifstream, читающий блоками, идёт вровень с read. Дело в способе чтения. Построчный getline медленнее в пять раз, потому что на каждую строку выполняет поиск разделителя, копирование в std::string и работу форматирующего слоя, — а строк в файле триста тысяч.

Значит, правило звучит так: важен размер порции, а не выбор API. Читайте блоками — и разница между read и ifstream окажется в пределах шума.

Почему тогда в mycat всё-таки read? Потому что утилита обязана работать с любым источником, включая тот, у которого нет ни размера, ни возможности перемотки: канал, сокет, терминал. С сырым дескриптором это естественно, а ifstream вокруг стандартного ввода — уже фокус с gdbuf. Плюс мы в системной части книги, и полезно видеть механику без обёрток.

32.3 mywc

Вторая утилита считает строки, слова и байты. Выглядит просто, но в ней прячутся два поучительных места: определение слова и работа с UTF-8.

```
struct Counts {
    long long lines = 0;
    long long words = 0;
    long long bytes = 0;
    long long chars = 0;    // UTF-8 «первые байты»
};
```



```
Counts count_stream(std::istream& in) {
    Counts c;
    char ch;
    bool in_word = false;
    while (in.get(ch)) {
        ++c.bytes;

        // UTF-8: считаем «не-продолжающие» байты. (b & 0xC0) != 0x80
        if ((static_cast<unsigned char>(ch) & 0xC0) != 0x80) {
            ++c.chars;
        }

        if (ch == '\n') ++c.lines;

        bool ws = (ch == ' ' || ch == '\t' || ch == '\n' ||
                   ch == '\r' || ch == '\f' || ch == '\v');
        if (ws) {
            in_word = false;
        } else if (!in_word) {
            in_word = true;
            ++c.words;
        }
    }
    return c;
}
```

Подсчёт слов — крошечный конечный автомат с единственным флагом `in_word`. Слово начинается в тот момент, когда непробельный символ встречается после пробельного, и именно там мы увеличиваем счётчик. Такой подход правильно обрабатывает и несколько пробелов подряд, и пробелы в начале строки, и файл без завершающего перевода строки. Попытка считать слова как «число разделителей плюс один» разваливается на первом же двойном пробеле.

Строки считаются по числу символов `\n`. Отсюда известная особенность: если файл не заканчивается переводом строки, последняя строка не посчитается. Настоящий `wc` ведёт себя так же — он считает не строки, а именно завершённые переводы.

32.3.1 UTF-8 в две строки

Теперь самое интересное. Как посчитать символы, а не байты, если строка русская?

Вспомним устройство UTF-8 из главы 2. Символ кодируется одним-четырьмя байтами, и старшие биты первого байта говорят, сколько их всего:

Двоичный вид байта	Смысл
0xxxxxxx	односимвольный ASCII
110xxxxx	первый байт из двух
1110xxxx	первый байт из трёх
11110xxx	первый байт из четырёх
10xxxxxx	продолжение предыдущего

Ключевое наблюдение: продолжающий байт всегда начинается с битов 10, и никакой другой — нет. Значит, чтобы посчитать символы, достаточно посчитать байты, которые **не** начинаются с 10:

```
if ((static_cast<unsigned char>(ch) & 0xC0) != 0x80) ++c.chars;
```

Маска 0xC0 — это 11000000, она оставляет только два старших бита. Сравнение с 0x80 — с 10000000 — отвечает на вопрос «это продолжение?». Если нет, значит, начался новый символ.

Приведение к unsigned char обязательно. Тип char в C++ может быть знаковым (на x86 обычно так), и байт 0xD0 в нём станет отрицательным числом; при побитовых операциях он расширится единицами до int, и сравнение сломается. Это классическая ловушка, из-за которой «работало на моей машине» превращается в «на ARM всё поехало» — там char беззнаковый по умолчанию.

Прелесть приёма в том, что мы не декодируем UTF-8 и вообще не знаем, какие там символы. Никаких таблиц, никакой библиотеки — два бита и одно сравнение.

32.3.2 Аргументы и вывод

```
int main(int argc, char* argv[]) {
    bool show_chars = false;
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) {
        std::string a = argv[i];
        if (a == "-m") show_chars = true; // символы (POSIX wc -m)
        else if (a == "-c") show_chars = false; // байты (POSIX wc -c) — и так по умолчанию
    }
}
```

```

        else files.push_back(a);
    }

    if (files.empty()) {
        Counts c = count_stream(std::cin);
        print_counts(c, "", show_chars);
        return 0;
    }

    Counts total;
    int rc = 0;
    for (const auto& f : files) {
        std::ifstream in(f);
        if (!in.is_open()) {
            std::cerr << "mywc: " << f << ": " << std::strerror(errno) << "\n";
            rc = 1;
            continue;
        }
        Counts c = count_stream(in);
        print_counts(c, f, show_chars);
        total.lines += c.lines;
        total.words += c.words;
        total.bytes += c.bytes;
        total.chars += c.chars;
    }
    if (files.size() > 1) {
        print_counts(total, "total", show_chars);
    }
    return rc;
}

```

Флаги названы так же, как в POSIX, и это стоило исправления: у настоящего `wc` ключ `-c` означает байты (character в старом смысле, когда символ и байт совпадали), а символы — это `-m`. Наша первая версия использовала `-c` для символов, и такая утилита в чужой цепочке вела бы себя обманчиво: `mywc -c` возвращал бы не то, что `wc -c`. Совместимость с общепринятым поведением важнее удобства обозначений.

Вывод форматируется через `std::setw(7)` из `<iomanip>` — числа выравниваются по правому краю в колонке шириной семь, как это делает настоящий `wc`. Итоговая строка `total` печатается, только когда файлов больше одного.

32.3.3 Проверка

Сверимся с системной утилитой — лучший вид проверки для клона:

```
$ echo "Привет мир" | ./build/mywc
      1      2      20
$ echo "Привет мир" | wc
      1      2      20

$ echo "Привет мир" | ./build/mywc -m
      1      2      11
$ echo "Привет мир" | wc -m
      11
```

Числа сходятся с системными. Проверим их вручную: «Привет» — шесть букв, «мир» — три, каждая кириллическая буква в UTF-8 занимает два байта, плюс пробел и перевод строки по одному. Байт получается $6 \times 2 + 1 + 3 \times 2 + 1 = 20$, символов — $6 + 1 + 3 + 1 = 11$. Именно это обе утилиты и напечатали.

32.4 mygrep

Третья утилита ищет строки по регулярному выражению. Здесь появляется работа с аргументами посерьёзнее и стандартная библиотека регулярных выражений.

```
struct Options {
    bool ignore_case = false;
    bool show_line_numbers = false;
    bool invert = false;
    std::string pattern;
    std::vector<std::string> files;
};

bool parse_args(int argc, char* argv[], Options& opts) {
    int i = 1;
    while (i < argc) {
        std::string a = argv[i];
```

```

    if (a == "-i") { opts.ignore_case = true; ++i; }
    else if (a == "-n") { opts.show_line_numbers = true; ++i; }
    else if (a == "-v") { opts.invert = true; ++i; }
    else if (a == "--") { ++i; break; }
    else if (!a.empty() && a[0] == '-' && a != "--") {
        std::cerr << "mygrep: неизвестная опция: " << a << "\n";
        return false;
    }
    else break;
}
if (i >= argc) {
    std::cerr << "mygrep: нужен паттерн\n";
    return false;
}
opts.pattern = argv[i++];
while (i < argc) opts.files.push_back(argv[i++]);
return true;
}

```

Разбор аргументов написан руками, и для трёх флагов это оправданно. Когда флагов станет десяток и появятся значения через пробел (`-A 3`), стоит взять `getopt` из `<unistd.h>` — стандартную функцию, которая умеет и группировку (`-in` вместо `-i -n`), и разбор значений.

Два соглашения в этом коде стоят внимания. Двойное тире `--` означает «опции кончились, дальше только данные»; без него нельзя искать шаблон, начинающийся с минуса. Одиночное тире `-`, наоборот, опцией не считается — это имя, означающее стандартный ввод, и потому в условии стоит `a != "--`.

32.4.1 Поиск

```

long long grep_stream(std::istream& in, const std::regex& re,
                     const Options& opts, const std::string& filename) {
    std::string line;
    long long line_no = 0;
    long long printed = 0;
    bool multi_file = opts.files.size() > 1;
    while (std::getline(in, line)) {
        ++line_no;
        bool matched = std::regex_search(line, re);
    }
}

```

```

        if (matched == opts.invert) continue;

        if (multi_file && !filename.empty()) std::cout << filename << ":";
        if (opts.show_line_numbers)         std::cout << line_no << ":";
        std::cout << line << "\n";
        ++printed;
    }
    return printed;
}

```

Строчка `if (matched == opts.invert) continue;` — компактная запись двух случаев сразу. Без инверсии (`invert == false`) она пропускает строки, где совпадения не было. С инверсией — наоборот. Такой приём читается не мгновенно, зато избавляет от дублирования ветки.

`std::regex_search` ищет совпадение **где угодно** в строке, а `std::regex_match` требует, чтобы шаблон покрывал строку целиком. Для `grep` нужен первый: `mygrep error` должен находить строку `2026-07-30 error: timeout`, а не только строку, состоящую из одного слова `error`.

Имя файла печатается перед строкой только тогда, когда файлов несколько, — снова поведение настоящего `grep`, и оно осмысленно: при одном файле имя было бы шумом, при нескольких без него не разобрать, где что нашлось.

32.4.2 Компиляция шаблона

```

auto flags = std::regex::ECMAScript;
if (opts.ignore_case) flags |= std::regex::icase;

std::regex re;
try {
    re = std::regex(opts.pattern, flags);
} catch (const std::regex_error& e) {
    std::cerr << "mygrep: bad regex '" << opts.pattern << "': " << e.what() << "\n";
    return 2;
}

```

Регулярное выражение компилируется один раз, до цикла. Это не микрооптимизация: построение `std::regex` — дорогая операция, она разбирает шаблон и строит автомат. Создавать объект внутри цикла по строкам — верный способ получить программу, которая работает в сотни раз медленнее необходимого. Правило общее для всех языков, где есть регулярные выражения.

Диалект по умолчанию — ECMAScript, тот же, что в JavaScript. Он близок к расширенному POSIX, который понимает `grep -E`, но не совпадает с ним: в ECMAScript есть `\d`, `\w`, `\b`, ленивые квантификаторы `*`, а в POSIX их нет. Если нужен именно POSIX-диалект, `std::regex` умеет и его — через флаги `std::regex::extended` или `std::regex::basic`.

Ошибка в шаблоне приходит исключением `std::regex_error`, и ловить его обязательно: `mygrep "[a-z]"` без обработки просто уронил бы программу с невнятным сообщением о необработанном исключении.

32.4.3 Код возврата: три состояния

У `grep` не два исхода, а три, и это одно из самых полезных соглашений в Unix:

```
long long found = 0;

if (opts.files.empty()) {
    found = grep_stream(std::cin, re, opts, "");
    return found > 0 ? 0 : 1;
}

bool had_error = false;
for (const auto& f : opts.files) {
    ...
}
if (had_error) return 2;
return found > 0 ? 0 : 1;
```

Ноль означает «нашёл хотя бы одну строку», единица — «отработал нормально, но не нашёл ничего», двойка — «что-то сломалось: нет файла, плохой шаблон».

Различие между единицей и двойкой — не педантизм. На нём стоят все скрипты вида:

```
if grep -q ERROR app.log; then
    echo "в логе есть ошибки"
fi
```

Условие в `if` смотрит именно на код возврата. Если бы `grep` возвращал ноль всегда, ветка срабатывала бы независимо от содержимого лога. В первой версии нашей утилиты так и было — `grep_stream` возвращал ноль, чем бы дело ни кончилось. Теперь она считает найденные строки и сообщает честно:

```
$ echo "hello" | ./build/mygrep hello > /dev/null; echo $?
0
```

```
$ echo "hello" | ./build/mygrep xyz > /dev/null; echo $?  
1  
$ ./build/mygrep foo /nosuchfile > /dev/null 2>&1; echo $?  
2
```

Точно те же три числа выдаёт системный grep.

32.4.4 Сколько стоит std::regex

Честно измерим себя на фоне взрослого инструмента. Файл — журнал на 300 тысяч строк, 16 мегабайт, ищем слово timeout, обе программы собраны с оптимизацией:

```
mygrep (std::regex):    0.97 c  
системный grep:       0.16 c
```

В шесть раз медленнее. Причин две, и обе поучительные.

Первая — реализация std::regex в стандартной библиотеке действительно медленная. Она разбирает шаблон в дерево и обходит его на каждой строке, вместо того чтобы построить конечный автомат один раз. Это известная беда стандарта: интерфейс задан так, что быстрые реализации сделать сложно, и все три крупные библиотеки (libstdc++, libc++, MSVC) проигрывают специализированным движкам вроде RE2 или PCRE2 на порядок.

Вторая — системный grep не использует регулярные выражения там, где может обойтись без них. Увидев шаблон без спецсимволов, он ищет подстроку алгоритмом Бойера — Мура, который пропускает целые куски текста, не глядя на них.

Практический вывод: std::regex годится для разбора конфигов, проверки полей формы, обработки пользовательского ввода — везде, где строк единицы и тысячи. Для потоков в миллионы строк берите готовый быстрый движок или проверьте сначала простым line.find(pattern) != npos, не обходится ли задача без регулярного выражения вовсе.

32.5 myhead и умение вовремя остановиться

Четвёртая утилита короткая, но показывает свойство, которого нет у первых трёх: она читает не всё.

```
// Печатает не больше limit строк. Возвращает, сколько напечатала.  
long long head_stream(std::istream& in, long long limit) {  
    std::string line;  
    long long printed = 0;  
    while (printed < limit && std::getline(in, line)) {
```



```

        std::cout << line << "\n";
        ++printed;
    }
    return printed;
}

```

Условие `printed < limit` стоит **перед** чтением, и это принципиально: как только напечатано нужное количество, программа не делает лишнего `getline` и сразу выходит. Проверить легко — натравим её на бесконечный поток:

```

$ time (yes | ./build/myhead -n 3)
y
y
y
( yes | ./build/myhead -n 3; )  0.00s user 0.00s system 0.006 total

```

Шесть миллисекунд на бесконечном источнике. Если бы условие стояло после чтения или если бы мы, скажем, сначала собирали все строки в вектор, а потом печатали первые три, — программа не закончилась бы никогда.

Разбор аргументов заслуживает отдельного слова, потому что `-n` — первый ключ со значением:

```

if (a == "-n") {
    if (i + 1 >= argc) {
        std::cerr << "myhead: y -n нет значения\n";
        return 2;
    }
    char* end = nullptr;
    long long v = std::strtoll(argv[++i], &end, 10);
    if (!end || *end != '\0' || v < 0) {
        std::cerr << "myhead: плохое число строк: " << argv[i] << "\n";
        return 2;
    }
    limit = v;
} else if (a.size() > 2 && a.compare(0, 2, "-n") == 0) {
    // Слитная форма: -n5
    ...
}

```

`std::strtoll` из `<cstdlib>` разбирает число и через второй параметр сообщает, где остановилась. Проверка `*end != '\0'` ловит мусор вроде `-n 5x`: без неё `strtoll` вернул бы пятёрку и молча

съел бы х. Именно поэтому `strtoll` предпочтительнее `atoi`, который на плохом вводе просто возвращает ноль и не говорит ни слова.

Слитная форма `-n5` разобрана отдельно, потому что традиция допускает и её. Когда таких ключей становится много, ручной разбор надоедает — и тогда берут `getopt`:

```
#include <unistd.h>

int opt;
while ((opt = getopt(argc, argv, "n:iv")) != -1) {
    switch (opt) {
        case 'n': limit = std::atoll(optarg); break;
        case 'i': ignore_case = true; break;
        case 'v': invert = true; break;
        default: return 2;
    }
}
// после цикла optind указывает на первый неопционный аргумент
```

Двоеточие после буквы в строке `"n:iv"` означает «у этого ключа есть значение». `getopt` сам разбирает и слитную форму, и группировку `-iv`, и `--` как конец опций. Для утилиты с пятью и более ключами это правильный выбор.

Наконец, при нескольких файлах `head` печатает заголовки — и мы повторяем формат в точности:

```
$ ./build/myhead -n 2 Makefile include/pipeline.h
==> Makefile <==
# Makefile для demo-shell.
# Зеркало стиля demo-rpg/Makefile: -MMD для авто-зависимостей,

==> include/pipeline.h <==
#ifndef SHELL_PIPELINE_H
#define SHELL_PIPELINE_H
```

Совпадение с системной утилитой проверяется всё тем же способом:

```
$ head -3 Makefile | cmp - <(. /build/myhead -n 3 Makefile)
совпадает с head -3
```

32.6 Что происходит, когда читатель уходит раньше

У раннего выхода есть последствие для соседа слева по конвейеру. Соберём цепочку, где `myscat` читает шестнадцатимегабайтный журнал, а `myhead` берёт одну строку:

```
$ ./build/mycat big.log | ./build/myhead -n 1
0 debug retry ok timeout info timeout failed info
$ echo "коды: ${PIPESTATUS[0]} ${PIPESTATUS[1]}"
коды: 141 0
```

Код 141 у первой команды — это, как мы считали в главе 27, $128 + 13$, то есть SIGPIPE. `myhead` напечатал строку и закрыл свой стандартный ввод; `myscat` попытался писать в канал без читателя и был убит ядром.

Это не поломка, а именно тот механизм, благодаря которому `head` на гигабайтном файле возвращается мгновенно. Никто не дочитывает файл до конца — источник просто убивают, как только результат перестаёт быть нужным.

Для утилиты-фильтра отсюда следует практический вывод: не надо ловить SIGPIPE и продолжать работу. Смерть от него — правильное поведение в конвейере. А вот для сервера или демона, который пишет в сокет, всё наоборот — там сигнал нужно отключить и обрабатывать EPIPE как обычную ошибку. Мы этим займёмся в Части V.

32.7 Буферизация: почему вывод иногда «залипает»

Есть ещё одна особенность, о которой стоит знать заранее, потому что она порождает жалобы вида «программа зависла, ничего не печатает».

Стандартный вывод буферизуется по-разному в зависимости от того, куда направлен. Если это терминал — буферизация построчная, и каждая строка появляется сразу. Если файл или канал — буферизация полная, и данные уходят порциями по несколько килобайт.

Проверить, куда мы пишем, можно вызовом `isatty` из `<unistd.h>`:

```
#include <unistd.h>

if (isatty(STDOUT_FILENO)) {
    // человек смотрит на экран
} else {
    // нас перенаправили в файл или конвейер
}
```

Утилиты этим пользуются. Тот же `grep` при выводе в терминал подсвечивает совпадения цветом, а при перенаправлении — нет, чтобы управляющие последовательности не портили файл. Проверяется это ровно через `isatty`.

Побочный эффект полной буферизации проявляется на длинных конвейерах с долгой работой. Скажем, `./monitor | grep ERROR` может молчать минутами: `monitor` накапливает вывод в буфере, `grep` ждёт данных, а вы смотрите на пустой экран и думаете, что всё повисло. Лечится это со стороны источника — сбросом буфера после каждой записи (`std::cout.flush()` или `std::endl`) либо переключением потока в небуферизованный режим. Снаружи помогает утилита `stdbuf -oL` на Linux, которая заставляет чужую программу перейти на построчный режим.

В наших утилитах специальных мер нет, и это осознанно: они работают быстро и завершаются, так что заметить задержку негде. Но если бы `mygrep` работал на потоке, приходящем в реальном времени, флаг `-L` для построчного вывода стал бы первой же просьбой пользователей.

32.8 Всё вместе

Соберём цепочку из наших программ и запустим её в нашей оболочке:

```
myshell$ ./build/mycat Makefile | ./build/mygrep -i make | ./build/mywc
      2      11      124
myshell$ ./build/mygrep -n "^CXX" Makefile
5:CXX      := g++
6:CXXSTD   := -std=c++11
18:CXXFLAGS := $(CXXSTD) $(WARNINGS) $(OPT) -Iinclude -MMD -MP
myshell$ echo "Привет, мир" | ./build/mywc -m
      1      2      12
```

Первая строка — полный круг. Наша оболочка разобрала команду, создала два канала, запустила три процесса, переставила им дескрипторы. Первый процесс прочитал файл системным вызовом `read` и отдал байты в канал. Второй прочитал их оттуда, прогнал через регулярное выражение и отдал дальше. Третий посчитал и напечатал в терминал. Каждая часть этой цепочки написана нами и разобрана в предыдущих главах.

Обратите внимание, что утилиты ничего не знают о своём окружении. `mygrep` не подозревает, что слева `mycat`, а справа `mywc`; для него это просто стандартный ввод и стандартный вывод. Поменяйте местами, вставьте системный `sort` посередине, отправьте результат в файл — всё продолжит работать. Это и есть та самая композиция, ради которой Unix устроен именно так.

32.9 Что не сделано

Настоящие утилиты обросли десятками ключей. У cat есть -n для нумерации строк, -E для показа концов строк, -T для табуляций. У wc — ключи -l, -w, -c, -m для вывода одного числа вместо трёх и -L для максимальной длины строки. У grep — рекурсивный обход каталогов -r, вывод только имён файлов -l, подсчёт совпадений -c, показ соседних строк -A и -B, поиск фиксированной строки без регулярных выражений -F.

Ни одна из этих возможностей не требует новых знаний — это работа с аргументами и небольшие изменения в цикле. Несколько штук вынесены в упражнения; остальное допишете, когда понадобится.

Отдельно упомяну приём, который иногда даёт заметный выигрыш на больших файлах, — отображение файла в память:

```
#include <sys/mman.h>
#include <sys/stat.h>

int fd = open("big.log", O_RDONLY);
struct stat st;
fstat(fd, &st);
char* data = static_cast<char*>(
    mmap(nullptr, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0));
// теперь data[0..st.st_size-1] — содержимое файла, как обычный массив
munmap(data, st.st_size);
close(fd);
```

После mmap файл доступен как массив байтов: ядро подгружает страницы по мере обращения, и лишнего копирования из своего кэша в наш буфер не происходит. Для поиска по большому файлу это иногда даёт двукратное ускорение. Но у приёма есть жёсткое ограничение: отобразить можно только то, у чего есть размер и произвольный доступ. Канал, сокет, терминал отобразить нельзя — а именно из них наши утилиты чаще всего и читают. Плюс на файле, лежащем на сетевой файловой системе, исчезновение файла даёт SIGBUS вместо аккуратной ошибки. Поэтому в универсальной утилите-фильтре read остаётся правильным выбором, а mmap берут для специализированных задач.

32.10 Часть III закончена

Шесть глав назад у нас не было ничего, кроме желания написать оболочку. Сейчас в demo-shell/ лежит около тысячи строк кода: сама оболочка в src/, четыре утилиты в utils/, тесты токенизатора в tests/.

Что теперь умеет myshell: запускать программы, соединять их в цепочки любой длины, перенаправлять ввод, вывод и поток ошибок в шести формах, разбирать кавычки и экранирование, выполнять шесть встроенных команд, переживать Ctrl+C, помнить историю между запусками.

Что теперь умеете вы. Из системного интерфейса Unix — fork, exec, wait, pipe, dup2, open, read, write, close, chdir, getcwd, setenv, sigaction, kill. Из общих приёмов — токенизатор на конечном автомате, разделение разбора и выполнения, обработка коротких чтений и EINTR, подсчёт символов UTF-8 без декодирования, регулярные выражения стандартной библиотеки и понимание их цены.

Это и есть системное программирование в его повседневном виде. Той же техникой пишутся утилиты сборки, обёртки для запуска сервисов, обработчики логов и всё, что живёт в командной строке.

32.11 Главные правила главы

1. Утилита без аргументов читает стандартный ввод — это то, что позволяет ей встать в конвейер.
2. read и write могут обработать меньше запрошенного; обе всегда оборачивают в цикл.
3. EINTR — не ошибка, а просьба повторить вызов.
4. Читайте блоками, а не по строкам: разница в пять раз, и она не про выбор между read и ifstream.
5. Символы UTF-8 считаются подсчётом байтов, у которых старшие два бита не равны 10.
6. Приводите char к unsigned char перед побитовыми операциями — знаковость char зависит от платформы.
7. Регулярное выражение компилируют один раз до цикла; std::regex медленный, для больших объёмов ищите альтернативу.
8. Коды возврата — часть интерфейса программы: у grep ноль, единица и двойка означают три разные вещи, и на этом стоят скрипты.

32.12 Упражнения

1. Соберите утилиты (make) и сверьте каждую с системной: mycat через str, mywc на файле с кириллицей, mygrep по кодам возврата. Найдите хотя бы одно расхождение с системной версией и объясните его.
2. Проверьте обработку коротких чтений: запустите ./build/mycat < /dev/urandom | head -c 1000000 | ./build/mywc -c. Убедитесь, что счёт сходится и ничего не потерялось.

3. Добавьте в `mycat` ключ `-n` — нумерацию строк, как у настоящего `cat`. Учтите, что нумеровать надо сквозным счётом по всем файлам, а не начинать заново для каждого.
4. Добавьте в `mywc` ключи `-l`, `-w`, `-c`, `-m` в их POSIX-смысле: при указании любого из них печатается только запрошенное число, при нескольких — только они и в фиксированном порядке.
5. Добавьте в `mygrep` ключ `-c` (печатать только количество совпадений) и `-l` (только имена файлов, где нашлось). Проверьте, что код возврата остаётся правильным.
6. Замерьте `mygrep` на своём большом файле и сравните с системным `grep`. Затем добавьте оптимизацию: если в шаблоне нет спецсимволов регулярных выражений, искать через `line.find`. Насколько сократился разрыв?
7. (Сложнее) Добавьте в `myhead` ключ `-c N` — печатать первые `N` **байт**, а не строк. Проверьте на бинарном файле, что счёт точный и лишнего не читается.
8. (Сложнее) Напишите `mytail` — последние `N` строк. В отличие от `head`, конец потока заранее неизвестен: понадобится кольцевой буфер на `N` строк. Для файла можно схитрить и читать с конца через `lseek`, но сначала сделайте общий вариант, работающий и с каналом.

32.13 Что дальше

Начинается Часть IV — мини-СУБД. Восемь глав, за которые мы напишем `demo-db/`: страничный файл с кэшем, B+-дерево, журнал упреждающей записи, крошечный SQL и планировщик запросов, умеющий выбирать между индексом и полным перебором.

Это будет другая работа. В Части III важнее всего было договориться с операционной системой; в Части IV главными станут корректность и производительность. Что значит «база не теряет данные при выключении питания», почему поиск по индексу быстрее перебора в сорок раз и как выглядит EXPLAIN изнутри — всё это мы построим сами, начиная с пустого файла и одной структуры на диске.

Глава 33

Глава 32. Бинарный формат данных

Начинается Часть IV, и в ней восемь глав мы будем строить базу данных. Настоящую, хотя и маленькую: со страничным файлом на диске, кэшем страниц, B+-деревом, журналом упреждающей записи, крошечным диалектом SQL, планировщиком запросов и командой EXPLAIN. Лежать всё это будет в `demo-db/`, а называться `mydb`.

Работа предстоит другая, чем в Части III. Там мы договаривались с операционной системой; здесь главными станут два слова — корректность и производительность. Что значит «база не теряет данные, даже если выдернуть питание посреди записи». Почему поиск по индексу быстрее перебора в десятки раз. Как выглядит план запроса изнутри и почему он иногда выбирает не то, что вы ожидали.

Начнём с самого низа — с байтов. В Части II мы сохраняли мир RPG в текстовом виде, и это было правильным решением: файл читается глазами, отлаживается тривиально, а размеры игрушечные. Для базы данных такой формат не годится, и в этой главе мы разберёмся почему, а заодно напишем фундамент — код, который укладывает числа и строки в байты так, чтобы прочитать их можно было завтра, на другой машине и другим компилятором.

33.1 Текст против байтов

Возьмём одну запись: `id = 42, name = "Alice"`.

В текстовом виде это девять байт: `42,Alice`. Компактно и понятно. Но чтобы получить из этого число 42, программа должна пройти по символам, проверить каждый на принадлежность цифрам, умножить накопленное на десять, прибавить очередную цифру — и всё это ради одного маленького целого. Разбор одного числа занимает десятки операций.

В бинарном виде та же запись выглядит так:


```
2A 00 00 00 05 00 00 00 41 6C 69 63 65
[--- id ---] [-- длина -] [---- "Alice" ----]
```

Тринадцать байт — больше, чем текстовых девять. Зато чтобы получить число, достаточно взять четыре байта по известному смещению и сложить их в `uint32_t`. Никакого разбора: одно чтение, три сдвига.

Разница в размере обманчива и зависит от данных. Число 42 в тексте занимает два байта, а в бинарном виде четыре. Зато 1234567890 в тексте — десять байт, а в бинарном по-прежнему четыре. Для базы данных с миллионами записей выигрывает бинарный формат, и не столько по размеру, сколько по цене чтения: разобрать миллион текстовых чисел — это миллион маленьких разборов, а прочитать миллион бинарных — это миллион обращений к памяти.

Платить приходится читаемостью. Бинарный файл не откроешь в редакторе, не поправишь руками, не помотришь глазами, что там не так. Поэтому в этой главе будет отдельный раздел про то, чем смотреть.

33.1.1 Насколько именно быстрее

Разговоры о скорости стоят немного без чисел, поэтому измерим. Программа готовит миллион 32-битных чисел в двух видах — текстом через пробел и четырьмя байтами каждое, — а потом складывает их, разбирая тем и другим способом:

```
// разбор текста
std::istringstream in(text);
std::uint32_t v;
while (in >> v) sum1 += v;

// чтение байтов
for (int i = 0; i < N; ++i) {
    const unsigned char* p = bin.data() + i * 4;
    std::uint32_t v = static_cast<std::uint32_t>(p[0]) |
                     (static_cast<std::uint32_t>(p[1]) << 8) |
                     (static_cast<std::uint32_t>(p[2]) << 16) |
                     (static_cast<std::uint32_t>(p[3]) << 24);
    sum2 += v;
}
```

Сборка с `-O2`, результат:

текстом: 10719377 байт

бинарно: 4000000 байт

разбор текста: 347 мс

чтение байтов: 451 мкс

суммы совпали

Данных в текстовом виде почти в три раза больше, а разбор занимает в семьсот с лишним раз больше времени. Триста сорок семь миллисекунд против половины миллисекунды — на миллионе чисел. Именно из-за этой разницы ни одна база данных не хранит строки на диске в текстовом виде: цена разбора платится при каждом чтении.

Оговорюсь: `std::istream` — не самый быстрый способ разбирать числа, ручной разбор был бы в несколько раз быстрее. Но и он не приблизится к чтению готовых байтов, потому что байты читать не надо — их надо просто взять.

33.2 Что компилятор делает со структурой

Первое, что приходит в голову: раз нужен бинарный формат — возьмём структуру и запишем её в файл как есть. Прежде чем объяснять, почему так делать нельзя, посмотрим, что вообще лежит в памяти.

```
struct Row {  
    std::uint32_t id;  
    std::uint16_t age;  
    std::uint64_t balance;  
};
```

Наивный подсчёт даёт $4 + 2 + 8 = 14$ байт. Спросим у компилятора — программа `utils/binary_demo.cpp` в нашем проекте это и делает:

```
$ ./build/binary_demo  
=== Как компилятор разложил Row ===  
sizeof(Row)           = 16  
alignof(Row)          = 8  
offsetof(Row, id)      = 0  
offsetof(Row, age)     = 4  
offsetof(Row, balance) = 8  
sizeof(RowPacked)      = 14
```

Шестнадцать, а не четырнадцать. И поле `balance` начинается не с шестого байта, а с восьмого — между ним и `age` образовалась дыра в два байта.

Виновато выравнивание. У каждого типа есть требование к адресу, по которому он может лежать: `uint8_t` — по любому, `uint16_t` — по чётному, `uint32_t` — кратному четырём, `uint64_t`

— кратному восьми. Компилятор обязан разложить поля так, чтобы требование соблюдалось для каждого, и если для этого нужно вставить пустые байты — он их вставляет.

Требование не выдуманно на пустом месте. На части процессоров обращение к невыровненному числу — это аппаратная ошибка и падение программы; на x86 оно работает, но медленнее, потому что процессору приходится делать два обращения к памяти и склеивать результат. Пустые байты — плата за то, чтобы каждое обращение было одним.

Дыры могут появиться и в конце структуры. Если после последнего поля размер не кратен `alignof`, компилятор добавит хвост — иначе в массиве второй элемент оказался бы невыровненным.

Инструменты для разглядывания всего этого — `sizeof`, `alignof` и макрос `offsetof` из `<stddef>`. Первые два — операторы языка, третий достался в наследство от С и принимает имя типа и имя поля.

Есть и обратная возможность — потребовать выравнивание побольше:

```
struct alignas(64) CacheLine {
    int data;
};
// sizeof(CacheLine) == 64
```

Это нужно, когда структура должна занимать отдельную строку кэша процессора: в многопоточном коде две переменные, попавшие в одну строку, начинают мешать друг другу, даже если логически не связаны. Об этом ложном разделении речь пойдёт в главе 42.

Уменьшить выравнивание стандартными средствами C++ нельзя, но компиляторы дают расширения. У GCC и Clang это `__attribute__((packed))`, у MSVC — `#pragma pack`:

```
struct __attribute__((packed)) RowPacked {
    std::uint32_t id;
    std::uint16_t age;
    std::uint64_t balance;
};
// sizeof(RowPacked) == 14 — дыр нет
```

Четырнадцать байт, как мы и считали изначально. Только теперь `balance` лежит по адресу, кратному шести, и каждое обращение к нему компилятор вынужден собирать из кусочков.

Здесь принято говорить, что плотная упаковка обходится дорого. Проверим — миллион структур, двадцать проходов, суммируем поле `balance`:

```
выровненная: sizeof=16, 21 мс
packed      : sizeof=14, 17 мс
```

Плотная версия оказалась быстрее. Не потому, что невыровненное чтение бесплатно, а потому, что на x86 оно стоит очень мало, зато структура на два байта короче — и в кэш процессора помещается больше записей. При обходе миллиона элементов подряд выигрывает тот, у кого меньше данных.

Не делайте из этого вывод «пакуйте всё». Тот же тест на ARM или на процессоре без быстрого невыровненного доступа дал бы обратный результат, а на некоторых архитектурах программа бы просто упала. Вывод правильнее сформулировать иначе: утверждения о производительности проверяются измерением на своей платформе, а не берутся из общих соображений.

Плотная упаковка по-настоящему оправдана, когда структура обязана точь-в-точь соответствовать чужому формату — например, заголовку сетевого пакета, где каждое поле стоит на месте, предписанном стандартом. Для своих данных лучше не трогать выравнивание вовсе и, как мы сейчас увидим, вообще не полагаться на раскладку структуры.

33.3 Почему структуру нельзя писать в файл

Соблазн велик:

```
Row r{42, 25, 1000};
std::ofstream out("data.bin", std::ios::binary);
out.write(reinterpret_cast<const char*>(&r), sizeof(r));
```

Это компилируется, работает и даже читается обратно — на той же машине, тем же компилятором, в тот же день. Дальше начинаются неприятности, и их четыре.

33.3.1 Дыры не инициализируются

Те два байта между `age` и `balance` компилятор не обязан ничем заполнять. Что там лежит — вопрос везения: остатки предыдущих значений на стеке, куски освобождённой памяти, что угодно. Посмотрим своими глазами. Демо-программа специально пачкает память байтами `0xAB`, потом кладёт туда структуру и заполняет поля:

```
=== Мусор в padding-байтах ===
00000000 2a 00 00 00 19 00 ab ab e8 03 00 00 00 00 00 00 |*.....|
байты 6 и 7 — padding, их никто не инициализировал
```

Всё на месте: `2a 00 00 00` — это 42, `19 00` — это 25, `e8 03 ...` — это 1000. А между ними торчат `ab ab` — тот самый мусор, который мы туда положили до присваивания полей.

Для файла это означает две беды. Побайтовое сравнение двух логически одинаковых записей может дать «не равны». А ещё в дыры может утечь то, чего в файле быть не должно: если

раньше в этой памяти лежал пароль или кусок чужих данных, он окажется в файле. Реальные утечки такого рода случались в сетевых протоколах не раз.

33.3.2 Размеры типов разные

`sizeof(long)` равен восьми на Linux и четырём на Windows. Записали на одной системе, читаете на другой — сдвиг на четыре байта и полный мусор дальше. Лечится это просто: в форматах используются только типы фиксированного размера из `<stdint.h>` — `std::int32_t`, `std::uint64_t` и их родня. О них шла речь в главе 8, и в нашей базе других не будет.

33.3.3 Порядок байтов

Число `0x12345678` в памяти x86 и ARM лежит как `78 56 34 12` — младший байт первым, это порядок `little-endian`. Есть машины с обратным порядком, `big-endian`, где то же число лежит как `12 34 56 78`; в этом же порядке принято передавать числа в сетевых протоколах, отчего его называют сетевым.

Записали структуру на одной машине, прочитали на другой — числа развернулись. Причём не сломались явно, а тихо превратились в другие числа, что гораздо хуже.

33.3.4 Разные компиляторы раскладывают по-разному

Правила выравнивания задаются не языком, а соглашением о двоичном интерфейсе (ABI) конкретной платформы. Для простых структур все современные компиляторы совпадают, но стоит появиться битовым полям, наследованию или необычным типам — и `sizeof` начинает различаться между GCC и MSVC.

Вывод простой и без исключений: **структуру нельзя писать в файл как есть**, если файл проживёт дольше одного запуска или уедет на другую машину. Писать надо поле за полем, в явно заданном порядке, байт за байтом.

33.4 Явная сериализация

Весь фундамент умещается в один заголовок. `include/binary.h`:

```
#ifndef DB_BINARY_H
#define DB_BINARY_H

#include <stdint.h>
#include <cstring>
#include <string>
```

```
#include <vector>

namespace db {

    // Запись/чтение чисел в little-endian с фиксированным размером.
    // Не полагаемся на raw struct dump'ы – портбельность важнее микро-эффективности.

    inline void write_u16(std::vector<unsigned char>& out, std::uint16_t v) {
        out.push_back(static_cast<unsigned char>(v & 0xFFu));
        out.push_back(static_cast<unsigned char>((v >> 8) & 0xFFu));
    }

    inline void write_u32(std::vector<unsigned char>& out, std::uint32_t v) {
        for (int i = 0; i < 4; ++i) {
            out.push_back(static_cast<unsigned char>((v >> (i * 8)) & 0xFFu));
        }
    }

    inline void write_u64(std::vector<unsigned char>& out, std::uint64_t v) {
        for (int i = 0; i < 8; ++i) {
            out.push_back(static_cast<unsigned char>((v >> (i * 8)) & 0xFFu));
        }
    }

    inline void write_string(std::vector<unsigned char>& out, const std::string& s) {
        // Длина (u32) + байты.
        write_u32(out, static_cast<std::uint32_t>(s.size()));
        out.insert(out.end(), s.begin(), s.end());
    }

    inline std::uint16_t read_u16(const unsigned char* p) {
        return static_cast<std::uint16_t>(p[0]) |
            (static_cast<std::uint16_t>(p[1]) << 8);
    }

    inline std::uint32_t read_u32(const unsigned char* p) {
        std::uint32_t v = 0;
        for (int i = 0; i < 4; ++i) {
```

```

        v |= static_cast<std::uint32_t>(p[i]) << (i * 8);
    }
    return v;
}

inline std::uint64_t read_u64(const unsigned char* p) {
    std::uint64_t v = 0;
    for (int i = 0; i < 8; ++i) {
        v |= static_cast<std::uint64_t>(p[i]) << (i * 8);
    }
    return v;
}

inline std::string read_string(const unsigned char* p, std::size_t& consumed) {
    std::uint32_t len = read_u32(p);
    std::string s(reinterpret_cast<const char*>(p + 4), len);
    consumed = 4 + len;
    return s;
}

} // namespace db

#endif

```

Логика записи одинакова во всех трёх функциях: берём число, отрезаем от него по байту начиная с младшего, кладём в буфер. Выражение $(v \gg (i * 8)) \& 0xFF$ при $i = 0$ даёт младший байт, при $i = 1$ — следующий, и так далее. Чтение делает обратное: берёт байты по порядку и сдвигает каждый на своё место.

Ключевое свойство этого кода — он не зависит от того, как число лежит в памяти. Мы не смотрим на память числа, мы вычисляем байты арифметикой. Поэтому на big-endian машине функции выдадут ровно тот же файл, и прочитают его так же. Порядок байтов в файле мы задали сами, и он little-endian — не потому, что так лежит на x86, а потому, что мы так написали.

Буфер — `std::vector<unsigned char>`, и знаковость тут важна. У `char` она зависит от платформы, а с отрицательными значениями побитовые операции ведут себя неожиданно: при расширении до `int` знаковый байт `0xD0` превращается в `0xFFFFFD0`. С `unsigned char` таких сюрпризов нет.

Функция `read_string` возвращает через параметр `consumed` количество съеденных байт — вызывающему нужно знать, где начинается следующее поле. Приём распространённый, хотя в C++17 его вытесняет возврат пары или `string_view` на исходный буфер.

Проверим на деле — тот же `binary_demo` пишет три значения подряд и печатает шестнадцатеричный дамп:

=== Явная сериализация ===

записано 21 байт

```
00000000  78 56 34 12 ef be ad de  be ba fe ca 05 00 00 00  |xV4.....|
00000010  68 65 6c 6c 6f                                     |hello|
```

читаем обратно:

```
u32 = 0x12345678
```

```
u64 = 0xcafebabedeadeadbeef
```

```
str = "hello" (9 байт)
```

Каждый байт на своём месте: 78 56 34 12 — это 0x12345678 младшим байтом вперёд, затем восемь байт числа, затем четыре байта длины строки (05 00 00 00) и пять байт самой строки. Двадцать один байт, ни одного лишнего.

33.4.1 Строки с префиксом длины

Строка записана как длина плюс тело, а не как в C — с нулевым байтом в конце. Разница принципиальная.

Нулевой байт в конце заставляет читателя идти по строке и искать этот ноль — то есть заранее не известно, сколько читать. Ещё хуже, что строка с нулевым байтом внутри становится непредставимой, а в базе данных это вполне законные данные: двоичный ключ, кусок картинки, что угодно.

Префикс длины снимает обе проблемы. Прочитали четыре байта — знаем, сколько читать дальше, можем сразу выделить память нужного размера и прочитать одним куском. Любые байты внутри допустимы. Именно так устроены почти все двоичные форматы, от Protocol Buffers до сетевого протокола, который мы напишем в главе 44.

33.4.2 Знаковые числа

Отдельных функций для знаковых чисел в заголовке нет, и они не нужны:

```
write_u64(buf, static_cast<std::uint64_t>(some_int64));
std::int64_t back = static_cast<std::int64_t>(read_u64(p));
```


Работает это благодаря дополнительному коду, о котором шла речь в главе 2. Число `-1` в нём представлено единичными битами во всех разрядах; приведение к беззнаковому типу битов не меняет, оно лишь меняет их истолкование. Записали `ff ff ff ff ff ff ff ff`, прочитали обратно, привели к знаковому — получили `-1`.

=== Отрицательные числа ===

```
00000000 ff ff ff ff ff ff ff ff 00 00 00 00 00 00 80 |.....|
```

прочитали обратно: `-1`

Вторые восемь байт — это `INT64_MIN`, самое маленькое представимое число: единица в старшем бите и нули во всех остальных. Обратите внимание, что старший бит оказался в последнем байте, — это и есть порядок `little-endian`.

Начиная с C++20 дополнительный код закреплён в стандарте, до этого формально допускались и другие представления, но живых платформ с ними не осталось.

33.4.3 Числа с плавающей точкой и правило про темсру

Сдвигами можно разобрать целое число, но не `double` — сдвигать вещественные типы нельзя. А хранить их иногда нужно.

Приём тут такой: взять биты числа как целое, а дальше действовать как обычно. Соблазнительный способ взять биты — привести указатель:

```
double d = 3.14159;
std::uint64_t bits = *reinterpret_cast<std::uint64_t*>(&d); // так нельзя
```

Так делать нельзя, и дело не в занудстве. Стандарт запрещает читать объект одного типа через указатель на несовместимый тип — правило называется строгим соответствием типов (`strict aliasing`). Компилятор им пользуется при оптимизации: он вправе считать, что запись в `double` не может изменить то, что читается через `uint64_t*`, и переставить операции местами. Код при этом обычно работает — до тех пор, пока однажды не перестанет, причём только в сборке с оптимизацией.

Законный способ один — скопировать байты:

```
double d = 3.14159;
std::uint64_t bits = 0;
std::memcpy(&bits, &d, sizeof(bits));
// дальше bits пишем через write_u64
```

```
$ ./aliasing
```

```
double 3.14159 в битах: 0x400921f9f01b866e
```

```
обратно: 3.14159
первый байт числа 1: 1 → little-endian
```

Опасение, что `memscr` — это дорогой вызов функции, беспочвенно: при известном размере компилятор превращает его в одну команду пересылки. Я проверил ассемблер, который выдал `g++ -O2` для этой программы: ни одного вызова `memscr` в нём нет.

Тот же приём даёт законный способ узнать порядок байтов своей машины: положить в `uint32_t` единицу и посмотреть на первый байт через `memscr`. Если там единица — младший байт идёт первым, машина `little-endian`.

С форматом чисел с плавающей точкой на практике проблем не бывает: все живые платформы используют IEEE 754, и биты `double` означают на них одно и то же. Проверить это в коде можно через `std::numeric_limits<double>::is_iec559`.

33.4.4 Сетевой порядок байтов

Раз речь зашла о порядке, стоит упомянуть готовые функции из `<arpa/inet.h>`:

```
std::uint32_t nbo = htonl(0x12345678); // из порядка машины в сетевой
std::uint32_t hbo = ntohl(nbo);       // обратно
```

`htonl` и `ntohl` переводят между порядком машины и сетевым (`big-endian`); есть ещё пара `htons/ntohs` для 16-битных. На `big-endian` машине они ничего не делают, на `little-endian` переставляют байты.

Пользоваться ими имеет смысл, когда формат задан снаружи и требует сетевого порядка, — так устроены заголовки TCP, кадры HTTP/2 и почти все старые протоколы. Наш формат файла мы придумываем сами, поэтому берём `little-endian`: на подавляющем большинстве машин это порядок процессора, и лишних перестановок не будет.

33.5 Чем смотреть в бинарный файл

Отладка бинарного формата без инструмента невозможна, поэтому инструмент стоит завести сразу. В системе есть `xxd` (и `hexdump -C`, если `xxd` не установлен):

```
$ xxd /tmp/waldemo/data.wal
00000000: 1100 0000 0101 0000 0000 0000 0064 0000 .....d..
00000010: 0000 0000 00fe 9b83 7911 0000 0001 0200 .....y.....
00000020: 0000 0000 0000 c800 0000 0000 0000 f900 .....
00000030: 0bef ..
```

Слева адрес, в середине байты, справа те же байты как символы — точка вместо непечатаемых. Это журнал нашей базы после двух вставок, и в нём уже можно кое-что разобрать: 1100 0000 — длина записи (17), 01 — тип «вставка», дальше восемь байт ключа (единица), восемь байт значения (64 — это 100), затем четыре байта контрольной суммы. Вторая запись начинается со смещения 25 и содержит ключ 2 и значение 200 (с8). Формат журнала мы разберём в главе 35, а пока важно другое: открыв файл так, вы за минуту находите смещение, где формат разъехался.

Иметь такую же печать внутри программы удобно вдвойне: в отладчике не всегда есть ххd, а вывести дамп куска памяти прямо из кода — дело двадцати строк:

```
void hexdump(const unsigned char* data, std::size_t len) {
    for (std::size_t i = 0; i < len; i += 16) {
        std::cout << std::hex << std::setw(8) << std::setfill('0') << i << " ";

        for (std::size_t j = 0; j < 16; ++j) {
            if (i + j < len) {
                std::cout << std::setw(2) << std::setfill('0')
                    << static_cast<int>(data[i + j]) << " ";
            } else {
                std::cout << " ";
            }
            if (j == 7) std::cout << " ";
        }

        std::cout << " |";
        for (std::size_t j = 0; j < 16 && i + j < len; ++j) {
            unsigned char c = data[i + j];
            std::cout << static_cast<char>((c >= 32 && c < 127) ? c : '.');
        }
        std::cout << "|\n";
    }
    std::cout << std::dec << std::setfill(' ');
}
```

Тонкость одна, и она про манипуляторы потоков. `std::hex`, `std::setfill` и `std::setw` ведут себя по-разному: первые два переключают режим потока навсегда, а `setw` действует только на ближайший вывод. Поэтому `setw` стоит перед каждым числом, а вот вернуть поток в десятичный режим и восстановить заполнитель нужно один раз в конце — иначе весь последующий вывод программы окажется шестнадцатеричным с ведущими нулями. Ошибка распространённая и ищется долго.

Приведение `static_cast<int>(data[i + j])` обязательно: без него `std::cout` напечатал бы байт как символ, а не как число.

33.6 Метка формата и версия

Последнее, что нужно заложить в фундамент, — способ узнать, что за файл нам подсунули.

Мы будем открывать файл и читать из него страницы, доверяя, что там наши данные. А если это чужой файл? Или файл, записанный прошлой версией нашей же программы, где формат был другим? Без проверки программа честно прочитает мусор и начнёт делать вид, что это дерево, — с непредсказуемыми последствиями вплоть до бесконечного цикла.

Лечится это меткой в самом начале. В нашей базе нулевая страница отведена под метаданные дерева, и первые восемь байт мы разложили так: четыре байта — номер корневой страницы, затем три байта метки MDB и один байт версии формата.

```
constexpr unsigned char MAGIC_0 = 'M';
constexpr unsigned char MAGIC_1 = 'D';
constexpr unsigned char MAGIC_2 = 'B';
constexpr unsigned char FORMAT_VERSION = 1;

void BPlusTree::save_metadata() {
    Page meta = make_page();
    write_u32_at(meta, 0, root_page_id_);
    meta[4] = MAGIC_0;
    meta[5] = MAGIC_1;
    meta[6] = MAGIC_2;
    meta[7] = FORMAT_VERSION;
    pm_.write_page(0, meta);
}

void BPlusTree::load_metadata() {
    Page meta = pm_.read_page(0);
    if (meta[4] != MAGIC_0 || meta[5] != MAGIC_1 || meta[6] != MAGIC_2) {
        throw PageError("это не файл mydb (нет метки MDB в странице 0)");
    }
    if (meta[7] != FORMAT_VERSION) {
        throw PageError("версия формата " + std::to_string(static_cast<int>(meta[7])) +
            " не поддерживается (нужна " +
            std::to_string(static_cast<int>(FORMAT_VERSION)) + ")");
    }
}
```

```

    }
    root_page_id_ = read_u32_at(meta, 0);
}

```

Проверка стоит два сравнения и окупается первым же случаем, когда кто-то укажет базе на посторонний файл:

```

$ ./build/mydb /tmp/aliendir
mydb REPL — таблица 'items(id INT PRIMARY KEY, value INT)' по умолчанию.
Open: /tmp/aliendir
Не удалось открыть базу в /tmp/aliendir: это не файл mydb (нет метки MDB в странице 0)
$ echo $?
1

```

Отдельная польза от версии — она даёт путь к развитию формата. Когда в будущем понадобится добавить в узел новое поле, вы поднимете версию до двойки; старая программа честно откажется читать новый файл, а новая сможет распознать единицу и прочитает старый файл по старым правилам. Без версии обе программы будут читать чужие данные и делать вид, что всё в порядке.

Ту же метку мы уже заводили в формате сохранения RPG в главе 24 — там она называлась RPGSAVE. Приём универсальный: любой формат, который проживёт дольше одного запуска, начинается с метки и версии.

33.7 Переменная длина: как сэкономить на маленьких числах

У формата с фиксированной шириной есть очевидная слабость: число 3 занимает те же восемь байт, что и три миллиарда. В базе, где большинство значений маленькие, это заметная переплата.

Лечится это кодированием переменной длины. Приём известен под именем varint и применяется в Protocol Buffers, в форматах Git, в SQLite. Идея: в каждом байте семь бит данных, а старший бит означает «есть продолжение».

```

void write_varint(std::vector<unsigned char>& out, std::uint64_t v) {
    while (v >= 0x80) {
        out.push_back(static_cast<unsigned char>((v & 0x7F) | 0x80));
        v >>= 7;
    }
    out.push_back(static_cast<unsigned char>(v));
}

```

```
std::uint64_t read_varint(const unsigned char* p, std::size_t& consumed) {
    std::uint64_t v = 0;
    int shift = 0;
    std::size_t i = 0;
    while (true) {
        unsigned char b = p[i++];
        v |= static_cast<std::uint64_t>(b & 0x7F) << shift;
        if ((b & 0x80) == 0) break;
        shift += 7;
    }
    consumed = i;
    return v;
}
```

Запись отрезает по семь младших бит, пока число не станет меньше 128; на каждом байте, кроме последнего, поднимает старший бит. Чтение собирает обратно, останавливаясь на байте без старшего бита.

0: 1 байт (u64 занял бы 8)

1: 1 байт

127: 1 байт

128: 2 байт

300: 2 байт

100000: 3 байт

4294967295: 5 байт

Выигрыш на маленьких числах трёх-восьмикратный. Плата — трёхкратная: во-первых, размер записи заранее неизвестен, и её нельзя пропустить, не разобрав; во-вторых, разбор стоит цикла с ветвлением вместо простого чтения; в-третьих, самые большие числа занимают уже не восемь байт, а десять.

Из-за первого пункта мы varint в нашей базе не используем. Внутри страницы ключи лежат подряд, и поиск по ним делается двоичным — а он требует, чтобы к элементу с номером i можно было обратиться по смещению, без разбора предыдущих. С фиксированной шириной это $\text{HEADER} + i * 8$, с переменной пришлось бы читать всё подряд.

Общее правило звучит так: переменная длина хороша там, где данные всё равно читаются подряд (журналы, сетевые сообщения, архивы), и мешает там, где нужен произвольный доступ внутри блока.

Отдельная тонкость — отрицательные числа. Наивный varint для -1 даст десять байт, потому что в дополнительном коде это все единицы. Поэтому знаковые числа сначала переводят зигзагом: $0 \rightarrow 0, -1 \rightarrow 1, 1 \rightarrow 2, -2 \rightarrow 3$, что записывается как $(v < 1) \wedge (v > 63)$. После такого преобразования маленькие по модулю числа остаются маленькими.

33.8 Развитие формата

Метка и версия дают возможность менять формат, но саму возможность надо ещё уметь использовать. Скажем, завтра нам понадобится хранить в узле дерева не только ключ и значение, но и время изменения. Есть три способа это пережить.

Самый простой — поднять версию и отказаться читать старые файлы. Годится для промежуточных данных, кэшей, временных индексов: пользователь просто пересоздаст файл. Для базы, где лежат чьи-то данные, не годится совсем.

Второй — читать обе версии. Разбор смотрит на номер версии и выбирает соответствующую функцию; при первой же записи файл переписывается в новом формате. Так поступает большинство программ, и это честная работа: старый код разбора приходится хранить, пока живы старые файлы.

Третий — сделать формат расширяемым заранее, чтобы новые поля не ломали старый разбор. Ровно для этого в Protocol Buffers каждое поле снабжено номером и типом: незнакомое поле старый код пропускает, не понимая, но и не ломаясь. Цена — служебные байты у каждого поля.

В нашем формате узла заложен маленький задел на будущее: заголовок узла занимает шестнадцать байт, а осмысленных полей в нём три — тип, число ключей и ссылка на следующий лист, всего восемь байт. Оставшиеся восемь зарезервированы. Добавив туда поле, мы не сдвинем ключи и значения, а значит, старый и новый форматы останутся совместимы по расположению данных.

Резерв в заголовке — приём почти бесплатный (несколько байт на страницу в четыре килобайта) и не раз выручавший. Закладывайте его в свои форматы.

33.9 Разберём чужой формат руками

Всё, о чём шла речь, легко проверить на настоящей базе данных. Возьмём SQLite — он есть на любой машине, — создадим крошечную базу и посмотрим в неё:

```
$ sqlite3 demo.sqlite "CREATE TABLE t(a INT, b TEXT); INSERT INTO t VALUES (42,'Alice');"
$ xxd demo.sqlite | head -3
00000000: 5351 4c69 7465 2066 6f72 6d61 7420 3300  SQLite format 3.
```

```
00000010: 1000 0101 0c40 2020 0000 0002 0000 0002  ....@ .....
00000020: 0000 0000 0000 0000 0000 0001 0000 0004  .....
```

Первые шестнадцать байт — метка формата, причём человекочитаемая: строка SQLite format 3 и нулевой байт. Ровно то, о чём мы говорили в разделе про версии, только здесь метку сделали текстовой, чтобы утилита file могла узнать базу по содержимому.

Дальше начинается заголовок, и первое же поле в нём — размер страницы, два байта по смещению 16: 10 00. Здесь порядок байтов **старшим вперёд**, как в сетевых протоколах, поэтому это 0x1000 — четыре тысячи девяносто шесть. Те самые четыре килобайта, которые мы возьмём в следующей главе.

Размер файла подтверждает картину:

```
$ ls -l demo.sqlite
-rw-r--r-- 1 serik wheel 8192 ...
```

Восемь килобайт — ровно две страницы: заголовочная и одна с данными. Файл не растёт по байту, он растёт страницами.

А вот и наши данные, в самом конце файла:

```
$ xxd demo.sqlite | tail -2
00001fe0: 0000 0000 0000 0000 0009 0103 0117 2a41  ....*A
00001ff0: 6c69 6365 0000 0000 0000 0000 0000 0000  lice.....
```

Байт 2a — это 42, наше целое, а следом лежат буквы Alice. Строка записана без нулевого байта в конце и без кавычек — её длина закодирована в заголовке записи, там же, где типы полей. Совсем как наш формат с префиксом длины, только упакованный плотнее.

Одна деталь стоит отдельного внимания: запись лежит в **конце** страницы, а не в начале. Так устроены страницы почти во всех базах — служебная информация растёт от начала страницы вперёд, записи кладутся с конца назад, а свободное место остаётся посередине. Мы к этой схеме вернёмся в главе 33.

Двадцать минут с xxd и описанием формата — и чужой файл перестаёт быть чёрным ящиком. Умение так смотреть окупается каждый раз, когда что-то «не читается», а документация молчит.

33.10 Как выглядит остаток Части IV

Фундамент заложен, дальше будем строить слоями. В главе 33 появится страничный файл: файл станет массивом блоков по четыре килобайта, а над ним встанет кэш с вытеснением давно не использованных страниц. В главе 34 на страницах вырастет B+-дерево — структура,

благодаря которой поиск по ключу обходится в считанные обращения к диску вместо перебора. Глава 35 добавит журнал упреждающей записи и ответит на вопрос, что происходит с базой при внезапном выключении питания.

Дальше — ближе к пользователю. Глава 36 даст крошечный SQL с разбором в дерево, глава 37 — вторичные индексы и планировщик запросов с EXPLAIN, глава 38 — многострочный REPL с историей, глава 39 — измерения, где мы наконец увидим в числах, ради чего всё затевалось.

Байтовый слой при этом останется тем же самым: все страницы, узлы и записи журнала пишутся функциями из `binary.h`.

33.11 Главные правила главы

1. Структуру нельзя записывать в файл целиком: дыры от выравнивания не инициализированы, размеры типов и правила раскладки зависят от платформы.
2. Сериализуйте поле за полем, вычисляя байты арифметикой, — тогда формат не зависит от порядка байтов машины.
3. В форматах используйте только типы фиксированного размера из `<cstdint>`.
4. Буфер байтов — это `unsigned char`, иначе побитовые операции преподнесут сюрприз на отрицательных значениях.
5. Строки пишут с префиксом длины, а не с нулевым байтом в конце: так можно хранить любые байты и читать одним куском.
6. Знаковые числа сериализуются приведением к беззнаковым — дополнительный код сохраняет биты.
7. Любой долгоживущий формат начинается с метки и номера версии, и проверка обязана быть при открытии.
8. Заведите привычку смотреть на файл через `xxd` — без этого отладка бинарного формата превращается в гадание.

33.12 Упражнения

1. Соберите `demo-db` и запустите `./build/binary_demo`. Сопоставьте каждый байт вывода с числами, которые записывались. Найдите в дампе длину строки и убедитесь, что она записана младшим байтом вперёд.
2. Добавьте в `Row` поле `std::uint8_t flags` — сначала в конец структуры, потом между `id` и `age`. Как в каждом случае меняются `sizeof` и смещения? Объясните разницу.
3. Напишите `write_i32/read_i32` через приведение к беззнаковому типу и проверьте на трёх значениях: `-1`, `INT32_MIN`, `INT32_MAX`. Сравните байты с тем, что даёт `xxd`.

4. Запишите тысячу записей Row двумя способами — явной сериализацией и `write` структуры целиком — и сравните размеры файлов. Объясните разницу в байтах.
5. Испортите один байт в метке формата (`xxd` покажет, где она) и запустите `mydb`. Затем поменяйте байт версии на двойку. Разные ли сообщения об ошибке вы получили?
6. Реализуйте зигзаг-преобразование для знаковых чисел и скормите его `varint`. Проверьте, что `-1`, `-2` и `1` занимают по одному байту, а `INT64_MIN` — десять.
7. (Сложнее) Напишите функцию, которая печатает разницу двух буферов: те же строки шестнадцатеричного дампа, но байты, которые различаются, помечены. Такая печать бесценна, когда «файл после записи не совпадает с ожидаемым».
8. (Сложнее) Проверьте порядок байтов своей машины, не полагаясь на макросы: положите `std::uint32_t` со значением `1` и посмотрите на его первый байт через `unsigned char*`. Объясните, почему такой приём допустим, а чтение `int` через указатель на `float` — нет.

33.13 Что дальше

Глава 33 — страничный файл и кэш. Мы перестанем работать с файлом как с потоком байтов и начнём — как с массивом страниц по четыре килобайта. Разберёмся, почему именно четыре, что такое `pread` и `pwrite` и чем они лучше пары `lseek` плюс `read`, зачем нужен кэш страниц в памяти и как устроено вытеснение по давности использования. А в конце соберём `PageManager` — компонент, на котором будет стоять всё остальное в этой части.

Глава 34

Глава 33. Страничный файл и кэш

В прошлой главе мы научились превращать числа в байты. Теперь надо решить, как эти байты лежат в файле и как база с ними работает.

Ответ, к которому за полвека пришли все — PostgreSQL, MySQL, SQLite, ClickHouse, — одинаков: файл базы данных считается **массивом страниц** одинакового размера, обычно по четыре килобайта. Не потоком записей переменной длины, а именно массивом блоков с номерами. Поверх этого массива живёт кэш в памяти, потому что обращение к диску в тысячи раз дороже обращения к оперативной памяти.

К концу главы у нас будет PageManager — компонент, на котором стоит вся Часть IV. Он открывает файл, выдаёт страницы по номеру, держит недавно использованные в памяти, помечает изменённые и по команде сбрасывает всё на диск.

34.1 Почему страницы, а не записи

Представим, что база хранит записи как есть, каждую по своему смещению, вплотную друг к другу. Первая беда обнаружится при удалении: на месте записи остаётся дыра, а дыры надо где-то учитывать и как-то переиспользовать, причём дыра размером 40 байт не подойдёт для записи в 50. Вторая беда — при изменении: если запись выросла на байт, её больше нельзя оставить на месте, надо переносить и править все ссылки на неё.

Со страницами всё проще. Файл нарезан на равные куски, каждый кусок имеет номер, и любой кусок можно прочитать или записать целиком, зная только номер. Пустое место внутри страницы — забота самой страницы, а не всего файла. Ссылка на данные становится парой «номер страницы, смещение внутри» и не ломается при перекладывании соседей.

Есть и вторая причина, поважнее удобства. Диск физически работает блоками: даже если попросить один байт, устройство прочитает целый сектор, а файловая система — целый блок.

Читать меньше блока бессмысленно, читать не кратно блоку — значит трогать лишний блок. Раз всё равно платим за блок, будем и работать блоками.

34.1.1 Почему 4096

Число не случайное и совпадает сразу с несколькими другими.

Страница виртуальной памяти на x86 и ARM — четыре килобайта. Блок в ext4, APFS, NTFS по умолчанию — четыре килобайта. Размер страницы по умолчанию в SQLite — тоже четыре, в PostgreSQL — восемь. Мы это уже видели своими глазами в прошлой главе: в заголовке настоящего файла SQLite по смещению 16 лежало 10 00, то есть 4096.

Совпадение полезное. Когда размер страницы базы кратен блоку файловой системы, запись страницы — это запись целых блоков, без чтения соседних. Если бы страница была, скажем, 3000 байт, то запись первой страницы задела бы часть второго блока, и файловой системе пришлось бы сначала прочитать этот блок, вставить в него кусок и записать обратно. Такое чтение-перед-записью — известный источник тормозов.

Размер побольше выгоден при последовательном чтении: меньше обращений на тот же объём. Размер поменьше выгоден при случайном доступе: не тратим канал на данные, которые не нужны. Четыре килобайта — компромисс, устоявшийся десятилетиями.

Объявляем всё это в `include/page.h`:

```
#ifndef DB_PAGE_H
#define DB_PAGE_H

#include <stddef>
#include <stdint>
#include <vector>

namespace db {

// Размер страницы — 4 KiB. Совпадает с размером страницы памяти в большинстве ОС
// и размером блока в большинстве файловых систем. Чтение/запись страницы — это
// один блок IO, без частичных операций.
constexpr std::size_t PAGE_SIZE = 4096;

// Адрес страницы — индекс в файле. Файл рассматривается как массив страниц:
// страница 0 это байты 0..4095, страница 1 — 4096..8191, и так далее.
using PageId = std::uint32_t;
```

```
// Страница — буфер из 4 KiB байтов. Используем vector, а не array, чтобы можно  
// было std::move без копий.  
using Page = std::vector<unsigned char>;  
  
inline Page make_page() {  
    return Page(PAGE_SIZE, 0);  
}  
  
} // namespace db  
  
#endif
```

Адрес страницы — 32-битное беззнаковое число, и это ограничивает базу четырьмя миллиардами страниц, то есть шестнадцатью терабайтами. Для учебной базы с запасом; в настоящих системах номер страницы обычно 64-битный или составной — номер файла плюс номер страницы внутри.

Выбор `std::vector` вместо `std::array<unsigned char, 4096>` сделан ради перемещения. Массив фиксированного размера всегда копируется целиком — четыре килобайта на каждую передачу; вектор при перемещении отдаёт указатель, и копирования не происходит вовсе. Это прямое применение того, о чём шла речь в главе 19.

34.2 Адресация: страница номер N

Перевод номера страницы в смещение в файле — умножение:

смещение = `pid * PAGE_SIZE`

Страница 0 занимает байты 0–4095, страница 1 — 4096–8191, страница 7 начинается с 28672. Обратный перевод — деление.

Важная мелочь: умножать надо в достаточно широком типе. `PageId` у нас 32-битный, и `pid * PAGE_SIZE` при большом `pid` переполнится ещё до того, как результат попадёт в 64-битную переменную. Поэтому в коде везде стоит явное приведение:

```
off_t off = static_cast<off_t>(pid) * static_cast<off_t>(PAGE_SIZE);
```

`off_t` — тип смещения в файле, на 64-битных системах он 64-битный. Ошибка «переполнение при вычислении смещения» относится к тем, которые не проявляются в тестах и вылезают на большом файле у пользователя.

34.3 pread и pwrite

Обычные `read` и `write` работают с текущей позицией в файле, которую надо предварительно выставить вызовом `lseek`:

```
lseek(fd, offset, SEEK_SET);
read(fd, buf, PAGE_SIZE);
```

Для страничного доступа это неудобно и опасно. Во-первых, два системных вызова вместо одного. Во-вторых, позиция — общее состояние дескриптора: если тот же дескриптор используется в другом потоке, он сдвинет позицию между вашими `lseek` и `read`, и вы прочитаете не то. Мы это свойство наблюдали в главе 27, когда родитель и ребёнок делили позицию в файле после `fork`.

POSIX даёт пару, свободную от обеих проблем:

```
ssize_t pread(int fd, void* buf, size_t count, off_t offset);
ssize_t pwrite(int fd, const void* buf, size_t count, off_t offset);
```

Смещение передаётся аргументом, позиция дескриптора не трогается вообще. Один вызов, никакого общего состояния, безопасно из нескольких потоков.

Как и обычные `read` с `write`, они могут обработать меньше запрошенного и прерваться сигналом, поэтому вызываются в цикле — том самом, который мы написали в главе 31 для `mycat`:

```
ssize_t written = 0;
while (written < static_cast<ssize_t>(PAGE_SIZE)) {
    ssize_t n = ::pwrite(fd_, e.page.data() + written,
                        PAGE_SIZE - written, off + written);

    if (n < 0) {
        if (errno == EINTR) continue;
        throw PageError(std::string("pwrite: ") + std::strerror(errno));
    }
    ++stats_.disk_writes;
    written += n;
}
```

34.4 Зачем кэш

Числа, которые стоит держать в голове. Обращение к оперативной памяти — около 100 наносекунд. Чтение блока с SSD — около 100 микросекунд, то есть в тысячу раз дольше. Позиционирование головки жёсткого диска — около 10 миллисекунд, в сто тысяч раз дольше.

Значит, всё, что можно не читать с диска, читать не нужно. Кэш страниц в памяти делает ровно это: держит недавно использованные страницы под рукой и отдаёт их без обращения к диску.

Работает это благодаря локальности обращений. В базе данных они распределены крайне неравномерно: верхние узлы индекса читаются при каждом запросе, метаданные — постоянно, а какая-нибудь страница с данными за позапрошлый год — раз в год. Даже маленький кэш ловит большинство обращений, потому что горячих страниц немного.

Кэш ограничен по размеру, значит, при заполнении надо кого-то выселять. Стратегий много, самая известная — вытеснять ту страницу, к которой дольше всего не обращались; её называют LRU, *least recently used*. Идея простая: если страницу не трогали дольше всех, вероятно, и дальше не тронут.

34.4.1 Структура данных для LRU

Нужны две операции, обе за постоянное время: найти страницу по номеру и переместить страницу в «самые свежие». Ни хеш-таблица, ни список поодиночке этого не дают, а вместе — дают.

Держим двусвязный список номеров страниц: в начале самая свежая, в конце самая старая. И хеш-таблицу из номера страницы в пару «данные плюс итератор на позицию в списке».

```
using LruList = std::list<PageId>;
using LruIter = LruList::iterator;

LruList lru_;    // front = самая свежая
std::unordered_map<PageId, std::pair<Entry, LruIter>> cache_;
```

Обращение к странице — переносим её узел в начало списка:

```
void PageManager::touch(PageId pid) {
    auto it = cache_.find(pid);
    if (it == cache_.end()) return;
    lru_.erase(it->second.second);
    lru_.push_front(pid);
    it->second.second = lru_.begin();
}
```

Вытеснение — берём с конца списка:

```
void PageManager::evict_one_if_full() {
    if (cache_.size() < cache_size_) return;
    // Жертва — самый старый элемент (хвост LRU).
    PageId victim = lru_.back();
    auto it = cache_.find(victim);
    flush_entry(victim, it->second.first);
    ++stats_.evictions;
    lru_.pop_back();
    cache_.erase(it);
}
```

Именно ради этого сочетания выбран `std::list`, а не `std::vector`: у списка есть свойство, которого нет у большинства контейнеров, — итератор на элемент остаётся действительным после вставок и удалений где угодно ещё. Поэтому итератор можно хранить в хеш-таблице. У `std::vector` при первой же вставке всё бы протухло.

Заметьте, что `erase` плюс `push_front` в `touch` — это не перевыделение памяти, а перевешивание указателей: узел списка остаётся на месте, меняются связи. Правда, `std::list::splice` сделал бы то же самое одной операцией и без переприсваивания итератора; это упражнение в конце главы.

34.5 Грязные страницы

Если бы страницы только читались, кэш был бы просто копией диска и выбрасывался бы без последствий. Но их изменяют, и изменённая в памяти страница отличается от той, что на диске. Такую страницу называют грязной.

Правило простое: грязную страницу нельзя выбросить, не записав. Поэтому у каждой записи в кэше есть флаг:

```
struct Entry {
    Page page;
    bool dirty = false;
};
```

`write_page` ставит флаг, `flush_entry` при записи снимает:

```
void PageManager::flush_entry(PageId pid, Entry& e) {
    if (!e.dirty) return;
    // ... pwrite в цикле ...
```



```
e.dirty = false;  
}
```

Отложенная запись — не просто оптимизация, а серьёзный выигрыш. Если страницу изменили десять раз подряд, на диск уйдёт одна запись вместо десяти. В базе данных, где верхний узел индекса меняется при каждой вставке, разница огромна.

Плата за это — окно, в котором изменения существуют только в памяти. Что случится при внезапном выключении питания, разбирается в главе 35, где мы напишем журнал.

34.6 fsync: когда данные действительно на диске

Здесь легко попасть в ловушку. Вызов `pwrite` вернул успех — значит ли это, что данные на диске?

Нет. Он значит, что данные скопированы в кэш ядра. Когда именно ядро запишет их на устройство, решает оно само, обычно в пределах десятков секунд. Если в этот промежуток выключить питание, данные пропадут, хотя программа была уверена, что записала их.

Заставить ядро довести дело до конца можно вызовом `fsync`:

```
void PageManager::sync() {  
    // 1) сбросить все dirty-страницы.  
    for (auto& kv : cache_) {  
        flush_entry(kv.first, kv.second.first);  
    }  
    // 2) fsync — заставить ядро записать данные с буферов на диск.  
    if (::fsync(fd_) < 0) {  
        throw PageError(std::string("fsync: ") + std::strerror(errno));  
    }  
}
```

Он возвращается только тогда, когда устройство подтвердило запись. Стоит это дорого — миллисекунды, — поэтому вызывать его на каждую страницу нельзя, а нужно в моменты, когда важна сохранность: при контрольной точке, при закрытии базы, после записи в журнал.

Есть родственник `fdatasync`, который сбрасывает только содержимое, не трогая метаданные вроде времени изменения. Он чуть быстрее и обычно достаточен, если размер файла не менялся.

Насколько дорого — измерим. Двести записей по четыре килобайта, сначала без синхронизации и с одним `fsync` в конце, потом с `fsync` после каждой записи:

200 записей без fsync: 2711 мкс, финальный fsync: 529 мкс

200 записей с fsync после каждой: 31191 мкс (155 мкс на запись)

Разница почти десятикратная, и это на быстром SSD; на вращающемся диске она была бы стократной. Отдельная запись стоит около тринадцати микросекунд, а fsync — сто пятьдесят пять, то есть в двенадцать раз дороже самой записи.

Отсюда становится понятной вся конструкция, которую мы строим. Синхронизировать каждую изменённую страницу нельзя — база превратится в улитку. Не синхронизировать вовсе нельзя — потеряем данные. Выход, к которому пришли все базы: синхронно и часто пишется только маленький последовательный журнал, а страницы данных сбрасываются редко, большими пачками, в контрольных точках. Журналом займёмся в главе 35, и там же вернёмся к этим числам.

Отдельно предупрежу про честность самих устройств. У диска есть собственный кэш записи, и некоторые дешёвые накопители подтверждают запись, когда данные ещё лежат в нём. На таком железе fsync не спасает; лечится это отключением кэша записи или использованием устройств с защитой от потери питания. Знать об этом стоит хотя бы потому, что рано или поздно вы услышите фразу «база потеряла подтверждённую транзакцию».

В деструкторе PageManager тоже стоит sync, но с проглатыванием исключений:

```
PageManager::~~PageManager() {
    if (fd_ >= 0) {
        // На деструкторе пытаемся синхронизироваться. Исключения проглатываем —
        // в деструкторе их бросать нельзя.
        try { sync(); } catch (...) {}
        ::close(fd_);
    }
}
```

Бросать из деструктора нельзя — если это случится во время раскрутки стека из-за другого исключения, программа немедленно завершится через std::terminate (глава 22). Поэтому в деструкторе остаётся только попытка. Правильное решение для важных данных — явный метод закрытия, который возвращает ошибку и который вызывающий обязан проверить; деструктор тогда становится страховкой на случай, если про метод забыли.

34.7 Класс PageManager

Соберём всё вместе. Заголовок include/page_manager.h:

```

class PageManager {
public:
    // Открыть файл (создаст, если не существует). cache_size – сколько страниц
    // держать в памяти.
    PageManager(const std::string& path, std::size_t cache_size = 64);
    ~PageManager();

    // Запрет копирования.
    PageManager(const PageManager&) = delete;
    PageManager& operator=(const PageManager&) = delete;

    Page read_page(PageId pid);
    void write_page(PageId pid, const Page& page);
    PageId allocate_page();
    PageId page_count() const { return page_count_; }
    void sync();

    // Счётчики для наблюдения за кэшем.
    struct Stats {
        long long hits = 0;           // страница была в кэше
        long long misses = 0;        // пришлось читать с диска
        long long evictions = 0;     // вытеснено из кэша
        long long disk_reads = 0;    // вызовов pread
        long long disk_writes = 0;   // вызовов pwrite
    };
    const Stats& stats() const { return stats_; }
    void reset_stats() { stats_ = Stats(); }

private:
    struct Entry {
        Page page;
        bool dirty = false;
    };

    using LruList = std::list<PageId>;
    using LruIter = LruList::iterator;

    int fd_;

```

```

PageId page_count_;
std::size_t cache_size_;
Stats stats_;

LruList lru_;
std::unordered_map<PageId, std::pair<Entry, LruIter>> cache_;

void evict_one_if_full();
void flush_entry(PageId pid, Entry& e);
Entry& load_into_cache(PageId pid);
void touch(PageId pid);
};

```

Копирование запрещено через `= delete` — приём из главы 18. `PageManager` владеет файловым дескриптором, и второй владелец того же дескриптора означал бы двойное закрытие. Это тот случай, когда запрет копирования не ограничение, а описание сути объекта.

Счётчики в открытой части класса — не украшение. Кэш трудно обсуждать умозрительно, и возможность спросить «сколько раз ты попал» превращает догадки в измерения. В главе 39 мы будем смотреть на них при настройке размера кэша.

34.7.1 Открытие

```

PageManager::PageManager(const std::string& path, std::size_t cache_size)
: fd_(-1), page_count_(0), cache_size_(cache_size) {
    fd_ = ::open(path.c_str(), O_RDWR | O_CREAT, 0644);
    if (fd_ < 0) {
        throw PageError("не открыт: " + path + ": " + std::strerror(errno));
    }
    struct stat st;
    if (::fstat(fd_, &st) < 0) {
        ::close(fd_);
        throw PageError(std::string("fstat: ") + std::strerror(errno));
    }
    if (st.st_size % PAGE_SIZE != 0) {
        ::close(fd_);
        throw PageError("файл не выровнен по PAGE_SIZE");
    }
}

```

```

    page_count_ = static_cast<PageId>(st.st_size / PAGE_SIZE);
}

```

Количество страниц нигде не хранится — оно вычисляется из размера файла. Так меньше состояния, которое может рассинхронизироваться: файл сам себя описывает.

Проверка кратности размера — дешёвая защита от чужого или обрезанного файла. Обратите внимание на порядок в ветках ошибок: сначала закрываем дескриптор, потом бросаем исключение. Пропустить `close` здесь — классическая утечка, потому что деструктор объекта, конструктор которого бросил, не вызывается.

`fstat` вместо `stat` — потому что дескриптор у нас уже есть, и спрашивать про файл по имени было бы и лишней работой, и гонкой: между `open` и `stat` файл могли подменить.

34.7.2 Чтение и запись

```

Page PageManager::read_page(PageId pid) {
    if (pid >= page_count_) {
        throw PageError("страница вне диапазона: " + std::to_string(pid));
    }
    Entry& e = load_into_cache(pid);
    return e.page; // copy
}

void PageManager::write_page(PageId pid, const Page& page) {
    if (pid >= page_count_) {
        throw PageError("страница вне диапазона: " + std::to_string(pid));
    }
    if (page.size() != PAGE_SIZE) {
        throw PageError("неправильный размер page");
    }
    Entry& e = load_into_cache(pid);
    e.page = page;
    e.dirty = true;
}

```

`read_page` возвращает **копию** страницы — четыре килобайта на каждый вызов. Это осознанное упрощение, и в настоящей базе так не делают: там возвращают указатель на страницу в кэше вместе с механизмом закрепления (`pin`), который запрещает вытеснять страницу, пока на неё кто-то смотрит. Мы платим копированием за то, что не нужно думать о времени

жизни: страница, полученная из `read_page`, остаётся действительной, даже если её оригинал вытеснили. В главе 39 мы увидим, во что это обходится по времени.

Проверка `pid >= page_count_` превращает обращение за границу в исключение вместо чтения мусора. За такие проверки на границах компонентов платят наносекундами, а получают часы сэкономленной отладки.

34.7.3 Загрузка в кэш

```
PageManager::Entry& PageManager::load_into_cache(PageId pid) {
    auto it = cache_.find(pid);
    if (it != cache_.end()) {
        ++stats_.hits;
        touch(pid);
        return it->second.first;
    }
    ++stats_.misses;

    evict_one_if_full();

    Entry e;
    e.page = make_page();
    e.dirty = false;

    off_t off = static_cast<off_t>(pid) * static_cast<off_t>(PAGE_SIZE);
    ssize_t read_total = 0;
    while (read_total < static_cast<ssize_t>(PAGE_SIZE)) {
        ssize_t n = ::pread(fd_, e.page.data() + read_total,
                           PAGE_SIZE - read_total, off + read_total);

        if (n < 0) {
            if (errno == EINTR) continue;
            throw PageError(std::string("pread: ") + std::strerror(errno));
        }
        if (n == 0) {
            // Дочитали до конца файла — остальное оставляем нулями.
            break;
        }
        ++stats_.disk_reads;
        read_total += n;
    }
}
```

```

    }

    lru_.push_front(pid);
    auto inserted = cache_.emplace(pid, std::make_pair(std::move(e), lru_.begin()));
    return inserted.first->second.first;
}

```

Ветка с `n == 0` заслуживает объяснения. Ноль от `pread` означает, что мы читаем за концом файла. Так бывает штатно: страницу выделили в памяти, ещё не сбросили на диск, а кто-то её читает. Файл до неё не дорос, читать нечего — и мы оставляем страницу нулевой, что как раз и есть её содержимое.

Возвращается ссылка на запись **внутри** хеш-таблицы. Это безопасно ровно до следующей операции, которая может вытеснить эту страницу, — и именно поэтому `read_page` тут же делает копию. У `std::unordered_map` ссылки на элементы переживают вставки, в отличие от итераторов, но не переживают удаление самого элемента.

34.7.4 Выделение страницы

```

PageId PageManager::allocate_page() {
    PageId new_id = page_count_;
    page_count_++;

    // Расширяем файл, чтобы он действительно содержал эту страницу.
    Entry e;
    e.page = make_page();
    e.dirty = true;

    evict_one_if_full();
    lru_.push_front(new_id);
    cache_.emplace(new_id, std::make_pair(std::move(e), lru_.begin()));
    return new_id;
}

```

Новая страница создаётся сразу грязной, хотя в ней одни нули. Иначе она никогда бы не попала на диск, файл не вырос бы, и `page_count_` при следующем открытии оказался бы меньше — база потеряла бы страницы.

Свободные страницы мы не переиспользуем: номера всегда растут. В настоящей базе есть список свободных страниц (`freelist`), куда попадают освобождённые при удалении данных, и

`allocate_page` сначала смотрит туда. Нам это не нужно, потому что удаление данных мы не реализуем; в упражнениях есть задание сделать `freelist`.

34.8 Кэш в работе

Посмотрим на всё это в действии — программа `utils/page_demo.cpp` записывает страницы, переоткрывает файл и гоняет разные образцы обращений:

```
$ ./build/page_demo
=== Запись трёх страниц ===
page_count = 3, размер файла будет 12288 байт
после записи: попаданий 3, промахов 0, вытеснено 0, pread 0, pwrite 3

=== Переоткрытие файла ===
page_count восстановлен из размера файла: 3
  страница 0: page 0 payload.....
  страница 1: page 1 payload.....
  страница 2: page 2 payload.....
три первых чтения: попаданий 0, промахов 3, вытеснено 0, pread 3, pwrite 0
ещё 30 чтений: попаданий 30, промахов 0, вытеснено 0, pread 0, pwrite 0
```

Три записи — три попадания в кэш (страницы только что выделены и лежат в памяти) и ни одного чтения с диска. После `sync` — три вызова `pwrite`. При переоткрытии первые три чтения промахиваются, а следующие тридцать попадают все: страницы уже в памяти, диск не потревожен ни разу.

Теперь интереснее — кэш на четыре страницы при шестидесяти трёх используемых:

```
=== Кэш меньше рабочего набора ===
обход по кругу: попаданий 0, промахов 189, вытеснено 189, pread 189, pwrite 0
горячие 4 страницы: попаданий 196, промахов 4, вытеснено 4, pread 4, pwrite 0
```

Первая строка — полный провал: 189 обращений, 189 промахов, ноль попаданий. Кэш не помог совсем. Вторая строка — почти идеал: 196 попаданий из 200.

Разница только в образце обращений, и она объясняет главную слабость LRU. При обходе по кругу к моменту, когда мы вернёмся к первой странице, её уже вытеснили — как раз потому, что к ней дольше всех не обращались. Кэш работает точно наоборот, чем нужно. Это явление называют вымыванием кэша последовательным чтением, и настоящие базы борются с ним отдельно: PostgreSQL для больших последовательных чтений выделяет крошечное кольцо буферов, чтобы такое чтение не выбрасывало из кэша всё полезное.

Вторая строка показывает случай, ради которого кэш и существует: небольшой горячий набор, который целиком помещается в память. Именно так выглядит доступ к верхним узлам индекса.

34.9 У операционной системы тоже есть кэш

Здесь возникает законный вопрос. Ядро и так кэширует содержимое файлов: прочитанный блок остаётся в памяти ядра, и повторное чтение того же места не доходит до устройства. Зачем базе свой кэш поверх ядерного?

Причин несколько, и все практические.

Ядро кэширует байты файла, а база работает со структурами. В нашем случае в кэше лежат страницы уже в том виде, в котором с ними работает дерево; ядерный кэш заставил бы каждый раз копировать четыре килобайта из ядра в память процесса. Копирование не бесплатно, а на горячих страницах оно происходит постоянно.

Ядро не знает смысла страниц. Для него все блоки равны, а база отлично знает, что корень индекса нужен всегда, а страница с архивными данными — почти никогда. Своё вытеснение можно сделать умнее.

И главное — база знает, чего ещё не записала. Именно от неё зависит порядок: сначала журнал, потом данные. Ядро о таких зависимостях не подозревает и вправе записать блоки в любом порядке.

Взрослые системы иногда идут дальше и отключают ядерный кэш вовсе — флагом `O_DIRECT` на Linux или `F_NOCACHE` на macOS. Тогда чтение и запись идут мимо ядра, прямо между устройством и буфером процесса, и данные не хранятся в памяти дважды. Расплата — жёсткие требования: буфер и смещение должны быть выровнены по размеру блока устройства, а всё кэширование ложится на программу. MySQL InnoDB так и делает по умолчанию, PostgreSQL сознательно не делает и полагается на кэш ядра.

Мы `O_DIRECT` не используем: наш кэш живёт поверх ядерного, и часть промахов на самом деле обслуживается памятью ядра, а не диском. Для учебной базы это нормально, но при замерах об этом надо помнить — счётчик `disk_reads` считает вызовы `pread`, а не реальные обращения к устройству.

34.10 Сколько страниц держать

Наш кэш по умолчанию — 64 страницы, то есть 256 килобайт. Мало по меркам настоящей базы, но нам хватает.

Общее правило: кэш должен вмещать горячий набор — те страницы, к которым обращаются постоянно. Для индекса это верхние уровни дерева; при миллионе записей и ветвлении в сотню верхних уровней всего два-три, и это десятки страниц. Пропуски станут заметны, если кэш меньше этого.

Настоящие базы отдают кэшу существенную долю оперативной памяти: в PostgreSQL параметр `shared_buffers` обычно ставят около четверти памяти машины, в MySQL InnoDB буферный пул нередко доводят до трёх четвертей. Спорить с этими цифрами бессмысленно без измерений — а измерения начинаются с тех самых счётчиков попаданий и промахов, которые мы завели.

34.11 Посмотрим на настоящий файл

Всё, что мы построили, уже работает — базой можно пользоваться, и её файл можно разглядывать. Вставим две записи и заглянем внутрь:

```
$ printf 'INSERT INTO items VALUES (10, 1000);\nINSERT INTO items VALUES (20, 2000);\n.checkpoint\n.ex
| ./build/mydb /tmp/pgview
$ ls -l /tmp/pgview/data.db
-rw-r--r-- 1 serik wheel 8192 ...
```

Восемь килобайт — ровно две страницы. Первая:

```
$ xxd -l 32 /tmp/pgview/data.db
00000000: 0100 0000 4d44 4201 0000 0000 0000 0000 ....MDB.....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000 .....
```

Четыре байта номера корневой страницы (единица), метка MDB, версия формата — та самая метаданная страница, которую мы завели в главе 32. Дальше нули: страница выделена целиком, а занято в ней восемь байт.

Вторая страница — сам корень, и он пока лист:

```
$ xxd -s 4096 -l 64 /tmp/pgview/data.db
00001000: 0100 0200 0000 0000 0000 0000 0000 0000 .....
00001010: 0a00 0000 0000 0000 1400 0000 0000 0000 .....
00001020: e803 0000 0000 0000 d007 0000 0000 0000 .....
```

Здесь уже виден формат узла, который мы напишем в следующей главе. `01 00` — тип узла (лист), `02 00` — в нём два ключа, дальше четыре байта ссылки на следующий лист (нулевая — следующего нет) и восемь байт про запас. С шестнадцатого байта идут ключи: `0a` — это 10, `14` — это 20. Затем значения: `e8 03` — это 1000, `d0 07` — это 2000.

Ключей два, а места — четыре килобайта. Расточительно? Для двух записей — да. Но именно фиксированный размер позволяет читать узел одним обращением и не заботиться о том, что он вырос: пока ключи помещаются, страница остаётся той же самой.

34.12 Что здесь упрощено

Честный список, чтобы вы не приняли учебный код за образец промышленного.

Мы возвращаем копию страницы вместо указателя с закреплением — просто, но копирует четыре килобайта на каждое чтение. Мы не переиспользуем освобождённые страницы. У нас нет ни блокировок, ни атомарности: PageManager рассчитан на один поток, и попытка работать с ним из двух потоков сломает и кэш, и список LRU. Наконец, при вытеснении грязной страницы мы пишем её сразу и синхронно, тогда как настоящие базы делают это фоновым процессом, чтобы запрос не ждал диска.

Есть и опасность, о которой стоит знать заранее: разрыв страницы. Если питание пропадёт посреди записи четырёх килобайт, на диске может оказаться половина новой страницы и половина старой. Для SSD запись сектора обычно атомарна, но целая страница из нескольких секторов — уже нет. PostgreSQL решает это записью полного образа страницы в журнал при первом изменении после контрольной точки, MySQL — двойной записью в отдельную область. Наш ответ будет проще и тоже рабочий: журнал операций, из которого дерево восстанавливается заново. Это глава 35.

34.13 Главные правила главы

1. База данных работает с файлом как с массивом страниц одинакового размера, а не как с потоком записей.
2. Размер страницы выбирают кратным блоку файловой системы; 4096 — устоявшийся выбор.
3. Смещение считают в достаточно широком типе: `static_cast<off_t>(pid) * PAGE_SIZE`.
4. `pread` и `pwrite` не трогают позицию дескриптора — это один вызов вместо двух и безопасность при многопоточности.
5. LRU строится на паре «двусвязный список плюс хеш-таблица»; итераторы `std::list` не портятся при вставках и удалениях.
6. Изменённую страницу помечают грязной; выбросить её без записи нельзя.
7. Успешный `pwrite` не означает, что данные на диске; гарантию даёт только `fsync`.
8. Последовательный обход большого файла сводит пользу LRU к нулю — это надо учитывать, а не удивляться.

34.14 Упражнения

1. Соберите demo-db и запустите ./build/page_demo. Повторите опыт с разными размерами кэша (поправьте число в исходнике): 2, 8, 64. Как меняется доля попаданий в каждом из двух образцов обращений?
2. Добавьте в page_demo третий образец: случайные обращения к 63 страницам при кэше на 16. Посчитайте долю попаданий и сравните с ожидаемой — при равномерно случайном доступе она примерно равна отношению размера кэша к размеру набора.
3. Замените в touch пару erase плюс push_front на lru_.splice(lru_.begin(), lru_, it->second.second). Убедитесь, что тесты проходят, и объясните, почему после splice итератор не нужно обновлять.
4. Уберите пометку dirty = true в allocate_page, пересоберите и запустите make tests && ./build/tests/test_db. Какие проверки упали и почему? Верните обратно.
5. Сделайте так, чтобы sync записывал страницы в порядке возрастания номера, а не в произвольном порядке обхода хеш-таблицы. На вращающемся диске это заметно быстрее — объясните почему.
6. Добавьте список свободных страниц: метод free_page(PageId) складывает номер в вектор, allocate_page сначала берёт оттуда. Продумайте, где хранить список между запусками, чтобы он не потерялся.
7. (Сложнее) Замените LRU на политику «второго шанса»: у каждой страницы бит обращения, при вытеснении идём по кругу, сбрасывая биты, и выселяем первую страницу с нулевым битом. Сравните на образце «обход по кругу» — стало ли лучше?
8. (Сложнее) Сделайте read_page, возвращающий const Page& вместо копии, и добавьте закрепление: pin(pid) запрещает вытеснение, unpin(pid) разрешает. Что должно происходить, если все страницы кэша закреплены?

34.15 Что дальше

Глава 34 — В+-дерево. У нас есть страницы, но пока нет способа быстро найти нужную. Полный перебор при миллионе записей — это миллион чтений; дерево превращает поиск в три-четыре обращения. Разберёмся, чем В+-дерево отличается от двоичного и почему для диска годится именно оно, как узел укладывается в страницу, что происходит при переполнении узла и почему листья связаны в цепочку.

Глава 35

Глава 34. В+-дерево

У нас есть страничный файл, но пока в нём нет никакого порядка: чтобы найти запись, придётся прочитать все страницы подряд. При тысяче страниц это тысяча обращений к диску на один запрос.

В этой главе построим структуру, которая превращает поиск в три-четыре обращения независимо от размера базы. Называется она В+-дерево, и на ней держатся индексы во всех реляционных базах: PostgreSQL, MySQL InnoDB, SQLite, Oracle, SQL Server. Не потому, что нет альтернатив, а потому, что она устроена под диск лучше остальных.

К концу главы дерево будет работать: вставка с разделением переполненных узлов, поиск, обход по возрастанию ключей, поддержка повторяющихся ключей. Всё — на страницах из прошлой главы.

35.1 Зачем вообще дерево

Разберём на числах. Пусть в базе миллион записей, в странице помещается сто записей — значит, десять тысяч страниц.

Поиск полным перебором прочитает в среднем половину, пять тысяч страниц. При ста микросекундах на страницу это половина секунды на один запрос. Индекс должен превратить эти пять тысяч обращений в считанные единицы.

Способов два: хеш-таблица и упорядоченная структура. Хеш-таблица находит по точному совпадению за одно обращение и проигрывает во всём остальном: она не умеет отвечать на «все записи с ключом больше 100», не умеет отдавать данные по порядку, плохо переносит рост — перехеширование миллиона записей на диске событие, которое хочется не пережить. Поэтому основным индексом почти везде служит упорядоченная структура, а хеш-индексы существуют как дополнение для точечных запросов.

Упорядоченная структура — это дерево поиска. Только не то, к которому привыкли по учебникам.

35.1.1 Почему не двоичное дерево

В обычном двоичном дереве поиска у узла два потомка, и высота при миллионе записей — около двадцати. Двадцать узлов на пути от корня к листу, и если каждый узел лежит в своей странице, то двадцать обращений к диску.

Проблема в том, что за одно обращение мы всё равно получаем четыре килобайта, а двоичный узел использует из них байт двадцать. Мы платим за страницу, а забираем крохи.

Отсюда идея В-дерева: сделать узел размером со страницу и напихать в него столько ключей, сколько влезет. Если в узел помещается 250 ключей, у него 251 потомок, и высота дерева считается логарифмом по основанию 251:

Записей	Двоичное дерево	Ветвление 251
1 000	10 уровней	2 уровня
1 000 000	20 уровней	3 уровня
1 000 000 000	30 уровней	4 уровня

Миллиард записей — четыре обращения к диску. Причём верхние уровни постоянно лежат в кэше, так что реальных обращений будет одно-два.

Это и есть главная мысль всей главы: **структуры для диска проектируют вокруг размера блока**, а не вокруг красоты алгоритма. Двоичное дерево прекрасно в памяти и негодно на диске.

35.1.2 В-дерево и В+-дерево

Различие между ними одно, но важное: где лежат данные.

В классическом В-дереве значения хранятся во всех узлах, включая внутренние. Поиск может закончиться на середине пути — и это выглядит выигрышем, пока не начинаются диапазонные запросы: чтобы обойти ключи по порядку, приходится ходить по дереву вверх-вниз.

В В+-дереве значения лежат **только в листьях**, а внутренние узлы содержат исключительно ключи-разделители, указывающие, в какого потомка спускаться. Из этого следуют три свойства.

Внутренние узлы становятся компактнее: в них нет значений, а значит, ключей помещается больше и ветвление выше. Любой поиск проходит одинаковый путь до листа — производительность

предсказуема. И, наконец, листья можно связать в цепочку: у каждого есть ссылка на следующего, и обход диапазона превращается в проход по односвязному списку без единого возврата вверх.

Последнее свойство и решило спор. Запросы вида «все заказы с датой между такой-то и такой-то» — обычное дело, а на B+-дереве они обходятся почти даром.

35.2 Узел на странице

Наш узел занимает страницу целиком. Раскладка описана в заголовке `include/btree.h`:

Layout узла (4096 байт):

```
[0..1]  u16 type (0=Internal, 1=Leaf)
[2..3]  u16 num_keys
[4..7]  u32 next_leaf (только для leaf, для internal — 0)
[8..15] reserved
[16..]  keys (int64), потом children (PageId u32) или values (int64)
```

Шестнадцать байт заголовка, дальше массив ключей, дальше — в зависимости от типа — массив значений (для листа) или массив ссылок на потомков (для внутреннего узла). Ключей n , значений тоже n , а потомков всегда $n+1$: между каждой парой соседних ключей и по краям.

Посмотрим на настоящие байты. Вставим в базу семь записей и заглянем в файл:

```
$ xxd -l 16 /tmp/treeview/data.db
00000000: 0300 0000 4d44 4201 0000 0000 0000 0000  ....MDB.....
```

Метаданные: корень — страница 3, метка MDB, версия 1.

```
$ xxd -s $((4096*3)) -l 48 /tmp/treeview/data.db
00003000: 0000 0200 0000 0000 0000 0000 0000 0000  .....
00003010: 0300 0000 0000 0000 0500 0000 0000 0000  .....
00003020: 0100 0000 0200 0000 0400 0000 0000 0000  .....
```

Корень: тип `00 00` — внутренний, ключей `02 00` — два, ссылка на следующий лист нулевая (внутренним узлам она не нужна). С шестнадцатого байта два ключа-разделителя: 3 и 5. Затем три ссылки на потомков по четыре байта: страницы 1, 2 и 4.

```
$ xxd -s 4096 -l 48 /tmp/treeview/data.db
00001000: 0100 0200 0200 0000 0000 0000 0000 0000  .....
00001010: 0100 0000 0000 0000 0200 0000 0000 0000  .....
00001020: 6400 0000 0000 0000 c800 0000 0000 0000  d.....
```

Левый лист: тип 01 00 — лист, два ключа, ссылка на следующий лист — страница 2. Ключи 1 и 2, значения 64 и с8, то есть 100 и 200.

Всё сходится с картинкой: разделитель 3 в корне означает, что ключи меньше трёх лежат слева, а от трёх и выше — справа.

35.2.1 Чтение и запись узла

Поскольку страница — это просто байты, нужны функции перевода в удобную структуру и обратно:

```
struct NodeData {
    NodeType type = Leaf;
    std::vector<std::int64_t> keys;
    std::vector<std::int64_t> values;
    std::vector<PageId> children;
    PageId next_leaf = 0;
};

BPlusTree::NodeData BPlusTree::read_node(PageId pid) const {
    Page p = pm_.read_page(pid);
    NodeData n;
    n.type = static_cast<NodeType>(read_u16_at(p, 0));
    std::uint16_t nk = read_u16_at(p, 2);
    n.next_leaf = read_u32_at(p, 4);

    std::size_t off = HEADER_BYTES;
    n.keys.resize(nk);
    for (std::uint16_t i = 0; i < nk; ++i) {
        n.keys[i] = read_i64_at(p, off);
        off += 8;
    }
    if (n.type == Leaf) {
        n.values.resize(nk);
        for (std::uint16_t i = 0; i < nk; ++i) {
            n.values[i] = read_i64_at(p, off);
            off += 8;
        }
    } else {
        n.children.resize(static_cast<std::size_t>(nk) + 1);
    }
}
```



```

    for (std::uint16_t i = 0; i <= nk; ++i) {
        n.children[i] = read_u32_at(p, off);
        off += 4;
    }
}
return n;
}

```

Обратите внимание на `i <= nk` в цикле потомков — их на одного больше, чем ключей. Ошибиться здесь легко, а проявится ошибка не сразу: дерево будет работать, пока не понадобится самый правый потомок.

Запись симметрична: создаём чистую страницу через `make_page`, раскладываем поля по тем же смещениям и отдаём в `write_page`. Заметьте, что узел всегда пишется целиком, даже если изменился один ключ, — так устроена работа со страницами.

Такое чтение с распаковкой в вектора удобно, но расточительно: на каждое обращение к узлу мы выделяем память под три вектора и копируем данные. Промышленная реализация работает с байтами страницы напрямую, находя ключ двоичным поиском прямо в буфере. Мы платим за читаемость, и в главе 39 увидим, сколько именно.

35.3 Сколько ключей в узле

В коде стоит нарочито маленькое число:

```

// Малый порядок — для демонстрации split'ов на 10–20 вставках.
static constexpr std::size_t MAX_KEYS = 4;

```

Четыре ключа на узел — игрушечное значение, выбранное ради наглядности: дерево начинает ветвиться уже на пятой вставке, и все интересные случаи видны на глаз.

Сколько влезло бы на самом деле? Для листа на каждый ключ приходится 8 байт ключа плюс 8 байт значения, плюс 16 байт заголовка:

$$(4096 - 16) / 16 = 255 \text{ ключей}$$

Для внутреннего узла — 8 байт ключа плюс 4 байта ссылки, и одна лишняя ссылка:

$$(4096 - 16 - 4) / 12 = 339 \text{ ключей}$$

То есть настоящее значение — примерно 250 для листьев и 340 для внутренних узлов. При таком ветвлении дерево на миллион записей имеет высоту три, а на миллиард — четыре.

Менять `MAX_KEYS` на 250 в учебном коде я не стал сознательно: тогда все примеры этой главы уместились бы в один узел, и ни одного разделения вы бы не увидели. В упражнениях есть задание поменять число и посмотреть, что произойдёт.

35.4 Поиск

Поиск состоит из двух шагов: спуститься до листа, потом найти ключ внутри листа.

```
PageId BPlusTree::find_leaf(std::int64_t key, std::vector<PageId>* path,
                           bool lower) const {
    PageId pid = root_page_id_;
    while (true) {
        NodeData n = read_node(pid);
        if (n.type == Leaf) return pid;
        if (path) path->push_back(pid);

        auto it = lower ? std::lower_bound(n.keys.begin(), n.keys.end(), key)
                        : std::upper_bound(n.keys.begin(), n.keys.end(), key);
        std::size_t idx = static_cast<std::size_t>(it - n.keys.begin());
        pid = n.children[idx];
    }
}
```

Спуск — цикл, а не рекурсия: глубина дерева невелика, но и стек тратить незачем.

Выбор потомка делает `std::upper_bound` — алгоритм из главы 14, находящий первый элемент строго больше искомого. Его позиция и есть номер нужного потомка. Проверим на нашем корне с разделителями [3, 5] и потомками [1, 2, 4]: для ключа 1 `upper_bound` вернёт позицию 0 — идём в страницу 1; для ключа 3 — позицию 1, идём в страницу 2; для ключа 7 — позицию 2, идём в страницу 4. Ровно то, что нужно: разделитель 3 означает «от трёх и выше — направо».

Параметр `lower` переключает алгоритм на `std::lower_bound`, и зачем это нужно, разберём в разделе про дубликаты. Параметр `path` — накопитель пройденных внутренних узлов; он понадобится вставке, чтобы подниматься обратно.

Второй шаг тривиален:

```
bool BPlusTree::find(std::int64_t key, std::int64_t& out_value) const {
    PageId leaf_pid = find_leaf(key, nullptr);
    NodeData leaf = read_node(leaf_pid);
    auto it = std::lower_bound(leaf.keys.begin(), leaf.keys.end(), key);
    if (it == leaf.keys.end() || *it != key) return false;
```

```

    out_value = leaf.values[static_cast<std::size_t>(it - leaf.keys.begin())];
    return true;
}

```

Ключи в листе отсортированы, поэтому ищем двоичным поиском. `lower_bound` возвращает первый элемент **не меньше** искомого; если это конец или не равно — ключа нет.

Цена поиска — по одному чтению страницы на уровень. В нашем демо дерево высоты три, и счётчик показывает четыре обращения (три на спуск плюс повторное чтение листа в `find`):

поиск ключа 13: страниц в файле 11, обращений к страницам 4 (промахов 0)

Промахов ноль — все страницы уже в кэше. Лишнее чтение листа легко убрать, вернув узел из `find_leaf`; это вынесено в упражнения.

35.5 Вставка

Вставка начинается как поиск: спускаемся до листа, запоминая путь.

```

void BPlusTree::insert(std::int64_t key, std::int64_t value) {
    std::vector<PageId> path;
    PageId leaf_pid = find_leaf(key, &path);
    NodeData leaf = read_node(leaf_pid);

    auto it = std::lower_bound(leaf.keys.begin(), leaf.keys.end(), key);
    std::size_t idx = static_cast<std::size_t>(it - leaf.keys.begin());

    if (it != leaf.keys.end() && *it == key) {
        // Replace.
        leaf.values[idx] = value;
        write_node(leaf_pid, leaf);
        return;
    }

    leaf.keys.insert(leaf.keys.begin() + idx, key);
    leaf.values.insert(leaf.values.begin() + idx, value);

    if (leaf.keys.size() <= MAX_KEYS) {
        write_node(leaf_pid, leaf);
        return;
    }
}

```

```

    // Split.
    std::int64_t sep = 0;
    PageId right = split_leaf(leaf_pid, leaf, sep);
    insert_in_parent(path, sep, right);
}

```

Три случая. Ключ уже есть — меняем значение и уходим. Ключ новый и лист не переполнился — вставляем в нужную позицию, чтобы сохранить порядок, и пишем. Лист переполнился — начинается самое интересное.

35.5.1 Разделение листа

```

PageId BPlusTree::split_leaf(PageId leaf_pid, NodeData& leaf,
                             std::int64_t& sep_key) {
    // Разделяем: левый получает first half, правый — second half.
    std::size_t mid = leaf.keys.size() / 2;

    NodeData right;
    right.type = Leaf;
    right.keys.assign(leaf.keys.begin() + mid, leaf.keys.end());
    right.values.assign(leaf.values.begin() + mid, leaf.values.end());
    right.next_leaf = leaf.next_leaf;

    leaf.keys.resize(mid);
    leaf.values.resize(mid);

    PageId right_pid = pm_.allocate_page();
    leaf.next_leaf = right_pid;

    write_node(leaf_pid, leaf);
    write_node(right_pid, right);

    sep_key = right.keys.front(); // первый ключ правого — separator
    return right_pid;
}

```

Половина ключей уезжает в новую страницу, половина остаётся. Разделителем становится **первый ключ правого узла**, и он именно **копируется** вверх, а не переносится: сам ключ со

своим значением остаётся в листе, потому что значения хранятся только в листьях, и потерять его нельзя.

Три строки про цепочку листьев легко проглядеть, а они важны. Новый правый узел наследует ссылку на следующий лист от левого, и только потом левый начинает указывать на правый. Порядок обратный — сначала запомнили, потом перезаписали, — иначе цепочка порвётся, и половина дерева станет недостижимой при обходе.

35.5.2 Разделение внутреннего узла

```
PageId BPlusTree::split_internal(PageId int_pid, NodeData& node,
                                std::int64_t& sep_key) {
    // В internal middle ключ ВЫНОСИТСЯ в parent, а не копируется.
    std::size_t mid = node.keys.size() / 2;
    sep_key = node.keys[mid];

    NodeData right;
    right.type = Internal;
    right.keys.assign(node.keys.begin() + mid + 1, node.keys.end());
    right.children.assign(node.children.begin() + mid + 1, node.children.end());

    node.keys.resize(mid);
    node.children.resize(mid + 1);

    PageId right_pid = pm_.allocate_page();
    write_node(int_pid, node);
    write_node(right_pid, right);
    return right_pid;
}
```

Здесь разделитель **выносится** наверх и в потомках не остаётся — обратите внимание на `mid + 1` в границах правой половины. Разница с листьями принципиальная: во внутреннем узле ключ — это не данные, а указатель направления, и наверху он выполняет ту же роль, что выполнял внизу. Оставить его копию значило бы завести лишнюю развилку.

Если перепутать и скопировать вместо переноса, дерево не сломается сразу — оно станет тихо возвращать неверные результаты на границах диапазонов. Такие ошибки ищут долго, поэтому правило стоит запомнить: **в листьях копируем, во внутренних выносим**.

Потомков в левой половине остаётся `mid + 1`, потому что ключей `mid`. Ещё одно место, где легко ошибиться на единицу.

35.5.3 Подъём наверх

Разделение породило новый узел, и на него надо сослаться из родителя. Родитель тоже может переполниться — тогда разделяется он, и так далее до корня.

```
void BPlusTree::insert_in_parent(const std::vector<PageId>& path,
                                std::int64_t sep_key,
                                PageId right_child) {
    if (path.empty()) {
        // Root split — создаём новый root.
        NodeData new_root;
        new_root.type = Internal;
        new_root.keys.push_back(sep_key);
        new_root.children.push_back(root_page_id_);
        new_root.children.push_back(right_child);
        PageId new_root_pid = pm_.allocate_page();
        write_node(new_root_pid, new_root);
        root_page_id_ = new_root_pid;
        save_metadata();
        return;
    }

    PageId parent_pid = path.back();
    NodeData parent = read_node(parent_pid);

    auto it = std::upper_bound(parent.keys.begin(), parent.keys.end(), sep_key);
    std::size_t idx = static_cast<std::size_t>(it - parent.keys.begin());
    parent.keys.insert(parent.keys.begin() + idx, sep_key);
    parent.children.insert(parent.children.begin() + idx + 1, right_child);

    if (parent.keys.size() <= MAX_KEYS) {
        write_node(parent_pid, parent);
        return;
    }

    // Parent переполнен — split рекурсивно.
    std::int64_t new_sep = 0;
    PageId new_right = split_internal(parent_pid, parent, new_sep);
    std::vector<PageId> grand_path(path.begin(), path.end() - 1);
```

```
insert_in_parent(grand_path, new_sep, new_right);
}
```

Пустой путь означает, что разделился сам корень. Тогда создаётся новый корень с одним ключом и двумя потомками, а номер корня записывается в метаданные — иначе после перезапуска база пошла бы искать данные не оттуда.

Здесь обнаруживается особенность B-деревьев, которая отличает их от прочих: **дерево растёт вверх, а не вниз**. Новые уровни появляются только над корнем, и потому все листья всегда оказываются на одной глубине. Дерево остаётся идеально сбалансированным без каких-либо поворотов и перебалансировок, которыми занимаются красно-чёрные и AVL-деревья.

Вставка ссылки в родителя — $idx + 1$, потому что новый узел встаёт **справа** от разделителя. Ещё одно место для ошибки на единицу; таких мест в B-дереве много, и это главная причина, по которой его считают неприятным в реализации.

Рекурсия в конце копирует путь без последнего элемента. Красивее было бы передавать индекс, но копирование вектора из трёх-четырёх чисел ничего не стоит, а код читается яснее.

35.6 Как это выглядит вживую

Программа `utils/btree_demo.cpp` вставляет ключи по одному и показывает, что происходит с деревом:

```
$ ./build/btree_demo
```

```
MAX_KEYS = 4 (в учебном дереве нарочно мало, чтобы сплиты были видны)
```

вставка	страниц	корень	
1	2	1	
2	2	1	
3	2	1	
4	2	1	
5	4	3	← сплит, новых страниц: 2 ← новый корень
6	4	3	
7	5	3	← сплит, новых страниц: 1
8	5	3	
9	6	3	← сплит, новых страниц: 1
10	6	3	
11	7	3	← сплит, новых страниц: 1
12	7	3	
13	10	9	← сплит, новых страниц: 3 ← новый корень

14	10	9	
15	11	9	← сплит, новых страниц: 1
16	11	9	

Высота дерева: 3

Читается как история роста. Первые четыре ключа помещаются в корень-лист. Пятый переполняет его: лист делится надвое, появляется новый корень — сразу три страницы вместо одной, и высота становится два.

Дальше каждый второй ключ вызывает разделение листа: узлы маленькие, заполняются быстро. Тринадцатый ключ вызывает цепную реакцию — переполняется лист, разделение поднимается в корень, корень тоже переполнен, делится и он, появляется новый корень. Три новых страницы за одну вставку, высота стала три.

Обход по цепочке листьев выдаёт всё в порядке возрастания:

Обход по цепочке листьев: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

А счётчики показывают, во что обходятся операции:

поиск ключа 13: обращений к страницам 4

поиск всех 16 ключей: обращений к страницам 64

полный обход: обращений к страницам 10

Четыре обращения на поиск при высоте три. Полный обход шестнадцати записей — десять обращений, по числу листьев плюс спуск; заметьте, что обход дешевле, чем шестнадцать поисков по одному, — ровно ради этого листья и связаны в цепочку.

35.7 Обход диапазона

Цепочка листьев делает обход тривиальным:

```
void BPlusTree::scan(std::function<void(std::int64_t, std::int64_t)> cb) const {
    // Идём в самый левый лист.
    PageId pid = root_page_id_;
    while (true) {
        NodeData n = read_node(pid);
        if (n.type == Leaf) break;
        pid = n.children.front();
    }
    while (pid != 0) {
```



```

    NodeData leaf = read_node(pid);
    for (std::size_t i = 0; i < leaf.keys.size(); ++i) {
        cb(leaf.keys[i], leaf.values[i]);
    }
    pid = leaf.next_leaf;
}
}

```

Спускаемся по самым левым потомкам до первого листа, дальше идём по ссылкам, пока они есть. Ноль в `next_leaf` означает конец — нулевая страница занята метаданными, поэтому её можно смело использовать как признак «дальше никого».

Обратная сторона такого интерфейса — обход всегда идёт до конца. Настоящий диапазонный запрос должен уметь начаться с заданного ключа и остановиться на другом; для этого нужен либо колбэк, умеющий сказать «хватит», либо итератор. Итератор был бы правильнее и вписался бы в алгоритмы STL, но потребовал бы аккуратной работы с состоянием: текущий лист, позиция в нём, и что делать, если лист вытеснили из кэша. В упражнениях есть задание сделать хотя бы диапазонную версию с ранним выходом.

35.8 Повторяющиеся ключи

До сих пор мы считали, что ключ уникален: `insert` при совпадении заменяет значение. Для первичного ключа это правильно — двух записей с одинаковым идентификатором быть не должно.

А вот для вторичного индекса — нет. Если индекс построен по колонке «город», то на значение «Алматы» приходится тысячи строк, и все они должны сохраниться. Обычный `insert` тут молча теряет данные: каждая следующая строка затирает предыдущую.

Именно это и произошло в первой версии нашей базы. Вставляем три строки, две из которых имеют одинаковое значение, строим индекс — и запрос находит одну:

```

mydb> SELECT * FROM items WHERE value = 100;
id      | value
-----+-----
2       | 100
(1 rows)

```

Строка с идентификатором 1 просто исчезла из выдачи, хотя в таблице она есть. Ошибка неприятная вдвойне: база не падает и ничего не сообщает, она отдаёт неполный ответ.

Лечится это отдельной операцией вставки, которая не заменяет, а добавляет:

```

void BPlusTree::insert_dup(std::int64_t key, std::int64_t value) {
    std::vector<PageId> path;
    PageId leaf_pid = find_leaf(key, &path);
    NodeData leaf = read_node(leaf_pid);

    // upper_bound — новая запись встает ПОСЛЕ уже имеющихся с тем же ключом.
    auto it = std::upper_bound(leaf.keys.begin(), leaf.keys.end(), key);
    std::size_t idx = static_cast<std::size_t>(it - leaf.keys.begin());

    leaf.keys.insert(leaf.keys.begin() + idx, key);
    leaf.values.insert(leaf.values.begin() + idx, value);

    if (leaf.keys.size() <= MAX_KEYS) {
        write_node(leaf_pid, leaf);
        return;
    }

    std::int64_t sep = 0;
    PageId right = split_leaf(leaf_pid, leaf, sep);
    insert_in_parent(path, sep, right);
}

```

Отличие от обычной вставки в двух местах: нет ветки замены, и позиция ищется через `upper_bound`, то есть новая запись становится последней среди равных. Порядок вставки сохраняется, что приятно, хотя и не обязательно.

35.8.1 Ловушка, которую видно не сразу

Теперь надо научиться доставать все записи с одинаковым ключом. Казалось бы, просто: спустились в лист, нашли первое совпадение, идём вправо, пока ключ тот же.

Не работает. Представьте лист с ключами [7, 7, 7, 7, 7], который переполнился и разделился пополам: слева осталось [7, 7], справа оказалось [7, 7, 7], разделителем стало число 7. Теперь обычный спуск с `upper_bound` для ключа 7 уйдёт **направо** — разделитель не больше искомого, — и две левые семёрки останутся ненайденными.

Именно из-за этого в `find_leaf` появился параметр `lower`. Со `std::lower_bound` спуск уходит в самое левое поддерево, где ключ ещё может встретиться:

```

void BPlusTree::find_all(std::int64_t key,
                        std::function<void(std::int64_t)> cb) const {
    PageId pid = find_leaf(key, nullptr, /*lower=*/true);
    while (pid != 0) {
        NodeData leaf = read_node(pid);
        for (std::size_t i = 0; i < leaf.keys.size(); ++i) {
            if (leaf.keys[i] < key) continue;
            if (leaf.keys[i] > key) return;    // отсортировано — дальше только больше
            cb(leaf.values[i]);
        }
        pid = leaf.next_leaf;
    }
}

```

Спустились в левый возможный лист, идём по цепочке вправо, отдавая совпадения, и останавливаемся на первом ключе больше искомого. Дубликаты, разъехавшиеся по нескольким листьям, собираются полностью.

Проверка в тестах гоняет двенадцать записей с одним ключом при `MAX_KEYS = 4` — они гарантированно занимают несколько листьев:

```

for (int i = 1; i <= 12; ++i) tree.insert_dup(7, i);
...
tree.find_all(7, [&](std::int64_t v) { found.push_back(v); });
check_eq(found.size(), 12, "find_all находит все дубликаты");

```

После исправления запрос из начала раздела отвечает правильно:

```
mydb> SELECT * FROM items WHERE value = 100;
```

```

id      | value
-----+-----
1       | 100
2       | 100
(2 rows)

```

Эту историю я рассказываю подробно не ради самого исправления, а ради урока. Ошибка была не в алгоритме дерева — он работал безупречно. Ошибка была в предположении, что вторичный ключ уникален, — предположении, которое никто не записал и не проверил. Такие ошибки не находятся чтением кода; они находятся тестом, который спрашивает: а что, если данные не такие, как я думал?

35.9 Удаление

В нашем дереве его нет, и это самое заметное упрощение Части IV.

Причина не в лени, а в том, что удаление устроено заметно сложнее вставки. Убрать ключ из листа легко, но если лист опустел ниже половины, дерево перестаёт быть сбалансированным: надо либо забрать ключ у соседа (перераспределение), либо слить два узла в один — а слияние может опустошить родителя и подняться до корня, уменьшив высоту дерева. Плюс отдельная работа с разделителями, которые ссылаются на удалённые ключи.

Как поступают взрослые системы, знать полезно. Многие вообще не удаляют записи сразу, а помечают их надгробием (tombstone) и физически убирают позже, при перестроении страницы. PostgreSQL так и делает: DELETE только помечает версию строки мёртвой, а чистит потом VACUUM. Это и быстрее — не надо перестраивать дерево, — и необходимо для многоверсионности: старые транзакции ещё могут читать удалённое.

Реализация удаления с перераспределением и слиянием — хорошее большое упражнение, оно есть в конце главы.

35.10 Чего ещё нет в учебной версии

Несколько вещей, которые отличают наше дерево от промышленного.

Ключи и значения у нас — восьмибайтовые целые. В настоящей базе ключом может быть строка произвольной длины, и это меняет всё: в узле помещается переменное число ключей, а сам узел устроен как массив смещений плюс область данных, растущая с другого конца страницы, — ту же схему мы видели в файле SQLite в главе 32. Кроме того, для строковых ключей применяют сжатие общих префиксов: в узле с ключами Алматы, Алматинская, Алма-Ата общая часть хранится один раз.

Ничего нет и про одновременную работу. Если два потока начнут вставлять одновременно, они разрушат дерево при первом же разделении. Классическое решение — блокировки с перехватом (latch crabbing): спускаясь, держим блокировку на родителе, пока не убедимся, что потомок не переполнится, и только тогда отпускаем. Современные системы идут дальше, к структурам без блокировок вроде Bw-tree.

Наконец, наше дерево плохо заполняется при последовательной вставке. Разделение пополам при вставке возрастающих ключей оставляет левый узел заполненным ровно наполовину — и так навсегда, потому что в него больше никогда не вставят. Файл получается вдвое больше нужного. Базы решают это особым правилом: если ключ вставляется в самый правый узел, делить надо не пополам, а оставив всё слева, — тогда узлы заполняются полностью.

35.11 Главные правила главы

1. Структуры для диска строят вокруг размера страницы: узел размером в страницу даёт ветвление в сотни раз и высоту в три-четыре уровня на миллионах записей.
2. В В+-дереве значения лежат только в листьях, а листья связаны в цепочку — отсюда дешёвый обход диапазона.
3. Потомков в узле на одного больше, чем ключей; это источник большинства ошибок на единицу.
4. При разделении листа разделитель копируется наверх, при разделении внутреннего узла — выносится.
5. Дерево растёт вверх: новый уровень появляется только при разделении корня, поэтому все листья всегда на одной глубине.
6. Номер корня хранится в метаданных и обновляется при каждом разделении корня.
7. Вторичный ключ не уникален; спуск за дубликатами делают через `lower_bound`, иначе часть записей окажется недостижимой.
8. Отсутствие ошибки не означает правильности: неполный ответ на запрос — самая тихая из возможных поломок.

35.12 Упражнения

1. Соберите `demo-db` и запустите `./build/btree_demo`. Проследите по выводу, на каких вставках произошли разделения. Объясните, почему на тринадцатой вставке появилось сразу три новые страницы.
2. Поменяйте `MAX_KEYS` на 250, пересоберите и запустите демо снова. Сколько разделений осталось? Какой стала высота дерева на шестнадцати ключах?
3. Уберите из `find_all` параметр `lower` — спускайтесь обычным способом — и запустите тесты. Какая проверка упала и с каким числом? Объясните, чем расположение дубликатов в этом тесте отличается от простого случая.
4. Сделайте так, чтобы `find` не читал лист повторно: пусть `find_leaf` возвращает уже прочитанный `NodeData`. На сколько уменьшилось число обращений к страницам в демо?
5. Добавьте метод `scan_range(int64 from, int64 to, cb)`, который начинает с нужного листа и останавливается на верхней границе. Сравните число обращений к страницам с полным обходом.
6. Реализуйте правило правого края: при вставке в самый правый лист делить не пополам, а оставлять всё слева. Вставьте сто тысяч возрастающих ключей до и после изменения и сравните размеры файла.

7. (Сложнее) Напишите проверку целостности дерева: все листья на одной глубине, ключи в каждом узле отсортированы, ключи потомка лежат в границах, заданных разделителями родителя, цепочка листьев проходит через все листья ровно один раз. Прогоните её после тысячи случайных вставок.
8. (Сложнее) Реализуйте удаление с перераспределением и слиянием. Начните с простого случая — удаление из листа, который не опустился ниже половины, — и добавьте проверку целостности из предыдущего упражнения в качестве теста.

35.13 Что дальше

Глава 35 — журнал упреждающей записи. Дерево работает, но живёт наполовину в памяти: изменённые страницы лежат в кэше и попадают на диск только при `sync`. Если в этот момент выключить питание, часть данных исчезнет, а часть дерева может оказаться в несогласованном виде — со ссылками на недописанные страницы.

Разберёмся, почему нельзя просто чаще звать `fsync` — мы уже измерили, сколько он стоит, — что такое упреждающая запись, зачем журналу контрольные суммы и как из журнала восстанавливается состояние после аварии. И проверим на деле: убьём базу посреди работы и посмотрим, что она скажет при следующем запуске.

Глава 36

Глава 35. Журнал упреждающей записи

К этому моменту база умеет хранить данные в дереве и быстро их находить. Но живёт она наполовину в памяти: изменённые страницы лежат в кэше и попадают на диск только при `sync`. Если в этот промежуток выключить питание, пропадут не просто последние записи — может пострадать само дерево.

Представьте: разделение листа успело записать новую страницу, но не успело записать родителя со ссылкой на неё. После перезапуска половина ключей окажется недостижимой, а дерево при этом будет выглядеть исправным. Такие поломки называют нарушением согласованности, и они хуже потери данных: потерю видно, а несогласованность обнаруживается через месяц, когда запросы начинают отвечать неправильно.

В этой главе добавим третий слой — журнал упреждающей записи, `write-ahead log`. Идея, придуманная в семидесятых для System R, лежит в основе транзакций во всех современных базах. К концу главы мы убьём нашу базу сигналом посреди работы и убедимся, что она восстановится.

36.1 Почему нельзя просто чаще звать `fsync`

Очевидное решение — синхронизировать страницы после каждой вставки. Оно правильное по надёжности и негодное по скорости, и мы уже знаем почему: в главе 33 замер показал, что `fsync` стоит около 155 микросекунд, тогда как сама запись страницы — около тринадцати.

Но дело даже не в одном вызове. Одна вставка в дерево меняет от одной до нескольких страниц, разбросанных по файлу: лист, при разделении — второй лист и родителя, при

разделении корня — ещё и метаданные. Записать их все синхронно — это несколько разрозненных записей плюс `fsync`, и всё ради одной строки данных.

Хуже того, надёжности это всё равно не даёт. Между записью листа и записью родителя питание может пропасть — и мы получим ровно ту несогласованность, от которой убегали. Нужна не частая синхронизация, а **атомарность**: либо изменение целиком есть, либо его целиком нет.

36.2 Идея упреждающей записи

Приём состоит из одного правила: **сначала запиши, что собираешься сделать, потом делай**.

Порядок действий при вставке становится таким. Пишем в журнал запись «вставить ключ K со значением V» и синхронизируем журнал. Только после этого меняем страницы дерева в памяти. Страницы попадают на диск когда-нибудь потом, при контрольной точке.

Что даёт такой порядок при аварии в любой момент? Если авария случилась до записи в журнал — операции просто не было, база в прежнем состоянии. Если после записи в журнал, но до сброса страниц — при следующем запуске мы прочитаем журнал и повторим операцию над деревом. Если после сброса страниц — журнал уже не нужен, а повторное применение операции ничего не испортит.

Последнее свойство называется идемпотентностью: повторить вставку пары «ключ — значение» в дерево можно сколько угодно раз, результат один и тот же. Именно поэтому нам не нужно точно знать, какие страницы успели дойти до диска, — достаточно повторить всё, что есть в журнале.

Выигрыш в скорости берётся из того, что журнал — **последовательный** файл. Запись в него всегда идёт в конец, головке диска не надо куда прыгать, а `fsync` последовательного файла на порядок дешевле, чем синхронизация нескольких разрозненных страниц.

Проверим на нашей реализации: две тысячи записей в журнал с синхронизацией каждой занимают 96 миллисекунд, то есть 48 микросекунд на запись. Втрое дешевле, чем `fsync` страницы данных, — и это при том, что надёжность выше.

36.2.1 Где здесь транзакции

Ту же схему используют для транзакций, и стоит понимать связь. В журнал пишут записи «начало транзакции», потом операции, потом «фиксация». При восстановлении применяют только те операции, у которых нашлась фиксация; всё остальное отбрасывают

как незавершённое. Отсюда буква A в аббревиатуре ACID — атомарность: транзакция либо есть целиком, либо её нет.

Буква D, устойчивость, обеспечивается тем самым `fsync` журнала: как только он вернулся, операция переживёт выключение питания. Именно поэтому базы так болезненно относятся к честности накопителя — если диск врёт про запись, ломается вся конструкция.

У нас транзакций нет: каждая вставка сама себе транзакция и считается зафиксированной сразу. Это упрощение, а не другой подход — добавить границы транзакций к готовому журналу несложно, и в упражнениях есть такое задание.

36.3 Формат записи

Журнал — последовательность записей, каждая описывает одну операцию. Наш формат объявлен в `include/wal.h`:

Layout файла — последовательность записей:

```
u32 length    (длина полезной нагрузки, всегда 17)
u8  type      (1 = insert)
i64 key
i64 value
u32 crc       (от type+key+value)
```

Итого 25 байт на запись.

Три части: длина, полезная нагрузка, контрольная сумма. Каждая нужна.

Длина спереди позволяет пропустить запись, не разбирая её, и — что важнее — обнаружить обрыв: если до конца файла осталось меньше, чем обещает длина, значит, запись не дописана.

Тип операции нужен для расширения. Сейчас он всегда единица, но когда появятся удаление и границы транзакций, потребуются другие значения. Незнакомый тип при восстановлении — повод остановиться, а не гадать.

Контрольная сумма в конце ловит повреждение данных. Она вычисляется по нагрузке и проверяется при чтении; несовпадение означает, что запись испорчена и доверять ей нельзя.

Посмотрим на настоящий журнал после двух вставок:

```
$ xxd /tmp/waldemo/data.wal
00000000: 1100 0000 0101 0000 0000 0000 0064 0000  ....d..
00000010: 0000 0000 00fe 9b83 7911 0000 0001 0200  ....y.....
00000020: 0000 0000 0000 c800 0000 0000 0000 f900  ....
00000030: 0bef                                     ..
```

Первая запись: 11 00 00 00 — длина 17, 01 — тип «вставка», восемь байт ключа (единица), восемь байт значения (64 — это 100), четыре байта контрольной суммы. Со смещения 25 начинается вторая запись — ключ 2, значение с8, то есть 200.

36.3.1 Про нашу контрольную сумму

Честное предупреждение, которое есть и в коде:

```
std::uint32_t crc32_simple(const unsigned char* data, std::size_t n) {  
    // Очень простой полиномиальный хеш — не настоящий CRC32.  
    // Этого достаточно для отлова случайных повреждений в учебной СУБД.  
    std::uint32_t h = 0xCAFEBAFEu;  
    for (std::size_t i = 0; i < n; ++i) {  
        h = h * 31u + data[i];  
    }  
    return h;  
}
```

Это не CRC32, а обычный полиномиальный хеш — тот самый, которым в Java считается хеш строки. Он ловит случайные повреждения (перевёрнутый бит, обрезанный блок) и совершенно не годится против намеренной подделки: подобрать данные с нужным значением тривиально.

Настоящий CRC32 построен на арифметике многочленов над полем из двух элементов и даёт гарантии: он обнаруживает любую одиночную ошибку, любую двойную, любой пакет ошибок длиной до 32 бит. У полиномиального хеша таких гарантий нет — он просто «скорее всего заметит». В настоящей базе берут CRC32C, у которого есть аппаратная поддержка в процессоре, так что он ещё и быстрее наивной реализации.

Заменить нашу функцию на настоящий CRC32 — упражнение в конце главы: таблица на 256 элементов и цикл в четыре строки.

36.4 Запись в журнал

```
void WAL::log_insert(std::int64_t key, std::int64_t value) {  
    unsigned char payload[PAYLOAD_BYTES];  
    payload[0] = 1;    // тип = insert  
    std::uint64_t uk = static_cast<std::uint64_t>(key);  
    std::uint64_t uv = static_cast<std::uint64_t>(value);
```

```

for (int i = 0; i < 8; ++i) payload[1 + i] = static_cast<unsigned char>((uk >> (i * 8)) & 0xFF);
for (int i = 0; i < 8; ++i) payload[9 + i] = static_cast<unsigned char>((uv >> (i * 8)) & 0xFF);
std::uint32_t crc = crc32_simple(payload, PAYLOAD_BYTES);

unsigned char rec[RECORD_BYTES];
std::uint32_t len = PAYLOAD_BYTES;
for (int i = 0; i < 4; ++i) rec[i] = static_cast<unsigned char>((len >> (i * 8)) & 0xFF);
std::memcpy(rec + 4, payload, PAYLOAD_BYTES);
for (int i = 0; i < 4; ++i) rec[4 + PAYLOAD_BYTES + i] = static_cast<unsigned char>((crc >> (i * 8)) & 0xFF);

if (!write_all(fd_, rec, RECORD_BYTES)) {
    throw WALError(std::string("WAL write: ") + std::strerror(errno));
}
if (::fsync(fd_) < 0) {
    throw WALError(std::string("WAL fsync: ") + std::strerror(errno));
}
}

```

Запись собирается целиком в буфере на стеке и отдаётся ядру **одним** вызовом `write`. Это не микрооптимизация, а требование корректности: три отдельные записи (заголовок, тело, сумма) могли бы прерваться посередине, и в журнале осталась бы запись с правильным заголовком и мусором вместо тела.

Файл открыт с флагом `O_APPEND`:

```
fd_ = ::open(path.c_str(), O_RDWR | O_CREAT | O_APPEND, 0644);
```

Флаг делает запись атомарной относительно позиции: ядро само помещает данные в конец файла, и два процесса, пишущих в один журнал, не затрут друг друга. Мы это свойство обсуждали в главе 29, когда разбирали перенаправление `>>`.

`fsync` после каждой записи — то место, где покупается устойчивость. Убрать его — и вставки станут в разы быстрее, а гарантия «подтверждённое не теряется» исчезнет.

Вспомогательные `write_all` и `read_all` — знакомые циклы вокруг системных вызовов с обработкой частичных операций и `EINTR`; впервые мы их написали в главе 31 для `myscat`.

36.5 Восстановление

При открытии базы журнал читается с начала, и каждая целая запись применяется к дереву:

```

void WAL::replay(std::function<void(std::int64_t, std::int64_t)> cb) {
    // Открываем отдельный fd для чтения с offset 0.
    int rfd = ::open(path_.c_str(), O_RDONLY);
    if (rfd < 0) {
        throw WLError("WAL read open: " + path_);
    }

    unsigned char header[4];
    while (read_all(rfd, header, 4)) {
        std::uint32_t len = 0;
        for (int i = 0; i < 4; ++i) len |= static_cast<std::uint32_t>(header[i]) << (i * 8);
        if (len != PAYLOAD_BYTES) {
            // Битый заголовок – обрыв.
            break;
        }
        unsigned char body[PAYLOAD_BYTES];
        if (!read_all(rfd, body, PAYLOAD_BYTES)) break;
        unsigned char tail[4];
        if (!read_all(rfd, tail, 4)) break;
        std::uint32_t got_crc = 0;
        for (int i = 0; i < 4; ++i) got_crc |= static_cast<std::uint32_t>(tail[i]) << (i * 8);
        if (got_crc != crc32_simple(body, PAYLOAD_BYTES)) {
            // Повреждение – обрываем replay.
            break;
        }
        if (body[0] != 1) {
            break; // неизвестный тип записи
        }
        std::uint64_t uk = 0, uv = 0;
        for (int i = 0; i < 8; ++i) uk |= static_cast<std::uint64_t>(body[1 + i]) << (i * 8);
        for (int i = 0; i < 8; ++i) uv |= static_cast<std::uint64_t>(body[9 + i]) << (i * 8);
        cb(static_cast<std::int64_t>(uk), static_cast<std::int64_t>(uv));
    }

    ::close(rfd);
}

```

Ключевое проектное решение здесь — что делать при первой же испорченной записи. Мы **останавливаемся**, а не пропускаем её и не пытаемся читать дальше.

Логика такая. Журнал пишется последовательно, и повреждение почти всегда означает обрыв на конце: питание пропало посреди записи. Всё, что было раньше, — целое; всё, что после, — не существует. Значит, правильное поведение — применить целую часть и объявить, что на этом данные кончились. Попытка перескочить через испорченный кусок привела бы к применению записей, которые, возможно, никогда не были подтверждены пользователю.

Обратите внимание на отдельный дескриптор для чтения. Основной открыт с `O_APPEND` и предназначен для записи; читать через него, перематывая позицию, было бы источником путаницы. Отдельный дескриптор на чтение — дешевле и понятнее.

Тесты проверяют оба вида порчи. Обрыв на середине записи:

```
int fd = ::open(path.c_str(), O_WRONLY);
::ftruncate(fd, 25 + 10); // вторая запись обрезана посередине
::close(fd);

db::WAL wal(path);
int count = 0;
wal.replay([&](std::int64_t, std::int64_t) { ++count; });
check_eq(count, 1, "оборванная запись в конце игнорируется");
```

И порча байта внутри записи, которую ловит контрольная сумма:

```
int fd = ::open(path.c_str(), O_RDWR);
unsigned char bad = 0xFF;
::pwrite(fd, &bad, 1, 25 + 6); // портим ключ второй записи
::close(fd);
// ... replay даёт одну запись вместо двух
```

36.6 Контрольная точка

Журнал не может расти вечно. Раз в какое-то время база сбрасывает все грязные страницы на диск, и тогда журнал становится ненужным — состояние дерева на диске уже включает всё, что в нём записано.

```
void Database::checkpoint() {
    pm_>sync();
    wal_>truncate();
    for (auto& kv : indexes_) {
```

```

        kv.second.pm->sync();
        kv.second.wal->truncate();
    }
}

```

Порядок здесь строгий и обратный порядку записи: **сначала данные, потом обрезаем журнал**. Если сделать наоборот и авария случится между двумя действиями, журнал уже пуст, а страницы ещё не записаны — данные потеряны безвозвратно. Это одно из тех мест, где перестановка двух строк превращает надёжную систему в ненадёжную.

Обрезание — вызов `ftruncate`:

```

void WAL::truncate() {
    if (::ftruncate(fd_, 0) < 0) {
        throw WALError(std::string("ftruncate: ") + std::strerror(errno));
    }
    // O_APPEND следит, чтобы новые записи писались с offset 0 после truncate.
    if (::fsync(fd_) < 0) {
        throw WALError(std::string("WAL truncate fsync: ") + std::strerror(errno));
    }
}

```

Флаг `O_APPEND` выручает и здесь: после обрезания следующая запись автоматически пойдёт с нулевого смещения, хотя позиция дескриптора могла остаться где-то в середине.

Настоящие базы устраивают контрольные точки хитрее: не по команде, а по расписанию или по объёму журнала, и не мгновенно, а размазывая запись страниц во времени, чтобы не создавать всплеск нагрузки на диск. В PostgreSQL этим занимается отдельный процесс, а параметры вроде `checkpoint_timeout` и `max_wal_size` — предмет постоянной настройки. У нас контрольная точка вызывается вручную командой `.checkpoint` и автоматически при выходе.

36.7 Проверим на аварии

Теория закончилась, устроим базе настоящую смерть. Запускаем, вставляем три записи и не даём завершиться штатно — убиваем сигналом, который нельзя перехватить:

```

$ (printf 'INSERT INTO items VALUES (1, 100);\nINSERT INTO items VALUES (2, 200);\nINSERT INTO items V
  | ./build/mydb /tmp/crashtest &
$ sleep 2; pkill -9 mydb

```

Девятый сигнал не даёт ни единого шанса прибратсья: ни деструкторы, ни контрольная точка не выполняются (глава 28). Смотрим, что осталось на диске:

```
$ ls -l /tmp/crashtest/
-rw-r--r--  1 serik  wheel  8192 data.db
-rw-r--r--  1 serik  wheel    75 data.wal
```

Журнал — 75 байт, ровно три записи по 25. Файл данных существует, но что в нём — неизвестно: страницы могли и не дойти. Запускаем базу заново:

```
$ ./build/mydb /tmp/crashtest
mydb> SELECT * FROM items;
id      | value
-----+-----
1       | 100
2       | 200
3       | 300
(3 rows)
```

Все три записи на месте. При открытии Database прочитал журнал и применил операции к дереву:

```
Database::Database(const std::string& dir) : dir_(dir) {
    ::mkdir(dir.c_str(), 0755);

    pm_ = mk<PageManager>(dir + "/data.db", 64);
    tree_ = mk<BPlusTree>(*pm_);
    wal_ = mk<WAL>(dir + "/data.wal");

    wal_>replay([this](std::int64_t k, std::int64_t v) {
        tree_>insert(k, v);
    });
    ...
}
```

Четыре строки, и база переживает выдёргивание питания. Ради этого всё и затевалось.

Обратите внимание, что мы не проверяли, дошли ли страницы дерева до диска, — мы просто применили журнал заново. Если страницы дошли, повторная вставка тех же пар ничего не изменит; если не дошли, она их восстановит. Идемпотентность операции избавляет от необходимости знать, что именно успело записаться.

36.8 Сколько это стоит

Соберём числа вместе. Две тысячи вставок через полный путь — журнал с синхронизацией плюс дерево в памяти:

2000 вставок с fsync на каждую: 1.03 с (515 мкс на вставку, 1943 вставок/с)

Отдельно журнал, без дерева:

2000 записей в WAL (fsync на каждую): 96 мс (48 мкс на запись), файл 50000 байт

replay 2000 записей: 5107 мкс

Журнал сам по себе занимает меньше десятой доли времени; остальное уходит на дерево — на наши расточительные копии страниц и разбор узлов в вектора. Восстановление двух тысяч записей — пять миллисекунд, то есть при аварии база стартует практически мгновенно.

Тысяча девятьсот вставок в секунду — скромно, но это и есть цена честной устойчивости на каждую отдельную операцию. Настоящие базы обходят её групповой фиксацией: если одновременно пришли сто транзакций, их записи складываются в буфер и синхронизируются **одним** fsync. Каждая транзакция ждёт чуть дольше, зато пропускная способность растёт в десятки раз. Реализовать групповую фиксацию у нас нигде — база однопоточная и запросы приходят по одному, — но знать про приём стоит: это первое, что делают, когда упираются в скорость журнала.

36.9 Разбор по моментам аварии

Полезно проговорить все точки, в которых может пропасть питание, и что произойдёт в каждой. Возьмём одну вставку и пройдем по ней шаг за шагом.

Момент аварии	Что на диске	Что будет при запуске
до write в журнал	ничего нового	операции не было — как будто пользователь её не вводил
после write, до fsync	запись, возможно, в кэше ядра	может быть, а может не быть; пользователю мы ещё не ответили — обе развязки честные
после fsync журнала	запись в журнале точно есть	replay применит её к дереву
после изменения страниц в памяти	там же, что и выше	то же самое: страницы всё равно не на диске
после sync страниц, до truncate	и данные, и журнал	replay применит запись повторно — ничего не изменится

Момент аварии	Что на диске	Что будет при запуске
после truncate	только данные	журнал пуст, данные на месте

Ни в одной строке нет варианта «данные потеряны после того, как мы ответили пользователю» — а это и есть определение устойчивости. Есть вариант «операция не выполнялась, и мы не успели ответить», и он допустим.

Отдельно стоит вторая строка. Между write и fsync существует окно, в котором запись есть в кэше ядра, но не на диске. Именно поэтому log_insert возвращается только после fsync, а Database::execute_insert печатает 1 row уже после этого. Порядок «сначала сделать устойчивым, потом подтвердить» — не формальность: поменяйте местами, и база начнёт врать пользователю.

36.10 Как это устроено у взрослых

Наш журнал повторяет схему, принятую в PostgreSQL, MySQL и SQLite в режиме WAL, но существует и другой подход, и его тоже полезно знать.

SQLite по умолчанию работает не так. У него журнал отката: перед изменением страницы её **старая** версия копируется в отдельный файл, и только потом страница меняется в основном файле. При аварии восстановление возвращает старые версии на место — то есть откатывает незавершённую транзакцию. Файл этот видно:

```
$ python3 -c "... " # открыли транзакцию, вставили строку, не зафиксировали
во время транзакции: ['rj.db', 'rj.db-journal']
после фиксации:      ['rj.db']
```

Файл rj.db-journal появляется на время транзакции и исчезает после фиксации. Логика обратная нашей: мы пишем в журнал новое и накатываем при восстановлении, SQLite пишет старое и откатывает.

С 2010 года у SQLite есть и наш режим — включается командой PRAGMA journal_mode=WAL:

```
$ sqlite3 wj.db "PRAGMA journal_mode=WAL; CREATE TABLE t(a); INSERT INTO t VALUES (1);"
wal
$ ls -l wj.db*
-rw-r--r-- 8192 wj.db
-rw-r--r--    0 wj.db-wal
-rw-r--r-- 32768 wj.db-shm
```

Появился файл `-wal` — тот самый журнал с новыми данными, и `-shm` — разделяемая память для координации между процессами.

Зачем понадобился второй режим, если первый работал двадцать лет? Из-за читателей. При журнале отката изменение идёт прямо в основной файл, поэтому читать во время записи нельзя — читатели ждут писателя. При упреждающей записи основной файл не трогают до контрольной точки, и читатели спокойно работают со старой версией, пока писатель дописывает журнал. Отсюда правило, которое можно встретить в советах по SQLite: если у вас много одновременных читателей, включайте режим WAL.

Ещё одна разница — в цене фиксации. Журналу отката приходится синхронизировать и журнал, и основной файл; упреждающей записи — только журнал. На нагрузке из множества мелких транзакций разница заметна.

36.11 Чего нет в нашем журнале

Список честных упрощений.

В журнал попадают только вставки. Изменения структуры дерева не журналируются — впрочем, они и не нужны, раз дерево восстанавливается повторным применением вставок.

Нет границ транзакций: каждая вставка фиксируется сама по себе, откатить группу операций нельзя.

Нет отмены. Настоящий журнал хранит не только «что сделать», но и «как вернуть» — записи redo и undo, — потому что при аварии на диске могут оказаться страницы незавершённой транзакции, и их нужно откатить. Нам это не нужно ровно потому, что незавершённых транзакций у нас не бывает.

Нет номеров записей журнала. Настоящие системы помечают каждую страницу номером последней применённой к ней записи и при восстановлении применяют только то, что новее, — иначе повторное применение неидемпотентных операций испортило бы данные. Наш ход конём — сделать все операции идемпотентными — работает только потому, что база очень простая.

И нет сжатия журнала при восстановлении: если в журнале миллион вставок одного и того же ключа, мы применим их все по очереди.

36.12 Главные правила главы

1. Сначала журнал, потом данные — правило упреждающей записи, из которого следует всё остальное.

2. Устойчивость даёт не запись, а `fsync`; журнал синхронизируют, страницы данных — нет.
3. Последовательная запись в конец файла на порядок дешевле разрозненных записей по всему файлу.
4. Запись журнала собирают целиком в буфере и отдают одним вызовом — иначе она может порваться посередине.
5. Контрольная сумма ловит повреждение; при первой испорченной записи восстановление останавливается, а не пропускает её.
6. В контрольной точке порядок обратный: сначала сбросить страницы, потом обрезать журнал.
7. Идемпотентность операций избавляет от необходимости знать, какие страницы успели записаться.
8. Групповая фиксация — первый приём, когда упираются в скорость журнала: много операций, один `fsync`.

36.13 Упражнения

1. Повторите опыт с аварией: запустите базу, вставьте несколько записей, убейте её через `kill -9`, запустите снова и проверьте данные. Затем сделайте `.checkpoint` перед убийством — что изменилось в размере журнала?
2. Испортите журнал руками: найдите в `xxd` первый байт второй записи и поменяйте его. Запустите базу — сколько записей восстановилось и почему именно столько?
3. Уберите `fsync` из `log_insert`, пересоберите и замерьте скорость вставки. Во сколько раз быстрее? Объясните, что именно вы потеряли вместе с этими микросекундами.
4. Поменяйте местами две строки в `checkpoint` — сначала `truncate`, потом `sync`. Опишите сценарий, при котором такая перестановка теряет данные, и воспроизведите его через `kill -9`.
5. Замените полиномиальный хеш на настоящий CRC32: таблица на 256 элементов, полином `0xEDB88320`. Проверьте, что старые журналы после этого перестают читаться, и объясните, почему это правильное поведение.
6. Добавьте в журнал тип записи «удаление» (значение 2) и версию формата в начало файла. Продумайте, что должно происходить при чтении журнала с незнакомой версией.
7. (Сложнее) Добавьте границы транзакций: записи `BEGIN` и `COMMIT`. При восстановлении применяйте только операции между парой `BEGIN/COMMIT`, а незавершённый хвост отбрасывайте.

8. (Сложнее) Реализуйте групповую фиксацию: пусть `log_insert` только складывает запись в буфер, а отдельный метод `commit()` пишет накопленное одним вызовом и делает один `fsync`. Замерьте скорость на тысяче вставок с фиксацией каждые сто.

36.14 Что дальше

Глава 36 — мини-SQL. У нас есть дерево и журнал, но общаться с базой приходится вызовами методов на C++. Напишем свой диалект: `CREATE TABLE`, `INSERT`, `SELECT` с условием, `CREATE INDEX` и `EXPLAIN`. Лексер, разбор рекурсивным спуском, дерево разбора — всё то же, что мы делали для командной строки в главе 29, только грамматика посерьёзнее.

Глава 37

Глава 36. Мини-SQL

Хранилище готово: страницы, дерево, журнал. Но чтобы вставить строку, пользователю пришлось бы писать на C++ и знать про `BPlusTree::insert`. Между человеком и хранилищем не хватает языка.

В этой главе напишем свой диалект SQL — маленький, но настоящий: `CREATE TABLE`, `INSERT`, `SELECT` с условием, `CREATE INDEX` и `EXPLAIN`. После этого с базой можно будет разговаривать так:

```
SELECT * FROM items WHERE value > 250;
```

Разбор устроен так же, как токенизатор командной строки из главы 29: сначала лексер режет текст на токены, потом парсер собирает из токенов дерево. Разница в том, что грамматика здесь заметно богаче, и разбирать её будем методом рекурсивного спуска — тем самым, которым устроено большинство компиляторов.

Это четвёртый разбор в книге: мир RPG в главе 23, сохранение в главе 24, командная строка в главе 29 и теперь SQL. К этому моменту схема должна быть уже знакомой, и в главе будет больше внимания к тому, что отличает язык запросов от остальных.

37.1 Зачем вообще язык

Вызов `tree.insert(1, 100)` — это императивный интерфейс: вы говорите, **что делать**, и знаете, с какой структурой данных работаете. Если завтра дерево заменят на хеш-таблицу, весь код, который им пользовался, придётся переписать.

SQL устроен наоборот. Запрос описывает, **что нужно получить**, и молчит о том, как это делать:

```
SELECT * FROM items WHERE value > 250;
```

Ни слова о деревьях, индексах и порядке обхода. Решение о способе выполнения принимает база — и может решить по-разному в зависимости от того, есть ли индекс и сколько строк в таблице. Именно это разделение и сделало реляционные базы тем, чем они стали: запросы, написанные в семидесятых, работают на движках, о которых их авторы не подозревали.

Обратная сторона — планировщик может ошибиться, и тогда запрос, работавший миллисекунду, начинает работать минуту. Отсюда команда EXPLAIN, показывающая выбранный план; мы её тоже сделаем, в следующей главе.

37.2 Что поддерживаем

Грамматика нашего диалекта уместается в несколько строк:

```
CREATE TABLE <имя> (<колонка> INT [PRIMARY KEY], <колонка> INT)
CREATE INDEX <имя> ON <таблица> (<колонка>)
INSERT INTO <таблица> VALUES (<число>, <число>)
[EXPLAIN] SELECT <колонки> | * FROM <таблица> [WHERE <колонка> <оператор> <число>]
```

Операторы сравнения — все шесть: =, !=, <, <=, >, >=.

Чего нет, тоже стоит проговорить сразу: нет DELETE и UPDATE (в дереве нет удаления, глава 34), нет соединения таблиц, нет AND и OR в условии, нет строковых колонок, нет ORDER BY, GROUP BY и LIMIT. Таблица в базе одна, колонок в ней ровно две, обе целые.

Такое подмножество выглядит скромно, но его хватает, чтобы показать всю схему целиком: от текста запроса до обращения к дереву. Каждая недостающая возможность — это либо ещё одна ветка в парсере, либо ещё один способ выполнения, а каркас остаётся тем же.

37.3 Три ступени

Обработка запроса проходит те же три ступени, что и обработка команды в оболочке:

текст запроса

↓ лексер — режет на токены, узнаёт ключевые слова и числа

токены

↓ парсер — собирает Statement по правилам грамматики

Statement

↓ исполнитель — обращается к дереву и журналу, печатает результат

результат

Разделение даёт то же, что и раньше: каждая ступень проверяема отдельно, а сообщения об ошибках получаются осмысленными — лексер жалуется на неизвестный символ, парсер на неожиданный токен, исполнитель на неизвестную таблицу.

37.4 Лексер

Токен у нас — пара из типа и текста:

```
struct Token {
    enum Type {
        kIdent,    // имя таблицы/колонки
        kKeyword,  // CREATE, TABLE, INSERT, INTO, ...
        kNumber,
        kStar,     // *
        kComma,
        kLParen,
        kRParen,
        kOp,       // =, <, >, <=, >=, !=
        kSemi,
        kEnd
    };
    Type type;
    std::string text;
};
```

Тип `kEnd` — не настоящий токен, а метка конца, которую лексер дописывает в конец списка. С ней парсер может всегда смотреть на `peek()` без проверки на выход за границу — приём, экономящий десяток проверок.

Сам разбор — цикл по символам:

```
std::vector<Token> tokenize(const std::string& s) {
    std::vector<Token> out;
    std::size_t i = 0;
    while (i < s.size()) {
        char c = s[i];
        if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue; }

        if (c == '(') { out.push_back({Token::kLParen, "("}); ++i; continue; }
        if (c == ')') { out.push_back({Token::kRParen, ")"}); ++i; continue; }
```

```

if (c == ',') { out.push_back({Token::kComma, ","}); ++i; continue; }
if (c == ';') { out.push_back({Token::kSemi, ";"}); ++i; continue; }
if (c == '*') { out.push_back({Token::kStar, "*"}); ++i; continue; }

if (c == '=' || c == '<' || c == '>' || c == '!') {
    std::string op(1, c);
    if (i + 1 < s.size() && s[i + 1] == '=') { op += '='; i += 2; }
    else { ++i; }
    if (op != "=" && op != "<" && op != ">" && op != "<=" && op != ">=" && op != "!=") {
        throw SqlError("плохой оператор: " + op);
    }
    out.push_back({Token::kOp, op});
    continue;
}

if (std::isdigit(static_cast<unsigned char>(c)) ||
    (c == '-' && i + 1 < s.size() && std::isdigit(static_cast<unsigned char>(s[i + 1])))) {
    std::size_t j = i;
    if (c == '-') ++j;
    while (j < s.size() && std::isdigit(static_cast<unsigned char>(s[j]))) ++j;
    out.push_back({Token::kNumber, s.substr(i, j - i)});
    i = j;
    continue;
}

if (std::isalpha(static_cast<unsigned char>(c)) || c == '_') {
    std::size_t j = i;
    while (j < s.size() && (std::isalnum(static_cast<unsigned char>(s[j])) || s[j] == '_')) ++j;
    std::string text = s.substr(i, j - i);
    std::string upper = to_upper(text);
    if (is_keyword(upper)) {
        out.push_back({Token::kKeyword, upper});
    } else {
        out.push_back({Token::kIdent, text});
    }
    i = j;
    continue;
}

```



```
        throw SqlError(std::string("неизвестный символ: '" + c + "'");
    }
    out.push_back({Token::kEnd, ""});
    return out;
}
```

Разберём места, где принимались решения.

Односимвольные знаки разбираются первыми и без затей. Операторы сравнения требуют заглядывания вперёд: увидев <, надо проверить, не идёт ли следом =. Это тот же приём, что при разборе >> в командной строке.

Отрицательные числа распознаются по минусу перед цифрой. Решение спорное: в настоящем SQL минус — это оператор, и value - 5 должно разбираться как три токена. У нас арифметики нет вовсе, поэтому минус может означать только знак числа, и лексер сразу приклеивает его. Как только появится сложение, это придётся переделать.

Ключевые слова отличаются от имён проверкой по списку:

```
bool is_keyword(const std::string& u) {
    static const std::vector<std::string> kw = {
        "CREATE", "TABLE", "INT", "PRIMARY", "KEY",
        "INDEX", "ON",
        "INSERT", "INTO", "VALUES",
        "SELECT", "FROM", "WHERE",
        "EXPLAIN",
    };
    for (const auto& k : kw) if (u == k) return true;
    return false;
}
```

Сравнение идёт по версии в верхнем регистре — поэтому select, Select и SELECT работают одинаково, как и положено в SQL. А вот имена таблиц и колонок сохраняются как есть: в тексте токена лежит исходное написание, и items не превращается в ITEMS. Это соответствует поведению большинства баз, где ключевые слова регистронезависимы, а имена — как повезёт (в PostgreSQL они приводятся к нижнему регистру, в MySQL зависят от файловой системы).

Список из четырнадцати слов перебирается линейно. Для четырнадцати это быстрее хеш-таблицы: сравнение коротких строк стоит наносекунды, а хеширование — тоже наносекунды, но с накладными расходами. Когда слов станет сто, стоит взять std::unordered_set.

Приведение `static_cast<unsigned char>` перед `isspace`, `isdigit` и `isalpha` обязательно, и это одна из самых известных ловушек стандартной библиотеки C. Эти функции принимают `int`, и передавать в них отрицательное значение — неопределённое поведение. А `char` на x86 знаковый, и любой байт русского текста в UTF-8 отрицателен. Забыть приведение — значит получить падение или мусор на первом же запросе с кириллицей в имени.

37.5 Парсер

Разбор идёт методом рекурсивного спуска: на каждое правило грамматики — своя функция, которая читает токены слева направо и при необходимости зовёт другие функции. Состояние — позиция в списке токенов:

```
struct Parser {
    const std::vector<Token>& tokens;
    std::size_t pos = 0;

    const Token& peek() const { return tokens[pos]; }
    Token consume() { return tokens[pos++]; }

    Token expect(Token::Type t) {
        if (peek().type != t) {
            throw SqlError("ожидался токен, получено '" + peek().text + "'");
        }
        return consume();
    }

    Token expect_keyword(const std::string& kw) {
        if (peek().type != Token::kKeyword || peek().text != kw) {
            throw SqlError("ожидалось '" + kw + "', получено '" + peek().text + "'");
        }
        return consume();
    }
    ...
};
```

Три метода — `peek`, `consume`, `expect` — составляют весь инструментарий. `peek` смотрит, не сдвигаясь; `consume` берёт и сдвигается; `expect` берёт, если тип совпал, и бросает исключение, если нет. Дальше вся грамматика выражается через них.

Точка входа выбирает разбор по первому слову:

```

Statement parse() {
    if (peek().type != Token::kKeyword) {
        throw SqlError("ожидался keyword");
    }
    if (peek().text == "EXPLAIN") {
        consume();
        Statement st = parse_select();
        st.is_explain = true;
        return st;
    }
    const std::string& kw = peek().text;
    if (kw == "CREATE") return parse_create();
    if (kw == "INSERT") return parse_insert();
    if (kw == "SELECT") return parse_select();
    throw SqlError("неизвестный statement: " + kw);
}

```

EXPLAIN обрабатывается как приставка: съели слово, разобрали обычный SELECT, поставили флаг. Так же он устроен и в настоящих базах — это не отдельная команда, а модификатор.

Разбор вставки — самая простая из функций и хорошо показывает стиль:

```

Statement parse_insert() {
    Statement st;
    st.kind = Statement::kInsert;
    expect_keyword("INSERT");
    expect_keyword("INTO");
    st.table = expect(Token::kIdent).text;
    expect_keyword("VALUES");
    expect(Token::kLParen);
    while (true) {
        st.values.push_back(parse_number());
        if (peek().type == Token::kComma) { consume(); continue; }
        break;
    }
    expect(Token::kRParen);
    if (peek().type == Token::kSemi) consume();
    return st;
}

```

Функция читается как сама грамматика: INSERT, INTO, имя, VALUES, скобка, числа через запятую, закрывающая скобка. В этом и состоит удобство рекурсивного спуска — код совпадает с описанием языка, и по нему видно, что именно принимается.

Цикл по списку значений — типичный образец «элемент, потом пока есть запятая — ещё элемент». Он не позволяет ни пустого списка, ни висящей запятой, и это правильно: VALUES () и VALUES (1,) должны быть ошибками.

Точка с запятой в конце необязательна — мы её съедаем, если она есть. В REPL завершающая точка с запятой обязательна по другой причине: по ней он понимает, что команда закончилась и её пора выполнять (глава 38).

Разбор выборки чуть сложнее из-за необязательных частей:

```
Statement parse_select() {
    Statement st;
    st.kind = Statement::kSelect;
    expect_keyword("SELECT");
    if (peek().type == Token::kStar) {
        consume();
        st.select_columns.push_back("*");
    } else {
        while (true) {
            st.select_columns.push_back(expect(Token::kIdent).text);
            if (peek().type == Token::kComma) { consume(); continue; }
            break;
        }
    }
    expect_keyword("FROM");
    st.table = expect(Token::kIdent).text;
    if (peek().type == Token::kKeyword && peek().text == "WHERE") {
        consume();
        st.has_where = true;
        st.where_col = expect(Token::kIdent).text;
        st.where_op = expect(Token::kOp).text;
        st.where_val = parse_number();
    }
    if (peek().type == Token::kSemi) consume();
    return st;
}
```

Необязательные части разбираются по одной схеме: заглянули через peek, и если там ожидаемое слово — разобрали, иначе прошли мимо. Флаг has_where нужен, потому что отсутствие условия и условие «всегда истинно» — разные вещи для планировщика.

Условие у нас предельно жёсткое: колонка, оператор, число. Ни `5 > value`, ни `value > other_column`, ни скобок. Полноценное выражение потребовало бы разбора с приоритетами операций — того самого, что пишут для калькуляторов, — и дерева выражений вместо трёх полей. Это большое, но полезное упражнение, оно есть в конце главы.

37.5.1 Дерево разбора

Результат — одна структура на все виды команд:

```
struct Statement {
    enum Kind { kCreate, kCreateIndex, kInsert, kSelect };
    Kind kind = kSelect;

    bool is_explain = false;
    std::string table;

    // CREATE TABLE: список (имя_колонки, тип).
    // CREATE INDEX: только колонки[0].first = имя индексируемой колонки.
    std::vector<std::pair<std::string, std::string>> columns;
    std::string index_name;

    // INSERT: значения по позициям.
    std::vector<std::int64_t> values;

    // SELECT: какие колонки (или "*"), плюс опциональное WHERE.
    std::vector<std::string> select_columns;
    bool has_where = false;
    std::string where_col;
    std::string where_op;
    std::int64_t where_val = 0;
};
```

Одна структура с полями на все случаи — решение спорное, и стоит объяснить его честно. Правильнее была бы иерархия классов с общим предком (`CreateStatement`, `InsertStatement`, `SelectStatement`) и виртуальным методом выполнения — ровно то, что мы делали для предметов RPG в главе 17. Или, если бы у нас был C++17, — `std::variant`, о котором пойдёт речь в главе 48.

Но для четырёх видов команд с десятком полей иерархия добавила бы больше кода, чем сэкономила. Цена такого решения — половина полей в каждом конкретном разборе не используется, и об этом приходится помнить. Как только видов станет десять, а полей полсотни, надо переделывать.

Обратите внимание на комментарии над полями: они объясняют, какое поле для какого вида команды осмысленно. В структуре без иерархии это единственный способ не запутаться, и пренебрегать им нельзя.

37.5.2 Проверка конца

Последний штрих в точке входа:

```
Statement parse_sql(const std::string& sql) {
    auto tokens = tokenize(sql);
    Parser p(tokens);
    Statement st = p.parse();
    if (p.peek().type != Token::kEnd) {
        throw SqlError("лишние токены после statement: '" + p.peek().text + "'");
    }
    return st;
}
```

Проверка на kEnd ловит запросы вроде `SELECT * FROM items garbage` — разбор успешно завершился, но токены не кончились. Без неё лишний текст молча игнорировался бы, а опечатки превращались бы в неправильные запросы.

37.6 Исполнитель

Разобранная команда попадает в `Database::execute`:

```
void Database::execute(const std::string& sql, std::ostream& out) {
    Statement st = parse_sql(sql);
    switch (st.kind) {
        case Statement::kCreate:      execute_create(st, out);      break;
        case Statement::kCreateIndex: execute_create_index(st, out); break;
        case Statement::kInsert:      execute_insert(st, out);      break;
        case Statement::kSelect:      execute_select(st, out);      break;
    }
}
```

Обратите внимание на `switch` без `default`. При таком написании компилятор с `-Wall` предупредит, если в перечисление добавят новый вид, а ветку забудут. С `default`: предупреждения не будет — и новый вид тихо провалится в него. Это общее правило для `switch` по перечислению: не пишите `default`, если хотите, чтобы компилятор помогал.

Вывод идёт в `std::ostream&`, а не в `std::cout` напрямую. Разница проявляется в тестах: туда можно передать `std::ostringstream` и проверить результат строкой, что мы и делаем:

```
std::ostringstream sel;
d.execute("SELECT * FROM items WHERE value = 100", sel);
check(sel.str().find("(2 строки)") != std::string::npos, ...);
```

Привязка к `std::cout` внутри библиотечного кода — частая ошибка, из-за которой потом невозможно ни протестировать, ни перенаправить вывод.

37.6.1 Создание таблицы

```
void Database::execute_create(const Statement& st, std::ostream& out) {
    if (st.columns.size() != 2) {
        throw SqlError("мини-СУБД поддерживает таблицы только из 2 колонок");
    }
    table_name_ = st.table;
    col_names_ = {st.columns[0].first, st.columns[1].first};
    out << "OK (table '" << table_name_ << "')\n";
}
```

Вся команда сводится к запоминанию имён. Ни новых файлов, ни схемы на диске: таблица у нас всегда одна, а её структура зашита в код — первая колонка становится ключом дерева, вторая значением.

Отсюда следует неприятная особенность, о которой лучше сказать прямо: имена колонок не сохраняются между запусками. Создали таблицу с колонками `sku` и `price`, закрыли базу, открыли — и колонки снова зовутся `id` и `value`, потому что так написано в конструкторе. Данные при этом на месте.

Чтобы это исправить, нужна системная таблица — место, где база хранит собственную схему. В настоящих СУБД это `pg_catalog` в PostgreSQL или `sqlite_master` в SQLite, и устроены они как обычные таблицы, только заполняет их сам движок. Нам хватило бы одной страницы с именами; это упражнение в конце главы.

37.6.2 Вставка

```
void Database::execute_insert(const Statement& st, std::ostream& out) {
    if (st.table != table_name_) throw SqlError("неизвестная таблица: " + st.table);
    if (st.values.size() != 2) throw SqlError("ожидается 2 значения");

    std::int64_t id = st.values[0];
    std::int64_t value = st.values[1];

    wal->log_insert(id, value);
    tree->insert(id, value);

    for (auto& kv : indexes_) {
        const std::string& col = kv.first;
        std::int64_t key = (col == col_names_[0]) ? id : value;
        kv.second.wal->log_insert(key, id);
        kv.second.tree->insert_dup(key, id);
    }

    out << "1 строка\n";
}
```

Порядок здесь ровно тот, что мы обсуждали в главе 35: сначала журнал, потом дерево. Поменяйте местами — и авария между двумя строками потеряет запись.

Каждый вторичный индекс — отдельное дерево со своим журналом, и вставка обновляет их все. Это и есть цена индексов, о которой всегда говорят: чтение ускоряется, запись замедляется. Три индекса — четыре дерева и четыре записи в журналы на каждую строку. Подробнее об этом в следующей главе.

Заметьте `insert_dup` вместо `insert` для индексов — та самая правка из главы 34, без которой строки с одинаковым значением затирали бы друг друга.

37.6.3 Выборка

```
bool Database::matches_where(const Statement& st,
                             std::int64_t id, std::int64_t value) const {
    if (!st.has_where) return true;
    std::int64_t left = (st.where_col == col_names_[0]) ? id : value;
    const auto r = st.where_val;
```



```

const auto& op = st.where_op;
if (op == "=") return left == r;
if (op == "!=") return left != r;
if (op == "<") return left < r;
if (op == "<=") return left <= r;
if (op == ">") return left > r;
if (op == ">=") return left >= r;
throw SqlError("неизвестный оператор: " + op);
}

```

Проверка условия для одной строки. Сравнение операторов строками — не самое быстрое решение: на каждой строке таблицы выполняется до шести сравнений коротких строк. Правильнее было бы превратить оператор в перечисление ещё в парсере, и на большой таблице это дало бы заметную разницу. Для учебной базы оставлено как есть, но приём общий: **разбор должен превращать текст в числа один раз, а не заставлять исполнителя разбирать его снова на каждой строке.**

Самый простой способ выполнения — обойти всё дерево и отфильтровать:

```

void Database::run_full_scan(const Statement& st, const std::vector<std::string>& cols,
                           std::ostream& out) {

    int rows = 0;
    tree_>scan([&](std::int64_t id, std::int64_t value) {
        if (!matches_where(st, id, value)) return;
        for (std::size_t i = 0; i < cols.size(); ++i) {
            if (i) out << " | ";
            std::int64_t cell = (cols[i] == col_names_[0]) ? id : value;
            out << std::setw(8) << std::left << cell;
        }
        out << "\n";
        ++rows;
    });
    print_row_count(rows, out);
}

```

Это полный перебор — тот самый, от которого нас должен спасать индекс. В следующей главе появятся ещё три способа выполнения и планировщик, который между ними выбирает.

Согласование числительных вынесено в отдельную функцию — мелочь, но приятная:

```
// Согласование числительного: 1 строка, 2–4 строки, 5+ строк.
void print_row_count(int rows, std::ostream& out) {
    int last_two = rows % 100;
    int last = rows % 10;
    const char* word = "строк";
    if (last == 1 && last_two != 11) word = "строка";
    else if (last >= 2 && last <= 4 && (last_two < 12 || last_two > 14)) word = "строки";
    out << "(" << rows << " " << word << ")\n";
}
```

Правило русского языка требует смотреть на две последние цифры: 21 строка, но 11 строк; 22 строки, но 12 строк. Такую функцию стоит написать один раз и носить с собой из проекта в проект.

37.7 Пробуем

```
$ ./build/mydb /tmp/sqldemo
mydb> CREATE TABLE items (id INT PRIMARY KEY, value INT);
OK (table 'items')
mydb> INSERT INTO items VALUES (1, 100);
1 строка
mydb> INSERT INTO items VALUES (2, 250);
1 строка
mydb> INSERT INTO items VALUES (3, 300);
1 строка
mydb> SELECT * FROM items;
id      | value
-----+-----
1       | 100
2       | 250
3       | 300
(3 строки)
mydb> SELECT id FROM items WHERE value > 200;
id
-----
2
3
(2 строки)
```

```
mydb> SELECT * FROM items WHERE id = 2;
id      | value
-----+-----
2       | 250
(1 строка)
mydb> select * from items where value <= 100;
id      | value
-----+-----
1       | 100
(1 строка)
mydb> SELECT * FROM items WHERE value > 999;
id      | value
-----+-----
(0 строк)
```

Работают и выбор колонок, и все операторы сравнения, и нижний регистр ключевых слов. Пустой результат честно печатает шапку и ноль строк — так делают настоящие базы, потому что клиенту важно знать имена колонок даже при пустой выборке.

Ошибки выглядят понятно:

```
mydb> DROP TABLE items;
Error: SQL: ожидался keyword
mydb> INSERT INTO items VALUES (1, 2, 3);
Error: SQL: ожидается 2 значения
mydb> SELECT * FROM other;
Error: SQL: неизвестная таблица: other
```

Первое сообщение стоило бы сделать точнее — DROP не является ключевым словом нашего диалекта, поэтому лексер счёл его именем, а парсер пожаловался на отсутствие ключевого слова. Пользователю было бы понятнее прочитать «неизвестная команда: DROP». Это упражнение в конце главы, и оно полезнее, чем кажется: качество сообщений об ошибках — половина впечатления от инструмента.

37.8 Тесты разбора

Парсер — чистая функция от строки к структуре, и тестировать его так же удобно, как токенизатор оболочки:

```

db::Statement s = db::parse_sql("CREATE TABLE items (id INT PRIMARY KEY, value INT)");
check(s.kind == db::Statement::kCreate, "CREATE TABLE распознан");
check_eq(s.table, std::string("items"), "имя таблицы");
check_eq(s.columns.size(), static_cast<std::size_t>(2), "две колонки");

s = db::parse_sql("select * from items where value >= -5");
check(s.kind == db::Statement::kSelect, "SELECT в нижнем регистре распознан");
check(s.has_where, "WHERE распознан");
check_eq(s.where_op, std::string(">="), "оператор >=");
check_eq(s.where_val, static_cast<std::int64_t>(-5), "отрицательное число");

```

Отдельно проверяется, что плохой ввод отвергается, а не разбирается как попало:

```

const char* bad[] = {
    "SELECT",
    "INSERT INTO items VALUES (1",
    "SELECT * FROM items WHERE",
    "DROP TABLE items",
};

for (...) {
    bool threw = false;
    try { db::parse_sql(bad[i]); } catch (const db::SqlError&) { threw = true; }
    check(threw, std::string("плохой SQL отвергается: ") + bad[i]);
}

```

Тесты на отрицательные случаи в парсерах важнее положительных. Разбор правильного запроса вы проверите руками при первом же запуске, а вот что делает программа с обрубком `SELECT * FROM items WHERE` — узнаете, только если специально спросите. Ответ должен быть «внятная ошибка», а не «падение» и не «молча принял».

37.9 Главные правила главы

1. Язык запросов отделяет «что нужно» от «как получить» — благодаря этому движок можно менять, не трогая запросы.
2. Схема разбора одна и та же для любого языка: лексер, парсер, исполнитель; каждая ступень проверяется отдельно.
3. Метка конца в списке токенов избавляет парсер от проверок на выход за границу.
4. Рекурсивный спуск пишется так, что код повторяет грамматику; это его главное достоинство.

5. Ключевые слова в SQL регистронезависимы, имена — нет; сравнивайте по приведённой копии, храните исходное написание.
6. Перед `isalpha` и родней обязательно приводите символ к `unsigned char`, иначе кириллица даст неопределённое поведение.
7. После разбора проверьте, что токены кончились, — иначе лишний текст молча пропадёт.
8. Разбор должен превращать текст в числа и перечисления один раз; исполнитель не должен разбирать строки на каждой записи.

37.10 Упражнения

1. Соберите базу и повторите сессию из главы. Затем проверьте разные написания: `SeLeCt`, лишние пробелы, отсутствие точки с запятой, точку с запятой в середине.
2. Сделайте сообщение об ошибке для неизвестной команды понятнее: `DROP TABLE items` должно давать «неизвестная команда: DROP», а не «ожидался keyword». Подсказка — проверьте тип первого токена и его текст.
3. Добавьте в сообщения об ошибках позицию в строке: лексер знает смещение, достаточно донести его до токена. Проверьте на `SELECT * FROM items WHERE value >> 5`.
4. Замените строковый оператор в `Statement` на перечисление, разбираемое в парсере. Убедитесь, что тесты проходят, и объясните, где именно это ускорит выполнение.
5. Добавьте `ORDER BY <колонка> [DESC]`. Для колонки-ключа сортировка бесплатна (дерево уже отдаёт по возрастанию), для второй колонки придётся собрать строки и отсортировать.
6. Добавьте `LIMIT <число>`. Важно, чтобы обход прекращался по достижении предела, а не отбрасывал лишнее после сбора всего.
7. (Сложнее) Сделайте системную таблицу: заведите страницу, где хранятся имя таблицы и имена колонок, пишите её при `CREATE TABLE` и читайте при открытии базы. Проверьте, что после перезапуска колонки называются так, как их создали.
8. (Сложнее) Поддержите `AND` и `OR` в условии. Понадобится дерево выражений вместо трёх полей и разбор с приоритетами: `OR` слабее `AND`, скобки сильнее всего. Начните с грамматики на бумаге.

37.11 Что дальше

Глава 37 — вторичные индексы и планировщик. Сейчас любой запрос с условием выполняется полным перебором: даже `WHERE id = 42` честно обходит всё дерево, хотя ключ у нас именно `id`.

Научим базу выбирать способ выполнения — по первичному ключу, по вторичному индексу, диапазонным сканированием индекса или полным перебором, — и покажем выбранный план по команде EXPLAIN. А заодно измерим, во сколько раз индекс ускоряет чтение и на сколько замедляет запись.

Глава 38

Глава 37. Вторичные индексы и план запроса

Наша база умеет быстро находить строку по первичному ключу — за этим и строилось дерево. А что с остальными колонками?

```
SELECT * FROM items WHERE value = 250;
```

Дерево упорядочено по `id`, а не по `value`, поэтому единственный способ ответить — обойти все строки и проверить каждую. На тысяче записей это уже заметно, на миллионе — катастрофа.

В этой главе научим базу строить вторичные индексы — дополнительные деревья, упорядоченные по другой колонке, — и научим её выбирать способ выполнения запроса. А выбранный способ будем показывать по команде `EXPLAIN`, как это делают взрослые базы.

Заодно измерим обе стороны сделки: во сколько раз индекс ускоряет чтение и во что обходится при записи.

38.1 Насколько всё плохо без индекса

Замер на тысяче строк, программа `utils/bench.cpp`:

Find by id (B+tree): 226 ms на 1000 запросов, 4411 запросов/с

Find by value (полный перебор, 100 запросов): 3386 ms, 29.5 запросов/с

Разница в полтора раза, и она будет расти линейно с числом строк: поиск по ключу — логарифм, полный перебор — линия. На миллионе строк перебор станет в тысячу раз медленнее, чем сейчас, а поиск по ключу — всего вдвое.

Обратите внимание и на абсолютные числа: 4411 поисков в секунду по ключу — совсем не быстро для базы, где всё помещается в памяти. Виноваты наши упрощения: копирование страницы при каждом чтении и разбор узла в три вектора. Об этом подробнее в главе 39.

38.2 Что такое вторичный индекс

Идея простая: завести второе дерево, где ключом будет значение колонки, а значением — идентификатор строки.

Основное дерево (по id):	Вторичный индекс (по value):
1 → 100	100 → 1
2 → 250	250 → 2
3 → 250	250 → 3

Теперь запрос `WHERE value = 250` выполняется в два шага: находим в индексе все идентификаторы с ключом 250, потом по каждому идентификатору достаём строку из основного дерева. Два поиска по логарифму вместо линейного перебора.

Почему во вторичном индексе хранятся идентификаторы, а не строки целиком? Из-за дублирования. Копия строки в каждом индексе означала бы, что при изменении строки надо обновить все копии, а при трёх индексах данные хранятся вчетверо. Ссылка на строку решает обе проблемы ценой второго обращения при чтении.

Впрочем, у копирования данных в индекс есть и своя ниша: если индекс содержит все колонки, которые нужны запросу, база может ответить, не заглядывая в таблицу вообще. Такие индексы называют покрывающими, и они бывают очень эффективны — но у нас всего две колонки, и говорить о покрытии особо не о чем.

38.3 Реализация

Каждый вторичный индекс — это отдельный файл со своим деревом, своим страничным менеджером и своим журналом:

```
struct SecondaryIndex {
    std::unique_ptr<PageManager> pm;
    std::unique_ptr<BPlusTree> tree;
    std::unique_ptr<WAL> wal;
    std::string column;
};

std::unordered_map<std::string, SecondaryIndex> indexes_;
```


Ключ в отображении — имя колонки, потому что именно по нему планировщик ищет подходящий индекс.

Полный набор из трёх компонентов на каждый индекс выглядит расточительно, но так устроено и в настоящих базах: индекс — это самостоятельное хранилище со своим жизненным циклом. Общий журнал был бы возможен и даже лучше (одна синхронизация вместо нескольких), но потребовал бы записывать в него, к какому дереву относится операция.

`std::unique_ptr` здесь потому, что `PageManager` не копируем — он владеет дескриптором, — а класть не копируемый объект в контейнер по значению не выйдет. Умный указатель решает и это, и вопрос времени жизни: закрылась база — закрылись все индексы.

38.3.1 Создание

```
void Database::execute_create_index(const Statement& st, std::ostream& out) {
    if (st.table != table_name_) throw SqlError("неизвестная таблица: " + st.table);
    if (st.columns.size() != 1) throw SqlError("ожидается одна колонка");
    const std::string& col = st.columns[0].first;
    if (col != col_names_[0] && col != col_names_[1]) {
        throw SqlError("неизвестная колонка: " + col);
    }
    if (indexes_.count(col)) {
        out << "Index уже существует\n";
        return;
    }

    open_index(st.index_name, col);
    SecondaryIndex& idx = indexes_[col];

    int populated = 0;
    tree_>scan([&](std::int64_t row_id, std::int64_t row_value) {
        std::int64_t key = (col == col_names_[0]) ? row_id : row_value;
        idx.wal->log_insert(key, row_id);
        idx.tree->insert_dup(key, row_id);
        ++populated;
    });
    idx.pm->sync();
    idx.wal->truncate();

    save_catalog();
}
```

```

out << "Index '" << st.index_name << "' on " << col
    << " (" << populated << " rows indexed)\n";
}

```

Главное здесь — заполнение уже существующими данными. Индекс, построенный на непустой таблице, обязан сразу знать про все строки, иначе запросы начнут возвращать неполные ответы — та самая тихая поломка, о которой шла речь в главе 34.

Заполнение идёт полным обходом основного дерева, и на большой таблице это долгая операция. В нашем замере построение индекса на тысяче строк заняло 470 миллисекунд — примерно столько же, сколько вставка этой тысячи. В настоящих базах создание индекса на большой таблице занимает часы и блокирует запись, поэтому существуют отдельные режимы вроде `CREATE INDEX CONCURRENTLY` в PostgreSQL, которые строят индекс, не мешая работе.

После заполнения — `sync` и очистка журнала: индекс на диске полон, повторять нечего.

Вставки идут через `insert_dup`, а не `insert`. Это принципиально: вторичный ключ не уникален, и обычная вставка затирала бы строки с одинаковым значением. Историю этой ошибки мы разбирали в главе 34.

38.3.2 Обновление при вставке

Индекс бесполезен, если он отстаёт от данных. Поэтому каждая вставка обновляет все индексы:

```

wal->log_insert(id, value);
tree->insert(id, value);

for (auto& kv : indexes_) {
    const std::string& col = kv.first;
    std::int64_t key = (col == col_names_[0]) ? id : value;
    kv.second.wal->log_insert(key, id);
    kv.second.tree->insert_dup(key, id);
}

```

Вот она, цена индексов. Одна вставка при трёх индексах — это четыре записи в деревья и четыре синхронизации журналов. Отсюда правило, знакомое всем администраторам баз: индексы ускоряют чтение и замедляют запись, поэтому на таблицах, куда пишут потоком, их держат по минимуму.

Ещё одно следствие — при массовой загрузке данных индексы принято удалять перед загрузкой и создавать после. Построить индекс один раз по готовой таблице дешевле, чем поддерживать его на каждой из миллиона вставок.

38.3.3 Каталог: чтобы индекс пережил перезапуск

Первая версия базы теряла индексы при закрытии. Файлы оставались на диске, но при следующем запуске никто про них не знал, и запрос снова уходил в полный перебор:

```
mydb> EXPLAIN SELECT * FROM items WHERE value = 250;  
Plan: FullScan with filter (value = 250)
```

Ответ при этом оставался правильным — только медленным. Тихая потеря производительности, которую легко не заметить.

Лечится это каталогом — местом, где база хранит сведения о себе самой. У нас он предельно простой, текстовый файл `catalog.txt`:

```
goods  
sku price  
idx_price price
```

Первая строка — имя таблицы, вторая — имена колонок, дальше по строке на индекс: имя и колонка.

```
void Database::save_catalog() {  
    std::ofstream out(dir_ + "/catalog.txt");  
    if (!out.is_open()) return;  
    out << table_name_ << "\n";  
    out << col_names_[0] << " " << col_names_[1] << "\n";  
    for (const auto& kv : index_names_) {  
        out << kv.second << " " << kv.first << "\n";  
    }  
}  
  
void Database::load_catalog() {  
    std::ifstream in(dir_ + "/catalog.txt");  
    if (!in.is_open()) return;  
  
    std::string table;  
    if (!std::getline(in, table) || table.empty()) return;  
    table_name_ = table;
```

```

...
std::string line;
while (std::getline(in, line)) {
    std::istringstream ls(line);
    std::string index_name, column;
    if (!(ls >> index_name >> column)) continue;
    open_index(index_name, column);
}
}

```

Открытие индекса вынесено в отдельный метод, потому что нужно в двух местах — при создании и при загрузке каталога:

```

void Database::open_index(const std::string& index_name, const std::string& column) {
    if (indexes_.count(column)) return;

    SecondaryIndex idx;
    idx.column = column;
    idx.pm = mk<PageManager>(dir_ + "/" + idx_ + index_name + ".db", 32);
    idx.tree = mk<BPlusTree>(*idx.pm);
    idx.wal = mk<WAL>(dir_ + "/" + idx_ + index_name + ".wal");
    idx.wal->replay([&idx](std::int64_t k, std::int64_t v) {
        idx.tree->insert_dup(k, v);
    });

    indexes_.emplace(column, std::move(idx));
    index_names_[column] = index_name;
}

```

Обратите внимание, что при открытии индекса проигрывается его журнал — точно так же, как для основного дерева. Индекс — полноценное хранилище и восстанавливается после аварии по тем же правилам.

Теперь всё переживает перезапуск:

```

mydb> EXPLAIN SELECT * FROM goods WHERE price = 250;
Plan: SecondaryLookup(price=250)
mydb> SELECT * FROM goods WHERE price = 250;
sku      | price
-----+-----
2        | 250

```

3 | 250

(2 строки)

И имена колонок теперь тоже сохраняются — `sku` и `price`, как их создали, а не `id` и `value` из конструктора. Тот же каталог решил обе задачи.

Текстовый формат каталога — сознательное упрощение. Настоящие базы хранят каталог в обычных таблицах внутри той же базы (`pg_class`, `pg_index` в PostgreSQL; `sqlite_master` в SQLite), и это красиво: движок описывает сам себя своими же средствами, а запрос к каталогу выполняется обычным механизмом запросов. Нам для двух колонок и одного индекса хватит трёх строк текста.

38.4 Планировщик

Теперь самое интересное: как база выбирает способ выполнения.

```
std::string Database::plan_select(const Statement& st) const {
    if (!st.has_where) return "FullScan(primary)";
    if (st.where_col == col_names_[0] && st.where_op == "=") {
        return "PrimaryLookup(id=" + std::to_string(st.where_val) + ")";
    }
    auto it = indexes_.find(st.where_col);
    if (it != indexes_.end()) {
        if (st.where_op == "=") {
            return "SecondaryLookup(" + it->first + "=" + std::to_string(st.where_val) + ")";
        }
        return "SecondaryScan(" + it->first + " " + st.where_op + " "
            + std::to_string(st.where_val) + ")";
    }
    return "FullScan with filter (" + st.where_col + " " + st.where_op + " "
        + std::to_string(st.where_val) + ")";
}
```

Четыре стратегии, и выбор между ними — цепочка проверок сверху вниз.

Нет условия — обходим всё дерево, других вариантов и нет. Условие на первичный ключ с равенством — один спуск по дереву, самый быстрый случай. Есть индекс по нужной колонке: при равенстве — точечный поиск в индексе, при неравенстве — обход индекса. Ничего не подошло — полный перебор с проверкой условия на каждой строке.

Такой планировщик называют основанным на правилах: он выбирает по жёсткому списку предпочтений, не заглядывая в данные. Так работали первые версии Oracle, и так до сих пор работают простые встроенные базы.

Взрослые системы используют стоимостное планирование: для каждого способа оценивается стоимость в условных единицах, и выбирается самый дешёвый. Оценка строится на статистике — сколько всего строк, сколько различных значений в колонке, как они распределены. Именно поэтому в PostgreSQL есть команда ANALYZE, собирающая статистику, и именно поэтому после массовой загрузки данных запросы иногда «тупеют», пока статистику не пересоберут.

Разница видна на простом примере. Пусть в таблице миллион строк и индекс по колонке «город». Запрос `WHERE city = 'Алматы'`, если алматинцев в таблице десятков, отлично выполнится через индекс. Но если алматинцев девятьсот тысяч, поход через индекс окажется **медленнее** полного перебора: девятьсот тысяч случайных обращений к основному дереву против одного последовательного прохода. Планировщик по правилам этого не увидит, стоимостный — увидит, если знает распределение.

38.4.1 EXPLAIN

Показать выбранный план — три строки:

```
void Database::execute_select(const Statement& st, std::ostream& out) {
    if (st.table != table_name_) throw SqlError("неизвестная таблица: " + st.table);

    std::string plan = plan_select(st);

    if (st.is_explain) {
        out << "Plan: " << plan << "\n";
        return;
    }
    ...
}
```

И вот как это выглядит в работе:

```
mydb> EXPLAIN SELECT * FROM items;
Plan: FullScan(primary)
mydb> EXPLAIN SELECT * FROM items WHERE id = 2;
Plan: PrimaryLookup(id=2)
mydb> EXPLAIN SELECT * FROM items WHERE value = 250;
Plan: FullScan with filter (value = 250)
```

```
mydb> CREATE INDEX idx_val ON items (value);
Index 'idx_val' on value (3 rows indexed)
mydb> EXPLAIN SELECT * FROM items WHERE value = 250;
Plan: SecondaryLookup(value=250)
mydb> EXPLAIN SELECT * FROM items WHERE value > 100;
Plan: SecondaryScan(value > 100)
```

Один и тот же запрос до и после создания индекса выполняется по-разному — и EXPLAIN это показывает. Именно так им и пользуются на практике: запрос стал медленным, смотрим план, видим полный перебор там, где ожидали индекс, — и разбираемся почему.

Наш EXPLAIN печатает одну строку, потому что план у нас плоский. В настоящих базах план — дерево из соединений, сортировок и агрегатов, и EXPLAIN выдаёт его с отступами, оценкой строк и стоимости, а EXPLAIN ANALYZE — ещё и реальное время каждого узла.

38.5 Выполнение

Каждой стратегии соответствует своя функция. Поиск по первичному ключу — самый прямой:

```
void Database::run_primary_eq(const Statement& st, const std::vector<std::string>& cols,
                             std::ostream& out) {
    std::int64_t v = 0;
    if (tree_>find(st.where_val, v)) {
        ...
        print_row_count(1, out);
    } else {
        print_row_count(0, out);
    }
}
```

Работа с индексом сложнее — нужен второй шаг, поход в основное дерево за строкой:

```
void Database::run_secondary(const Statement& st, const std::vector<std::string>& cols,
                             const SecondaryIndex& idx, std::ostream& out) {
    int rows = 0;
    auto emit = [&](std::int64_t row_id) {
        std::int64_t row_value = 0;
        if (!tree_>find(row_id, row_value)) return;
        ...
        ++rows;
    };
}
```

```

if (st.where_op == "=") {
    // Вторичный ключ не уникален: на одно значение может приходиться
    // много строк, поэтому берём ВСЕ совпадения, а не первое.
    idx.tree->find_all(st.where_val, emit);
} else {
    idx.tree->scan([&](std::int64_t key, std::int64_t row_id) {
        const auto r = st.where_val;
        const auto& op = st.where_op;
        bool ok =
            (op == "<" && key < r) ||
            (op == "<=" && key <= r) ||
            (op == ">" && key > r) ||
            (op == ">=" && key >= r) ||
            (op == "!=" && key != r);
        if (ok) emit(row_id);
    });
}
print_row_count(rows, out);
}

```

Лямбда `emit` — общая часть обеих ветвей: получить идентификатор, достать строку, напечатать. Проверка `if (!tree->find(row_id, row_value)) return;` защищает от рассогласования индекса и таблицы: если в индексе есть ссылка на несуществующую строку, мы её молча пропустим, а не напечатаем мусор.

Ветка с неравенством выглядит подозрительно, и подозрение справедливо: она обходит **весь** индекс целиком, проверяя каждую запись. То есть `WHERE value > 100` через индекс делает столько же работы, сколько полный перебор, плюс ещё поход в основное дерево на каждое совпадение — то есть работает **медленнее**.

Правильно было бы спуститься по индексу к границе диапазона и идти по цепочке листьев, пока условие выполняется. У дерева для этого уже почти всё есть — не хватает метода `scan_range`, который вынесен в упражнения главы 34. Честно говоря, наш `SecondaryScan` — это заготовка под правильную реализацию, а не сама реализация, и в плане запроса стоило бы это отражать.

38.6 Сколько выигрывают и сколько теряют

Полный замер, тысяча строк:

=== Бенчмарк мини-СУБД, N = 1000 ===

Insert: 453 ms, 2205 ops/s

Checkpoint: 2.2 ms

Find by id (B+tree): 227 ms, 4411 ops/s

Find by value (полный перебор, 100 запросов): 3386 ms, 29.5 ops/s

Build index: 470 ms

Find by value (индекс): 464 ms, 2157 ops/s

Ускорение find-by-value с индексом: 73x

Семьдесят три раза на тысяче строк. И это ещё скромно: выигрыш растёт линейно с размером таблицы, потому что перебор линеен, а поиск по индексу — логарифмичен. На ста тысячах строк разница была бы уже в тысячи раз.

Обратная сторона: поиск через индекс (2157 запросов в секунду) вдвое медленнее поиска по первичному ключу (4411), потому что делает два спуска вместо одного. Это нормально и ожидаемо — за универсальность платят.

Построение индекса на тысяче строк заняло 470 миллисекунд, столько же, сколько сама вставка этой тысячи. Логично: построение — это полный обход плюс тысяча вставок в новое дерево плюс тысяча записей в журнал.

38.7 Чего мы не сделали

Составных индексов — по нескольким колонкам сразу. В настоящей базе индекс по паре «город, дата» отвечает и на запросы по городу, и на запросы по городу с датой, но не на запросы по одной дате. Правило левого префикса — первое, что узнаёшь, работая с индексами всерьёз.

Уникальных индексов, которые не просто ускоряют поиск, а запрещают дубликаты. У нас первичный ключ уникален по построению (обычная вставка заменяет), но объявить уникальной вторую колонку нельзя.

Удаления индексов: команды `DROP INDEX` нет, а файлы индекса живут вечно.

Оптимизации диапазона по первичному ключу. Запрос `WHERE id > 100` у нас получает план `FullScan with filter`, хотя дерево упорядочено именно по `id` и могло бы начать обход с нужного места:

```
mydb> EXPLAIN SELECT * FROM items WHERE id > 1;
```

Plan: FullScan with filter (id > 1)

Формально ответ правильный, но работы делается больше необходимого. Это самая заметная недоделка планировщика, и чинится она тем же `scan_range`.

И, конечно, нет статистики и стоимостного выбора — планировщик действует по жёстким правилам и не подозревает, сколько строк подойдёт под условие.

38.8 Главные правила главы

1. Вторичный индекс — отдельное дерево «значение колонки → идентификатор строки»; чтение по нему требует двух спусков.
2. Индекс, построенный на непустой таблице, обязан быть заполнен существующими данными сразу.
3. Каждая вставка обновляет все индексы: чтение ускоряется, запись замедляется пропорционально их числу.
4. При массовой загрузке индексы выгоднее удалить и построить заново.
5. Сведения о таблице и индексах живут в каталоге, иначе после перезапуска база забудет про индексы и тихо станет медленной.
6. Планировщик по правилам выбирает из списка предпочтений; стоимостный смотрит на статистику и иногда предпочитает полный перебор индексу.
7. EXPLAIN — первый инструмент при разборе медленного запроса: он показывает, что база решила сделать, а не что вы задумали.
8. Диапазонный запрос через индекс без остановки на границе не имеет смысла: он делает больше работы, чем полный перебор.

38.9 Упражнения

1. Соберите базу, создайте таблицу с индексом, вставьте несколько строк с повторяющимися значениями. Посмотрите EXPLAIN для запросов по обеим колонкам, с равенством и неравенством. Перезапустите базу и убедитесь, что план не изменился.
2. Удалите `catalog.txt` при закрытой базе и запустите её снова. Что покажет EXPLAIN? Объясните, почему данные при этом не потерялись.
3. Запустите `./build/bench 100`, `./build/bench 1000` и `./build/bench 5000`. Постройте зависимость ускорения от числа строк и объясните её форму.
4. Замерьте вставку тысячи строк без индекса и с двумя индексами. Во сколько раз медленнее? Сходится ли это с числом деревьев, которые надо обновить?

5. Реализуйте `DROP INDEX <имя>`: уберите индекс из отображения, удалите файлы, обновите каталог. Проверьте, что план после этого возвращается к полному перебору.
6. Добавьте план `PrimaryScan` для диапазона по первичному ключу: спуск к границе и обход по цепочке листьев с остановкой. Сравните время с `FullScan with filter` на десяти тысячах строк.
7. (Сложнее) Соберите простейшую статистику: при построении индекса посчитайте число различных значений. Научите планировщик выбирать полный перебор, если под условие подходит больше трети таблицы, и проверьте на данных, где одно значение встречается у большинства строк.
8. (Сложнее) Сделайте уникальный индекс: `CREATE UNIQUE INDEX` должен отвергать вставку строки с уже существующим значением. Продумайте, что делать, если в момент создания индекса дубликаты уже есть.

38.10 Что дальше

Глава 38 — оболочка базы. У нас есть язык, но общаться с базой пока приходится через `printf` в конвейере. Сделаем нормальный REPL в стиле `psql`: многострочный ввод с продолжением, служебные команды с точкой, история между запусками, обработка ошибок без падения. Заодно разберёмся, почему разбор ввода в интерактивной программе — отдельная задача, не сводящаяся к разбору языка.

Глава 39

Глава 38. Оболочка базы данных

Движок готов, язык есть. Не хватает того, через что с базой общается человек, — оболочки.

Все клиенты баз данных устроены одинаково: `psql`, `mysql`, `sqlite3` — это циклы «прочитать, выполнить, напечатать». Внешне просто, но в деталях набирается работа: запрос может занимать несколько строк, служебные команды должны отличаться от запросов, история должна переживать выход, а ошибка в запросе не должна ронять программу.

В этой главе доведём `mydb` до состояния, в котором им приятно пользоваться. И заодно разберём, почему разбор пользовательского ввода — задача, не сводящаяся к разбору языка.

39.1 Цикл

Скелет знаком по оболочке из главы 26:

```
while (true) {
    std::cout << (first_line ? "mydb> " : " ... ");
    std::cout.flush();

    std::string line;
    if (!std::getline(std::cin, line)) {
        std::cout << "\n";
        break;
    }
    ...
}
```

Два вида приглашения — `mydb>` для начала команды и `...` для продолжения — это соглашение всех клиентов баз. Увидев `...`, пользователь понимает, что программа ждёт продолжения, а не зависла.

`flush` после приглашения обязателен по той же причине, что и в оболочке: приглашение не заканчивается переводом строки и без сброса буфера осталось бы невидимым.

Проверка `!std::getline(...)` ловит конец ввода — `Ctrl+D` в терминале или конец файла, если команды поданы конвейером. Именно она позволяет пользоваться базой из скриптов:

```
$ printf 'INSERT INTO items VALUES (1, 10);\nSELECT * FROM items;\n' | ./build/mydb
```

39.2 Многострочный ввод

Первое настоящее отличие от оболочки: команда SQL не заканчивается на переводе строки. Запрос может занимать десяток строк, и признаком конца служит точка с запятой.

Отсюда накопитель: строки складываются в буфер, пока в нём не появится `;`.

```
if (!buffer.empty()) buffer += " ";
buffer += t;
first_line = false;

// В строке может быть несколько команд: `INSERT ...; SELECT ...;`.
// Выполняем ВСЕ завершённые, а незавершённый хвост оставляем в буфере
// до следующей строки.
while (true) {
    std::size_t semi = buffer.find(';');
    if (semi == std::string::npos) break;

    std::string stmt = buffer.substr(0, semi + 1);
    buffer = trim(buffer.substr(semi + 1));

    try {
        db.execute(stmt, std::cout);
    } catch (const std::exception& e) {
        std::cout << "Error: " << e.what() << "\n";
    }
    history_add(trim(stmt));
}
```

```
first_line = buffer.empty();
```

Строки склеиваются через пробел, а не через перевод строки: для нашего лексера это одно и то же, зато в историю команда попадает одной строкой.

Внутренний цикл — результат исправления ошибки, о которой стоит рассказать подробно, потому что она поучительна.

39.2.1 Ошибка, которую нашла проверка вручную

В первой версии вместо цикла стояла одна проверка: нашли точку с запятой — выполнили команду, остаток положили обратно в буфер, читаем следующую строку. Выглядит разумно и работает, пока в строке одна команда.

А вот что получалось, если написать две:

```
mydb> INSERT INTO items VALUES (1, 10); INSERT INTO items VALUES (2, 20);
```

1 строка

...

Первая команда выполнена, вторая осталась в буфере, приглашение сменилось на ... — и всё, программа ждёт продолжения того, что уже полное. Хуже того, служебные команды перестают работать: они распознаются только в начале ввода, а буфер не пуст. Набранный .exit уходит в накопитель, и выйти из программы можно только через Ctrl+D.

Именно так это и выглядело при проверке: три служебные команды подряд провалились в никуда, и сессия не завершилась.

Исправление — обрабатывать буфер, пока в нём есть завершённые команды. Три строки разницы, а поведение меняется принципиально.

Урок здесь не про цикл. Урок про то, что интерактивные программы нужно проверять сценариями, а не по одной команде. Ошибка не проявлялась, пока команды вводились по одной, — то есть ровно в том режиме, в котором обычно и проверяют, дописав функцию.

39.2.2 Чего не умеет наш поиск точки с запятой

Мы ищем ; обычным find, не глядя на контекст. Для нашего диалекта этого достаточно: строк в языке нет, а значит, точка с запятой не может оказаться внутри данных.

Как только появятся строковые литералы, всё сломается:

```
INSERT INTO t VALUES ('привет; пока');
```

Здесь ; внутри кавычек, а наш разбор оборвёт команду посередине. Правильное решение — искать разделитель тем же лексером, который разбирает язык: он про кавычки знает. Клиенты настоящих баз так и делают, причём `psql` умеет ещё и вложенные `$$`-блоки для функций.

Это тот случай, когда простое решение годится ровно до определённого момента, и важно понимать, до какого именно.

39.3 Служебные команды

Команды, которые адресованы не базе, а клиенту, принято отличать по точке в начале — так сделано в `sqlite3`, а в `psql` для этого служит обратная косая черта:

```
bool handle_meta(const std::string& cmd, db::Database& db) {
    std::string c = trim(cmd);
    if (c == ".quit" || c == ".exit") return false;
    if (c == ".help") { show_help(); return true; }
    if (c == ".history") { show_history(); return true; }
    if (c == ".checkpoint") {
        db.checkpoint();
        std::cout << "(checkpoint done)\n";
        return true;
    }
    std::cout << "Неизвестная meta-команда: " << c << " (.help для справки)\n";
    return true;
}
```

Возвращаемое значение означает «продолжать ли работу»: `false` только у команд выхода. Такой способ выхода из вложенной функции чище, чем `std::exit`, — он позволяет циклу выйти нормально и выполнить всё, что положено при завершении: контрольную точку и сохранение истории.

Обратите внимание, что распознаются служебные команды **только в начале ввода**:

```
if (first_line && !t.empty() && t[0] == '.') {
    if (!handle_meta(t, db)) break;
    continue;
}
```

Условие `first_line` защищает от неверного толкования: если пользователь набирает многострочный запрос и одна из строк начинается с точки, это часть запроса, а не команда клиенту.

Справка по `.help` — не украшение. Программа, которая не может рассказать о себе, вынуждает лезть в исходники:

```
mydb> .help
```

Meta-команды:

<code>.help</code>	справка
<code>.history</code>	показать историю
<code>.checkpoint</code>	принудительный sync на диск
<code>.quit/.exit</code>	выход

SQL:

```
CREATE TABLE <name> (<col> INT [PRIMARY KEY], <col> INT);
CREATE INDEX <name> ON <table>(<column>);
INSERT INTO <name> VALUES (<n>, <n>);
SELECT <cols>|* FROM <name> [WHERE <col> <op> <n>];
EXPLAIN SELECT ...;
```

Команды многострочные, завершаются ``;``.

39.4 Комментарии

SQL-комментарий начинается с двух дефисов и идёт до конца строки. Убираем его до всякого разбора:

```
// Удалить SQL-комментарии: «-- до конца строки».
std::string strip_comment(const std::string& s) {
    auto pos = s.find("--");
    if (pos == std::string::npos) return s;
    return s.substr(0, pos);
}
```

Четыре строки, и у них тот же изъян, что у поиска точки с запятой: два дефиса внутри строкового литерала обрежут команду. И то же оправдание — строк в нашем языке нет.

Блочные комментарии `/* ... */` мы не поддерживаем: они могут занимать несколько строк, и для них потребовалось бы состояние «мы внутри комментария», переживающее переход к следующей строке. Это упражнение в конце главы.

39.5 История

Устроена так же, как в оболочке из главы 30: список строк в памяти, файл в домашней папке, загрузка при старте и сохранение при выходе.

```
// Сколько команд помним. Без ограничения файл истории растёт вечно.
const std::size_t kHistoryLimit = 1000;

void history_add(const std::string& q) {
    if (q.empty()) return;
    if (!g_history.empty() && g_history.back() == q) return;
    g_history.push_back(q);
    if (g_history.size() > kHistoryLimit) {
        g_history.erase(g_history.begin(),
                        g_history.begin() +
                            static_cast<long>(g_history.size() - kHistoryLimit));
    }
}
```

Ограничение появилось не из теоретических соображений. Пока я гонял примеры к этим главам, файл истории дорос до двух с лишним тысяч строк, и `.history` начал выдавать простыню на весь экран. Тысяча последних команд — разумный предел, столько же по умолчанию держит `bash`.

Обрезание сделано одним вызовом `erase` с диапазоном, а не циклом удалений с начала. Разница принципиальна: `erase` с диапазоном сдвигает хвост один раз, а тысяча одиночных удалений сдвинула бы его тысячу раз. Это общее правило работы с `std::vector`: удаляйте диапазоном.

В историю попадает разобранная команда, а не введённая строка, — то есть если пользователь написал две команды в одной строке, в истории их будет две:

```
mydb> INSERT INTO items VALUES (1, 10); INSERT INTO items VALUES (2, 20);
1 строка
1 строка
mydb> .history
  1  INSERT INTO items VALUES (1, 10);
  2  INSERT INTO items VALUES (2, 20);
  3  SELECT * FROM items;
```

Так удобнее для повторного использования: каждая запись истории — самостоятельная команда.

39.6 Ошибки не должны ронять программу

Пожалуй, главное требование к интерактивной программе. Опечатка в запросе — обычное дело, и клиент, падающий от неё, никуда не годится.

```
try {
    db.execute(stmt, std::cout);
} catch (const std::exception& e) {
    std::cout << "Error: " << e.what() << "\n";
}
```

Ловим `std::exception` — общего предка и `SqlError`, и `PageError`, и `WALError`. Это тот случай, когда широкий захват уместен: на верхнем уровне цикла нас интересует не вид ошибки, а факт, что команда не выполнялась.

```
mydb> SELECT * FROM other;
Error: SQL: неизвестная таблица: other
mydb> INSERT INTO items VALUES (1, 2, 3);
Error: SQL: ожидается 2 значения
mydb> SELECT * FROM items;
id      | value
-----+-----
1       | 10
(1 строка)
```

После двух ошибок база продолжает работать как ни в чём не бывало.

Ошибка при открытии базы обрабатывается отдельно и до цикла — там продолжать нечего:

```
std::unique_ptr<db::Database> db_ptr;
try {
    db_ptr.reset(new db::Database(dir));
} catch (const std::exception& e) {
    std::cerr << "Не удалось открыть базу в " << dir << ": " << e.what() << "\n";
    return 1;
}
db::Database& db = *db_ptr;
```

Указатель здесь нужен из-за `try`: объект, созданный внутри блока, за его пределами не виден, а ссылку на него хочется иметь во всей функции. Разыменование сразу после — способ дальше писать `db.execute(...)`, а не `db_ptr->execute(...)`.

Возврат 1 из `main` — не мелочь: по нему скрипт, вызвавший базу, узнает, что дело не пошло.

39.7 Завершение

При штатном выходе нужно прибратся:

```
db.checkpoint();
history_save();
return 0;
```

Порядок важен и здесь: сначала данные, потом история. Если что-то пойдёт не так, потерять историю команд неприятно, а потерять данные — недопустимо.

Слово «штатном» здесь ключевое. Выход по `.exit` или `Ctrl+D` сюда попадает, а вот `kill -9` — нет, и в этом случае данные спасает журнал (глава 35).

39.8 Как это работает целиком

```
$ ./build/mydb /tmp/demo
mydb REPL — таблица 'items(id INT PRIMARY KEY, value INT)' по умолчанию.
Open: /tmp/demo
.help для справки, .quit для выхода.

mydb> SELECT *
... FROM items
... WHERE id > 0;
id      | value
-----+-----
(0 строк)
mydb> INSERT INTO items VALUES (1, 10); INSERT INTO items VALUES (2, 20);
1 строка
1 строка
mydb> SELECT * FROM items; -- комментарий в конце
id      | value
-----+-----
1       | 10
2       | 20
(2 строки)
mydb> .checkpoint
(checkpoint done)
mydb> .exit
```

Работают многострочный ввод, несколько команд в строке, комментарии и служебные команды.

39.9 Чего не хватает

Стрелки вверх и вниз не листают историю — по той же причине, что и в оболочке из главы 30: для этого нужен сырой режим терминала и разбор управляющих последовательностей. Подключение библиотеки `readline` или `libedit` даёт это в три строки, и такое упражнение есть в конце главы.

Нет автодополнения имён таблиц и ключевых слов, нет подсветки синтаксиса, нет постраничного вывода длинных результатов (в `psql` для этого запускается внешний `less`), нет вывода в форматах CSV и JSON, нет выполнения файла со скриптом.

Всё это — обвязка вокруг того же цикла, и ни одна из этих возможностей не меняет устройства программы. Именно поэтому клиенты баз данных выглядят так похоже: внутри у них одно и то же.

39.10 Главные правила главы

1. Интерактивная программа читает по строкам, но команда может занимать несколько строк — нужен накопитель и признак завершённости.
2. Обрабатывать надо все завершённые команды в накопителе, а не первую: иначе остаток блокирует ввод.
3. Служебные команды клиента отличают приставкой и распознают только в начале ввода.
4. Ошибка выполнения не должна прерывать цикл; ошибка открытия базы, наоборот, должна прекращать работу с ненулевым кодом возврата.
5. Поиск разделителя без учёта кавычек работает ровно до появления строковых литералов в языке.
6. У истории должен быть предел, иначе файл растёт бесконечно; обрезайте диапазоном, а не по одному элементу.
7. Приглашение без перевода строки требует явного `flush`.
8. При штатном завершении сначала сохраняют данные, потом всё остальное.

39.11 Упражнения

1. Соберите базу и проверьте все режимы: многострочный запрос, две команды в одной строке, комментарий, служебные команды, ошибочный запрос. Убедитесь, что после ошибки клиент продолжает работать.

2. Верните старое поведение — замените внутренний цикл на одну проверку — и воспроизведите ошибку с двумя командами в строке. Опишите, что при этом происходит с приглашением и служебными командами.
3. Добавьте служебную команду `.tables`, печатающую имя таблицы и список её колонок, и `.indexes` — список индексов с колонками. Данные для этого уже есть в каталоге из главы 37.
4. Добавьте `.read <файл>`: прочитать файл и выполнить все команды из него. Учтите, что в файле могут быть комментарии и многострочные запросы.
5. Сделайте вывод результата в формате CSV по служебной команде `.mode csv`. Заголовок — имена колонок через запятую, дальше строки.
6. Добавьте поддержку блочных комментариев `/* ... */`, в том числе многострочных. Понадобится флаг «мы внутри комментария», переживающий переход к следующей строке.
7. (Сложнее) Замените поиск точки с запятой на поиск через лексер: добавьте в него режим «найти конец команды», понимающий кавычки. Проверьте на запросе с точкой с запятой внутри строкового литерала (строки придётся сначала добавить в язык).
8. (Сложнее) Подключите `libedit` и получите стрелки, поиск по `Ctrl+R` и редактирование строки. Не забудьте отдавать историю библиотеке через `add_history` и освобождать строку, которую возвращает `readline`.

39.12 Что дальше

Глава 39 закрывает Часть IV. Соберём все замеры вместе и посмотрим, где именно наша база теряет время, — а теряет она его в предсказуемых местах, и мы их назовём поимённо. Сравним устройство `mysql` с `SQLite` и `PostgreSQL`: что у нас такое же, что упрощено и что отсутствует принципиально. И честно перечислим, чего не хватает до настоящей СУБД, — список получится длинный, но не бесконечный.

Глава 40

Глава 39. Замеры и итоги Части IV

База готова. В `demo-db/` лежит около двух тысяч восьмисот строк: страничный файл с кэшем, B+-дерево, журнал упреждающей записи, мини-SQL с разбором в дерево, вторичные индексы, планировщик, оболочка и тесты.

В этой главе не будет нового кода. Будет то, чего обычно не хватает учебным проектам: честный разбор, где эта база теряет время, что в ней устроено как во взрослых системах, а что упрощено до неузнаваемости. И один опыт, который переворачивает выводы предыдущих глав.

40.1 Измеряем

Программа `utils/bench.cpp` прогоняет базу по типичным операциям. Две тысячи строк, сборка с оптимизацией:

```
Insert:                358 мс, 5589 вставок/с
Find by id (B+tree):    113 мс, 17742 запросов/с
Find by value (полный перебор, 100 запросов): 4911 мс, 20 запросов/с
Build index:           395 мс
Find by value (индекс): 203 мс, 9865 запросов/с
```

Ускорение `find-by-value` с индексом: 484x

файл данных: 6135808 байт

Числа выглядят красиво: индекс ускоряет поиск в четыреста восемьдесят четыре раза. Именно такие числа любят приводить в статьях про индексы.

Только вот файл. Шесть мегабайт на две тысячи строк — это по три килобайта на строку, состоящую из двух восьмибайтовых чисел. Триста раз больше, чем данных.

40.2 Опыт, который меняет выводы

Причина в MAX_KEYS. Мы поставили четыре ключа на узел, чтобы разделения были видны на первых же вставках, — и с тех пор все замеры делались на дереве, у которого узлы заполнены на четыре тысячных процента.

Поменяем одну константу на 128 и прогоним тот же тест:

	MAX_KEYS = 4	MAX_KEYS = 128	
Insert	5589 /с	11104 /с	вдвое быстрее
Find by id	17742 /с	64799 /с	вчетверо быстрее
Полный перебор	20 /с	16496 /с	в 800 раз быстрее
Построение индекса	395 мс	166 мс	
Поиск по индексу	9865 /с	53608 /с	
Ускорение от индекса	484х	3.2х	
Файл данных	6.1 МБ	135 КБ	в 45 раз меньше

Изменилась одна цифра в заголовочном файле — изменилось всё.

Файл ужался в сорок пять раз: узлы перестали быть почти пустыми, и вместо тысяч страниц по четыре килобайта дерево уместилось в тридцать три страницы.

Полный перебор ускорился в восемьсот раз. Он и был медленным не потому, что перебор, а потому, что обходил полторы тысячи страниц вместо тридцати. Диск и кэш тут ни при чём — работа была лишней.

И главное: ускорение от индекса упало с 484 до 3.2 раза. На двух тысячах строк, помещающихся в тридцать три страницы, полный перебор — это тридцать три чтения из кэша, и обгонять его в сотни раз попросту негде.

Отсюда вывод, который стоит унести из всей Части IV дальше своей практики: **число, полученное измерением, верно только для тех условий, в которых измеряли.** Ускорение в 484 раза не было ложью — оно было правдой про дерево с четырьмя ключами в узле. Стоило привести дерево в чувство, и правда стала другой.

Именно так и получают те самые статьи, где новая база быстрее старой в сорок раз: измеряют в условиях, где старая работает не так, как в жизни. Проверять всегда стоит не результат, а условия.

Ускорение от индекса, конечно, вернётся с ростом таблицы — перебор линейен, поиск логарифмичен. Но начинается выигрыш не с двух тысяч строк, а заметно позже, и это тоже полезно понимать: на маленькой таблице индекс может не окупить своей стоимости при записи.

40.3 Где теряется время

Даже с нормальным `MAX_KEYS` наша база далека от быстрой. Шестьдесят пять тысяч поисков в секунду по данным, целиком лежащим в памяти, — результат посредственный. Разберём, куда уходит время; все причины мы честно назвали по ходу дела.

Первая — копирование страницы. `read_page` возвращает копию, то есть четыре килобайта пересылки памяти на каждое обращение к узлу. При высоте дерева три это двенадцать килобайт на поиск. Лечится возвратом ссылки с закреплением страницы, о чём шла речь в главе 33.

Вторая — распаковка узла. `read_node` создаёт три вектора и переливает в них ключи, значения и ссылки — то есть выделяет память при каждом обращении к узлу. Промышленные реализации ищут двоичным поиском прямо в байтах страницы, ничего не выделяя.

Третья — сравнение операторов строками. В `matches_where` на каждой строке таблицы выполняется до шести сравнений `std::string`. Разбор должен был превратить оператор в перечисление один раз, а не оставлять эту работу исполнителю.

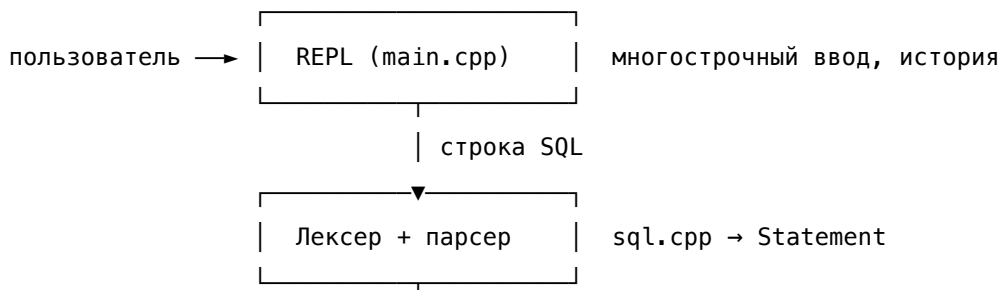
Четвёртая — синхронизация журнала на каждую вставку. Сто пятьдесят микросекунд из ста восьмидесяти, которые занимает вставка, — это `fsync`. Групповая фиксация убрала бы почти всё.

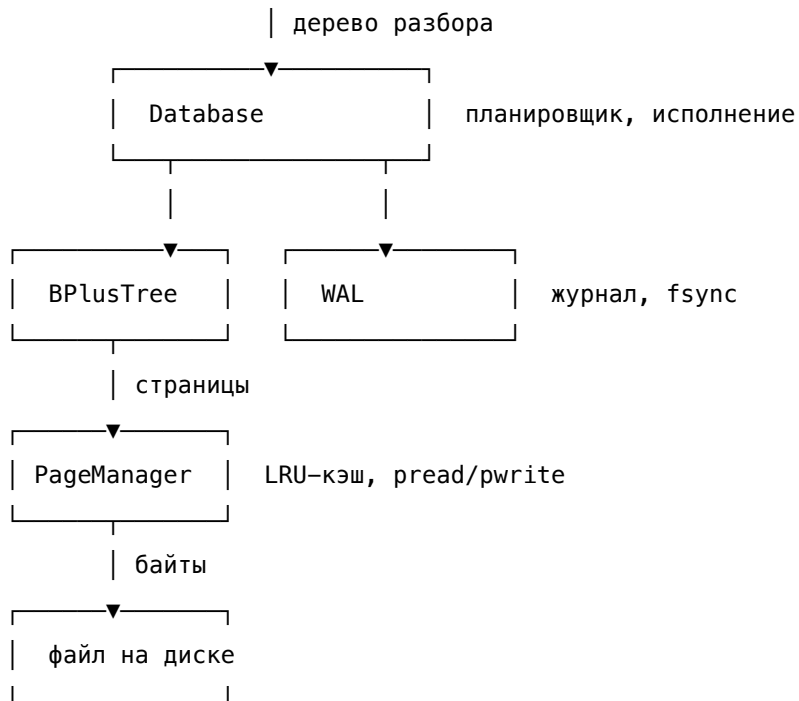
Ни одна из этих проблем не относится к устройству базы — все четыре относятся к реализации. Устройство у нас правильное: страницы, дерево, журнал, планировщик стоят там, где им положено стоять.

Кстати, о разнице замеров. В главе 35 вставка через оболочку показывала 1943 операции в секунду, а здесь через прямой вызов — 5589. Разница не в базе: там каждая команда проходила через конвейер, разбор строки и вывод результата. Полезное напоминание о том, что измерять надо ту часть, о которой делаете вывод.

40.4 Что мы построили

Соберём картину целиком.





Слои разделены так же, как в настоящих системах, и это не случайное совпадение — просто других разумных способов разложить эти обязанности не придумали. Каждый слой знает только про соседний снизу: дерево не подозревает про SQL, страничный менеджер не подозревает про дерево, а файл — просто файл.

Размеры по файлам:

Файл	Строк	Что делает
page_manager.cpp	161	страницы, кэш, fsync
btree.cpp	353	дерево: поиск, вставка, разделение, обход
wal.cpp	143	журнал: запись, восстановление, обрезание
sql.cpp	245	лексер и разбор
database.cpp	328	планировщик, исполнение, каталог
main.cpp	200	оболочка
tests/test_db.cpp	404	51 проверка
утилиты (bench, три демо)	340	замеры и демонстрации

Около полутора тысяч строк основного кода на работающую базу данных. Это не должно удивлять: сложность промышленных систем растёт не от базовых механизмов, а от всего того, чего у нас нет.

40.5 Как это соотносится с настоящими базами

Полезно разложить по трём стопкам: что устроено так же, что упрощено и чего нет вовсе.

Так же, как у взрослых. Файл как массив страниц фиксированного размера. Кэш страниц с вытеснением. В+-дерево с разделением узлов и связанными листьями. Журнал упреждающей записи с контрольными суммами и восстановлением. Разделение разбора и выполнения. Планировщик, выбирающий способ выполнения. Каталог с описанием схемы. Команда EXPLAIN.

Упрощено. Кэш возвращает копии вместо ссылок с закреплением. Узел распаковывается в вектора вместо работы с байтами. Планировщик действует по правилам вместо оценки стоимости. Каталог — текстовый файл вместо системных таблиц. Контрольная сумма — полиномиальный хеш вместо CRC32. Размер узла — четыре ключа вместо двухсот пятидесяти.

Отсутствует принципиально. Удаление и изменение строк. Транзакции с откатом. Многоверсионность и изоляция. Одновременный доступ и блокировки. Соединения таблиц. Строковые и вещественные типы. Составные и уникальные индексы. Ограничения целостности. Репликация. Сетевой протокол и клиенты. Права доступа.

Последняя стопка и объясняет, почему PostgreSQL — это полтора миллиона строк, а не полторы тысячи. Каждый пункт тянет за собой не функцию, а подсистему, и подсистемы взаимодействуют: многоверсионность меняет формат страницы, изоляция меняет планировщик, репликация меняет журнал.

Сравнение по устройству выглядит так:

	mydb	SQLite	PostgreSQL
Страница	4 КБ	4 КБ (настраивается)	8 КБ
Индекс	В+-дерево	В-дерево	В-дерево, GiST, GIN, BRIN, хеш
Журнал	свой WAL	журнал отката или WAL	WAL с LSN
Планировщик	по правилам	по стоимости	по стоимости, со статистикой
Транзакции	нет	есть	есть, с многоверсионностью
Одновременность		блокировка файла	процесс на соединение
Строк кода	~1400	~150 000	~1 500 000

40.6 Чему научила эта часть

Если отвлечься от базы данных, в Части IV было несколько вещей, которые пригодятся в любой работе с данными.

Как устроен двоичный формат и почему структуру нельзя писать в файл как есть. Как считается смещение и почему для этого нужен широкий тип. Что `write` не означает «на диске», а `fsync` — означает и стоит миллисекунду.

Почему структуры данных для диска проектируют вокруг размера блока и как это меняет выбор алгоритма: двоичное дерево, безупречное в памяти, на диске проигрывает вчистую.

Что такое упреждающая запись и почему порядок «сначала журнал, потом данные» — не рекомендация, а условие корректности. Как из идемпотентности операций получается простое восстановление после аварии.

Как разбирается язык: лексер, парсер рекурсивным спуском, дерево разбора, исполнитель. Мы делали это в четвёртый раз, и, надеюсь, к этому моменту схема стала привычной.

И самое ценное — как измерять и как не обмануться. Считать не догадками, а счётчиками. Сомневаться в удобных числах. Проверять условия, а не выводы.

40.7 Главные правила Части IV

1. Двоичный формат сериализуют полем за полем; структуру в файл не пишут.
2. Долгоживущий формат начинается с метки и версии.
3. Файл базы — массив страниц; размер страницы кратен блоку файловой системы.
4. Кэш нужен потому, что диск в тысячи раз медленнее памяти; вытеснение по давности бессильно против последовательного обхода.
5. Гарантию сохранности даёт только `fsync`, и он дорогой — поэтому синхронно пишут маленький последовательный журнал, а не разрозненные страницы.
6. B+-дерево даёт три-четыре обращения к диску на любой поиск, потому что узел размером в страницу даёт ветвление в сотни раз.
7. Индексы ускоряют чтение и замедляют запись; выигрыш зависит от размера таблицы и доли подходящих строк.
8. Числа из замера верны только для условий замера — сначала проверяйте условия.

40.8 Упражнения

1. Повторите опыт с `MAX_KEYS`: прогоните `./build/bench 2000` при значениях 4, 16, 64 и 250. Постройте таблицу по времени поиска и размеру файла. При каком значении рост прекращается и почему?
2. Прогоните замер при разных размерах кэша страниц (второй аргумент конструктора `PageManager`): 4, 16, 64, 512. Сопоставьте с числом промахов из `stats()`.

3. Уберите `fsync` из журнала и повторите замер вставки. Какую долю времени он занимал? Верните обратно.
4. Замените сравнение операторов строками на перечисление, как предлагалось в главе 36, и замерьте полный перебор до и после на десяти тысячах строк.
5. Соберите базу с `-O0` и с `-O2` и сравните все числа. Где разница больше всего и почему?
6. Напишите нагрузку, ближе к реальной: восемьдесят процентов чтений и двадцать процентов вставок вперемешку. Сравните два варианта — с индексом по второй колонке и без него.
7. (Сложнее) Профилируйте базу (`perf` на Linux, Instruments на macOS) на полном переборе. Подтвердите или опровергните утверждение, что основное время уходит на копирование страниц и распаковку узлов.
8. (Сложнее) Реализуйте возврат страницы по ссылке с закреплением вместо копии. Замерьте выигрыш и опишите, какие новые ошибки стали возможны.

40.9 Часть IV закрыта

Восемь глав назад у нас был пустой каталог. Сейчас есть база данных, которая переживает выключение питания, отвечает на запросы за логарифм от числа строк и умеет объяснить, почему она решила выполнить запрос именно так.

Она маленькая и во многом наивная, но принципиально не игрушечная: те же механизмы, в том же порядке, с теми же обязанностями, что и у систем, на которых работает половина интернета. Разница — в количестве проработанных случаев, а не в идеях.

40.10 Что дальше

Часть V — сетевое программирование. Восемь глав, за которые мы напишем `demo-chat`: TCP-сервер и клиент, многопоточную версию с мьютексами, атомарные операции, однопоточный сервер на цикле событий, собственный двоичный протокол поверх TCP, чат с комнатами, простое шифрование и разбор его слабостей, наконец сборку в контейнер размером пять мегабайт.

Работа снова будет другой. В Части III мы разговаривали с операционной системой, в Части IV боролись за корректность на диске. В Части V главным станет одновременность: несколько клиентов, несколько потоков, гонки данных и способы их избежать. Начнём с самого начала — с сокета.

Глава 41

Глава 40. Сокеты

Начинается Часть V, и в ней восемь глав мы будем писать сетевой чат. Сервер, клиент, свой двоичный протокол, комнаты, многопоточность, цикл событий, шифрование и упаковка в контейнер. Лежать всё это будет в `demo-chat/`.

Главной темой части станет одновременность. В Части III мы разговаривали с операционной системой, в Части IV боролись за корректность на диске — а здесь появится десяток клиентов, которые обращаются к серверу когда им вздумается, и данные, к которым обращаются сразу несколько потоков.

Но начнём с основы: как две программы вообще передают друг другу байты. Интерфейс для этого придумали в Беркли в начале восьмидесятых, назвали сокетами, и с тех пор он почти не изменился — тот же код работает на Linux, macOS, BSD и, с косметическими правками, на Windows.

К концу главы у нас будет работающий эхо-сервер и клиент к нему.

41.1 Что такое сокет

Сокет — это конечная точка соединения, и в программе он выглядит как файловый дескриптор. То самое небольшое целое число из главы 27, только за ним стоит не файл, а сетевое соединение.

Отсюда следует приятное: с сокетом работают те же вызовы, что и с файлом. `read` и `write` на сокете работают, `close` закрывает его, `dup2` подменяет — можно, например, направить в сокет стандартный вывод запускаемой программы, и она будет отвечать по сети, ничего об этом не подозревая. Ровно так устроен классический `inetd`.

Отдельные вызовы `recv` и `send` существуют потому, что у них есть четвёртый аргумент — флаги, которых у `read` и `write` нет. Мы им воспользуемся: флаг `MSG_NOSIGNAL` в главе 43 спасёт сервер от смерти.

41.2 TCP и UDP

Прежде чем создавать сокет, надо выбрать вид.

TCP даёт поток байтов с гарантиями: что отправлено, то и придёт, в том же порядке, без потерь и дубликатов. Достигается это подтверждениями, повторными отправками и нумерацией — всё внутри ядра, программе видна только чистая труба. Плата — установление соединения тремя пакетами, задержка при потерях (потерянный кусок ждут, а следующие за ним данные лежат в очереди) и состояние на обеих сторонах.

UDP отправляет отдельные пакеты и ничего не обещает: пакет может потеряться, продублироваться, прийти не в том порядке. Зато никакого соединения, никакой задержки на подтверждения — отправил и забыл.

Для чата нужен TCP: терять сообщения нельзя, порядок важен, объёмы маленькие. UDP выбирают там, где свежесть данных важнее полноты, — голос, видео, игры, где потерянный кадр проще пропустить, чем ждать.

Есть третий вариант, который стоит держать в голове: поверх UDP делают собственные протоколы с нужным набором гарантий. Так устроен QUIC, на котором работает HTTP/3, — ему понадобилось избавиться от блокировки очереди, которую TCP навязывает по своей природе.

41.3 Порядок вызовов

У сервера и клиента разные последовательности, и их полезно видеть рядом.

Сервер			Клиент	
<code>socket()</code>	создать сокет		<code>socket()</code>	
<code>bind()</code>	занять адрес и порт			
<code>listen()</code>	начать слушать			
<code>accept()</code>	ждать клиента	←	<code>connect()</code>	подключиться
<code>recv/send</code>	общаться	↔	<code>send/recv</code>	
<code>close()</code>			<code>close()</code>	

Ключевое отличие в `accept`: он не принимает данные, а порождает **новый** сокет — отдельный на каждое соединение. Слушающий сокет продолжает слушать, а разговор идёт через новый.

Это сбивает с толку при первом знакомстве, зато объясняет, как один сервер работает с сотней клиентов.

41.4 Создание сокета

```
int srv = ::socket(AF_INET, SOCK_STREAM, 0);
if (srv < 0) {
    std::cerr << "socket: " << std::strerror(errno) << "\n";
    return 1;
}
```

Первый аргумент — семейство адресов: `AF_INET` для IPv4, `AF_INET6` для IPv6, `AF_UNIX` для сокетов внутри одной машины, работающих через файловую систему. Второй — тип: `SOCK_STREAM` для TCP, `SOCK_DGRAM` для UDP. Третий — протокол внутри типа; ноль означает «выбери сам», и для пары `AF_INET` плюс `SOCK_STREAM` это всегда TCP.

Двойное двоеточие перед именем — `::socket` — указывает глобальную область видимости. Привычка полезная: в программе может оказаться свой `socket` в пространстве имён проекта, и без уточнения компилятор выберет его.

41.5 Адрес

Адрес для IPv4 описывается структурой `sockaddr_in`:

```
sockaddr_in addr;
std::memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(static_cast<std::uint16_t>(port));
addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

Обнуление через `memset` обязательно: в структуре есть поле `sin_zero`, оставшееся от древних времён, и мусор в нём приводит к трудноуловимым ошибкам.

`htons` и `htonl` — те самые преобразователи порядка байтов из главы 32. В сети принят порядок старшим байтом вперёд, и номер порта с адресом обязаны быть записаны именно так. Забыть `htons` — классическая ошибка: сервер запускается, но слушает не тот порт. Порт 9001 без преобразования превращается в 10531.

`INADDR_ANY` означает «все адреса машины»: сервер примет соединение и через сетевую карту, и через локальную петлю. Альтернатива — `INADDR_LOOPBACK`, тогда достучаться можно будет только с этой же машины. Для отладки второе безопаснее.

Дальше начинается самое неприятное в этом интерфейсе — приведение типов:

```
::bind(srv, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
```

Функции принимают указатель на `sockaddr` — обобщённую структуру, а мы передаём `sockaddr_in`, специфичную для IPv4. Для IPv6 передавали бы `sockaddr_in6`. Это способ сделать полиморфизм в языке C, где его нет: все структуры начинаются с одинакового поля семейства, по которому ядро понимает, что ему дали.

С точки зрения C++ такое приведение — нарушение правил о совместимости типов (глава 32), но здесь оно узаконено самим стандартом POSIX, и другого способа просто нет.

41.6 bind, listen, accept

```
if (::bind(srv, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) < 0) {  
    std::cerr << "bind: " << std::strerror(errno) << "\n";  
    ::close(srv);  
    return 1;  
}  
  
if (::listen(srv, 16) < 0) {  
    std::cerr << "listen: " << std::strerror(errno) << "\n";  
    ::close(srv);  
    return 1;  
}
```

`bind` закрепляет за сокетом адрес и порт. Самая частая ошибка здесь — `EADDRINUSE`, «адрес уже используется»; о ней отдельный разговор ниже.

Порты меньше 1024 — привилегированные: занять их может только процесс с правами администратора. Поэтому веб-сервер запускается от `root`, занимает 80-й порт и сразу сбрасывает права. Мы берём 9001 — свободный и непривилегированный.

`listen` переводит сокет в режим ожидания соединений. Вторым аргументом — длина очереди уже установленных, но ещё не принятых соединений. Если очередь переполнена, новые попытки подключения ядро отвергает. Шестнадцать — разумно для учебного сервера; у нагруженных значение доводят до тысяч.

```
sockaddr_in cli;  
socklen_t cli_len = sizeof(cli);  
int c = ::accept(srv, reinterpret_cast<sockaddr*>(&cli), &cli_len);  
if (c < 0) {
```



```
if (errno == EINTR) continue;
std::cerr << "accept: " << std::strerror(errno) << "\n";
break;
}
```

accept блокирует выполнение, пока кто-нибудь не подключится, и возвращает новый дескриптор для разговора с этим клиентом. Заодно он заполняет структуру адресом клиента — отсюда и второй с третьим аргументы.

Третий аргумент — указатель на длину, и это ещё одна особенность интерфейса. На входе он говорит, сколько места в структуре, на выходе — сколько ядро записало. Такой параметр называют «значение-результат», и в сокетном интерфейсе их несколько.

Проверка на EINTR знакома по главе 28: блокирующий вызов может прерваться сигналом, и это не ошибка, а повод повторить.

Адрес клиента печатаем человекочитаемо:

```
char ip[INET_ADDRSTRLEN] = {0};
::inet_ntop(AF_INET, &cli.sin_addr, ip, sizeof(ip));
std::cout << "Клиент " << ip << ":" << ntohs(cli.sin_port) << " подключён\n";
```

inet_ntop переводит двоичный адрес в строку вида 192.168.1.5; INET_ADDRSTRLEN — константа с достаточным размером буфера. Обратное преобразование делает inet_pton. Старые функции inet_ntoa и inet_addr встречаются в примерах из девяностых, но у них нет поддержки IPv6 и есть проблемы с многопоточностью — не используйте.

41.7 Обмен данными

```
char buf[1024];
while (true) {
    ssize_t n = ::recv(c, buf, sizeof(buf), 0);
    if (n == 0) { std::cout << "клиент отключился\n"; break; }
    if (n < 0) {
        if (errno == EINTR) continue;
        std::cerr << "recv: " << std::strerror(errno) << "\n";
        break;
    }
    ssize_t sent = 0;
    while (sent < n) {
        ssize_t s = ::send(c, buf + sent, n - sent, 0);
```

```

    if (s < 0) {
        if (errno == EINTR) continue;
        break;
    }
    sent += s;
}
}

```

Три исхода `recv` те же, что у `read` в главе 31, но со своими значениями. Ноль означает, что собеседник закрыл соединение, — это не ошибка, а нормальное завершение. Отрицательное — ошибка, и `EINTR` среди них особая. Положительное — сколько байт пришло, и оно почти никогда не равно запрошенному.

Отправка обёрнута в цикл по той же причине, что и запись в файл: `send` вправе отправить меньше, чем попросили, если буфер отправки заполнен. Не проверять его результат — значит терять хвосты сообщений при высокой нагрузке, а это ошибка, которая не воспроизводится в тестах и вылезает в бою.

41.7.1 TCP — поток, а не сообщения

Здесь скрыто самое важное свойство TCP, и его лучше увидеть, чем прочитать. Клиент отправляет три сообщения по двенадцать байт тремя вызовами `send`, а сервер делает один `recv`:

```

::send(fd, "ПЕРВОЕ", 12, 0);
::send(fd, "ВТОРОЕ", 12, 0);
::send(fd, "ТРЕТЬЕ", 12, 0);

```

```

$ ./stream

```

три вызова `send` по 12 байт → один `recv` вернул 36 байт:

```

"ПЕРВОЕВТОРОЕТРЕТЬЕ"

```

Границы сообщений исчезли. TCP не знает, что вы считаете сообщением, — он передаёт поток байтов и вправе склеивать их и разрезать как угодно. Три отправки могут прийти одним куском, одна отправка — тремя кусками.

Из этого следует, что **любой протокол поверх TCP обязан сам размечать границы сообщений**. Способов три: разделитель (например, перевод строки, как в HTTP и SMTP), префикс длины (как в нашем формате из главы 32) или фиксированный размер. Наш эхо-сервер работает без разметки только потому, что ему всё равно: что пришло, то и отправил обратно.

В главе 44 мы напишем протокол с префиксом длины и накопителем — и заодно поймём, почему эта задача сложнее, чем кажется.

41.8 Клиент

У клиента вместо bind и listen один вызов connect, но перед ним — разрешение имени:

```
addrinfo hints;
std::memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;      // и IPv4, и IPv6
hints.ai_socktype = SOCK_STREAM;

addrinfo* res = nullptr;
int rc = ::getaddrinfo(host, port, &hints, &res);
if (rc != 0) {
    std::cerr << "getaddrinfo: " << ::gai_strerror(rc) << "\n";
    return 1;
}

int fd = -1;
for (addrinfo* p = res; p; p = p->ai_next) {
    fd = ::socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (fd < 0) continue;
    if (::connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;
    ::close(fd);
    fd = -1;
}
::freeaddrinfo(res);
```

getaddrinfo — единственная правильная функция для превращения имени в адрес. Она умеет и доменные имена, и цифровые адреса, и IPv4, и IPv6, и имена служб ("http" вместо "80").

Возвращает она **список** адресов, и перебор этого списка — не формальность. У домена может быть несколько адресов, часть из них может не отвечать, IPv6 может быть настроен, но не работать. Правильный клиент пробует все по очереди — именно так делают браузеры, только они ещё и параллельно.

Ошибки getaddrinfo расшифровываются не через strerror, а через gai_strerror: у неё собственный набор кодов.

`freeaddrinfo` обязателен — список выделен в куче. Здесь напрашивается умный указатель со своим удалителем (глава 18); в учебном коде оставлено как есть, но в настоящем проекте стоит обернуть.

Дальше клиент читает строки со стандартного ввода, отправляет и печатает ответ:

```
std::string line;
while (true) {
    std::cout << "> ";
    std::cout.flush();
    if (!std::getline(std::cin, line)) break;
    if (line.empty()) break;
    line += '\n';
    // ... send в цикле ...
    char buf[1024];
    ssize_t n = ::recv(fd, buf, sizeof(buf), 0);
    if (n <= 0) {
        std::cerr << "сервер закрыл соединение\n";
        break;
    }
    std::cout << "<- " << std::string(buf, n);
}
```

Обратите внимание: клиент отправляет запрос и сразу ждёт ответ. Для эха годится, для чата — нет: там сообщения приходят когда угодно, а не в ответ на наши. Как быть с этим, разберёмся в главе 41, где появится второй поток.

41.9 Проверка

```
$ ./build/echo_server 9101 &
Echo-server слушает на 0.0.0.0:9101

$ ./build/echo_client 127.0.0.1 9101
Подключён к 127.0.0.1:9101. Пустая строка — выход.

> привет
<- привет
> вторая строка
```

```
<- вторая строка  
>
```

Сервер работает и с обычным telnet или nc, потому что никакого своего протокола у него пока нет:

```
$ nc localhost 9101  
проверка  
проверка
```

Посмотреть, что порт действительно слушается, можно системными средствами: `lsof -i :9101` покажет процесс, `netstat -an | grep 9101` — состояние сокета.

41.10 TIME_WAIT и SO_REUSEADDR

Остался вызов, который я до сих пор не объяснил:

```
int yes = 1;  
if (::setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes)) < 0) { ... }
```

Без него сервер, который только что работал и был остановлен, при перезапуске получает `bind: Address already in use` — хотя никто порт не занимает. Проверим на опыте: программа поднимает сервер, принимает соединение, закрывается и сразу пытается занять порт снова.

```
$ ./twait  
без SO_REUSEADDR: повторный bind: Address already in use  
$ ./twait reuse  
с SO_REUSEADDR: повторный bind сразу удался
```

Причина — состояние `TIME_WAIT`. Сторона, закрывшая соединение первой, держит его в этом состоянии около минуты (на macOS и Linux — от 30 до 120 секунд в зависимости от настроек). Нужно это затем, чтобы опоздавшие пакеты старого соединения не были приняты новым, случайно занявшим ту же пару адресов и портов. Пока соединение в `TIME_WAIT`, его порт считается занятым.

`SO_REUSEADDR` говорит ядру: «разреши занять порт, даже если на нём висят соединения в `TIME_WAIT`». Это ровно то, что нужно серверу при перезапуске.

Заодно развеим распространённое заблуждение. Многие думают, что `SO_REUSEADDR` позволяет двум серверам слушать один порт. Не позволяет:

```
$ ./reuse 9188  
без SO_REUSEADDR: bind удался
```

```
без SO_REUSEADDR: bind не удался – Address already in use  
с SO_REUSEADDR: bind не удался – Address already in use
```

Первый занял порт, и второй не может занять его ни с флагом, ни без. Для нескольких процессов на одном порту существует другой флаг — `SO_REUSEPORT`, — и он появился гораздо позже, для распределения нагрузки между процессами. Не путайте их.

41.11 Чего наш сервер не умеет

Один клиент за раз. Пока идёт разговор с одним, остальные ждут в очереди `listen`. Это следующая глава.

Ничего, кроме эха. Никакого протокола, никаких имён, никакой рассылки — всё впереди.

Не переживает разрыв. Если клиент исчез без закрытия соединения — выдернули кабель, ушёл ноутбук, — сервер узнает об этом не скоро: TCP обнаружит обрыв только при попытке отправки, а если ничего не отправлять, соединение может висеть часами. Лечится это либо своими сообщениями «я жив» на уровне протокола, либо флагом `SO_KEEPALIVE`, включающим проверки на уровне TCP (правда, по умолчанию первая проверка происходит через два часа).

Не ограничивает никого и ни в чём. Клиент может отправить гигабайт в одном сообщении, открыть тысячу соединений, вообще ничего не отправлять и держать соединение занятым. Все три случая для настоящего сервера — поводы для защиты.

41.12 Главные правила главы

1. Сокет — это файловый дескриптор; с ним работают те же `read`, `write` и `close`, а `recv` и `send` добавляют флаги.
2. TCP даёт надёжный поток байтов, UDP — ненадёжные отдельные пакеты; выбор делается по тому, что важнее — полнота или свежесть.
3. ассерт возвращает новый сокеты; слушающий продолжает слушать.
4. Номер порта и адрес записываются в сетевом порядке байтов — `htons` и `htonl` обязательны.
5. TCP не хранит границ сообщений: разметку обязан обеспечивать протокол поверх него.
6. `recv` и `send` обрабатывают меньше запрошенного; оба зовут в цикле, ноль от `recv` означает закрытие соединения.
7. Имя в адрес переводит `getaddrinfo`, и перебирать надо весь возвращённый список.
8. `SO_REUSEADDR` снимает запрет на занятие порта, оставшегося в `TIME_WAIT`, но не позволяет двум серверам слушать один порт.

41.13 Упражнения

1. Соберите `demo-chat` и запустите эхо-сервер с клиентом. Затем подключитесь к серверу через `nc localhost 9001` и через `telnet`. Объясните, почему это работает без всякого клиента.
2. Подключите к серверу двух клиентов одновременно. Что происходит со вторым? Посмотрите состояние соединений через `netstat -an | grep 9001` и найдите очередь.
3. Уберите `SO_REUSEADDR`, пересоберите, запустите сервер, подключитесь клиентом, остановите сервер и сразу запустите снова. Сколько времени порт остаётся занятым на вашей системе?
4. Повторите опыт со склейкой сообщений: отправьте три коротких сообщения подряд и посмотрите, сколько байт вернёт один `recv`. Затем вставьте паузу в полсекунды между отправками — изменится ли результат?
5. Замените в сервере `recv` и `send` на `read` и `write` и убедитесь, что всё работает. Какие возможности вы при этом потеряли?
6. Сделайте так, чтобы сервер печатал длину каждого полученного куска. Отправьте клиентом строку в сто килобайт (например, `python3 -c "print('x'*100000)" | nc localhost 9001`) и посмотрите, на сколько кусков она разбилась.
7. (Сложнее) Добавьте серверу таймаут на бездействующего клиента через `setsockopt` с `SO_RCVTIMEO`. Проверьте, что `recv` возвращает `-1` с `errno == EAGAIN` по истечении времени.
8. (Сложнее) Перепишите клиент так, чтобы он работал и с IPv6: уберите привязку к `AF_INET` там, где она осталась, и проверьте подключение к `:::1`. Что придётся поменять в сервере, чтобы он слушал оба протокола?

41.14 Что дальше

Глава 41 — многопоточность. Наш сервер обслуживает одного клиента за раз, и это никуда не годится для чата. Заведём по потоку на соединение, познакомимся с `std::thread`, увидим первую гонку данных своими глазами и научимся защищать общие данные мьютексом. А заодно поймём, почему модель «поток на соединение» проста, удобна и плохо масштабируется.

Глава 42

Глава 41. Поток и мьютексы

Эхо-сервер из прошлой главы обслуживает одного клиента за раз. Пока идёт разговор с первым, второй ждёт в очереди `listen` и не понимает, почему сервер молчит.

Для чата это не годится по определению: чат — это когда несколько человек говорят одновременно. Значит, серверу нужно вести несколько разговоров сразу.

Самый прямой способ — по потоку на соединение. В этой главе мы его и применим: научимся создавать потоки, увидим первую гонку данных своими глазами (и не в виде страшилки, а в виде упавшей программы), защитим общие данные мьютексом и разберёмся, почему эта простая модель плохо растёт вширь.

42.1 Три способа обслуживать многих

Прежде чем писать код, стоит увидеть развилку целиком.

Процесс на клиента. После `accept` делаем `fork`, и ребёнок разговаривает с клиентом. Так работали веб-серверы девяностых, так до сих пор устроен PostgreSQL. Плюсы — полная изоляция: упавший обработчик не заденет остальных. Минусы — процесс дорог (мы измеряли в главе 30: около трёх миллисекунд), а общих данных нет, и списком клиентов придётся делиться через разделяемую память или трубы.

Поток на клиента. После `accept` создаём поток. Дешевле процесса, память общая — список клиентов лежит в обычном векторе. Расплата за общую память — гонки данных, ради которых и написана эта глава.

Один поток и цикл событий. Никаких потоков вообще: один цикл спрашивает у ядра, на каких сокетах что-то произошло, и обрабатывает готовые. Так работают `nginx`, `Redis`, `Node.js`. Плюс —

нет гонок, потому что нет одновременности. Минус — код выворачивается наизнанку, а одна долгая операция останавливает всех. Этим займёмся в главе 43.

Мы пройдем второй и третий пути, чтобы сравнить их на одной и той же задаче.

42.2 Поток за две строки

```
#include <thread>

std::thread(handle_client, c).detach();
```

Первый аргумент конструктора — функция, дальше её аргументы. Поток стартует сразу, ещё до того, как конструктор вернулся.

`detach` отвязывает поток от объекта: тот работает сам по себе, а когда закончит — уберётся сам. Альтернатива — `join`, дождаться завершения; но серверу ждать нельзя, ему надо возвращаться к `accept`.

Забудь и то и другое — фатально. Деструктор `std::thread`, у которого поток ещё привязан, вызывает `std::terminate` и убивает программу:

```
{
    std::thread t(work);
} // ← здесь программа завершится
```

Решение выглядит грубым, но оно осознанное. Комитет рассматривал варианты «неявно дождаться» и «неявно отвязать» и отверг оба: первый порождает незаметные зависания, второй — работу с уничтоженными объектами. Лучше явная ошибка сразу, чем тихая беда потом.

Аргументы копируются в поток. Если нужна ссылка, оборачивают в `std::ref`:

```
std::thread t(work, std::ref(shared_data));
```

Приём одновременно и удобный, и опасный: копия защищает от того, что объект исчезнет раньше потока, а `std::ref` эту защиту снимает. Правило простое: передавать ссылку можно только на то, что гарантированно переживёт поток.

42.3 Гонка данных вживую

Теперь то, ради чего глава написана. Возьмём общий вектор и будем добавлять в него из четырёх потоков:

```
std::vector<int> g_v;

void worker_unsafe(int n) {
    for (int i = 0; i < n; ++i) g_v.push_back(i);
}

int main() {
    std::vector<std::thread> ts;
    for (int i = 0; i < 4; ++i) ts.push_back(std::thread(worker_unsafe, 25000));
    for (std::size_t i = 0; i < ts.size(); ++i) ts[i].join();
    std::cout << "ждали 100000, получили " << g_v.size() << "\n";
}
```

Запускаем:

```
$ ./racevec
→ программа упала, код 134
```

Не «получили неправильное число», а именно упала — сто тридцать четыре означает аварийное завершение. Причина понятна, если вспомнить, что делает `push_back`: он смотрит, есть ли место, при необходимости выделяет новый буфер вдвое больше, копирует туда старое содержимое, освобождает старый буфер и увеличивает счётчик размера. Когда два потока делают это одновременно, один освобождает буфер, которым второй в этот момент пользуется.

Добавим мьютекс:

```
std::mutex g_m;

void worker_safe(int n) {
    for (int i = 0; i < n; ++i) {
        std::lock_guard<std::mutex> lock(g_m);
        g_v.push_back(i);
    }
}
```

```
$ ./racevec safe
с мьютексом: ждали 100000, получили 100000
```

Разница в две строки. Отсюда правило, из которого нет исключений: **любые данные, к которым обращаются несколько потоков и хотя бы один из них пишет, должны быть защищены.**

42.3.1 Инструмент, который находит гонки

Полагаться на то, что программа упадёт, нельзя: гонка может годами не проявляться, а потом выстрелить на машине пользователя. Поэтому существуют детекторы.

Соберём ту же программу с `-fsanitize=thread`:

```
$ g++ -std=c++11 -O1 -g -fsanitize=thread -pthread racevec.cpp -o racevec_tsan
$ ./racevec_tsan
=====
WARNING: ThreadSanitizer: data race (pid=56677)
  Read of size 8 at 0x000102c4e048 by thread T2:
    #0 std::vector<int>::push_back(int const&) vector.h:455
    #1 worker_unsafe(int) racevec.cpp:12

  Previous write of size 8 at 0x000102c4e048 by thread T1:
    #0 std::vector<int>::push_back(int const&) vector.h:455
    #1 worker_unsafe(int) racevec.cpp:12

  Location is global 'g_v' at 0x000102c4e040
```

Отчёт называет всё: адрес, размер обращения, оба потока, обе строки исходника и даже имя переменной. Найти такую ошибку глазами почти невозможно, а детектор находит её за секунду.

ThreadSanitizer замедляет программу раз в десять и увеличивает потребление памяти, поэтому в бою его не держат. Правильное место — тесты и отладочные сборки, а на многопоточном сервере это первое, что стоит включить.

Есть и родственники, о которых полезно знать: `-fsanitize=address` находит выход за границы и обращение к освобождённой памяти, `-fsanitize=undefined` — неопределённое поведение вроде переполнения знакового целого. Все три работают в GCC и Clang.

42.4 Мьютекс

Мьютекс — это замок, который в каждый момент держит не больше одного потока. Остальные ждут.

Пользоваться им напрямую — через `lock` и `unlock` — можно, но не нужно:

```
g_mutex.lock();
g_clients.push_back(client);
g_mutex.unlock();           // ← а если между ними бросят исключение?
```

Исключение между двумя вызовами оставит замок закрытым навсегда, и все потоки, которые к нему придут, повиснут. Ровно та же задача, что с освобождением памяти в главе 11, и решается она тем же приёмом — захват ресурса при создании объекта:

```
void register_client(int fd, const std::string& name) {  
    std::lock_guard<std::mutex> lock(g_mutex);  
    g_clients.push_back({fd, name});  
}
```

`std::lock_guard` закрывает замок в конструкторе и открывает в деструкторе. Как бы функция ни завершилась — обычным возвратом, `return` из середины, исключением — деструктор отработает и замок откроется.

В C++11 есть ещё `std::unique_lock` — то же самое, но с возможностью открыть и закрыть замок вручную посреди жизни объекта, а также передать владение. Он нужен для условных переменных и стоит чуть дороже; когда достаточно `lock_guard`, берите его.

42.5 Сервер чата

Общее состояние — список клиентов и мьютекс к нему:

```
struct Client {  
    int fd;  
    std::string name;  
};  
  
std::mutex g_mutex;  
std::vector<Client> g_clients;  
  
// Счётчик имён. Его увеличивают потоки-обработчики, значит, обычный int  
// здесь — гонка данных: два клиента, подключившиеся одновременно, могли бы  
// получить одно имя. atomic делает инкремент неделимым (глава 42).  
std::atomic<int> g_next_id(1);
```

Про счётчик стоит сказать отдельно, потому что в первой версии он был обычным `int`. Выглядело безобидно: подумаешь, увеличить число. Но `g_next_id++` — это чтение, прибавление и запись, три отдельных действия, и два потока могут прочитать одно и то же значение. Тогда два клиента получают имя `user_3`, а `user_4` не будет существовать вовсе.

Проявляется такое редко — нужно, чтобы двое подключились в одну и ту же микросекунду, — и именно поэтому опасно: тест не поймает, а в бою однажды случится. Исправление — `std::atomic<int>`, о котором подробно в следующей главе.

Три операции над списком, и каждая под замком:

```
void broadcast(int from_fd, const std::string& msg) {
    std::lock_guard<std::mutex> lock(g_mutex);
    for (const auto& c : g_clients) {
        if (c.fd == from_fd) continue;
        // MSG_NOSIGNAL — не получать SIGPIPE, если клиент уже отключился.
        ::send(c.fd, msg.data(), msg.size(), MSG_NOSIGNAL);
    }
}

void register_client(int fd, const std::string& name) {
    std::lock_guard<std::mutex> lock(g_mutex);
    g_clients.push_back({fd, name});
}

void unregister_client(int fd) {
    std::lock_guard<std::mutex> lock(g_mutex);
    g_clients.erase(
        std::remove_if(g_clients.begin(), g_clients.end(),
                      [fd](const Client& c) { return c.fd == fd; }),
        g_clients.end());
}
```

Удаление написано идиомой «удалить-стереть» из главы 14: `remove_if` сдвигает подходящие элементы в конец и возвращает границу, `erase` отрезает хвост. В C++20 появился `std::erase_if`, который делает то же самое одним вызовом.

Флаг `MSG_NOSIGNAL` в `send` — защита от `SIGPIPE`. Клиент мог отключиться секунду назад, его дескриптор ещё в списке, и запись в закрытое соединение по умолчанию убила бы весь сервер (глава 27). С флагом `send` просто вернёт ошибку, которую мы игнорируем: отключившийся клиент разберётся сам, когда его поток дойдёт до `unregister_client`.

42.5.1 Обработчик

```
void handle_client(int fd) {
    std::string name = "user_" + std::to_string(g_next_id.fetch_add(1));
    register_client(fd, name);

    std::string greet = "* " + name + " вошёл в чат\n";
```

```

broadcast(fd, greet);

std::string hello = "Привет, " + name + ". Пишите сообщения. /quit для выхода.\n";
::send(fd, hello.data(), hello.size(), MSG_NOSIGNAL);

char buf[1024];
std::string acc;
while (true) {
    ssize_t n = ::recv(fd, buf, sizeof(buf), 0);
    if (n == 0) break;
    if (n < 0) { if (errno == EINTR) continue; break; }
    acc.append(buf, static_cast<std::size_t>(n));

    // Разбираем сообщения по '\n'.
    std::size_t pos;
    while ((pos = acc.find('\n')) != std::string::npos) {
        std::string line = acc.substr(0, pos);
        acc.erase(0, pos + 1);

        if (line == "/quit") {
            goto disconnect;
        }

        std::string out = "<" + name + "> " + line + "\n";
        broadcast(fd, out);
    }
}

disconnect:
    unregister_client(fd);
    ::close(fd);
    std::string bye = "* " + name + " вышел из чата\n";
    broadcast(/*from_fd=*/-1, bye);    // не исключаем никого
}

```

Здесь впервые появляется накопитель — переменная `acc`, в которую складываются пришедшие байты. Он нужен ровно потому, что ТСП не хранит границ сообщений (глава 40): одно сообщение может прийти двумя кусками, а два — одним. Накопитель копит байты, а внутренний цикл вынимает из него все завершённые строки.

Разделителем служит перевод строки — самый простой из трёх способов разметки. Он и подводит первым: сообщение с переводом строки внутри разъедется на два. В главе 44 мы заменим разделитель на префикс длины.

Про `goto` в коде на C++ принято ругаться, и обычно за дело. Здесь он выпрыгивает из двух вложенных циклов сразу к общему завершению — то есть делает ровно то, для чего в языках без `goto` заводят флаг или выносят цикл в отдельную функцию. Читается такое честнее, чем `bool done = false;` с проверкой в двух местах. Если `goto` вас смущает, вынесите тело цикла в функцию, возвращающую признак завершения, — это упражнение в конце главы.

42.6 Проверка

Запускаем сервер и двух клиентов:

клиент 1:

```
> <- Привет, user_1. Пишите сообщения. /quit для выхода.
```

```
> <- * user_2 вошёл в чат
```

```
<user_2> я второй
```

```
* user_2 вышел из чата
```

клиент 2:

```
> <- Привет, user_2. Пишите сообщения. /quit для выхода.
```

Первый клиент видит, как второй входит, пишет и выходит, — сервер разослал ему все три события. Это уже настоящий чат, пусть и без имён по выбору пользователя.

Обратите внимание на асимметрию: клиент из главы 40 умеет только «отправил — жду ответ», поэтому сообщения от других он увидит лишь после того, как отправит что-то сам. Чтобы принимать сообщения в любой момент, клиенту тоже нужен второй поток: один читает клавиатуру, другой — сокет. Это упражнение, и оно полезное: там обнаружится, что вывод из двух потоков в один терминал тоже надо защищать.

42.7 Чем плоха модель «поток на клиента»

Она отлично работает на десятках соединений и разваливается на тысячах, причём по нескольким причинам сразу.

Память. Каждому потоку выделяется стек — по умолчанию восемь мегабайт виртуального пространства на Linux, около половины мегабайта на macOS. Физической памяти сначала тратится мало, но на десяти тысячах потоков одно только адресное пространство под стеки съест десятки гигабайт.

Переключение контекста. Планировщик ядра переключается между потоками, и каждое переключение стоит порядка микросекунды: сохранить регистры, сменить таблицы, испортить кэш процессора. Когда потоков тысячи, а работы у каждого мало, машина занята в основном переключениями.

Состязание за мьютекс. Наш `broadcast` держит замок на всё время рассылки. Пока один поток рассылает сообщение сотне клиентов, остальные девяносто девять ждут — и параллельность превращается в очередь.

Именно эти три причины и породили в конце девяностых так называемую проблему десяти тысяч соединений и, как ответ на неё, цикл событий. Им мы займёмся в главе 43.

Заметьте, впрочем, что двадцать лет спустя маятник качнулся обратно: сопрограммы в C++20, горутины в Go, виртуальные потоки в Java — всё это попытки вернуть удобство модели «поток на задачу», сделав сами потоки дешёвыми. Модель была хороша, дороги были потоки.

42.8 Что ещё есть в стандартной библиотеке

Кроме `std::thread` и `std::mutex` в C++11 появилось несколько вещей, о которых стоит знать.

`std::condition_variable` — способ ждать события, не крутя цикл проверок. Поток говорит «разбуди меня, когда в очереди появится работа», и спит, не занимая процессор. Пара «мьютекс плюс условная переменная» — классический способ построить очередь задач между потоками. Нам она не понадобилась, потому что наши потоки ждут не друг друга, а сокет.

`std::async` и `std::future` — запустить вычисление и получить результат позже. Удобно для «посчитай это в фоне, а я потом заберу», и совершенно не годится для долгоживущих обработчиков.

`std::recursive_mutex` — мьютекс, который можно закрыть дважды из одного потока. Звучит удобно, но почти всегда означает, что структура кода неудачна: функция под замком зовёт другую, которая тоже берёт замок. Лучше разделить на «публичную функцию с замком» и «внутреннюю без замка».

`std::shared_mutex` из C++17 — замок с двумя режимами: много читателей одновременно либо один писатель. Для данных, которые читают часто, а меняют редко, он снимает состязание. Наш список клиентов — как раз такой случай: рассылка только читает, а меняют список только при подключении и отключении.

42.9 Главные правила главы

1. Данные, к которым обращаются несколько потоков и хотя бы один пишет, обязаны быть защищены — исключений нет.
2. `std::thread` требует `join` или `detach` до разрушения объекта, иначе программа завершается.
3. Замок берут через `std::lock_guard`, а не вручную: при исключении ручной `unlock` не выполнится.
4. Инкремент обычного `int` — не одна операция; счётчик, общий для потоков, должен быть атомарным.
5. `MSG_NOSIGNAL` при отправке защищает сервер от смерти по `SIGPIPE`, если клиент уже отключился.
6. Накопитель для входящих байтов нужен всегда: ТСП не хранит границ сообщений.
7. Гонки ищут `ThreadSanitizer`, а не глазами; в отладочной сборке он должен быть включён.
8. Поток на соединение хорош до сотен клиентов; дальше упирается в память, переключения контекста и состязание за замки.

42.10 Упражнения

1. Соберите `demo-chat`, запустите многопоточный сервер и подключите к нему двух-трёх клиентов (можно через `nc`). Проверьте, что сообщения доходят всем, кроме отправителя, и что вход и выход объявляются.
2. Повторите опыт с гонкой: соберите пример с общим вектором без мьютекса и запустите несколько раз. Всегда ли падает? Затем соберите с `-fsanitize=thread` и прочитайте отчёт.
3. Верните `g_next_id` к обычному `int`, соберите сервер с `-fsanitize=thread` и подключите десяток клиентов одновременно (`for i in $(seq 10); do nc localhost 9001 & done`). Появился ли отчёт о гонке?
4. Уберите `MSG_NOSIGNAL` из `broadcast`. Подключите двух клиентов, одного убейте резко (`kill -9` на `nc`), второму отправьте сообщение. Что произойдёт с сервером?
5. Замените `goto disconnect` на явный флаг или на вынесенную функцию. Сравните читаемость обоих вариантов.
6. Сделайте команду `/nick <имя>`, меняющую имя клиента. Подумайте, где именно нужен замок и что будет, если два клиента одновременно возьмут одно имя.
7. (Сложнее) Напишите клиент с двумя потоками: один читает клавиатуру и отправляет, другой принимает и печатает. Обнаружьте, что вывод из двух потоков перемешивается, — защитите его мьютексом.

8. (Сложнее) Замените `std::mutex` на `std::shared_mutex` (C++17): рассылка берёт разделяемый замок, изменение списка — исключительный. Замерьте разницу на пятидесяти клиентах, которые активно пишут.

42.11 Что дальше

Глава 42 — атомарные операции. Мы уже наткнулись на них, спасая счётчик имён, но там всё обошлось одной строкой. Разберёмся, что такое атомарность на самом деле, почему `counter++` из двух потоков теряет половину прибавлений, чем атомарная переменная отличается от переменной под мьютексом и во сколько раз она быстрее. И заглянем в модель памяти C++ — самую сложную часть языка, о которой большинству достаточно знать одно правило.

Глава 43

Глава 42. Атомарные операции

В прошлой главе мы спасли счётчик имён одним словом — заменили `int` на `std::atomic<int>`. Работает, но объяснения это не заменяет: что именно происходит при `counter++` из двух потоков, почему атомарная переменная быстрее мьютекса и что такое пресловутая модель памяти, о которой все говорят с придыханием.

Разберёмся. Материала здесь на целую книгу, но практической пользы — на одну главу и одно правило, которого хватает в девяти случаях из десяти.

43.1 Почему ++ теряет половину

Возьмём самый простой возможный код: два потока, каждый увеличивает общий счётчик миллион раз.

```
int counter = 0;
std::thread a([&] { for (int i = 0; i < ITER; ++i) ++counter; });
std::thread b([&] { for (int i = 0; i < ITER; ++i) ++counter; });
a.join(); b.join();
```

Ожидаем два миллиона. Получаем:

Без защиты: counter=1032826 (ожидалось 2000000), 6.4 ms

Половина прибавлений исчезла. Не «немного разошлось», а именно половина.

Причина в том, что `++counter` — не одна операция, а три. Процессор читает значение из памяти в регистр, прибавляет единицу, записывает обратно. Между чтением и записью может вклиниться другой поток:

поток A: читает 100

поток В: читает 100 ← оба видят одно и то же
 поток А: пишет 101
 поток В: пишет 101 ← прибавление А потерялось

Два прибавления, результат один. Чем чаще потоки попадают в это окно, тем больше теряется — при плотном цикле почти всегда.

Обратите внимание и на время: небезопасная версия отработала за шесть миллисекунд. Запомним это число, оно понадобится для сравнения.

43.2 Мьютекс: правильно и дорого

Самое очевидное решение — замок из прошлой главы:

```
std::mutex m;
auto work = [&] {
    for (int i = 0; i < ITER; ++i) {
        std::lock_guard<std::mutex> lock(m);
        ++counter;
    }
};
```

Mutex: counter=2000000, 97.2 ms

Ответ верный, но время выросло в пятнадцать раз. И это на операции, которая сама по себе занимает наносекунду.

Дорого обходится не сам замок — в неоспариваемом случае он стоит десятки наносекунд. Дорого обходится состязание: когда второй поток приходит к закрытому замку, он засыпает, а это системный вызов, переключение контекста и последующее пробуждение. При миллионе прибавлений столкновения происходят постоянно.

43.3 Атомарная переменная

Мьютекс — универсальный инструмент: он защищает произвольный кусок кода любой длины. Но нам нужно защитить одну-единственную операцию, и для этого у процессора есть встроенная поддержка.

```
std::atomic<int> counter(0);
auto work = [&] { for (int i = 0; i < ITER; ++i) ++counter; };
```

atomic (seq_cst): counter=2000000, 42.2 ms

Верно и вдвое быстрее мьютекса. Причём заметьте: медленнее незащищённой версии всё равно в семь раз — атомарность не бесплатна, просто дешевле замка.

Что происходит внутри, видно в ассемблере. Соберём с оптимизацией и посмотрим, во что превратился `fetch_add`:

```
inc_seq:
    lock inc dword ptr [rip + a]
```

Одна инструкция с приставкой `lock`. Она заставляет процессор выполнить чтение, изменение и запись как единое целое: на время операции строка кэша захватывается в исключительное владение, и другие ядра не могут её тронуть. Никакого засыпания, никаких системных вызовов — только задержка на согласование кэшей между ядрами.

Отсюда и разница в цене. Мьютекс при столкновении уходит в ядро, атомарная операция остаётся в пользовательском коде.

43.3.1 Что умеет `std::atomic`

Кроме привычных `++` и `--` у атомарной переменной есть именованные операции:

```
counter.fetch_add(1);           // прибавить и вернуть прежнее значение
counter.fetch_sub(1);
counter.load();                 // прочитать
counter.store(42);              // записать
counter.exchange(99);           // записать и вернуть прежнее
```

И главная из них, на которой строится всё остальное:

```
int expected = 5;
if (counter.compare_exchange_strong(expected, 10)) {
    // было 5, стало 10
} else {
    // не было 5; в expected теперь лежит фактическое значение
}
```

«Сравнить и обменять» — атомарная проверка с условной записью. На ней строятся все алгоритмы без блокировок: прочитали значение, вычислили новое, попытались записать, если никто не вмешался; вмешались — повторили.

```
// Атомарно умножить на два — операции fetch_mul не существует.
int old = counter.load();
while (!counter.compare_exchange_weak(old, old * 2)) {
```

```
// old обновилось фактическим значением, пробуем снова  
}
```

Слабая версия `compare_exchange_weak` вправе иногда «промахнуться» без причины, зато на некоторых процессорах дешевле; поскольку она всё равно вызывается в цикле, ложный промах безвреден. Сильная версия нужна там, где цикла нет.

Важно понимать границу применимости: атомарной может быть одна переменная, но не связка из двух. Если нужно согласованно поменять счётчик и список, атомарные переменные не помогут — нужен мьютекс.

43.4 Модель памяти и одно правило

Теперь та часть, вокруг которой ходят легенды.

Проблема в том, что и компилятор, и процессор переставляют операции. Компилятор — при оптимизации, процессор — при выполнении, чтобы не простаивать в ожидании памяти. В однопоточной программе это незаметно: перестановка не меняет наблюдаемый результат. В многопоточной — меняет, потому что другой поток видит промежуточные состояния.

Классический пример:

```
// Поток A                // Поток B  
data = 42;                if (ready) {  
ready = true;              use(data); // а точно ли 42?  
                           }  
                           }
```

Ничто не мешает процессору сделать запись `ready` раньше записи `data` — они не связаны между собой. Поток B увидит поднятый флаг и мусор в данных.

Атомарные операции решают и это: у каждой из них есть параметр упорядочивания, который говорит, что можно, а что нельзя переставлять вокруг неё. Вариантов в C++11 шесть, и разбирать их все здесь ни к чему. Достаточно знать три.

`memory_order_seq_cst` — по умолчанию. Самый строгий: все атомарные операции всех потоков выстраиваются в единый глобальный порядок, и ничего вокруг них не переставляется. Именно поэтому пример выше починится сам, если `ready` сделать атомарным.

`memory_order_relaxed` — самый слабый: гарантируется только атомарность самой операции, никаких обещаний про порядок. Годится для счётчиков, которые ни с чем не синхронизируют, — например, для сбора статистики.

`memory_order_acquire` и `memory_order_release` — пара для передачи данных между потоками: писатель делает `store(..., release)`, читатель `load(..., acquire)`, и всё, что писатель сделал

до своей записи, гарантированно видно читателю **после** его чтения. Это дешевле полного порядка и достаточно почти всегда.

Правило, которого хватает большинству: **берите умолчание**. seq_cst рассуждается проще всего, ошибиться в нём нельзя, а разница в цене на практике редко заметна.

43.4.1 Насколько редко — измерим

Наш демо-стенд сравнивает seq_cst и relaxed на том же миллионе прибавлений:

```
atomic (seq_cst):  counter=2000000, 42.2 ms
```

```
atomic (relaxed):  counter=2000000, 44.2 ms
```

Разницы нет вовсе — расслабленная версия даже чуть медленнее, что укладывается в разброс измерений.

Объяснение — в ассемблере. Обе версии компилируются в одну и ту же инструкцию:

```
inc_seq:      lock inc dword ptr [rip + a]
```

```
inc_relaxed:  lock inc dword ptr [rip + a]
```

Дело в том, что у x86 сильная модель памяти: процессор сам почти ничего не переставляет, и большинство гарантий получается бесплатно. Разница между режимами проявляется на ARM, где модель слабая, — и там relaxed действительно дешевле.

Но не для всех операций. Посмотрим на запись:

```
store_seq:    xchg dword ptr [rip + a], edi    ; неявно атомарна и ставит барьер
```

```
store_rlx:    mov  dword ptr [rip + a], edi    ; обычная запись
```

Здесь разница есть даже на x86: строгая запись превращается в xchg, которая дороже обычного mov. А вот чтение в обоих режимах — обычный mov.

Вывод практический: чтобы рассуждать о цене упорядочивания, надо смотреть на конкретную операцию и конкретный процессор. И ещё раз — если вы не пишете структуру данных без блокировок, оставьте умолчание.

43.5 Ложное разделение

Есть у атомарных операций неочевидная ловушка, о которой стоит знать, потому что она бьёт по производительности незаметно.

Процессор работает с памятью не байтами, а строками кэша — обычно по 64 байта. Если две атомарные переменные попали в одну строку, то ядра, работающие с разными переменными,

всё равно дерутся за одну строку кэша. Это называют ложным разделением: логически данные независимы, физически — нет.

```
struct Counters {
    std::atomic<long> a;    // эти две переменные почти наверняка
    std::atomic<long> b;    // окажутся в одной строке кэша
};
```

Лечится выравниванием — тем самым `alignas` из главы 32:

```
struct Counters {
    alignas(64) std::atomic<long> a;
    alignas(64) std::atomic<long> b;
};
```

Структура станет вдвое больше, зато ядра перестанут мешать друг другу. В C++17 для этого появилась константа `std::hardware_destructive_interference_size` — размер строки кэша, узнанный у компилятора.

Замечать такое стоит только тогда, когда счётчики действительно горячие и обновляются миллионы раз в секунду. В остальных случаях выравнивание — преждевременная оптимизация.

43.6 Что выбрать

Короткая таблица, которой хватает для принятия решения:

Задача	Инструмент
Счётчик, флаг, одна переменная	<code>std::atomic</code>
Несколько связанных переменных	<code>std::mutex</code>
Структура данных (список, отображение)	<code>std::mutex</code>
Много читателей, редкие записи	<code>std::shared_mutex</code> (C++17)
Один пишет, один читает, порядок важен	атомарный флаг с <code>release/acquire</code>
Очередь задач между потоками	мьютекс плюс <code>condition_variable</code>

Отдельно стоит сказать про соблазн написать структуру данных без блокировок. Выглядит это заманчиво — очередь на атомарных операциях, никаких замков, — а на практике оказывается одной из самых сложных областей программирования. Там ждут проблема АВА (значение вернулось к прежнему, но объект уже другой), сложности с освобождением памяти

(когда безопасно удалить узел, на который может смотреть другой поток) и почти полная невозможность проверить корректность тестами.

Практический совет: начинайте с мьютекса. Если измерения покажут, что он мешает, — сначала уменьшайте область блокировки, потом смотрите в сторону `shared_mutex`, и только потом, с очень большой осторожностью, в сторону структур без блокировок. И берите готовую библиотеку, а не пишите свою.

43.7 В нашем сервере

Атомарная переменная у нас ровно одна:

```
// Счётчик имён. Его увеличивают потоки-обработчики, значит, обычный int  
// здесь — гонка данных: два клиента, подключившиеся одновременно, могли бы  
// получить одно имя. atomic делает инкремент неделимым (глава 42).  
std::atomic<int> g_next_id(1);  
...  
std::string name = "user_" + std::to_string(g_next_id.fetch_add(1));
```

Это ровно тот случай из таблицы: одна независимая переменная, никакой связи с другими данными. Мьютекс здесь был бы избыточен, а `fetch_add` возвращает прежнее значение — то самое, которое нам и нужно для имени.

Список клиентов, наоборот, защищён мьютексом, и правильно: изменение списка — это не одна операция над одной переменной, а перестройка структуры.

43.8 Главные правила главы

1. `counter++` — это чтение, прибавление и запись; из двух потоков теряется значительная часть прибавлений.
2. Атомарная операция выполняется процессором как единое целое — одной инструкцией с приставкой `lock` на x86.
3. Атомарная переменная вдвое быстрее мьютекса на счётчике, потому что не уходит в ядро при столкновении.
4. Атомарной может быть одна переменная, но не связка из нескольких — там нужен мьютекс.
5. `compare_exchange` — основа всех алгоритмов без блокировок: прочитал, вычислил, попытался записать, повторил при неудаче.
6. По умолчанию берите `seq_cst`; расслабленные режимы дают выигрыш не везде и требуют внимательности.

7. На x86 `relaxed` и `seq_cst` для прибавления компилируются в одну инструкцию, а для записи — в разные; проверять надо конкретную операцию.
8. Две горячие атомарные переменные в одной строке кэша мешают друг другу; лечится `alignas(64)`.

43.9 Упражнения

1. Запустите `./build/atomic_demo` несколько раз. Насколько отличается потерянная доля прибавлений от запуска к запуску? Объясните разброс.
2. Уменьшите число итераций до тысячи и запустите снова. Пропала ли гонка? Сделайте вывод о том, можно ли доверять отсутствию ошибки в тесте.
3. Посмотрите ассемблер сами: соберите маленький файл с `fetch_add` в двух режимах через `g++ -O2 -S -masm=intel` и найдите инструкции. Затем то же самое для `store` и `load`.
4. Замените в демо `++counter` на `counter.fetch_add(1, std::memory_order_relaxed)` и замерьте. Есть ли разница на вашей машине? А если запустить на телефоне или Raspberry Pi?
5. Реализуйте атомарное умножение на два через `compare_exchange_weak` в цикле. Проверьте на двух потоках, что результат совпадает с ожидаемым.
6. Соберите демо с `-fsanitize=thread` и убедитесь, что детектор находит гонку в первом варианте и молчит в трёх остальных.
7. (Сложнее) Воспроизведите ложное разделение: два потока увеличивают два счётчика в одной структуре, замерьте время; добавьте `alignas(64)` и замерьте снова. Какой выигрыш?
8. (Сложнее) Напишите очередь «один писатель, один читатель» на двух атомарных индексах и кольцевом буфере. Проверьте, что ни одно сообщение не теряется и не дублируется на миллионе передач.

43.10 Что дальше

Глава 43 — цикл событий. Мы построили сервер на потоках и знаем его пределы: память под стеки, переключения контекста, состязание за замки. Пойдём другим путём — один поток, который спрашивает у ядра, на каких сокетах что-то произошло, и обрабатывает только их. Познакомимся с `poll`, разберёмся, что такое неблокирующий режим и зачем каждому клиенту нужен собственный буфер отправки, и поймём, почему код при этом выворачивается наизнанку.

Глава 44

Глава 43. Цикл событий

Сервер на потоках работает, но мы знаем его пределы: стек на каждое соединение, переключения контекста, состязание за мьютекс при рассылке. На сотне клиентов это незаметно, на десяти тысячах — уже нет.

В этой главе построим тот же чат другим способом: один поток, который спрашивает у ядра «на каких сокетах что-то произошло» и обрабатывает только их. Такую схему называют циклом событий или реактором, и на ней работают `nginx`, `Redis`, `Node.js` и почти всё, что держит много соединений.

Заодно узнаем, что такое неблокирующий режим, зачем каждому клиенту собственный буфер отправки и почему при переходе на события код выворачивается наизнанку.

44.1 Почему потоков не хватает

Сравним два наших сервера на ста подключённых клиентах — просто посмотрим, что показывает система:

```
сервер на потоках:  потоков 102, память 1.7 МБ  
реактор:           потоков 2,   память 1.0 МБ
```

Сто два потока против двух. Память пока отличается несильно, потому что стеки выделены виртуально и почти не тронуты, — но на десяти тысячах клиентов разница станет решающей: адресное пространство под стеки, память ядра под структуры потоков, а главное — планировщик, который вынужден раскладывать десять тысяч потоков по восьми ядрам.

Ещё одна причина, менее очевидная: в чате потоки почти всё время спят. Клиент пишет сообщение раз в минуту, а поток занят соединением постоянно. Мы платим за поток, а работы у него на доли процента.

Идея цикла событий отсюда и растёт: раз потоки всё равно ждут, пусть ждёт один — сразу за всех.

44.2 poll

Системный вызов, который умеет ждать сразу на многих дескрипторах, называется `poll`:

```
#include <poll.h>

struct pollfd {
    int    fd;           // за каким дескриптором следим
    short  events;       // чего ждём: POLLIN, POLLOUT
    short  revents;      // что произошло — заполняет ядро
};

int poll(struct pollfd fds[], nfds_t nfds, int timeout);
```

Программа готовит массив: для каждого сокета — номер и набор интересующих событий. `poll` засыпает, пока хоть на одном что-нибудь не случится, и возвращает количество готовых дескрипторов, проставив в `revents` то, что произошло.

Флаги, которые нам нужны: `POLLIN` — есть данные для чтения (или соединение закрыто), `POLLOUT` — можно писать без блокировки, `POLLHUP` — собеседник отключился, `POLLERR` — ошибка на сокете. Первые два ставим мы, вторые два ядро выставляет само, о них просить не нужно.

Тайм-аут — в миллисекундах; `-1` означает «ждать бесконечно», `0` — «посмотреть и сразу вернуться».

Есть три родственника, и разница между ними важна для больших нагрузок. `select` — самый старый, ограничен 1024 дескрипторами и неудобен. `poll` — без ограничения на число, но каждый вызов передаёт ядру весь массив целиком, а ядро проходит по нему линейно; при десяти тысячах сокетов это десять тысяч проверок на каждое событие. `epoll` на Linux и `kqueue` на macOS и BSD решают эту проблему: список отслеживаемых дескрипторов хранится в ядре и меняется отдельными вызовами, а при пробуждении отдаются только готовые.

Мы берём `poll`, потому что он работает и на Linux, и на macOS, а идея у всех трёх одна. Переход на `epoll` в упражнениях.

44.3 Неблокирующий режим

Цикл событий работает только с неблокирующими сокетами, и это принципиально.

Представьте: poll сказал, что на сокете есть данные, мы делаем recv и читаем всё, что пришло. Данных было мало, а буфер большой — и recv в блокирующем режиме уснёт, ожидая продолжения. Один клиент остановит весь сервер.

Перевод в неблокирующий режим:

```
void set_nonblocking(int fd) {
    int flags = ::fcntl(fd, F_GETFL, 0);
    ::fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}
```

Сначала читаем текущие флаги, потом добавляем свой — именно добавляем, а не назначаем. Написать fcntl(fd, F_SETFL, O_NONBLOCK) — значит стереть все остальные флаги, и это распространённая ошибка.

После этого поведение вызовов меняется: если данных нет, recv не ждёт, а сразу возвращает -1 с errno равным EAGAIN (то же самое, что EWOULDBLOCK — два имени одного значения). Аналогично send при переполненном буфере отправки и accept при отсутствии ожидающих соединений.

Отсюда правило: **EAGAIN — не ошибка, а сообщение «пока ничего нет»**. Оно повторяется в коде постоянно:

```
ssize_t r = ::recv(fd, buf, sizeof(buf), 0);
if (r < 0) {
    if (errno == EAGAIN || errno == EWOULDBLOCK) break; // всё прочитали
    if (errno == EINTR) continue; // сигнал, повторим
    gone = true; break; // настоящая ошибка
}
```

Три ветки на каждый вызов. Многословно, но каждая нужна.

44.4 Состояние клиента

В потоковом сервере состояние разговора хранилось в локальных переменных обработчика: имя лежало в name, накопитель — в acc, а место, где мы находимся, — в счётчике команд. Всё это исчезает, когда обработчиков больше нет.

Значит, состояние надо хранить явно:

```
struct ClientState {
    int fd;
    std::string name;
    std::string recv_buf; // буфер для частичных recv
}
```

```
std::string send_buf;    // что мы хотим послать (пока не отправлено)
};

std::unordered_map<int, ClientState> clients;
```

Отображение из дескриптора в состояние — типичная для реакторов структура: событие приносит номер дескриптора, по нему находим, с кем имеем дело.

Буфер приёма нужен по той же причине, что и раньше: сообщение может прийти по частям.

А вот буфер отправки — новинка, и он появляется именно из-за неблокирующего режима. Раньше мы просто звали `send` и ждали, пока он отправит. Теперь `send` может отправить половину и вернуть `EAGAIN` — буфер сокета переполнен, клиент не успевает читать. Блокироваться нельзя, значит, остаток надо сохранить и попробовать позже:

```
void broadcast(std::unordered_map<int, ClientState>& clients,
              int from_fd, const std::string& msg) {
    for (auto& kv : clients) {
        if (kv.first == from_fd) continue;
        kv.second.send_buf += msg;
    }
}
```

Обратите внимание, что рассылка теперь **ничего не отправляет**. Она только раскладывает сообщение по буферам, а отправкой займётся цикл, когда ядро скажет, что сокет готов принимать. Это и есть переворот, о котором я говорил: код перестаёт делать вещи и начинает планировать их.

44.5 Цикл

```
while (true) {
    // Собираем массив pollfd на эту итерацию.
    std::vector<pollfd> pfd;
    pfd.reserve(clients.size() + 1);
    pfd.push_back({srv, POLLIN, 0});
    for (const auto& kv : clients) {
        short events = POLLIN;
        if (!kv.second.send_buf.empty()) events |= POLLOUT;
        pfd.push_back({kv.first, events, 0});
    }
}
```

```

int n = ::poll(pfds.data(), pfds.size(), -1 /*ms, infinite*/);
if (n < 0) {
    if (errno == EINTR) continue;
    std::cerr << "poll: " << std::strerror(errno) << "\n";
    break;
}
...
}

```

Массив собирается заново на каждой итерации — расточительно, но наглядно. В настоящем сервере его держат постоянным и правят точно, а с `epoll` вопрос отпадает сам: список живёт в ядре.

Ключевая строка — условие про `POLLOUT`. Просить готовность к записи надо **только тогда, когда есть что писать**. Иначе `poll` будет просыпаться непрерывно: свободное место в буфере отправки есть почти всегда, и цикл превратится в бесконечную прокрутку с полной загрузкой ядра. Это классическая ошибка новичков в реакторах, и её легко заметить по стопроцентной загрузке процессора на пустом сервере.

44.5.1 Новые соединения

```

if (pfds[0].revents & POLLIN) {
    while (true) {
        int c = ::accept(srv, nullptr, nullptr);
        if (c < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            if (errno == EINTR) continue;
            break;
        }
        set_nonblocking(c);

        ClientState st;
        st.fd = c;
        st.name = "user_" + std::to_string(next_id++);
        clients.emplace(c, std::move(st));
        broadcast(clients, c, "* " + clients[c].name + " вошёл\n");
        clients[c].send_buf += "Привет, " + clients[c].name + "\n";
    }
}

```

```

    }
}

```

Принимаем в цикле, пока ассерт не скажет EAGAIN. Причина в том, что за одно пробуждение могло прийти несколько соединений, а poll сообщает лишь факт готовности, не количество. Приняв одно и вернувшись в poll, мы бы разобрали очередь по одному за круг — работало бы, но медленнее.

Слушающий сокет тоже переведён в неблокирующий режим — иначе ассерт в этом цикле однажды заснёт.

Заметьте, что счётчик next_id здесь — обычный int, и никакой гонки нет: поток один. Это первая приятная особенность реактора — исчезает целый класс ошибок.

44.5.2 Чтение

```

if (re & POLLIN) {
    char buf[1024];
    bool gone = false;
    while (true) {
        ssize_t r = ::recv(fd, buf, sizeof(buf), 0);
        if (r == 0) { gone = true; break; }
        if (r < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            if (errno == EINTR) continue;
            gone = true; break;
        }
        s.recv_buf.append(buf, static_cast<std::size_t>(r));
    }
    // Парсим сообщения по \n
    std::size_t pos;
    while ((pos = s.recv_buf.find('\n')) != std::string::npos) {
        std::string line = s.recv_buf.substr(0, pos);
        s.recv_buf.erase(0, pos + 1);
        if (line == "/quit") { gone = true; break; }
        std::string out = "<" + s.name + "> " + line + "\n";
        broadcast(clients, fd, out);
    }
    if (gone) {
        std::string bye = "* " + s.name + " вышел\n";

```



```

        ::close(fd);
        clients.erase(it);
        broadcast(clients, -1, bye);
        continue;
    }
}

```

Читаем тоже в цикле до EAGAIN — по той же причине, что и принимаем.

Ноль от `recv` означает, что клиент закрыл соединение. Обратите внимание на порядок при отключении: сначала запоминаем прощальное сообщение в локальную переменную, потом закрываем сокет, удаляем клиента и только затем рассылаем. Если бы мы рассылали до удаления, ушедший клиент получил бы собственное прощание в буфер отправки, которого уже не существует.

44.5.3 Запись и ловушка с висящей ссылкой

```

// Можно писать. Осторожно: если отправка сорвалась, клиента надо
// удалить — и после этого ссылка s указывает в никуда, поэтому
// выходим сразу, а не продолжаем работать с ней.
if ((re & POLLOUT) && !s.send_buf.empty()) {
    bool dropped = false;
    while (!s.send_buf.empty()) {
        ssize_t w = ::send(fd, s.send_buf.data(), s.send_buf.size(), MSG_NOSIGNAL);
        if (w < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            if (errno == EINTR) continue;
            std::cout << "[server] " << s.name << ": send: "
                      << std::strerror(errno) << "\n";
            ::close(fd);
            clients.erase(it);
            dropped = true;
            break;
        }
        s.send_buf.erase(0, static_cast<std::size_t>(w));
    }
    if (dropped) continue;
}

```

```
// Хост-отключение/ошибка
if (re & (POLLHUP | POLLERR | POLLNVAL)) {
    std::cout << "[server] " << s.name << " (hup/err)\n";
    ::close(fd);
    clients.erase(it);
}
```

Здесь была настоящая ошибка, и стоит рассказать о ней подробно, потому что она характерна для такого кода.

В первой версии после неудачной отправки клиент удалялся, но цикл не прерывался — а условие цикла читает `s.send_buf`, где `s` — ссылка на удалённый элемент. Обращение к освобождённой памяти в чистом виде. Хуже того, дальше управление доходило до проверки `POLLHUP`, которая печатала `s.name`, — второе обращение к тому же трупу.

Ошибка не проявлялась в обычной работе: неудачная отправка случается редко, и надо было специально ронять клиента посреди рассылки. Именно такие ошибки — редкие, зависящие от времени, ведущие к порче памяти — и делают серверный код сложным.

Лечится флагом `dropped` и немедленным выходом. Более надёжный подход — не удалять клиентов сразу, а пометать их и вычищать в конце итерации цикла: тогда никакие ссылки не протухают в середине обработки. Так делают в настоящих реакторах, и это упражнение в конце главы.

Флаг `MSG_NOSIGNAL` в `send` — та же защита от `SIGPIPE`, что в главе 41. На `macOS`, кстати, этого флага нет (там вместо него настройка сокета `SO_NOSIGPIPE`), но заголовки его определяют, и код собирается.

44.6 Проверка

клиент 1:

```
> <- Привет, user_1
> <- * user_2 вошёл
<user_2> второй тут
```

клиент 2:

```
> <- Привет, user_2
```

Поведение неотличимо от потокового сервера — а внутри всё устроено иначе.

44.7 Чем приходится платить

Реактор решает проблему масштаба, но забирает взамен несколько вещей.

Код становится сложнее. Сравните два обработчика: в потоковой версии логика читается сверху вниз — поздоровались, читаем, разбираем, отвечаем. В реакторе она разорвана на куски, между которыми возврат в цикл, а связь между кусками держится на структуре состояния. Это та же разница, что между обычным кодом и цепочкой обратных вызовов.

Одна долгая операция останавливает всех. Если в обработке сообщения окажется запрос к базе данных на сто миллисекунд, все клиенты будут ждать эти сто миллисекунд. В потоковой версии ждал бы только один. Отсюда правило реакторов: **в цикле событий не должно быть ничего долгого** — ни синхронного ввода-вывода, ни тяжёлых вычислений, ни ожидания замков.

Не используются ядра. Один поток — одно ядро, как бы ни была мощна машина. Промышленное решение — несколько реакторов, по одному на ядро, каждый со своим набором соединений: так устроены nginx с его рабочими процессами и Node.js с модулем кластера.

44.8 Как это выглядит у взрослых

Наш цикл — учебный, но идея в промышленных серверах ровно та же, отличается техника.

Вместо poll берут epoll (Linux) или kqueue (BSD, macOS): список наблюдаемых дескрипторов живёт в ядре, а при пробуждении отдаются только готовые. Разница видна на тысячах соединений, где poll тратит время на обход всего массива.

Реакторов заводят несколько — по числу ядер, и распределяют соединения между ними. Иногда с помощью SO_REUSEPORT, о котором шла речь в главе 40: несколько процессов слушают один порт, а ядро само раскладывает подключения.

Тяжёлую работу выносят в отдельный пул потоков: реактор принимает запрос, отдаёт его рабочему потоку и возвращается к событиям, а результат отправляет, когда работа закончится.

И почти всегда берут готовую библиотеку — libuv (на ней стоит Node.js), Boost.Asio, libevent. Писать цикл событий руками имеет смысл ровно один раз — чтобы понять, как он устроен.

Стоит упомянуть и направление, в котором всё это движется. В Linux с 2019 года есть io_uring — интерфейс, где программа и ядро общаются через две кольцевые очереди в разделяемой памяти, почти без системных вызовов. А в C++20 появились сопрограммы, позволяющие писать асинхронный код в привычном линейном виде: co_await вместо разрыва на обратные вызовы. Об обоих — коротко в главе 51.

44.9 Главные правила главы

1. Цикл событий заменяет много спящих потоков одним, который ждёт сразу за всех.
2. Реактор работает только с неблокирующими сокетами; `EAGAIN` в нём означает «пока ничего нет», а не ошибку.
3. Флаги дескриптора меняют чтением и добавлением бита, а не назначением — иначе сотрутся остальные.
4. Состояние разговора приходится хранить явно, потому что стека обработчика больше нет.
5. Буфер отправки у каждого клиента обязателен: `send` может отправить часть и вернуть `EAGAIN`.
6. `POLLOUT` просят только при непустом буфере отправки, иначе цикл крутится вхолостую на полной загрузке.
7. Принимать и читать надо в цикле до `EAGAIN`: событие говорит о готовности, но не о количестве.
8. Удаление клиента посреди обработки оставляет висящую ссылку — либо выходите сразу, либо откладываете удаление на конец итерации.

44.10 Упражнения

1. Запустите оба сервера и подключите к каждому по сотне клиентов (`for i in $(seq 100); do sleep 30 | nc localhost PORT & done`). Сравните число потоков (`ps -M PID`) и потребление памяти.
2. Уберите условие `if (!kv.second.send_buf.empty())` — просите `POLLOUT` всегда. Запустите сервер с одним клиентом и посмотрите загрузку процессора через `top`. Объясните результат.
3. Уберите `set_nonblocking(c)` для клиентских сокетов. Подключите двух клиентов и заставьте одного молчать посреди отправки. Что произойдёт со вторым?
4. Замените в `set_nonblocking` добавление флага на назначение (`F_SETFL, O_NONBLOCK`). Проверьте, что сокет всё ещё работает, и объясните, какие флаги вы при этом потеряли.
5. Переделайте удаление клиентов на отложенное: собирайте дескрипторы на удаление в вектор и вычищайте в конце итерации. Убедитесь, что ссылки больше не протухают.
6. Добавьте тайм-аут в `poll` (например, тысячу миллисекунд) и раз в секунду отключайте клиентов, молчавших дольше минуты. Понадобится хранить время последней активности в состоянии клиента.

7. (Сложнее) Перепишите цикл на `epoll` (Linux) или `kqueue` (macOS). Сравните код: что стало проще, что сложнее. Замерьте разницу на тысяче соединений.
8. (Сложнее) Сделайте несколько реакторов: главный поток принимает соединения и раздаёт их по очереди рабочим потокам, у каждого из которых свой цикл `poll`. Продумайте, как передать дескриптор в чужой цикл, не разбудив его насильно.

44.11 Что дальше

Глава 44 — свой двоичный протокол. Мы до сих пор разделяем сообщения переводом строки, и это работает ровно до тех пор, пока в сообщении нет перевода строки. Заменяем разделитель на префикс длины, напишем кодирование и разбор с накопителем, добавим версию протокола и ограничение на размер — и проверим всё это на нарезанном по байтам потоке, потому что именно так и приходят данные в реальной сети.

Глава 45

Глава 44. Свой двоичный протокол

Наш чат до сих пор разделяет сообщения переводом строки. Работает — ровно до тех пор, пока в сообщении нет перевода строки. Стоит пользователю вставить многострочный текст, и одно сообщение превратится в несколько.

Это не единственная беда текстового разделителя. Отправить двоичные данные — картинку, файл — через него нельзя вовсе: в них встретится любой байт, включая разделитель. Нет и способа отличить одно действие от другого: «привет, я Аня» и «войти в комнату общая» — просто строки, и серверу приходится гадать по содержимому.

В этой главе напишем настоящий протокол: с префиксом длины, версией, типом сообщения и ограничением на размер. И проверим его на самом злом входе, какой бывает в сети, — потоке, нарезанном по одному байту.

45.1 Три способа разметить границы

ТСР, как мы выяснили в главе 40, границ сообщений не хранит. Способов расставить их самому ровно три.

Разделитель — то, что у нас сейчас: сообщения отделяются символом, который в данных не встречается. Так устроены HTTP, SMTP, IRC и вообще большинство текстовых протоколов. Плюсы — читаемость, отладка через telnet, простота. Минусы — разделитель нельзя использовать в данных, а если очень надо, приходится экранировать, и разбор усложняется.

Фиксированный размер — все сообщения одинаковой длины. Просто и негибко; годится для телеметрии, где записи однотипны.

Префикс длины — сначала число, потом столько байт данных. Читатель узнаёт размер заранее, может выделить память одним куском и не разбирать данные посимвольно. Любые

байты допустимы. Так устроены Protocol Buffers, кадры HTTP/2, сообщения Redis, наш формат журнала из главы 32.

Для чата берём третий: сообщения могут содержать что угодно, а разбор должен быть быстрым.

45.2 Формат кадра

Формат фрейма:

```
[0..3]  u32 LE length (= 1 + 1 + payload.size(), т.е. size after length)
[4]      u8 version
[5]      u8 type
[6..]    payload (length-2 байт)
```

Шесть байт заголовка, дальше данные. Каждое поле заслуживает объяснения.

Длина идёт первой и считает всё, что следует за ней. Такой выбор — вопрос вкуса; главное, чтобы обе стороны считали одинаково. Четыре байта хватило бы на четыре гигабайта, но мы ограничим размер отдельно.

Версия протокола позволяет менять формат в будущем и не ломать старых клиентов сразу. Проверка версии стоит одно сравнение, а спасает от попыток разобрать чужие данные как свои — та же мысль, что с меткой формата в главе 32.

Тип сообщения превращает поток данных в набор осмысленных действий:

```
enum MsgType : std::uint8_t {
    kHello    = 1,    // payload = nickname (utf-8)
    kMessage  = 2,    // payload = текст сообщения
    kNotify   = 3,    // payload = серверное уведомление (вошёл/вышел и т.д.)
    kJoin     = 4,    // payload = имя комнаты
    kLeave     = 5,    // payload = имя комнаты
};

constexpr std::uint8_t  PROTO_VERSION = 1;
constexpr std::size_t   HEADER_BYTES = 4 + 1 + 1;
constexpr std::uint32_t MAX_PAYLOAD = 64 * 1024;    // 64 KiB — здравый лимит
```

Указание : std::uint8_t после имени перечисления задаёт его базовый тип — гарантирует, что значение занимает ровно байт. Без него компилятор вправе выбрать int, и запись в кадр пришлось бы приводить вручную.

Предел на размер данных — не формальность, а защита. Без него клиент, приславший в поле длины четыре миллиарда, заставил бы сервер попытаться выделить четыре гигабайта. Это готовый способ уронить сервер одним пакетом, и такие уязвимости находят в настоящем софте регулярно.

45.3 Кодирование

```
std::vector<std::uint8_t> encode(std::uint8_t type, const std::string& payload) {
    if (payload.size() > MAX_PAYLOAD) {
        throw std::runtime_error("payload too large");
    }
    std::vector<std::uint8_t> out;
    // length = version(1) + type(1) + payload(N)
    std::uint32_t len = static_cast<std::uint32_t>(2 + payload.size());
    out.reserve(4 + len);
    write_u32_le(out, len);
    out.push_back(PROTO_VERSION);
    out.push_back(type);
    out.insert(out.end(), payload.begin(), payload.end());
    return out;
}
```

Проверка размера стоит первой — до всякой работы. `reserve` избавляет от перевыделений при сборке кадра: размер известен заранее, зачем растить вектор по частям.

Запись числа — та же функция, что в главе 32, младшим байтом вперёд:

```
void write_u32_le(std::vector<std::uint8_t>& out, std::uint32_t v) {
    for (int i = 0; i < 4; ++i) {
        out.push_back(static_cast<std::uint8_t>((v >> (i * 8)) & 0xFF));
    }
}
```

Здесь можно было бы взять сетевой порядок через `htonl` — так принято в старых протоколах. Мы выбрали `little-endian`, потому что на всех машинах, где будет работать наш чат, это родной порядок процессора, а преобразование всё равно делается арифметикой и от платформы не зависит. Главное — записать выбор в документацию протокола, что мы и сделали комментарием в заголовке.

Посмотрим на результат:

Hello 'Alice' (11 байт):

07 00 00 00 01 01 41 6c 69 63 65

Message 'Привет всем!' (28 байт):

18 00 00 00 01 02 d0 9f d1 80 d0 b8 d0 b2 d0 b5

d1 82 20 d0 b2 d1 81 d0 b5 d0 bc 21

Первый кадр: длина 7 (версия, тип и пять букв имени), версия 1, тип 1, дальше Alice в ASCII. Второй: длина 24, тип 2, дальше русский текст в UTF-8 — по два байта на букву, как мы разбирали в главе 31.

Обратите внимание, что кодировка нас вообще не касается. Для протокола данные — это байты; что они значат, договариваются приложения. Мы условились, что имена и тексты в UTF-8, и записали это в комментарии рядом с типами.

45.4 Разбор

Вот здесь начинается настоящая работа. Кодирование — это сборка одного кадра, а разбор должен справляться с потоком, который приходит произвольными кусками.

```
bool try_decode(std::vector<std::uint8_t>& acc,
               std::uint8_t& out_type,
               std::string& out_payload) {
    if (acc.size() < 4) return false;
    std::uint32_t len = read_u32_le(acc.data());
    if (len < 2 || len > MAX_PAYLOAD + 2) {
        throw std::runtime_error("invalid frame length: " + std::to_string(len));
    }
    if (acc.size() < 4 + len) return false;

    std::uint8_t version = acc[4];
    if (version != PROTO_VERSION) {
        throw std::runtime_error("unsupported version: " + std::to_string(version));
    }
    out_type = acc[5];
    out_payload.assign(reinterpret_cast<const char*>(acc.data() + 6), len - 2);

    acc.erase(acc.begin(), acc.begin() + 4 + len);
    return true;
}
```

Функция работает с накопителем — тем самым `recv_buf` из состояния клиента. Возвращает `true`, если удалось вынуть кадр, `false`, если данных пока мало, и бросает исключение, если данные испорчены. Три исхода, и каждый обрабатывается по-своему.

Порядок проверок здесь не случаен, и переставлять их нельзя.

Сначала смотрим, пришёл ли заголовок целиком: без четырёх байт длину не прочесть. Если нет — просто ждём, никакой ошибки.

Потом проверяем саму длину — **до** того, как ею воспользоваться. Слишком маленькая означает кадр без версии и типа, слишком большая — либо мусор, либо злой умысел. Именно эта проверка защищает от попытки выделить гигабайты.

И только затем смотрим, пришло ли тело. Тут снова возможен честный ответ «пока нет» — с исключением его путать нельзя.

Разница между «данных мало» и «данные испорчены» — ключевая мысль всего разбора потоковых протоколов. Первое случается постоянно и лечится ожиданием, второе означает, что соединение испорчено и его надо закрывать. Свалить их в один ответ — верный способ либо закрывать исправные соединения, либо вечно ждать данных, которых не будет.

Последняя строка удаляет разобранный кадр из накопителя. Тут прячется неэффективность: `erase` с начала вектора сдвигает весь остаток. При потоке из тысяч мелких сообщений это заметно, и настоящие реализации держат вместо вектора кольцевой буфер или пару индексов «начало данных, конец данных». Для учебного чата сдвиг допустим, а в упражнениях есть задание его убрать.

45.5 Проверка на злом входе

Кадр, разрезанный по байтам, — не выдумка: так бывает при плохой сети, при работе через прокси, при отправке маленькими порциями. Проверим, что разбор это переживает.

```
std::vector<std::uint8_t> acc;
std::vector<std::string> got;
for (std::size_t i = 0; i < wire.size(); ++i) {
    acc.push_back(wire[i]);           // добавляем ровно один байт
    while (chat::try_decode(acc, type, payload)) {
        got.push_back(payload);
    }
}
check_eq(got.size(), 3, "по одному байту собрались все три фрейма");
```

Три кадра, нарезанные на отдельные байты, собираются полностью. Работает и обратный случай — когда несколько кадров приходят одним куском:

```
std::vector<std::uint8_t> acc = chat::encode(chat::kHello, "Ann");
std::vector<std::uint8_t> second = chat::encode(chat::kMessage, "два");
acc.insert(acc.end(), second.begin(), second.end());

int n = 0;
while (chat::try_decode(acc, type, payload)) ++n;
check_eq(n, 2, "из склеенного куска разобраны оба фрейма");
```

Обратите внимание на `while` в обоих случаях. Разбирать надо **в цикле**, пока кадры извлекаются: за один `recv` могло прийти сколько угодно сообщений. Написать `if` вместо `while` — распространённая ошибка, из-за которой сообщения начинают приходить с задержкой в одно следующее сообщение, а при паузе в разговоре застревают навсегда.

Проверяются и все три вида порчи: чужая версия, абсурдная длина, слишком короткий кадр. Каждый должен давать исключение, а не тихое непонятное поведение:

```
$ ./build/tests/test_protocol
все проверки прошли: 25/25
```

Протокол — идеальный кандидат для тестов: чистые функции, никакой сети, всё воспроизводимо. Отлаживать разбор на живом соединении — мучение, а на тестах он пишется за полчаса.

45.6 Как это встраивается в сервер

В состоянии клиента накопитель меняет тип со строки на вектор байтов — данные теперь двоичные:

```
std::vector<std::uint8_t> recv_buf;
std::vector<std::uint8_t> send_buf;
```

Чтение превращается в «добавить в накопитель и разобрать всё, что собралось»:

```
s.recv_buf.insert(s.recv_buf.end(), buf, buf + r);

std::uint8_t type;
std::string payload;
try {
    while (chat::try_decode(s.recv_buf, type, payload)) {
        process_message(s, type, payload);
    }
}
```

```
} catch (const std::runtime_error& e) {  
    // Испорченный кадр — закрываем соединение.  
    drop_client(s.fd);  
}
```

Обработка сообщения теперь начинается с выбора по типу, а не с разбора текста:

```
switch (type) {  
    case chat::kHello:    /* установить имя */      break;  
    case chat::kJoin:     /* войти в комнату */     break;  
    case chat::kMessage: /* разослать в комнату */ break;  
    ...  
}
```

Разница с текстовым протоколом принципиальная: раньше сервер разбирал строку и гадал, что она значит; теперь намерение клиента закодировано явно. Это и есть главная польза типизированного протокола — не экономия байтов, а исчезновение двусмысленности.

Отправка тоже упрощается: вместо склейки строк — кодирование кадра в буфер отправки.

```
std::vector<std::uint8_t> frame = chat::encode(chat::kNotify, "вы вошли в комнату");  
s.send_buf.insert(s.send_buf.end(), frame.begin(), frame.end());
```

Полностью на новый протокол сервер переходит в главе 45, где появятся комнаты.

45.7 Плата за двоичный формат

Главная потеря — отладка. К текстовому протоколу можно подключиться `telnet` и поговорить руками; к двоичному — нет. Нужен свой клиент, а на первых порах — печать кадров в шестнадцатеричном виде, как в нашей демонстрационной программе.

Именно поэтому многие протоколы остаются текстовыми даже там, где двоичный был бы эффективнее: HTTP/1.1 текстовый, и вся его экосистема — `curl`, логи, прокси — построена на том, что запрос можно прочитать глазами. Только когда цена стала слишком высокой, появился двоичный HTTP/2 — и вместе с ним отдельные инструменты для разглядывания трафика.

Вторая потеря — жёсткость. Добавить поле в текстовый протокол легко, старые клиенты просто не заметят лишнюю строку. В двоичном формате с фиксированным заголовком любое изменение — новая версия. Поэтому в серьёзных протоколах поля описываются самоописывающимся способом, как в Protocol Buffers, где у каждого поля есть номер и тип, а незнакомые поля пропускаются.

45.8 Чего нет в нашем протоколе

Контрольной суммы. В журнале базы данных она была нужна, потому что диск может отдать испорченные данные, а здесь за целостность отвечает TCP: у него своя проверка на каждый сегмент. Добавлять свою имеет смысл, если данные проходят через недоверенные преобразования — например, шифрование, — но об этом в главе 46.

Сжатия. Текст сжимается втрое-вчетверо, и на большом трафике это заметно. Обычно сжимают не отдельные кадры, а поток целиком, чтобы словарь накапливался.

Идентификаторов запросов. Если бы клиент мог послать несколько запросов и ждать ответов не по порядку, каждому кадру понадобился бы номер, а ответу — ссылка на него. Так устроен HTTP/2 с его потоками.

Согласования версии. Мы просто отвергаем чужую версию; правильнее было бы обмениваться поддерживаемыми версиями при подключении и выбрать общую. Так делают TLS и почти все протоколы, которым нужна долгая совместимость.

45.9 Главные правила главы

1. Поверх TCP границы сообщений размечает протокол: разделителем, фиксированным размером или префиксом длины.
2. Префикс длины позволяет читать двоичные данные и заранее знать размер, но лишает возможности отлаживать протокол вручную.
3. В заголовке должны быть версия и тип: первая спасает от чужих данных, второй убирает двусмысленность намерений.
4. Предел на размер данных обязателен и проверяется до выделения памяти.
5. Разбор различает три исхода: кадр готов, данных пока мало, данные испорчены — и путать второе с третьим нельзя.
6. Разбирать надо в цикле, пока кадры извлекаются: за одно чтение может прийти несколько сообщений.
7. Разбор потокового протокола проверяется на потоке, нарезанном по одному байту, — это худший и вполне реальный случай.
8. Удаление разобранного кадра сдвигом всего буфера работает, но на потоке мелких сообщений становится узким местом.

45.10 Упражнения

1. Соберите `demo-chat`, запустите `./build/protocol_demo` и разберите вывод: найдите в шестнадцатеричном дампе длину, версию, тип и данные каждого кадра.

2. Запустите тесты (`make tests && ./build/tests/test_protocol`). Затем сломайте что-нибудь намеренно — например, уберите проверку версии — и убедитесь, что тест это заметил.
3. Замените `while` на `if` в цикле разбора внутри демонстрационной программы. Какие сообщения перестали доходить и почему?
4. Уберите проверку `len > MAX_PAYLOAD + 2` и отправьте серверу кадр с длиной `0xFFFFFFFF` (можно через `printf` и `nc`). Что произойдёт с сервером?
5. Добавьте тип сообщения `kPing` и `kPong`: клиент раз в тридцать секунд шлёт `kPing`, сервер отвечает `kPong`. Это стандартный способ обнаружить мёртвое соединение.
6. Замените в накопителе `std::vector` с `erase` на пару индексов «начало данных, конец данных» с периодическим уплотнением. Замерьте разницу на потоке из ста тысяч мелких сообщений.
7. (Сложнее) Добавьте согласование версии: клиент присылает список поддерживаемых версий, сервер выбирает максимальную общую и отвечает выбором. Что делать, если общей нет?
8. (Сложнее) Напишите утилиту-снифер: она подключается к серверу как клиент, но вместо разговора печатает все входящие кадры в человекочитаемом виде — тип, длина, данные. Такой инструмент окупается на первой же отладке.

45.11 Что дальше

Глава 45 — комнаты. Соединим протокол из этой главы с циклом событий из предыдущей и получим чат, где сообщения видны только участникам той же комнаты. Появятся две связанные структуры данных вместо одной, состояние клиента превратится в маленький автомат, а мы разберём, почему удаление участника из комнаты — операция, в которой легко ошибиться порядком действий.

Глава 46

Глава 45. Комнаты

У нас есть цикл событий из главы 43 и двоичный протокол из главы 44. Соединим их и получим чат, в котором сообщения видны не всем подряд, а только участникам одной комнаты.

Заодно напишем наконец нормального клиента — того, который понимает наш протокол и умеет принимать сообщения в любой момент, а не только в ответ на свои.

46.1 Что добавляется

К реактору из главы 43 добавляются три вещи.

У клиента появляются имя и комната. Имя приходит в сообщении Hello, комната — в Join. Пока имени нет, клиент почти ничего не может.

Появляется вторая структура данных — отображение из имени комнаты в множество дескрипторов её участников.

И появляется разбор по типам сообщений вместо разбора текста: Hello, Join, Leave, Message. Каждый тип — отдельное действие.

46.2 Две структуры вместо одной

```
struct ClientState {
    int fd;
    std::string name;           // пусто, пока не прислал Hello
    std::string room;          // пусто = не в комнате
    std::vector<std::uint8_t> recv_buf;
    std::vector<std::uint8_t> send_buf;
```

```
};

std::unordered_map<int, ClientState> clients;
std::unordered_map<std::string, std::unordered_set<int>> rooms;
```

Данные хранятся дважды, и это осознанно. По дескриптору надо быстро узнать, кто это (clients), а по имени комнаты — быстро получить список участников для рассылки (rooms). Одна структура не даёт обеих операций за постоянное время.

Плата за двойное хранение — необходимость держать их согласованными. Клиент, попавший в rooms, но исчезнувший из clients, — источник обращений к несуществующим данным. Все места, где эта связь может порваться, надо знать поимённо, и мы их сейчас разберём.

unordered_set для участников выбран потому, что нужны быстрые вставка и удаление и не нужен порядок. Обычный вектор дал бы удаление за линейное время, а при рассылке в комнату на тысячу человек это заметно.

46.3 Обработка сообщений

Диспетчер по типу — это и есть весь протокол уровня приложения:

```
void process_message(std::unordered_map<int, ClientState>& clients,
                    std::unordered_map<std::string, std::unordered_set<int>>& rooms,
                    int fd, std::uint8_t type, const std::string& payload) {

    auto ci = clients.find(fd);
    if (ci == clients.end()) return;
    ClientState& c = ci->second;

    switch (type) {
        case chat::kHello: {
            c.name = payload;
            enqueue_frame(c, chat::kNotify,
                          "Привет, " + c.name + ". /join <room> для входа.");
            break;
        }
        case chat::kJoin: {
            if (c.name.empty()) {
                enqueue_frame(c, chat::kNotify, "сначала Hello");
                break;
            }
        }
    }
```



```

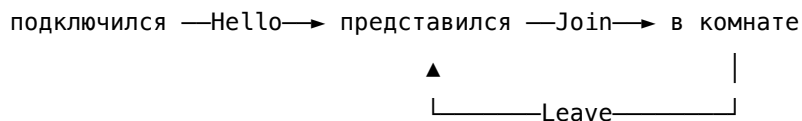
        leave_room(clients, rooms, fd);
        c.room = payload;
        rooms[c.room].insert(fd);
        enqueue_frame(c, chat::kNotify, "вы в комнате " + c.room);
        broadcast_room(clients, rooms, c.room, fd, chat::kNotify,
                        c.name + " вошёл в " + c.room);

        break;
    }
    case chat::kLeave: {
        leave_room(clients, rooms, fd);
        enqueue_frame(c, chat::kNotify, "вы вышли");
        break;
    }
    case chat::kMessage: {
        if (c.room.empty()) {
            enqueue_frame(c, chat::kNotify, "не в комнате");
            break;
        }
        std::string out = "<" + c.name + "> " + payload;
        broadcast_room(clients, rooms, c.room, fd, chat::kMessage, out);
        break;
    }
    default:
        enqueue_frame(c, chat::kNotify, "неизвестный тип");
        break;
}
}

```

Обратите внимание на две проверки состояния: Join без имени и Message без комнаты. Клиент, приславший их не вовремя, получает вежливое уведомление, а не падение сервера.

Дело в том, что состояние клиента — это маленький автомат:



Три состояния, переходы по сообщениям. В каждом состоянии допустим свой набор сообщений, и всё, что не допустимо, надо отвергать явно.

Это общее правило работы с сетью: **никогда не полагайтесь на то, что клиент присылает сообщения в правильном порядке**. Ваш клиент, может, и присылает, а вот случайно подключившийся сканер портов, старая версия программы или злоумышленник — нет. Сервер обязан выживать при любой последовательности входящих данных.

Заметьте ещё, что Join в другую комнату сначала зовёт leave_room. Иначе клиент остался бы участником двух комнат сразу, а поле room помнило бы только последнюю — и выйти из первой он бы уже не смог никогда.

46.4 Выход из комнаты: порядок решает

Функция, в которой легче всего ошибиться:

```
void leave_room(std::unordered_map<int, ClientState>& clients,
               std::unordered_map<std::string, std::unordered_set<int>>& rooms,
               int fd) {
    auto ci = clients.find(fd);
    if (ci == clients.end() || ci->second.room.empty()) return;
    std::string room = ci->second.room;
    rooms[room].erase(fd);
    if (rooms[room].empty()) rooms.erase(room);
    broadcast_room(clients, rooms, room, fd, chat::kNotify,
                  ci->second.name + " вышел из " + room);
    ci->second.room.clear();
}
```

Пять строк, и в каждой — решение.

Имя комнаты **копируется** в локальную переменную. Дальше мы будем чистить ci->second.room, и если бы работали со ссылкой на него, то к моменту рассылки читали бы уже пустую строку.

Участник удаляется из множества **до** рассылки. Иначе ушедший получил бы уведомление о собственном уходе. Формально broadcast_room исключает отправителя по from_fd, так что вреда бы не было, но полагаться на две защиты вместо одной — плохая привычка.

Опустевшая комната удаляется целиком. Без этой строки rooms копил бы пустые множества, и через месяц работы сервер держал бы в памяти тысячи комнат, в которых никого нет. Мелочь, из которой получаются утечки.

И только после всего — очистка поля room у клиента.

Ещё одна тонкость связана с отключением. При обрыве соединения `leave_room` надо звать **до** `close` и удаления из `clients`:

```
if ((re & (POLLHUP | POLLERR | POLLNVAL)) || drop) {
    leave_room(clients, rooms, fd);
    ::close(fd);
    clients.erase(it);
}
```

Порядок обратный порядку создания, как и положено при уборке. Если сначала удалить клиента из `clients`, то `leave_room` не найдёт его и молча ничего не сделает — а дескриптор останется в множестве участников комнаты. Дальше рассылка попытается отправить на закрытый дескриптор; с флагом `MSG_NOSIGNAL` это безопасно, но участник-призрак останется в комнате навсегда.

46.5 Рассылка

```
void broadcast_room(std::unordered_map<int, ClientState>& clients,
                   std::unordered_map<std::string, std::unordered_set<int>>& rooms,
                   const std::string& room, int from_fd,
                   std::uint8_t type, const std::string& payload) {
    auto it = rooms.find(room);
    if (it == rooms.end()) return;
    for (int fd : it->second) {
        if (fd == from_fd) continue;
        auto ci = clients.find(fd);
        if (ci == clients.end()) continue;
        enqueue_frame(ci->second, type, payload);
    }
}
```

Функция ничего не отправляет — она только раскладывает кадры по буферам отправки, как мы договорились в главе 43. Отправкой займётся цикл, когда ядро сообщит о готовности сокетов.

Проверка `ci == clients.end()` — вторая линия обороны на случай, если согласованность двух структур всё-таки нарушилась. В правильно работающем сервере она не срабатывает никогда; оставлена потому, что цена — одно сравнение, а результат её отсутствия — обращение к несуществующему клиенту.

Кодирование кадра происходит для каждого получателя заново:

```
void enqueue_frame(ClientState& c, std::uint8_t type, const std::string& payload) {
    auto frame = chat::encode(type, payload);
    c.send_buf.insert(c.send_buf.end(), frame.begin(), frame.end());
}
```

При рассылке в комнату на тысячу человек это тысяча одинаковых кодирований. Правильнее закодировать один раз и вставлять готовые байты — задание в упражнениях, и на большой комнате разница заметна.

46.6 Клиент, понимающий протокол

Эхо-клиент из главы 40 для такого чата не годится: он отправляет строку и ждёт ответ, а сообщения в чате приходят когда угодно. Нужен клиент с двумя потоками — один читает клавиатуру, другой сокет.

```
std::mutex g_out_mutex;           // вывод из двух потоков нужно защищать
std::atomic<bool> g_running(true);

void say(const std::string& line) {
    std::lock_guard<std::mutex> lock(g_out_mutex);
    std::cout << line << "\n";
    std::cout.flush();
}
```

Первое, обо что спотыкаются при написании такого клиента, — вывод. Два потока, пишущие в один терминал без защиты, перемешивают строки; иногда прямо посреди слова. Мьютекс вокруг печати обязателен, и это ровно тот случай из главы 41, когда общий ресурс — не переменная, а устройство.

Поток приёма читает сокет, накапливает байты и разбирает кадры — тем же способом, что сервер:

```
void receiver(int fd) {
    std::vector<std::uint8_t> acc;
    char buf[1024];

    while (g_running) {
        ssize_t r = ::recv(fd, buf, sizeof(buf), 0);
        if (r == 0) { say("[соединение закрыто сервером]"); break; }
        if (r < 0) {
            if (errno == EINTR) continue;
        }
    }
}
```

```

        say(std::string("[recv: ") + std::strerror(errno) + "]");
        break;
    }
    acc.insert(acc.end(), reinterpret_cast<std::uint8_t*>(buf),
               reinterpret_cast<std::uint8_t*>(buf) + r);

    try {
        std::uint8_t type = 0;
        std::string payload;
        while (chat::try_decode(acc, type, payload)) {
            if (type == chat::kMessage)    say(payload);
            else if (type == chat::kNotify) say("* " + payload);
            else                           say("[тип " + std::to_string(type) +
                                                "]" + payload);
        }
    } catch (const std::exception& e) {
        say(std::string("[ошибка протокола: ") + e.what() + "]");
        break;
    }
}
g_running = false;
}

```

Главный поток читает строки и превращает их в кадры:

```

if (line == "/quit") {
    break;
} else if (line.compare(0, 6, "/join ") == 0) {
    ok = send_frame(fd, chat::kJoin, line.substr(6));
} else if (line == "/leave") {
    ok = send_frame(fd, chat::kLeave, "");
} else {
    ok = send_frame(fd, chat::kMessage, line);
}

```

И последняя деталь, без которой клиент не завершается:

```

g_running = false;
::shutdown(fd, SHUT_RDWR); // разбудит receiver, зависший в recv

```

```
rx.join();  
::close(fd);
```

Флага `g_running` недостаточно: поток приёма в этот момент спит внутри `recv` и никаких флагов не проверяет. Разбудить его можно, закрыв соединение, — `shutdown` делает это, и `recv` немедленно возвращает ноль.

Разница между `shutdown` и `close` тут существенна. `close` закрывает дескриптор, и если его в этот момент читает другой поток, поведение непредсказуемо: номер может быть переиспользован новым сокетом. `shutdown` не трогает дескриптор, а разрывает само соединение — все ожидающие вызовы просыпаются, и только потом, после `join`, дескриптор можно спокойно закрыть.

46.7 Проверка

Запускаем сервер и двух клиентов:

alice:

Подключён как alice. /join <комната>, /leave, /quit.

* Привет, alice. /join <room> для входа.

* вы в комнате general

* bob вошёл в general

<bob> привет от боба

* bob вышел из general

bob:

Подключён как bob. /join <комната>, /leave, /quit.

* Привет, bob. /join <room> для входа.

* вы в комнате general

Алиса видит появление Боба, его сообщение и уход. Сам Боб своих сообщений не получает — рассылка исключает отправителя.

А третий клиент в другой комнате не видит ничего из этого разговора:

carol:

* Привет, carol. /join <room> для входа.

* вы в комнате secret

Изоляция работает.

46.8 Слабые места

Список честный, потому что в сетевом коде такой список важнее, чем описание работающего.

Буфер отправки не ограничен. Если клиент перестал читать — завис, ушёл в спящий режим, — его `send_buf` растёт с каждым сообщением в комнате. Сотня активных собеседников и один зависший дадут гигабайты в памяти сервера. Настоящие серверы ставят предел и отключают клиента, который в него упёрся; иногда вместо отключения выбрасывают старые сообщения. Это первое, что нужно добавить перед выпуском в мир.

Нет ограничения на частоту. Клиент может слать тысячу сообщений в секунду и занять сервер целиком. Лекарство — счётчик сообщений в единицу времени на каждого клиента.

Нет проверки имён. Двое могут назваться одинаково, имя может быть пустым, длиной в 64 килобайта или содержать управляющие символы, которые испортят терминал получателем. Любое поле, пришедшее от клиента, нужно проверять — это относится не только к чатам.

Клиент в одной комнате. В настоящих чатах человек состоит сразу в нескольких, и это меняет структуру данных: вместо поля `room` — множество комнат, а в сообщении появляется указание, куда именно оно адресовано. В IRC для этого служит команда `PRIVMSG #канал :текст`.

Ничего не сохраняется. Перезапустили сервер — комнаты пусты, история потеряна. Историей займёмся в следующей главе.

46.9 Главные правила главы

1. Две связанные структуры данных — цена быстрых операций в обе стороны; согласовывать их приходится вручную.
2. Состояние клиента — автомат; сервер обязан отвергать сообщения, недопустимые в текущем состоянии.
3. Не полагайтесь на порядок сообщений от клиента: сервер должен выживать при любой последовательности.
4. При выходе из комнаты порядок действий строгий: скопировать имя, удалить участника, разослать, очистить поле.
5. Опустевшие комнаты надо удалять, иначе они копятся навсегда.
6. При обрыве соединения `leave_room` вызывается до удаления клиента, иначе в комнате остаётся призрак.
7. Вывод из нескольких потоков в один терминал защищают мьютексом — иначе строки перемешиваются.
8. Разбудить поток, спящий в `recv`, можно через `shutdown`; закрывать дескриптор до `join` нельзя.

46.10 Упражнения

1. Соберите чат и проверьте изоляцию комнат: три клиента, двое в одной комнате, третий в другой. Убедитесь, что третий не видит чужих сообщений.
2. Отправьте Message до Join и Join до Hello. Какие уведомления пришли? Найдите в коде проверки, которые их выдали.
3. Уберите удаление пустых комнат и добавьте служебную команду, печатающую размер rooms. Повходите и повыходите десяток раз — что происходит с числом комнат?
4. Поменяйте местами `leave_room` и `clients.erase` при отключении. Подключите двух клиентов в одну комнату, одного убейте резко и посмотрите, что происходит при следующей рассылке.
5. Сделайте, чтобы `broadcast_room` кодировал кадр один раз, а не для каждого получателя. Замерьте разницу на комнате из пятидесяти клиентов.
6. Добавьте команды `/who` (список участников комнаты) и `/list` (список комнат). Первая — обход множества, вторая — обход отображения.
7. (Сложнее) Ограничьте размер буфера отправки: если он превысил мегабайт, отключайте клиента с уведомлением. Проверьте, подключившись клиентом, который не читает данные вовсе.
8. (Сложнее) Поддержите участие в нескольких комнатах: замените поле `room` на множество, добавьте в сообщение указание комнаты. Продумайте, что произойдёт со старыми клиентами, которые про это не знают.

46.11 Что дальше

Глава 46 — шифрование и история. Разберём, почему простое шифрование исключаящим ИЛИ выглядит убедительно и ломается за пять минут, — причём ломаем его сами, восстановив ключ по известному куску текста. И добавим серверу память: журнал сообщений на диске, из которого новый участник получает несколько последних реплик.

Глава 47

Глава 46. Шифрование и история

Две темы в одной главе, и обе про то, что происходит с сообщением после отправки.

Сначала шифрование — точнее, разговор о том, почему самодельное шифрование не работает. Мы напишем самый популярный из самодельных шифров, убедимся, что он выглядит убедительно, и сломаем его за пять минут, восстановив ключ. Это не упражнение в криптографии, а прививка от очень распространённой ошибки.

Потом — история сообщений на диске, чтобы новый участник видел несколько последних реплик, а перезапуск сервера не стирал разговор.

47.1 Шифр, который придумывают все

Идея выглядит красиво: возьмём ключ и наложим его на текст побитовым исключающим ИЛИ.

```
void xor_in_place(std::string& data, const std::string& key) {  
    for (std::size_t i = 0; i < data.size(); ++i) {  
        data[i] = static_cast<char>(data[i] ^ key[i % key.size()]);  
    }  
}
```

Свойство операции делает шифрование и расшифровку одной функцией: $(a \oplus k) \oplus k = a$. Ключ короче текста — повторяем его по кругу.

Смотрим на результат:

Plain: 'Hello, this is a confidential message!'

Plain hex: 48 65 6c 6c 6f 2c 20 74 68 69 73 20 69 73 20 61 ...

Cipher hex: 3b 00 0f 1e 0a 58 11 46 5b 1a 16 43 1b 16 54 50 ...

Decrypted: 'Hello, this is a confidential message!'

Выглядит убедительно: осмысленный текст превратился в невнятные байты и вернулся обратно. На этом месте обычно и останавливаются — и зря.

47.2 Ломаем за пять минут

У шифра есть уязвимость, которую даже не надо искать: она следует из самой конструкции. Если известен хотя бы кусок исходного текста и соответствующий ему кусок шифра, ключ достаётся тем же исключаящим ИЛИ:

```
шифр = текст ^ ключ  
шифр ^ текст = ключ
```

В нашем чате известный кусок найти проще простого: все сообщения начинаются с <имя>, служебные — со слова «Привет». Даже не зная ничего, можно предположить типичное начало.

```
std::string known = "Hello, ";  
std::string recovered_key;  
for (std::size_t i = 0; i < known.size(); ++i) {  
    recovered_key.push_back(known[i] ^ cipher[i]);  
}
```

=== Атака: known-plaintext ===

Зная начало plaintext, восстановили ключ:

```
Восстановленный: 'secret1'  
Реальный:       'secret1'
```

Семь символов известного текста — семь символов ключа. Ещё немного известного текста — весь ключ, и дальше читается вся переписка.

Это называется атакой по известному открытому тексту, и повторяющийся ключ беззащитен перед ней в принципе. Причём для взлома даже не нужен известный текст: если ключ короткий, его длина определяется статистически (метод Касиски, известный с 1863 года), а дальше шифр распадается на несколько простых замен, которые вскрываются частотным анализом.

Есть ещё две беды. Одинаковые сообщения дают одинаковый шифр — противник видит повторы, даже не расшифровывая. И шифр никак не защищён от подмены: перевернув бит в шифре, вы перевернёте тот же бит в расшифрованном тексте, и получатель не заметит подлога.

Любопытно, что схема с одноразовым ключом длиной с сообщение — единственный шифр с доказанной невзламываемостью. Но ключ должен быть по-настоящему случайным, длиной с текст и использоваться ровно один раз. Стоит применить его дважды — и всё рушится: сложив два шифра, противник получает сложение двух открытых текстов, а это уже поддаётся разбору. Именно на таком повторном использовании ключей строилась операция «Венона», в ходе которой американцы десятилетиями читали советские шифровки.

47.3 Что делать вместо

Правило, которому стоит следовать без исключений: **не изобретайте криптографию и не реализуйте её сами.**

Даже правильный алгоритм легко испортить реализацией. Сравнение хешей обычным `memcmp` выдаёт информацию через время выполнения — по нему подбирают значение побайтно. Случайные числа из `rand()` предсказуемы. Ключ, оставшийся в памяти после использования, попадёт в дамп при падении. Всё это не гипотетические опасности, а реальные уязвимости в реальных продуктах.

Для сетевого соединения ответ один — TLS. Его дают библиотеки OpenSSL, BoringSSL, mbedTLS, и в них уже решены вопросы, о которых самодельная реализация даже не подозревает: согласование версий и наборов шифров, проверка сертификатов, защита от повторной отправки, безопасное обновление ключей.

Если нужно зашифровать данные, а не соединение, берите готовые библиотеки высокого уровня — `libsodium` даёт разумные функции, в которых трудно ошибиться, и по умолчанию выбирает надёжные алгоритмы.

В нашем чате шифрования нет — и это честнее, чем самодельная схема, создающая ложное ощущение защищённости. Правильный путь для `demo-chat` — обернуть соединение в TLS, и это большое упражнение в конце главы.

47.4 История сообщений

Вторая половина главы — память сервера. Сейчас перезапуск стирает всё, а новый участник входит в комнату, где, судя по всему, никто никогда не говорил.

Формат журнала знаком по главе 35 — записи с префиксом длины:

Формат записи (как в WAL):

```
[4]  length u32 LE = 4 + 2 + 2 + 2 + name.size() + room.size() + text.size()
[4]  timestamp u32 LE (секунды от epoch)
[2]  name_len u16 LE
```

```
[2]    room_len u16 LE
[2]    text_len u16 LE
[...] name + room + text (по len байтов)
```

Три длины подряд, потом три строки подряд. Такая раскладка — «сначала все размеры, потом все данные» — позволяет прочитать заголовок фиксированного размера и сразу знать, сколько занимает каждое поле.

Запись — только в конец, как и положено журналу:

```
std::vector<std::uint8_t> framed;
w_u32(framed, static_cast<std::uint32_t>(body.size()));
framed.insert(framed.end(), body.begin(), body.end());

std::size_t off = 0;
while (off < framed.size()) {
    ssize_t w = ::write(fd_, framed.data() + off, framed.size() - off);
    if (w < 0) {
        if (errno == EINTR) continue;
        throw std::runtime_error("history write");
    }
    off += static_cast<std::size_t>(w);
}
::fsync(fd_);
```

Запись собирается целиком и отдаётся одним вызовом — по той же причине, что в журнале базы данных: половина записи в файле хуже, чем её отсутствие.

А вот `fsync` на каждое сообщение — решение спорное. Для базы данных оно оправданно: потерять подтверждённую транзакцию нельзя. Для чата — вопрос вкуса: сообщение уже разослано участникам, и потеря последних записей истории при внезапном выключении не катастрофа. Зато каждое сообщение стоит полтора сотен микросекунд, которые мы измеряли в главе 33. Разумный компромисс — синхронизировать не чаще раза в секунду; это упражнение.

47.4.1 Чтение последних N

```
std::vector<HistoryRecord> History::recent(const std::string& room, std::size_t n) {
    std::vector<HistoryRecord> all;
    int rfd = ::open(path_.c_str(), O_RDONLY);
    if (rfd < 0) return all;
```

```

// Верхний предел на запись. Без него испорченный файл с мусором в поле
// длины заставил бы нас выделить гигабайты – та же защита, что в разборе
// сетевого протокола (глава 44).
const std::uint32_t kMaxRecord = 1u << 20;    // 1 MiB

std::uint8_t hdr[4];
while (read_all(rfd, hdr, 4)) {
    std::uint32_t body_len = r_u32(hdr);
    if (body_len < 10 || body_len > kMaxRecord) break;    // битая запись – дальше не идём
    std::vector<std::uint8_t> body(body_len);
    if (!read_all(rfd, body.data(), body_len)) break;

    HistoryRecord rec;
    rec.timestamp = r_u32(body.data());
    std::uint16_t nl = r_u16(body.data() + 4);
    std::uint16_t rl = r_u16(body.data() + 6);
    std::uint16_t tl = r_u16(body.data() + 8);
    if (10u + nl + rl + tl > body_len) break;            // длины не сходятся
    std::size_t pos = 10;
    rec.name.assign(reinterpret_cast<const char*>(body.data() + pos), nl); pos += nl;
    rec.room.assign(reinterpret_cast<const char*>(body.data() + pos), rl); pos += rl;
    rec.text.assign(reinterpret_cast<const char*>(body.data() + pos), tl);

    if (room.empty() || rec.room == room) {
        all.push_back(std::move(rec));
    }
}
::close(rfd);

if (all.size() > n) {
    all.erase(all.begin(), all.end() - static_cast<std::ptrdiff_t>(n));
}
return all;
}

```

Две проверки в начале цикла появились не сразу, и стоит объяснить, почему они важны.

Первая ограничивает размер записи. Без неё файл, испорченный обрывом питания или чьей-то шалостью, мог бы содержать в поле длины четыре миллиарда — и `std::vector<std::uint8_t> body(body_len)` попытался бы выделить четыре гигабайта. Ровно та же уязвимость, что мы разбирали в сетевом протоколе главы 44, только источник данных другой — не сеть, а диск. Правило одно: **любая длина, пришедшая извне, проверяется до того, как ею воспользоваться**, и неважно, откуда она пришла.

Вторая проверка сверяет три длины с общим размером записи. Без неё поле `text` читалось бы за границей выделенного буфера — классическое чтение за пределами массива, ведущее к падению или утечке чужой памяти.

Заметьте, что при любой несостыковке мы прекращаем чтение, а не пропускаем запись. Логика та же, что в журнале базы: испорченный хвост означает обрыв, и всё, что после него, доверия не заслуживает.

47.4.2 Наивность, о которой надо знать

Функция читает файл **целиком**, чтобы вернуть последние несколько записей. На истории в сто мегабайт это сто мегабайт чтения ради десяти строк.

Так делать нельзя, и в комментарии к классу это честно записано. Правильных решений два. Можно читать с конца файла: отступить, скажем, на десять килобайт, найти границу записи и разобрать хвост, — но границу в потоке записей переменной длины ещё надо найти, и придётся либо хранить смещения, либо добавить в формат маркер начала записи. Можно вести отдельный индекс: файл со смещениями последних записей каждой комнаты.

Есть и третий путь, самый практичный: ротация. История режется на файлы по дате или размеру, старые сжимаются и убираются, а `rescent` смотрит только в последний. Так устроены почти все системы журналирования.

Мы оставили наивную реализацию, потому что она короткая и понятная, а объём учебной истории измеряется килобайтами. Но знать её предел обязательно — как раз о таких местах и говорят «работало на тесте, легло на продакшене».

```
=== Последние 10 в general ===
```

```
[1785407300] [general] <Alice> первое сообщение
```

```
[1785407301] [general] <Bob> ответ
```

```
[1785407303] [general] <Alice> ещё одно
```

```
=== Все записи (по всем комнатам) ===
```

```
[1785407300] [general] <Alice> первое сообщение
```

```
[1785407301] [general] <Bob> ответ
```

[1785407302] [tech] <Carol> что-то про код

[1785407303] [general] <Alice> ещё одно

Фильтр по комнате работает, пустое имя комнаты означает «все».

47.5 Как это встроить в сервер

Две точки подключения.

При получении сообщения — записать в журнал перед рассылкой:

```
case chat::kMessage: {
    if (c.room.empty()) { ... }
    HistoryRecord rec;
    rec.timestamp = static_cast<std::uint32_t>(std::time(nullptr));
    rec.name = c.name;
    rec.room = c.room;
    rec.text = payload;
    history.append(rec);

    std::string out = "<" + c.name + "> " + payload;
    broadcast_room(clients, rooms, c.room, fd, chat::kMessage, out);
    break;
}
```

При входе в комнату — отдать новичку последние реплики:

```
for (const HistoryRecord& r : history.recent(c.room, 10)) {
    enqueue_frame(c, chat::kMessage, "<" + r.name + "> " + r.text);
}
```

Второе — та самая мелочь, которая отличает работающий чат от учебного примера: человек, вошедший в комнату, видит, о чём говорят, а не пустой экран.

Порядок здесь важен: сначала история, потом уведомление «вы вошли». Иначе новичок увидит сообщения о своём приходе раньше того, что было до него.

47.6 Хранить ли переписку вообще

Вопрос не технический, и обойти его нельзя.

Журнал сообщений — это персональные данные, и в большинстве стран их хранение регулируется законом: нужны основание, срок хранения и возможность удаления по

требованию. Хранить переписку вечно «на всякий случай» — не нейтральное техническое решение, а осознанный выбор с последствиями.

Отсюда практические выводы, которые стоит принимать до того, как система начнёт работать. Определите срок хранения и удаляйте старое автоматически. Не пишите в журнал больше, чем нужно: адрес, устройство, точное время — всё это данные, которые потом придётся защищать. Шифруйте хранилище на диске, если переписка хоть сколько-нибудь чувствительная. И помните, что сквозное шифрование, при котором сервер вообще не видит содержимого, снимает большую часть этих вопросов — но делает невозможной серверную историю.

Для учебного чата всё это неважно. Для настоящего — важнее, чем выбор структуры данных.

47.7 Главные правила главы

1. Шифр с повторяющимся ключом вскрывается по известному куску текста: сложение шифра с текстом даёт ключ.
2. Одинаковые сообщения при таком шифровании дают одинаковый шифр — утечка информации без всякого взлома.
3. Не изобретайте и не реализуйте криптографию сами; для соединения берите TLS, для данных — готовую библиотеку.
4. Отсутствие шифрования честнее самодельного: оно не создаёт ложного ощущения защищённости.
5. Записи журнала пишут целиком одним вызовом, чтобы не оставить половину при обрыве.
6. Любая длина, пришедшая извне — из сети или из файла, — проверяется до того, как под неё выделяют память.
7. При несостыковке в журнале чтение прекращают, а не пропускают запись: испорченный хвост доверия не заслуживает.
8. Чтение всего файла ради последних записей работает на килобайтах и не работает на гигабайтах — нужен индекс или ротация.

47.8 Упражнения

1. Запустите `./build/xor_demo` и повторите атаку руками: возьмите первые семь байт шифра и первые семь байт известного текста, сложите их и убедитесь, что получился ключ.
2. Зашифруйте одним ключом два разных сообщения, сложите шифры между собой и посмотрите на результат. Что можно сказать об исходных текстах, не зная ключа?

3. Запустите `./build/history_demo`, посмотрите файл истории через `xxd` и найдите в нём длины, метку времени и строки.
4. Испортите файл истории: поменяйте первый байт длины на `0xFF`. Что делает `recent`? Уберите проверку `body_len > kMaxRecord` и повторите — чем закончится?
5. Встройте историю в сервер комнат: запись при каждом сообщении, выдача последних десяти при входе. Проверьте, что новый участник видит разговор.
6. Замените `fsync` на каждую запись синхронизацией не чаще раза в секунду. Замерьте, как изменилась скорость приёма сообщений.
7. (Сложнее) Сделайте ротацию: когда файл истории превысил мегабайт, переименуйте его с меткой времени и начните новый. `recent` должен уметь дочитать предыдущий файл, если в текущем записей не хватило.
8. (Сложнее) Оберните соединение в TLS через OpenSSL: серверу понадобится сертификат (самоподписанный сгодится), клиенту — проверка. Сравните объём работы с самодельным шифрованием и подумайте, почему разница именно такая.

47.9 Что дальше

Глава 47 закрывает Часть V. Соберём чат в контейнер: разберёмся, чем статическая сборка отличается от динамической и почему статически собрать программу с `glibc` не так просто, как кажется; сделаем многоступенчатый образ, в котором остаётся только наш бинарник и ничего больше; и подведём итоги всей части — что построено, что упрощено и куда идти дальше.

Глава 48

Глава 47. Сборка, упаковка и итоги Части V

Чат работает — на нашей машине, из каталога с исходниками, запущенный руками. Между этим состоянием и «работает у людей» лежит ещё одна работа, о которой в учебниках говорят редко.

В этой главе разберёмся, что мешает просто скопировать бинарник на другую машину, чем статическая сборка отличается от динамической, почему статически собрать программу с glibc не так просто, как кажется, и как упаковать сервер в контейнер размером в несколько мегабайт. И подведём итоги всей Части V.

48.1 Почему бинарник не переносится

Скопируем собранный сервер на другую машину — и, скорее всего, получим отказ запускаться. Причина в том, что программа собрана динамически: сама она содержит только свой код, а всё остальное берёт из библиотек системы.

Посмотреть, от чего зависит программа, можно системными средствами — `ldd` на Linux, `otool -L` на macOS:

```
$ otool -L build/rooms_chat_server
build/rooms_chat_server:
    /usr/lib/libc++.1.dylib (compatibility version 1.0.0, current version 2100.43.0)
    /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1356.0.0)
```

Две зависимости: стандартная библиотека C++ и системная библиотека C. На чужой машине их может не оказаться вовсе или оказаться другая версия, несовместимая по внутреннему устройству.

Именно отсюда растёт знаменитая жалоба «у меня работает». Программа, собранная на свежей Ubuntu, не запустится на старой CentOS, потому что там другая версия glibc, и наоборот — собранная на старой обычно запускается на новой, потому что совместимость поддерживают в одну сторону.

Наш сервер в динамической сборке занимает 310 килобайт — это почти чистый наш код. Всё остальное берётся снаружи.

48.2 Статическая сборка

Альтернатива — вкомпилировать библиотеки внутрь:

```
$ g++ -static -static-libstdc++ -static-libgcc ...
```

Тогда бинарник становится самодостаточным: скопировал и запустил, ничего не требуется. Размер вырастает до нескольких мегабайт, но зависимостей нет.

У подхода есть и обратная сторона, о которой стоит помнить. Обновление библиотеки с исправленной уязвимостью больше не приходит из системы — программу надо пересобирать и раскладывать заново. Для сервера, живущего в контейнере, это нормально: его и так пересобирают на каждый выпуск. Для программы, установленной у пользователей, — уже вопрос.

48.2.1 Что не так со статическим glibc

На Linux попытка собрать статически даёт неожиданное:

```
warning: Using 'getaddrinfo' in statically linked applications requires  
at runtime the shared libraries from the glibc version used for linking
```

Предупреждение, а не ошибка: программа соберётся и в большинстве случаев даже заработает. Но не всегда.

Причина в устройстве glibc. Разрешение имён устроено через подключаемые модули: как искать имя — в файле /etc/hosts, через DNS, через LDAP — решается настройкой, и соответствующий модуль подгружается на ходу. Подгрузка означает динамическую загрузку, а её невозможно вкомпилировать заранее: набор модулей известен только на целевой машине.

Итог: статически собранная программа с glibc работает, пока не понадобится разрешить имя. А нашему клиенту оно нужно — getaddrinfo вызывается при подключении.

Обойти это можно тремя способами. Не разрешать имена вовсе, работая только с числовыми адресами, — годится для сервера, но не для клиента. Собрать с другой реализацией стандартной библиотеки C. Или не собирать статически, а класть программу вместе с нужными библиотеками — так делают контейнеры.

48.2.2 musl

Вторая реализация, о которой речь, — musl. Написана с нуля, компактнее glibc, лицензия свободнее, и главное — рассчитана на статическую сборку: разрешение имён в ней встроено, никаких подгружаемых модулей.

Дистрибутив, построенный на musl, — Alpine Linux, самый популярный выбор для контейнеров именно по этой причине. Там статическая сборка получается честной:

```
# внутри alpine
$ apk add build-base
$ g++ -static -static-libstdc++ -static-libgcc -O2 -std=c++11 ... -o server
$ file server
server: ELF 64-bit LSB executable, statically linked, ...
```

Расплата — мелкие несовместимости. musl строже относится к нарушениям стандарта, у неё меньше расширений GNU, и программы, привыкшие к особенностям glibc, иногда требуют правок. Ещё у musl заметно медленнее стандартный распределитель памяти, что бьёт по программам, активно выделяющим память.

48.3 Контейнер

Многоступенчатая сборка решает противоречие «для сборки нужен компилятор, а в готовом образе он не нужен»:

```
# Многоступенчатый Dockerfile для chat-server.
# Stage 1: builder на alpine с musl — компилируем статически.
# Stage 2: scratch (пустой образ) + только бинарь.
# Итог: < 5 MB образ, никаких зависимостей.
```

```
FROM alpine:3.19 AS builder
RUN apk add --no-cache build-base make
WORKDIR /src
```

```

COPY . .
RUN make MODE=release CXX=g++ \
    CXXFLAGS="-std=c++11 -O2 -DNDEBUG -static -static-libstdc++ -static-libgcc -pthread -Iinclude -MM"
    LDFLAGS="-static -pthread"

FROM scratch
COPY --from=builder /src/build/rooms_chat_server /chat-server
EXPOSE 9001
ENTRYPOINT ["/chat-server", "9001"]

```

Первая ступень — полноценная система с компилятором, около полугигабайта. В ней собирается статический бинарник.

Вторая ступень начинается с `scratch` — совершенно пустого образа. В него копируется единственный файл: наш сервер. Ни оболочки, ни утилит, ни библиотек.

Итоговый образ — размер бинарника плюс несколько килобайт метаданных. Кроме размера это даёт безопасность: в контейнере, где нет ни оболочки, ни `cat`, злоумышленнику, нашедшему уязвимость, буквально нечем воспользоваться. Тот же принцип лежит в основе образов `distroless` от Google.

Расплата — отлаживать такой контейнер нечем: `docker exec` не запустит оболочку, потому что её нет. Отлаживают либо на промежуточном образе, либо подключая отладочный контейнер к тому же пространству имён.

```

$ docker build -t my-chat .
$ docker images
my-chat    latest    abc123    5.2MB

$ docker run -p 9001:9001 my-chat
Rooms chat-server слушает на :9001

```

Строка `EXPOSE 9001` — только документация; настоящий проброс делает `-p 9001:9001` при запуске. Сервер внутри контейнера обязан слушать `0.0.0.0`, а не `127.0.0.1`, иначе снаружи до него не достучаться, — у нас так и есть, `INADDR_ANY` из главы 40.

48.3.1 Чего в нашем контейнере не хватает

Сервер работает от имени администратора — в `scratch` других пользователей просто нет. Правильно добавить непривилегированного пользователя и указать `USER`.

Нет проверки живости: оркестратор не знает, работает сервер или завис. Обычно для этого добавляют HEALTHCHECK или отдельную ручку состояния.

Нет обработки сигнала завершения. Docker при остановке шлёт SIGTERM и через десять секунд добывает SIGKILL. Наш сервер не ловит SIGTERM, значит, его всегда убивают жёстко — и история сообщений теряет последние записи. Как это чинится, мы разбирали в главе 28: обработчик выставляет флаг, цикл его проверяет.

48.4 Что построено в Части V

Восемь глав назад мы не умели передать по сети ни байта. Сейчас в demo-chat/ лежит около полутора тысяч строк:

Файл	Что делает
echo_server.cpp, echo_client.cpp	сокеты: socket, bind, listen, accept, connect
threaded_chat_server.cpp	поток на клиента, мьютекс на общий список
atomic_demo.cpp	гонка данных и три способа её устранить
reactor_chat_server.cpp	цикл событий на poll, неблокирующие сокеты
protocol.cpp	двоичный протокол с префиксом длины
rooms_chat_server.cpp	комнаты поверх реактора и протокола
chat_client.cpp	клиент с двумя потоками, понимающий протокол
history.cpp	журнал сообщений на диске
xor_demo.cpp	почему самодельное шифрование не работает
tests/test_protocol.cpp	25 проверок протокола и истории

Три архитектуры одной задачи — последовательная, многопоточная и событийная — на одном и том же примере. Это, пожалуй, главное, что стоит унести из части: выбор между ними определяется не модой, а числом соединений и характером работы.

48.5 Чему научила эта часть

Что TCP — поток байтов, а не сообщений, и любой протокол поверх него обязан сам размечать границы. Что recv и send обрабатывают сколько успели, а не сколько попросили. Что закрытие соединения — это ноль от recv, а не ошибка.

Что общие данные в многопоточной программе должны быть защищены — без исключений; что гонку данных находит не внимательность, а ThreadSanitizer; что атомарная переменная годится для счётчика и не годится для структуры.

Что цикл событий заменяет тысячи спящих потоков одним и требует за это вывернуть код наизнанку, хранить состояние явно и никогда не блокироваться.

Что в протоколе нужны версия, тип и предел на размер, а разбор должен различать «данных мало» и «данные испорчены».

И что любая длина, пришедшая извне, проверяется до того, как под неё выделяют память, — эта мысль повторилась в части трижды, в трёх разных местах, и это не совпадение.

48.6 Главные правила Части V

1. Сокет — файловый дескриптор; сетевой код работает теми же приёмами, что файловый, плюс несколько своих.
2. Границы сообщений размечает протокол: разделителем, фиксированным размером или префиксом длины.
3. Данные, доступные нескольким потокам, защищают всегда; счётчику хватает атомарной переменной, структуре нужен мьютекс.
4. Поток на соединение прост и хорош до сотен клиентов; дальше нужен цикл событий.
5. В цикле событий нельзя блокироваться и нельзя делать ничего долгого — иначе встают все клиенты.
6. Каждому клиенту нужен собственный буфер отправки, потому что `send` может отправить часть.
7. Любое поле, пришедшее по сети, проверяется: длина, версия, тип, содержимое.
8. Криптографию не пишут сами — берут TLS или проверенную библиотеку.

48.7 Упражнения

1. Посмотрите зависимости своих бинарников через `ldd` или `otool -L`. Соберите сервер статически и сравните размер и список зависимостей.
2. Соберите образ по нашему `Dockerfile` (понадобится Docker и Linux или Docker Desktop). Проверьте размер и запустите сервер в контейнере, подключившись клиентом снаружи.
3. Добавьте в образ непривилегированного пользователя и укажите `USER`. Убедитесь, что сервер по-прежнему может занять порт 9001 (и объясните, почему не смог бы занять 80-й).
4. Научите сервер аккуратно завершаться по `SIGTERM`: обработчик выставляет флаг, цикл `poll` получает тайм-аут и проверяет флаг, при завершении закрываются все соединения. Проверьте через `docker stop`.

5. Добавьте предел на размер буфера отправки и отключение зависших клиентов — то, чего нам не хватало в главе 45.
6. Соберите сервер с `-fsanitize=address` и погоняйте под нагрузкой (сотня клиентов, тысячи сообщений). Нашлось ли что-нибудь?
7. (Сложнее) Сделайте нагрузочный клиент: подключает N соединений, каждое шлёт сообщение каждые M миллисекунд, считает задержку между отправкой и получением. Сравните три сервера — многопоточный, реактор и комнатный — на 500 соединениях.
8. (Сложнее) Замените `poll` на `epoll` (Linux) или `kqueue` (macOS) в комнатном сервере и повторите нагрузочный тест. Где разница стала заметной?

48.8 Что дальше

Часть VI — короткая, четыре главы, и она про инструменты, которых у нас до сих пор не было. Всю книгу мы писали на C++11, чтобы код собирался хоть чем и хоть где. Теперь посмотрим, что добавили следующие стандарты и как они меняют уже написанный код: `std::optional` и `std::variant` вместо самодельных признаков «значения нет», `std::filesystem` вместо системных вызовов, `std::string_view` вместо копирования строк. А в последней главе — обзор того, что появилось в C++20 и C++23, и разговор о том, куда двигаться дальше.

Глава 49

Глава 48. C++17: optional, variant, структурные привязки

Всю книгу мы держались C++11. Причина была практическая: этот стандарт понимает любой компилятор старше 2013 года, и код из книги соберётся у школьника на старом ноутбуке и на сервере с древним дистрибутивом.

Но с 2011 года вышли ещё три стандарта, и кое-что в них не просто удобнее — оно меняет способ писать код. Часть VI — короткая, четыре главы — про то, что появилось и как оно относится к уже написанному нами.

Начнём с четырёх возможностей C++17, которые встречаются чаще всего. Все примеры лежат в `demo-cpp17/` и собираются с `-std=c++17`.

49.1 `std::optional`: значение или ничего

Задача старая как мир: функция ищет что-то и может не найти. Как об этом сообщить?

Способ первый — особое значение. Так делает `find_age_oldstyle` в нашем демо:

```
int find_age_oldstyle(const std::string& name) {  
    if (name == "Alice") return 30;  
    return -1; // «не найден» — но что если возраст реально -1?  
}
```

Работает, пока особое значение невозможно в данных. Возраст `-1` невозможен, а вот баланс счёта — вполне, и температура тоже. Такие функции ломаются не сразу, а когда в данных появляется то самое значение.

Способ второй — вернуть признак отдельно: `bool find(const std::string&, int& out)`. Мы так делали в `BPlusTree::find` в главе 34. Честно, но неуклюже: результат приходится объявлять заранее, а функцию нельзя использовать в выражении.

Способ третий — исключение. Годится, когда «не нашли» — исключительная ситуация, и не годится, когда это обычный ход дела.

C++17 даёт четвёртый:

```
std::optional<int> find_age(const std::string& name) {
    static const std::unordered_map<std::string, int> ages = {
        {"Alice", 30}, {"Bob", 25}
    };
    auto it = ages.find(name);
    if (it == ages.end()) return std::nullopt;
    return it->second;
}
```

Тип возвращаемого значения сам говорит: значение может быть, а может не быть. Ни особых значений, ни выходных параметров.

Пользоваться удобно:

```
auto a = find_age("Alice");
if (a.has_value()) {
    std::cout << "Alice: " << *a << "\n";
}

// Сокращение: optional в bool-контексте.
if (auto b = find_age("Charlie")) {
    std::cout << "Charlie: " << *b << "\n";
} else {
    std::cout << "Charlie не найден\n";
}

// value_or — значение по умолчанию.
int c_age = find_age("Charlie").value_or(0);
```

Проверка через `if (auto b = ...)` объявляет переменную прямо в условии и ограничивает её областью видимости `if` — это тоже подарок C++17, раньше так было можно только в цикле `for`.

Отличие *от .value() существенно. Оператор разыменования не проверяет наличие значения: обращение к пустому optional — неопределённое поведение, как разыменование нулевого указателя. Метод value() проверяет и бросает исключение:

```
try {
    std::cout << find_age("ghost").value() << "\n"; // бросит bad_optional_access
} catch (const std::bad_optional_access&) {
    std::cout << "bad_optional_access поймали\n";
}
```

Правило простое: * — когда наличие уже проверено, value() — когда хочется гарантию.

Есть тонкость с перемещением, о которой легко забыть:

```
std::optional<std::string> name = "Hello";
std::string taken = std::move(*name);
// *name теперь moved-from, но optional всё ещё has_value.
name.reset();
```

Перемещение забирает содержимое строки, но не делает optional пустым — он по-прежнему считает, что значение есть, просто значение это теперь пустая строка. Чтобы действительно опустошить, нужен reset(). Ошибка распространённая: переместили и думают, что has_value() вернёт false.

Важно понимать, что optional хранит значение внутри себя, а не в куче — никаких выделений памяти. Он занимает размер типа плюс флаг наличия плюс выравнивание.

49.2 std::variant: одно из нескольких

Второй тип решает другую задачу: значение может быть одного из нескольких типов.

В C++11 для этого писали объединение с тегом:

```
struct Item {
    enum Kind { Weapon, Armor, Potion } kind;
    union {
        int damage;
        int defense;
        int heal;
    };
};
```

Компактно и опасно: ничто не мешает записать `damage`, а прочитать `heal`. Компилятор не проверит, а объединение с нетривиальными типами (той же `std::string`) потребует ручного вызова конструкторов и деструкторов.

Второй способ — иерархия классов с виртуальными функциями. Именно так устроены предметы нашей RPG в главе 17. Способ хороший, но требует выделения в куче и указателя на каждый объект.

`std::variant` даёт третий путь — типобезопасное объединение:

```
using Item = std::variant<Weapon, Armor, Potion>;

std::vector<Item> items = { Weapon{8}, Armor{4}, Potion{15} };
```

Значения лежат по значению, без кучи, а тип отслеживается внутри. Прочитать не тот тип нельзя — попытка кончится исключением или ошибкой компиляции.

Разбирают `variant` двумя способами. Первый — спросить, что внутри:

```
if (std::holds_alternative<Weapon>(item)) {
    std::cout << "weapon dmg " << std::get<Weapon>(item).damage << "\n";
}
```

Второй, более идиоматичный, — посетитель:

```
std::visit([](const auto& x) {
    using T = std::decay_t<decltype(x)>;
    if constexpr (std::is_same_v<T, Weapon>) {
        std::cout << "Weapon, dmg=" << x.damage << "\n";
    } else if constexpr (std::is_same_v<T, Armor>) {
        std::cout << "Armor, def=" << x.defense << "\n";
    } else {
        std::cout << "Potion, heal=" << x.heal << "\n";
    }
}, item);
```

=== `std::visit` + `if constexpr` ===

Weapon, dmg=8

Armor, def=4

Potion, heal=15

`std::visit` вызывает переданную функцию с фактически хранящимся значением. Если функция — обобщённая лямбда (с `auto` в параметре), компилятор порождает по версии на каждый возможный тип, а `if constexpr` выбирает нужную ветку на этапе компиляции.

Главное преимущество перед иерархией классов — проверка полноты. Если добавить в `variant` четвёртый тип и забыть обработать его в посетителе, компилятор скажет об этом. С виртуальными функциями забытая реализация обнаружилась бы только во время работы.

Когда что выбирать? Иерархия классов лучше, когда набор типов открыт (кто угодно может добавить свой) и когда объекты крупные. `variant` лучше, когда набор типов закрыт и известен заранее, а объекты мелкие: нет выделений памяти, нет косвенности, полнота проверяется компилятором.

Размер `variant` — размер самого большого из типов плюс тег. Это стоит помнить: `std::variant<char, std::array<char, 1000>>` займёт килобайт независимо от того, что в нём лежит.

49.3 Структурные привязки

Мелочь, которая ощутимо меняет читаемость. Было:

```
auto result = m.insert({"Carol", 40});
if (result.second) {
    std::cout << "вставили " << result.first->first << "\n";
}
```

Что такое `second`? Что такое `first->first`? Приходится помнить.

Стало:

```
auto [it, inserted] = m.insert({"Carol", 40});
if (inserted) {
    std::cout << "вставили " << it->first << "\n";
}
```

Имена вместо номеров. Работает с парами, кортежами, структурами и массивами фиксированного размера:

```
auto [id, name, score] = std::make_tuple(42, "Alice", 5.7);
auto [x, y] = point;           // структура из двух полей
```

Особенно заметен выигрыш в циклах по отображениям:

```
for (const auto& [name, age] : ages) {
    std::cout << name << " = " << age << "\n";
}
```

Alice = 30

Bob = 25

Carol = 40

Вместо `kv.first` и `kv.second` — осмысленные имена. Наш код по всей книге пестрит обращениями к `kv.first`, и в C++17 каждое из них стало бы читаемое.

Ограничение: имена в привязке нельзя пропустить — если из трёх полей нужно одно, объявить придётся все три. Неиспользованные переменные вызовут предупреждение компилятора; в C++17 для этого есть атрибут `[[maybe_unused]]`.

49.4 if constexpr

Ветвление, которое разрешается при компиляции. Ветка, не выбранная условием, не просто не выполняется — она вообще не компилируется:

```
template <typename T>
void describe(const T& value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "integer: " << value << " (x2 = " << value * 2 << ")\n";
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "float: " << value << " (sqrt = " << std::sqrt(value) << ")\n";
    } else if constexpr (std::is_same_v<T, std::string>) {
        std::cout << "string: '" << value << "' (size " << value.size() << ")\n";
    } else {
        std::cout << "other type\n";
    }
}
```

integer: 42 (x2 = 84)

float: 3.14 (sqrt = 1.772)

string: 'hello' (size 5)

other type

Почему это важно, видно на строке: `value.size()` есть у `std::string`, но нет у `int`. С обычным `if` шаблон, вызванный с `int`, не скомпилировался бы — компилятор проверяет обе ветки, даже недостижимую. С `if constexpr` невыбранная ветка исключается из компиляции целиком.

До C++17 то же самое делали через специализацию шаблонов или хитрости с подстановкой типов — техника под названием SFINAE, читать которую было мучением. `if constexpr` заменяет её обычным `if`, и это одно из тех улучшений, ради которых стоит переходить на новый стандарт.

49.5 Что бы изменилось в нашем коде

Пройдёмся по книге и посмотрим, где C++17 сделал бы код лучше.

BPlusTree::find из главы 34 возвращает bool и пишет результат в выходной параметр:

```
bool find(std::int64_t key, std::int64_t& out_value) const;

// ВЫЗОВ
std::int64_t v = 0;
if (tree.find(42, v)) { ... }
```

C optional это стало бы:

```
std::optional<std::int64_t> find(std::int64_t key) const;

// ВЫЗОВ
if (auto v = tree.find(42)) { ... }
```

chat::try_decode из главы 44 возвращает bool и заполняет два выходных параметра. Просится std::optional<Frame>, где Frame — структура с типом и данными.

Разбор SQL из главы 36 хранит все виды команд в одной структуре с полями «на все случаи». Идеальный кандидат на std::variant<CreateStatement, InsertStatement, SelectStatement> с посетителем вместо switch — и компилятор проследит, чтобы ни один вид не забыли.

Обходы отображений — а их в книге десятки — стали бы читаемее со структурными привязками.

Значит ли это, что книгу надо было писать на C++17? Нет. Совместимость со старыми компиляторами тоже чего-то стоит, и понимание того, что optional делает под капотом, приходит именно от опыта с выходными параметрами. Но когда вы начнёте свой проект и никто не заставляет держаться старого стандарта — берите новый.

49.6 Что ещё есть в C++17

Коротко о том, что не поместилось в отдельные разделы.

std::any — хранилище значения любого типа с проверкой при извлечении. Нужен реже, чем кажется: почти всегда набор типов известен, и variant подходит лучше.

std::string_view — невладеющая ссылка на строку. Ему посвящена целая глава 50, потому что вокруг него есть тонкости.

std::filesystem — работа с путями и файлами вместо системных вызовов. Это глава 49.

Встроенные переменные (`inline`) — способ объявить переменную в заголовке, не нарушая правило одного определения. Мелочь, но избавляет от возни с отдельным файлом ради одной константы.

Гарантированное устранение копий при возврате временного объекта — стандарт теперь требует то, что компиляторы и так делали. Заодно это позволяет возвращать по значению объекты, которые нельзя ни копировать, ни перемещать.

Параллельные алгоритмы — про них будет разговор в главе 50, и разговор невесёлый.

49.7 Главные правила главы

1. `std::optional` заменяет особые значения и выходные параметры: тип сам сообщает, что значения может не быть.
2. `*` не проверяет наличие, `value()` проверяет и бросает исключение — выбирайте осознанно.
3. Перемещение из `optional` не делает его пустым; для этого есть `reset()`.
4. `std::variant` — типобезопасное объединение; в отличие от иерархии классов не требует кучи, а компилятор проверяет полноту разбора.
5. Размер `variant` определяется самым большим из вариантов.
6. Структурные привязки убирают `first/second` и делают циклы по отображениям читаемыми.
7. `if constexpr` исключает невыбранную ветку из компиляции — это заменяет громоздкие приёмы работы с шаблонами.
8. Переходить на новый стандарт стоит тогда, когда компиляторы целевых систем его поддерживают; в остальном ограничения бывают только внешние.

49.8 Упражнения

1. Соберите `demo-cpp17` (`cd demo-cpp17 && make`) и запустите все демонстрации. Разберитесь в выводе каждой.
2. Обратитесь к пустому `optional` через `*` вместо `value()`. Соберите с `-fsanitize=undefined` и посмотрите, что скажет детектор.
3. Добавьте в `variant` из демо четвёртый тип и не обрабатывайте его в посетителе. Какую ошибку выдаст компилятор?
4. Перепишите `BPlusTree::find` из главы 34 на `std::optional`. Что изменилось в коде вызывающей стороны?

5. Замените в любом файле книги обход отображения на структурные привязки. Сравните читаемость.
6. Напишите шаблон `print_size`, который для контейнеров печатает `size()`, для чисел — `sizeof`, а для остальных типов сообщает, что не знает. Используйте `if constexpr`.
7. (Сложнее) Перепишите `Statement` из главы 36 на `std::variant` с посетителем. Проверьте, что забытый вид команды теперь ловится компилятором.
8. (Сложнее) Реализуйте свой упрощённый `optional<T>` без выделения памяти: хранилище через `alignas` и `std::aligned_storage`, размещающее создание через `placement new`, деструктор, копирование и перемещение. Это упражнение показывает, сколько работы прячется за простым типом.

49.9 Что дальше

Глава 49 — `std::filesystem`. Всю книгу мы работали с файлами через системные вызовы: `open`, `stat`, `mkdir`, `opendir`. В C++17 появилась библиотека, делающая то же самое переносимо и без возни с `errno`. Разберём, как она устроена, чем `path` лучше `std::string`, почему у каждой функции есть две версии — с исключением и с кодом ошибки, — и где эта библиотека всё-таки не поможет.

Глава 50

Глава 49. `std::filesystem`

Всю книгу мы работали с файлами через системные вызовы. Открыть — `open`, узнать размер — `fstat`, создать каталог — `mkdir`, обойти его — `opendir` с `readdir`. Всё это POSIX, то есть Linux и macOS; на Windows пришлось бы писать другой код с другими именами.

В C++17 появилась библиотека `<filesystem>`, которая делает то же самое переносимо. В этой главе разберём, что она умеет, чем `path` отличается от строки и почему у каждой функции есть две версии.

50.1 Путь — не строка

Первое, к чему надо привыкнуть: путь представлен отдельным типом `std::filesystem::path`, а не `std::string`.

Причин несколько. Разделители различаются: на Windows это обратная косая черта, в остальном мире — прямая. Кодировка различается: на Windows имена файлов хранятся в UTF-16, в Linux это просто байты, на macOS — UTF-8 в особой нормализованной форме. И, наконец, у пути есть структура: каталог, имя, расширение, — которую по строке приходится вычислять руками.

Тип `path` всё это прячет. Его можно строить из строки, из строкового литерала, из другого пути:

```
namespace fs = std::filesystem;

fs::path base = "/tmp/fs_demo";
fs::path file = base / "a.txt";      // оператор / собирает путь
```

Оператор деления вместо склейки строк — самая заметная особенность. Он сам подставляет нужный разделитель и не даёт получить двойную косую черту:

Состав: "data/config/app.toml"

Разбор пути на части — набор методов:

```
fs::path p = "/Users/alice/docs/report.pdf";
std::cout << "parent_path: " << p.parent_path() << "\n";
std::cout << "filename:    " << p.filename() << "\n";
std::cout << "stem:       " << p.stem() << "\n";
std::cout << "extension:   " << p.extension() << "\n";
```

```
parent_path: "/Users/alice/docs"
filename:    "report.pdf"
stem:       "report"
extension:   ".pdf"
```

Раньше это писали руками через `rfind('/')` и `rfind('.')`, каждый раз заново и каждый раз с ошибками на краевых случаях: путь без каталога, файл без расширения, имя вроде `.gitignore` (у него расширения нет, это имя целиком).

Обратите внимание, что вывод пути в поток даёт кавычки. Так сделано намеренно: в именах файлов бывают пробелы, и кавычки показывают границы. Если нужна чистая строка — есть метод `string()`.

50.2 Операции с файлами и каталогами

```
fs::create_directories(base / "subdir1" / "deeper");
```

Создаёт всю цепочку сразу — как `mkdir -p`. Есть и `create_directory`, создающий один уровень и требующий существования родителя.

```
fs::remove(file);           // удалить файл или пустой каталог
fs::remove_all(base);       // удалить рекурсивно, как rm -rf
fs::copy(from, to);
fs::rename(from, to);
```

Всё это раньше требовало либо системных вызовов, либо запуска внешних программ. Обратите особое внимание на `remove_all` — он опасен ровно настолько, насколько опасен `rm -rf`, и ошибка в вычислении пути обойдётся дорого.

Свойства файла:

```
std::cout << "exists: " << fs::exists(file) << "\n";
std::cout << "is_regular_file: " << fs::is_regular_file(file) << "\n";
std::cout << "file_size: " << fs::file_size(file) << "\n";
```

```
exists: 1
is_regular_file: 1
file_size: 7
```

Здесь стоит сделать предупреждение, которое относится не к библиотеке, а к работе с файловой системой вообще. Проверка «файл существует» и последующее действие с ним — это две операции, между которыми мир может измениться. Файл могли удалить, подменить символической ссылкой, заменить чужим. В защищённом коде такую пару называют уязвимостью проверки-и-использования и избегают: вместо «проверить и открыть» просто открывают и разбирают ошибку.

50.3 Обход каталогов

Раньше это выглядело как цикл с `opendir`, `readdir` и `closedir`, а рекурсия писалась руками. Теперь:

```
for (const auto& entry : fs::recursive_directory_iterator(base)) {
    if (entry.is_regular_file()) {
        std::cout << entry.path().string()
                  << " (" << entry.file_size() << " bytes)\n";
    } else if (entry.is_directory()) {
        std::cout << entry.path().string() << "/\n";
    }
}
```

```
=== Все файлы в "/tmp/fs_demo" ===
/tmp/fs_demo/a.txt (7 bytes)
/tmp/fs_demo/subdir1/
/tmp/fs_demo/subdir1/deeper/
/tmp/fs_demo/subdir1/deeper/c.txt (7 bytes)
/tmp/fs_demo/subdir1/b.txt (7 bytes)
```

Есть два итератора: `directory_iterator` обходит один уровень, `recursive_directory_iterator` — всё дерево. Оба работают в цикле по диапазону.

Порядок обхода не определён — обратите внимание на вывод: `b.txt` идёт после содержимого `deeper`. Файловая система выдаёт записи в своём порядке, и если нужен отсортированный список, его надо собрать и отсортировать самому.

Элемент обхода — не путь, а `directory_entry`, и это важно для производительности. Он кэширует сведения о файле, полученные при обходе, поэтому `entry.is_regular_file()` и `entry.file_size()` обычно не делают дополнительных обращений к диску, а вот `fs::file_size(entry.path())` сделал бы. На каталоге в сто тысяч файлов разница заметна.

У рекурсивного итератора есть полезная возможность — не заходить в текущий каталог:

```
for (auto it = fs::recursive_directory_iterator(base);
     it != fs::recursive_directory_iterator(); ++it) {
    if (it->is_directory() && it->path().filename() == ".git") {
        it.disable_recursion_pending();    // не заходить внутрь
    }
}
```

Так пропускают служебные каталоги при обходе проекта.

50.4 Ошибки: две версии каждой функции

У большинства функций библиотеки есть два варианта — с исключением и с кодом ошибки:

```
// Бросает fs::filesystem_error при неудаче.
std::uintmax_t size = fs::file_size(p);

// Не бросает: ошибка попадает в ec.
std::error_code ec;
std::uintmax_t size = fs::file_size(p, ec);
if (ec) {
    std::cerr << "не удалось: " << ec.message() << "\n";
}
```

Выбор зависит от того, насколько ошибка ожидаема. Отсутствие файла, который должен быть, — исключительная ситуация, тут уместно исключение. Проверка сотни путей, из которых половина может не существовать, — обычный ход дела, и версия с кодом ошибки избавит от исключений в цикле.

Тип `std::error_code` появился ещё в C++11 и устроен любопытно: он хранит числовой код и указатель на категорию, которая умеет превращать код в сообщение. Благодаря этому один тип годится и для ошибок файловой системы, и для сетевых, и для ваших собственных.

50.5 Чего библиотека не делает

Она не заменяет системные вызовы там, где нужен точный контроль. Нашему PageManager из главы 33 нужны `pread` и `pwrite` со смещением и `fsync` — ничего этого в `filesystem` нет и не будет: библиотека занимается путями и метаданными, а не содержимым файлов.

Она не даёт атомарности. Между `exists` и `remove` файл может измениться, и никакие обёртки этого не исправят.

Она не решает вопросы прав доступа сложнее базовых. Владелец, группа, списки контроля доступа — всё это осталось за пределами стандарта, потому что модели прав в Unix и Windows слишком разные.

И у неё есть особенность, о которой стоит знать при сборке. В первых реализациях (GCC 8, Clang 7) библиотека требовала явного подключения: `-lstdc++fs` для GCC, `-lc++fs` для Clang. В современных версиях это не нужно, но если сборка падает на неопределённых ссылках при вполне правильном коде — причина, скорее всего, в этом.

50.6 Как это выглядело бы в нашей книге

`Database::Database` из главы 37 создаёт каталог системным вызовом:

```
::mkdir(dir.c_str(), 0755);
```

Три проблемы: не создаёт промежуточные каталоги, не переносится на Windows, молча игнорирует ошибку. С библиотекой:

```
fs::create_directories(dir);
```

Проверка размера файла в `PageManager` делается через `fstat` — и это правильно, потому что дескриптор уже открыт. А вот проверка существования файла базы до открытия просилась бы на `fs::exists`.

Утилита `myscat` из главы 31 могла бы принимать каталоги и обходить их рекурсивно — с итератором это десять строк.

50.7 Главные правила главы

1. Путь — отдельный тип, а не строка: он знает про разделители, кодировку и структуру.
2. Пути собирают оператором `/`, а не склейкой строк.
3. Разбор на каталог, имя, основу и расширение делают методами `path`, а не поиском символов.

4. `directory_iterator` обходит уровень, `recursive_directory_iterator` — дерево; порядок обхода не определён.
5. Сведения из `directory_entry` уже прочитаны при обходе — пользуйтесь ими вместо повторных запросов.
6. У функций две версии: с исключением и с `std::error_code`; выбирайте по ожидаемости ошибки.
7. Проверка существования и последующее действие — не атомарная пара; в защищённом коде так не делают.
8. Библиотека занимается путями и метаданными, но не содержимым файлов — для чтения и записи по-прежнему нужны потоки или системные вызовы.

50.8 Упражнения

1. Соберите и запустите `./build/filesystem_demo`. Затем добавьте в дерево файл с пробелом в имени и посмотрите, как он печатается.
2. Напишите программу, которая считает суммарный размер каталога рекурсивно. Сравните результат с `du -sb` (Linux) или `du -sk` (macOS) и объясните расхождение, если оно есть.
3. Сделайте обход, пропускающий каталоги `.git` и `node_modules`, через `disable_recursion_pending`. Проверьте на каталоге с проектом.
4. Напишите функцию, которая находит все файлы с заданным расширением. Сравните две версии: с `entry.path().extension()` и с ручным поиском точки в строке.
5. Вызовите `fs::file_size` для несуществующего файла в обеих версиях — с исключением и с `error_code`. Напечатайте сообщение из `ec.message()`.
6. Замените `::mkdir` в `Database` на `fs::create_directories` и проверьте, что база создаётся во вложенном пути вроде `data/2026/july/db`.
7. (Сложнее) Напишите утилиту-дубликатор: находит файлы одинакового размера, для них считает хеш содержимого и печатает группы совпадающих. Обратите внимание, где нужен `filesystem`, а где — обычное чтение файла.
8. (Сложнее) Реализуйте безопасное создание временного файла: без проверки существования, атомарно, с правами только для владельца. Разберитесь, почему `exists` плюс `create` — плохое решение и что предлагает вместо этого POSIX.

50.9 Что дальше

Глава 50 — `std::string_view` и параллельные алгоритмы. Первый убирает копирование строк там, где нужно только посмотреть, и приносит с собой новый класс ошибок — висящие ссылки. Второй обещает ускорение алгоритмов одним параметром, а на практике оказывается историей о том, почему не всё, что есть в стандарте, доступно на вашей машине.

Глава 51

Глава 50. `string_view` и параллельные алгоритмы

Две темы, объединённые общим свойством: обе выглядят как бесплатное ускорение, и обе требуют понимания, что происходит внутри.

Первая — `std::string_view`, способ смотреть на чужую строку без копирования. Он действительно ускоряет разбор текста, и он же приносит с собой новый класс ошибок, которых в C++ раньше не было.

Вторая — параллельные алгоритмы, обещающие ускорение сортировки одним лишним параметром. С ними история поучительнее: сначала о том, что они дают, потом о том, почему на моей машине они не работают вовсе.

51.1 Проблема копирования строк

Всякий раз, когда мы разбирали текст в этой книге, мы копировали куски строки. Токенизатор командной строки из главы 29 копирует каждое слово. Лексер SQL из главы 36 копирует каждый токен. Разбор мира RPG в главе 23 копирует каждое поле.

Каждая копия — выделение памяти в куче (если строка длиннее пятнадцати-двадцати символов) и пересылка байтов. Для строки, которая нужна только на время проверки, это чистая трата.

Обойти проблему можно было и раньше — парой «указатель плюс длина», — но такая пара неудобна: её надо передавать двумя аргументами, ей нельзя пользоваться как строкой, у неё нет методов.

`std::string_view` — это и есть указатель с длиной, но с интерфейсом строки:

```
std::string_view sv = "какая-то строка";
sv.size();
sv.find(' ');
sv.substr(5, 3);
sv == "какая-то строка";
```

Всё как у `std::string`, кроме одного: `string_view` **не владеет** данными. Он смотрит на чужую память и обязан умереть раньше неё.

51.2 Сколько выигрывает

Проверим на разборе текста в миллион слов. Две реализации разбиения на слова, отличающиеся только типом результата:

```
std::vector<std::string> split_copy(const std::string& s);
std::vector<std::string_view> split_view(const std::string& s);
```

`std::string`: 20 мс (1000000 слов)

`std::string_view`: 10 мс (1000000 слов)

Вдвое быстрее. Выигрыш взялся из отсутствия выделений памяти: копирующая версия делает миллион выделений и миллион освобождений, версия с представлением — ни одного.

Выигрыш был бы ещё заметнее на длинных словах: короткие строки помещаются внутрь объекта `std::string` без обращения к куче (это называют оптимизацией коротких строк), и для них копирование почти бесплатно. Наши слова как раз короткие — отсюда «всего» вдвое.

Есть и второй выигрыш, менее очевидный. `substr` у `std::string` создаёт новую строку, у `string_view` — просто сдвигает указатель и длину:

=== срез не аллоцирует ===

исходная строка не тронута, срез = [move]

Значит, разбор текста можно строить целиком на срезах, не создавая ни одной новой строки, пока результат действительно не понадобится сохранить.

51.3 Опасность, которой раньше не было

За удобство приходится платить, и плата — новый вид ошибок.

```
std::string make_name() { return "Александра"; }
```

```
std::string_view bad = make_name();
std::cout << "после смерти строки: '" << bad << "'\n";
```

Временная строка, которую вернула функция, уничтожается в конце строки с инициализацией. string_view продолжает смотреть на освобождённую память. Результат:

```
после смерти строки: '000 0?00< 00f'
живая строка:      'Александра'
```

Мусор вместо имени — и это в лучшем случае; в худшем программа упадёт или отдаст чужие данные наружу.

К счастью, компиляторы такие случаи знают:

```
warning: object backing the pointer will be destroyed at the end of
the full-expression [-Wdangling-gsl]
```

А детектор адресов ловит обращение точно:

```
ERROR: AddressSanitizer: stack-use-after-scope
READ of size 20 ...
#6 0x0001098569f1 in main dangling.cpp:11
```

Отсюда правила, которые надо соблюдать буквально.

Никогда не возвращайте string_view на локальную строку — она умрёт раньше. Никогда не сохраняйте string_view в поле объекта, если не уверены, что строка переживёт объект. Никогда не создавайте string_view из временного значения — той самой функции, возвращающей строку.

А вот где string_view уместен: параметр функции, которая только читает строку и не сохраняет её. Это, пожалуй, главное его применение:

```
void log(std::string_view message);      // берёт и string, и литерал, и срез
bool starts_with(std::string_view s, std::string_view prefix);
```

Такая функция принимает без копирования и std::string, и строковый литерал, и кусок другой строки. Раньше для этого приходилось делать две перегрузки — для const std::string& и для const char*.

51.4 Ещё одна тонкость: нулевой байт

std::string_view не обязан заканчиваться нулевым байтом, потому что он может смотреть на середину другой строки.

Отсюда следует запрет: у него нет метода `c_str()`, и передавать его в функции языка C нельзя:

```
void open_file(std::string_view path) {  
    // ::open(path.data(), ...); // ОШИБКА: data() может не иметь \0 в конце  
    std::string tmp(path);        // придётся скопировать  
    ::open(tmp.c_str(), O_RDONLY);  
}
```

Метод `data()` существует, но возвращает указатель на начало без гарантии завершающего нуля. Использовать его вместе с `size()` — правильно, использовать как строку C — ошибка, которая проявится на срезе.

Именно поэтому для путей в главе 49 существует отдельный тип `std::filesystem::path`, а не `string_view`: пути в конце концов уходят в системные вызовы, которым нужен нулевой байт.

51.5 Где он пригодился бы нам

Токенизатор командной строки из главы 29 возвращает `std::vector<std::string>` — каждый токен копируется. С представлениями копий не было бы вовсе, а строка команды всё равно жива до конца разбора.

Лексер SQL из главы 36 копирует текст каждого токена в поле структуры. То же самое.

Разбор пути в `builtin_type` из главы 30 режет переменную `PATH` через `substr` — это выделение памяти на каждый каталог. С `string_view` разбор был бы без единого выделения; я даже упоминал это в той главе.

Общее правило: если строка живёт дольше, чем разбор, и результат разбора не переживает исходную строку — берите представления. Если результат надо сохранить надолго — копируйте.

51.6 Параллельные алгоритмы

Вторая тема главы обещает большее за меньшее. В C++17 у алгоритмов появился первый параметр — политика выполнения:

```
#include <execution>  
  
std::sort(std::execution::par, v.begin(), v.end());  
std::for_each(std::execution::par_unseq, v.begin(), v.end(), heavy_work);
```

Политик четыре. `seq` — последовательно, как обычно. `par` — параллельно на нескольких потоках. `par_unseq` — параллельно и с векторизацией, то есть с использованием команд, обрабатывающих несколько значений сразу. `unseq` (появилась в C++20) — только векторизация.

Идея прекрасна: одна строка, и алгоритм использует все ядра. Библиотека сама разбивает работу, создаёт потоки, собирает результат.

51.6.1 Что нужно понимать до применения

Требования к переданной функции строже, чем кажется. Она не должна иметь состояния, не должна обращаться к общим данным без защиты, не должна бросать исключения (при исключении в параллельном алгоритме вызывается `std::terminate`, а не раскрутка стека). Иными словами, всё, о чём говорилось в главах 41 и 42, здесь ложится на вас.

Выигрыш есть не всегда. Разбиение работы и синхронизация стоят времени; на массиве из тысячи чисел параллельная сортировка окажется медленнее обычной. Порог начинается где-то с десятков тысяч элементов и сильно зависит от того, насколько дорога операция над одним элементом.

И порядок обхода перестаёт быть определённым. `std::for_each` с политикой `par` может обойти элементы в любом порядке; полагаться на последовательность нельзя.

51.6.2 А теперь неприятное

Всё вышесказанное — теория, и вот почему.

```
$ g++ -std=c++17 exec_test.cpp -o exec_test
error: no member named 'par' in namespace 'std::execution'
```

Заголовок `<execution>` есть, а политик в нём нет. Реализация `libc++`, которая поставляется с Apple Clang, параллельные алгоритмы не поддерживает — точнее, поддерживает частично и только в свежих версиях, собранных особым образом.

С GCC на Linux ситуация другая, но тоже не безоблачная: там параллельные алгоритмы работают, если установлена библиотека Intel TBB, и линковать надо с `-ltbb`. Без неё код соберётся, но выполнится последовательно.

Отсюда практический вывод, который важнее самих алгоритмов: **наличие возможности в стандарте не означает её наличия у вас**. Между принятием стандарта и появлением рабочей реализации проходят годы, а на некоторых платформах не появляется вовсе.

Поэтому я не стал писать демонстрацию параллельных алгоритмов — она не собралась бы на моей машине, а печатать в книге непроверенный код нечестно. Если они вам нужны,

проверьте доступность на своей платформе и сравните с альтернативами: пул потоков, написанный руками, даёт больше контроля, а библиотеки вроде TVB — больше возможностей.

51.7 Главные правила главы

1. `string_view` — указатель плюс длина с интерфейсом строки; он не владеет данными.
2. Разбор текста на представлениях вдвое быстрее, потому что не делает выделений памяти.
3. `substr` у представления не выделяет память, а сдвигает границы.
4. Представление обязано умереть раньше строки, на которую смотрит; временные значения — источник висящих ссылок.
5. Компилятор предупреждает о таких ошибках (`-Wdangling-gsl`), а детектор адресов ловит их точно.
6. У представления нет `c_str()` и нет гарантии завершающего нуля — в функции языка C его передавать нельзя.
7. Лучшее применение — параметр функции, которая только читает строку.
8. Параллельные алгоритмы есть в стандарте, но не везде в реализациях; проверяйте доступность до того, как рассчитывать на них.

51.8 Упражнения

1. Соберите и запустите `./build/string_view_demo`. Разберитесь, где в нём происходит копирование, а где нет.
2. Повторите замер разбиения на слова на своей машине. Затем удлините слова до пятидесяти символов и повторите — как изменилось соотношение и почему?
3. Воспроизведите висящую ссылку: создайте `string_view` из временной строки. Соберите с предупреждениями, потом с `-fsanitize=address` и сравните, что скажет каждый инструмент.
4. Перепишите `tokenize` из главы 29 на `std::string_view`. Что придётся изменить в вызывающем коде и почему нельзя просто поменять тип?
5. Напишите функцию `bool has_extension(std::string_view path, std::string_view ext)` без единого выделения памяти. Проверьте на пути без расширения и на имени вроде `.gitignore`.
6. Проверьте на своей машине, доступны ли параллельные алгоритмы: соберите пример с `std::execution::par`. Если не собирается — найдите в документации своего компилятора, что для этого нужно.

7. (Сложнее) Замерьте параллельную и обычную сортировку на массивах в тысячу, сто тысяч и десять миллионов элементов (если параллельные алгоритмы доступны). Найдите порог, начиная с которого параллельная версия выигрывает.
8. (Сложнее) Напишите свой параллельный `for_each` на пуле потоков: разбейте диапазон на части по числу ядер, обработайте каждую в своём потоке, дождитесь всех. Сравните с последовательной версией на тяжёлой операции.

51.9 Что дальше

Глава 51 — последняя. Посмотрим, что появилось в C++20 и C++23: концепты вместо непонятных ошибок шаблонов, диапазоны вместо пар итераторов, сопрограммы вместо обратных вызовов, модули вместо заголовочных файлов. Разберём, что из этого уже можно применять, а что пока ждёт реализаций. И поговорим о том, куда двигаться дальше, когда книга закончилась.

Глава 52

Глава 51. Куда дальше — обзор C++20/23 и маршрут роста

Вот и финал. Вся книга стояла на C++11 — фундаменте, который соберётся даже на старом компиляторе. Две предыдущие главы показали, что добавил C++17. Эта, последняя, — взгляд вперёд: что принесли C++20 и C++23, и, главное, **куда расти**, когда вы закроете книгу.

Сразу честно: код в этой главе — **обзорный**. Многое из C++20 уже работает в свежих GCC, Clang и MSVC, но не везде и не всё (модули и корутины до сих пор местами болезненны). Поэтому не ждите, что каждый листинг соберётся «прямо сейчас» — читайте их как «вот к чему язык пришёл и куда движется». Цель главы — чтобы вы узнали эти штуки в чужом коде и понимали, зачем они.

52.1 Концепты — конец SFINAE-ужасов

Вспомните главу 20. Чтобы шаблон принимал только подходящие типы, мы городили SFINAE — `std::enable_if`, проверки через `decltype`, нечитаемые конструкции, которые я сам назвал страшными. И сообщения об ошибках были под стать: вместо «тип не подходит» компилятор вываливал три экрана подстановок.

C++20 ввёл **концепты** (concepts) — именованные требования к типам, написанные по-человечески:

```
// C++20
#include <concepts>

template <typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;
```



```
template <Numeric T>           // принимаем только числа
T square(T x) { return x * x; }

square(5);           // ок
square(3.14);        // ок
square("hi");         // ошибка компиляции — и сообщение понятное:
                      // "constraints not satisfied: 'std::string' is not Numeric"
```

Сравните с SFINAE из главы 20 — это небо и земля. Концепт `Numeric` читается как обычное условие, а ошибка прямо говорит, **что** не так. В стандартной библиотеке уже есть готовые концепты: `std::integral`, `std::floating_point`, `std::sortable`, `std::copyable` и десятки других.

Можно требовать конкретное «умение» типа через `requires`:

```
template <typename T>
concept Drawable = requires(T obj) {
    obj.draw();           // у типа должен быть метод draw()
};
```

Если бы наша иерархия `Item` в RPG (главы 17-18) была на шаблонах, концепт `requires(T it) { it.describe(); }` заменил бы виртуальный интерфейс там, где он нужен только на компиляции.

52.2 Ranges — конвейеры вместо «итератор, итератор»

Всю книгу алгоритмы STL мы вызывали через пару итераторов:

```
std::sort(v.begin(), v.end());
std::copy_if(v.begin(), v.end(), std::back_inserter(out), is_even);
```

Это работает, но многословно, и сцепить два алгоритма в цепочку нельзя — приходится заводить промежуточные контейнеры. C++20 **ranges** решает обе беды. Во-первых, алгоритм берёт контейнер целиком:

```
// C++20
#include <ranges>
#include <algorithm>

std::ranges::sort(v);           // вместо v.begin(), v.end()
```

Во-вторых — и это главное — появились **views** и оператор `|`, собирающий обработку в конвейер (если писали на JS, Python или знаете LINQ из C#/Stream из Java — узнаете идею):

```
// C++20
namespace rv = std::views;

std::vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8};

auto result = nums
    | rv::filter([](int n){ return n % 2 == 0; }) // только чётные
    | rv::transform([](int n){ return n * n; }); // возвести в квадрат
// result «лениво» даёт: 4, 16, 36, 64

for (int x : result) std::cout << x << " ";
```

Ключевое слово — **лениво** (lazy). `filter` и `transform` не создают промежуточных векторов; элементы протекают по конвейеру по одному, только когда вы их запрашиваете в цикле. Это и читабельнее цепочки `copy_if+transform`, и без лишних аллокаций.

52.3 Корутины — функции, умеющие приостанавливаться

В главе 43 мы написали реактор на `poll`: один поток жонглирует сотней соединений, вручную сохраняя состояние каждого клиента в структуре. Код получился рабочим, но громоздким — машина состояний размазана по обработчику событий.

Корутины (coroutines, C++20) — это функции, которые умеют **приостановиться** на середине, вернуть управление, а потом продолжить с того же места. Состояние между приостановками компилятор сохраняет за вас:

```
// C++20, обзорно
Task handle_client(Socket sock) {
    while (true) {
        std::string msg = co_await sock.read(); // приостановились, ждём данные
        if (msg.empty()) break;
        co_await sock.write("эхo: " + msg); // снова приостановились
    }
}
```

`co_await` — точка приостановки: «жди, пока данные придут, а пока отдай поток другим». Снаружи это выглядит как простой последовательный код — `read`, потом `write`, потом снова

— но под капотом работает тот самый реактор, только машину состояний строит компилятор, а не вы. Появились ключевые слова `co_await`, `co_yield`, `co_return`.

Честно: корутины в C++20 — **сложная** тема. Стандарт дал низкоуровневый механизм (`promise_type`, `coroutine_handle`), но не дал готовых высокоуровневых обёрток вроде `Task` из примера — их приходится либо писать самому, либо брать из библиотек. Это та область, где «легко пользоваться» пока не равно «легко устроено».

52.4 Модули — прощание с `#include`

Всю книгу мы подключали заголовки через `#include`. В главе 7 я объяснял, как это работает: препроцессор тупо **вставляет** текст заголовка в каждый `.cpp`. Отсюда болезни — медленная компиляция (один и тот же `<vector>` парсится в каждом файле заново), хрупкость порядка включений, война с `include guards`.

Модули (modules, C++20) должны это похоронить:

```
// C++20, обзорно
// файл math.ixx — объявляем модуль
export module math;

export int square(int x) { return x * x; } // export = видно снаружи

// файл main.cpp
import math;                               // не #include!

int main() { return square(5); }
```

`import math;` не вставляет текст — компилятор один раз компилирует модуль в бинарный формат и переиспользует. Быстрее, чище, без препроцессорной магии.

Почему «пока больно»: на момент написания поддержка модулей в сборочных системах (CMake и прочих) ещё сырая, флаги у GCC/Clang/MSVC разные, а большие проекты на модули переходят медленно. Механизм правильный, экосистема догоняет.

52.5 Несколько штрихов C++23

C++23 — не революция, а шлифовка. Три вещи из него встретятся вам чаще остальных.

`std::expected<T, E>` хранит либо значение, либо ошибку. В главе 22 мы обсуждали выбор между кодами возврата и исключениями и оставили его открытым; `expected` — аккуратный

третий путь. Функция возвращает `std::expected<Frame, ParseError>`, вызывающий проверяет, что пришло, и никаких выходных параметров с исключениями. По духу это то же, что `optional` из главы 48, только вместо «ничего» лежит причина неудачи.

`std::print` и `std::println` — человеческий вывод вместо цепочки `<<`:

```
std::println("сумма = {}, среднее = {:.2f}", total, avg);
```

Форматирование пришло в C++20 как `std::format` (по образцу Python), а печать добавилась в C++23. Разница с `std::cout` не только в краткости: формат разбирается при компиляции, и опечатка в строке формата становится ошибкой сборки, а не мусором в выводе.

`std::flat_map` и `std::flat_set` — словарь и множество поверх плоского отсортированного массива вместо дерева. Данные лежат подряд, обход дружелюбен к кэшу процессора, а поиск идёт двоичным поиском. Выигрывают, когда чтений много, а вставок мало, — и это ровно та мысль про размещение данных в памяти, которую мы обсуждали в главе 34, когда объясняли, почему B+-дерево устроено вокруг размера страницы.

Язык продолжает развиваться, C++26 уже на подходе. Но гнаться за каждой новинкой не нужно: фундамент, который вы освоили, никуда не денется.

52.6 Что из этого можно брать уже сейчас

Обзор без практических выводов бесполезен, поэтому подведу черту по каждой возможности.

Концепты работают везде — GCC 10, Clang 10, MSVC 2019 и новее. Брать можно смело, и выигрыш заметен сразу же, как только у вас появляется хоть один шаблон с требованиями к типу.

Диапазоны работают в свежих компиляторах, но с оговорками: часть представлений появилась только в C++23, а сообщения об ошибках внутри конвейеров пока бывают устрашающими. Для нового кода — да; переписывать работающие алгоритмы ради красоты — вряд ли.

Сопрограммы работают как механизм, но без готовых обёрток. Это значит, что использовать их напрямую тяжело, а через библиотеку — вполне; посмотрите на `srpcoro` или на асинхронную часть Boost. Если пишете сетевой сервер с нуля сегодня, это стоит рассмотреть всерьёз: тот же реактор из главы 43 на сопрограммах выглядел бы как последовательный код.

Модули пока не готовы к повседневной работе. Компиляторы их поддерживают, сборочные системы догоняют, а большие проекты переходят медленно. Следите, но не торопитесь.

`std::format` и `std::print` из C++20 и C++23 стоит брать сразу, как только доступны: это чистый выигрыш без подводных камней. Пока недоступны — есть библиотека `fmt`, с которой стандартный формат и списан.

52.7 Куда расти после этой книги

Вы прошли путь от «что такое бит» до собственной СУБД и сетевого чата. Это **много**. Но C++ — язык, который изучают годами. Вот честный маршрут, что делать дальше.

Закончите упражнения-«посложнее». В конце почти каждой главы есть задачи со звёздочкой, которые я нарочно не разбирал. Допилите RPG (новые предметы, бои, карта побольше), доведите `shell` до удобного (автодополнение, & для фона), научите СУБД делать JOIN двух таблиц, добавьте в чат приватные сообщения. Доделанный своими руками проект учит больше десяти прочитанных глав.

Освойте инструменты профессионала. В книге мы сознательно сидели на голом `make` и `g++`, чтобы видеть каждую стадию сборки. В работе вас ждёт другое.

CMake — стандарт сборки в индустрии, и рано или поздно вы столкнётесь с ним в любом проекте. Хорошее упражнение — взять любой наш демо-проект и перевести его сборку на CMake, сохранив режимы отладки и выпуска.

Санитайзеры мы уже применяли по ходу книги: детектор потоков нашёл гонку в главе 41, детектор адресов поймал висящую ссылку в главе 50. Прогоните под `-fsanitize=address` и `-fsanitize=undefined` весь код книги целиком — почти наверняка что-нибудь всплывёт, и это будет полезнее, чем кажется.

Отладчик — `gdb` или `lldb`. Точки останова, просмотр стека, пошаговое выполнение, разглядывание памяти. После отладки печатью это другой мир, и осваивать его стоит один раз и надолго.

Готовый фреймворк тестов вместо самодельного. Наши счётчики проверок работают, но `GoogleTest` или `doctest` дают понятные сообщения, группировку, параметризованные тесты и запуск по маске. Перенесите тесты RPG или базы — разница чувствуется сразу.

Профилировщик — `perf` на Linux, Instruments на macOS. В главе 39 мы гадали, где база теряет время; профилировщик отвечает на этот вопрос не догадками, а измерением.

Изучите то, что мы обошли. Книга — фундамент, а не энциклопедия. За её рамками остались многопоточность вглубь (`std::async`, `std::future`, структуры данных без блокировок), сетевые библиотеки вроде `Boost.Asio`, метапрограммирование шаблонов, свои распределители памяти и весь C++20 по-настоящему, а не обзорно.

Где брать надёжные сведения. Справочник srpreference.com — первое место, куда стоит идти с любым вопросом о типе или функции; он точен и содержит примеры. Тексты предложений на open-std.org отвечают на вопрос «почему язык такой» — читать их непривычно, но именно там объясняют мотивы. Доклады с конференций CppCon и Meeting C++ лежат в открытом доступе, и лучшие из них учат не синтаксису, а тому, как думать о задаче. И, наконец, чужой код: исходники хороших библиотек читать тяжело первые недели и незаменимо потом.

И последнее. Главного, чему я хотел научить, нет в синтаксисе. Это привычка **понимать, что происходит под капотом**: где аллокация, кто владеет памятью, когда вызовется деструктор, что делает каждая стадия компиляции. Мы ради этого и сидели на C++11 и голом `make` — чтобы ничего не пряталось за магией. Если теперь, глядя на любую строку C++, вы спрашиваете себя «а что здесь происходит на самом деле?» — значит, книга своё дело сделала.

Спасибо, что дошли до конца. Идите и пишите код.